

Nullexit: Unified Project Document

Omar Alaaeldein

July 23, 2026

Contents

1	Project Directory Tree	4
2	README.md	6
3	agent.md	14
4	devref.md	22
5	diagrams.md	76
6	toggle.sh	81
7	recover.sh	109
8	setup.sh	119
9	scripts/setup-linux.sh	120
10	docker-compose.yml	121
11	scripts/common.sh	124
12	scripts/watcher.sh	138
13	scripts/crypto.sh	142
14	.aiignore	143
15	.claude/settings.local.json	143
16	.env.example	144
17	.gitignore	145
18	Recover-Gateway.applescript	146
19	Sweep-Gateway.applescript	146
20	Toggle-Gateway.applescript	146
21	benchmarks/benchmark.sh	146
22	benchmarks/test_load.py	148

23	black_list.txt	149
24	cloud-init.yaml	150
25	docker/tor/Dockerfile	150
26	docker/tor/entrypoint.sh	151
27	launchd/com.nullexit.noise.plist	151
28	launchd/com.nullexit.taildrop-watch.plist	152
29	launchd/com.nullexit.wake-recovery.plist	153
30	post-rules.txt	154
31	scripts/Dockerfile.routing-fix	154
32	scripts/Dockerfile.rule-compiler	154
33	scripts/Toggle-Gateway.bat	155
34	scripts/check_circuits.py	157
35	scripts/device-scramble.sh	159
36	scripts/diagnose-host-leak.sh	161
37	scripts/dns-proxy.py	170
38	scripts/dns_anomaly_detector.py	171
39	scripts/fix-docker-bridge-collision.sh	177
40	scripts/fix-ssh-delay.sh	182
41	scripts/generate_tex.py	183
42	scripts/logger/go.mod	185
43	scripts/logger/main.go	185
44	scripts/noise.sh	189
45	scripts/pf.conf	193
46	scripts/pihole-mode.sh	194
47	scripts/profiles.sh	196
48	scripts/reinstall-host-tailscale.sh	198
49	scripts/routing-fix.sh	207
50	scripts/rule-compiler/go.mod	211
51	scripts/rule-compiler/main.go	211
52	scripts/setup-common.sh	222

53 scripts/sweep.sh	228
54 scripts/taildrop-watch.sh	234
55 scripts/unlock-files.sh	235
56 scripts/wifi-rejoin.sh	235
57 tor-bridges.txt	237
58 white_list.txt	237

1 Project Directory Tree

```
nullexit/
|-- .aignore
|-- .claude
|   |-- settings.local.json
|-- .env.example
|-- .gitignore
|-- LICENSE
|-- README.md
|-- Recover-Gateway.applescript
|-- Sweep-Gateway.applescript
|-- Toggle-Gateway.applescript
|-- agent.md
|-- benchmarks
|   |-- benchmark.sh
|   |-- test_load.py
|-- black_list.txt
|-- cloud-init.yaml
|-- devref.md
|-- diagrams.md
|-- docker
|   |-- tor
|       |-- Dockerfile
|       |-- entrypoint.sh
|-- docker-compose.yml
|-- launchd
|   |-- com.nullexit.noise.plist
|   |-- com.nullexit.taildrop-watch.plist
|   |-- com.nullexit.wake-recovery.plist
|-- post-rules.txt
|-- recover.sh
|-- scripts
|   |-- Dockerfile.routing-fix
|   |-- Dockerfile.rule-compiler
|   |-- Toggle-Gateway.bat
|   |-- check_circuits.py
|   |-- common.sh
|   |-- crypto.sh
|   |-- device-scramble.sh
|   |-- diagnose-host-leak.sh
|   |-- dns-proxy.py
|   |-- dns_anomaly_detector.py
|   |-- fix-docker-bridge-collision.sh
|   |-- fix-ssh-delay.sh
|   |-- generate_tex.py
|   |-- logger
|   |   |-- go.mod
|   |   |-- main.go
|   |-- noise.sh
|   |-- pf.conf
|   |-- pi-hole-mode.sh
|   |-- profiles.sh
|   |-- reinstall-host-tailscale.sh
```

```
| |-- routing-fix.sh
| |-- rule-compiler
| |   |-- go.mod
| |   `-- main.go
| |-- setup-common.sh
| |-- setup-linux.sh
| |-- sweep.sh
| |-- taildrop-watch.sh
| |-- unlock-files.sh
| |-- watcher.sh
| `-- wifi-rejoin.sh
|-- setup.sh
|-- toggle.sh
|-- tor-bridges.txt
`-- white_list.txt
```

2 README.md

```
1 # nullexit: Tailscale + Cloudflare WARP Docker Gateway
2
3 > **Last updated:** July 23, 2026 [Architecture & Flow Diagrams ](./diagrams.md) [Full Dev Reference ](./
  devref.md)
4
5 **nullexit** is a chained network gateway that routes all Tailscale exit-node traffic through a Cloudflare WARP
  VPN tunnel double-encrypting every packet, hiding your ISP metadata, and providing network-wide DNS ad-
  blocking (AdGuard Home) and kernel-level IP threat blocking (`ipset`/`iptables`) for every device on your
  mesh.
6
7 ---
8
9 ## 1. Prerequisites
10 - Docker and Docker Compose installed.
11 - A Tailscale account.
12 - **macOS:** Install the standalone Tailscale CLI via Homebrew (`brew install tailscale`) and register `
  tailscaled` as a system service (`brew services start tailscale`). No `.app` GUI install required.
13 - **OS & Python:** Developed and tested against **macOS 26.5.2**. The project has zero external Python
  dependencies (no `requirements.txt`) and explicitly targets the default Apple-provided **Python 3.9.6** (`/
  usr/bin/python3`). While the scripts could function on newer versions, `nullexit` explicitly avoids
  experimenting with or modifying the built-in system Python environment to prevent unexpected OS-level side
  effects.
14
15 ## 2. Installation & Setup
16
17 Run the interactive setup script for your OS. It downloads `wgcf`, generates fresh Cloudflare WARP WireGuard
  keys, prompts for a Tailscale Auth Key, and writes your `.env`.
18
19 ```bash
20 ./setup.sh # macOS
21 ./scripts/setup-linux.sh # Linux
22 ```
23
24 ### Reference `.env`
25 This is a quick reference of the common variables. For the **complete, authoritative list** of every `.env`
  variable (including all optional/advanced settings), see `devref.md 7`.
26
27 ```env
28 WIREGUARD_PRIVATE_KEY=<Generated>
29 WIREGUARD_PUBLIC_KEY=<Generated>
30 WIREGUARD_ADDRESSES=<Generated>
31 TS_AUTHKEY=tskey-auth-...
32 GATEWAY_RULE_PROFILE=medium # light | medium | heavy
33 # Safe MSS for double-tunneled traffic (Tailscale + WARP). Don't raise this
34 # devref.md 15.3.2 found 1180 (and even 1160) still too large and causes the
35 # exact mysterious mid-transfer stalls this value exists to prevent.
36 GATEWAY_MSS=1120
37 # Set to false to run as a 'Headless Server' (Docker acts as an exit node, but the host Mac/Linux's own
  internet is NOT hijacked).
38 GATEWAY_HIJACK_HOST=true
39 GATEWAY_USE_EXIT_NODE=true
40 WARP_FAIL_THRESHOLD=6 # consecutive warp=off polls before auto-shutdown (default 6 = 30s)
41 KILL_SWITCH=true # enforce strict PF lock that breaks SSH if VPN fails. Leave true this is
  the core leak guarantee.
42 # On campus/enterprise networks (WPA2-Enterprise / 802.1x), AP Client Isolation blocks direct
43 # LAN traffic between devices. Tailscale attempts a local P2P upgrade anyway, which causes
44 # macOS gvproxy to SNAT-mangle the reply's source port poisoning the phone's WireGuard
45 # endpoint and blackholing 100% of its traffic. Setting this to false drops those local
46 # hole-punch packets so the phone safely falls back to Tailscale DERP relays.
47 # watcher.sh auto-detects 802.1x and AP isolation on every Wi-Fi change and overrides
48 # this setting automatically you only need to set this manually if auto-detection fails.
49 TAILSCALE_ALLOW_LAN_P2P=false # auto-managed; true only on trusted hotspots/home networks
50 ADGUARD_USER=admin # Username for AdGuard Home (defaults to admin if not set)
51 ADGUARD_PASSWORD=nullexit # Shared password for AdGuard Home and Tor ControlPort (defaults to nullexit
  if not set)
52 BLOCKED_COUNTRIES="kp il" # dynamically block country IP ranges via ipdeny.com
53 TOR_EXCLUDE_EXIT_NODES={us},{gb},{fr},{de},{it},{ca},{jp},{kp},{il} # comma-separated country codes to exclude
  as Tor exit nodes
54 DEBUG_TRACE=false # true = mirror a full command-by-command trace of the toggle to output.log
  (see 9)
55
56 ---
57
58 ## 3. Deploy the Gateway
59
60 **macOS (Recommended):**
61 ```bash
62 ./toggle.sh
63 ```
```

```

62 Running it again stops the gateway and restores your network. Handles the full lifecycle: Colima VM, containers
    , DNS hijacking, Tailscale exit-node routing, rule compilation, and sleep prevention.
63
64 **Linux / Manual:**
65 ```bash
66 docker compose up -d
67 ```
68
69 ## 4. Post-Deployment
70
71 ### Approve the Exit Node
72 1. Go to [Tailscale Admin Machines](https://login.tailscale.com/admin/machines).
73 2. Find the `tailscale` device `...` **Edit route settings** enable as Exit Node.
74
75 ### Verify Traffic Flow
76 On any Tailscale client device, select this gateway as the exit node, then visit [api.ipify.org](https://api.
    ipify.org) or run:
77 ```bash
78 curl -s https://www.cloudflare.com/cdn-cgi/trace | grep -E '^(warp|ip|colo)='
79 # expect: warp=on
80 ```
81
82 ### AdGuard DNS Setup
83
84 To ensure client devices correctly route DNS through the AdGuard container (and do not bypass it via `
    tailscaled`'s internal resolver), you **must** configure your Tailscale Admin Console:
85 1. Go to the **DNS** tab in the [Tailscale Admin Console](https://login.tailscale.com/admin/dns).
86 2. Enable **MagicDNS**.
87 3. Under **Nameservers**, add a **Custom** nameserver and enter the gateway's Tailscale IPv4 address (run `
    tailscale ip -4` on the gateway machine to find it, e.g., `100.x.x.x`).
88 4. Click the **three dots (...)** next to the nameserver you just added and enable **"Use with exit node"**.
89 5. Delete any other Global Nameservers (like Google or Cloudflare).
90 6. Toggle ON **Override local DNS**.
91
92 Traffic will now be correctly forced into the AdGuard sinkhole, and mobile devices will be blocked from
    bypassing it via encrypted DNS (DoH/DoT).
93
94 > [!WARNING]
95 > **Disable "Private DNS" on mobile devices.** Android's Private DNS (DoT/DoH) bypasses port 53 interception
    entirely. Go to `Settings` `Connections` `More connection settings` `Private DNS` `Off`.
96
97 > [!WARNING]
98 > **Bandwidth doubling:** Streaming 1GB of video on a remote device consumes 1GB download + 1GB upload on the
    host's network. Be mindful on metered connections.
99
100 ### AdGuard Dashboard
101 Navigate to `http://<gateway-tailscale-ip>:3000` credentials: **admin` / `nullexit`**.
102
103 ### Access Any Mesh Device From Anywhere (SFTP/SSH)
104
105 Once the gateway exit node is online, **any device on your Tailscale mesh can reach any other mesh device,
    anywhere in the world** as long as both devices are on the mesh and the target device has an SSH server
    running. No port forwarding, no public IPs, no firewall holes. You can SSH into any device using its
    MagicDNS hostname (e.g. `ssh username@my-macbook`) or its assigned Tailscale IP (`100.x.x.x`).
106
107 > **Security Tip (Zero Local Attack Surface):** It is highly recommended to disable macOS's native "Remote
    Login" and "Screen Sharing" in System Settings to prevent exposing those ports (22, 5900) to physical Wi-Fi
    networks (which scanners can fingerprint). The gateway enforces `tailscale up --ssh=true` and the `pf`
    Kill-Switch automatically blocks local Wi-Fi access to these ports. Since both MagicDNS and Tailscale IPs
    route through the exact same encrypted tunnel, they are equally securebut **MagicDNS is recommended simply
    for ease of use**.
108
109 ### Extreme OPSEC: The Secret Tor-over-WARP Proxy
110 `nullexit` includes a completely isolated, headless Tor infrastructure proxy designed for terminals and hacking
    tools (e.g., `curl`, `sqlmap`, `nmap`).
111 To protect against local malware fingerprinting:
112 1. **Procedural Ports:** The proxy ports are randomized into the ephemeral range (10000-65000) on every single
    boot using a strict kernel socket collision-avoidance check (`netstat`).
113 2. **Honey-Port Tripwire:** The gateway spawns a fake trap port. If any malware attempts a sequential port-scan
    on `localhost` to find the proxy, it trips the Honey-Port, instantly triggering a macOS desktop
    notification and a critical log alert.
114
115 > [!NOTE]
116 > **Threat Model limitation:** Port randomization and the Honey-Port only defeat blind, generic port scanners.
    They do not protect against targeted local malware that runs `lsof -i` or reads the `/tmp/nullexit-ports.
    env` file directly. To actually prevent malicious local processes from reaching the proxy, you must run a
    host-level outbound firewall (like LuLu 4.3.0+ or Little Snitch) with **localhost/loopback filtering
    explicitly enabled** to gate access by process identity.
117
118 To use the Tor proxy, run `cat /tmp/nullexit-ports.env` to find your `$TOR SOCKS_PORT` for the current session,
    and point your hardened browser (LibreWolf/Tor Browser) or CLI tool to `127.0.0.1:$TOR SOCKS_PORT`.

```

```

119 > [!WARNING]
120 > **Preventing DNS Leaks (SOCKS5 vs. SOCKS5-Hostname):**
121 > When routing command-line tools through Tor, you must force the tool to delegate DNS resolution to the
122 Tor proxy. Using plain SOCKS5 (e.g., `socks5://` or curl's `--socks5` flag) will resolve hostnames locally
    * using the host's DNS settings (AdGuard/WARP) before establishing the connection. While this DNS traffic
    is encrypted inside the WARP tunnel, the destination domain name is still leaked to AdGuard and Cloudflare,
    undermining your anonymity.
123 >
124 > Use these correct protocols and configurations:
125 > - curl: Use `--socks5-hostname` or the `socks5h://` scheme (e.g., `curl -x socks5h://127.0.0.1:
    $TOR SOCKS_PORT https://check.torproject.org`). Do not use `--socks5` or `socks5://`.
126 > - sqlmap: Pass the proxy URL using the `socks5h://` scheme to ensure remote resolution: `--proxy="
    socks5h://127.0.0.1:$TOR SOCKS_PORT"`.
127 > - nmap: Nmap's built-in `--proxies` option resolves hostnames locally. To safely scan targets without
    DNS leaks, tunnel it using `proxychains-ng` configured with `proxy_dns` enabled (add `socks5 127.0.0.1 <
    PORT>` to `/etc/proxychains.conf` or a local config, then run `proxychains4 nmap <args>`).
128
129
130 ##### Transparent .onion Browsing & Dark Web Access
131 In addition to the randomized SOCKS5 proxy port, `nullexit` provides seamless, transparent `.onion` domain
    browsing for all devices on your Tailscale mesh:
132 - Zero-Config DNS: AdGuard Home automatically intercepts queries for `.onion` domains and resolves them to
    Tor's virtual mapping subnet.
133 - Tailscale Auto-Routing: The virtual subnet is mapped to the `198.18.0.0/15` benchmark range (RFC 2544).
    Since this range is not a private IP subnet, Tailscale's Exit Node mode automatically routes it to the
    gateway (bypassing local LAN exclusions).
134 - Transparent NAT Proxying: Kernel firewall rules in the gateway redirect all TCP traffic destined for
    `198.18.0.0/15` directly into Tor's transparent port (`9040`).
135 - Exit Node Exclusion: Supports the ability to exclude specific countries from being used as Tor exit nodes
    via the `TOR_EXCLUDE_EXIT_NODES` environment variable in `.env`.
136
137 ##### Enable in Web Browsers
138 To prevent accidental DNS leaks, web browsers block `.onion` lookups by default. To browse dark web sites
    natively:
139 - Firefox: Go to `about:config`, search for `network.dns.blockDotOnion`, and set it to `false`. Type any
    onion address (e.g. `duckduckgogg42xjoc72x3sjasoworfbgcmvfimaftt6twagswzczad.onion`) directly into the URL
    bar and it will load.
140 - Mobile Browsers (iOS/Android): Connect to your Tailscale exit node, open Firefox (with blockDotOnion
    disabled), and browse onion sites natively on the go.
141
142 ##### Hardening: Using obfs4 Tor Bridges
143 If you want to disguise your Tor traffic from the WARP exit node provider, you can optionally enable Tor
    bridges:
144 1. Obtain private `obfs4` bridge lines from [bridges.torproject.org](https://bridges.torproject.org) or the
    official Telegram bot.
145 2. Create a file named `tor-bridges.txt` in the root of the `nullexit` folder and paste your bridge lines
    inside it (one per line).
146 3. Set `TOR_USE_BRIDGES=true` in your `.env` file and restart the gateway. The proxy will refuse to start if
    the bridges file is empty or missing.
147
148 ##### Example: Browse your Android phone's files from your Mac
149
150
151 1. On your Android phone, install [Termux](https://termux.dev/) and [Termux:Boot](https://f-droid.org/
    packages/com.termux.boot/) from F-Droid.
152 2. In Termux, install and start the SSH server:
153 ```bash
154 pkg install openssh
155 sshd
156 ```
157 Termux defaults to port 8022 (ports below 1024 require root on Android).
158 3. Find your phone's Tailscale IP:
159 ```bash
160 tailscale ip -4 # e.g. 100.87.42.87
161 ```
162 4. Stop Android from killing Termux do ALL of these (Samsung One UI is especially aggressive):
163 - Settings Apps Termux Battery Unrestricted**
164 - Device Care Battery Background usage limits Sleeping apps / Deep sleeping apps** remove Termux if
    listed
165 - In Termux, run `termux-wake-lock` to hold a persistent wakelock
166 5. On your Mac/PC, open [Cyberduck](https://cyberduck.io/) (or any SFTP client: WinSCP, FileZilla, FE File
    Explorer on iOS) and connect to:
167 - Server: `100.87.42.87` (your phone's Tailscale IP)
168 - Port: `8022`
169 - Protocol: SFTP (SSH File Transfer Protocol)
170 - Username: (your Termux username, usually `u0_a123` run `whoami` in Termux to check)
171 - Password: (your Termux password set with `passwd` in Termux if you haven't already)
172
173 That's it you can now browse, download, and upload files to your phone from any device on the mesh, no matter
    where either device is physically located. This works identically for any mesh device: Mac Mac, Windows

```



```

Linux, phone NAS, etc.
174
175 > **Troubleshooting:** If the connection hangs for ~30 seconds before failing, your phone's SSH server isn't
    running. Open Termux and run `sshd` again. If Android keeps killing it despite Unrestricted battery, the
    termux-wake-lock` command is your fix.
176
177 ---
178
179 ## 5. Multi-Layer Firewall & Kill-Switch
180
181 ### Layer 1 DNS Sinkhole (AdGuard Home)
182 Blocks ads and tracking domains before they are downloaded. In automated Lighthouse tests on ad-heavy sites:
183 - **Time to Interactive:** ~22% faster (51.0s 41.8s)
184 - **Speed Index:** ~20% faster (15.3s 12.8s)
185
186 Customize blocking via `black_list.txt` and `white_list.txt`. Rule profiles (`light` / `medium` / `heavy`) are
    set in `.env`.
187
188 ### Layer 2 Kernel IP Firewall (`ipset`/`iptables`)
189 On every startup, the `rule-compiler` fetches threat-intelligence feeds (Spamhaus DROP, Feodo Tracker, Emerging
    Threats, CINS) and compiles **~16,700 unique malicious IPs/CIDRs** into the kernel `FORWARD` chain
    blocking both outbound C2 connections and inbound attack traffic. Zero configuration required.
190
191 ### Layer 3 Geo-IP Blocking
192 Dynamically blocks all traffic to and from specific countries using live IP ranges from `ipdeny.com`.
193 To add countries to your blocklist, open your `.env` file and set the `BLOCKED_COUNTRIES` variable with 2-
    letter ISO country codes (e.g. `BLOCKED_COUNTRIES="kp il cn ru"`), then restart the gateway.
194
195 ### Layer 4 macOS PF Kill-Switch
196 A native macOS Packet Filter (`pf`) kill-switch that enforces a strict default-deny policy on your physical Wi-
    Fi interface (`en0`). When `KILL_SWITCH=true` is set in your `.env`, it drops all outgoing traffic except
    the Cloudflare WARP endpoints and Tailscale DERP relays.
197
198 - **Failsafe Design:** If the VPN tunnel crashes or Docker dies, your Mac completely loses internet access
    rather than leaking your real IP.
199 - **Remote-Access Safe:** Carefully designed to whitelist Tailscale's control plane (`192.200.0.0/24`), `utun*`
    tunnel traffic, and local LAN subnets. This guarantees that your remote SSH sessions and local AirDrop
    will survive even when the kill switch engages.
200 - **Setup Requirement:** The toggle scripts need permission to manipulate the network and firewall in the
    background without prompting you for a password. You must configure your passwordless sudo config by
    running exactly:
201 ```bash
202 echo "$USER ALL=(root) NOPASSWD: /sbin/pfctl, /usr/sbin/networksetup, /usr/bin/dscacheutil, /usr/bin/killall,
    /usr/bin/pkill, /bin/kill, /sbin/route, /sbin/ifconfig, /usr/bin/true, /opt/homebrew/bin/brew, /usr/local/
    bin/brew, /usr/bin/python3 -I $PWD/scripts/dns-proxy.py, /opt/homebrew/bin/python3 -I $PWD/scripts/dns-
    proxy.py, /usr/local/bin/python3 -I $PWD/scripts/dns-proxy.py" | sudo tee /etc/sudoers.d/nullexit
203 ```
204
205 ---
206
207 ## 6. Toggle Scripts
208
209 ### macOS `toggle.sh`
210 Fully automated lifecycle script. Also supports a native `.app` launcher:
211 ```bash
212 osacompile -o "Toggle Gateway.app" Toggle-Gateway.applescript
213 ```
214
215 Optional `.env` settings (common subset; the full canonical table lives in `devref.md 7`):
216 - `GATEWAY_BYPASS_PING=true` proceed even if pre-flight connectivity checks fail.
217 - `GATEWAY_USE_EXIT_NODE=false` DNS-only mode, skip exit-node routing.
218 - `GATEWAY_HIJACK_HOST=false` skip DNS hijacking on the host (provides VPN/adblocking only to external
    Tailscale peers).
219 - `GATEWAY_MSS` override the TCP MSS clamp for the double-tunnel. Default (and only validated-safe value) is
    `1120`; devref.md 15.3.2 found 1180 and even 1160 still too large for the Tailscale+WARP double-
    encapsulation overhead, causing the exact mid-transfer stalls this clamp exists to prevent. Applies to both
    the container (`post-rules.txt`) and the host (`pf.conf`) they're kept in sync automatically.
220 - `HOST_MTU=1200` override the host Tailscale interface MTU. Defaults to 1200 to fit inside the 1280-byte WARP
    tunnel.
221 - `KILL_SWITCH=true` enforce strict network lock that breaks SSH if the VPN fails.
222 - `STOP_COLIMA_ON_EXIT=true` fully shut down the Colima VM on toggle-off (saves battery on dedicated hosts).
223 - `TAILSCALE_ALLOW_LAN_P2P=true` allow direct Tailscale LAN P2P connections between devices on the same
    network.
224 - **Default: `false`** On campus or enterprise Wi-Fi (WPA2-Enterprise / 802.1x), AP Client Isolation blocks
    local device-to-device traffic. When Tailscale still attempts a local P2P upgrade, macOS's `gvproxy` layer
    SNAT-mangles the reply's source port. WireGuard's endpoint roaming treats this as a legitimate address
    change and updates the phone's outbound path to the newly-mangled port which has no listener. This
    blackholes 100% of the phone's traffic while the DERP relay path is abandoned. Setting this to `false`
    preemptively drops all RFC1918-destined Tailscale UDP from the gateway so the phone receives a clean
    failure and safely falls back to DERP.

```

```

225 - **Auto-managed:** `watcher.sh` automatically detects WPA2-Enterprise via `system_profiler SPAirPortDataType`
    (the `airport` binary was removed in macOS 14.4) and probes for AP isolation by pinging the default
    gateway (`route -n get 1.1.1.1`) on every network change. It writes the detected value to `lan_p2p_detected`
    in the repo root, which `routing-fix.sh` re-reads every 30 seconds **no restart required
** Only set this manually if auto-detection fails. See `devref.md 15.7.1` for the full SNAT endpoint
    poisoning analysis and `devref.md 15.11.6` for the `airport` removal background. *(Note: Direct P2P between
    the gateway container and the host Mac itself is always permitted via the Docker bridge to bypass DERP
    relays, regardless of this setting).*
226 - Set to `true` on trusted home networks or Windows/mobile hotspots (no AP isolation) for a faster, lower-
    latency direct path.
227 - **The real payoff untrusted-but-open networks (hotels, cafes, airports). ** These aren't enterprise/802.1x
    networks, so they typically don't AP-isolate `watcher.sh`'s auto-detection usually finds P2P viable and
    flips this on by itself. That's two kinds of protection at once with zero manual setup on that specific
    network: every byte between your mesh devices is WireGuard-encrypted whether it goes direct or via DERP, so
    another guest on the same open hotel Wi-Fi who can technically reach your device at Layer 2 still can't
    read or tamper with anything; and since you're already Tailscale-enrolled, none of this needs configuring
    per-network no VPN profile to import, no setting to flip at each new hotspot. The same phone that was
    safely DERP-only on an isolated campus network this afternoon gets fast, secure P2P at the hotel tonight
    automatically, because `watcher.sh` re-probes isolation on every Wi-Fi change.
228 - `WARP_FAIL_THRESHOLD=3` number of consecutive failed checks before forcing a gateway teardown (default: 6
    checks = 30s).
229 - `WARP_ENDPOINT_1` / `WARP_ENDPOINT_2` override the Cloudflare WARP WireGuard endpoints.
230
231 ## Linux `toggle.sh` / `recover.sh`
232 ```bash
233 ./toggle.sh # toggle ON/OFF (cross-platform, dispatches on $OSTYPE)
234 ./recover.sh # recovery tool (cross-platform)
235 ```
236
237 ## Sleep Prevention (macOS)
238 When the gateway is ON, `caffeinate -i -s` runs in the background: `-i` blocks idle system sleep (even on
    battery), and `-s` additionally blocks sleep outright whenever the Mac is on AC power (a no-op on battery,
    so unplugged runs still sleep normally). The display can still sleep normally. Without `-s`, real idle-
    sleep cycles were still slipping through and dropping every mesh device's exit node until the next wake-
    recovery cycle see `devref.md 15.5.8`.
239
240 ## Auto-Recovery (macOS post-wake / post-roam)
241 Install the launchd LaunchAgent so the gateway self-heals after sleep/wake and Wi-Fi roams:
242 ```bash
243 cp ./launchd/com.nullexit.wake-recovery.plist ~/Library/LaunchAgents/
244 sed -i '' "s|__NULLEXIT_HOME__|$(pwd)|" ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
245 launchctl load -w ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
246 ```
247 Wi-Fi roams and network changes are caught reliably. Lid-close/wake is
248 **best-effort on macOS Sonoma ** Apple suspends LaunchAgents during forced
249 sleep rather than re-triggering them on every wake, so the watcher can miss
250 the specific wake event (other self-healing the DNS Watcher, Tailscale's
251 DERP fallback, gluetun's healthcheck still recovers within ~30-60s
252 regardless). If lid-wake responsiveness matters more than that window,
253 `brew install sleepwatcher` and hook `recover.sh --post-wake` to `~/wake`
254 instead. See `devref.md 15.5.1` for the full deep dive.
255
256 ## Wi-Fi auto-rejoin (macOS)
257 If this Mac drops Wi-Fi and macOS **auto-join is off **, it will otherwise just sit
258 there with no internet (and, if the gateway is up, the WARP Watcher would
259 eventually mistake the dead uplink for a dead tunnel see `devref.md 15.6.2`).
260 The same always-on `watcher.sh` daemon now also re-associates your Wi-Fi for you.
261
262 It never scans for or hops onto arbitrary networks macOS hides the live SSID
263 from every script unless you grant Location (which this project deliberately does
264 not use; see `devref.md 15.11.10`), so instead you **remember** your network
265 name(s) once and it only ever rejoins one of those:
266 ```bash
267 bash scripts/wifi-rejoin.sh set "Your-Wi-Fi-Name" # remember a network
268 bash scripts/wifi-rejoin.sh list # show remembered networks
269 bash scripts/wifi-rejoin.sh forget "Your-Wi-Fi-Name" # stop remembering it
270 ```
271 The remembered list lives in `last_wifi_ssid` (one per line, gitignored). When
272 the Wi-Fi interface has no routable IP, `watcher.sh` calls
273 `scripts/wifi-rejoin.sh rejoin`, which waits a short grace period (let DHCP
274 self-heal first), then re-associates a remembered network via
275 `networksetup -setairportnetwork` pulling the password from your Keychain with
276 progressive backoff and a give-up cap so it never churns a network that won't come
277 up. It is leak-safe: it only acts when the interface has **no** address, i.e. no
278 egress path exists. Because the driver is the always-on daemon (not the gateway's
279 WARP Watcher), it reconnects you whether the gateway is up, down, or off. See
280 `devref.md 15.5.9`.
281
282 ## Config profiles taming the flag sprawl
283 The opt-in features below add up to several interacting `.env` switches. Rather than
284 hand-tune them, `scripts/profiles.sh` bundles the intent-level flags into three

```

```

285 known-good presets. It is **config-only** it just edits `.env`, never touches the
286 live gateway so changes apply on your next `.toggle.sh --restart`, and it forces
287 `KILL_SWITCH=true` in every profile.
288
289 ```bash
290 bash scripts/profiles.sh show # the whole current config, grouped + explained
291 bash scripts/profiles.sh preview home # exactly what 'apply' would change (read-only)
292 bash scripts/profiles.sh apply home # write it (backs up `.env`; never restarts)
293 ```
294
295 Profiles: **campus** (AP-isolated enterprise, conservative) **home** (trusted, fast
296 direct P2P, FaceTime + LAN Pi-hole) **hardened** (cover traffic + Tor bridges, no
297 real-IP exposure).
298
299 ### Cover traffic / dummy padding (macOS optional, opt-in)
300 `scripts/noise.sh` emits verifiable-random UDP padding toward the exit so a passive
301 on-path observer sees a continuously varying encrypted flow instead of your real
302 traffic envelope (a metadata / traffic-flow correlation defence). Unlike the rest
303 of the toolkit it is **active** it puts traffic on the wire so it stays inert
304 unless you opt in. **One switch controls everything:** set `NOISE_ENABLED=true` in
305 `.env`. There is no separate "persistent vs one-shot" option installing the
306 LaunchAgent below simply means padding auto-starts whenever `NOISE_ENABLED=true`,
307 and the job is a clean no-op whenever it is `false`.
308
309 ```bash
310 # One-off (this session only):
311 bash scripts/noise.sh verify # prove the noise is statistically random (entropy/monobit/chi-square)
312 bash scripts/noise.sh start # arm padding now; `status` / `stop` to manage
313
314 # Persistent (survives reboot/login) install the LaunchAgent once:
315 cp ./launchd/com.nullexit.noise.plist ~/Library/LaunchAgents/
316 sed -i '' "s|_NULLEXIT_HOME_|$(pwd)|" ~/Library/LaunchAgents/com.nullexit.noise.plist
317 launchctl load -w ~/Library/LaunchAgents/com.nullexit.noise.plist
318 # To stop persisting: launchctl bootout gui/$UID/com.nullexit.noise
319 ```
320
321 Tunables (rate, packet size, jitter, target) live in `.env` under the
322 `NOISE_PAD_*` keys; defaults are a modest 64 kbps toward the gateway. Honest
323 limits: this is one-way random padding it blurs uplink volume + timing, not a
324 full traffic-analysis-resistant shaper. See the `noise.sh` / `Cover Traffic &`
325 `Padding` notes in the knowledge graph.
326
327 ### LAN Pi-hole mode (macOS optional, opt-in)
328 nullexit is already a Pi-hole for **mesh** devices AdGuard Home filters DNS for
329 everything that routes through the gateway over Tailscale. This mode extends that
330 filtering to **non-Tailscale** devices on the same LAN (a smart TV, a guest, IoT):
331 point their DNS server at this Mac's IP and they get the same ad/tracker/threat
332 blocking. It reuses `dns-proxy.py` bound to the LAN interface, forwarding to
333 AdGuard.
334
335 ```bash
336 # `.env`: PIHOLE_LAN_MODE=true (default false)
337 sudo bash scripts/pihole-mode.sh enable # bind the LAN DNS forwarder (:53 needs root)
338 bash scripts/pihole-mode.sh test # prove it resolves clean + sinkholes blocked
339 bash scripts/pihole-mode.sh status # running? listen addr, AdGuard reachability
340 sudo bash scripts/pihole-mode.sh disable
341 ```
342
343 Honest limits: it only works on a **trusted, non-isolated** network on
344 AP-isolated WPA2-Enterprise (the same isolation that forces phones onto DERP)
345 other devices can't reach this Mac at all. And it's **DNS filtering only** it
346 does not route those devices through WARP. See the `pihole-mode.sh` note in the
347 knowledge graph.
348
349 ### Device Scramble Mode (macOS optional, opt-in)
350 Randomizes the identifiers a network/observer uses to fingerprint your device (hostname and MAC address). This
351 is useful on open / no-auth Wi-Fi (caf, hotel, airport) where these IDs are the only handle on you.
352 * Note: MAC spoofing requires root. The `test-mac` command is fail-safe and detects Apple-Silicon spoof-
353 blocking without stranding you.
354
355 ```bash
356 bash scripts/device-scramble.sh status # show current + saved identity
357 sudo bash scripts/device-scramble.sh scramble # random hostname (add --mac to also rotate MAC)
358 sudo bash scripts/device-scramble.sh restore # put your real identity back
359 sudo bash scripts/device-scramble.sh test-mac # safe test to see if this network takes a random MAC
360 ```
361
362 ---
363
364 ## 7. Networking Notes

```

```

363 - **Port conflicts:** None. Tailscale uses dynamic UDP ports; WARP uses outbound UDP:2408. The gateway binds
    `0.0.0.0:41642/udp` on the host for Tailscale's WireGuard listener this is intentional and required to
    receive inbound hole-punch packets from peers on the internet. All other internal ports (AdGuard :5335,
    SOCKS5 :1080) are bound to `127.0.0.1` only and are not reachable from the LAN.
364 - **Tailscale P2P vs DERP:** On the same Wi-Fi, Tailscale establishes a fast P2P connection. On cellular (CGNAT
    ), it always falls back to a DERP relay (~80-200ms). See `devref.md 5.1`.
365 - **AirDrop / AirPlay:** Unaffected the gateway only touches standard Wi-Fi/Ethernet interfaces, not `awdl0`.
366 - **VPN coexistence:** Do **not** run a local VPN client (WARP, Mullvad, NordVPN) simultaneously with the exit
    node. Both fight for the default route and will blackhole your connection.
367 - **Banking & financial sites:** May intermittently block logins through WARP. Banks use anti-fraud databases
    that flag datacenter IP ranges (like Cloudflare's `104.28.x.x`) as VPN/proxy traffic. Success fluctuates
    depending on which WARP IP you land on. If a site blocks you, either disable the exit node temporarily for
    that session or use the bank's mobile app (which often uses different fraud detection).
368 - **MSS clamping (Bufferbloat):** Double-tunnelling adds ~120-160 bytes of overhead per packet. This is now
    automatically handled natively via the macOS `pf` firewall, which universally applies TCP MSS Clamping to
    all outbound packets to strictly prevent MTU fragmentation and bufferbloat. No manual `.env` configuration
    is required. Additionally, the host's Tailscale interface MTU is explicitly lowered to 1200 (configurable
    via `HOST_MTU` in `.env`) to guarantee packets fit inside the container's WARP tunnel without fragmenting.
369
370 ---
371
372 ## 8. Privacy & Security Hardening
373
374 The stack shifts trust across four layers: WARP hides traffic from your ISP, AdGuard blocks tracking at DNS,
    Tailscale encrypts device-to-device, and `ipset` blocks known malicious IPs at the kernel.
375
376 For higher security, the gateway supports these drop-in upgrades:
377
378 | Layer | Default | Hardened |
379 |---|---|---|
380 | VPN Exit | Cloudflare WARP (US) | Mullvad (Swedish, no-logs, anonymous payment) |
381 | DNS Resolver | AdGuard via WARP | AdGuard via Mullvad DoH |
382 | Coordination | Tailscale (US) | Headscale (self-hosted) |
383 | Auth | Persistent OAuth keys | Ephemeral auth keys |
384 | Content | WireGuard asymmetric | WireGuard + Pre-Shared Keys (PSK) |
385
386 #### Python Runtime Airgap (Isolated Mode)
387 **Never** install third-party `pip` packages (like `requests` or `pyyaml`) into the Python environment that `
    nullexit` uses. All python scripts in this project (e.g., the `dns-proxy.py` daemon) are strictly
    engineered to run on the zero-dependency Python Standard Library. To strictly isolate the environment from
    supply-chain attacks, `nullexit` executes these scripts using **Python's Isolated Mode** (`python3 -I`).
    This deliberately blinds the execution environment to any malicious user-installed `pip` packages on the
    host, permanently air-gapping the gateway from PyPI.
388
389 See `devref.md 9` for the full threat model, Snowden programme analysis, and trust assumptions.
390
391 ---
392
393 ## 9. Debugging
394
395 - **`output.log`** All stderr from `toggle.sh`, `recover.sh`, `setup.sh`, and the rule-compiler is written
    here. The WARP Watcher (`WARP_FAIL_THRESHOLD`) also writes its events here `WARP DOWN`, `WARP RECOVERED`,
    and `WARP SHUTDOWN` notices, plus `WARP watcher PAUSED/RESUMED` when it detects the physical uplink is down
    (15.6.2). The Wi-Fi auto-rejoin helper logs `wifi-rejoin:` lines (re-associate attempts, backoff, give-up)
    . All timestamps are **UTC** (unified 2026-07-23 the log used to mix UTC and local time). Check this first
    .
396 - Every run now records lifecycle breadcrumbs regardless of `DEBUG_TRACE`: an `EXEC:` / `EXIT <code>:` line
    brackets each heavyweight command (`colima start`, `docker compose up -d --build`, `docker compose down`),
    and an `EXIT: code=<N> phase=<init|stop|start>` line is written when the script exits **for any reason**
    including a killed terminal, `pkill`, or OOM. This closes a prior blind spot where an interrupted **START**
    (e.g. a `--restart` race while Wi-Fi was re-associating) left the log ending abruptly after the teardown
    with no indication of what failed. If a toggle "does nothing" or the gateway ends up half-up, `grep -E '
    EXEC:|EXIT ' output.log | tail` shows the last command run and its result.
397 - **`DEBUG_TRACE=true`** (in `.env`) Full command-by-command execution trace of `toggle.sh`, mirrored to `
    output.log` for deep post-mortems of intermittent failures. Each traced line is timestamped and tagged with
    `file:line:function` (via a custom `PS4`), so you can follow the *exact* code path a run took including
    subshells, Docker/Colima calls, and where it aborted. Off by default because it is verbose (the log grows
    quickly); enable it only while reproducing a specific problem, then set it back to `false`. Implementation
    note: it prefers bash's `BASH_XTRACEFD` (bash 4.1, e.g. most Linux) to send the trace to the log while
    keeping your terminal clean; on macOS's stock `/bin/bash` 3.2 it transparently falls back to tee'ing stderr
    to the log (trace also appears in the terminal). It never uses the dynamic-fd (`exec {var}>>`) form, so it
    is safe on 3.2. To read a trace: `grep -nE '\+' output.log`.
398 - **`bash scripts/diagnose-host-leak.sh`** One-shot host-routing diagnostic. Runs 8 checks and classifies your
    state into one of four known scenarios (System Extension conflict, SOCKS5 fallback, IPv6 leak, route-table
    freeze) or OK, and prints the exact fix command. Check 5 uses `route -n get 1.1.1.1` to ask the kernel
    which interface real traffic actually resolves through (more accurate than `netstat -rn | grep default` on
    macOS). Pass `--fix` to apply the remediation automatically. Pass `--watch` (or `--watch 30`) to run a full
    baseline diagnostic then continuously monitor warp/IPv6/default-route for leaks, alerting on any state
    change.
399 - **`bash scripts/sweep.sh`** One-shot, read-only gateway health sweep. Runs 7 checks containers up/healthy,
    double-tunnel/IP-leak (host egress == container egress with `warp=on`), direct P2P to the gateway (no DERP

```

```

relay), AdGuard compiled-rule counts + live DNS interception, PF kill-switch armed, tailnet reachability (a
*burst* of `tailscale ping`s to every online peer, reporting loss % and RTT jitter not just one-shot
latency and flagging DERP-relayed peers as throughput-degraded), and real throughput (host path vs. a WARP
-only baseline via the container's SOCKS5 proxy, against a fast CDN no exec/install inside the container)
then rolls them into a PASS/WARN/FAIL verdict. Writes a timestamped `sweep-<UTC>.txt` report to the repo
root (diffable across runs) and exits non-zero on FAIL, so it can gate launchd hooks or CI. Pass `--quiet`
for just the verdict line. It never mutates routing, PF, DNS, or containers safe to run against a live
gateway at any time.
400 - **`bash scripts/fix-docker-bridge-collision.sh`** One-shot fix for Docker bridge IP conflicts. If your local
Wi-Fi router assigns you an IP in the `172.17.0.0/16` range, it will violently collide with Docker's
default internal bridge, permanently killing your internet. This script detects the collision and auto-
patches your Colima VM to use a safe subnet (e.g. `10.200.0.1/24`).
401 - **`devref.md`** Complete architecture reference, routing deep-dives, and full resolved-issues log. Read this
before modifying any routing or firewall logic.
402
403 > [!CAUTION]
404 > **Two infinite routing loops are possible.** (1) If a hotspot device providing internet to the gateway host
is *also* using the gateway as its exit node, packets bounce forever between the two. (2) When the host's
default route is redirected to `utun*`, WARP endpoint packets can loop back into the tunnel. Both are
documented in `devref.md 15.2.1` and `15.2.2` and are automatically handled by `toggle.sh`.
405
406 ---
407
408 ## 10. Unified Project Document (Quine)
409
410 The project includes a `generate_tex.py` script that compiles the entire source code, configurations, and
documentation into a single, unified LaTeX document (`nullexit_unified.tex` / `.pdf`).
411
412 Funnily enough, this document acts as a "project-level quine." It encapsulates the entirety of its own source
code, its documentation, and the exact Python code required to generate itself. It serves as a self-
contained, long-term archiving strategy providing everything needed to reconstruct and understand the `
nullexit` system from a single file.
413
414 ---
415
416 ## 11. Acknowledgements
417 - **[SyameimaruKoa](https://github.com/SyameimaruKoa):** Dual-stack TCP MSS clamping, `SIGHUP` state-tracking
in the routing sidecar, and `TS_AUTH_ONCE` integration.
418
419 ## 12. License
420 GNU Affero General Public License v3. See [LICENSE](./LICENSE).
421
422 ## 13. Changelog
423
424 - **July 23, 2026** Wi-Fi resilience + logging cleanup. (1) **Wi-Fi auto-rejoin**: if the Mac drops Wi-Fi with
auto-join off it no longer just sits there the always-on `watcher.sh` daemon re-associates a *remembered*
network (`scripts/wifi-rejoin.sh`; no Location, no SSID scanning) with a grace period, backoff, and give-
up cap. Driven by the always-on daemon, not the gateway's WARP Watcher, so it works whether the gateway is
up, down, or off. See `README 6` and `devref.md 15.5.9`. (2) **WARP Watcher no longer false-kills the
gateway on a Wi-Fi drop**: it now checks the physical uplink before counting a `warp=off` as a tunnel
failure and PAUSEs (leak-safe: no routable IP = no egress) instead of nuking the gateway root cause of the
2026-07-22 outage. See `devref.md 15.6.2`. (3) **Logging unified to UTC** `output.log`, `blocked.log`,
and the AdGuard query log previously mixed UTC and local (EDT); everything is now UTC. See `devref.md
15.12.26`.
425 - **July 18, 2026** Fixed two real bugs surfaced by live sweep evidence: the QUIC/DoT `REJECT` rules in `post-
rules.txt` were shadowed by an earlier blanket `ACCEPT` (0 packets ever matched the intended Instagram/
QUIC-fragmentation fix from `devref.md 15.3.3` was never actually enforced), now reordered ahead of it; and
the host-side `pf.conf` MSS clamp was stuck at the pre-`15.3.2` value of `1160` while the container had
already moved to `1120` `scripts/common.sh` now syncs both from `GATEWAY_MSS` so they can't drift apart
again. Also fixed `setup.sh`/`scripts/setup-common.sh` never generating `NULLEXIT_SEED` (every fresh
install hard-failed its first `toggle.sh` on a missing-seed crypto error) and defaulting `KILL_SWITCH` to `
false` for new installs see `devref.md 15.12.23`. `scripts/sweep.sh` gained a 7th check (**throughput**,
host path vs. a WARP-only baseline against a fast CDN) see `devref.md 15.3.6` for why the target choice
matters and `9` below for the check itself. Check 6 (tailnet reachability) was upgraded from a single ping
to a **burst** (loss % + RTT jitter per peer) a peer can be 100% reachable and still have real quality
problems a single ping hides; caught live on an S24 over DERP (0% loss, 665ms jitter). Confirmed that
jitter isn't gateway-fixable (mobile-radio power state + DERP's TCP-relay timing, verified via a Wi-Fi-vs-
cellular comparison) and that forcing P2P through confirmed AP isolation isn't possible either (Layer 2
block, below anything Tailscale operates at) see `devref.md 16.4`.
426 - **July 17, 2026** `scripts/sweep.sh` (the read-only automated health sweep added July 15, mirroring `sweep.
md 1`) gained a 6th check: **tailnet reachability** every Tailscale peer reported online is probed with `
tailscale ping` (TSMP, so it works even on phones that drop ICMP) and rolled into the PASS/WARN/FAIL
verdict. Documented in 9 above.
427 - **July 12, 2026** Tor ControlPort dynamic password authentication (unified with `ADGUARD_PASSWORD`),
enabling live circuit inspection via `sweep`. See `devref.md 15.10.2`.
428 - **July 11, 2026** Tor container hardening: `obfs4proxy` restored (base image `debian:bookworm-slim`),
robust bridge-file validation, and image pinning. See `devref.md 15.10.1`.
429 - **July 10, 2026** Roaming/startup stabilization suite: fixed the Wi-Fi-roam host-IP leak (raw ISP `132.x`)
caused by the WARP Watcher racing post-wake recovery, plus a batch of pre-flight/concurrency/kill-switch/
Tailscale-flag bugs and the macOS SNAT endpoint-poisoning P2P blackhole. Full post-mortems in the Incident
Log see `devref.md 15.2.7`, `15.5.3`-`15.5.5`, `15.2.8`-`15.2.9`, `15.7.1`, `15.7.3`-`15.7.5`, `15.12.18`.

```



```

430 - **July 6, 2026** Edge-case security and stability hardening: DNS Watcher interface independence, robust lock
      -file PID verification, fail-closed crypto integrity check, strict `iptables` pinning for tailscale0tun0,
      Docker port binding to `127.0.0.1` to prevent LAN exposure, routing verification via `netstat -nr`. See `
      devref.md 17`.
431 - **July 4, 2026** Colima bridged networking, SOCKS5 proxy migrated natively to Tailscale, headless prompt
      crashes fixed, proxy bypass domains added to fix LAN P2P. Python dependencies completely eliminated: `
      logger` and `rule-compiler` ported to blazing-fast Go multi-stage Docker builds. Added `crypto.sh` to
      enforce cryptographic signature validation on all core bash scripts. Massively refactored `toggle.sh` and `
      recover.sh` to eliminate ~200 lines of duplicated code by migrating network and process logic to `scripts/
      common.sh`. Added a concurrency mutex to `toggle.sh`, resolved unbound `TS_AUTHKEY` check crashes on setup,
      and integrated Go validation to the CI pipeline.
432 - **July 3, 2026** **Critical Security Updates:** Addressed the overnight silent IP leak and improved geo-
      blocking. Introduced `scripts/host-leak-probe.sh` (a sub-second host-egress leak detector) and a
      statistical threshold auto-shutdown watcher. Added `unlock-files.sh` to safely reset file permissions.
      Resolved numerous startup regressions and system bugs.
433 - **July 2, 2026** Two infinite routing loops resolved (`devref.md 15.2.2`): host-VM tunnel loop (WARP bypass
      routes now via gateway IP + manual `utun*` redirection) and Docker Compose subnet takeover (`10.200.1.0/24`
      lock). `--accept-routes=true` now explicit on all `tailscale up --reset` calls.
434 - **July 2, 2026** Auto-recovery daemon documented in 6 above (`scripts/watcher.sh` + `launchd plist`). See `
      devref.md 15.5.1`.
435 - **July 2, 2026** Linux scripts (`scripts/toggle-linux.sh`, `recover-linux.sh`, `setup-linux.sh`) documented
      in 6.
436 - **July 1, 2026** `recover.sh --post-wake` ships; wired to watcher daemon. Triggered on every sleep/wake or
      network-state change.
437 - **June 28, 2026** 8 internal scripts moved to `scripts/`. User-facing files (`toggle.sh`, `recover.sh`, `
      setup.sh`, `docker-compose.yml`) stay at repo root.
438 - **June 28, 2026** All hardcoded install paths removed. `SCRIPT_DIR`-relative resolution and `
      _NULLEXIT_HOME` placeholder used throughout.
439
440 ### Cryptographic Integrity Verification
441
442 nullexit uses a built-in cryptographic integrity checker designed to provide tamper-*evidence* against
      accidental edits, logic bugs, or non-privileged malware. During setup, a 256-bit `NULLEXIT_SEED` is
      injected into your `.env` file. The core endpoint scripts (`toggle.sh`, `recover.sh`, etc.) are hashed
      using HMAC-SHA256, and their signatures are stored in `.signatures`.
443
444 *(Note: This is not a strict boundary against an attacker who already has write access to the repo, as they
      could simply read the seed and re-sign the scripts. It is designed as a strict footgun-catcher).*
445
446 If any script is modified without authorization, the gateway will loudly fail to boot. If you make intentional
      edits to the bash scripts, you must re-sign them by running:
447 ```bash
448 ./scripts/crypto.sh --sign
449 ```

```

3 agent.md

```

1 # nullexit Detailed Agent Analysis
2
3 > Complete per-file breakdown of every function, constant, duplication, and bug found.
4 > **Note (July 2026):** `sync-rules.py` and `logger.py` have been fully ported to Go (`scripts/rule-compiler/
      main.go` and `scripts/logger/main.go`) to dramatically improve performance and reduce container footprint
      via multi-stage Alpine builds. The analysis below reflects their previous Python states.
5
6 ## Important Agent Instruction
7 - **NEVER use `cat << EOF` or terminal redirections to write or modify files.** You must always use your native
      file-editing tools (`replace_file_content` or `multi_replace_file_content`). Using `cat EOF` leads to
      duplicated text, broken markdown headers, and structural corruption (which previously destroyed the section
      numbering in `devref.md`).
8 - **Always run `bash scripts/crypto.sh --sign` after making any modifications to the core bash scripts.** This
      is required to update the cryptographic HMAC-SHA256 signatures; otherwise, the startup integrity checks
      will fail and block execution.
9 - **Always run `git diff` and print the changes to the user *before* requesting or executing any git commit
      command.** The user must be shown the diff so they can review and understand the changes. Based on this
      diff, formulate a highly precise, detailed, and descriptive commit message (explaining exactly what was
      changed and why) rather than a generic summary, and present it to the user alongside the diff.
10 - **Always run `git status` before `git diff` to identify all modified and untracked files, ensuring no files (
      such as new scripts or configuration assets) are missed before formulating commit diffs and staging.**
11 - **Always use the `--restart` flag when asked to restart the toggle (e.g., run `./toggle.sh --restart`).**
12 - **NEVER execute any `sudo` commands (especially `sudo route`, `sudo dscacheutil`, or any network-modifying
      commands) without explicitly explaining to the user what the command does and why it is needed FIRST. Do
      not assume implicit permission. Wait for the user's approval before running it.**
13 - **When fixing an error and using logs to debug, always show the user the reasoning and the specific logs that
      support this theory.**
14 - **When the user tells you to "push", this means read git diffs and prepare commit messages that correspond to
      these changes (do not ignore any changes). If the changes can be atomized into multiple commits for easier
      understanding, do so.**

```

```

15 - **When the user says "latex this", it means you should run `python3 scripts/generate_tex.py` to regenerate
    the unified LaTeX document.**
16 - **NEVER apply changes to running gateway containers directly via `docker compose up` or `docker compose
    restart`.** Recreating or restarting containers (especially the `warp` container which hosts the network
    namespace) breaks the shared network stack for all other services (`adguardhome`, `tailscale`, `routing-fix`
    `), severing host connectivity and freezing macOS DNS. If you need to apply changes or rebuild containers,
    cleanly stop the gateway first using `./toggle.sh`, or trigger `recover.sh --post-wake` to rebuild and re-
    verify the network stack safely in the correct dependency order. Do NOT run `./toggle.sh --restart` or any
    network-rebuilding commands yourself if the execution could sever host network access; instead, explain the
    changes and ask the USER to run `./toggle.sh --restart` (or `./toggle.sh` stop and start) themselves to
    safely apply the modifications.
17
18 ## How to Verify Gateway is Working
19 Whenever modifications are made to the gateway scripts, routing, or containers, execute these verification
    checks in order:
20 1. **Container Health:** Run `docker compose ps` to ensure all containers (`warp`, `adguardhome`, `routing-fix`
    `), `tailscale`) are `running` and `healthy`.
21 2. **DNS Hijacking Status:** Run `networksetup -getdnsservers "Wi-Fi"` (or appropriate network service) and
    ensure it is set exclusively to the gateway's Tailscale static IP (typically `100.100.21.8`).
22 3. **DNS Query Interception:** Run `dig @100.100.21.8 google.com` (using the gateway static IP from `.`
    gateway_ip`) to ensure AdGuard Home is active, intercepting, and resolving queries.
23 4. **Double-Tunnel Egress:** Run `curl -s https://www.cloudflare.com/cdn-cgi/trace | grep warp` to verify the
    egress is routed through Cloudflare WARP (`warp=on`).
24 5. **Host Leak Monitoring:** Run `tail -n 30 output.log` and verify the background watchers (DNS + WARP) report
    healthy status without `LEAK` warnings. For a deeper on-demand audit, run `bash scripts/diagnose-host-leak
    .sh` (add `--watch` to poll, `--fix` to remediate).
25
26 **Automated:** `bash scripts/sweep.sh` runs a read-only 7-check health sweep covering the above plus the PF
    kill-switch, tailnet-wide peer reachability, and real throughput, writes a timestamped `sweep-<UTC>.txt`
    report to the repo root, and exits non-zero on FAIL (`--quiet` for just the verdict). It never mutates
    routing/PF/DNS/containers, so it is safe against a live gateway. See its entry in Part 3.
27
28 ---
29
30 ## Part 1: macOS Main Scripts
31
32 ### toggle.sh (2191 lines, unified macOS + Linux)
33
34 > **Refreshed 2026-07-15.** Post-unification (`$OSTYPE` switch at L20) + the July-2026 `common.sh` extraction,
    the tables below reflect the *current* file. Most former "toggle.sh functions" now live in `scripts/common.
    sh`; the host-leak-probe pair was removed. For the ordered execution flow see **"toggle.sh Execution Map &
    Critical Path"** above.
35
36 **Functions still defined in `toggle.sh`** several exist in *both* halves (Linux L63848 / macOS L8782191):
37
38 | Function | Linux L | macOS L | Purpose |
39 |-----|-----|-----|-----|
40 | `_log_exit_breadcrumb()` | 30 | 885 | Write lifecycle-phase breadcrumb on exit (post-mortem trail) |
41 | `_get_free_port()` | 67 | 944 | Pick a random unused port (self- + kernel-collision checked) |
42 | `_run_gui_cmd()` | 1033 | Run a command as the logged-in console GUI user |
43 | `_write_gateway_active_marker()` | 1053 | Write `/tmp/nullexit-gateway-active.marker` (macOS only) |
44 | `_clear_gateway_active_marker()` | 1059 | Remove active marker + `TUNNEL_FAILED_CLOSED.marker` |
45 | `_start_sleep_prevention()` | 145 | 1091 | Start `caffeinate` w/ revert trap |
46 | `_stop_sleep_prevention()` | 188 | Stop caffeinate via PID file |
47 | `_stop_dns_watcher()` | 202 | Stop DNS watcher (macOS uses `stop_pidfile_daemon`) |
48 | `_start_warp_watcher()` | 1164 | Background WARP liveness monitor `recover.sh` after N fails |
49 | `_stop_warp_watcher()` | 1241 | Stop WARP watcher via PID file |
50 | `_cleanup_handler()` | 215 | 1258 | Fail-closed teardown trap (guarded by `SUCCESS_RUN`) |
51 | `_restart_tailscaled_daemon()` | 327 | *(common)* | Restart tailscaled (Linux-half local copy; macOS uses
    common.sh) |
52 | `_disconnect_tailscale_host()` | 337 | *(common)* | `tailscale down` w/ retry (Linux-half local copy; macOS
    uses common.sh) |
53 | `_setup_exit_node_routing()` | 1430 | Override default route through the Tailscale utun interface |
54 | `_cleanup_network_state()` | 354 | 1464 | Nuclear cleanup: proxies, DNS cache, routes, Wi-Fi, sharing services
    |
55
56 **Delegated to `scripts/common.sh`** (moved out of toggle.sh in the July-2026 refactor; line = common.sh):
57
58 | Function | common.sh L | Function | common.sh L |
59 |-----|-----|-----|-----|
60 | `_run_with_timeout()` | 58 | `_enable_socks_proxy()` | 769 |
61 | `_is_gateway_active()` | 168 | `_disable_socks_proxy()` | 803 |
62 | `_gateway_state()` | 232 | `_reset_dns()` | 824 |
63 | `_get_active_service()` | 284 | `_stop_local_dns_proxy()` | 898 |
64 | `_restart_tailscaled_daemon()` | 338 | `_start_local_dns_proxy()` | 909 |
65 | `_disconnect_tailscale_host()` | 352 | `_force_dns_to_gateway()` | 988 |
66 | `_stop_pidfile_daemon()` | 393 | `_add_warp_bypass_routes()` | 484 |
67 | `_start_dns_watcher()` | 715 | `_remove_warp_bypass_routes()` | 551 |
68
69 **Removed:** `start_host_leak_probe()` / `stop_host_leak_probe()` and the whole `HOST_LEAK_PROBE` prober
    subsystem no longer defined or called anywhere in `toggle.sh`, `recover.sh`, or `common.sh`, and `

```

```

HOST_LEAK_PROBE` is no longer an env var (all verified). The only surviving host-leak tool is the
standalone, on-demand `scripts/diagnose-host-leak.sh` (there is **no** `scripts/host-leak-probe.sh`).
70
71 **Constants / anchors (current):**
72
73 | Constant | Value | Line |
74 |-----|-----|-----|
75 | `LOG_FILE` | `$PWD/output.log` | L22 (Linux) / L863 (macOS) |
76 | `LOCK_FILE` | `/tmp/nullexit-toggle.lock` | L1020 |
77 | `PID_FILE` (caffeinate) | `/tmp/nullexit-caffeinate.pid` | L142 |
78 | `DNS_WATCHER_PID_FILE` | `/tmp/nullexit-dns-watcher.pid` | L200 |
79 | `WARP_WATCHER_PID_FILE` | `/tmp/nullexit-warp-watcher.pid` | L1144 |
80 | Gateway active marker | `/tmp/nullexit-gateway-active.marker` | L1054 (write) |
81 | TUNNEL_FAILED_CLOSED marker | `### toggle.sh Execution Map & Critical Path (verified 2026-07-15)
91
92 > Traced from the current unified `toggle.sh` (2191 lines). The file carries **two halves** selected by `
    $OSTYPE` at **L20** (if [[ "$OSTYPE" != "darwin"* ]] runs the self-contained **Linux path L63848** then
    `exit 0` at L845; darwin falls through to the **macOS path L8782191**). Line numbers below are the **
    macOS path** (the deployment target). The halves are *similar in intent but not identical* see "Linux path
    where it diverges" below. `~Line` = current, approximate.
93
94 #### Entry & dispatch
95
96 ...
97 ./toggle.sh [ (no-arg) | --restart | restart ]
98
99 | -- verify HMAC signatures (crypto.sh --verify) --fail--> exit 1 (L857)
100 | -- rotate output.log if > 1GB
101 | -- arm teardown trap: ERR/INT/TERM/HUP -> cleanup_handler (L1317-1320)
102 | -- generate random ports (Tor / SOCKS / DNS) (L940)
103 |
104 | -- --restart --> gateway_state? running -> run STOP, then START (L999)
105 |
106 | -- no-arg --> gateway_state? (L1535)
107 |         running -> STOP branch (L1538-1584)
108 |         stopped -> START branch (L1590-2191)
109
110 ...
111 `gateway_state` is driven by `/tmp/nullexit-gateway-active.marker` (persisted intent), falling back to a live
    container probe.
112
113 #### START synchronous critical path (all must complete)
114
115 | # | Step | ~Line | Effect if it fails / is interrupted |
116 |---|-----|-----|-----|
117 | 1 | `disconnect_tailscale_host` (clean slate) | 1595 | host left mesh |
118 | 2 | `colima_start_until_ready` (readiness poll, 15.8.7) | 1631 | no Docker VM abort |
119 | 3 | honey-port reap (`pkill`) + arm (`nc -l`, disowned) | 1695 | tripwire missing (non-fatal) |
120 | 4 | rule-compile (hash-gated, OFF critical path 15.8.8) | 1708 | reuse prior compiled rules |
121 | 5 | build-gate routing-fix / tor images | 1743 | rebuild only on change |
122 | 6 | `docker compose up -d` warp, tailscale, routing-fix, adguard, tor | 1758 | no gateway |
123 | 7 | wait tailscaled reachable | 1907 | host tailscale unusable |
124 | 8 | **host tailscale set --exit-node=$TS_IP** | 2046 | **host not routed via gateway REAL-IP LEAK** |
125 | 9 | `add_warp_bypass_routes` + `setup_exit_node_routing` | 2048 | WARP endpoint unreachable / wrong route |
126 | 10 | **force_dns_to_gateway** (+ `start_local_dns_proxy`) | 2078 | host DNS not hijacked |
127 | 11 | `enable_socks_proxy` + verify SOCKSWARP | 2100 | SOCKS off |
128 | 12 | print **"STARTED in Ns"* (timing marker) | 2127 | cosmetic only NOT the commit point |
129 | 13 | `start_sleep_prevention` (caffeinate) | 2130 | Mac may sleep |
130 | 14 | `start_dns_watcher` (re-hijack DNS every 30s) | 2134 | no roam DNS recovery |
131 | 15 | `start_warp_watcher` (warp liveness recover.sh) | 2138 | no WARP self-heal |
132 | 16 | **write_gateway_active_marker** | 2146 | watcher believes gateway is DOWN |
133 | 17 | **SUCCESS_RUN=true** disarms the teardown trap | 2164 | *(true commit point)* |
134 | 18 | print "You can close this terminal window now." | 2191 | |
135
136 #### STOP critical path
137
138 `disconnect_tailscale_host` (1543) `docker compose down --remove-orphans` (1552) `colima stop` (1558) `
    reset_dns` (1568) `stop_local_dns_proxy` (1571) stop caffeinate (1574) stop DNS watcher (1575) `
    stop_warp_watcher` (1576) `clear_gateway_active_marker` (1577) "STOPPED in Ns" (1584).
139
140 #### Long-lived background processes spawned by START

```



```

141 | Process | Started | PID / reap | Role |
142 |-----|-----|-----|-----|
143 | honey-port `nc -l localhost $PORT` (disowned) | L1695 | `kill -f "nc -l localhost"` | port-scan tripwire |
144 | `caffeinate` | L2130 | `/tmp/nullexit-caffeinate.pid` | prevent system sleep |
145 | DNS watcher loop | L2134 | `/tmp/nullexit-dns-watcher.pid` | re-hijack DNS every 30s (Wi-Fi roam) |
146 | WARP watcher loop | L2138 | `/tmp/nullexit-warp-watcher.pid` | monitor warp; fire recover.sh after N fails |
147 | local DNS proxy `dns-proxy.py` | L2089 | (killed by `stop_local_dns_proxy`) | UDP:53 gateway DNS fallback |
148
149 Separately, the launchd `scripts/watcher.sh` daemon (post-wake / post-roam recovery) runs independently of
150 toggle and invokes `recover.sh --post-wake`; toggle does not spawn it.
151
152 ##### `cleanup_handler` the fail-closed teardown trap
153
154 Armed on `ERR/INT/TERM/HUP` (L1317-1320), guarded by `SUCCESS_RUN`. While `SUCCESS_RUN=false` it runs: `
155   reset_dns`, `disconnect_tailscale_host`, `stop_local_dns_proxy`, stop watchers, `
156   clear_gateway_active_marker`, `docker compose down` (+ `colima stop` on `ERR`). This is the fail-closed
157   guarantee: a half-built gateway is torn down, not left leaking.
158
159 ##### Correctness findings (verified against source)
160
161 1. "STARTED in Ns" (L2127) is NOT the commit point `SUCCESS_RUN=true` (L2164) is. The marker write (L2146)
162   and watcher starts (L2130-2138) sit between them. Any TERM/INT including a wrapping tool's timeout
163   in the L2127L2164 window fires cleanup_handler` a FULL teardown` (containers + colima + host tailscale),
164   leaving the marker ABSENT and the host on its raw ISP link, despite the printed success. This is exactly
165   the 2026-07-15 incident. (The Linux half has the same shape but a much narrower window: STARTED L814 `
166   SUCCESS_RUN=true` L820, and it maintains no marker.)
167
168 2. Never pipe toggle.sh` through a reader` (`tail`, `grep`). The disowned children (honey-port,
169   watchers, dns-proxy, caffeinate) inherit stdout, so the pipe never EOFs and the reader hangs long after
170   toggle has exited which then trips the wrapper's timeout SIGTERM finding #1. Run detached with a file
171   redirect instead: nohup ./toggle.sh --restart > /tmp/t.log 2>&1 &`, then poll the file.
172
173 3. Host wiring is correctly on the synchronous critical path (steps 8 & 10 precede "STARTED"), so a 
174   completed START cannot leave the host unrouted/leaking the only way to that state is an interruption per
175   finding #1.
176
177 4. Namespace ordering: step 6's docker compose up` creates warp's network namespace that `tailscale`/`
178   routing-fix`/`adguard` share (compose `depends_on: warp healthy` enforces order). Never docker compose
179   restart warp` on a live gateway it detaches the shared netns (see Agent Instruction, L16).
180
181 ##### Linux path where it diverges (L63848)
182
183 The Linux half is intent-similar but not a line-for-line mirror of macOS. Verified differences:
184
185 | Aspect | macOS (L8782191) | Linux (L63848) |
186 |-----|-----|-----|
187 | VM layer | Colima (`colima_start_until_ready`, L1631) | none native Docker |
188 | Host exit-node assert | `$TS_BIN` set --exit-node="$TS_IP" (L2046) | `$TS_BIN` up --reset --exit-node="
189   $TS_IP" (L735) |
190 | gateway-active marker | written (L2146) | not maintained (L119) `gateway_state` degrades to a live
191   container probe |
192 | Commit window | STARTED L2127 `SUCCESS_RUN` L2164 | STARTED L814 `SUCCESS_RUN` L820 |
193 | Port gen tooling | `jot` / `netstat` | `shuf`/`$RANDOM` / `ss` |
194
195 The up --reset` vs `set` distinction matters: --reset` re-applies all prefs and forces a route re-eval,
196   whereas set` mutates only the exit-node pref the same asymmetry recover.sh --post-wake` relies on (
197   devref 15.12.21).
198
199 ---
200
201 ##### recover.sh (566 lines, 23KB)
202
203 Functions (7 total):
204
205 | # | Function | Lines | Purpose |
206 |---|-----|-----|-----|
207 | 1 | `step()` | L40 | Print bold step header |
208 | 2 | `ok()` | L41 | Print green success message |
209 | 3 | `warn()` | L42 | Print yellow warning message |
210 | 4 | `fail()` | L43 | Print red failure message |
211 | 5 | `die()` | L44 | Print red error and exit |
212 | 6 | `run_with_timeout()` | L5674 | Run command with timeout (bash-native, no GNU coreutils) |
213 | 7 | `restart_tailscaled_daemon()` | L118129 | Restart tailscaled daemon via brew services |
214
215 Constants:
216
217 | Constant | Value | Line |
218 |-----|-----|-----|
219 | Color codes | `RED`, `GREEN`, `YELLOW`, `NC` | L37-38 |
220 | `SCRIPT_DIR` | `${dirname "${BASH_SOURCE[0]}"}` | L50 |
221 | `PATH` export | `/opt/homebrew/bin:/usr/local/bin:$PATH` | L77 |
222 | `PID_FILE` | `/tmp/nullexit-caffeinate.pid` | L387 |
223 | `DNS_WATCHER_PID_FILE` | `/tmp/nullexit-dns-watcher.pid` | L407 |

```

```

202 | WARP endpoints | `162.159.192.1`, `162.159.193.1` | L329-336 |
203 | TUNNEL_FAILED_CLOSED marker | `${SCRIPT_DIR}/TUNNEL_FAILED_CLOSED.marker` | L51 |
204
205 **Duplicated logic from toggle.sh:**
206 *(Note: As of July 2026, **ALL** of the below duplicated logic has been successfully extracted to `scripts/
    common.sh`!)*
207 - `run_with_timeout()` simpler subshell version vs toggle's PID-tracking version
208 - `restart_tailscaled_daemon()` near-identical (uses warn()/ok() vs echo)
209 - Active network service detection inline at L217-233 (toggle has `get_active_service()` function)
210 - ENO service resolution inline at L230-233 (same as toggle L515-516)
211 - WARP bypass routes inline at L329-337 (toggle has `add_warp_bypass_routes()`)
212 - Exit node routing setup inline at L340-345 (toggle has `setup_exit_node_routing()`)
213 - DNS cache flushing L296-302 (same as toggle L611-616)
214 - Wi-Fi power-cycling L442-461 (similar to toggle L626-637)
215 - Proxy disabling L282-288 (same as toggle L604-608)
216 - Sharing services reset L586 (same as toggle L642)
217 - PID file stop pattern L387-399, L407-419, L423-435 (same as toggle L157-167, L193-203, L312-322)
218 - KILL_SWITCH check L194 (same as toggle L141, L469)
219 - .gateway_ip reading L138-142, L209-213 (same pattern as toggle L1080)
220
221 ---
222
223 ### setup.sh (410 lines, 18KB)
224
225 **Functions (4 total):**
226
227 | # | Function | Lines | Purpose |
228 |---|-----|-----|-----|
229 | 1 | `step()` | L10 | Print bold blue step header |
230 | 2 | `ok()` | L11 | Print green success message |
231 | 3 | `warn()` | L12 | Print yellow warning message |
232 | 4 | `die()` | L13 | Print red error and exit |
233
234 **Constants:**
235
236 | Constant | Value | Line |
237 |-----|-----|-----|
238 | Color codes | `RED`, `GREEN`, `YELLOW`, `BLUE`, `BOLD`, `NC` | L7-8 |
239 | `RECOMMENDED_TS_VERSION` | `1.98.5` | L71 |
240 | `ADGUARD_PASSWORD` | `nullexit` | L215 |
241 | Default .env values | `GATEWAY_RULE_PROFILE=medium`, `GATEWAY_MSS=1120`, etc. | L240-248 |
242
243 ---
244
245 ## Part 2: Linux Scripts
246
247 ### toggle.sh Linux path (unified into the root cross-platform script)
248
249 The Linux toggle is no longer a separate file. `scripts/toggle-linux.sh` was
250 merged into the repo-root `toggle.sh`, which dispatches on `OSTYPE`: an early
251 `if [[ "$OSTYPE" != darwin* ]]` block runs the self-contained Linux toggle
252 (shuf / ss / nmcli / ip / systemd) and exits before the macOS body. The shared
253 DNS/proxy/daemon primitives it used were already moved to `common.sh` in the
254 `313dc32` unification, so both branches call them by the same names.
255
256 ---
257
258 ### recover.sh Linux path (unified into the root cross-platform script)
259
260 Linux recovery is no longer a separate file. `scripts/recover-linux.sh` was
261 merged into the repo-root `recover.sh`, which dispatches on `OSTYPE`: the Linux
262 branch runs a self-contained teardown (resolvectl / nmcli / ip) and exits before
263 the macOS dual-mode body. All formatting/timeout helpers come from `common.sh`.
264
265 ---
266
267 ### scripts/setup-linux.sh (416 lines, 18KB)
268
269 **Functions (4 total):**
270
271 | # | Function | Line | Purpose |
272 |---|-----|-----|-----|
273 | 1 | `step()` | L10 | Print formatted step header |
274 | 2 | `ok()` | L11 | Print green success message |
275 | 3 | `warn()` | L12 | Print yellow warning message |
276 | 4 | `die()` | L13 | Print red error + exit 1 |
277
278 ***~95% identical to macOS setup.sh.*** Only differences:
279 1. Desktop shortcuts (L366-391 Linux `.desktop` files vs macOS `.app` via `osacompile`)
280 2. Final instructions text
281 3. .env template: macOS adds `GATEWAY_USE_EXIT_NODE`, `WARP_FAIL_THRESHOLD`, `KILL_SWITCH` Linux omits these

```

```

282 ---
283 ---
284
285 ## Part 3: Utility Scripts
286
287 ### scripts/diagnose-host-leak.sh (722 lines, 38KB)
288
289 **Purpose:** Comprehensive host-routing diagnostic. 8 checks classify host IP leaks. Supports `--fix` and `--
    watch` modes.
290
291 **Duplicated logic:**
292 - `resolve_active_service()` identical pattern as toggle.sh, recover.sh, fix-docker-bridge-collision.sh
293 - `run_with_timeout()` reimplemented timeout
294 - Color codes RED/GREEN/YELLOW/BOLD/NC with tty detection
295 - WARP check via `cloudflare.com/cdn-cgi/trace`
296 - `export PATH="/opt/homebrew/bin:/usr/local/bin:$PATH"`
297 - .gateway_ip reading pattern
298
299 ### scripts/fix-docker-bridge-collision.sh (402 lines, 18KB)
300
301 **Purpose:** Fixes Docker bridge subnet collisions with `10.200.1.0/24`.
302
303 **Duplicated logic:**
304 - `resolve_active_iface()` same interface detection as diagnose-host-leak.sh
305 - Color codes, step()/ok()/warn()/fail()/die() formatting helpers
306
307 ### scripts/device-scramble.sh (159 lines, 7.8KB)
308
309 **Purpose:** Randomizes hostname and MAC address for Wi-Fi anonymity. Includes fail-safe MAC test and
    disassociate-first logic.
310
311 **Duplicated logic:**
312 - `need_root()` same sudo check as other scripts
313 - `WARP_INHIBIT` marker interacts with watcher.sh to prevent gateway shutdown during Wi-Fi blip
314
315 ### scripts/fix-ssh-delay.sh (63 lines, 2.7KB)
316
317 **Purpose:** One-shot fix for ~20s SSH delay. Standalone, minimal duplication (uses hardcoded / instead of
    color functions).
318
319 ### scripts/reinstall-host-tailscale.sh (740 lines, 31KB)
320
321 **Purpose:** Complete uninstall + reinstall of host Tailscale.
322
323 **Duplicated logic:**
324 - `with_timeout()` yet another bash timeout implementation (polling-based, different from diagnose and toggle
    versions)
325 - Color codes + logging functions
326 - System Extension detection logic (similar to diagnose-host-leak.sh)
327
328 ### scripts/routing-fix.sh (266 lines, 10.5KB)
329
330 **Purpose:** Runs INSIDE warp container. Maintains routing table 200, FORWARD rules, country blocking, IP
    blocklists, tunnel health checks.
331
332 **Duplicated logic:**
333 - (Fixed) WARP_ENDPOINT defaults are now centralized in common.sh
334 - WARP health check via cdn-cgi/trace (another instance)
335 - iptables dual-backend pattern (for-loop over `iptables iptables-legacy`)
336
337 ### scripts/sweep.sh (491 lines, 28KB)
338
339 **Purpose:** Read-only post-restart health sweep answers "after a toggle/restart/wake, is the gateway actually
    healthy end-to-end including under real load?" Never mutates routing, PF, DNS, or containers, so it is
    safe to run against a live gateway at any time (including from a remote session). `errexit` is deliberately
    OFF: probes are *expected* to fail on an unhealthy gateway and each exit status is recorded as the
    diagnostic signal.
340
341 **The 7 checks** (each recorded PASS/WARN/FAIL; exit 0 only when nothing FAILs, so it can gate CI/launchd/post-
    restart hooks):
342 1. **Containers** `docker compose ps`: warp/adguardhome/tailscale/tor/routing-fix up (warp + adguardhome
    additionally `healthy`).
343 2. **Double-tunnel/leak** container WARP egress IP == host egress IP, both `warp=on` (gluetun ships wget not
    curl, so the in-container trace falls back to wget).
344 3. **Hostcontainer path** the gateway peer line in `tailscale status` shows `direct <ip:port>`; WARNs on a
    DERP relay.
345 4. **AdGuard filtering** `compiled_rules.txt` block counts present + live interception (`dig @` the gateway IP
    from `.gateway_ip`).
346 5. **PF kill-switch** `pfctl` reports Enabled AND anchor `com.apple/nullexit` carries rules (needs
    passwordless sudo, else WARN).

```

```

347 6. **Tailnet reachability** (added 2026-07-17; upgraded to burst-ping 2026-07-18) every peer `tailscale status`
    reports online (self and `offline` peers skipped; a `` status means online-but-idle and is kept) is sent
    a *burst* of `tailscale ping --timeout=3s --c $SWEEP_PING_BURST --until-direct=false` (default 8; TSMP
    probes the WireGuard data path even on devices that drop ICMP, answered by the Tailscale client itself with
    no app/listener needed, so it works identically over DERP or direct P2P). A single ping only proves "
    reachable right now"; the burst additionally reports loss % and RTT min/avg/max/jitter, which is what
    actually predicts whether a call/video/QUIC session on that device holds up a peer can be 100% reachable
    and still have real quality problems a single ping would hide (confirmed live: an S24 over DERP showed 0%
    loss but 665ms of RTT jitter). 0% loss PASS; some loss (<40%) WARN; 40% loss or no pong FAIL. DERP-
    relayed peers are also flagged inline as throughput-degraded, pointing at check 7.
348 7. **Throughput** (added 2026-07-18) pulls a real file over HTTPS from a fast CDN (Hetzner; Cloudflare's own
    speed-test endpoint 403s WARP's shared/CGNAT egress IPs, so it's avoided) twice: once over the host's
    normal double-tunnel route, once through the `warp` container's own SOCKS5 proxy (port read from the live
    `/tmp/nullexit-ports.env`) to isolate WARP from the Tailscale-wrap hop without exec'ing into or installing
    anything in the container. Verdict is read from curl's actual exit code, not a byte-count guess: `56` (`
    Recv failure: Connection reset by peer`) is a real mid-transfer stall (WARN see devref.md 15.3.1); `28` is
    just the bounded test window ending on an otherwise-healthy transfer (PASS). A slow but supposedly "fast"
    CDN target was deliberately avoided after `speedtest.tele2.net` (the initial choice) turned out to be the
    bottleneck itself, not nullexit see devref.md 15.3.6.
349
350 **Output:** coloured stdout summary + a timestamped plain-text report `sweep-<TIMESTAMP>.txt` in the repo root
    (one file per run, so runs can be diffed); `--quiet` prints only the verdict. Signed in `.signatures` re-
    run `bash scripts/crypto.sh --sign` after editing it.
351
352 ### scripts/watcher.sh (320 lines, 15KB)
353
354 **Purpose:** Long-running launchd daemon (`com.nullexit.wake-recovery`) for post-wake/post-roam recovery. Runs
    independently of the gateway. Four backgrounded listeners: (1) WAKE via `log stream`, (2) NETWORK via `
    scutil n.watch`, (3) `TUNNEL_FAILED_CLOSED.marker` poll, (4) **Wi-Fi uplink keepalive** (added 2026-07-23).
355
356 **Key function `detect_lan_p2p_mode():`**
357 Added July 10, 2026. Called on startup and on every network state change (Listener 2 NET: events). Determines
    whether the current Wi-Fi network safely allows direct Tailscale LAN P2P connections:
358 1. **WPA2-Enterprise check:** `system_profiler SPAirPortDataType` reads the `Security:` field for the current
    association. Enterprise = 802.1x = always AP-isolated sets `allow=false`.
359 2. **AP isolation probe:** `route -n get 1.1.1.1` finds default gateway IP, then `ping -c 3 -t 1 -q "$gw"` if
    0 responses on a non-empty network, AP isolation is active sets `allow=false`.
360 3. **Output:** Writes `true` or `false` to `.lan_p2p_detected` in the repo root.
361 4. **Consumer:** `routing-fix.sh` reads this file every 30 seconds and enforces/relaxes the RFC1918 DROP rule
    accordingly no restart required.
362
363 **Listener 4 Wi-Fi uplink keepalive (added 2026-07-23, devref 15.5.9):**
364 Polls the Wi-Fi interface every 15s. When it has no routable IP (empty or `169.254.*`) `bash scripts/wifi-
    rejoin.sh rejoin`; when it has one `... reset`. This is the always-on driver for Wi-Fi auto-rejoin the
    WARP Watcher (`toggle.sh`) that used to be the only driver runs solely while the gateway is up, so a Wi-Fi
    drop with the gateway down/off left the Mac stranded. Leak-safe: only acts when there is no routable
    address (no egress path).
365
366 **Duplicated logic:**
367 - PATH export (similar to toggle.sh/recover.sh)
368 - Marker file pattern (`/tmp/nullexit-gateway-active.marker`)
369
370 ### scripts/wifi-rejoin.sh (130 lines) unsigned helper
371
372 **Purpose:** Re-associates ONLY user-remembered Wi-Fi networks (never arbitrary APs). Subcommands: `set`/`
    forget`/`list` manage `.last_wifi_ssid` (gitignored, one SSID per line); `rejoin`/`reset` are internal,
    called by `watcher.sh` Listener 4 (and the WARP Watcher while the gateway is up). `rejoin` enforces a 15s
    grace, progressive backoff (30cap 300s), and a give-up cap (`MAX_TRIES=6`) via `/tmp/nullexit-wifi-*`
    markers, and joins by name via `sudo -n networksetup -setairportnetwork` (Keychain password). No Location,
    no live-SSID read (devref 15.11.10). NOT in the crypto `.signatures` list.
373
374 ### scripts/dns-proxy.py (92 lines, 2.9KB) Clean, standalone
375
376 ### scripts/logger.py (144 lines, 5.9KB) Clean, standalone
377
378 ---
379
380 ## Part 4: Cross-Cutting Issues
381
382 ### Functions Identical Across macOS Linux
383
384 | Function | macOS toggle | macOS recover | macOS setup | Linux toggle | Linux recover | Linux setup |
385 |-----|-----|-----|-----|-----|-----|-----|
386 | `run_with_timeout()` | | | identical | identical | | |
387 | `stop_local_dns_proxy()` | | | identical | | | |
388 | `start_local_dns_proxy()` | | | ~95% similar | | | |
389 | `is_gateway_active()` | | | ~80% similar | | | |
390 | `step/ok/warn/die` | | | identical | identical | | |
391
392 ### Platform-Adapted Functions (same logic, different OS commands)
393

```

```

394 | Function | Linux Command | macOS Command |
395 |-----|-----|-----|
396 | `get_active_service()` | `ip route get 1.1.1.1 \| awk '{print $5}'` | `route get default` + `networksetup -
    listnetworkserviceorder` |
397 | `reset_dns()` | `resolvectl revert` | `networksetup -setdnsservers` |
398 | `cleanup_network_state()` | `ip link set dev down/up` | `networksetup -setairportpower off/on` + proxy
    disable |
399 | `force_dns_to_gateway()` | `resolvectl dns` + `resolvectl domain -ts.net` | `networksetup -setdnsservers` +
    `--setsearchdomains ts.net` |
400 | `start_sleep_prevention()` | `systemd-inhibit --what=sleep` | `caffeinate -i` |
401 | `enable/disable_socks_proxy()` | Stub (no-op on Linux) | `networksetup -setsocksfirewallproxy` |
402 | DNS cache flush | `resolvectl flush-caches` | `dscacheutil -flushcache` + `killall -HUP mDNSResponder` |
403
404 ### Features Missing From Linux Scripts
405
406 These exist in `toggle.sh`'s macOS branch but NOT its Linux branch:
407
408 1. `write_gateway_active_marker()` / `clear_gateway_active_marker()`
409 2. `restart_tailscaled_daemon()`
410 3. `start_warp_watcher()` / `stop_warp_watcher()` WARP liveness monitor
411 5. `add_warp_bypass_routes()` / `remove_warp_bypass_routes()` / `setup_exit_node_routing()`
412 6. `LOG_FILE` structured logging (timestamps to output.log)
413 7. MSS clamping injection (step 4c)
414 8. Rule count reporting
415 9. `--post-wake` mode in recover
416 10. Container teardown on failure in cleanup_handler
417
418 ---
419
420
421
422 ## Part 5: Constant Duplication Map
423
424 | Value | Files |
425 |-----|-----|
426 | `162.159.192.1` (WARP endpoint) | docker-compose.yml, routing-fix.sh (now dynamically loaded via .env/common.
    sh!) |
427 | `5335` (AdGuard DNS port) | docker-compose.yml, AdGuardHome.yaml, post-rules.txt |
428 | `5354` (mapped DNS port) | docker-compose.yml, dns-proxy.py |
429 | `41642` (Tailscale WireGuard) | docker-compose.yml, routing-fix.sh, post-rules.txt |
430 | `1080` (SOCKS5 port) | docker-compose.yml, toggle.sh |
431 | `1120` (MSS clamp) | post-rules.txt, .env |
432 | `admin:nullexit` (AdGuard creds) | AdGuardHome.yaml (bcrypt) |
433 | `10.200.1.0/24` (Docker subnet) | (Fixed) no longer duplicated |
434 | `America/New_York` (timezone) | docker-compose.yml (4 services) |
435 | `/tmp/nullexit-caffeinate.pid` | (Fixed) centralized in common.sh |
436 | `/tmp/nullexit-dns-watcher.pid` | (Fixed) centralized in common.sh |
437 | `/opt/homebrew/bin:...` (PATH) | (Fixed) centralized in common.sh |
438
439 ---
440
441 ## Part 6: Refactoring Outcome (Implemented July 2026)
442
443 **Decision: "Scripts-only" architecture**
444 Instead of introducing a new `lib/` directory, all shared code was moved into the existing `scripts/` directory
    to keep the project hierarchy flat and simple.
445
446 ### 1. `scripts/common.sh` (288 lines, 10KB)
447 Extracts the fundamental primitives and networking logic used across orchestrator scripts:
448 - **Formatters:** `step()`, `ok()`, `warn()`, `fail()`, `die()`
449 - **Timeout Wrapper:** `run_with_timeout()` (Canonical version with PID tracking, graceful SIGTERM, and SIGKILL
    fallback)
450 - **Daemon Lifecycle:** `restart_tailscaled_daemon()`, `stop_pidfile_daemon()` (collapsed caffeinate, DNS
    watcher, and WARP watcher teardown logic)
451 - **Network Interface/Router:** `get_active_service()`, `get_en0_service()`, `bounce_wifi_interfaces()`,
    `add_warp_bypass_routes()`, `remove_warp_bypass_routes()`
452 - **Proxy/DNS Cleanup:** `disable_all_proxies()`, `flush_dns_cache()`, `reset_sharing_services()`
453 - **Configuration Helpers:** `read_adguard_ip()`, `is_kill_switch_enabled()`, `get_warp_endpoint_1()`,
    `get_warp_endpoint_2()` (which centralizes the hardcoded 162.159.192.1 / .193.1 into .env logic)
454
455 ### 2. `scripts/setup-common.sh` (~350 lines)
456 Extracts the heavy, duplicated installation logic shared between `setup.sh` (macOS) and `scripts/setup-linux.sh`
    :
457 - Docker and Colima prerequisite checks
458 - `wgcf` binary download and WARP key generation
459 - `.env` configuration file generation
460 - Interactive Tailscale Auth Key and AdGuard Rule Profile prompts
461
462 ### Sourcing Pattern
463 - macOS root scripts (`toggle.sh`, `recover.sh`, `setup.sh`) source via:
464 `source "$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/scripts/common.sh`

```

```

465 - Scripts under `scripts/` (`watcher.sh`, `diagnose-host-leak.sh`, etc.) source via:
466 `source "${dirname "${BASH_SOURCE[0]}"}"/common.sh`
467
468 ## Agent Verbs / Protocols
469
470 ### `/sweep` (or `sweep the gateway`)
471 When the user invokes this verb, the agent MUST perform a full end-to-end connectivity, health, security, and
472 performance audit of the gateway:
473 1. **WARP Validation:** Run `curl -s https://www.cloudflare.com/cdn-cgi/trace | grep warp=` (must equal `on`).
474 2. **Tor Proxy & Control Validation:**
475 - Source the dynamically generated ports from `/tmp/nullexit-ports.env` (or identify them using `docker
476 compose ps`).
477 - **SOCKS5 Check:** Run `curl -s --socks5-hostname 127.0.0.1:$TOR SOCKS_PORT https://check.torproject.org/
478 api/ip` to confirm the SOCKS5 proxy successfully connects and routes through the Tor network.
479 - **Control Port Check:** Query the Tor Control Port using netcat: `echo "PROTOCOLINFO" | nc 127.0.0.1:
480 $TOR_CONTROL_PORT`. Verify it returns a standard `250-PROTOCOLINFO` response, confirming the Control Port
481 is active and communicating.
482 - **Circuit & Node Lookup:** Query the Tor Control Port for the active path (`GETINFO circuit-status`). For
483 each active built circuit, parse the relay fingerprints, lookup their IP addresses (`GETINFO ns/id/<
484 fingerprint>`), and resolve their country codes using Tor's local GeoIP database (`GETINFO ip-to-country/<
485 IP>`). Include the list of active circuits, showing the role (Guard/Middle/Exit), nickname, IP, and country
486 of each hop in the final sweep report.
487 - **Transparent Onion Validation:** Run `nslookup duckduckgogg42xjoc72x3sjasowoarfbgcmvfimaftt6twagswzczad.
488 onion` to verify it resolves to an IP within `198.18.0.0/15` benchmark range. Verify transparent dark web
489 routing by running `curl -sI -H "Host: duckduckgogg42xjoc72x3sjasowoarfbgcmvfimaftt6twagswzczad.onion" http
490 ://<resolved-ip>/` (should return HTTP 301/200).
491 3. **DNS Leak Validation:**
492 - Audit the upstream resolver currently utilized by the host. Run `curl -s https://edns.ip-api.com/json` to
493 fetch details of the DNS resolver processing the client's requests.
494 - Verify that the resolver IP and organization returned belong to Cloudflare (e.g., AS13335) or match the
495 WARP tunnel's exit range, and that **no** DNS traffic leaks to the user's raw physical ISP DNS.
496 4. **Performance Benchmark Audit:**
497 - Run the python benchmark script `python3 benchmarks/test_load.py`.
498 - Report the overall load speed and the ratio of successfully fetched vs. blocked/failed subresources for
499 the targeted domains to ensure ad-blocking and MTU encapsulation are performing correctly.
500 5. **Tailscale Mesh Validation:**
501 - Run `ping -c 1 100.100.21.8` to ensure the gateway is reachable.
502 - Run `tailscale status` to identify all active/online nodes in the mesh.
503 - For every online peer returned in the status output, execute `tailscale ping --timeout=3s --c 8 --until-
504 direct=false <peer-ip>` (a burst, not a single ping TSMP works even on phones that drop ICMP) to verify
505 connectivity and read loss % + RTT spread across the burst, not just one-shot latency. `bash scripts/sweep.
506 sh` automates this as its check 6/7 and reports each peer's path (direct endpoint vs `DERP(...)`), loss %,
507 and RTT min/avg/max/jitter; its check 7/7 additionally measures real throughput (host path vs. a WARP-only
508 baseline).
509 6. **Log Audit:** Run `tail -n 100 output.log | grep -iE "(error|fail|warn)"` to catch any underlying component
510 failures or warnings.
511
512 ### `/latex` (or `generate latex`)
513 When the user invokes this verb, the agent MUST run `python3 -I scripts/generate_tex.py` to regenerate the
514 documentation source code (`nullexit_unified.tex`). The agent MUST NOT attempt to compile the `.tex` file
515 into a PDF (the user handles PDF compilation locally).

```

4 devref.md

```

1 # nullexit Development Reference & Resolved Issues
2
3 > **Last updated:** July 19, 2026
4 > **Purpose:** Provide any LLM or developer with complete project understanding, debugging history, and
5 resolved issues so they can make informed changes without re-reading every file.
6 > **Diagrams:** See [diagrams.md](./diagrams.md) for system architecture, toggle.sh flowcharts, monitoring
7 layer, traffic sequence, recover.sh decision tree, and the full failureself-healing map.
8
9 This reference is organized into four parts:
10 > - **Part I Architecture & Reference** (18)
11 > - **Part II Design Decisions & Threat Model** (914)
12 > - **Part III Incident Log & Resolved Issues** (15)
13 > - **Part IV Observations, Changelog & TODO** (1618)
14
15 ---
16
17 # Part I Architecture & Reference
18
19 ## 1. What Is nullexit?
20
21 nullexit is a **Tunnel-in-Tunnel VPN gateway** that chains **Tailscale** (mesh VPN) through **Cloudflare WARP**
22 (exit VPN). It runs on a macOS host inside Docker containers managed by **Colima** (lightweight Linux VM).

```



```

    The goal: any device on the user's Tailscale mesh can use this gateway as an exit node, and all traffic
    exits through Cloudflare WARP achieving double encryption, ISP invisibility, and network-wide ad-blocking
    via **AdGuard Home**.
20
21 ### Traffic Flow
22 ---
23 Phone/Laptop  Tailscale mesh (WireGuard)  Mac host  Docker container
24 Tailscale decrypts  Gluetun re-encrypts  WARP tun0  Cloudflare  Internet
25 ---
26
27 ### SOCKS5 Failover Path (if exit node is unavailable)
28 ---
29 DNS:  Browser  host 127.0.0.1:53  Python proxy  TCP:5354  AdGuard  WARP DNS
30 TCP:  Apps  macOS SOCKS5 proxy (127.0.0.1:1080)  container  tun0  WARP  Internet
31 ---
32
33 ---
34
35 ## 2. Architecture
36
37 ### Containers (all share `warp`'s network namespace via `network_mode: service:warp`)
38 | Container | Image | Role |
39 |-----|-----|-----|
40 | `warp` | `qmcgaw/gluetun:v3.41.1` | Gluetun WireGuard client  Cloudflare WARP. Owns the network namespace.
    Strict firewall. |
41 | `tailscale` | `tailscale/tailscale:v1.98.4` | Advertises as exit node on the mesh. Also provides the built-in
    SOCKS5 proxy fallback via `TS_SOCKS5_SERVER=1080`. |
42 | `routing-fix` | `alpine:3.20` | Sidecar that maintains routing tables + iptables rules every 30 seconds. |
43 | `rule-compiler` | `golang:1.22-alpine` / `alpine:3.20` | One-shot startup container that compiles 500k+ DNS
    block rules in <2s and immediately exits. |
44 | `adguardhome` | `adguard/adguardhome:v0.107.77` | DNS sinkhole for ads/trackers. Listens on port 5335.
    Upstream DNS: `127.0.0.1:53` (through WARP). |
45
46 ### Port Mappings (on host via Colima)
47 | Host Port | Container Port | Protocol | Purpose |
48 |-----|-----|-----|-----|
49 | 5354 | 5335 | TCP+UDP | AdGuard DNS |
50 | (Tailscale IP):3000 | 3000 | TCP | AdGuard web UI (Internal to Mesh) |
51 | 41641 | 41641 | UDP | Tailscale WireGuard direct |
52 | 1080 | 1080 | TCP | SOCKS5 proxy |
53
54 ### Host Components
55 - **Colima VM** Linux VM running Docker (600MB RAM + 400MB swap)
56 - **tailscaled** Standalone Tailscale daemon via `brew install tailscale` + `brew services start tailscale` (
    NOT the GUI `.app`)
57 - **macOS network settings** DNS, SOCKS proxy, routing all manipulated by `toggle.sh`
58
59 ---
60
61 ## 3. Key Files
62
63 | File | Lines | Purpose |
64 |-----|-----|-----|
65 | `toggle.sh` | ~2109 | **Main script (macOS + Linux).** Dispatches on `$OSTYPE` an early Linux block (shuf/ss
    /nmcli/ip, systemd) runs and exits before the macOS body (jot/netstat/networksetup, launchd/pf). Detects
    state, toggles gateway ON/OFF. Handles DNS hijacking, Tailscale exit node, SOCKS proxy, sleep prevention,
    timeouts, cleanup. |
66 | `setup.sh` | 406 | **One-time setup.** Installs deps (Docker, Tailscale, wgcf), generates WARP keys, writes
    `.env`, configures AdGuard via API, starts containers. |
67 | `docker-compose.yml` | 194 | Service definitions for all 5 containers. |
68 | `scripts/routing-fix.sh` | 201 | Maintains routing tables (table 200 for SOCKS5, table 52 for CGNAT) and
    FORWARD iptables rules in both nftables and legacy backends. Loads and enforces the IP blocklist via `ipset`
    . Runs in a 30-second loop. |
69 | `post-rules.txt` | 18 | Gluetun iptables rules loaded at container start. FORWARD accept for tailscale0, NAT
    MASQUERADE on tun0, DNS redirect to port 5335, IPv6 drop, TCP MSS clamping. |
70
71 | `scripts/dns-proxy.py` | 91 | Local DNS proxy. UDP:53 TCP:5354 with proper 2-byte length prefix. Fallback
    when exit node is unavailable. |
72 | `scripts/rule-compiler/` | ~800 | Unified threat compiler (ported to Go from Python). **DNS pipeline:**
    fetches remote blocklists, deduplicates against AdGuard native filters, optimizes subdomains (~50%
    reduction), compiles to AdGuard syntax. **IP pipeline:** fetches threat-intel feeds (Spamhaus, Feodo, ET,
    CINS), strips private/Tailscale ranges, outputs atomic `ipset` restore file. Built dynamically in Docker
    multi-stage build. |
73 | **`adguard/work/`** | | **Bind-mounted container data folder. Off-limits from outside Docker READ-ONLY-
    EXCEPT-VIA-`sync-rules`. See README 6.** |
74 | `recover.sh` | ~742 | Nuclear recovery script (**macOS + Linux** dispatches on `$OSTYPE`; the Linux branch
    runs a self-contained teardown via `resolvectl`/`nmcli`/`ip` and exits, macOS falls through to the dual-
    mode body). Gain: use `--post-wake` (macOS) for non-destructive refresh after sleep/wake or Wi-Fi roam
    keeps the gateway live while still re-hijacking DNS + refreshing Tailscale exit-node + force-recreating
    warp if unhealthy. Resets DNS on ALL services, disables proxies, flushes routes, stops containers, kills
    sleep prevention, power-cycles Wi-Fi. Cryptographically verified. |

```

```

75 | `scripts/diagnose-host-leak.sh` | 580+ | **One-shot host-routing diagnostic.** Runs 8 checks (Tailscale,
    | SOCKS5, CDN-cgi/trace, IPv6 leak, default route, output.log hints, gateway IP, Tailscale System Extension
    | conflict), classifies into ONE of three known leak scenarios (A=SOCKS5 fallback, B=IPv6 leak, C=route
    | freeze) or OK, writes a timestamped report to `host-leak-diagnostic-<UTC>.txt`, and prints a ready-to-run
    | fix on stdout. Pass `--fix` to apply the matched remediation and re-verify egress. Pass `--watch` (or `--
    | watch 30`) to run a full baseline then continuously monitor warp/IPv6/default-route every N seconds,
    | alerting on any state change. See 15.6.1. |
76 | `scripts/host-leak-probe.sh` | | **REMOVED (2026-07-15, verified deleted/untracked).** Was a continuous sub-
    | second host-egress probe auto-launched via `HOST_LEAK_PROBE`. Its fail-closed role is now covered by the
    | **PF kill-switch** (`enable_killswitch` in `common.sh`) + the in-container **WARP Watcher** (15.6.2); on-
    | demand auditing is `scripts/diagnose-host-leak.sh`. See 15.6.1 (retained as historical rationale). |
77 | `scripts/fix-docker-bridge-collision.sh` | ~290 | **One-shot fix for 15.2.3 (Docker bridge subnet collision)
    | ** Detects Docker's `172.17.0.0/16` bridge colliding with the host Wi-Fi subnet (the symptom that produces
    | `default 172.17.0.1 UGScg en0` in `netstat -rn`), picks a non-colliding `docker.bip` from a small
    | candidate set, idempotently edits `~/colima/default/colima.yaml` with a timestamped backup, restarts
    | Colima, rebinds Wi-Fi DHCP, verifies the new default route is no longer Docker's bridge, and re-runs `
    | toggle.sh` + `scripts/diagnose-host-leak.sh`. Supports `--dry` and `--skip-toggle`. |
78 | `scripts/watcher.sh` | 320 | **Long-running post-wake + post-roam daemon.** Launched by launchd LaunchAgent;
    | subscribes to `com.apple.powermanagement` events via `log stream` and to network-state changes via `scutil
    | n.watch`. Both fire `recover.sh --post-wake`. A 4th listener (added 2026-07-23) polls the Wi-Fi interface
    | and drives `scripts/wifi-rejoin.sh` when the uplink has no routable IP independent of gateway state
    | (15.5.9). Single-instance lock + SCRIPT_DIR-relative resolve of `RECOVER`. See 15.5.1. |
79 | `scripts/wifi-rejoin.sh` | 130 | **Wi-Fi keepalive helper (unsigned).** Re-associates ONLY networks the user
    | remembered in `last_wifi_ssid` (`set`/`forget`/`list`), never arbitrary APs. `rejoin` enforces a grace
    | period, progressive backoff, and a give-up cap; `reset` clears the down-timer once the link returns. No
    | Location, no root plist read, no live-SSID read (15.11.10). Driven by `watcher.sh` (always-on) and, when
    | the gateway is up, the WARP Watcher; the two share `/tmp` markers. See 15.5.9. |
80 | `scripts/common.sh` | ~40 | **Shared script primitives.** Centralizes formatted echo statements (`step`, `ok
    | `, `warn`, `die`) and the safe background `run_with_timeout` execution wrapper. Sourced by all orchestrator
    | scripts across macOS and Linux. |
81 | `scripts/setup-common.sh` | ~350 | **Shared installation logic.** Centralizes logic for verifying Docker,
    | installing Tailscale/wgcf, generating `.env`, and prompting for auth keys. Sourced by `setup.sh` and `setup
    | -linux.sh`. |
82 | `scripts/setup-linux.sh` | ~80 | **Linux sibling of `setup.sh`. Sources `setup-common.sh`. Distro detection
    | (apt/dnf/pacman), installs Linux-specific dependencies, creates `.desktop` shortcuts. |
83 | `scripts/Toggle-Gateway.bat` | ~300 | **Windows Toggle helper.** Moved from root to `scripts/`. Helps invoke
    | the framework on Windows if applicable. |
84 | `scripts/reinstall-host-tailscale.sh` | ~740 | **Nuke-and-reinstall tool for host Tailscale.** Completely
    | purges Tailscale, its settings, macOS Background Items (`.btm` database), and stale System Extensions.
    | Wipes ALL Login Items via `sftool reset-login-items` (requires sudo). Useful for crisis recovery but
    | destructive to other login items. |
85 | `scripts/fix-ssh-delay.sh` | ~100 | **Fixes SSH connection delays.** One-shot script to address SSH delays
    | caused by DNS or routing issues, useful in crisis situations. |
86 | `scripts/logger/` | ~100 | **Internal logger piped from `scripts/routing-fix.sh`** inside the `warp`
    | container. Ported to Go from Python. Built via Docker multi-stage build. |
87 | `black_list.txt` | 86 | Custom domains to block (ads, trackers, telemetry, cross-platform device-
    | identification endpoints). Supports `$important` modifier. |
88 | `white_list.txt` | 231 | Domains to force-allow (YouTube, Apple services, etc.). Always wins over blocks. |
89 | `.env` | ~14 | WARP WireGuard keys, Tailscale auth key, rule profile. **Contains secrets & NULLEXIT_SEED.** |
90 | `.gateway_ip` | 1 | Static gateway Tailscale IP (fallback for dynamic resolution). |
91 | `scripts/unlock-files.sh` | ~20 | **One-shot stale permission fix.** Uses atomic rename (`cp` + `mv`) to
    | replace locked inodes (mode `000`/`0444` from old `chmod` decisions) with fresh writable ones no `chmod`
    | called. |
92 | `scripts/crypto.sh` | ~50 | **Cryptographic integrity enforcement.** Uses `NULLEXIT_SEED` from `.env` to sign
    | core bash scripts (HMAC-SHA256) and strictly verify them on start to prevent manipulation. Pass `--sign`
    | or `--verify`. |
93 | `scripts/pf.conf` | 35 | **macOS native firewall ruleset.** Enforces the default-deny kill-switch, allows
    | local LAN / Tailscale traffic, and applies TCP MSS Clamping (`max-mss 1160`) to prevent WireGuard MTU
    | fragmentation. |
94
95 ---
96
97 ## 4. toggle.sh Flow
98
99 ### State Detection
100 `is_gateway_active()` returns true if EITHER containers are running OR host DNS is not `1.1.1.1`. This dual
    | check prevents the script from starting when it should stop (e.g., containers crashed but DNS is still
    | hijacked).
101
102 ### START Path (gateway OFF ON)
103 1. **Reset DNS to 1.1.1.1** Prevents deadlocks during startup
104 2. **Disconnect host Tailscale** `tailscale down` (prevents exit-node routing during container boot)
105 3. **Compile DNS rules** `docker compose run --rm rule-compiler` (Go binary inside multi-stage Docker build)
106 4. **Boot Colima VM** (if not running) via `colima_start_until_ready()`, which returns as soon as Docker is
    | reachable (~12s) instead of blocking on Colima's ~2-min post-provision hang (15.8.7). The networking mode
    | is **gated on the LAN-P2P signal** (`.lan_p2p_detected`, falling back to `TAILSCALE_ALLOW_LAN_P2P` in `.env
    | `): a trusted home network requests `--network-address --network-mode bridged` (or `shared` on WPA2-
    | Enterprise), while the common AP-isolated / untrusted case boots plain fast user-mode networking (`colima
    | start --memory 0.6 --vm-type vz`).
107 - **Why Bridged Networking (when enabled)?** The `--network-mode bridged` and `--network-address` flags
    | force the VM to receive its own IP directly from your local router's DHCP server, placing it on the same

```



```

physical subnet as the host. This bypasses macOS network isolation and is what lets **external LAN devices
** (phones, laptops on the same LAN), mDNS, and local service discovery reach across the container boundary
. It requires the `socket_vmnet` helper and is only worthwhile on a trusted LAN, which is why it is gated
rather than always requested. **It is *not* required for direct hostcontainer P2P** that path (the gateway
its own Mac) works even in user-mode networking via the routing-fix `.host_ips` RETURN-rule carve-out;
see 15.3.5 and 15.8.7.
108 - **Enterprise Wi-Fi Fallback & Dynamic Roaming:** `toggle.sh` dynamically queries `system_profiler
SPAirPortDataType` on boot. If it detects a **WPA2-Enterprise** connection (802.1X), it forces Colima to
fall back to `--network-mode shared`. This prevents aggressive network intrusion systems on corporate/
university Wi-Fi switches from terminating the host's Wi-Fi connection due to "MAC spoofing" (detecting
multiple MAC addresses originating from a single authenticated port). P2P connections are impossible on
these networks anyway due to AP Client Isolation.
109 - **Dynamic Roam Downgrading:** `toggle.sh` writes its selected network mode to `/tmp/nullexit_colima_mode
.txt`. When roaming between networks (e.g., Home to University), `recover.sh` actively checks this state
against the new network's security requirement (`.lan_p2p_detected`). If a mismatch is detected (e.g.,
bridged Colima on an Enterprise network), `recover.sh` automatically forces a full `toggle.sh --restart` in
the background to safely transition the Virtual Machine's network mode and protect the host from de-
authentication.
110 5. **Configure VM swap** 400MB swap file inside the VM to prevent OOM
111 6. **Clean corrupted AdGuard config** Remove empty `AdGuardHome.yaml`
112 7. **Start containers** `docker compose up -d`
113 8. **Wait for gateway Tailscale** Poll `tailscale status` for "offers exit node" (up to 60s, abort at 40
consecutive NoState)
114 9. **Resolve gateway IP** From `.gateway_ip` or `docker compose exec tailscale tailscale ip -4`
115 10. **Connect host to mesh** Verify `tailscaled` is running (auto-start if needed), then `tailscale up --reset
--ssh=true --accept-dns=false --accept-routes=true --exit-node=` ( `--accept-routes=true` must be explicit
`--reset` reverts it to `false` otherwise, silently preventing the default route from switching to `utun
*`)
116 11. **Pre-flight checks** [1/3] `tailscale ping` gateway, [2/3] `dig +tcp` AdGuard DNS, [3/3] WARP container
internet
117 12. **If all pass** Set exit node + hijack DNS to gateway IP (single server, no fallback)
118 13. **If any fail** Enable SOCKS5 proxy + local DNS proxy as fallback
119 14. **Final DNS re-force** `force_dns_to_gateway` called again after exit-node transition (tailscaled clobbers
DNS briefly)
120 15. **Enable sleep prevention** `caffeinate -i` in background to prevent idle sleep
121
122 ### STOP Path (gateway ON OFF)
123 1. Disconnect host Tailscale (`tailscale down`)
124 2. `docker compose down -t 5`
125 3. Reset DNS to 1.1.1.1
126 4. Stop local DNS proxy
127 5. **Stop sleep prevention** Kill caffeinate process
128 6. Full network state cleanup (proxies off, DNS cache flush, route flush, Wi-Fi power cycle)
129
130 ### Safety Mechanisms
131 - `run_with_timeout()` Pure-bash watchdog for every system command (15s120s)
132 - `cleanup_handler()` Trap on ERR/INT/TERM/HUP restores DNS to 1.1.1.1, kills caffeinate, and tears down
Tailscale
133 - `force_dns_to_gateway()` Sets DNS and VERIFIES via `networksetup -getdnsservers` (3 attempts with backoff)
134 - `SUCCESS_RUN` flag Cleanup only runs if script didn't complete successfully
135 - `start_sleep_prevention()` / `stop_sleep_prevention()` Manages `caffeinate -i` via PID file at `/tmp/
nullexit-caffeinate.pid`
136
137 ---
138
139 ## 5. Routing & Firewall Architecture (Inside Container)
140
141 ### IP Rules (`ip rule show`)
142 | Priority | Match | Table | Purpose |
143 |-----|-----|-----|-----|
144 | 99 | `to 100.64.0.0/10` | 52 | CGNAT range Tailscale routes (injected by routing-fix) |
145 | 100 | `from 172.18.0.2` | 200 | Container's own traffic (SOCKS5 proxy) tun0 |
146 | 101 | all | 51820 | Gluetun's WireGuard fwmark tun0 |
147
148 ### Table 200 (SOCKS5 proxy traffic)
149 - `162.159.192.1` via $DOCKER_GW dev eth0` WARP endpoint exception (prevents tunnel loop)
150 - `default dev tun0` Everything else through WARP
151
152 ### iptables (TWO separate stacks!)
153 - **nftables backend** (`iptables`) Used by Gluetun's `post-rules.txt`. FORWARD policy DROP.
154 - **legacy backend** (`iptables-legacy`) Used by Tailscale's `ts-forward`. FORWARD policy ACCEPT.
155 - Both stacks apply to every packet. FORWARD RELATED,ESTABLISHED must exist in BOTH.
156
157 ### post-rules.txt (loaded by Gluetun)
158 ...
159 FORWARD: ACCEPT for tailscale0 in/out
160 INPUT: ACCEPT for tailscale0
161 NAT POSTROUTING: MASQUERADE on tun0
162 MANGLE: TCP MSS clamp to 1180
163 NAT PREROUTING: Redirect DNS (53) 5335 on tailscale0 and eth0
164 ip6tables: DROP FORWARD on tailscale0

```

```

165 ---
166 ---
167 ---
168
169 ### 5.1 Tailscale Local Network Discovery (DERP Bypass)
170 By default, Gluetun's strict NAT swallows all of Tailscale's UDP hole-punching packets, forcing Tailscale to
    fall back to incredibly slow DERP relay servers (often adding 500ms+ latency).
171
172 To fix this, we use `FIREWALL_OUTBOUND_SUBNETS=192.168.0.0/16,172.16.0.0/12,10.0.0.0/8` in `docker-compose.yml`
    (on the `warp` container). This creates `ip rule` bypasses (Priority 99, Table 199) that route all packets
    destined for private IP ranges *outside* the WARP tunnel directly to the host's `eth0`.
173 - **172.16.0.0/12** is especially critical because many modern Wi-Fi routers (and Docker itself) assign IPs in
    this block (e.g., `172.17.x.x`).
174 - This bypass allows Tailscale's UDP packets to reach devices on the same Wi-Fi directly, establishing a fast
    peer-to-peer connection while the actual web traffic inside the tunnel remains fully routed through WARP.
175
176 #### 5.1.1 Why This Matters: Untrusted, Non-Enterprise Networks
177
178 Most public/consumer hotspots hotels, cafes, airports, a friend's home router are **untrusted** (any other
    guest is a stranger, no admin-enforced isolation) but **not** enterprise-hardened the way WPA2-Enterprise
    /802.1x campus networks are (see 15.7.1's SNAT poisoning analysis for why *those specifically* get P2P
    disabled). Because open networks usually lack AP client isolation, `watcher.sh`'s `detect_lan_p2p_mode()`
    probe (a ping to the default gateway see 16.4) typically finds them P2P-capable and flips `
    TAILSCALE_ALLOW_LAN_P2P` on automatically.
179
180 This is the scenario where nullexit's mesh-first design pays off over a conventional single-device VPN app:
181
182 1. **Encryption doesn't depend on P2P succeeding.** Every packet between mesh devices is WireGuard-encrypted
    whether it goes direct or via DERP. An open network's *lack* of isolation is itself the risk anyone else
    on that hotel Wi-Fi can reach your device at L2 but it doesn't expose your traffic's content, only its
    existence/metadata to that local segment.
183 2. **Zero configuration per network.** The device only needs to be Tailscale-enrolled once, ever. `watcher.sh`
    re-runs its isolation probe on every Wi-Fi change (`scutil n.watch` on `State:/Network/Global/IPv4{4,6}`,
    15.5.1) and rewrites `.lan_p2p_detected` accordingly, which `routing-fix.sh` picks up within 30s no app to
    reopen, no profile to import, no manual toggle per hotspot. The same phone gets DERP-only (safe, no SNAT
    risk) on an isolated campus network and fast direct P2P (safe, isolation genuinely absent) at a hotel that
    night, entirely automatically.
184 3. **The auto-detection is what makes this safe to leave alone.** A user who manually forced `
    TAILSCALE_ALLOW_LAN_P2P=true` globally and never touched it again would carry the P2P attempt onto the *
    next* enterprise network they visit, reproducing the 15.7.1 SNAT-poisoning blackout. The watcher's per-
    network re-probe is what lets the setting behave correctly on untrusted-but-open networks without that risk
    following the user everywhere.
185
186 ---
187
188 ### 5.2 Firewalling & Per-Device Access Control
189 Because every mesh device's traffic passes through the `warp` container's network namespace, this namespace
    serves as the ultimate choke point for network-wide firewall rules.
190
191 **Where to Put Rules**
192 The right place to add firewall rules is `scripts/routing-fix.sh`. It runs a 30-second loop inside the
    namespace and already handles re-injecting rules that Gluetun resets on reconnect. **Do not use `post-rules
    .txt` for dynamic rules**, as Gluetun flushes and resets it upon VPN reconnects.
193
194 **The Dual iptables Backend Constraint (CRITICAL)**
195 The container runs two simultaneous iptables backends: `iptables` (for the nftables backend used by Gluetun)
    and `iptables-legacy` (for Tailscale). Every rule **must** be added to both stacks, or it will silently
    fail for certain traffic.
196
197 **Avoiding Duplication**
198 Because `scripts/routing-fix.sh` runs in a loop, you must always check for a rule's existence with `-C` before
    appending with `-A`. Otherwise, your rules will duplicate every 30 seconds.
199
200 **Per-Device Identification & Filtering**
201 Each mesh device has a stable `100.x.x.x` Tailscale IP, which is visible as the `src` IP in the `FORWARD` chain
    . This is how you identify devices for filtering.
202 * **Domain-level:** AdGuard Home (port 3000, credentials `admin/nullexit`) provides a REST API that supports
    per-client blocking rules based on their Tailscale IP.
203 * **IP-level:** Use `iptables` in `scripts/routing-fix.sh` (e.g., `iptables -I FORWARD -s 100.x.x.x -d <
    blocked_ip> -j DROP`).
204 * **Country-level:** Add `ipset` to the `routing-fix` apk install step in `docker-compose.yml`, and load CIDR
    ranges from `ipdeny.com` into an ipset, then block that set in the `FORWARD` chain. See 5.3 (Geo-IP
    Blocking) for the full configuration flow.
205
206 ### 5.3 Geo-IP Blocking
207 To block traffic to and from specific countries, the system dynamically downloads country-specific IP ranges
    from [ipdeny.com](http://www.ipdeny.com/ipblocks/data/countries/) and drops them via iptables `FORWARD`
    rules.
208
209 To add a new country to the blocklist:

```

```

210 1. Find the 2-letter ISO country code (e.g., `cn` for China, `ru` for Russia, `il` for Israel, `kp` for North
211    Korea).
212 2. Open your `.env` file.
213 3. Update the `BLOCKED_COUNTRIES` variable: e.g., `BLOCKED_COUNTRIES="kp il cn ru"`.
214 4. Run `toggle.sh` to restart the gateway and apply the new environment variables to the routing-fix container.
215
216 > [!NOTE]
217 > **Limitations of Geo-IP Blocking:** IP-based blocking is highly effective at blocking servers, direct apps,
218    and raw infrastructure physically located and registered in a specific country (e.g., `www.gov.il`).
219    However, it will not block high-profile websites (e.g., `jpost.com`) that route their traffic through
220    global Content Delivery Networks (CDNs) like Google Cloud, Cloudflare, or Fastly. Because the CDN's IP
221    addresses are registered in the US or globally, the network traffic physically never routes to the blocked
222    country, allowing it to bypass the Geo-IP firewall.
223
224 ## 6. macOS Quirks to Remember
225
226 1. Colima SSH tunnel is TCP-only UDP ports (like 5354/udp) are NOT accessible from host. Use `dig +tcp` or
227    the Python DNS proxy (UDPTCP converter).
228 2. docker compose exec -T injects `r` Always `tr -d '\r'` when capturing output for comparisons or `
229    networksetup` commands.
230 3. brew services` can get stuck Fix with `sudo brew services restart tailscale`. Common state: `brew
231    services list` shows `started` but `tailscale status` shows "Tailscale is stopped."
232 4. No `timeout` command macOS lacks GNU `timeout`. Use the `run_with_timeout()` bash function.
233 5. sudo -v` prompts even with NOPASSWD Use `sudo -n` (non-interactive) everywhere.
234 6. DNS is scoped to en0 macOS scutil DNS resolver is scoped to en0 (Wi-Fi). DNS changes on other
235    interfaces (like USB Ethernet) are ignored unless en0's service is also updated. The script always sets DNS
236    on BOTH `ACTIVE_SERVICE` and `ENO_SERVICE`.
237 7. tailscaled clobbers DNS during exit-node transition `force_dns_to_gateway` is called a second time at
238    the end of the ENABLE branch to counteract this.
239 8. tailscale up --reset resets all unspecified flags to defaults Every `--reset` call MUST also pass `--
240    accept-dns=false` explicitly or tailscaled re-enables DNS management.
241 9. docker compose ps` CWD pitfall `docker compose` commands require a `docker-compose.yml` in the current
242    working directory. Use `docker ps` for raw container listing from any CWD; `docker compose ps` requires
243    the project directory.
244
245 ---
246
247 ## 7. Environment Variables
248
249 ### `.env` File
250 | Variable | Purpose |
251 |-----|-----|
252 | `WIREGUARD_PRIVATE_KEY` | WARP WireGuard private key (from `wgcf generate`) |
253 | `WIREGUARD_PUBLIC_KEY` | WARP WireGuard public key |
254 | `WIREGUARD_ADDRESSES` | WARP WireGuard address (e.g., `172.16.0.2/32`) |
255 | `TS_AUTHKEY` | Tailscale auth key for the container |
256 | `NULLEXIT_SEED` | 256-bit seed for HMAC-SHA256 script integrity signing (generated by `setup.sh`) |
257 | `GATEWAY_RULE_PROFILE` | Rule compilation tier: `light`, `medium`, `heavy` |
258 | `GATEWAY_BYPASS_PING` | (Optional) `true` to proceed even if pre-flight checks fail |
259 | `GATEWAY_USE_EXIT_NODE` | (Optional) `false` to skip exit node, DNS-only mode |
260 | `GATEWAY_HIJACK_HOST` | (Optional) `false` to skip DNS hijacking on the host (VPN/adblocking for Tailscale
261    peers only) |
262 | `GATEWAY_MSS` | (Optional) TCP MSS clamp value. Default and only validated-safe value is `1120` 15.3.2 found
263    1180 (and 1160) still too large for the double-encapsulation overhead and causes mid-transfer stalls.
264    Applied to both the container (`post-rules.txt`) and the host (`pf.conf`), kept in sync. |
265 | `WARP_FAIL_THRESHOLD` | (Optional) Consecutive `warp-off` polls before auto-shutdown (default 6 = 30s; 3 = 15
266    s) |
267 | `WARP_ENDPOINT_1` | (Optional) Override Cloudflare WARP WireGuard endpoint IP 1 (default `162.159.192.1`) |
268 | `WARP_ENDPOINT_2` | (Optional) Override Cloudflare WARP WireGuard endpoint IP 2 (default `162.159.193.1`) |
269 | `KILL_SWITCH` | (Optional) `true` to enforce strict PF lock drops all host traffic if VPN fails. Breaks SSH
270    if VPN dies. |
271 | `STOP_COLIMA_ON_EXIT` | (Optional) `true` to fully shut down the Colima VM on toggle-off (saves battery on
272    dedicated hosts) |
273 | `ADGUARD_USER` | (Optional) AdGuard Home web UI username (default: `admin`) |
274 | `ADGUARD_PASSWORD` | (Optional) Shared password for AdGuard Home and Tor ControlPort (default: `nullexit`) |
275 | `BLOCKED_COUNTRIES` | Space-separated 2-letter ISO codes to block via ipdeny.com CIDR ranges (e.g. `kp il cn
276    ru`) |
277
278 ---
279
280 ## 8. Diagnostic Commands
281
282 ### Container Health
283 ```bash
284 docker ps --format '{{.Names}} {{.Status}}'
285 docker compose exec -T tailscale tailscale status
286 docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace
287 ```
288
289 ### SOCKS5 Proxy
290 ```bash

```

```

269 curl --socks5-hostname 127.0.0.1:1080 --max-time 5 -s https://www.cloudflare.com/cdn-cgi/trace
270 ```
271
272 ### AdGuard DNS
273 ```bash
274 dig +tcp @127.0.0.1 -p 5354 google.com +short +timeout=5
275 ```
276
277 ### Firewall & Routing
278 ```bash
279 # nftables FORWARD chain
280 docker compose exec -T warp iptables -L FORWARD -v --line-numbers
281 # Legacy FORWARD chain
282 docker compose exec -T warp iptables-legacy -L FORWARD -v --line-numbers
283 # IP rules + routing tables
284 docker compose exec -T warp ip rule show
285 docker compose exec -T warp ip route show table 199
286 docker compose exec -T warp ip route show table 200
287 ```
288
289 ### Host macOS
290 ```bash
291 tailscale status
292 brew services list | grep tailscale
293 networksetup -getdnsservers Wi-Fi
294 networksetup -getsocksfirewallproxy Wi-Fi
295 sysctl net.inet.ip.forwarding
296 ```
297
298 ---
299
300 # Part II Design Decisions & Threat Model
301
302 ## 9. Privacy Architecture & Threat Model
303
304 ### Layered Defense
305 The gateway stacks four layers targeting different attack surfaces:
306
307 - **Layer 1 ISP-Level Surveillance (WARP):** Your ISP sees only an encrypted WireGuard tunnel to a Cloudflare
  IP. In the US, ISPs can legally sell your browsing metadata and are subject to secret NSLs (National
  Security Letters). WARP removes their visibility entirely.
308 - **Layer 2 DNS Tracking (AdGuard Home):** DNS is the phone book of the internet. By default, queries go to
  your ISP's resolver, giving them a complete log of your traffic. AdGuard intercepts all DNS from mesh
  devices, blocks trackers, and resolves queries through the WARP tunnel.
309 - **Layer 3 Device Identity & IP Exposure (Tailscale):** On untrusted networks (coffee shops, public Wi-Fi),
  your device's IP is exposed. Tailscale creates an encrypted WireGuard mesh between your devices, routing
  all traffic to your gateway exit node.
310 - **Layer 4 Kernel-Level IP Blocking (ipset/iptables):** Sophisticated malware bypasses DNS entirely with
  hardcoded IPs. The `rule-compiler` fetches ~16,700 threat-intelligence IPs/CIDRs (Spamhaus, Feodo Tracker,
  Emerging Threats, CINS) and enforces them as `DROP` rules in the kernel `FORWARD` chain blocking both
  outbound C2 connections and inbound attack traffic.
311
312 ### The Documented Threat: Why This Matters
313 This architecture is calibrated against documented mass-surveillance programs revealed in the Snowden
  disclosures:
314 - **PRISM:** Bulk collection of communication content from major tech providers.
315 - **UPSTREAM:** Tapping physical backbone fiber, collecting metadata and content in bulk.
316 - **XKeyscore:** Retroactive search of unencrypted traffic.
317 - **MUSCULAR:** Infiltration of unencrypted internal datacenter interconnect links.
318
319 Passive bulk collection is not paranoia it is a documented reality. By double-encrypting and routing traffic,
  nullexit prevents your ISP from harvesting your metadata.
320
321 ### Trust Assumptions & Shifting
322 Every component shifts trust rather than eliminating it:
323 1. **Physical Device Integrity:** You trust your physical devices are not compromised.
324 2. **Cloudflare Integrity:** You trust Cloudflare not to correlate your WARP tunnel with exit traffic.
325 3. **Tailscale Integrity:** You trust Tailscale's coordination server not to MITM WireGuard keys.
326
327 The goal: no single provider has a complete picture. Your ISP sees an encrypted tunnel. WARP sees traffic with
  no account identity. Tailscale sees node topology but not content.
328
329 ### 9.1 Exit Node IP Rotation (Design Decision)
330
331 A common question for privacy architectures is whether the system should aggressively rotate its public exit IP
  (e.g., every 5 minutes, like a Tor circuit or a proxy rotator).
332
333 `nullexit` uses Cloudflare WARP via Gluetun. WARP provides **anonymity in a crowd** rather than aggressive
  rotation. Thousands of users share the same Cloudflare Edge IP simultaneously, making your traffic
  indistinguishable from the herd. The IP may occasionally rotate on its own (historically surfaced as a `
  ROTATE` event in `output.log`), but it remains generally stable.

```

```

334
335 **Why aggressive IP rotation was intentionally excluded:**
336 1. **Broken TCP Connections:** Every rotation instantly drops all long-lived TCP sessions (SSH, large downloads
    , WebSockets, gaming, Zoom).
337 2. **CAPTCHA & 2FA Hell:** Modern web security (e.g., Cloudflare, Akamai) aggressively flags IP-hopping as bot
    behavior. Aggressive rotation guarantees the user will be bombarded with "Verify you are human" CAPTCHAs
    and "New Login Detected" 2FA emails constantly.
338 3. **WireGuard Architecture:** WireGuard is stateless and does not natively support IP hopping on the fly. To
    rotate an IP, the VPN client must actively drop the tunnel, request a new endpoint, and re-handshake. This
    introduces latency gaps where the kill-switch would repeatedly trigger and blackhole the user's internet.
339
340 **Verdict:** For a daily-driver secure gateway, shared-IP crowding (WARP) is superior to aggressive rotation.
    As an added benefit, Cloudflare WARP IPs are actually highly static (tied to the persistent WireGuard
    public key generated for your device identity). Because your IP stays static and is part of Cloudflare's
    own trusted network, it builds a high "trust score," virtually eliminating CAPTCHA loops while still hiding
    you within the massive crowd of other users in that data center. All mesh devices routing through your
    gateway will reliably share this exact same trusted IP.
341
342 *(Note: The only exception where aggressive IP rotation is genuinely required is for specialized offensive
    security tasks like high-volume web scraping or dark web research where avoiding IP bans or active tracking
    overrides the need for TCP stability.)*
343
344 ### 9.2 Cloudflare WARP Privacy & Logging
345
346 Because `nullexit` uses Cloudflare WARP (via Gluetun) as its upstream exit node, the system inherits Cloudflare
    's consumer privacy policy.
347
348 **What Cloudflare DOES NOT log (Strict No-Logs Policy):**
349 * **Browsing History:** They do not log the domains or URLs you visit.
350 * **Traffic Destinations:** They do not log the IP addresses of the servers you connect to.
351 * **Content:** They do not inspect or log the payload/data of your traffic.
352 * **Data Brokering:** They explicitly guarantee they will never sell or rent your data to advertisers.
353
354 **What Cloudflare DOES log (Minimal Telemetry):**
355 * **Data Transfer Volume:** Total bandwidth used (required to enforce 10GB/mo quotas if you are not using a
    paid WARP+ key).
356 * **Performance Metrics:** Anonymized connection speeds and latency to optimize their network routing.
357 * **Aggregate Volume:** Total traffic volumes to popular destinations in aggregate (e.g., "datacenter X sent 50
    TB to Netflix"), stripped of all user IDs.
358 * **Device ID (Public Key):** They store the persistent WireGuard public key generated by your container to
    authorize the connection.
359
360 **Privacy Verdict:**
361 Cloudflare WARP provides excellent **historical privacy**. Because they do not log your destinations, they
    cannot comply with historical subpoenas asking what websites you visited yesterday. However, it is **not
    absolute anonymity. They still know your real ISP IP address while you are actively connected. For daily-
    driver privacy against ISPs, hackers, and corporate trackers, it is top-tier. For nation-state evasion, use
    Tor (see 11).
362
363 ### 9.3 OPSEC: Python Runtime Airgap (Isolated Mode)
364 `nullexit` explicitly relies on the host's native system Python 3 runtime for critical components like `scripts
    /dns-proxy.py`. Because this script runs with elevated privileges (`sudo -n`) to bind to the privileged UDP
    /53 socket, modifying or polluting this Python environment is strictly forbidden.
365
366 **The Zero-Dependency Rule & Isolated Mode (-I)**
367 You must **NEVER** install third-party `pip` packages (e.g., `requests`, `pyyaml`) into the system Python
    environment that `nullexit` uses.
368 - All Python scripts in this repository must be written using *only* the Python Standard Library (`socket`, `
    urllib`, `json`, `ssl`).
369 - Third-party packages introduce a massive supply-chain attack vector. A compromised `pip` package installed
    globally could allow local malware to hook into the Python runtime and exploit the `sudo` privilege of the
    DNS proxy to gain full root access.
370 - To enforce strict immunity against user-installed malicious packages, `toggle.sh` explicitly invokes the
    proxy using **Python's Isolated Mode** (`python3 -I`). This flag forces the interpreter to completely
    ignore `PYTHONPATH`, `PYTHONHOME`, and the user's `site-packages` directory, executing exclusively from the
    read-only, Apple-signed system framework.
371
372 ### 9.4 Recommended Upgrades
373
374 | Layer | Default | Upgraded |
375 |---|---|---|
376 | VPN Exit | Cloudflare WARP (US) | Mullvad (Swedish, proven no-logs, anonymous payment) |
377 | DNS Resolver | AdGuard via WARP | AdGuard via Mullvad DoH (Cloudflare-blind) |
378 | Coordination | Tailscale (US, OAuth) | Headscale (self-hosted, no third-party) |
379 | Auth | OAuth / persistent keys | Ephemeral auth keys (no identity persistence) |
380 | Content | WireGuard asymmetric | WireGuard + PSKs (coordination server blind) |
381
382 #### Mullvad (Replacing WARP)
383 Swedish-based. Police raided their offices in 2023 and left empty-handed no logs exist to seize. Accounts are
    random numbers. Payment by cash or Monero. Replace `VPN_SERVICE_PROVIDER=custom` with `VPN_SERVICE_PROVIDER
    =mullvad` in `docker-compose.yml`.

```

```

384
385 ##### Mullvad DoH Upstreams
386 Point AdGuard's upstream resolvers to `https://adblock.dns.mullvad.net/dns-query`. Cloudflare WARP only sees
    encrypted HTTPS to Mullvad's IPs completely blind to your DNS queries.
387
388 ##### Headscale
389 Self-hosted Tailscale coordination server. Eliminates Tailscale the company, keeping all node topology under
    your control.
390
391 ##### Ephemeral Auth Keys
392 Generate in the Tailscale admin panel. Used once to authenticate; node disappears from the admin panel after
    disconnect. Breaks persistent identity linkage.
393
394 ##### Pre-Shared Keys (PSKs)
395 Add a WireGuard PSK between peer nodes. Adds a symmetric encryption layer that Tailscale's coordination server
    has no knowledge of, blinding it from reading transit content.
396
397 ### 9.5 Structural Limitations
398 - **Traffic Analysis / Metadata:** Passive fiber tapping (UPSTREAM) can observe timestamps, data volumes, and
    IP ranges without reading content. Defeating traffic analysis requires mixing networks (Tor) or continuous
    dummy packet padding both heavily degrade performance.
399 - **Targeted State-Level Adversaries:** Consumer-grade VPNs are not a complete shield against a nation-state
    actively targeting you. This stack is designed to defeat bulk passive surveillance and corporate dragnet
    collection.
400
401 ---
402
403 ## 10. Per-Device Access Control
404
405 Because every mesh device routes internet-bound traffic through the `warp` container's `FORWARD` chain, they
    all appear with their stable `100.x.x.x` Tailscale IP as the `src` address. This gives two powerful
    enforcement surfaces:
406
407 ### IP & Port Level (`scripts/routing-fix.sh`)
408 Write raw `iptables` rules targeting specific devices. The 30-second idempotent loop auto-survives Gluetun
    reconnects:
409
410 ```bash
411 # Block a specific device from a target subnet
412 iptables -I FORWARD -s 100.x.x.x -d 203.0.113.0/24 -j DROP
413
414 # Restrict a device to only HTTP/HTTPS
415 iptables -I FORWARD -s 100.x.x.x -p tcp ! --dport 3000 -j DROP
416 iptables -I FORWARD -s 100.x.x.x -p tcp ! --dport 443 -j DROP
417
418 # Block a device from internet egress entirely (mesh only)
419 iptables -I FORWARD -s 100.x.x.x -j DROP
420
421 # Time-based (e.g., cut off 10 PM 7 AM)
422 iptables -I FORWARD -s 100.x.x.x -m time --timestart 22:00 --timestop 07:00 -j DROP
423 ```
424
425 ### DNS Level (AdGuard REST API)
426 AdGuard Home supports per-client rules keyed by Tailscale IP:
427
428 ```bash
429 curl -X POST http://localhost:80/control/clients/add \
430   -H "Content-Type: application/json" \
431   -d '{
432     "name": "kids-ipad",
433     "ids": ["100.x.x.x"],
434     "blocked_services": ["youtube", "tiktok", "instagram"],
435     "safesearch_enabled": true
436   }'
437 ```
438
439 ---
440
441 ## 11. Tor & OPSEC Architecture
442
443 ### 11.1 Tor-over-WARP Topology
444
445 When extreme anonymity is required, integrating the Tor network into `nullexit` provides a powerful Tor-over-
    VPN topology:
446 * **The University/ISP** sees only an encrypted Cloudflare WireGuard tunnel, rendering them completely blind to
    the fact that you are using Tor (effectively bypassing Enterprise firewall Tor blocks).
447 * **Cloudflare** sees an encrypted TCP stream heading to a Tor Guard Node. They cannot see your DNS lookups,
    the destination website, or the content of your traffic.
448 * **The Tor Exit Node** sees your traffic, but not your real IP (only the Cloudflare IP).
449
450 ##### The "Transparent Proxy" Trap (What NOT to do)

```


451 It is tempting to build a system-wide "Transparent Tor Proxy" that forces all host traffic (e.g., standard Chrome/Safari browsers, background iCloud syncs) through Tor automatically. ****This is a massive OPSEC trap and destroys anonymity.****

452 Modern operating systems and browsers will instantly leak your identity through background syncing (e.g., Apple Mail, Google Drive) and browser fingerprinting (canvas fingerprints, WebRTC, screen resolution). The Tor network protects your ***IP***, but if your payload contains identifying cookies or unique fingerprints, the Tor Exit Node (which could be run by malicious actors) will instantly tie your session to your real identity.

453

454 **#### The nullexit Implementation: Isolated Procedural Proxy**

455 To safely integrate Tor without triggering the transparent proxy trap, ``nullexit`` implements an ****Isolated Tor SOCKS5 Proxy**** with ****Procedurally Generated Ports****:

456 1. ****Isolated Container:**** A lightweight ``tor`` Docker container runs securely inside the ``warp`` network namespace. It does not hijack system traffic.

457 2. ****Opt-In Usage:**** You must consciously configure a highly-hardened browser (like the official Tor Browser or a dedicated Firefox profile) to use the proxy. This prevents accidental background sync leaks.

458 3. ****Procedurally Generated Ports:**** Rather than hardcoding the standard proxy ports (like ``127.0.0.1:9050`` or ``1080``), ``toggle.sh`` randomizes all internal proxy ports on every boot into the ephemeral range (10000-65000) and silently writes them to ``/tmp/nullexit-ports.env``. This completely defeats static fingerprinting by malware.

459 4. ****Kernel-Level Collision Avoidance:**** To prevent the randomizer from picking a port already in use by a root daemon or Apple service (which would crash the gateway), the script directly queries the macOS kernel socket table (``netstat -an -p tcp``) to verify the port is absolutely free before assigning it.

460 5. ****The Honey-Port Tripwire:**** While procedural ports defeat static fingerprinting, advanced malware might attempt a full sequential port scan of ``localhost`` to find the hidden proxy. To counter this, ``toggle.sh`` spawns a "Honey-Port" (a fake random port with a netcat listener attached). If any local application attempts to connect to the Honey-Port, it instantly logs a critical security alert to ``output.log`` and fires a macOS desktop notification, serving as an early-warning Intrusion Detection System (IDS) for local malware.

461

462 > ****Threat Model note:**** The Tor SOCKS proxy and Honey-Port Tripwire only bind to ``127.0.0.1`` (loopback). Port randomization and the Honey-Port only defeat blind, generic port-scanners they do NOT protect against a targeted local process that reads ``/tmp/nullexit-ports.env``, greps process memory, or runs ``lsOf -i``. The proper mitigation is a host-level loopback-filtering firewall (LuLu/Little Snitch). See 15.9 (Threat Model: Local Proxy Discovery) for the full analysis and the rogue-binary verification test.

463

464 **### 11.2 Hardening: Tor Bridges and obfs4 Pluggable Transports**

465 For users operating under severe Deep Packet Inspection (DPI), ``nullexit`` supports configuring Tor to connect exclusively through private bridges using the ``obfs4`` pluggable transport.

466

467 When ``TOR_USE_BRIDGES=true`` is set:

468 - ****What it does:**** It hides the entry IP from being matched against the public Tor consensus list, and disguises the traffic's protocol fingerprint from the WARP exit provider (Cloudflare).

469 - ****What it does NOT do:**** It does ***not*** add any extra layers of anonymity beyond what Tor already provides. It is purely a censorship-circumvention and obfuscation mechanism.

470 - ****Limitations:**** ``obfs4`` is highly effective against passive DPI, but it is not guaranteed to defeat active-probing-resistant detection by a highly sophisticated state-level adversary.

471

472 > ****Architecture Rule:**** The ``nullexit`` gateway deliberately does NOT bundle, hardcode, or automatically fetch bridge lines. Bridge lines are highly sensitive and expire. Users must obtain their own bridge lines from ``https://bridges.torproject.org`` and manually populate the ``tor-bridges.txt`` file. If bridges are enabled but the file is missing or empty, the Tor container will intentionally crash and fail to start rather than silently falling back to public guard relays.

473

474 ****The Bootstrapping Paradox (Out-of-Band Key Exchange)****

475 Automating the bridge acquisition process (e.g., scripting the gateway to log into the Telegram ``@GetBridgesBot`` via MTPROTO API) introduces catastrophic OPSEC flaws:

476 1. ****Identity Leaks:**** Baking a Telegram session into the container explicitly links the "anonymous" gateway to a real-world physical phone number.

477 2. ****The Chicken & Egg Deadlock:**** In heavily censored regimes, Telegram itself is usually blocked. If the container requires Telegram to fetch bridges to bypass the firewall, it will fail because it cannot bypass the firewall to reach Telegram.

478 3. ****Bridge Exhaustion & CAPTCHAs:**** Automated scraping behaves identically to state-sponsored scrapers, triggering Tor's anti-bot CAPTCHAs which leads to silent proxy failures and exhausts the limited public bridge pool.

479

480 Instead, users should rely on a less secure layer to manually reach Telegram, obtain the bridges, and populate ``tor-bridges.txt``. This can be done via the standard ``nullexit`` WARP tunnel, a mobile phone on cellular data, Mullvad VPN, or a secure remote VPS. ***(Note: We plan on fully integrating Mullvad and VPS architectures directly into `nullexit` in the future, but have not had the time to build it yet).*** This intentionally "air-gaps" the cryptographic key exchange from the proxy infrastructure, leaving zero API tokens and zero network traces for an adversary to exploit.

481

482 **### 11.3 Tor Container Kill-Switch Inheritance & Fail-Closed Egress Verification**

483

484 **#### Context**

485 Unlike the host's SOCKS5 fallback path (whose kill-switch interaction is documented in 15.7), the ``tor`` container's connection status and fail-closed behavior under VPN tunnel drops is not governed by macOS ``pf`` rules. Instead, it relies on network namespace inheritance inside Docker Compose.

486

487 **#### Mechanics**


```

488 1. **Shared Network Namespace:** The `tor` service is configured with `network_mode: service:warp` in `docker-
compose.yml`. This shares the network namespace of the `warp` (Gluetun) container.
489 2. **Inherited Egress Rules:** All socket communication within the namespace is bound to the same networking
stack. Gluetun manages this stack by redirecting traffic through `tun0` (the WireGuard/WARP tunnel) and
applying `iptables` rules that explicitly drop outbound traffic originating on `eth0` (the physical/bridge
interface), except for permitted local subnets or the VPN server's endpoint.
490 3. **Tunnel Fail-Closed:** If the WARP connection drops or the `tun0` interface is brought down, the routing
table entries for `tun0` become invalid or disappear. Because the `iptables` rules in the shared namespace
continue to drop any outgoing internet traffic attempting to exit via `eth0`, the `tor` container cannot
leak raw traffic to the host's underlying networks. Egress fails closed.
491
492 #### Verification / Manual Test Case
493 To manually verify that the Tor container fails closed and does not leak traffic when the tunnel dies:
494
495 1. **Verify active path:** Ensure the gateway is active (`toggle.sh start`) and fetch the Tor port (`
$TOR SOCKS_PORT` in `/tmp/nullexit-ports.env`). Verify Tor traffic routes correctly:
496 ```bash
497 curl --socks5-hostname 127.0.0.1:$TOR SOCKS_PORT https://check.torproject.org/api/ip
498 ```
499 *(This should output a Tor exit node IP).*
500
501 2. **Simulate tunnel drop:** Bring down the virtual interface inside the shared network namespace:
502 ```bash
503 docker compose exec warp ip link set dev tun0 down
504 ```
505
506 3. **Re-run the egress check:**
507 ```bash
508 curl --socks5-hostname 127.0.0.1:$TOR SOCKS_PORT https://check.torproject.org/api/ip
509 ```
510 * **Expected Result:** The request must hang and eventually time out, or immediately fail with a proxy error
(e.g., `curl: (97) SOCKS5: connection failed` or `curl: (7) Failed to connect`).
511 * **Leaked Egress Check:** Check the host's Wi-Fi router or firewall logs. No outbound packets from the Tor
container should route directly over the bridge network to the internet.
512
513 ### 11.4 Transparent .onion Routing via RFC 2544 Subnet (198.18.0.0/15)
514
515 #### Context
516 To enable seamless dark web browsing across the entire mesh, the gateway requires a mechanism to dynamically
map `.onion` domains to IP addresses that the host operating system and network layers can route.
517
518 #### IP Address Conflict & Solution
519 1. **Initial Conflict:** Initially, the Tor virtual address network was configured to map `.onion` sites within
the `10.192.0.0/10` range. However, the Colima Docker daemon dynamically allocated `10.200.1.0/24` to the
containers. Because this Docker subnet mathematically falls within the `10.192.0.0/10` block, a routing
loop occurred: all host packets destined for the Docker containers on ports like `9050` or `9051` were
intercepted by the transparent proxy NAT rules and dropped, causing proxy failures.
520 2. **Tailscale Private IP Exclusion:** Shifting the subnet to another private range like `10.123.0.0/16`
resolved the port conflict, but led to connection timeouts. On macOS, Tailscale's Exit Node routing
actively excludes RFC 1918 private subnets (e.g. `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`) to
maintain accessibility to local LAN resources (like printers/routers). Thus, packets targeting `10.123.x.x`
were dropped locally by Tailscale before reaching the gateway tunnel.
521 3. **The Benchmarking Subnet Solution:** To bypass these private IP exclusions without manual static routing
tables, Tor's virtual network was changed to `198.18.0.0/15` (reserved by RFC 2544 for benchmarking).
Since this is not a private RFC 1918 range, Tailscale exit-node routing automatically encapsulates and
routes all traffic destined for `198.18.0.0/15` through the tunnel to the gateway.
522 4. **Transparent NAT Redirection:** The `routing-fix` sidecar applies NAT rules in the shared network namespace
:
523 ```bash
524 iptables -t nat -I PREROUTING -d 198.18.0.0/15 -p tcp -j REDIRECT --to-ports 9040
525 ```
526 This intercepts all incoming TCP traffic targeting the mapped `.onion` range and transparently proxies it
through Tor's transparent port (`9040`).
527
528 #### Exit Node Exclusions
529 The gateway supports the ability to exclude specific countries from being used as exit nodes (e.g. to avoid
certain jurisdictions or nodes with poor network latency). This is configured globally via the `
TOR_EXCLUDE_EXIT_NODES` environment variable in `.env`, which gets dynamically appended to Tor's `
ExcludeExitNodes` and strictly enforced with `StrictNodes 1`.
530
531 ---
532
533 ## 12. Censorship-Resistant Transport & Egress Compartmentalization
534
535 ### 12.1 Why this is needed
536 WARP can be blocked from deployment countries with strict internet controls. The most likely cause isn't that "
encrypted traffic is blocked" in general it's that WireGuard has a recognizable handshake signature, and `
WIREGUARD_ENDPOINT_IP=162.159.192.1` on `WIREGUARD_ENDPOINT_PORT=2408` is a fixed, easily-enumerable target
(Cloudflare's known WARP range). That combination is exactly what IP/ASN-based and DPI-based blocking is
good at catching.
537

```

```

538 ### 12.2 Important correction: Gluetun's `SHADOWSOCKS=on` is the wrong direction
539 Gluetun's built-in Shadowsocks option runs a Shadowsocks **server** inside the already-connected VPN tunnel, so
    other LAN devices can reach out through Gluetun's *existing* connection via the SS protocol. It is not a
    way to make Gluetun itself connect **outbound** through a Shadowsocks server. Gluetun only speaks OpenVPN
    or WireGuard as its actual transport. There is no `VPN_TYPE=shadowsocks`. Confirmed against Gluetun's own
    maintainer/community discussion: people have asked for a "Shadowsocks-client" mode specifically to chain
    through Gluetun, and the standing answer is "run a separate Shadowsocks client container."

540
541 ### 12.3 Diagnose before building anything
542 The right fix depends entirely on what's actually being blocked:
543 - Point Gluetun at a **different provider/IP/ASN** (any other Gluetun-supported WireGuard/OpenVPN provider, or
    a self-hosted WireGuard server) and test. If that gets through, the block is Cloudflare/WARP-specific, not
    a general WireGuard block. **no new component needed**, just swap `VPN_SERVICE_PROVIDER` / the endpoint.
544 - If WireGuard fails regardless of endpoint, the block is protocol-level (DPI fingerprinting the handshake)
    proceed to obfuscation.

545
546 ### 12.4 Path A: Swap WARP for a different Gluetun-native transport
547 Simplest fix, only applies if 12.3 shows the block is Cloudflare-specific. Change `VPN_SERVICE_PROVIDER` / `
    VPN_TYPE` / endpoint in `docker-compose.yml` and in the now-centralized `WARP_ENDPOINT_*` values in `.env`.
    Nothing else in the stack needs to change.

548
549 ### 12.5 Path B: Add an obfuscation hop
550 Needed if WireGuard itself is being fingerprinted/blocked. Two ways to slot it in:
551
552 **B1 Wrap (recommended):** Run a Shadowsocks (or obfs4/v2ray) client as a new sidecar container. Point Gluetun
    's `WIREGUARD_ENDPOINT_IP` at `127.0.0.1:<local-port>` instead of Cloudflare directly; the sidecar relays
    that traffic to a Shadowsocks server you run yourself abroad. Keeps Gluetun's kill-switch, healthchecks,
    and the entire rest of the stack (Tailscale, AdGuard, ipset, `routing-fix.sh`) untouched, since none of
    them know the transport changed underneath `warp`.

553
554 **B2 Replace:** Swap the `warp` service entirely for a Shadowsocks-client + `tun2socks` container that owns
    the network namespace instead of Gluetun. More invasive. loses Gluetun's built-in kill-switch/healthcheck,
    which would need to be rebuilt by hand (see Section 12.9 for the pluggable architecture to support this
    natively).

555
556 **Recommendation: B1.** Smaller surface area, preserves the self-healing architecture already built and
    documented.

557
558 ### 12.6 Fingerprinting caveat
559 Plain Shadowsocks can still be caught by active-probing DPI in more aggressive censorship environments. If
    plain SS gets blocked too, the next step up is an obfuscation plugin (`v2ray-plugin`, `Cloak`) or a newer
    protocol designed against active probing (VLESS+Reality, Hysteria2). Test plain SS first. don't over-build
    before confirming it's needed.

560
561 ### 12.7 AmneziaWG (The High-Speed Alternative)
562 If you want to maintain the bare-metal speeds of WireGuard while bypassing DPI (e.g. in Egypt or Russia), the
    best alternative to Shadowsocks/V2Ray is AmneziaWG. AmneziaWG is a fork of WireGuard that pads the
    handshake packets with randomized "junk" data, entirely destroying the 148-byte DPI signature that
    firewalls look for.
563 - **Pros:** Because it runs entirely as UDP without TCP-over-TCP overhead, it completely avoids "TCP meltdown"
    latency spikes associated with Shadowsocks/V2Ray wrappers.
564 - **Cons:** You cannot use Cloudflare WARP. Furthermore, it requires completely ripping Gluetun out of the `
    nullexit` stack and building a custom Docker container, meaning you lose Gluetun's built-in kill-switches
    and health-check logic.

565
566 ### 12.8 Infrastructure Costs & Privacy
567 To use either Shadowsocks (Path B) or AmneziaWG, you cannot use the free Cloudflare WARP infrastructure,
    because Cloudflare servers only speak standard WireGuard. The software for both custom protocols is 100%
    free and open-source, but you must host the remote server infrastructure yourself.
568 - **Free Tier (Oracle Cloud):** You can host your remote server on Oracle Cloud's "Always Free" tier for $0.
    However, this routes your highly sensitive, censorship-evading traffic directly through Oracle's massive
    corporate data broker. If privacy is the core objective, handing all your metadata to Oracle defeats the
    purpose.
569 - **Paid Tier (Hetzner / Mullvad):** The recommended path is to rent a standard ~$4/month Virtual Private
    Server (VPS) in a free country (e.g., Hetzner in Germany, subject to strict EU GDPR privacy laws) and self-
    host the server. Alternatively, Mullvad VPN (5/month) provides native v2ray/Shadowsocks bridges built into
    their network, allowing you to bypass censorship with a strict zero-logs policy. It is highly recommended
    to pay with untrackable payment methods to maintain complete anonymity, which these providers typically
    support without requiring local residency.

570
571 ### 12.9 The Future: Egress Compartmentalization (Pluggable Architecture)
572 To natively support invasive replacement paths like **Path B** or **AmneziaWG** (Section 12.7) without
    breaking the core `nullexit` routing and monitoring logic, the architecture must be refactored into a
    modular, "pluggable" design.

573
574 1. **The Egress Plugin System:** Currently, WARP-specific logic (like `cdn-cgi/trace` health checks and
    hardcoded interface names) is scattered across `toggle.sh` and `scripts/routing-fix.sh`. This should be
    abstracted into a `scripts/plugins/` directory, where each egress method (`warp.sh`, `v2ray.sh`) implements
    standardized functions: `get_egress_interface()`, `check_health()`, and `get_bypass_ips()`.
575 2. **Dynamic Docker Compose:** Based on an `EGRESS_TYPE` variable in `.env`, dynamic compose file generation
    would spin up only the required egress containers (e.g., skipping the `warp` container when `EGRESS_TYPE=

```

```

v2ray` is set).
576 3. **Agnostic Routing & Watchers:** `routing-fix.sh` would dynamically read the `EGRESS_INTERFACE` from the
active plugin to apply `iptables` rules, and the background watchers in `toggle.sh` would become an
agnostic `Egress Watcher` that invokes `check_health()` periodically.
577
578 This compartmentalization transforms `nullexit` into a universal, censorship-resistant gateway where the entire
egress layer can be swapped by changing a single `.env` variable and running `./toggle.sh --restart`.
579
580 ---
581
582 ## 13. Roaming & Recovery Design: How nullexit Mimics Commercial VPNs
583
584 The `nullexit` roaming/network recovery subsystem replicates the exact engineering mechanics used by commercial
, premium VPN clients (like Mullvad or NordVPN) to transition across Wi-Fi networks safely and seamlessly.
This is the design overview; the specific bugs encountered and fixed along the way live in the Incident Log
(15.5 Roaming/Sleep-Wake, 15.7 Tailscale P2P/SNAT).
585
586 ### The 4-Step Transition Cycle
587 When your Mac disconnects from one Wi-Fi and associates with another, `nullexit` coordinates the following
events:
588
589 1. **Firewall Lock (Kill-Switch):** The macOS Packet Filter (PF) anchor `com.apple/nullexit` remains locked on
the physical interface (`en0`), blocking all direct outbound IPv4 and IPv6 traffic. This guarantees that
your real ISP IP or unencrypted DNS requests **never leak** onto the new network while the connection is
rebuilding.
590 2. **System Event Interception:** A launchd LaunchAgent (`watcher.sh`) listens to the macOS System
Configuration framework for the `State:/Network/Global/IPv4` key changes, spawning `recover.sh --post-wake`
in under 500ms when a change is detected.
591 3. **Bypass Routing:** The script dynamically resolves the new physical network's IP gateway (e.g.
`192.168.137.1`), cleans up stale bypass routes from the previous network, and adds static bypass routes
pointing to Cloudflare's WARP endpoints and the Tailscale control plane directly via the new gateway. This
allows the VPN client to negotiate its handshake outside of the tunnel.
592 4. **Hole-Punching / NAT Re-negotiation:** The script runs a target check on the `warp` container's tunnel. If
the WireGuard connection is wedged by the network switch, it force-recreates the container to establish a
fresh NAT binding to Cloudflare, restoring connection in ~15 seconds.
593
594 ---
595
596 ## 14. Deployment, Packaging & Known Limitations
597
598 ### 14.1 Packaging nullexit as a macOS Application (.dmg)
599
600 nullexit can be packaged into a native `.app` bundle, but this introduces nested virtualization requirements.
601
602 nullexit uses **Colima** (Apple Virtualization Framework `vz`) to run a Linux VM hosting Docker. Installing the
`.app` inside a virtualized macOS environment (Parallels, cloud Mac) means running a Linux VM inside a
macOS VM requiring hardware-level **nested virtualization**.
603
604 #### Hardware Support
605 - **Supported:** Apple Silicon M3/M4 (macOS Sequoia 15+), Intel Macs with VMX passthrough.
606 - **Unsupported:** M1, M2, A18 Pro. Colima crashes silently; the terminal (`setup.sh`) surfaces crash logs.
607
608 #### Build Steps
609 ```bash
610 # 1. Verify nested virtualization support
611 sysctl -a | grep hv_nested_virt_supported # expect: 1
612
613 # 2. Compile the AppleScript launcher
614 osacompile -o "Nullexit.app" "Toggle Gateway.applescript"
615
616 # 3. Embed scripts into app resources
617 mkdir -p "Nullexit.app/Contents/Resources/scripts"
618 cp toggle.sh setup.sh recover.sh docker-compose.yml "Nullexit.app/Contents/Resources/"
619
620 # 4. Package into DMG
621 hdiutil create -volname "Nullexit Installer" -srcfolder "./Nullexit.app" -ov -format UDZO Nullexit.dmg
622
623 # 5. Bypass Gatekeeper (unsigned app)
624 xattr -cr /Applications/Nullexit.app
625 ```
626
627 Without a paid Apple Developer certificate ($99/yr), users see an "App is damaged" Gatekeeper warning and need
to run step 5.
628
629 ### 14.2 Moonlight/Sunshine + Tailscale Race Condition
630
631 When streaming from a remote Windows host running Sunshine over a Tailscale mesh (e.g., while the client is on
a cellular hotspot), a race condition can occur on boot:
632
633 1. Both Tailscale and Sunshine are set to start automatically on boot.

```

634 2. Tailscale takes a few seconds to fully initialize its `100.x.x.x` virtual adapter and establish a connection

635 3. ****Sunshine starts too fast.**** It launches before Tailscale is fully ready. Since Sunshine only binds to network interfaces that are active at the exact moment of startup, it misses the Tailscale adapter entirely

636 4. Moonlight clients (even when configured with the correct Tailscale IP) will fail to connect because Sunshine isn't listening on that interface.

637

638 ****The "Magic Toggle" Symptom:****

639 If the user manually enables or disables Wi-Fi on the host PC, it triggers a global Windows network change event. Sunshine listens for these events, re-enumerates all network adapters, and says, "Oh, there's a Tailscale adapter here!" and finally binds to it. Suddenly, Moonlight can connect.

640

641 ****The Permanent Fix:****

642 On the Windows host, open `services.msc`, locate the ****Sunshine Service****, and change its Startup type from ****Automatic**** to ****Automatic (Delayed Start)****. This forces Windows to wait a minute or two after booting before launching Sunshine, ensuring Tailscale's virtual adapter is fully initialized first.

643

644 **### 14.3 Chrome Remote Desktop Performance (UDP NAT)**

645

646 **#### Observation (July 11, 2026)**

647 When the user connects to their host Mac using Chrome Remote Desktop (CRD) while `nullexit` is active, the connection suffers from massive latency, lag, and degraded video quality. AdGuard Home's `blocked.log` shows zero blocked DNS requests for Google, WebRTC, or `chromoting` endpoints, indicating this is not a DNS sinkhole issue. *(See also 15.11 for the separate CRD-on-remote-device connection-failure quirk.)*

648

649 **#### Root Cause**

650 This is a fundamental limitation of tunneling all host traffic through a strict commercial NAT (Cloudflare WARP).

651 1. Chrome Remote Desktop attempts to use ****STUN (UDP hole-punching)**** to establish a lightning-fast, direct peer-to-peer WebRTC connection between the client device and the host Mac.

652 2. Because all non-Tailscale traffic is forcefully routed through the `warp` container, the STUN packets exit the network from a Cloudflare IP.

653 3. Cloudflare's enterprise NAT infrastructure strictly drops unsolicited inbound UDP packets. The UDP hole-punch fails.

654 4. Because a direct P2P connection cannot be established, CRD falls back to ****Relay Mode**** (TURN), bouncing all video data back and forth through Google's cloud servers instead of sending it directly over the local or mesh network. This relaying introduces massive latency.

655

656 **#### Workaround / Fix**

657 This lag is the accepted "tax" for maintaining absolute cryptographic anonymity; the only way to restore CRD speed would be to leak its UDP traffic outside the tunnel (violating zero-trust).

658 To achieve low-latency remote access, users should bypass CRD entirely and use macOS's built-in ****Screen Sharing (VNC)**** directly over the Tailscale IP. Tailscale's sophisticated custom DERP/WireGuard architecture successfully UDP hole-punches through strict NATs, providing a direct, encrypted, high-speed connection without relying on external cloud relays.

659

660 **### 14.4 Dynamic IP Fetch vs. MagicDNS**

661

662 **#### Observation**

663 The `nullexit` gateway uses a dynamically fetched IP address (cached in `.gateway\_ip`) to configure host routing and the `pf` kill-switch, rather than utilizing Tailscale's user-friendly MagicDNS hostname (e.g., `nullexit-gateway`).

664

665 **#### Why MagicDNS is Not Used**

666 1. ****The "Chicken and Egg" Bootstrapping Problem:****

667 macOS's Packet Filter (`pf`) and low-level routing commands (`route add`) operate strictly at Layer 3 and require raw IP addresses. If `toggle.sh` or `recover.sh` attempted to use `nullexit-gateway`, the host Mac would need to perform a DNS lookup to resolve it. However, the very first step of the gateway's initialization is to completely hijack the host's DNS and point it ****at the gateway itself****. The Mac cannot resolve the gateway's MagicDNS name if it needs the gateway's IP to know where to send the DNS query!

668 2. ****Dynamic Self-Healing (Avoiding Stale Cache):****

669 To solve the bootstrapping problem without causing bugs if the node is deleted and re-authenticated on the Tailscale Admin Console, `toggle.sh` explicitly runs `docker compose exec ... tailscale ip -4` during boot to fetch the absolute freshest IP dynamically. It saves this to `.gateway\_ip` so that fast-roaming scripts like `recover.sh` can instantly retrieve the routing target without incurring the latency of querying Docker.

670

671 **### 14.5 Battery & Power Measurement Quirks**

672

673 When trying to optimize this framework (or any daemon) for battery life, developers often look for a way to programmatically measure the exact Wattage consumed by a specific process (e.g., "How many Watts is `toggle.sh` using?").

674

675 ****This is a hardware impossibility on macOS and Linux.****

676

677 **#### Why Activity Monitor is Lying to You**

678 Your machine's logic board only has physical power sensors for the ****entire**** CPU package, the GPU, and the RAM. It knows the CPU is pulling 5W total, but it has no physical hardware capability to know whether Docker is using 3W and Chrome is using 2W.

679

```

680 The "Energy Impact" score you see in macOS Activity Monitor is a **synthetic, heuristic score** calculated by `
    powerd`. It penalizes apps for waking up the CPU (idle wakes), doing disk I/O, or keeping the screen awake.
    It is a relative score, not actual Wattage.
681
682 ##### The Proper A/B Testing Methodology
683 If you want to truly know how many Watt-hours or mAh your background daemons (`routing-fix.sh`, the DNS/WARP
    watchers, etc.) are costing you, you must perform a hardware-level A/B drain test:
684
685 1. Unplug the laptop, run the gateway, and note your exact battery mAh using:
686     ```bash
687     ioreg -l | grep "AppleRawCurrentCapacity"
688     ```
689 2. Leave the laptop idle for exactly 1 hour.
690 3. Run the command again to see exactly how many mAh were drained.
691 4. Turn the gateway off and repeat the test for another hour.
692
693 The delta in mAh drained between the two tests is the true, exact cost of running the software. When optimizing
    polling loops (e.g., changing `sleep 5` to `sleep 30`), this is the only reliable way to measure the
    actual battery life returned to the user.
694
695 ### 14.6 Host Lockdown Mode (Why It's Impossible on macOS)
696
697 A common privacy feature request is a "Host Lockdown Mode" (Mode 3): A mode where the Docker exit node remains
    perfectly functional for remote devices, but the host machine itself is completely blocked from accessing
    the internet (to prevent even encrypted host traffic from leaking metadata).
698
699 **Why this is impossible on macOS:**
700 On Linux (e.g., a Raspberry Pi), Docker runs natively. The host's traffic traverses the `OUTPUT` iptables chain
    , while Docker traffic traverses the `FORWARD` chain. We could easily drop all `OUTPUT` traffic to kill the
    host's internet, and Docker would continue routing traffic perfectly.
701
702 However, Docker on macOS runs inside a Linux Virtual Machine (via `qemu` or Apple `Virtualization.framework`).
    This VM runs as a standard macOS user-space application. If you configure the macOS firewall (`pf`) to drop
    all host outbound traffic, it will indiscriminately drop the VM application's traffic as well, instantly
    killing the Cloudflare WARP tunnel and the exit node.
703
704 Writing complex macOS `pf` rules to "allow the VM app, allow Cloudflare WARP IPs, allow Tailscale DERP IPs, but
    block everything else" is highly fragile and heavily discouraged. Since the current `toggle.sh` default
    mode already fully encrypts the host's traffic via the WARP tunnel, the host is already protected from ISP
    leaks.
705
706 ---
707
708 # Part III Incident Log & Resolved Issues
709
710 This is the single, deduplicated log of every incident, bug, and resolved issue. Entries are organized into
    subsystem groups (15.115.12). Each entry follows a **Symptom / Root Cause / Fix** shape where applicable;
    packet-level analyses are preserved as clearly-labeled **Deep dive** blocks. The terse dated changelog
    lives in 17; this section holds the full post-mortems.
711
712 ## 15. Incident Log
713
714 ### 15.1 Exit-Node Return Path & SOCKS5 Failover
715
716 ##### 15.1.1 Exit Node Return Path (`rx 0`) & The SOCKS5 Failover RESOLVED
717 **Symptom:** Gateway showed `tx 936 rx 0` transmitted but never received return traffic. For days, Tailscale's
    exit node refused to return traffic back to the client, leading to the assumption that Tailscale's
    userspace forwarding was bypassing `iptables` and ignoring the WARP `tun0` interface. In a desperate
    attempt to fix this, a **SOCKS5 proxy** (`scripts/socks5-proxy.py`) was introduced as a replacement.
718
719 **Root Cause:** Tailscale's userspace forwarding was *not* the problem. `docker-compose.yml` included `
    FIREWALL_OUTBOUND_SUBNETS=100.64.0.0/10`. Gluetun created a strict routing rule (`ip rule add to
    100.64.0.0/10 lookup 199`, priority 99) that forced all Tailscale CGNAT traffic to bypass the VPN via the
    unencrypted Docker bridge (`eth0`). Return packets were sent to the macOS host instead of back through `
    tailscale0`, blackholing all return packets back to the client.
720
721 **Fix:** Removed `FIREWALL_OUTBOUND_SUBNETS` entirely, immediately restoring the Tailscale exit node. `scripts/
    routing-fix.sh` now injects `lookup 52`, and return packets correctly flow back into `tailscale0`. Instead
    of removing the SOCKS5 proxy, we repurposed it into a **bulletproof failover** for when restrictive Wi-Fi
    networks block Tailscale UDP ports.
722
723 Architecture after the fix:
724 ```
725 PRIMARY (Exit Node):
726 DNS/TCP/UDP: Apps Tailscale Exit Node gateway container tun0 WARP internet
727
728 FAILOVER (SOCKS5 - if Tailscale is blocked by local Wi-Fi):
729 DNS: Browser 127.0.0.1:53 Python proxy TCP:5354 AdGuard WARP
730 TCP: Apps SOCKS5:1080 gateway container tun0 WARP internet
731 Mesh: SSH/ping 100.x.x.x utun5 WireGuard peers (independent of proxy)
732 ```

```

```

733
734 **Deep dive: Exit Node Return Path Analysis (June 26, 2026).**
735 This preserves the detailed packet-level analysis that led to identifying and fixing the `rx 0` bug.
736
737 *The exit node was probably never going to work in this container topology.* Here's the packet path traced by
    reading `ip rule show` and the routing tables live:
738
739 **Outbound (Phone-1 internet):**
740 1. Phone-1 sends packet to gateway via Tailscale mesh
741 2. Gateway's tailscale0 (kernel TUN, `TS_USERSPACE=false`) decrypts packet enters FORWARD chain
742 3. Packet has src=`100.87.42.87` (Phone-1), dst=some internet IP
743 4. FORWARD chain (nftables): Rule 1 matches (`-i tailscale0 -j ACCEPT`)
744 5. Routing decision for the forwarded packet. Which ip rule matches?
745   - Rule 98: `to 172.18.0.0/16` no (dst is internet)
746   - Rule 99: `to 100.64.0.0/10` no (dst is internet)
747   - Rule 100: `from 172.18.0.2` **NO** (src is `100.87.42.87`, not the container IP!)
748   - Rule 101: `not fwmark 0xca6c lookup 51820` **YES** (no fwmark on forwarded packets)
749 6. Table 51820: `default dev tun0` packet goes out through WARP
750 7. NAT POSTROUTING: `MASQUERADE on tun0` src becomes WARP IP
751 8. Packet exits through WARP to internet (in theory)
752
753 **Return (internet Phone-1):**
754 1. Return packet arrives on tun0, conntrack de-MASQUERADES dst back to `100.87.42.87`
755 2. Routing decision for dst=`100.87.42.87`:
756   - Rule 99: `to 100.64.0.0/10 lookup 199` **YES**
757 3. Table 199: `100.64.0.0/10 via 172.18.0.1 dev eth0` **sends it to the Docker gateway!**
758 4. **This is wrong.** The packet should go to tailscale0 (to be re-encrypted and sent to Phone-1), but table
    199 sends it out eth0 to the Docker bridge host.
759
760 **Table 199 is the smoking gun.** It has one route: `100.64.0.0/10 via 172.18.0.1 dev eth0`. This sends ALL
    Tailscale CGNAT return traffic out the Docker bridge to the host not back through tailscale0. The return
    packet leaves the container's network namespace entirely, hits the host's routing stack, and gets dropped
    because the host has no idea what to do with a raw packet for `100.87.42.87`.
761
762 *Why table 199 exists and why it's wrong for exit node traffic.* Table 199 + the CGNAT rule (pref 99) was
    designed so that the **container itself** can reach other Tailscale peers (e.g., for mesh management, DERP
    relay). For that use case, routing CGNAT traffic via the Docker bridge to the host (which has its own
    tailscaled) makes sense. But for **forwarded exit node traffic**, the return packet needs to go back
    through tailscale0 inside the container not out to the host. The CGNAT rule intercepts the return packet
    before it can be routed back to tailscale0 through the main table or Tailscale's own routing (table 52,
    which is at pref 5270).
763
764 *The SOCKS5 path works because it dodges all of this.* SOCKS5 proxy creates connections **from the container's
    own IP** (`172.18.0.2`). So:
765 - ip rule 100 (`from 172.18.0.2 lookup 200`) matches
766 - Table 200 has `default dev tun0`
767 - Return traffic comes back to `172.18.0.2` on tun0, conntrack de-NATs, proxy sends response to client
768
769 No FORWARD chain involved. No CGNAT rule involved. No table 199. It just works.
770
771 *Summary of findings:*
772
773 | Finding | Status | Evidence |
774 |-----|-----|-----|
775 | Table 199 CGNAT route hijacks exit node return traffic | **Fixed** | Changed `lookup 199` to `lookup 52` in `
    scripts/routing-fix.sh`. Return packets now route via tailscale0. |
776 | FORWARD RELATED, ESTABLISHED missing | **Fixed** | Installed `iptables` via apk in `docker-compose.yml`. Rule
    now injects successfully. |
777 | DOCKER_GW variable parsing error | **Fixed** | Added `head -1` in `scripts/routing-fix.sh`. |
778 | `FIREWALL_OUTBOUND_SUBNETS` was the root cause | **Fixed** | Removed from `docker-compose.yml`. Gluetun no
    longer hijacks CGNAT routing. |
779 | Host tailscaled error state from aggressive `tailscale down` | **Mitigated** | Toggle script now detects and
    auto-restarts. |
780
781 #### 15.1.2 Container Default Route Bypasses WARP
782 **Problem:** Inside the WARP container's network namespace, the main routing table's default route was via `
    eth0` (Docker bridge), not `tun0` (WARP tunnel). While gluetun uses policy routing (rule 101 table 51820
    `tun0`) for most traffic, traffic originating from the container's own IP (`172.18.0.2`) hits rule 100 (`
    from 172.18.0.2 lookup 200`), whose default was also via `eth0`. This caused the SOCKS5 proxy's outbound
    connections to bypass WARP.
783
784 **Fix:** Updated the `routing-fix` container to:
785 1. Change the main table's default route to `default dev tun0`
786 2. Change table 200's default route to `default dev tun0`
787 3. Add a specific route for the WARP WireGuard endpoint (`162.159.192.1`) through `eth0` via the Docker gateway
    in table 200, preventing a tunnel loop
788 4. Re-assert all routes every 30 seconds (gluetun may reset them on health checks)
789 5. Dynamically detect the Docker subnet from `ip route show dev eth0` instead of hardcoding `172.18.0.0/16`
790
791 #### 15.1.3 Hardcoded Docker Subnet in Routing Fix

```



```

792 **Problem:** The `routing-fix` container hardcoded `172.18.0.0/16` and `172.18.0.1` for the Docker bridge
      subnet and gateway. If Docker's bridge network changed (after `docker network prune`, Colima restart, or on
      a different machine), these routes would be wrong.
793
794 **Fix:** Detection is now dynamic using `ip route show`:
795 - `DOCKER_NET=$(ip route show dev eth0 | grep -v default | head -1 | awk '{print $1}')
796 - `DOCKER_GW=$(ip route show default | awk '{print $3}')
797
798 This extracts the actual subnet and gateway directly from the routing table, working with any subnet size
      (`/16`, `/24`, `/20`, etc.).
799
800 ### 15.2 Routing Loops & Subnet Collisions
801
802 #### 15.2.1 The Infinite Recursive Routing Loop (Hotspot Paradox)
803 **Problem:** A critical network topology crash occurs if the host Mac connects to a Mobile Hotspot (e.g., a
      Windows PC or Phone) that is *also* connected to the Tailscale mesh and has "Use Exit Node" enabled.
      Because the Mac is hosting the Exit Node, it routes its internet traffic to the Hotspot. The Hotspot
      intercepts the Mac's traffic on its way to the cellular tower and routes it *back* to the Exit Node (the
      Mac) via Tailscale. The Mac receives it, tries to send it out again, and the Hotspot intercepts it again.
      This creates an infinite, recursive encryption loop that instantly destroys network bandwidth and drops all
      connections.
804
805 **Fix:** This is a physical routing paradox that cannot be resolved in software. The upstream physical router
      providing internet to the Mac **must** be allowed to route its traffic directly to the physical WAN
      interface. The user must manually disable "Use Exit Node" on the upstream device providing the hotspot.
806
807 **Advanced Workaround:** If the upstream hotspot is a Windows machine and the user explicitly wants it to
      remain connected to the Exit Node, a permanent static route can be injected into the Windows routing kernel
      to forcefully bypass the Tailscale WFP interception for WARP packets. By binding the route to the physical
      adapter's Interface Index (IF), it becomes a permanent, roaming-aware bypass.
808
809 On Windows (Admin Command Prompt):
810 ```cmd
811 route print (find Interface List, note the IF number for the active Wi-Fi adapter, e.g. 15)
812 route -p add 162.159.192.1 mask 255.255.255.255 0.0.0.0 IF 15
813 route -p add 162.159.193.1 mask 255.255.255.255 0.0.0.0 IF 15
814 ```
815
816 On Linux (Root Terminal):
817 ```bash
818 ip route add 162.159.192.1 dev wlan0
819 ip route add 162.159.193.1 dev wlan0
820 ```
821
822 On macOS (Root Terminal):
823 ```bash
824 route add -host 162.159.192.1 -interface en0
825 route add -host 162.159.193.1 -interface en0
826 ```
827
828 *(Note: If the upstream router is an iPhone or Android device, you cannot inject static routes without
      jailbreak/root. You must simply disable "Use Exit Node" in the mobile Tailscale app).*
829
830 This punches a tiny hole straight through the paradox, letting the Mac's WARP packets escape to the physical
      internet while keeping the rest of the upstream router's traffic securely trapped in the Tailscale Exit
      Node.
831
832 #### 15.2.2 Infinite Egress Routing Loops & Subnet Takeover Paradoxes
833 **Problem:** Two distinct network loops can break host egress connectivity entirely when the exit node is
      active:
834
835 1. **The Recursive Host-VM Tunnel Loop:**
836 - **Mechanism:** The host Mac's default route points to the Tailscale exit node (`utun*`). The exit node
      container sends its encrypted tunnel packets (destined for the WARP endpoint `162.159.192.1`) back to the
      host via the VM NAT. Without a static route exception on the host Mac, the host Mac routes those packets
      right back into the exit node (`utun*`), creating an infinite loop that blackouts all host network egress.
837 - **ARP-Scoping Pitfall:** Simply binding the bypass route to the interface (`route add -host 162.159.192.1
      -interface en0`) fails when the default route is deleted or redirected to `utun*`. Since `162.159.192.1` is
      not local to the physical link, macOS fails to resolve it via ARP, dropping all tunnel packets.
838 - **Fix:** Dynamically resolve the active physical gateway IP (e.g. `172.17.0.1`) on startup, and add the
      static host routes for the WARP endpoints via the gateway IP directly (`route add -host 162.159.192.1 <
      gateway_ip>`). Additionally, because standalone `tailscaled` on macOS does not automatically modify the
      system default route via the CLI, we manually redirect the default gateway to `utun*` using `
      setup_exit_node_routing`.
839
840 2. **The Docker Compose Subnet Takeover Collision:**
841 - **Mechanism:** To avoid collisions with the host's campus network, Colima's default bridge (`docker.bip`)
      is moved (e.g. to `10.200.0.1/24`). However, because `172.17.0.0/16` is now free inside the VM, Docker
      Compose's IPAM dynamically takes it over for the project's custom network (`nullexit_default`). Because the
      containers receive IPs on `172.17.0.0/16`, the host Mac cannot send local peer discovery packets (e.g.,
      Tailscale `magicsock` packets on `172.17.0.2:41641`) directly. The host routing table sends them out `en0`

```



```

842     to the physical Wi-Fi, returning `host is down` or `no route to host`.
843     - **Fix:** Explicitly configure a custom default network subnet `10.200.1.0/24` in `docker-compose.yml` to
      prevent Docker Compose from ever reclaiming the conflicting `172.17/172.18` ranges.
844
845 > See also 15.2.1 (The Hotspot Paradox) for the related infinite loop that occurs when the physical upstream
846     router is also a Tailscale exit-node client.
847
848 ##### 15.2.3 Docker Default Bridge vs Host Wi-Fi Subnet Collision (The 172.17.0.0/16 Subnet Overlap)
849 **Context:** When attempting to ping a local Windows client (`172.17.22.187`) or the default gateway
850     (`172.17.0.1`) directly over Wi-Fi, the Mac host returns `No route to host` (displaying a `REJECT` / `!`
851     flag in the routing table). Tailscale on the client also drops offline shortly after starting when using
852     the exit node, failing to establish direct P2P connections and falling back to slow DERP relays.
853
854 **Root Cause & Subnet Overlap Mechanism:**
855 1. **Docker Default Subnet:** By default, the Docker daemon initializes its default bridge network (`docker0`)
856     on the `172.17.0.0/16` IP range.
857 2. **Subnet Conflict:** The host's physical Wi-Fi network happens to be assigned the exact same
858     `172.17.0.0/16` range by the local network router.
859 3. **Routing Clash:** Colima launches a Linux VM on the Mac host to run the Docker engine. Because Docker
860     inside that VM creates `docker0` and claims the `172.17.0.0/16` subnet, any packet sent to `172.17.x.x`
861     from within the containers (such as Tailscale local discovery) is captured by the VM's internal virtual
862     bridge interface (which is down/empty) and is blackholed. On the Mac host, macOS dynamically marks routes
863     as `REJECT` (!) when local ARP resolution fails, causing local pings to immediately abort with `No route
864     to host`.
865 4. **Tailscale Relay Saturation:** Because local peer-to-peer discovery is blackholed, Tailscale falls back to
866     public DERP relay servers (e.g. `relay "tor"` in Toronto). When exit-node routing is enabled, all client
867     web traffic is routed through the relay. Public relays impose strict rate limits and throttle high-volume
868     traffic, resulting in packet drops. This saturation drops Tailscale's control packets, causing the
869     connection to time out and showing the client as offline.
870
871 **Fix:**
872 The canonical script for this is `scripts/fix-docker-bridge-collision.sh`. **One command applies the entire fix
873 :
874
875 ```bash
876 bash scripts/fix-docker-bridge-collision.sh          # apply
877 bash scripts/fix-docker-bridge-collision.sh --dry    # preview changes without applying
878 ```
879
880 It auto-detects the host's actual Wi-Fi subnet (no more guessing whether the campus DHCP handed out `172.x`,
881 `10.x`, or `192.168.x`), picks a non-conflicting `docker.bip` from a small candidate set (defaulting to
882 `172.26.0.1/24` if no overlap is matched), backs up `~/colima/default/colima.yaml` with an ISO-8601
883 timestamp, edits the file **in place** (idempotent replaces an existing `docker.bip` if present, appends
884 under an existing `docker:` block, or creates a fresh block), restarts Colima, rebinds Wi-Fi DHCP, verifies
885 the new default route is no longer Docker's bridge, and re-runs both `./toggle.sh` and `scripts/diagnose-
886 host-leak.sh` to print the post-fix verdict.
887
888 For reference, the underlying manual fix is: edit the Colima configuration file (`~/colima/default/colima.yaml
889 `) on your Mac to define:
890
891 ```yaml
892 docker:
893   bip: 172.26.0.1/24  # any /24 that does NOT overlap your Wi-Fi subnet
894 ```
895
896 Then, restart Colima to apply the new subnet inside the VM:
897
898 ```bash
899 colima restart
900 ```
901
902 This forces Docker to use `172.26.x.x` for its default bridge, freeing the physical `172.17.x.x` range and
903 letting pings and direct Tailscale peer-to-peer connections flow normally.
904
905 ##### 15.2.4 July 4, 2026: The "Network Unreachable" Host Leak & Post-Wake Container Destructions
906 **Symptom:**
907 1. Running `docker compose config` with dummy keys corrupted the user's `.env` file by appending invalid
908     WireGuard keys. This caused the `warp` container to become unhealthy on boot, which triggered a full
909     teardown of the gateway by `toggle.sh`.
910 2. After fixing `.env`, `toggle.sh` successfully booted the gateway, but the host's internet started bypassing
911     the tunnel entirely (a massive host leak). The Cloudflare trace returned `warp=off` and exposed the host's
912     physical ISP IP.
913 3. Running `recover.sh --post-wake` would violently force-recreate all gateway containers and drop the host's
914     internet connection.
915
916 **Root Cause:**
917 - **Bug 1 (The Host Leak):** In `toggle.sh`, the logic to find the Tailscale `utun` interface used `ifconfig |
918     grep -B4 "inet 100."`. Due to the 4 lines of before-context, this regex was accidentally capturing the
919     interface **above** the actual Tailscale interface (e.g. grabbing `utun4` instead of `utun5`). Because `utun4`
920     had no IP address, the subsequent `sudo route add -net 0.0.0.0/1 -interface utun4` silently failed with "
921     Network is unreachable", leaving the host's default route exposed.
922 - **Bug 2 (The Post-Wake Destructions):** The health check in `recover.sh --post-wake` relied on `docker
923     compose exec -T warp curl -sf https://www.cloudflare.com/cdn-cgi/trace`. However, the newer `qmcgaw/glutun`
924     Alpine-based Docker image no longer ships with `curl` pre-installed. The health check failed with `
925     executable file not found in $PATH`, tricking `recover.sh` into thinking the tunnel was completely dead and

```

```

needed to be forcefully recreated via `docker compose up -d --force-recreate`.
885 - **Bug 3 (Restart Flags):** Previous refactors incorrectly permitted `toggle.sh restart` instead of strictly
      enforcing `toggle.sh --restart`.
886
887 **Resolution:**
888 - Cleaned the `.env` file of duplicate dummy entries.
889 - Replaced the brittle `grep -B4` interface parsing in `toggle.sh` with a strict AWK parser: `awk '/^[a-z0
      -9]+:/{iface=$1} /inet 100\./{print iface; exit}' | tr -d ':'`. This mathematically isolates the exact
      interface possessing the Tailscale `100.x.x.x` IP address, preventing the `0.0.0.0/1` route from silently
      failing.
890 - Changed the health check in `recover.sh` to use `wget -qO- --timeout=3` which is natively installed in the
      Gluetun image. This stopped the unnecessary destructive recreations of the containers during post-wake
      events.
891 - Reverted the `restart` alias in `toggle.sh` to strictly require `--restart`.
892
893 #### 15.2.5 July 4-5, 2026: Tailscale Data-Plane Loop (The DERP Relay Deadlock)
894 **Symptom:**
895 After bypassing the Tailscale control plane (`192.200.0.0/16`) to fix the "offline" mesh status, the Mac
      appeared online. However, external peer devices (like the user's phone on cellular) could not successfully
      ping or SSH into the Mac. `tailscale netcheck` reported `Nearest DERP: unknown (no response to latency
      probes)`.
896
897 **Root Cause:**
898 While bypassing the control plane kept the daemon connected to the coordination servers, Tailscale's actual
      peer-to-peer data plane relies on STUN (for NAT traversal) and DERP relays. These relays span ~80
      dynamically allocated IP addresses across multiple cloud providers. Because we were forcing `0.0.0.0/1`
      through the `utun*` exit node, the Mac's own outbound connection attempts to DERP relays were being routed
      back into the tunnel. To prevent an infinite loop, Tailscale's daemon dropped its own packets when it saw
      them returning on the TUN interface. This effectively left the daemon deaf and blind to peer connections.
899
900 **Resolution:**
901 - Modified `add_warp_bypass_routes` in `scripts/common.sh` to execute a live API fetch (`curl -s https://login.
      tailscale.com/derpmap/default`) during startup.
902 - The script parses out all active DERP relay IPv4 addresses (~80 IPs) and adds a physical host bypass route
      for every single one of them.
903 - These IPs are written to a temporary file (`/tmp/nullexit-derp-ips.txt`) so they can be cleanly un-routed by
      `remove_warp_bypass_routes` when the gateway shuts down. Peer connectivity (ping, SSH, SFTP) was fully
      restored.
904
905 #### 15.2.6 July 5, 2026: Hardcoded WARP Endpoints in docker-compose
906 **Symptom:**
907 The bash scripts allowed overriding the default Cloudflare WARP IP endpoints via `.env` variables (`
      WARP_ENDPOINT_1` and `WARP_ENDPOINT_2`) to establish bypass routes. However, `docker-compose.yml`
      statically hardcoded `162.159.192.1` for the `warp` container's `WIREGUARD_ENDPOINT_IP`.
908
909 **Root Cause & Risk:**
910 If a user set a custom endpoint in `.env`, the script would successfully bypass the custom IP on the host level
      . However, Gluetun would still attempt to connect to the hardcoded `162.159.192.1`. Since `162.159.192.1`
      was no longer explicitly bypassed, its packets would be sucked into the `utun*` exit node, causing an
      infinite WireGuard routing loop that instantly breaks the gateway.
911
912 **Resolution:**
913 Modified `docker-compose.yml` to dynamically read the environment variable via `${WARP_ENDPOINT_1
      :-162.159.192.1}`, ensuring both the host routing scripts and the container use the exact same endpoint.
914
915 #### 15.2.7 Incident Post-Mortem: WARP Watcher Race vs Post-Wake (July 10, 2026)
916 **Symptom:** After switching Wi-Fi networks, the host IP appeared as the raw ISP IP (`132.x.x.x`) instead of a
      Cloudflare/WARP IP. The gateway appeared to complete post-wake recovery successfully in the logs, but
      nuclear shutdown fired immediately after and killed everything.
917
918 **Root Cause:** A fatal race condition between two concurrent processes:
919
920 1. **Wi-Fi roam** `watcher.sh` fires `recover.sh --post-wake`
921 2. **Post-wake** finds the `warp` container missing/dead calls `docker compose up -d --force-recreate` (~35
      seconds)
922 3. **In parallel**, the WARP Watcher (running as a child of `toggle.sh`) polls the warp container every 5s
      during failures
923 4. While the container is being recreated it returns `warp=off` for those 35 seconds 7 consecutive failures
      **WARP SHUTDOWN fires**, calling nuclear `recover.sh`
924 5. Nuclear `recover.sh` tears down Tailscale, resets DNS to 1.1.1.1, stops all containers **gateway dead, IP
      exposed**
925 6. The post-wake `docker compose up` finishes at ~35s mark, container healthy but the gateway is already gone
926
927 The evidence in `output.log`:
928 ```
929 [2026-07-10T06:07:27Z] NET: ... bash recover.sh --post-wake
930 [2026-07-10T06:07:47Z] WARP DOWN cdn-cgi/trace reports warp=off watcher starts counting
931 ...
932 add net 0.0.0.0: gateway utun5 post-wake successfully re-routes at 02:08:22
933 ...
934 [2026-07-10T06:09:04Z] WARP SHUTDOWN 6 consecutive failures watcher fires nuclear

```

```

935 ~~~
936
937 **Fix (July 10, 2026):** Added a WARP Watcher inhibit marker (`/tmp/nullexit-warp-inhibit.marker`):
938 - `recover.sh --post-wake` writes the marker at startup **before** any container operations
939 - The WARP Watcher loop checks for the marker at the top of every iteration; if present it resets `consec_off`
    =0` and sleeps 5s (skips the poll entirely)
940 - `recover.sh` removes the marker at the very end (on both success and failure via `trap cleanup_inhibit EXIT`)
941 - This guarantees the watcher never counts failures during the intentional container-down window of a post-wake
    force-recreate
942
943 The marker file path is hardcoded to `/tmp/nullexit-warp-inhibit.marker` in both `toggle.sh` and `recover.sh`.
944
945 #### 15.2.8 Incident Post-Mortem: Concurrency Collision between Nuclear Teardown and Post-Wake (July 10, 2026)
946 **Symptom:** When the background WARP Watcher triggers a nuclear shutdown (due to a real or simulated tunnel
    failure), the gateway is completely torn down. However, the network configuration changes made during the
    teardown trigger a `State:/Network/Global/IPv4` network change event. The LaunchAgent-based network watcher
    (`watcher.sh`) intercepts this event and concurrently spawns `recover.sh --post-wake` to heal/restore the
    gateway. This causes `docker compose down` (from nuclear teardown) and `docker compose up` (from post-wake)
    to run in parallel, resulting in container errors like `dependency failed to start: container warp exited
    (0)` and the gateway becoming wedged.
947
948 **Root Cause:** A lack of coordination/locking between the lifecycle control scripts. While `toggle.sh` used `/
    tmp/nullexit-toggle.lock` to prevent concurrent `toggle.sh` runs, `recover.sh` (both nuclear and `--post-
    wake` modes) did not utilize or check any lock files.
949
950 **Fix (July 10, 2026):** Implemented a shared lifecycle lock file (`/tmp/nullexit-toggle.lock`) across both
    scripts:
951 - **`recover.sh` (all modes):** Checks `/tmp/nullexit-toggle.lock` at startup. If the lock is held by another
    running lifecycle script (`toggle.sh` or `recover.sh`):
952 - In `--post-wake` mode, it logs a skip message to `output.log` and exits cleanly (`exit 0`). This safely
    prevents the auto-recovery agent from recreating containers during a teardown.
953 - In nuclear recovery mode, it exits with an error.
954 - **`recover.sh` Lock Cleanup:** Cleans up the lock file upon completion using `trap cleanup_recover EXIT` (
    which checks if the lock file belongs to the current PID).
955 - **`toggle.sh` update:** The lock check in `toggle.sh` was updated to look for both `toggle.sh` and `recover.
    sh` in the running processes to enforce proper exclusion.
956
957 #### 15.2.9 Incident Post-Mortem: Infinite Auto-Recovery Loop after WARP Watcher Nuclear Shutdown (July 10,
    2026)
958 **Symptom:** When the background WARP Watcher triggered a nuclear shutdown (due to a tunnel outage), it
    successfully executed `recover.sh` (nuclear). However, immediately after the teardown completed, the system
    started spawning `recover.sh --post-wake` and rebuilding the containers again. This created an infinite
    loop of tearing down and auto-restarting the gateway, keeping the host's internet route permanently wedged.
959
960 **Root Cause:** An active-state marker file leak. The LaunchAgent-based network watcher (`watcher.sh`) only
    fires `recover.sh --post-wake` when `/tmp/nullexit-gateway-active.marker` is present on disk. While `toggle
    .sh` (STOP path) correctly deleted this marker, the nuclear `recover.sh` script did not. When the WARP
    Watcher ran `recover.sh` (nuclear), the containers were stopped but the active-state marker was left on
    disk. The network changes from the teardown then triggered `watcher.sh`, which saw the marker, believed the
    gateway was still supposed to be active, and triggered `--post-wake` to start it back up.
961
962 **Fix (July 10, 2026):** Modified the start of `recover.sh` to explicitly delete `/tmp/nullexit-gateway-active.
    marker` if `POST_WAKE` is `false` (nuclear mode). This ensures that any subsequent network configuration
    changes made during the teardown are correctly ignored by `watcher.sh` because the marker file is gone.
963
964 ### 15.3 MTU, Fragmentation & Throughput
965
966 #### 15.3.1 Network Bufferbloat & MTU Optimizations (Double-VPN UDP Crash)
967 **Symptom:** When running aggressive UDP bandwidth tests (such as macOS `networkQuality`, which uses QUIC/HTTP3
    ), the `nullexit` tunnel completely crashes or exhibits severe lag (6,000ms+ bufferbloat).
968
969 **Root Cause:** The bug is a cascading MTU mismatch. Tailscale generates a UDP packet. Cloudflare WARP (also
    WireGuard) receives the packet, wraps it in another layer of encryption, pushing the packet size beyond the
    standard 1500 byte limit. Unlike TCP, UDP packets cannot be resized dynamically via MSS negotiation,
    resulting in massive IP packet fragmentation. The Apple Virtualization framework (`vz`), which Colima uses
    for bridged networking, silently drops heavily fragmented UDP packets under high load. This blackholes the
    entire UDP upload stream, instantly dropping the connection.
970
971 **Fix (Partial):** To prevent MTU fragmentation for standard web traffic, TCP MSS Clamping is strictly
    enforced via the macOS `pf` firewall. By adding `scrub out all max-mss 1160` to `scripts/pf.conf`, macOS
    unilaterally shrinks all outgoing TCP headers to fit comfortably inside the double-encrypted tunnel,
    bypassing Path MTU Discovery blackholes. `TS_DEBUG_MTU=1200` was also added to `docker-compose.yml` to
    minimize internal fragmentation. *(Superseded see 15.3.2: 1160 was later found still too large for double-
    tunneled traffic and tightened to 1120. That fix originally only reached the container's `post-rules.txt`;
    `pf.conf`'s host-side clamp stayed stale at 1160 until it was caught and synced to read `GATEWAY_MSS`
    dynamically, same as `post-rules.txt`, so the host and container can no longer drift apart.)*
972 **Unresolved:** The only way to solve the UDP crash bug natively is to artificially cap the maximum tunnel
    bandwidth using SQM (`fq_codel`). Because a static bandwidth cap would artificially throttle high-speed
    networks, `nullexit` leaves the bandwidth uncapped, optimizing perfectly for TCP while accepting UDP
    fragmentation under extreme edge-case load tests.

```

973 ***Why not Dynamic SQM (Aurate)?*** Implementing an EMA/PID-controlled aurate daemon (like `sqm-aurate`) requires a constant 100ms polling loop to measure the kernel packet queue, which severely degrades laptop battery life by preventing deep C-state CPU idling. Furthermore, macOS's native firewall (`dummynet`) only supports `fq\_codel` and does not support the newer Linux `CAKE` algorithm, which is the only algorithm that offers kernel-native, event-driven `aurate-ingress` (zero polling penalty). Therefore, dynamic SQM is computationally hostile to battery-powered macOS hosts.

974

975 **#### 15.3.2 Double-Tunneling MSS Clamping (Stalling Bug)**

976 ****Problem:**** Traffic double-wrapped through Tailscale (WireGuard) and WARP (WireGuard) suffered 120 bytes of MTU overhead (60 + 60). The default MSS clamp of 1180 was still slightly too large, causing large packets to fragment or drop, which resulted in mysterious web page stalls on strict-MSS endpoints.

977

978 ****Fix:**** Lowered the TCP MSS clamp in `post-rules.txt` to `1120` to guarantee double-tunneled payloads fit safely within standard internet MTUs.

979

980 **#### 15.3.3 Wi-Fi Edge Packet Loss via QUIC (UDP) Fragmentation**

981 ****Problem:**** Applications that heavily utilize HTTP/3 (QUIC) over UDP such as Facebook, Instagram, and Google services experience massive stuttering and stalled image loading when the client device is far away from the router (weak Wi-Fi signal). The issue did not occur when close to the router.

982

983 ****Root Cause:**** The `nullexit` gateway uses a double-encrypted tunnel (Tailscale + WARP). To prevent packets from fragmenting when entering these tunnels, a firewall rule in `post-rules.txt` uses `--set-mss 1120` to safely clamp TCP packets. However, because QUIC runs entirely over UDP, it bypasses the TCP MSS handshake completely. QUIC blasts maximum-MTU (1280 byte) UDP packets. The inner `tailscale0` interface (MTU 1200) forces these large UDP packets to fragment into two pieces. When the client is on the edge of Wi-Fi range (where 5-10% packet loss is common), losing *either* UDP fragment destroys the *entire* image frame. This exponential amplification of packet loss stalled QUIC streams.

984

985 ****Fix & Trade-offs:**** Appended an `iptables` rule to `post-rules.txt` that explicitly blocks `UDP --dport 443` (QUIC). When modern web apps detect QUIC is blocked, they instantly fall back to HTTP/2 (TCP 443). Because TCP is routed through the MSS clamp, the packets are resized *before* they leave, entirely eliminating IP fragmentation and restoring high-speed loading over weak Wi-Fi.

986 ***Consequences:*** This adds a minor latency penalty (TCP 3-way handshake) compared to QUIC's zero-RTT connection. It may also force WebRTC video calls (Google Meet/Discord) to fall back to TCP TURN servers, slightly inflating real-time video latency. However, in a constrained double-tunnel mesh, the absolute stability gained from eliminating fragmentation vastly outweighs these trade-offs.

987

988 **#### 15.3.4 Cellular Networks & PMTUD Blackholes (The 1280 Byte Limit)**

989 ****Experiment:**** Conducted a live packet sweep from the PC-1 host to Phone-1 connected via a cellular network + Tailscale DERP relay using `ping -D -s <size>`.

990

991 ****Result:****

992 - Packets with a payload of `1252` bytes (+ 28 bytes IP/ICMP headers = **1280 bytes total**) succeeded perfectly with ~60ms latency.

993 - Packets with a payload of `1253` bytes (**1281 bytes total**) and above resulted in a silent timeout.

994

995 ****Analysis:**** The cellular network (or the Tailscale DERP relay) enforces a strict MTU of 1280 bytes. Critically, it acts as a "PMTUD Blackhole" it silently drops packets larger than 1280 bytes instead of returning an ICMP "Fragmentation Needed" warning. If the exit node tries to send a standard 1500-byte internet packet to the phone over this link, it vanishes, causing web pages to stall permanently.

996

997 ****Conclusion:**** This empirically validates the absolute necessity of the TCP MSS clamping rule in `post-rules.txt`. By artificially clamping the MSS to `1120` (the value 15.3.2 above found necessary `1180` measured too large), we ensure the TCP payload + headers + double-WireGuard overhead never exceeds the strict 1280-byte ceiling of the cellular mesh link.

998

999 **#### 15.3.5 Low Gateway Throughput (DERP Relay & MTU Fragmentation) Fixed**

1000 ****Problem:**** The host Mac's internet throughput through the gateway dropped significantly (e.g., from 34Mbps raw to 2.1Mbps via gateway). This was caused by a combination of two bottlenecks:

1001

1002 ****1. Tailscale DERP Relay Bottleneck:****

1003 Because the host Mac and the Docker container operate behind the same public IP, standard hole-punching for Tailscale P2P failed, forcing traffic through Tailscale's NYC DERP relay. Normally, `routing-fix.sh` explicitly drops container Tailscale UDP packets destined for local subnets (when `TAILSCALE\_ALLOW\_LAN\_P2P=false` in `.env` or auto-detected). We nuke these P2P connections on purpose to prevent the severe macOS `gvproxy` SNAT endpoint poisoning issue we faced earlier (see 15.7 SNAT Endpoint Poisoning). However, this strict policy unintentionally blocked direct communication between the container and the Mac host itself via the Docker bridge network.

1004

1005 ****Fix:**** Tailscale's control plane registers the Mac host using its physical IP (e.g., `172.17.52.223`), not the Docker bridge IP. If this physical IP falls within the dropped local subnets (like `172.16.0.0/12`), the P2P handshake is dropped. To solve this natively, `scripts/common.sh` now features a `write\_host\_ips()` function that actively grabs all physical IPv4 addresses on the Mac and writes them to `.host\_ips`. `routing-fix.sh` dynamically reads this file every 30 seconds and injects priority `RETURN` rules for those exact physical IPs. This guarantees direct container-to-host P2P over the local bridge (bypassing DERP) while continuing to block external LAN devices (like phones) to prevent the 15.7 SNAT poisoning bug.

1006

1007 ****2. MTU Double-Encapsulation Fragmentation:****

1008 The host's Tailscale interface (`utun5`) defaulted to an MTU of 1280. The WARP tunnel inside the container also uses an MTU of 1280. When a full 1280-byte Tailscale packet from the host entered the container and was encapsulated by WARP (adding ~60 bytes of overhead), the resulting packet exceeded 1280 bytes, leading to

```

severe fragmentation and performance degradation.
1009
1010 **Fix:** In `toggle.sh`'s `setup_exit_node_routing` function, the host's Tailscale `utun` interface MTU is
explicitly lowered to `1200` (configurable via `HOST_MTU` in `.env`). This ensures that host-originated
packets can comfortably fit inside the container's WARP tunnel encapsulation without fragmenting.
1011
1012 > **Design Note Is direct hostcontainer P2P a Docker bug, or are we "exploiting a VM/host relationship"?
1013 > Neither. It is expected, well-defined behavior, and the RETURN-rule carve-out is a *defensive de-restriction
*, not an exploit. The reasoning:
1014 > - **On native Linux**, the container and host share one kernel; the Docker bridge is a first-class local
interface and hostcontainer is *trivially* direct. Nobody calls that an exploit. Our RETURN rules simply
reproduce that same local reachability on macOS.
1015 > - **The RETURN rules add nothing new; they *stop blocking* something legitimate.** `routing-fix.sh` broadly
DROPS containerLAN Tailscale UDP to prevent the real 15.7 `gvproxy` SNAT endpoint-poisoning bug caused by *
external* LAN peers (e.g. a phone on another AP). Tailscale's control plane registers the Mac host by its *
physical* LAN IP (e.g. `172.17.52.223`), which falls inside those DROPPed ranges so the host, the one peer
that is literally the same physical machine, became collateral damage. The `.host_ips` RETURN rules are
the minimal, precise exception that re-permits exactly that same-machine path and nothing else.
1016 > - **The genuinely VM-specific wart is `gvproxy` SNAT poisoning (15.7), and we route *around* it, not through
a hole in it.** Keeping hostcontainer traffic on the Docker bridge (direct) avoids letting it get NATed
through the userspace `gvproxy` endpoint rewriter that confuses Tailscale's peer-endpoint learning. That is
the opposite of exploiting the VM boundary it is respecting it.
1017 > - **What genuinely *doesn't* work on a VM host is UDP hole-punching to arbitrary *external* peers** (see
15.13). That is a real limitation of userspace VM networking, and it is why remote peers fall back to DERP
unless bridged mode is enabled on a trusted LAN. Direct hostcontainer P2P is the one case that works
regardless because both endpoints live on the same physical machine, no hole-punching is needed at all. It
was verified `direct 172.17.52.223` even in plain user-mode networking (no `--network-address`).
1018 >
1019 > In short: the DROP is the deliberate policy, the RETURN is a surgical exception for same-machine traffic, and
the whole thing is standard bridge networking not a Docker bug and not an exploit.
1020
1021 ##### 15.3.6 The "-4Mbps Ceiling" Was a Benchmark Artifact, Not a Tunnel Bottleneck (July 18, 2026)
1022 **Symptom:** Repeated throughput tests against `speedtest.tele2.net` from both the host (full double-tunnel)
and the `warp` container (isolated via its own SOCKS5 proxy) consistently landed around 3.3-4.7 Mbps,
regardless of which leg was tested. This looked like a hard nullexit-imposed ceiling and was the leading
suspect for the Instagram/general-slowness symptoms in 15.3.3-15.3.4.
1023
1024 **Investigation:** Before accepting "the tunnel caps at ~4Mbps," the same target was measured **raw** gateway
fully stopped, plain ISP egress, no tunnel at all and it **still** came back at ~3.2 Mbps. A second raw test
against a fast, nearby CDN (Cloudflare) returned ~32.2 Mbps on the same physical connection. `speedtest.
tele2.net` was the bottleneck, not nullexit; it's a slow/distant target that happened to be the first thing
tried.
1025
1026 **Confirmation:** Re-measured through the tunnel against a fast CDN that (unlike Cloudflare's own speed-test
endpoint, which 403s WARP's shared/CGNAT egress IPs as abuse-prevention) doesn't flag WARP IPs Hetzner's
public speed-test files. Result: **15.8-26.4 Mbps** tunneled, in the same ballpark as the 32.2 Mbps raw
baseline (roughly 20-50% overhead from the double WireGuard encapsulation, normal for this architecture).
CPU and memory on the Colima VM were also checked during sustained transfers (`colima ssh -- top -bn1 / `
free -m`) and ruled out as a bottleneck load average stayed near 0.00-0.07 and swap held flat around 250MB
throughout, on both a run that completed cleanly and one that reset mid-transfer.
1027
1028 **Fix:** `scripts/sweep.sh`'s throughput check (9, check 7/7) now defaults to the Hetzner target instead of `
speedtest.tele2.net`, so future sweeps report real tunnel performance instead of accidentally re-measuring
a slow third-party server and misattributing it to nullexit.
1029
1030 **Residual, unchanged:** the occasional mid-transfer connection reset (curl exit 56, `Recv failure: Connection
reset by peer`) observed on maybe 1-in-6 to 1-in-8 attempts is still unexplained by CPU/memory and remains
consistent with the gvproxy/UDP-fragmentation-under-load issue in 15.3.1 a real, separate, and still-open
question from the throughput-ceiling one this section resolves.
1031
1032 ##### 15.3.7 Field Test: Dense Cellular Congestion on a Mesh Device (July 19, 2026)
1033 **Setup:** An Android mesh device on cellular data, physically present in a very dense public crowd, exit-noded
through `nullexit-gateway` for the whole session. Goal: see whether heavy local RF/tower congestion
degrades the tunnel differently than normal cellular use.
1034
1035 **Method:** Confirmed the device was actively routing live (not just tailnet-idle) via AdGuard's query log
showing fresh DNS lookups from it seconds before the test. Then ran a large TSMP burst (`tailscale ping --
until-direct=false`, the same mechanism as sweep.sh check 6/7 but a much larger sample) to get a real loss/
jitter signal instead of a single ping.
1036
1037 **Result:** Every probe got a pong back 0% loss. The path was DERP-relayed for the entire burst, never direct,
which is expected on cellular (carrier-grade NAT blocks Tailscale's UDP hole-punching regardless of crowd
density, so this isn't specific to a busy area). RTT stayed in a tight, low band for the large majority of
probes, with a couple of outlier spikes consistent with intermittent RF contention/retransmission at the
cell tower from the sheer device density, not a gateway-side problem. All 5 containers stayed healthy
throughout; the WARP double-tunnel was confirmed live.
1038
1039 **Conclusion:** nullexit was not the bottleneck under real-world dense-cellular congestion zero packet loss is
a strong result. The DERP-vs-direct baseline and the latency spikes both trace to the cellular/CGNAT layer
, outside nullexit's control.
1040

```



```

1041 ### 15.4 DNS Interception & Proxy
1042
1043 ##### 15.4.1 DNS Proxy: TCP Wire Format Mismatch (socat / dnsmasq)
1044 **Problem:** When the Tailscale data plane is unavailable (DERP relay only), the script falls back to a local
DNS proxy that forwards queries from host UDP:53 to Docker's TCP:5354 (AdGuard). The `socat` and `dnsmasq`
approaches both failed.
1045
1046 - **socat** forwards raw bytes it does not prepend the 2-byte length prefix that DNS-over-TCP requires.
AdGuard parses the stream incorrectly and returns garbage.
1047 - **dnsmasq** tries UDP first to reach the upstream (`127.0.0.1#5354`), but Colima's SSH tunnel only forwards
TCP. With a single upstream server, dnsmasq doesn't fall back to TCP even with `--timeout=1`.
1048
1049 **Solution:** Replaced both with a **Python DNS proxy** (`scripts/dns-proxy.py`, ~25 lines) that properly
handles the DNS-over-TCP wire format: reads a UDP query from the host, prepends the 2-byte length prefix,
sends it over TCP to AdGuard, strips the prefix from the response, and sends it back over UDP.
1050
1051 *Architectural Note: Why is the DNS proxy in Python while the SOCKS5 proxy was moved to a compiled Go Docker
container?*
```

The SOCKS5 proxy handles heavy data streaming (e.g., video, downloads). Python introduces massive CPU/battery overhead for raw socket streaming, which is why we rely on the compiled Go implementation built directly into the `tailscaled` daemon via `TS\_SOCKS5\_SERVER=:1080`.

```

1052 Conversely, the DNS proxy only handles intermittent UDP queries (a few bytes per minute) generated solely by
the local host machine. The Python overhead for this is effectively 0.001 Watts. We deliberately kept `dns-
proxy.py` in Python because it runs natively on the macOS/Linux host rewriting it in Go would force users to
install a Go compiler (`brew install go`) on their host machine for zero noticeable battery gain. (Note:
If this architecture is ever adapted to serve as a public DNS resolver for thousands of users or for
massive automated web-scraping clusters, `dns-proxy.py` must be rewritten in Go to survive the concurrent
packet flood).
```

1054

```

1055 ##### 15.4.2 Tailscale MagicDNS Bypassing the AdGuard PREROUTING Intercept
1056 **Context:** The `routing-fix.sh` script applies an iptables NAT rule (`PREROUTING -i tailscale0 -p udp --dport
53 -j REDIRECT`) to intercept all incoming DNS queries from connected Tailscale peers and force them into
the AdGuard container on port 5335.
1057 **Problem:** When remote clients (like Android or iOS) have "Use Tailscale DNS settings" turned on, they query
Tailscale's MagicDNS IP (`100.100.100.100`). This packet enters the exit node, but the `tailscaled` daemon
running *inside* the gateway container intercepts the `100.100.100.100` packet internally. `tailscaled`
then generates a brand new, local DNS request to the upstream DNS server to resolve the query. Because this
new request is generated *locally* by the daemon (not arriving externally via the `tailscale0` interface),
it bypasses the `PREROUTING` chain completely. The request glides straight out the WARP tunnel to
Cloudflare/Google, entirely bypassing the AdGuard sinkhole.
1058 **Fix:** The only way to fix this blindspot is to force `tailscaled` to use AdGuard as its upstream. In the
Tailscale Admin Console, the gateway's Tailscale IPv4 address (e.g., `100.x.x.x`) must be added as the sole
Custom Global Nameserver, and "Override local DNS" must be enabled. Additionally, to block Android devices
from bypassing this via Private DNS (DNS-over-TLS), outbound Port 853 traffic is explicitly REJECTED in
both `routing-fix.sh` and `post-rules.txt`.
```

1059

```

1060 ##### 15.4.3 Seamless Wi-Fi Roaming & The macOS DNS Wipeout Loophole
1061 **Problem:** The Tailscale, WireGuard, and Colima NAT stacks are natively designed for connectionless roaming
and will seamlessly survive when the macOS host switches Wi-Fi networks (e.g., roaming from home Wi-Fi to a
cellular hotspot). However, the internet completely breaks upon network transition. This occurs because
macOS intentionally wipes out all custom DNS settings (`networksetup -setdnsservers`) and reverts to the
new network's default DHCP nameserver whenever the active Wi-Fi BSSID changes. Since the Mac is locked to
the exit node, its DNS requests are swallowed by WARP and dumped onto the public internet, which cannot
route private DHCP IPs. This results in a total DNS failure.
1062
1063 **Solution:** Implementing a silent background "DNS Watcher" daemon (`nullexit-dns-watcher`) in `toggle.sh`.
When the gateway starts, it spawns a background polling loop that checks the active Wi-Fi DNS every 30
seconds. If macOS resets the DNS during a network change, the watcher instantly detects the drift and
forcefully re-injects `100.100.21.8` via `networksetup`. When the gateway stops, the watcher process is
cleanly killed. This allows true, seamless roaming across physical networks without ever dropping the
gateway state.
```

1064

```

1065 ##### 15.4.4 docker compose exec `r` Injection
1066 **Fix:** All `docker compose exec` output piped through `tr -d '\r'`.
1067
1068 ##### 15.4.5 Stderr Invisibly Swallowed
1069 **Symptom:** Failures were silent due to stderr redirected to `output.log`.
1070
1071 **Fix:** Tailscale wait loop now shows output. Critical commands show errors inline.
```

1072

```

1073 ### 15.5 Roaming, Sleep/Wake & Auto-Recovery
1074
1075 ##### 15.5.1 Gateway Breakage on Lid Close / Wi-Fi Roam Post-Wake + Post-Roam Auto-Recovery
1076 **Context / Symptom:** Closing the lid (sleep/wake) OR losing + reconnecting Wi-Fi (e.g. in an elevator, caf,
or roaming between APs) leaves the gateway in a partly-dead state. Symptoms range from a ~30s window of
dead DNS to a permanent outage that only full `recover.sh` or `toggle.sh` fixes. The root cause is that
nullexit has **no daemon watching macOS sleep/wake or network-state-change events** the existing DNS
Watcher inside `toggle.sh` only runs in-process and is suspended along with the rest of the user shell on
lid close, so the gateway cannot self-heal after either event.
1077
1078 **Diagnosis:** Two distinct failure modes share the same root gap.
```

```

1079
1080 * **(A) Lid close sleep wake.** `caffeinate -i` (used by `toggle.sh`) blocks *idle* sleep but **does not
    block forced sleep from lid close**. Closing the lid hard-suspends Colima VM + every container + every
    Tailscale-quit-shells-except-tailscaled. On wake the VM clock needs ~5-30s to resync via NTP, during which
    TLS cert validation fails: Cloudflare WARP `162.159.192.1:2408` rejects the handshake gluetun's
    healthcheck (`interval: 2s, retries: 15`) eventually flips unhealthy Docker `unless-stopped` restarts the
    `warp` container (~30s gap). `mDNSResponder`'s in-memory cache still holds pre-sleep entries (stale A
    records for tailnet hosts) so the first dozen DNS queries after wake time out. `tailscaled` on the host
    often keeps its stale DERP relay preference and routes packets through a dead relay for another 30-60s.
1081 * **(B) Wi-Fi roam / loss in elevators, captive portals.** WARP is a UDP tunnel to Cloudflare's anycast edge.
    Carrier NATs and captive-portal networks silently invalidate the UDP binding on roam and on hotspot-bound
    re-association. macOS `networksetup` *should* preserve DNS settings on the same Wi-Fi service across a roam
    , but most captive-portal networks DHCP-replace DNS to the captive-portal resolver **during the captive-
    portal dance itself**, before the user has authenticated so even DNS hijack survives a clean roam and
    quietly breaks on captive-portal handoff.
1082
1083 **Resolution.** A single **launchd LaunchAgent** (`com.nullexit.wake-recovery`) runs a long-running shell
    daemon (`scripts/watcher.sh`) that listens for both event sources and, on each fire, executes `bash recover
    .sh --post-wake` a *lighter-touch* recovery mode that's the opposite of the existing `--nuclear` semantics
    : it keeps the gateway live while still refreshing every stale subsystem.
1084
1085 **Deep dive: Apple Power Management Primer (for the unfamiliar).**
1086 macOS exposes several relevant surfaces; the right primitive depends on what you actually need to detect. None
    of these are obvious and none of them have a standard splash screen in the docs.
1087
1088 * ***caffeinate <flags>** creates I/O Kit power assertions via `IOPMAssertionCreateWithName`. It does NOT
    block forced sleep (lid close, low-battery shutdown, sleep timer firing on a per-set Energy Settings policy
    ). Flags:
1089 * `-i` PreventIdleSleep (the only one `toggle.sh` uses; protects against the OS going idle while the user is
    active but a clamshell close still hard-puts it to sleep)
1090 * `-s` PreventSystemSleep (only honored on AC power; the user may be on battery)
1091 * `-u` DeclareUserActive (resets the idle timer every 30s by default; equivalent to keeping the cursor
    moving)
1092 * `-d` PreventDisplaySleep
1093 * `-m` PreventDiskIdleSleep
1094 * There is NO `-l` (lid-block) flag in any modern macOS. Lid close is a kernel-firmware-level event that
    ignores user-space assertions.
1095 * To prove any of this on live hardware: `pmset -g assertions` lists every active assertion with type /
    process / reason / timeout.
1096 * ***pmset** is the read-side of the same system:
1097 * `pmset -g log | grep -E 'Sleep|Wake|DarkWake'` shows the recent timeline.
1098 * `pmset -g history` is the per-day summary.
1099 * `pmset -g assertions` is the live assertion list (matches `-i -s -d -u` to processes).
1100 * ***launchd does NOT have a native "on-wake" hook. ** Not in any version of macOS as of Sonoma. The canonical
    recipe is `StartCalendarInterval` with a sub-minute cadence and let launchd queue missed intervals for
    replay-on-wake, or use a third-party daemon (Sleepwatcher). For our use case a long-running shell daemon
    listening to the unified log is more responsive than a polling agent.
1101 * ***com.apple.system.powermanagement.* Darwin notifications** (`willSleep`, `didWake`, `didDim`, `didUndim`)
    are the C-level signal. From a shell, they are best reached through the unified log with `log stream --
    predicate` see the watcher implementation.
1102 * ***scutil n.watch** is the canonical CLI for live-following network-state changes. Pairs with `n.add State:/
    Network/Global/IPv4` to get a single tap that fires on **any** IPv4 link/route/DNS/address change across **
    all** interfaces. This is what `scripts/watcher.sh`'s network-change listener uses.
1103 * ***com.apple.networkChange Darwin notification** is the C-level equivalent but has no shell-friendly
    listener; use `scutil` instead.
1104 * ***SCNetworkReachability** is the Objective-C API used by SOCKS5/HTTP clients to detect route changes per
    target. Never a fit for shell scripts.
1105
1106 **The Fix (component-by-component).**
1107
1108 1. ***recover.sh --post-wake** (new flag, **non-destructive** by design). Adding --post-wake to the existing
    nuclear recovery script inverts the semantics. The default mode still tears down Tailscale, resets DNS to
    empty, stops caffeinate, runs `docker compose down`, power-cycles Wi-Fi. The new mode does:
1109 * Skip Tailscale disconnect (we WANT it to keep the mesh connection)
1110 * **Re-hijack** DNS to the gateway Tailscale IP read from .gateway_ip (instead of resetting to empty)
    applies to both ACTIVE_SERVICE and ENO_SERVICE because the system resolver is scoped to en0
1111 * **Refresh** the exit-node preference with `tailscale up --reset --ssh=true --accept-dns=false --exit-node
    =\"$TS_IP\" --exit-node-allow-lan-access=true` (the exact flags `toggle.sh` START uses). This re-asserts
    DERP mapping without dropping the mesh
1112 * Inspect `docker inspect --format '{{.State.Health.Status}}' nullexit-warp-1` and **only** `docker compose
    up -d --force-recreate warp` if gluetun is unhealthy this single targeted recreate nudges the UDP NAT
    binding back to Cloudflare without disturbing Tailscale or AdGuard
1113 * Run the sharingd-reset step (existing fix from `toggle.sh` START path; see 15.11) so AirDrop doesn't freeze
    on the new IP lease
1114 * Verify the gateway with `dig +tcp @127.0.0.1 -p 5354 google.com` (AdGuard via TCP through Colima's SSH
    tunnel) and a 4-second `tailscale ping` to the gateway Tailscale IP, instead of the default mode's direct-
    to-1.1.1.1 check
1115 * Crucially: **skip `sudo -v` ** because the LaunchAgent has no TTY to prompt on; all privileged calls use `
    sudo -n` and lean on /etc/sudoers.d/nullexit NOPASSWD entries (networksetup, dnsmasq, dscacheutil,
    killall, pkill).

```



```

1116 2. **toggle.sh** writes and clears /tmp/nullexit-gateway-active.marker (an ISO-8601 UTC timestamp) at the
    bottom of the START path and the top of the STOP path / cleanup_handler. Two new helpers: write_gateway_active_marker and clear_gateway_active_marker. The watcher uses this file as its **only
    signal** that the gateway is live: if toggle.sh was stopped (or never started), the watcher no-ops. This
    prevents waking-up-only-on-counterfactual events from churning DNS.
1117 3. **scripts/watcher.sh** (long-running daemon, sibling-style source file). Two backgrounded pipelines:
1118 * **Wake listener**: log stream --predicate 'composedMessage CONTAINS[c] "didWake" OR composedMessage
    CONTAINS[c] "Wake from Sleep" OR composedMessage CONTAINS[c] "Waking from" OR composedMessage CONTAINS[c] "
    DarkWake"' --style compact --info` piped through while read; do run_recover "WAKE: ..."; done`. [c]
    makes the predicate case-insensitive (the unified log writes phrases in mixed case across versions).
1119 * **Network listener**: (echo "n.add State:/Network/Global/IPv4"; echo "n.watch"; while true; do sleep
    86400; done) | scutil 2>/dev/null` piped through a case match on n.state* / SCEventUpdate*.
1120 * **run_recover** checks the marker, debounces via /tmp/nullexit-watcher.last-recovery (default 10s,
    configurable via NULLEXIT_DEBOUNCE_SECONDS), and shells out to bash recover.sh --post-wake. The 10s
    debounce prevents a single wake event (which typically emits 2-3 log lines) from triggering multiple
    recoveries.
1121 * trap ... TERM INT` kill 0` cleanly tears down the process group on launchctl bootout` so the sleep
    86400` inside the scutil pipe also dies.
1122 * wait` blocks indefinitely while both pipelines feed it.
1123 4. **launchd/com.nullexit.wake-recovery.plist** (LaunchAgent in the user domain runs as the console user at
    every login without needing sudo).
1124 * Label: com.nullexit.wake-recovery`
1125 * ProgramArguments: ["/bin/bash", "<absolute-path-to-your-nullexit-install>/scripts/watcher.sh"]`
1126 * RunAtLoad: true` starts immediately on launchctl load` (no need to log out and back in)
1127 * KeepAlive: { SuccessfulExit: false, Crashed: false }` **don't** restart on clean SIGTERM exit (so launchctl bootout` doesn't get stuck in a relaunch loop). Also **don't** restart on hard crash: we observed
    that macOS Sonoma's sleep/wake suspend-resume path accumulates orphan watcher.sh PIDs because SIGCONT
    does not reliably clean up the prior instance's listener grandchildren, and a KeepAlive.Crashed=true`-
    driven relaunch actively races with the still-live prior process. The script enforces single-instance on
    its own via a flock`-style PID-file lock, so a relaunch-on-crash would be skipped by the lock and produce
    0 listener coverage until the live instance happened to die. Trade-off: a real crash leaves the user
    without auto-recovery until they run launchctl load -w` manually acceptable because (a) gateway still
    works without auto-recovery, (b) a relaunch wouldn't fix whatever caused the crash, and (c) the gateway-
    recovery itself is observably broken (no DNS, no exit-node) if it does go down.
1128 * ProcessType: Background` hint to App Nap not to suspend us.
1129 * StandardOutPath/StandardErrorPath: /tmp/nullexit-watcher.{out,err}.log` separates launchd-level
    diagnostics from the script's own /tmp/nullexit-watcher.log` (which exec >>>` redirects).
1130
1131 **Install (one-time)** The plist lives in the repo at **launchd/com.nullexit.wake-recovery.plist** for
    version control. The installed (live) copy must land in ~/Library/LaunchAgents/` so launchd picks it up on
    the user session.
1132
1133 ```bash
1134 # Copy the plist to the user-launchd location (run from repo root)
1135 cp ./launchd/com.nullexit.wake-recovery.plist \
1136 ~/Library/LaunchAgents/
1137
1138 # Substitute __NULLEXIT_HOME__ with your local install absolute path.
1139 # The plist uses WorkingDirectory + a relative program arg, so this
1140 # single substitution is the only place it references your machine.
1141 sed -i ' "s|__NULLEXIT_HOME__|$(pwd)|" \
1142 ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
1143
1144 # Validate the XML
1145 plutil -lint ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
1146
1147 # Load + persist this user session + every future login
1148 launchctl load -w ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
1149
1150 # Confirm it actually started
1151 launchctl list | grep nullexit
1152 ```
1153
1154 To **stop** the watcher (without uninstalling):
1155
1156 ```bash
1157 launchctl bootout gui/$UID/com.nullexit.wake-recovery
1158 # or:
1159 launchctl unload -w ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
1160 ```
1161
1162 To **uninstall** completely:
1163
1164 ```bash
1165 launchctl bootout gui/$UID/com.nullexit.wake-recovery 2>/dev/null || true
1166 rm ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
1167 ```
1168
1169 **Why this fix was preferred over alternatives.**
1170

```

```

1171 * **Polling with `StartCalendarInterval`** would have given ~30-60s detection latency per wake. The unified-log
    stream is sub-second.
1172 * **Sleepwatcher** would have covered wake events but not network changes. We'd still need a separate `scutil`
    listener, and introducing a third-party dependency for what amounts to ~150 lines of shell is unjustified.
1173 * **Forcing `caffeinate -l`** doesn't exist and the closest equivalent (`caffeinate -s`) only works on AC power
    . The whole point of the gateway is to stay usable on battery while traveling lid close must NOT silently
    kill the gateway.
1174 * **A kernel extension** is impossible (no entitlement) and out of scope.
1175
1176 **Reproduction recipe.** Test both events in isolation to confirm the fix actually closes the gap.
1177
1178 * ***(A) Lid-only repro**:* with the gateway up, run in a terminal: `pmset displaysleepnow && sleep 30 && pmset
    wake` (without physically closing the lid). Watch the wake wake-fires the watcher post-wake routine runs
    login should still work in <5s after wake. Compare to baseline pre-fix: pre-fix the gateway would be dead
    for 30-90s.
1179 * ***(B) Roam-only repro**:* with the gateway up: `networksetup -setairportpower off && sleep 30 && networksetup
    -setairportpower on` (or simply walk out of Wi-Fi range and back). The captive-portal WILL repave DHCP DNS
    for a few seconds; the watcher fires on the second `State:/Network/Global/IPv4` change (after the Wi-Fi re-
    associates) and the post-wake routine re-hijacks DNS within 2-3s.
1180
1181 **Operative files.**
1182
1183 * `recover.sh` added arg parsing; wrapped destructive sections behind `POST_WAKE`; added re-hijack DNS /
    refresh exit-node / force-recreate-if-unhealthy path.
1184 * `toggle.sh` added `write_gateway_active_marker` / `clear_gateway_active_marker` called at the END of START
    and TOP of STOP / cleanup_handler.
1185 * `scripts/watcher.sh` new long-running daemon.
1186 * `launchd/com.nullexit.wake-recovery.plist` launchd LaunchAgent descriptor.
1187 * `~/Library/LaunchAgents/com.nullexit.wake-recovery.plist` the installed copy (one-time copy).
1188
1189 **Honest assessment (read before testing).** This fix has two asymmetric halves:
1190
1191 * **Network / roam / captive-portal path RELIABLE.** `scutil n.watch` on `State:/Network/Global/IPv4{4,6}`
    catches every Wi-Fi roam, captive-portal DHCP-rebind, hotspot handoff, and VPN change with zero missed
    events. To validate, drop Wi-Fi for 30 s and re-add it; `/tmp/nullexit-watcher.log` will record a `NET:`
    trigger and `recover.sh --post-wake` will run within ~10 s of the network coming back.
1192 * **Lid close / wake path BEST-EFFORT on macOS Sonoma.** Apple *suspends* LaunchAgents process-state during
    forced sleep and *resumes* them on wake; it does NOT re-launch them every wake. As a result: (a) `log
    stream` is a live subscription events emitted during the agent's suppressed window are DROPPED, not
    buffered, so the wake event that prompted the resume is invisible to `log stream` on resume; (b) the "fire
    on startup" guard runs once on `launchctl load -w` and after a `KeepAlive` crash-recovery, NOT on every
    successive wake; (c) `subsystem == "com.apple.powermanagement"` will catch FUTURE emissions (display dim/
    restore, manual sleep/wake) but not the lid-closeopen wake directly.
1193
1194 In practice the lid-wake gap is short, because `toggle.sh`'s existing DNS Watcher already polls every 5 s and
    re-hijacks DNS (so DNS recovers within 5 s of any roam or wake), `tailscaled` self-heals via DERP fallback
    within 3060 s, and `gluetun` reconnects via its healthcheck retry within 30 s. The user-visible pain on lid
    open is ~30 s of wonky DNS, not a permanent outage.
1195
1196 **Test the ROAM path FIRST.** If ROAM works in your testing, the lid-wake gap is acceptable. If lid-wake
    handling is critical, the two clean follow-ups are:
1197 1. **Sleepwatcher** (one canonical install): `brew install sleepwatcher`; drop `recover.sh --post-wake` into
    `~/sleep` and `~/wake` so Sleepwatcher invokes it directly on every wake without the launchd propagation
    problem.
1198 2. **Move the watcher to a LaunchDaemon** (root) and use `IOPMSchedulePowerEvent` via a tiny C wrapper not
    portable to shell.
1199
1200 A truly reliable shell-only solution to "wake events from inside a LaunchAgent" does not exist on macOS Sonoma
    +.
1201
1202 **Empirical lessons from deployment.** Two findings came up while deploying this fix end-to-end; recording so
    the next operator doesn't lose an evening to each.
1203
1204 1. **Bash function-call-before-definition gotcha.** While introducing the gateway-active marker helpers, the
    defensive `clear_gateway_active_marker` call was inserted at the very top of `toggle.sh` (right after `set
    -e`), but the function definition lived near `PID_FILE="/tmp/nullexit-caffeinate.pid"` ~90 lines down. Bash
    parses top-to-bottom and resolves names at first invocation, so the call site exited with code 127 (`
    clear_gateway_active_marker: command not found`). The gateway stayed unreachable from the START path even
    though `bash -n` passed (the parser doesn't catch order-of-resolution errors). Rule: define functions
    BEFORE any block that calls them. Verify with `grep -n 'name{\'` followed by `grep -n 'name\s*$'`
    the latter must appear on a line number AFTER the former.
1205 2. **`networksetup -setairportpower off+on` is not a faithful roam reproduction.** Synthetic Wi-Fi radio-
    cycling (`sudo networksetup -setairportpower Wi-Fi off` for 30 s, then back on) is expected to produce a `
    NET:` trigger in `scutil n.watch` but produced ZERO during testing because `setairportpower` powers the
    radio at the driver level while leaving the network SERVICE in the "up" state from scutil's perspective,
    so no `n.state State:/Network/Global/IPv4` event is generated. A real roam (macOS briefly loses the AP and
    re-associates into a different one) DOES fire the event because the link actually changes.
1206 Better reproduction recipes in priority order:
1207 * Walk between two real SSIDs via `networksetup -switchtolocation` (or actual physical station movement)
    guaranteed to fire the scutil event.

```

```

1208 * Force a DHCP rebind on the same SSID with `sudo ipconfig set en0 DHCP` triggers `State:/Network/Global/
1209 IPv4` to update.
1210 * Trust the existing 30-second DNS Watcher inside `toggle.sh` for the DNS path: it re-hijacks DNS
1211 automatically. The post-wake watcher adds clear value mostly for tailscale exit-node re-assertion and warp
1212 gluetun force-recreate, both of which self-heal within ~30 s anyway.
1213
1214 ##### 15.5.2 July 8, 2026: Network Watcher Event Mismatch, Sudo-in-Background Crash, and Loop-Prevention
1215 **Symptom:**
1216 1. Switching Wi-Fi or waking the Mac from sleep did not trigger a post-wake recovery (`recover.sh --post-wake`)
1217 automatically.
1218 2. The network connection would eventually get completely sinkholed/blocked. If the user tried to restart or
1219 wait, it did not self-heal.
1220 3. During the recovery attempt, checking the public IP on the host would return the unencrypted campus/ISP IP (
1221 starting with `132.`) instead of the encrypted tunnel IP.
1222
1223 **Root Cause:**
1224 - **Bug 1 (Network Watcher Mismatch):** In `scripts/watcher.sh`, the network change listener watched `State:/
1225 Network/Global/IPv4` changes via `scutil n.watch` but matched them against `*.n.state*|*SCEventUpdate*`. On
1226 macOS, the actual output is `changedKey [0] = State:/Network/Global/IPv4`. Because the pattern did not
1227 match, all network switches were silently ignored.
1228 - **Bug 2 (Sudo Caching Background Crash):** When the background WARP Watcher eventually detected the broken
1229 tunnel (after 6 failures/30s), it triggered the default nuclear `recover.sh` (with `POST_WAKE=false`).
1230 Because there was no active TTY in the background context, the `sudo -v` caching check aborted instantly
1231 with a terminal required error. Due to `set -e`, `recover.sh` died immediately before resetting DNS,
1232 disabling the packet-filter killswitch, or stopping containers, leaving the host network completely
1233 sinkholed.
1234 - **Bug 3 (Post-Wake Infinite Loop):** Once the watcher patterns were fixed, `recover.sh --post-wake` triggered
1235 successfully on network changes. However, because the script deleted the default routing entries at the
1236 start, spent ~15 seconds rebuilding 88 DERP relay bypass routes, and re-added the default routes at the end
1237 , this execution time exceeded the watcher's 10-second debounce threshold. Furthermore, the watcher
1238 recorded the `start` timestamp in the debounce file. Consequently, the route changes made by the script
1239 itself triggered the watcher again immediately after completion, trapping the system in an infinite loop of
1240 recovery runs. During this loop, the VPN routes were constantly deleted, resulting in the host leaking
1241 traffic through the real ISP interface (an IP starting with `132.`) for 15s out of every cycle.
1242
1243 **Resolution:**
1244 - **Fixed Watcher Matching:** Updated `scripts/watcher.sh` to match `*changedKey*` and `*State:/Network/Global/
1245 IPv4*` to correctly catch network changes.
1246 - **Bypassed Background Sudo:** Added `--auto`/`--non-interactive` flags to `recover.sh` and added a TTY check
1247 `[ -t 0 ]` to skip interactive `sudo -v` authentication when running headlessly in the background. Updated
1248 the WARP Watcher call in `toggle.sh` to pass `--auto`.
1249 - **Prevented Infinite Loops:** Modified `scripts/watcher.sh` to write the `completion` timestamp of `recover
1250 .sh` to `DEBOUNCE_FILE` (instead of only the start timestamp). This ensures all buffered network config
1251 events generated during route reconstruction are evaluated against the end-of-run timestamp and
1252 successfully debounced, breaking the infinite loop.
1253 - **Centralized & Timestamped Logs:** Redirected `watcher.sh` stdout/stderr from `/tmp/nullexit-watcher.log` to
1254 the repository's main `output.log` and updated `scripts/common.sh`'s `step`, `ok`, `warn`, `fail`, and `
1255 die` helpers to prefix logs with a local `[YYYY-MM-DD HH:MM:SS]` timestamp.
1256
1257 ##### 15.5.3 Incident Post-Mortem: Startup Pre-Flight Check Tailscale Race (July 10, 2026)
1258 **Symptom:** Immediately after startup via `toggle.sh`, the pre-flight connectivity check `[1/3] Gateway
1259 reachable via Tailscale` returned `FAIL`, forcing `toggle.sh` to skip the full exit-node activation and
1260 fall back to SOCKS5/local DNS proxy mode.
1261
1262 **Root Cause:** A timing race condition during startup. In `toggle.sh`, the host joins the mesh using `
1263 tailscale up --reset` (Phase A) right before performing the pre-flight check. Because `tailscale up` resets
1264 and reconfigures the host's `tailscaled` daemon, the local daemon must fetch the latest netmap
1265 asynchronously from the coordination servers. This network map propagation and route establishment takes a
1266 few seconds. Running `tailscale ping` immediately after `tailscale up` returns (with no delay) causes the
1267 check to fail because the local daemon has not yet discovered the gateway node `100.100.21.8`.
1268
1269 **Fix (July 10, 2026):** Upgraded pre-flight Check 1 in both `toggle.sh` and `scripts/toggle-linux.sh` to use a
1270 5-attempt retry loop with a 1-second delay between checks. This allows up to 5 seconds for local tailnet
1271 routing paths to settle, ensuring the gateway is correctly reached and a pong is received before deciding
1272 whether to enable exit-node routing.
1273
1274 ##### 15.5.4 Incident Post-Mortem: Pre-Flight Check False Negatives due to VPN Firewall STUN Blocks (July 10,
1275 2026)
1276 **Symptom:** Immediately after a cold boot or full restart of the gateway, the pre-flight connectivity check
1277 `[1/3] Gateway reachable via Tailscale` failed, causing `toggle.sh` to skip the full exit-node
1278 configuration and fall back to SOCKS5/local DNS proxy mode. However, manually running `tailscale ping
1279 100.100.21.8` immediately after startup succeeded in 1ms.
1280
1281 **Root Cause:** An asynchronous Tailscale handshake delay caused by firewall restrictions. Inside the container
1282 network namespace, the `warp` container (gluetun) implements a strict firewall that blocks all outbound
1283 UDP traffic to the public internet except to the designated WireGuard endpoint. Because of this, the `
1284 tailscale` container's STUN requests to public STUN servers are blocked, causing the local daemon to report
1285 `UDP is blocked, trying HTTPS`. Without UDP STUN capability, Tailscale is forced to fall back to a slower
1286 DERP-assisted handshake (relayed via HTTPS/TCP). This negotiation and direct-path punch-through takes
1287 roughly 30 to 35 seconds to establish on cold boots (after a full `tailscale up --reset`). Since the pre-
1288 flight check only waited 15 seconds (15 attempts with 1-second sleeps), the check timed out and aborted

```

```

before the path completed.
1239
1240 **Fix (July 10, 2026):** Increased the pre-flight ping retry limit from 15 to 45 attempts (45 seconds) in both
`toggle.sh` and `scripts/toggle-linux.sh`. This provides sufficient time for the DERP-assisted handshake to
complete on cold boots, while still passing immediately (in 1-2 seconds) on warm starts or once the path
is established.
1241
1242 #### 15.5.5 Incident Post-Mortem: Docker Image Build DNS Failures on Immediate Restart (July 10, 2026)
1243 **Symptom:** When executing `toggle.sh --restart`, the script successfully stops the gateway but then
immediately fails during startup during `docker compose build` with DNS resolution errors:
1244 `failed to do request: Head "https://registry-1.docker.io/...": dial tcp: lookup registry-1.docker.io on
192.168.5.1:53: no such host`.
1245
1246 **Root Cause:** A race condition between physical link negotiation and virtual machine startup. The STOP path
of the restart command invokes `cleanup_network_state` which power-cycles the host's physical Wi-Fi
interface (`en0`) to flush stale routing states. Immediately after, `toggle.sh` starts the gateway boot
sequence. Because `toggle.sh` did not verify the status of the physical interface, it proceeded to boot
Colima and build Docker containers while `en0` was still negotiating a DHCP lease. Consequently, the host (
and by extension, the VM bridge) had no active internet gateway/DNS path, causing Docker's registry check
to fail.
1247
1248 **Fix (July 10, 2026):**
1249 1. Created a unified `wait_for_dhcp_settle` helper function in `scripts/common.sh` that polls `ipconfig
getpacket` and `ifconfig` until a valid IP and router are assigned. To handle typical macOS Wi-Fi
reassociation and DHCP negotiation latency (which commonly takes 15-20 seconds after a power-cycle), this
loop was configured to poll up to 60 times (30 seconds total).
1250 2. Injected a call to `wait_for_dhcp_settle` at the beginning of the `START` action in `toggle.sh` (right
before checking Colima status) to block container builds until the host's physical network is online.
1251 3. Refactored `recover.sh` to reuse the unified `wait_for_dhcp_settle` function.
1252
1253 #### 15.5.6 Watcher.sh Suppressed Exit Code Bug (Fixed)
1254 **Observation:** The `scripts/watcher.sh` script is a long-running daemon that invokes `recover.sh --post-wake`
when it detects system wake or network roam events. Previously, it contained the following bug:
1255 ```bash
1256 bash "$RECOVER" --post-wake
1257 local exit_code=$?
1258 ...
1259 return $exit_code || true
1260 ```
1261 The `|| true` mistakenly swallowed the actual `$exit_code` returned by `recover.sh`, causing `run_recover()` to
always return `0` (success), masking any underlying failures in the wake recovery process.
1262
1263 **Fix:** The `|| true` statement was removed from the return line:
1264 ```bash
1265 return $exit_code
1266 ```
1267 Since `watcher.sh` explicitly omits `set -e` (to prevent pipe breakages from killing the daemon), removing `||
true` safely allows the underlying exit code to propagate without risking daemon termination. The watcher
daemon was then reloaded (`launchctl kickstart -k`) to pick up the script changes in memory.
1268
1269 #### 15.5.7 Network Readiness Race Condition on `--restart` (Fixed)
1270 **Problem:** When running `./toggle.sh --restart`, the script tears down the gateway and immediately begins the
startup sequence. On systems with Wi-Fi (or any wireless interface), the OS takes a moment to re-associate
with the network after the teardown's DNS/routing changes. If the startup sequence ran before the OS had a
default route, `get_active_service` would return nothing, routing bypass targets would be wrong, and DNS/
WARP setup would silently fail resulting in a partially configured gateway that self-heals only once the
`watcher.sh` re-runs.
1271
1272 **Root cause:** The original code had no gate before the first network-dependent call (`ACTIVE_SERVICE=$(
get_active_service)` at the top of the start branch). This was a pure timing race.
1273
1274 **First fix (Wi-Fi specific):** A polling loop on `ipconfig getifaddr en0` was added to wait for an IPv4
address on `en0` before proceeding. This worked but was brittle it only handled Wi-Fi and would fail for
Ethernet, USB-C adapters, or iPhone USB tethering.
1275
1276 **Generalized fix:** Replaced the `en0` check with `route get default | grep 'gateway:'`. This detects a valid
default gateway on *any* interface, covering all connection types. The loop polls every 2 seconds with a
60-second hard timeout (then proceeds with a warning rather than hanging forever). The output prints the
detected gateway IP so failures are easy to diagnose in logs.
1277
1278 ```bash
1279 # In toggle.sh, before ACTIVE_SERVICE=$(get_active_service)
1280 while true; do
1281     if route get default 2>/dev/null | grep -q 'gateway: '; then
1282         _gw=$(route get default 2>/dev/null | awk '/gateway:/{print $2}')
1283         echo " Network ready (default gateway: $_gw)."
1284         break
1285     fi
1286     ...
1287 done
1288 ```

```

```

1289
1290 **Why not ping? Pinging an IP (e.g. `1.1.1.1`) during restart is unreliable because DNS may still be hijacked
    from the previous session, and the exit node routing may not yet be cleared, causing the ping to loop back
    through the tunnel. `route get default` is a pure local kernel query no packets sent.
1291
1292 #### 15.5.8 `caffeinate -i` Doesn't Survive Real Idle Sleep Mesh Devices Blacked Out on Every Cycle (July 18,
    2026)
1293 **Symptom: User reported mesh devices getting "infuriatingly slow" internet, self-resolving once the Mac
    stopped sleeping. `pmset -g log` showed **four** real `Idle Sleep` events in a 38-minute span (16:50,
    16:54, 16:58, 17:16) while the gateway believed itself protected. Each one suspended [[Colima VM|Colima]] (
    and with it all 5 containers [[toggle.sh]]'s exit-node/WARP/AdGuard/routing-fix/Tor stack), so every mesh
    device using this Mac as its Tailscale exit node lost that exit node outright, then had to ride out [[
    watcher.sh]]'s post-wake recovery (exit-node re-assert, DNS re-hijack, routing flush, container health re-
    verify) plus the [[Bug - Post-Wake Feedback Loop|45s post-recovery settle cooldown]] before things felt
    fast again.
1294
1295 **Root cause: `start_sleep_prevention()` (`toggle.sh` ~L1133) only ran `caffeinate -i`, which blocks *idle*
    system sleep but nothing else and per 15.5.1 was already known to not cover *forced* (lid) sleep either.
    In this session specifically, the gateway had been manually stopped (`toggle.sh` invoked 17:03:59, `phase=
    stop` 17:05:58) which also kills the `caffeinate` PID via `stop_pidfile_daemon`; before that stop and
    during the window the gateway *was* running, the four idle-sleep hits still got through, confirming *-i*
    alone is not a durable guarantee even while "protected."
1296
1297 **Fix: Added *-s* alongside *-i* `caffeinate -i -s &` (`toggle.sh` ~L1133). *-s* blocks sleep outright
    whenever the Mac is on AC power and is a documented no-op on battery, so unplugged runs still sleep
    normally (no unexpected battery drain) while a charging Mac the common case for a machine acting as a
    always-on gateway can no longer idle-sleep out from under the mesh. Does not change the lid-close caveat
    from 15.5.1 (*-s* / -i don't override a forced clamshell sleep); that remains best-effort via `watcher.sh`
    post-wake recovery.
1298
1299 #### 15.5.9 Wi-Fi Auto-Rejoin as an Always-On Listener in `watcher.sh` (July 23, 2026)
1300 **Context: With the WARP Watcher no longer false-killing the gateway on a Wi-Fi drop (15.6.2 point 5 /
    15.12.25), the remaining half of the user's actual problem was still open: *sometimes Wi-Fi disconnects
    this MacBook and if auto-join is off it just sits there without internet." *Pausing prevents the wrong
    action; it does not perform the right one (bring Wi-Fi back).
1301
1302 **Why "remember", not "detect". macOS Darwin 25 redacts the live SSID from every CLI tool (and even the on-
    disk known-networks store is a data-vault denied to root) unless the process holds Location Services see
    15.11.10. The user refused Location outright. So the current network cannot be read by any script. Joining
    a network **by name**, however, is *not* gated: `networksetup -setairportnetwork "$iface" "$ssid"`
    associates using the password already in the login Keychain. The design therefore inverts the problem the
    user remembers their network name(s) once, and we only ever rejoin one of those.
1303
1304 **The helper `scripts/wifi-rejoin.sh` (unsigned).
1305 * `set <SSID>` / `forget <SSID>` / `list` manage the remembered list in `.last_wifi_ssid` (one per line,
    gitignored). Seeded once with the user's primary network name.
1306 * `rejoin` (internal) arms a down-timer (`/tmp/nullexit-wifi-downsince`) and a try-counter on first call;
    holds a **15s grace** window so a same-AP DHCP blip self-heals before we force anything; then attempts with
    **progressive backoff** (`302` capped at 300s, tracked via `/tmp/nullexit-wifi-lastjoin`) and **gives up
    after MAX_TRIES=6 per down-episode (drops `/tmp/nullexit-wifi-gaveup`, logs "reconnect manually") so it
    never hammers a network that can't come up. Loops the remembered list, sends the re-associate request for
    the first one that's accepted. It also powers the radio on first if `networksetup -getairportpower` reports
    it off.
1307 * `reset` (internal) clears all four markers once the link is confirmed back.
1308
1309 **Where the driver lives and why that mattered. The first implementation drove `rejoin` from the WARP
    Watcher (`toggle.sh`) only. That watcher exists **solely while the gateway is up, so a Wi-Fi drop while
    the gateway was down or off left nothing to reconnect the Mac confirmed live when a retest "did nothing"
    because the gateway had shut down and taken its watcher with it (15.12.25, part 3). The fix moved the
    driver into the **always-on** `watcher.sh` LaunchAgent daemon (`com.nullexit.wake-recovery`), which runs
    independently of gateway state, as a 4th backgrounded listener alongside the wake / network-change / tunnel
    -fail listeners:
1310
1311 ```bash
1312 # Listener 4: Wi-Fi uplink keepalive polls the Wi-Fi iface every 15s
1313 phys_ip=$(ipconfig getifaddr "$wifi_iface")
1314 case "$phys_ip" in
1315     '|169.254.*') bash "$WIFI_HELPER" rejoin ;; # no routable IP try a remembered net
1316     *) bash "$WIFI_HELPER" reset ;; # link up clear the down-timer
1317 esac
1318 ```
1319
1320 It is leak-safe for the same reason the WARP-Watcher pause is: it only acts when the interface has **no**
    routable address, so no egress path exists. When the gateway *is* up, the WARP Watcher's pause path calls
    the same helper too; the two coordinate through the shared `/tmp/nullexit-wifi-*` markers (grace + backoff
    gate a second caller out) rather than double-associating, and `reset` on either path is idempotent.
    Verified: with Wi-Fi up the poller stays idle (no markers created); the true end-to-end proof is a manual
    drop test, which is what confirmed Keychain-based re-association actually works. Cross-refs: 15.5.1 (the
    daemon and its other listeners), 15.6.2 (the pause guard), 15.11.10 (SSID redaction), 15.12.25 (the
    originating incident).
1321

```



```

1322 ### 15.6 WARP Tunnel Liveness & Host Leaks
1323
1324 ##### 15.6.1 Host-Side Traffic Leaking Past the Exit Node (with Remote Clients Routing Correctly)
1325 **Symptom.** `toggle.sh` reports success. Remote tailnet clients (e.g. an S24 phone) correctly egress through
Cloudflare WARP and see a Cloudflare IP on `whatismyip.com`. But on the **HOST Mac running the gateway**, `
whatismyip.com` reports the underlying physical ISP's ASN (e.g. `campus_isp` for a university campus Wi-Fi)
. The gateway container itself is healthy; the leak is specifically on the host.
1326
1327 **Root causes.** Three distinct mechanisms can each produce this exact symptom on the host alone, while leaving
remote clients unaffected. They are mutually rankable so the diagnostic picks the active one
deterministically.
1328
1329 * **(A) SOCKS5 fallback lane active.** `toggle.sh`'s three Phase-B pre-flight checks (`tailscale ping` AdGuard
via `dig +tcp` WARP internet, `toggle.sh` ~lines 750-820) sometimes flake during the first minute. When ANY
of the three fails, the script sets `SKIP_EXIT_NODE=true` (toggle.sh:849) and instead activates the SOCKS5
fallback path (toggle.sh:894). The SOCKS5 fallback hijacks DNS to `127.0.0.1` (so AdGuard filtering still
works) but **modern browsers on macOS do NOT honor the system SOCKS5 proxy** they ignore `networksetup -
setsocksfirewallproxy`. Result: DNS gets ad-filtered but actual HTTP/S traffic exits direct through the
campus ISP. This branch is invisible from a remote tailnet client because the client's tunnel is unaffected
.
1330
1331 * **(B) IPv6 leak over campus Wi-Fi.** The Tailscale exit node and WARP tunnel are both IPv4-only (gluetun + `
post-rules.txt` drop all IPv6 forwarded traffic; see 15.12 IPv6 Exit-Node Leak). But many university and
enterprise APs are dual-stack the host happily accepts AAAA DNS responses and sends IPv6 traffic straight
out `en0`. macOS happily encrypts that traffic through Tailscale ONLY if Tailscale has IPv6 advertised;
with an IPv4-only exit node, IPv6 traffic skips Tailscale entirely. This branch is invisible from a phone
because iOS/Android Tailscale apps actively sinkhole IPv6 in their VPN profile, but the macOS host's `
tailscaled` does not.
1332
1333 * **(C) Route-table freeze and `--accept-routes` reset after `tailscaled` wake/toggle.** Running `tailscale up
--reset` clears the exit-node preference and reverts `--accept-routes` back to its default (`false`).
Without explicitly passing `--accept-routes=true`, `tailscaled` will not attempt to install the default
route through the `utun*` tunnel interface even if the exit node preference is reconciled. Additionally,
macOS occasionally freezes and fails to apply route-table changes (see 15.11 Standalone Tailscale Daemon
Freeze). In both cases, `netstat -rn` shows the default route still pointing to the physical Wi-Fi gateway
(e.g. `172.17.0.1`) instead of the Tailscale `utun*` interface, causing host traffic to exit direct while
remote clients route correctly.
1334
1335 **Why remote clients don't see this.** Each remote phone/laptop uses its OWN Tailscale tunnel to reach the
container's `100.x.x.x:443` over the mesh they're end-to-end encrypted regardless of how the host Mac
itself is configured. Only the HOST's own egress sockets (HTTP requests from the host's browser, curl, etc
.) are affected.
1336
1337 **Resolution.** A single script: `scripts/diagnose-host-leak.sh`. It runs the 8 checks, classifies into one of
A / B / C / OK, prints a verdict + a ready-to-run fix command on stdout, AND writes the full report to `
host-leak-diagnostic-<UTC>.txt` so the user can paste the file back instead of scrolling-and-copying from
the terminal. With `--fix`, the matched remediation is applied and the two checks that could change (warp +
ipv6) are re-verified. With `--watch` (or `--watch 30`), it runs the full baseline diagnostic then enters
a continuous loop checking warp/IPv6/default-route every N seconds, alerting only on state changes logs to
`host-leak-watch.log`.
1338
1339 ```bash
1340 # Diagnose only:
1341 bash scripts/diagnose-host-leak.sh
1342
1343 # Diagnose + apply matched fix + re-verify:
1344 bash scripts/diagnose-host-leak.sh --fix
1345 ```
1346
1347 **What it prints.** A 7-section report (`tailscale status`, SOCKS5 state, `cdn-cgi/trace` warp=, IPv6 leak
probe, host default route, last toggle.sh output.log hints, resolved gateway Tailscale IP), followed by a
classification block (`Scenario: A | B | C | OK`) with the exact command(s) the user needs to paste into
their terminal they don't have to figure out which `tailscale up` flags match their environment.
1348
1349 **What it does NOT do.** It does not touch `toggle.sh` or `toggle.sh`'s pre-flight logic, does not modify `.env
` (except `--fix` mode appends `GATEWAY_BYPASS_PING=true` when fixing Scenario A), and does not restart
Docker. It is a read-only diagnostic with an opt-in remediation flag. Safe to run on a healthy gateway the
worst case is a 30-line report that says "OK".
1350
1351 **Why cdn-cgi/trace over whatismyip.com for the verdict.** `whatismyip.com`'s ISP database routinely mislabels
campus IPv6 ranges as residential ISPs (that's why the local ISP's ASN appears even when you're routing
through Cloudflare). `cdn-cgi/trace` is hosted by Cloudflare ITSELF and reports `warp=on|off` plus the
exact edge colo serving the request. It is the only public endpoint that can truthfully confirm whether the
WARP tunnel is in use.
1352
1353 **Common false alarms.**
1354
1355 * **`warp=on` but whatismyip.com still shows the local ISP.** `whatismyip.com`'s ISP-detection database is
unreliable on academic networks. Trust `cdn-cgi/trace`'s `warp=on` over `whatismyip.com`'s ISP string. Use
`https://api.ipify.org` (returns just the IP, no ISP DB lookup) as a third confirmatory check.

```



```

1356 * tailscale status shows the host online but exit node preference is missing.** `tailscale up --reset --
    exit-node=` (empty value) clears any stale preference; `tailscale up --reset --exit-node=<100.x.x.x>` re-
    establishes it. The diagnostic prints the exact command with the resolved TS_IP filled in.
1357 * Both IPv6 leak AND route-freeze flags set simultaneously. Scenario B outranks Scenario C fixing IPv6
    makes IPv4 the only viable egress, after which the route-freeze usually self-resolves within ~30s (or one
    more `toggle.sh` cycle).
1358
1359 **Deep dive: Host Leak Probe Continuous Sub-Second Egress Monitor (`scripts/host-leak-probe.sh`).**
1360
1361 > ** STATUS: REMOVED (historical). `scripts/host-leak-probe.sh` and the `HOST_LEAK_PROBE` env flag no longer
    exist the file is deleted/untracked and nothing launches it (verified 2026-07-15). Its fail-closed role
    was superseded by the **PF kill-switch** (`enable_killswitch` in `common.sh`) plus the in-container **WARP
    Watcher** (15.6.2); the only surviving host-leak tool is the on-demand `scripts/diagnose-host-leak.sh`. The
    text below is kept for design rationale only treat every present-tense claim (auto-launch, PID file, `
    HOST_LEAK_PROBE=false` toggle) as past tense.
1362
1363 *What it is.* `scripts/host-leak-probe.sh` is a lightweight background daemon (~1.4 MB RSS, ~01% CPU) that
    polls `https://www.cloudflare.com/cdn-cgi/trace` every 300ms directly from the host via `curl` not via `
    docker compose exec`. This is a fundamentally different vantage point from the WARP Watcher (15.6.2): the
    WARP Watcher asks the WARP *container* whether it sees `warp=on`; the Host Leak Probe asks what a real
    browser request leaving the host's physical NIC looks like. The two blind spots it covers that the WARP
    Watcher cannot:
1364
1365 1. **Host-side flash-leaks** a Cloudflare anycast edge re-route, a browser reusing a stale pooled connection
    across a tunnel state change, or a split-second routing gap during a Gluetun healthcheck restart (see
    15.12.13 Routing Fix 30-second polling acknowledged 14s window).
1366 2. **Host egress while container is healthy** the container can report `warp=on` while the host's own traffic
    exits direct (the three-scenario leak class documented above in 15.6.1).
1367
1368 *Lifecycle.* Started by `toggle.sh`'s START path (`start_host_leak_probe`) immediately after `
    start_warp_watcher`. Controlled by:
1369
1370 | Signal path | What happens |
1371 |---|---|
1372 | `toggle.sh` STOP | `stop_host_leak_probe()` SIGTERM + PID file cleanup |
1373 | `toggle.sh` `cleanup_handler` (error/INT/TERM/HUP) | Same |
1374 | `recover.sh` (default, non `--post-wake`) | 6d block kill by PID file |
1375 | `recover.sh --post-wake` | Left running the gateway is still live |
1376
1377 PID file: `/tmp/nullexit-host-leak-probe.pid`. Toggle with `HOST_LEAK_PROBE=false` in `.env` to disable at next
    gateway start.
1378
1379 *Output format.* All events are written to `output.log` (repo root) with UTC timestamps. Only state *changes*
    are logged the probe is silent during normal healthy operation.
1380
1381 ```
1382 [09:10:26] LEAK warp=off ip=45.23.1.1 prev_warp=on prev_ip=104.28.246.50
1383 [09:10:26] ROTATE warp=on ip=104.28.248.11 prev_ip=104.28.246.50
1384 [09:10:26] HOST-PROBE failed/timeout (count=1)
1385 ```
1386
1387 curl transport errors (`TLS handshake failed`, `Connection refused`, etc.) are also appended to `output.log`
    via `2>>output.log` nothing is silenced to `/dev/null`.
1388
1389 *Grep cheatsheet.*
1390
1391 ```bash
1392 grep LEAK output.log          # real warp=off events from the host
1393 grep ROTATE output.log        # Cloudflare anycast edge rotations (harmless)
1394 grep 'HOST-PROBE' output.log  # curl failures / timeouts
1395 ```
1396
1397 *Resource budget.* ~1.4 MB RSS, ~0% CPU at rest, ~3 HTTPS requests/second to Cloudflare. Safe to leave running
    for hours. Back off the polling interval (`bash scripts/host-leak-probe.sh 1.0`) if you observe probe-
    failure pile-ups indicating Cloudflare rate-limiting.
1398
1399 *Detection vs prevention.* This script detects leaks; it does not prevent them. A sub-300ms flash-leak can
    still slip between two polls undetected. If `grep LEAK output.log` shows confirmed leak events, the next
    step is a kill-switch (prevention) an `iptables` rule on the host's `en0` that drops non-Tailscale, non-
    local egress whenever the gateway is active. That is a separate engineering task not yet implemented.
1400
1401 ##### 15.6.2 In-Flight WARP Tunnel Liveness Monitor & Statistical Auto-Shutdown
1402 **Why.** nullexit's core promise is that your IP always appears as Cloudflare. If the WARP tunnel silently dies
    due to a UDP NAT timeout, a Cloudflare edge rotation, a Colima VM clock skew causing TLS cert rejection,
    or any of a dozen other failure modes the host silently falls back to egressing through the physical ISP.
    The user sees the gateway as "up" (containers running, Tailscale connected, DNS hijacked) but their real IP
    is exposed. This is the exact class of failure that Ingo Blechschmidt documented in his chilling 2024
    Linux kernel post-mortem: for over two years, LUKS disk encryption on suspend was silently broken because a
    refactored kernel syscall returned success while doing nothing ([source](https://mathstodon.xyz/@iblech
    /116769502749142438)). The lesson: **a security mechanism that fails silently is worse than no mechanism at
    all** it creates a false sense of safety that prevents the user from taking corrective action.

```

```

1403
1404 **Design principle.** The WARP Watcher (`start_warp_watcher()` in `toggle.sh`) is a background daemon that
polls `cdn-cgi/trace` every 30 seconds and applies a **statistically calibrated threshold** before
triggering any safety response. This threshold distinguishes transient network blips (which self-heal
within seconds) from genuine outages (which require intervention). A single `warp=off` reading is
meaningless. Docker might be slow, the network might be congested, a container healthcheck might be
restarting. But 6 consecutive off readings (30 continuous seconds of downtime) has vanishingly low
probability of being a false positive: even at an unrealistically high 10% false-positive rate per poll,  $P$ 
(6 consecutive) =  $0.1 \cdot 0.0001\%$ . In practice, the per-poll false-positive rate is far lower than 10%, making
the real probability astronomically small.
1405
1406 **What it does.**
1407
1408 1. **Always logs.** Every state transition (onoff, offon) is timestamped to `output.log`. The user can `grep`
WARP output.log to audit the tunnel's entire lifetime. This is the "never fail silently" mandate.
1409
1410 2. **Tracks consecutive failures.** A counter increments on each `off` reading and resets to 0 on any `on`
reading. This ensures the watcher never fires on intermittent flapping.
1411
1412 3. **Auto-shuts down at threshold.** When the counter reaches `WARP_FAIL_THRESHOLD` (default 6, configurable in
`.env`), the watcher declares a statistically significant outage and runs `recover.sh` the nuclear
recovery script that disconnects Tailscale, stops Docker containers, resets DNS to `1.1.1.1`, flushes
routes, and power-cycles Wi-Fi. This instantly reverts the host to normal (un-encrypted) internet,
guaranteeing no traffic leaks through a dead tunnel while the user thinks they're protected. A trigger file
at `/tmp/nullexit-warp-shutdown-triggered` prevents double-firing.
1413
1414 4. **Exits cleanly.** After shutdown, the watcher exits (the gateway is dead, there's nothing left to monitor).
`stop_warp_watcher()` is also called by `toggle.sh`'s STOP path and `cleanup_handler` to terminate the
daemon cleanly during normal teardown.
1415
1416 5. **Pauses instead of killing when the *physical uplink* is down (added 2026-07-23).** The watcher cannot tell
"WARP tunnel failed" apart from "the Mac lost Wi-Fi" both read `warp=off`, because with no internet there
is no tunnel either. Before this guard, a plain Wi-Fi drop counted 6 `off` polls and nuked the whole
gateway (the 2026-07-22 outage, 15.5.9). Now, before counting a `warp=off` as a failure, the loop resolves
the physical interface (`networksetup -listallhardwareports`, same as `wait_for_dhcp_settle`) and checks `ipconfig getifaddr $phys_iface`. If it's empty or `169.254.*` (self-assigned) there is **no routable uplink
**, so it logs `WARP watcher PAUSED`, resets `consec_off=0`, calls `scripts/wifi-rejoin.sh rejoin`, sleeps,
and `continue`s it does **not** run `recover.sh`. When the interface gets an IP again it logs `RESUMED`
and calls `wifi-rejoin.sh reset`. This is **leak-safe**: the pause only ever fires when the interface has
no routable address, i.e. there is no egress path at all and therefore nothing that `can` leak. The kill-
switch semantics are unchanged for the case that actually matters "link up **and** `warp=off`" still trips
the nuclear shutdown. Note the pause covers the gateway-up case; the always-on `watcher.sh` (15.5.9) is
what drives rejoin when the gateway is down.
1417
1418 **Configuration.** Set `WARP_FAIL_THRESHOLD=3` in `.env` for a more aggressive 15s timeout, or `WARP_FAIL_THRESHOLD=12` for a conservative 60s window. The variable is parsed from `.env` using the same `grep`
pattern as `GATEWAY_MSS` and `GATEWAY_BYPASS_PING`.
1419
1420 **Log sample.** After a real outage (or a forced test via `docker compose pause warp`):
1421
1422 ```
1423 [2026-07-03T21:15:17Z] WARP DOWN cdn-cgi/trace reports warp=off
1424 [2026-07-03T21:15:47Z] WARP SHUTDOWN 6 consecutive failures (threshold=6). Running recover.sh to kill the
gateway and restore normal internet. Your IP is NO LONGER Cloudflare.
1425 [2026-07-03T21:15:52Z] WARP SHUTDOWN recover.sh completed, watcher exiting. Your IP is now your real ISP IP.
1426 ```
1427
1428 Or, if WARP self-recovers before the threshold:
1429
1430 ```
1431 [2026-07-03T21:15:17Z] WARP DOWN cdn-cgi/trace reports warp=off
1432 [2026-07-03T21:15:32Z] WARP RECOVERED warp=on after 3 consecutive off readings (threshold was 6)
1433 ```
1434
1435 **Relationship to other monitors.** The WARP Watcher is the **innermost safety layer** it runs as a child of `toggle.sh` and only while the gateway is active. It complements (does not replace) the `scripts/watcher.sh`
daemon (15.5.1, which handles sleep/wake and Wi-Fi roam) and `scripts/diagnose-host-leak.sh` (15.6.1,
which is a manual diagnostic tool). The WARP Watcher is the only component that actively verifies end-to-
end tunnel liveness and takes autonomous action when it fails.
1436
1437 #### 15.6.3 Incident: The Overnight Silent IP Leak (July 2026)
1438 **The Problem:** The host leaked traffic over its raw, unencrypted `eth0` IP continuously for roughly 8 hours,
completely bypassing the VPN. The `toggle.sh` state and the container liveness checks all reported the
gateway as healthy and active.
1439
1440 **The Investigation:**
1441 1. **Sleep/Wake Checks:** We initially suspected macOS sleep cycles broke the routing table. A check of `pmset`
`-g log` confirmed the laptop literally never slept (0 sleep events recorded since boot), ruling out all
sleep/wake code paths.
1442 2. **Keepalive Expiry:** We discovered `docker-compose.yml` was missing `WIREGUARD_PERSISTENT_KEEPLIVE_INTERVAL`. Without a keepalive, standard router NAT tables (which typically

```

```

garbage collect UDP after 30 to 120 seconds of silence) quietly expired the WireGuard port mapping
overnight.
1443 3. **Container State Masking:** Because WireGuard is stateless, the `warp` container didn't immediately crash.
It stayed "Up", which meant Docker's healthchecks and our `toggle.sh` liveness checks never noticed the
tunnel inside it was totally dead.
1444
1445 **The Solution:**
1446 1. Added `WIREGUARD_PERSISTENT_KEEPLIVE_INTERVAL=25s` (the WireGuard standard to beat standard 30s NAT
timeouts) to keep the UDP tunnel permanently pinned open through the router.
1447 2. Injected a **Fail-Closed Kill-Switch** into `routing-fix.sh`: A 30-second loop now actively verifies tunnel
health by running `curl` over `tun0` to Cloudflare's trace endpoint. If it fails 3 consecutive times, it
immediately rewrites the default route to `blackhole`, severing all traffic instead of letting it leak
through `eth0`.
1448 3. Added a listener in `watcher.sh` that detects this fail-closed event and instantly pushes a native macOS UI
notification to the desktop, alerting the user to manually intervene.
1449
1450 > [!NOTE]
1451 > **Why dual failure systems?**
1452 > The system now relies on two independent failure handlers that cover each other's blind spots:
1453 > 1. **Automated Self-Healing (WARP Watcher + `recover.sh`):** Triggered when the `warp` container logs `warp=
off`. This happens when the server actively kicks the client. Because the container *knows* it is
disconnected, it triggers `recover.sh` to safely reboot Docker and reconnect.
1454 > 2. **Fail-Closed Kill-Switch (`routing-fix.sh` + `watcher.sh`):** Triggered when a silent network failure
occurs (e.g., NAT timeouts). The container incorrectly believes it is connected (`warp=on`), so auto-
recovery never fires. Instead of auto-recovering (which risks falling back to raw Wi-Fi mid-process while
the user isn't looking), this kill-switch actively blackholes the internet and forces a manual intervention
via a desktop notification.
1455
1456 ### 15.7 Tailscale P2P, SNAT & Control Plane
1457
1458 #### 15.7.1 macOS SNAT Endpoint Poisoning & The P2P Blackhole (July 10, 2026)
1459 **Symptom:** When a Tailscale client (like an S24) connected to the exact same Wi-Fi network as the macOS
gateway, it completely lost internet connectivity. However, if the S24 connected to a Windows PC's hotspot
on the same network, the tunnel worked flawlessly.
1460
1461 **Root Cause:** Claude's critique correctly dismantled the port collision theory: the gateway daemon wasn't
failing to bind, and macOS wasn't load-balancing ports. The real culprit was **macOS SNAT Source Port
Mangling poisoning Tailscale's path discovery**.
1462
1463 Here is the exact step-by-step of the "infinite loop" blackhole:
1464 1. **The P2P Handshake:** The S24 and the Mac are on the same Wi-Fi (`192.168.1.x`). The S24 discovers the Mac's
local IP and sends a UDP hole-punch packet to `192.168.1.5:41641`. Colima successfully forwards this to
the gateway container.
1465 2. **The Bypass Rule:** The gateway replies. Because `routing-fix.sh` forces all Tailscale UDP traffic out `
eth0` to bypass WARP, the reply goes to the macOS host.
1466 3. **macOS SNAT Mangling:** macOS routes the reply out `en0` to the S24. Because the packet crosses from the
Colima VM bridge to the Wi-Fi interface, macOS applies Source NAT (SNAT). Critically, macOS **randomizes
the source port** (e.g., changes it from `41641` to `58291`).
1467 4. **Endpoint Poisoning:** The S24 receives the authenticated Tailscale packet from `192.168.1.5:58291`.
Tailscale's `magicsock` is designed to handle NAT dynamically, so it assumes the Mac has roamed to port
`58291`! The S24 updates its endpoint and starts sending all its encrypted internet traffic to
`192.168.1.5:58291`.
1468 5. **The Blackhole:** macOS has no port forward for `58291`. It drops 100% of the S24's outbound data packets.
The connection is completely dead.
1469
1470 **Why did the Hotspot work?** When the S24 connected to the PC hotspot, it was behind the PC's NAT. The S24
sent the packet, PC NATted it, and the Mac replied (mangled to `58291`). When the reply hit the PC, the PC's
strict NAT dropped it because the source port (`58291`) didn't match the destination port it originally
sent to (`41641`). Because the mangled packet was dropped, the S24 never poisoned its endpoint! It safely
gave up on P2P, fell back to the 100% reliable TCP DERP relay, and the internet worked perfectly.
1471
1472 **Fix & Resolution (July 10, 2026):**
1473
1474 **Phase 1 Port disambiguation:** Changed the gateway container's Tailscale listen port from the default
`41641` to `41642` across `docker-compose.yml`, `post-rules.txt`, and `routing-fix.sh`. This prevents any
bind conflict with the macOS host's native Tailscale daemon.
1475
1476 **Phase 2 RFC1918 DROP rules:** Updated `routing-fix.sh`'s `add_tailscale_p2p_bypass()` to explicitly drop all
outgoing Tailscale UDP packets destined for private subnets (`192.168.0.0/16`, `10.0.0.0/8`,
`172.16.0.0/12`) when `TAILSCALE_ALLOW_LAN_P2P=false`. This surgically kills the local P2P hole-punch
attempt *before* it can trigger the macOS gvproxy SNAT mangling. The S24 receives a clean failure and
immediately locks onto the public DERP relay with 0% packet loss.
1477
1478 **Phase 3 Automatic network detection:** Because `TAILSCALE_ALLOW_LAN_P2P=false` breaks P2P on trusted
hotspots that genuinely don't have AP isolation, a `.env` toggle and a full auto-detection system were
implemented:
1479
1480 - **`.env` toggle:** `TAILSCALE_ALLOW_LAN_P2P=true/false` static fallback, used only if auto-detection cannot
run.
1481 - **`watcher.sh` auto-detection (`detect_lan_p2p_mode`):** On every network change, `watcher.sh` runs two
checks:

```

1482 1. ****WPA2-Enterprise detection:**** `airport -I` is used to read the current Wi-Fi `link auth` field. If the
auth type does not contain `psk` (e.g., it is plain `wpa2` = 802.1x), the network is classified as
enterprise and P2P is forced off. *(Note: the `airport` binary was removed in macOS Sonoma 14.4 the
detection now uses `system\_profiler SPAirPortDataType`; see 15.11 for the full migration.)*

1483 2. ****AP Isolation probe:**** For WPA2-Personal networks, a short `ping` is sent to the default gateway. If the
gateway responds (it always should on a real home/hotspot network), P2P is allowed. If not, AP isolation is
inferred and P2P is forced off.

1484 - The result (`true` or `false`) is written to `.lan\_p2p\_detected` in the repo root.

1485 - ****`routing-fix.sh` dynamic reading:**** `add\_tailscale\_p2p\_bypass()` re-reads `.lan\_p2p\_detected` on every 30-
second loop iteration and atomically adds or removes the RFC1918 DROP rules. ****No container restart is
needed when switching networks.****

1486

1487 ****Known Unknowns (Pending Verification):****

1488 - The gvpnproxy SNAT source port mangling theory is mechanistically sound and backed by observed behavior (
gvpnproxy's UDP proxy changelog has documented edge cases in this exact code path), but has not yet been
confirmed via `tcpdump -i en0 udp portrange 40000-60000` while triggering a P2P handshake from the phone. A
packet capture cross-referenced with `tailscale ping` endpoint reports on the phone side would confirm the
theory definitively.

1489 - Claude's recommendation: run `tcpdump -i en0 udp port 41642 or portrange 40000-60000` on the Mac while
triggering the handshake from the phone to see whether the reply leaves with a different source port than
`41642`.

1490 - ****Client-Side VPN Cycling (Roaming Recovery):**** An observation (July 10, 2026) suggests that when a hung DNS
probe state is triggered by roaming between Access Points (APs) on a WPA2-Enterprise network, simply
toggling the VPN connection (Tailscale) off and on locally on the client phone immediately resolves the
issue. You do not need to cycle the phone's physical Wi-Fi. This reinforces the theory that roaming on
Enterprise networks leaves a stale/poisoned P2P endpoint in the phone's Tailscale magicsock state, which
gets instantly flushed upon a local VPN restart. (Status: Possible but not formally verified).

1491

1492 **#### 15.7.2 July 6, 2026: Packet Filter (pfctl) Defaulting Local LAN Rules to TCP-Only**

1493 ****Symptom:**** Devices on the exact same local Wi-Fi network as the gateway were experiencing ~500ms latency to
the gateway instead of the expected ~80ms.

1494

1495 ****Root Cause:**** When writing the `pf.conf` rules to whitelist local LAN traffic (`pass out quick on \$ext_if to
{ 192.168.0.0/16, ... }`), the rule omitted an explicit protocol. The macOS `pfctl` compiler implicitly
applies state tracking to all `pass` rules. However, because state tracking (`keep state`) natively
evaluates TCP sessions, `pfctl` automatically appended the `flags S/SA` modifier to the compiled rule in
the kernel. This silently restricted the whitelist to ****TCP only****, causing all local UDP traffic to be
dropped by the default-deny kill-switch. Since Tailscale relies on UDP port 41641 for direct P2P
connections, the local devices were unable to hole-punch and were forced to fall back to routing traffic
through a remote Tailscale DERP relay, massively inflating latency.

1496

1497 ****Resolution:**** Split the single implicit rule into explicit `proto tcp`, `proto udp`, and `proto icmp` rules
in `scripts/pf.conf` to guarantee the macOS compiler allows all three core transport protocols on the local
subnet without mutating the rule into a TCP-only lock.

1498

1499 **#### 15.7.3 Incident Post-Mortem: PF Kill-Switch Bricks Host Internet in SOCKS5 Fallback Mode (July 10, 2026)**

1500 ****Symptom:**** When the pre-flight checks fail (e.g. because host Tailscale was unauthenticated/logged out), the
script falls back to SOCKS5 proxy mode but the host immediately loses all internet connectivity and
tailscaled reports `You are logged out... connect: no route to host`. Even running `sudo tailscale up` to
log in fails because the connection to the control plane times out.

1501

1502 ****Root Cause:**** An architectural mismatch. The macOS Packet Filter (PF) Kill-Switch works at the IP level: it
blocks all outbound traffic on the physical interface (`en0`) except to explicitly whitelisted VPN
endpoints, expecting all other traffic to go through the virtual VPN interface (`utun*). In SOCKS5
fallback mode, the exit-node (`utun*) is NOT enabled; instead, only specific applications (like browsers)
use the localhost SOCKS5 proxy, while all other host traffic (including the `tailscaled` daemon's UDP
packets) continues to go through `en0`. Because the script still enabled the PF Kill-Switch during SOCKS5
fallback, it blocked all non-SOCKS5 traffic on `en0`, completely bricking the host's internet and locking
the Tailscale daemon out of the control plane.

1503

1504 ****Fix (July 10, 2026):**** Modified `toggle.sh` to remove the `enable\_killswitch` call from the SOCKS5 fallback
path. The firewall kill-switch is now strictly reserved for exit-node VPN mode, ensuring SOCKS5 fallback
leaves the host's direct internet route open for system daemons to authenticate.

1505

1506 **#### 15.7.4 Incident Post-Mortem: Host Tailscale Logouts due to Aggressive reset Flags (July 10, 2026)**

1507 ****Symptom:**** Every time the user runs `toggle.sh` or `recover.sh` which disconnects/reconnects the host client,
the host client randomly gets logged out, forcing them to re-run `sudo tailscale up` to authenticate.

1508

1509 ****Root Cause:**** An overly aggressive configuration reset. The host-side `tailscale up` calls (used in
`disconnect\_tailscale\_host` and the startup/enable exit node paths) all passed the `--reset` flag. On macOS,
`--reset` resets the entire configuration state and authentication token cache of the daemon. Since the
script does not pass a `TS\_AUTHKEY` to the host's commands (which are meant for the user's private
interactive client), the `--reset` flag effectively logged the user out of their tailnet, requiring
interactive re-login.

1510

1511 ****Fix (July 10, 2026):**** Removed the `--reset` flag from all host-side `tailscale up` invocations in `toggle.sh`
and `scripts/common.sh`. This ensures that exit-node state transitions (enabling/disabling) are handled
safely while preserving the host client's authenticated session.

1512

1513 **#### 15.7.5 Incident Post-Mortem: macOS Tailscale Flag Requirement Abort (July 10, 2026)**

```

1514 **Symptom:** When the `toggle.sh` stop trap or the roaming watcher triggered, the gateway failed to shut down
      or disconnect properly. The `output.log` showed the error: `Error: changing settings via 'tailscale up'
      requires mentioning all non-default flags. To proceed, either re-run your command with --reset...`
1515
1516 **Root Cause:** In a previous attempt to stop host-side logouts (15.7.4), we removed the `--reset` flag from
      all `tailscale up` invocations. However, if the Tailscale daemon on macOS is already running with non-
      default flags (like `--ssh` or `--accept-routes`), invoking `tailscale up` with only a subset of flags (
      like `--exit-node=`) triggers a strict safety check in the Tailscale CLI that aborts the command completely
      . Because the command aborted, exit-node routing remained stuck.
1517
1518 **Fix (July 10, 2026):** Migrated all specific preference changes in `toggle.sh` and `scripts/common.sh` from `
      tailscale up` to a two-step sequence:
1519 1. `tailscale up` (with no flags) to safely ensure the interface is brought up using the current authenticated
      state.
1520 2. `tailscale set --exit-node=...` to modify the specific target preference cleanly without triggering the
      missing flags error or requiring `--reset`.
1521
1522 ### 15.8 Docker / Colima Lifecycle
1523
1524 #### 15.8.1 Colima VM Memory / Swap Thrashing Latency
1525 **Symptom:** After running nullexit flawlessly for over 24 hours straight to test stability, the 0.5GB VM RAM
      configuration began experiencing severe network latency on the Tailscale mesh later in the day.
1526
1527 **Root Cause:** Services (like AdGuard, Tailscale, Docker logs) slowly accumulate cache and state over extended
      periods. Under the tight 512MB physical RAM limit, this slow creep forced the Linux kernel to aggressively
      swap memory to the 512MB SSD swap file. The constant disk I/O thrashing blocked process execution, causing
      DNS resolutions and packet forwarding to delay significantly (making the internet feel "very slow").
1528
1529 **Proof (Empirical Observation):** While in this OOM-thrashing state, direct DNS queries through the Tailscale
      mesh (`dig @100.100.21.8 google.com`) would intermittently hang or take several seconds to resolve. After
      restarting with the 600MB configuration, the exact same query immediately dropped to a lightning-fast **41
      ms** response time. (Note: Querying the mapped host port via `127.0.0.1:5354` will always time out due to
      Gluetun's strict leak-prevention firewall blocking non-VPN ingress traffic).
1530
1531 **Fix:** Increased VM base physical RAM to 600MB (`--memory 0.6`) and reduced the swap file to 400MB. This
      provides enough native RAM headroom for long-running state caches without forcing heavy I/O swapping.
      Subdomain deduplication still reduces blocklists by ~60%. Memory profiles (`light`/`medium`/`heavy`) let
      users trade off coverage for memory.
1532
1533 #### 15.8.2 Missing iptables in routing-fix Container
1534 **Root Cause:** `alpine:3.20` does not have iptables installed. Commands failed silently with `2>/dev/null ||
      true`.
1535
1536 **Fix:** Updated `docker-compose.yml` to `apk add --no-cache iptables iproute2` before `scripts/routing-fix.sh
     `.
1537
1538 #### 15.8.3 DOCKER_GW Variable Parsing Bug
1539 **Root Cause:** `ip route show default` printed two lines; `awk '{print $3}'` picked up both, causing route
      commands to fail.
1540
1541 **Fix:** Added `head -1` to the parsing chain.
1542
1543 #### 15.8.4 Hard Reboots Leave Stale Docker Socket in Colima VM
1544 **Problem:** If the macOS host machine is subjected to a hard reboot, sudden power loss, or a kernel panic, the
      Colima VM running in the background is abruptly killed. Upon next boot, running `toggle.sh` resulted in
      the contradictory output:
1545 `Colima is already running.` followed by `Cannot connect to the Docker daemon at unix:///./colima/default/
      docker.sock.`
1546
1547 **Root Cause:** The hard crash prevents the inner Docker daemon from executing its graceful shutdown routines.
      It leaves behind a corrupted `docker.sock` file inside the VM. On boot, the outer VM spins up (hence "
      already running"), but the inner Docker daemon sees the stale socket, assumes another instance is running,
      and crashes.
1548
1549 **Fix:** Added an Auto-Healing routine directly into `toggle.sh` (Step 4b). The script now explicitly tests the
      Docker daemon with `docker ps`. If the daemon is unresponsive despite Colima reporting as active, it
      assumes a stale socket and automatically runs `colima restart` in the background to cleanly rebuild the VM
      and socket before proceeding.
1550
1551 #### 15.8.5 Cloudflare WARP 24-Second Startup Delay (UDP Session Rate-Limiting)
1552 **Context:** When running `toggle.sh` to turn the gateway ON, `docker compose up -d` often hangs for exactly
      24-25 seconds waiting for the `warp` container to become `healthy`. Users might mistakenly blame the Go `
      rule-compiler` processing 400k+ rules for this delay, as Docker prints `rule-compiler Exited 24.3s` at the
      end of the wait.
1553 **Root Cause:** The `rule-compiler` actually finishes its work in <2 seconds. The 24-second blockage is
      entirely due to Cloudflare WARP's edge server rate-limiting. WireGuard is a stateless protocol with no "
      disconnect" packet. When the toggle is turned OFF, the old UDP session state remains active on Cloudflare's
      backend for 20-30 seconds. When the toggle is instantly turned back ON, Docker starts a new WireGuard
      connection from a new ephemeral UDP source port but uses the same static cryptographic key (from `.env
      `). Cloudflare detects this as a potential replay attack or key-sharing abuse, and intentionally drops all
      packets (rate-limits) for the new connection until the old session's ghost state fully expires (~20-25

```


seconds).

1554 ****Result:**** Gluetun's healthcheck fails repeatedly every 2 seconds until Cloudflare lifts the penalty box, at which point traffic flows and Docker unblocks the rest of the startup sequence. This is a known, expected behavior of reusing static keys rapidly on Cloudflare's network and cannot be bypassed.

1555

1556 **#### 15.8.6 July 4, 2026: Multi-stage Docker Builds for Go Ports & Teardown Hang Fix**

1557 ****Symptom:**** The Python implementations of `logger.py` and `sync-rules.py` were slow and required a heavy python runtime inside Alpine containers. Also, `routing-fix` container teardown was hanging for 30s during `toggle.sh` shutdown.

1558 ****Root Cause:****

1559 - Python is slower than Go and installing it dynamically via `apk add` added overhead and complexity to the containers.

1560 - Docker containers ignore `SIGTERM` if the init process doesn't handle it, causing a full 30s timeout on shutdown.

1561 ****Resolution:****

1562 - Ported `logger` and `sync-rules` to Go using Goroutines for massive concurrent speedups.

1563 - Implemented ****Multi-stage Docker builds**** (using `golang:1.22-alpine` as builder) to compile the static binaries inside Docker. The final containers are raw Alpine with zero dependencies.

1564 - Added `--build` to `docker compose up -d` in `toggle.sh` to ensure updates are consistently applied on boot.

1565 - Added `stop\_grace\_period: 1s` to `routing-fix` in `docker-compose.yml` which eliminates the 30-second teardown hang instantly.

1566 - Implemented a cryptographic integrity checker (`scripts/crypto.sh`) using HMAC-SHA256 and `NULLEXIT\_SEED` in `.env` to prevent tampering of bash scripts. (See also 15.9.4.)

1567

1568 **#### 15.8.7 Colima `start` Post-Provision Hang (~2 min) & the Docker-Readiness Poll (July 14, 2026)**

1569 ****Symptom:**** `toggle.sh` took ~197s of wall-clock time to bring the gateway up, and almost the entire delay lived in a single step: `colima start`. Timestamps in `output.log` showed the VM reaching `READY` and creating the `colima` Docker context in ~8-12 seconds, after which `colima start` simply *did not return* for ~110 more seconds until the `run\_with\_timeout 120` watchdog `SIGKILL`ed it. The rest of the boot (containers, routing) then proceeded normally.

1570

1571 ****Root Cause:**** A colima-internal hang that occurs *after* `Successfully created context colima` is printed. It is ****mode-independent****. It was initially misattributed to `--network-address` / `--network-mode bridged` needing the `socket\_vmnet` helper, but a controlled test with plain user-mode networking (`colima start --memory 0.6 --vm-type vz`, no `--network-address`) reproduced the identical ~120s hang (04:40:34 04:42:34, watchdog-killed) while Docker was fully usable the entire time. Crucially, killing the hung `colima start` process does ****not**** stop the VM and does ****not**** corrupt state `colima status` correctly reports `running` afterwards and `docker` keeps working via the `colima` context. The ~110s was therefore pure dead time waiting on a CLI that had already done its real work.

1572

1573 ****Fix:**** `scripts/common.sh` now provides `colima\_start\_until\_ready()`. It launches `colima start "\$@"` in the background and polls `docker --context colima info` every 3s. The moment Docker answers (~12s in practice) it stops waiting, reaps the (usually still-hung) starter with `kill`, runs `docker context use colima` to guarantee the default context is set, and returns 0. If Docker never answers within a 90s cap it returns non-zero and `toggle.sh` continues to the existing Step 4b `docker ps` auto-heal (15.8.4). Both Colima start call sites in `toggle.sh` the LAN-P2P bridged/shared path and the user-mode fast path now go through this helper instead of `run\_with\_timeout 120 colima start`.

1574

1575 ****Result:**** The Colima phase dropped from ~120s to ~12s and total `toggle.sh --restart` wall time from ~197s to ~106s, with the double-tunnel and direct hostcontainer P2P confirmed intact after the change. The key insight: treat "Docker is reachable" not "the `colima start` CLI returned" as the definition of *started*.

1576

1577 **#### 15.8.8 Toggle-On Startup Optimizations: Rule-Compiler Off the Critical Path (July 14, 2026)**

1578 ****Context:**** A genuine cold toggle-on (VM bounced for RAM, WARP down) measured ****95s****. Profiled per-phase with the now-fixed `DEBUG\_TRACE` (15.11.8-A): Colima cold start ****19s**** (near floor), `docker compose up` ****45s****, tailscale connect+poll ****21s****, final checks ****7s****.

1579

1580 ****Two wrong hypotheses first (documented so they aren't re-tried):**** (1) that BuildKit `--build` was the ~30s cost it wasn't; every layer was already `CACHED`, so gating `--build` saved only ~2s. (2) that `docker compose exec` was slow in the poll loop it isn't (0.18s steady-state vs 0.06s for `docker exec`); the ~5s/iteration was `tailscale status` ****blocking during cold connect**** and hitting the `run\_with\_timeout 5` cap.

1581

1582 ****Real bottleneck (evidence-backed):**** the 45s `compose up` was ****not**** WARP. On a genuine cold start WARP goes healthy fast (tor starts right with it no 15.8.5 same-key penalty; that only bites rapid offon). The long pole is the ****rule-compiler re-deduping ~340k rules inside the 600MB VM (~28s), every toggle**** `adguardhome` started 28s after warp because it waited on `rule-compiler: service\_completed\_successfully`, even though all 16 remote lists loaded from cache unchanged.

1583

1584 ****Fix hash-gated, off the critical path (kept in-container):****

1585 - `rule-compiler` is now a ****compile-profiled**** service, excluded from `docker compose up`. `adguardhome` and `routing-fix` no longer `depends\_on` it they boot instantly on the `compiled\_rules.txt` / `ip\_blocklist.ipset` already in `./adguard/work`.

1586 - `toggle.sh` runs it on-demand via `docker compose run --build --rm rule-compiler`, ****gated on `compute\_rules\_hash`**** = sha256 of `blacklist.txt` + `white\_list.txt` (the lists the user controls). Recompile only when that hash changes, when an artifact is missing, or when the compiled file is older than `RULES\_MAX\_AGE\_DAYS` (default 7, so the remote blocklists still refresh occasionally). A compile failure is ****non-fatal**** we keep the previous compiled rules rather than bricking the tunnel (ad-block the kill-switch).

1587 - Also trimmed the tailscale-connect poll timeouts (status `5s3s`, `ip -4` `10s5s`) to reduce overshoot.

1588

1589 ****Why NOT run the compiler natively on the Mac host (the tempting idea):**** rejected on purpose. There is no Go on the host, so "native" means either `brew install go` (a host toolchain dependency that drifts the project from *portable+secure-on-any-device* toward *faster-on-mine*) or committing a prebuilt arch-specific **binary blob** you'd then have to trust/sign. Worse, it would make the **host** write into the `./adguard/work` bind-mount violating the deliberate *single allowed writer is the rule-compiler container* invariant (hostcontainer UID/perms mismatches corrupt AdGuard's BoltDB store). Staying in-container preserves both properties; the gate delivers the speed.

1590

1591 ****Result:**** unchanged toggle `compose up` **45s 9s**, total `--restart` **~95106s 63s**, with AdGuard still serving the full 342,519 rules (it reuses the existing compiled file) and the double-tunnel + direct hostcontainer P2P intact. Editing `black\_list.txt`/`white\_list.txt` flips the hash one-off recompile new rules take effect. Both lists were added to the `crypto.sh` signed set (they're security inputs), so editing them requires a re-sign which is the same moment you'd want the recompile. Local state files `build\_hash` / `rules\_hash` are gitignored.

1592

1593 **### 15.9 Permissions, Crypto & Security Hardening**

1594

1595 **#### 15.9.1 Sudo Credential Caching Removed**

1596 ****Problem:**** The script used `sudo -v` at startup to cache sudo credentials, plus a background `SUDO\_KEEPER\_PID` loop refreshing them every 60 seconds. This required interactive password entry even with NOPASSWD sudoers configured, because `sudo -v` itself prompts.

1597

1598 ****Fix:**** Removed the entire `sudo -v` + `SUDO\_KEEPER\_PID` section. All privileged commands now use `sudo -n` (non-interactive), which works silently because the user has NOPASSWD rules in `/etc/sudoers.d/nullexit` for the specific commands needed (`networksetup`, `python3`, `dscacheutil`, `killall`, `pkill`, `route`, `ifconfig`).

1599

1600 **#### 15.9.2 Background Sudo Failures**

1601 ****Context:**** The `--post-wake` script is intentionally designed to skip the `sudo -v` interactive prompt because it is launched by `watcher.sh` running via `launchd` in the background (which has no TTY and cannot accept password input). Thus, it relies entirely on `sudo -n` (non-interactive sudo) for all its internal routing changes.

1602 ****Problem:**** If the user has not configured the `/etc/sudoers.d/nullexit` file properly, any automated background wake event will silently fail: `sudo -n` commands drop out, the WARP bypass routes fail to apply, and the `warp` container loses internet connectivity.

1603

1604 **#### 15.9.3 Tailscale Hijacking the Default Route (PHYSICAL_GW Bug)**

1605 ****Problem:**** When `--post-wake` runs, it clears the routing table using `sudo route -n flush` to ensure a clean slate after the Wi-Fi card roams or wakes up. Because it skips bouncing the Wi-Fi interface (to make the reconnect faster), macOS's DHCP client does not automatically rebuild the physical default route. To counteract this, the script must manually restore the physical default route. Originally, the script tried to capture the physical gateway via `route get default`. However, because the gateway is already running and Tailscale is active, the default route is often already pointing to `utunX`. When this happens, `route get default` does not output a `gateway:` field (it only outputs the `interface:`), causing the captured `PHYSICAL\_GW` variable to be empty.

1606 When `PHYSICAL\_GW` is empty, the physical default route is never restored, causing the `en0` interface to be isolated from the physical network. The WARP bypass routes then fall back to targeting `interface en0` (instead of routing via the gateway), which breaks WireGuard's remote connections completely.

1607 ****Fix:**** The script now bypasses the OS routing table entirely and directly queries the hardware DHCP lease (`ipconfig getpacket en0`) to reliably extract the physical router IP, ensuring the physical route is correctly restored even when Tailscale is active.

1608

1609 **#### 15.9.4 July 4, 2026: Consolidation of Core Bash Scripts (Refactoring)**

1610 ****Symptom:**** Over 200 lines of bash logic were directly duplicated across `toggle.sh` and `recover.sh` (e.g. `restart\_tailscaled\_daemon`, `stop\_sleep\_prevention`, WARP bypass routes). This caused drift when one script was updated without the other.

1611 ****Root Cause:**** Historical separation of responsibilities allowed `recover.sh` to grow complex alongside `toggle.sh`.

1612 ****Resolution:****

1613 - Massively refactored both scripts by extracting all duplicated networking logic, PID lifecycle management, and environmental configuration down to `scripts/common.sh`.

1614 - Specifically, the `stop\_sleep\_prevention`, `stop\_dns\_watcher`, `stop\_host\_leak\_probe`, and `stop\_warp\_watcher` functions were collapsed into a single `stop\_pidfile\_daemon()` abstraction.

1615 - Proxy disable loops were abstracted to `disable\_all\_proxies()`.

1616 - The `162.159.192.1/193.1` WARP edge IPs are now dynamically resolved from `.env` instead of hardcoded.

1617

1618 **#### 15.9.5 Threat Model: Local Proxy Discovery**

1619 The Tor SOCKS proxy and Honey-Port Tripwire only bind to `127.0.0.1` (loopback). The real security boundary here is "can an unprivileged local process reach the loopback interface," not "does the malware know the port number."

1620

1621 The port-randomization and the Honey-Port trap only catch blind or generic port-scanners that do not already know where to look. They **do not protect** against a process that directly reads the `/tmp/nullexit-ports.env` file, greps the `toggle.sh` memory space, or runs `lsof -i` to inspect open socket bindings.

1622

1623 Because modifying file ownership on `/tmp/nullexit-ports.env` via `chown`/`chmod` to restrict read-access introduces an unavoidable Time-Of-Check to Time-Of-Use (TOCTOU) race condition (the file is created user-owned before privilege escalation, meaning file descriptors can be snatched), it is deliberately left as a standard user-owned file.

1624

```

1625 The actual, proper mitigation for preventing a malicious local process from reaching the proxy is not hiding
    the port it is running a host-level outbound firewall with strict localhost/loopback filtering enabled (e.g
    ., LuLu 4.3.0+ or Little Snitch). These firewalls gate access by cryptographic process identity (binary
    signature) at the kernel/network-extension level, rather than by port secrecy.
1626
1627 **Verifying Localhost Filtering (The Rogue Binary Test).** To empirically validate that the host firewall
    properly intercepts local proxy hijacking, you can compile and execute a completely unknown, un-whitelisted
    "rogue" binary to attempt a connection to the proxy port:
1628
1629 ```c
1630 // lulu-test.c
1631 #include <stdio.h>
1632 #include <stdlib.h>
1633 #include <arpa/inet.h>
1634
1635 int main(int argc, char *argv[]) {
1636     int port = atoi(argv[1]);
1637     int sock = socket(AF_INET, SOCK_STREAM, 0);
1638     struct sockaddr_in server = { .sin_family = AF_INET, .sin_port = htons(port) };
1639     server.sin_addr.s_addr = inet_addr("127.0.0.1");
1640
1641     printf("Rogue binary attempting connection to 127.0.0.1:%d...\n", port);
1642     if (connect(sock, (struct sockaddr *)&server, sizeof(server)) < 0) {
1643         printf("Connection BLOCKED by firewall.\n");
1644         return 1;
1645     }
1646     printf("Connection SUCCESSFUL (Firewall bypassed!).\n");
1647     return 0;
1648 }
1649 ```
1650 Compile with `gcc -o lulu-test lulu-test.c`. When executed against the `$TOR SOCKS_PORT`, a properly configured
    host firewall (with "Allow Loopback" disabled in LuLu Preferences) will immediately pause the socket
    execution and generate a desktop alert for the unrecognized binary signature. This physically stops
    targeted local malware from exfiltrating data through the Tor tunnel, proving the threat model mitigation.
1651
1652 #### 15.9.6 Unlocking Mode-Locked Output Files Without `chmod` (`unlock-files.sh`)
1653 **The one-command fix:** `bash scripts/unlock-files.sh` replaces every known locked inode in the project with
    a fresh writable one. No `chmod`. Covers `.env` (mode `000`), `adguard/work/userfilters/compiled_rules.txt`
    , `ip_blocklist.ipset`, `cache/*.txt`, `cache/ip/*.txt`, and `adguard/work/data/filters/*.txt` (mode
    `0444`). Run it once after setup or after pulling an old repo; `toggle.sh` and `sync-rules.py` will then
    work without permission errors.
1654
1655 **Context:** The nullexit project has a strict project-wide policy of `no chmod from scripts` (see README 6).
    This rule exists because every prior `chmod 0444` (post-write tamper-proof lock) and `chmod 000` (post-
    toggle `.env` lock) call from `scripts/sync-rules.py` / `toggle.sh` could leave a file permanently
    unreadable on disk if the script exited early, was killed mid-flight, or simply set a restrictive mode
    without ever being re-flipped. We've stripped every `chmod` from the codebase. But files **written by older
    versions of those scripts** are still on disk in those restrictive modes (`0444` for `compiled_rules.txt`,
    `ip_blocklist.ipset`, `data/filters/<id>.txt`; `000` for `.env` after end-of-toggle lock). Re-running `
    toggle.sh` or `scripts/sync-rules.py` against those leftovers hits `PermissionError: [Errno 13] Permission
    denied` immediately.
1656
1657 **Why the stale permissions also block `mv`/`rm` of the parent folder:** The restrictive modes (`000` on `.env
    `, `0444` on output files) don't just block writes they prevent the kernel from unlinking the file's
    dentry during a `mv` or `rm` of the **containing directory** (`adguard/work/`). macOS enforces this at the
    VFS layer: you own the directory, but you can't delete a directory that contains entries you can't write to
    . This made the entire `adguard/work/` tree unmovable/undeletable until the locked inodes inside it were
    replaced. `unlock-files.sh` resolves this permanently by swapping each locked inode for a fresh `0644` one
    via atomic rename (`cp` + `mv`), which operates on the directory entry rather than the file contents no `
    chmod` called, no permission bypass needed.
1658
1659 **Problem:** You (the owner) cannot `cat`, `cp`, `rm`, `>` redirect, or any normal POSIX write against a `mode
    =000` file even though you own it the basic mode bits block ALL access for owner, group, and other. The
    script does not call `chmod` and you want a permanent fix that never relies on a chmod from any future run
    either.
1660
1661 **Solution:** Replace the locked inode with a fresh readable one. `mv` is atomic; mode bits live in the inode,
    not the directory entry. Two flavors cover every case we hit on a real machine:
1662
1663 **Flavor A locked file is mode `000` (owner can't even READ it):**
1664 NOPASSWD sudoers (`/etc/sudoers.d/nullexit`) already includes `/usr/bin/python3`. So we read via `sudo python3`
    (which has the DAC override root has) and pipe the bytes around as base64 in user space, where the
    redirect happens with the user's normal umask = `0644`:
1665
1666 ```bash
1667 sudo -n /usr/bin/python3 -c "import base64,sys; sys.stdout.write(base64.b64encode(open('<FILE>','rb').read()).
    decode())" > /tmp/snap.b64
1668 base64 -d < /tmp/snap.b64 > <FILE>.new
1669 mv -f <FILE>.new <FILE>
1670 ls -la <FILE>          # mode is now 0644
1671 ```

```

```

1671 The old `mode=000` inode is unlinked by `mv`; the new `mode=0644` inode (created by base64-decode via the
      calling user's umask) takes the path. No chmod ever called. The parent directory just needs to be writable
      by you (it always is, since you own it).
1672
1673 **Flavor B locked file is mode `0444` (you can READ but cannot WRITE; `scripts/sync-rules.py` needs to re-
      write):**
1674 You own the file and can read it, you just cannot truncate/overwrite it. The simplest path is to rename it
      aside and let the writer create a fresh inode from scratch (which gets the umask-default `0644`):
1675 ```bash
1676 mv <FILE> /tmp/old.<FILE>.$$
1677 python3 scripts/sync-rules.py      # now writes <FILE> cleanly at mode 0644
1678 ```
1679 Or apply Flavor A if you'd prefer to preserve the contents while resetting the mode.
1680
1681 **Why this is the canonical recovery, not a hack:**
1682 - Mode bits live in the inode, not the path. `mv -f` replaces the path's inode reference; the old locked inode
      drops out of the directory entirely (dentry eviction) and loses its last link, so the kernel reclaims it.
      The new inode (whatever it is: a freshly-written one, or a base64-decoded copy) gets the path.
1683 - The only prerequisite to apply Flavor B's `mv` is: you own the parent directory (so you can unlink entries
      from it) AND you have a NOPASSWD-capable binary that can read the byte stream if mode prohibits user read.
1684 - This pattern generalizes. Any time a script ends up producing a "locked file that the next run can't update"
      situation, the same trick applies without a single chmod anywhere.
1685
1686 **Empirical verification (user machine, July 1, 2026):**
1687 - `.env` was `mode=000` (owner $USER, i.e. you): Flavor A unblocked it in 3 commands, no chmod.
1688 - Before: `----- 1 $USER staff 899 Jun 28 07:07 .env`
1689 - After:  `-rw-r--r--@ 1 $USER staff 899 Jul 1 20:39 .env`
1690 - `docker compose config` immediately picked up `TS_AUTHKEY` + `WIREGUARD_PRIVATE_KEY` substitution.
1691 - `compiled_rules.txt`, `ip_blocklist.ipset`, `data/filters/1782645604.txt` were `mode=0444`: Flavor B (`mv`-
      aside) unblocked all three.
1692 - `scripts/sync-rules.py` then ran clean: 279587 block rules + 171 allow rules + 16710 IP entries compiled; new
      files came out at `0644`.
1693 - `toggle.sh` then went end-to-end in 48s (warp / routing-fix / adguardhome / tailscale all healthy, gateway
      mesh-joined, DNS hijack verified single-server).
1694
1695 **When this trick is appropriate vs. inappropriate:**
1696 - You own the parent directory and the file (typical desktop scenario).
1697 - The lock is a leftover from a removed `chmod` call that you want off disk forever.
1698 - NOPASSWD sudo includes at least one interpreter (`python3` on this system) that can be used to read mode-0
      files. If it does not, add it first: `**Reusable cookbook:**
1704
1705 ```bash
1706 # Quick diagnostic: figure out which flavor you need.
1707 WHOAMI=$(whoami)
1708 ls -la <FILE>
1709 stat -f 'mode=%Sp owner=%Su group=%Sg' <FILE>
1710 # mode=----- or mode=----r----- : you cannot even read it Flavor A.
1711 # mode=r--r--r-- but writer fails : you are blocked from write but can read Flavor B.
1712 # mode=rw-r--r-- : not actually locked at all; unrelated write failure.
1713
1714 # Flavor A (mode=000):
1715 sudo -n /usr/bin/python3 -c "import base64,sys; sys.stdout.write(base64.b64encode(open('<FILE>','rb').read()).
      decode())" > /tmp/snap.b64
1716 base64 -d < /tmp/snap.b64 > <FILE>.new && mv -f <FILE>.new <FILE> && rm -f /tmp/snap.b64
1717
1718 # Flavor B (mode=0444, file is readable):
1719 mv <FILE> /tmp/old.<FILE>.$(date +%s)
1720 # (let the original writer recreate <FILE>; new file lands at mode 0644)
1721
1722 # Verify after either flavor:
1723 ls -la <FILE>          # mode should be 0644
1724 <your normal validation command>
1725 ```
1726
1727 #### 15.9.7 sudo -n bash -c Silently Breaks the Scoped NOPASSWD Grant for dns-proxy.py (Fixed 2026-07-18)
1728 **Symptom:** Found while verifying `scripts/pihole-mode.sh` actually works, not from a reported failure `bash
      scripts/pihole-mode.sh enable` hung on an interactive password prompt despite passwordless sudo being
      correctly configured and working for every other privileged call (`pfctl`, `networksetup`, etc. all
      succeeded fine under `sudo -n`).
1729
1730 **Root cause:** `sudo -l` shows the live NOPASSWD grant for the DNS proxy is scoped to the *literal* command `/
      usr/bin/python3 /Users/.../scripts/dns-proxy.py` (plus Homebrew-path variants) no shell wrapper, no env-
      var prefix. But `pihole-mode.sh`'s `cmd_enable()` invoked it as `sudo -n bash -c "LISTEN_ADDR=...

```

TARGET_PORT=... python3 dns-proxy.py & echo \$! > pidfile". sudoers matches the *literal argv* passed to `sudo`; here that's `bash -c "<string>"`, which doesn't match a rule scoped to `python3 <path>` at all so it silently fell through to requiring interactive auth. Even fixing just the wrapper wouldn't be enough: `dns-proxy.py` reads its config (`LISTEN\_ADDR`, `LISTEN\_PORT`, `TARGET\_HOST`, `TARGET\_PORT`) purely from environment variables, and `sudo`'s `env\_reset` (macOS default) strips any custom env var not in `env\_keep` none of these four are in the default `env\_keep` list, and the sudoers rule carries no `SETENV` tag.

1731

1732 ****Same bug, dormant, in the primary codebase:**** `scripts/common.sh`'s `start\_local\_dns\_proxy()` the SOCKS5-fallback DNS path used when exit-node pre-flight checks fail used the identical `sudo -n bash -c "TARGET\_PORT=\$DNS\_PROXY\_PORT ..."` pattern. Never caught because this session (like most) achieves exit-node mode every time, so the fallback path never actually runs. Had it ever been needed, the one thing meant to keep DNS working when the primary path fails would have hung on a password prompt with no TTY to answer it.

1733

1734 ****Fix:**** Rather than fight `sudo`'s env-passing restrictions (which would require editing the live sudoers file a change requiring a password I can't supply non-interactively, and not something to script unprompted), `dns-proxy.py` now optionally reads `/tmp/nullexit-dns-proxy.conf` (written `0600` by the caller, checked on top of the existing env-var/default values) if present. Both callers now write their overrides to that file, then invoke `sudo -n "\$PYTHON" "\$DNS\_PROXY\_SCRIPT"` directly the exact, unwrapped, already-whitelisted command, no `bash -c`, no env prefix. Both `stop\_local\_dns\_proxy()` and `pihole-mode.sh` disable clean up the conf file. Verified end-to-end: `enable` no password prompt `test` resolves clean domains, sinkholes `doubleclick.net` `0.0.0.0` PASS `disable` removes both the process and the conf file.

1735

1736 ****Threat model note:**** the conf file is a bounded local-TOCTOU surface, same class already accepted for `/tmp/nullexit-ports.env` (15.9.5) restrictive perms, not a hard boundary against a co-resident attacker who could already do more damage with local code execution than redirect a DNS forwarder.

1737

1738 **### 15.10 Tor Container**

1739

1740 **#### 15.10.1 July 11, 2026: Tor Container Hardening & obfs4proxy Restoration**

1741 ****Symptom:****

1742 1. The Tor container entered a fatal crash loop upon boot, throwing `exec: "obfs4proxy": executable file not found in \$PATH` when `TOR\_USE\_BRIDGES=true`.

1743 2. When the user pasted Tor bridge lines containing comments (`#`) or blank lines into `tor-bridges.txt`, the container failed to validate the file properly and crashed.

1744

1745 ****Root Cause:****

1746 - ****Bug 1 (Missing Pluggable Transports):**** The Tor Dockerfile was built on `debian:bullseye-slim`, which had silently dropped or failed to resolve the `obfs4proxy` package in its latest apt repositories.

1747 - ****Bug 2 (Fragile Validation):**** The `entrypoint.sh` bridge validation check merely checked `[ -s tor-bridges.txt ]` (file size > 0). It did not verify if the file actually contained valid, uncommented bridge lines. Passing empty lines or comments tricked the validation script into appending invalid `Bridge` directives to the `torrc`, causing the Tor daemon to instantly exit.

1748

1749 ****Resolution:****

1750 - ****Upgraded Base Image:**** Shifted the Tor Dockerfile base image to `debian:bookworm-slim`, restoring access to the `obfs4proxy` pluggable transport package.

1751 - ****Robust Bridge Validation:**** Replaced the fragile `[ -s ]` check with a strict `grep -vE '\s\*#|\$)'` command that strips comments and empty lines. If no valid lines remain, the container safely falls back to standard public guard relays instead of crashing, ensuring Tor remains available even with a misconfigured bridge file.

1752 - ****Image Pinning:**** Explicitly pinned `image: nullexit-tor:v1.0.0` in `docker-compose.yml` to prevent arbitrary local builds from drifting to `:latest`.

1753

1754 **#### 15.10.2 July 12, 2026: Tor ControlPort Dynamic Password Authentication & User Config**

1755 ****Symptom:**** Querying the Tor Control Port (e.g., via the `/sweep` script or `check\_circuits.py`) returned empty responses or connection errors. Logs showed that Tor automatically closed its ControlPort:

1756 `You have a ControlPort set to accept unauthenticated connections from a non-local address. ... That's so bad that I'm closing your ControlPort for you.`

1757

1758 ****Root Cause:**** Docker port mapping (especially under Colima on macOS) requires container sockets to bind to `0.0.0.0` to be reachable from the host. However, when Tor's ControlPort is bound to `0.0.0.0` (non-local) with no authentication, Tor closes it by default as a safety precaution. Sourcing SOCKS5/ControlPort to `127.0.0.1` inside the container was not an option as it broke the Docker bridge routing path.

1759

1760 ****Resolution:****

1761 - ****Dynamic Password Authentication:**** Updated the Tor container's `entrypoint.sh` to read `TOR\_PASSWORD` and dynamically generate its HMAC hash via `tor --hash-password`, adding `HashedControlPassword <hash>` to `torrc`. This secures the ControlPort on `0.0.0.0` so Tor keeps it open.

1762 - ****Unified Credentials in .env:**** Exposed `ADGUARD\_USER=admin` and `ADGUARD\_PASSWORD=nullexit` in `.env` to serve as the unified gateway credentials. Sourced `TOR\_PASSWORD` directly from `ADGUARD\_PASSWORD` in `docker-compose.yml`.

1763 - ****Client Authentication:**** Updated `scripts/check\_circuits.py` to fetch `TOR\_PASSWORD` (defaulting to `ADGUARD\_PASSWORD` or `nullexit`) and authenticate with the ControlPort using `AUTHENTICATE "<password>"` to regain access.

1764

1765 > See also 11.3 (Tor Container Kill-Switch Inheritance & Fail-Closed Egress Verification) for the design-level fail-closed guarantee.

1766

1767 **### 15.11 macOS Platform Quirks**

1768

```

1769 ##### 15.11.1 macOS Application Firewall Silently Blocking AirDrop & Continuity
1770 **Problem:** Users of complex network extensions (Tailscale, LuLu, Docker, etc.) often run into an issue where
AirDrop, AirPlay, and Universal Clipboard silently fail, even with the gateway inactive and Wi-Fi/Bluetooth
correctly configured. The internal Apple Wireless Direct Link (`awdl0`) interface appears healthy, but
connections never establish.
1771
1772 **Root Cause:** The built-in macOS Application Firewall has a specific list of explicitly blocked applications.
Apple's own core networking services (e.g., `/usr/libexec/sharingd`, `/usr/libexec/rapportd`, `/usr/
libexec/avconferenced`) can be silently added to the "Block incoming connections" list either through
accidental "Deny" clicks on permission prompts, execution of aggressive privacy-hardening scripts, or a
known macOS bug that retains strict blocks even after toggling the "Block all incoming connections" setting
off. Since these are background daemons, macOS provides no visible error.
1773
1774 **Fix:** The firewall rules must be cleared via the terminal (or System Settings > Network > Firewall > Options
):
1775 ```bash
1776 sudo /usr/libexec/ApplicationFirewall/socketfilterfw --unblockapp /usr/libexec/sharingd
1777 sudo /usr/libexec/ApplicationFirewall/socketfilterfw --unblockapp /usr/libexec/rapportd
1778 ```
1779 Once unblocked, `sharingd` immediately resumes broadcasting mDNS/Bonjour discovery packets, and AirDrop
restores instantly without requiring a reboot.
1780
1781 ##### 15.11.2 Standalone Tailscale Daemon Freeze after macOS Sleep/Wake
1782 **Problem:** When the macOS host goes to sleep and wakes up, the network interfaces and routing tables are
rebuilt. The standalone `tailscaled` daemon (installed via Homebrew) occasionally fails to handle this
transition and becomes completely frozen/unresponsive. In this state, any command like `tailscale down` or
`tailscale status` hangs indefinitely, and all DNS requests sent to the local Tailscale resolver
(`100.100.21.8`) time out, causing a total internet blackout on the host.
1783
1784 **Fix:** Enhanced the Tailscale verification and teardown logic in `toggle.sh` and `recover.sh`:
1785 1. **Unresponsive Daemon Detection:** Instead of assuming the daemon is healthy just because the Unix socket `/
var/run/tailscaled.socket` exists, the scripts now actively run a status check with a 5-second timeout (`
run_with_timeout 5 tailscale status`). If the check times out (exit code 143), the daemon is classified as
unresponsive.
1786 2. **Auto-Recovery Loop:** If the daemon is unresponsive, the script now automatically attempts to restart the
service using `brew services restart tailscale` (falling back to `sudo -n brew services restart tailscale`)
.
1787 3. **Graceful Retry:** Once restarted, the script pauses for 3 seconds for the daemon to initialize before
proceeding with connection or teardown commands.
1788
1789 ##### 15.11.3 macOS Sharing Services (AirDrop/AirPlay) Freeze on Network Transitions
1790 **Problem:** When the gateway is turned on or off, the scripts perform network configuration cleanups which
include flushing the routing tables (`route -n flush`), restarting the `mDNSResponder` daemon (`killall -
HUP mDNSResponder`), and power-cycling the Wi-Fi interface (`setairportpower` or `ifconfig en0 down/up`).
While AirDrop BLE discovery continues to function, the actual file transfers stall at "Waiting..."
indefinitely.
1791
1792 **Root Cause:** The rapid tear-down and rebuild of the local routing table and Wi-Fi interface causes Apple's
core sharing and connection daemons (`sharingd` and `rapportd`) to lose their socket bindings to the mDNS/
Bonjour discovery interface. Instead of reconnecting gracefully, they enter a wedged state and try to route
peer-to-peer (AWDL) IPv6/TCP transfer traffic into the default gateway (the Tailscale interface `utun`)
instead of the direct wireless link, causing the TLS verification handshake to time out.
1793
1794 **Fix:** Integrated an automatic sharing services reset into both `toggle.sh` and `recover.sh`:
1795 1. The script now automatically runs `sudo -n killall sharingd rapportd` at the end of both the **START** path
and the **STOP** path (as well as inside `recover.sh`).
1796 2. Killing these daemons forces macOS to immediately relaunch them, binding them fresh to the newly initialized
network interfaces and routing tables, allowing AirDrop to work seamlessly while the gateway is active.
1797
1798 ##### 15.11.4 Chrome Remote Desktop Connection Failures (Tailscale on Remote Device)
1799 **Context:** When attempting to access a remote machine via Chrome Remote Desktop (CRD), the connection fails
or hangs if Tailscale is active **on the remote device**. The connection works fine even if Tailscale (and
the exit node) is active **on the local client/viewer device**, but only if Tailscale is disabled on the
remote machine. *(This is distinct from the CRD-through-WARP latency limitation in 14.3.)*
1800
1801 **Why:** Having Tailscale active on the remote machine creates virtual network interfaces and DNS/routing
modifications that conflict with Chrome Remote Desktop's WebRTC bindings and signaling traffic.
1802
1803 **Workaround:**
1804 - **Use Tailscale Native RDP/RustDesk (Recommended):** Connect directly to the remote device's Tailscale IP
(`100.X.Y.Z`) via an RDP or RustDesk client. This is faster and avoids third-party relay conflicts.
1805 - **Disable Tailscale on the Remote Device:** If you must use CRD, disable Tailscale on the remote machine
before connecting.
1806
1807 ##### 15.11.5 tailscaled Daemon Broken State (brew services)
1808 **Symptom:** `brew services list` shows `started` but `tailscale status` shows "stopped".
1809
1810 **Fix:** `sudo brew services restart tailscale`. Toggle script now detects and auto-restarts.
1811
1812 ##### 15.11.6 Apple Silently Removed the `airport` Binary in macOS Sonoma 14.4 (March 2024)
1813 **What happened:** `watcher.sh`'s `detect_lan_p2p_mode()` was written to use the `airport` CLI tool at:

```



```

1814 ~~~
1815 /System/Library/PrivateFrameworks/Apple80211.framework/Versions/Current/Resources/airport
1816 ~~~
1817 This path returned `exit 127: no such file or directory`. The function silently fell back to `reason="no-wifi"`
    even while the Mac was actively connected to WPA2 Enterprise Wi-Fi.
1818
1819 **Why Apple removed it:** `airport` was never a real public binary it lived inside a **private framework** and
    was always technically unsupported. Apple deprecated it quietly years ago and finally removed it in **
    macOS Sonoma 14.4 (March 2024)**. The removal is part of a broader privacy clampdown on Wi-Fi metadata:
1820 - Starting in **macOS 14.5**, BSSIDs and other Wi-Fi association details are **redacted** (`<redacted>`) in
    most tools, including the official replacement `wdutil`
1821 - Apple's official recommendation is to use the **CoreWLAN framework** (Swift/Objective-C) with proper
    entitlements useless for a bash script
1822 - The official CLI replacement `wdutil` requires `sudo` for most useful output also useless for a background
    LaunchAgent
1823
1824 **Fix:** Replaced `airport -I | awk '/link auth/'` with:
1825 ```bash
1826 system_profiler SPAirPortDataType | awk '/Current Network Information:/{found=1} found && /Security:/{print $NF
    ; exit}'
1827 ~~~
1828 This returns human-readable strings like `WPA2 Enterprise`, `WPA2 Personal`, or `None` for the currently
    associated network without `sudo`, and without any removed private binary.
1829
1830 **Confirmed working output on this machine:**
1831 ~~~
1832 P2P detect: security='Enterprise' allow=false (reason: 802.1x-enterprise)
1833 ~~~
1834
1835 > ** Risk: `system_profiler` may not survive forever either.** `system_profiler SPAirPortDataType` still works
    as of macOS Sequoia (15.x) without sudo and without redaction. However, given Apple's trajectory on Wi-Fi
    privacy, this could be locked down in a future release. If `security` comes back empty on a future macOS
    version while the Mac is actively on Wi-Fi, the function will silently fall back to `allow=false` (safe
    default), but the detection will be broken again. The long-term fix would be a small Swift helper using
    CoreWLAN that we compile once and bundle in the repo. *(This forward-looking caveat is also tracked as an
    observation in 16.)*
1836
1837 ##### 15.11.7 Changing macOS Local Hostname
1838 **Context:** Users may wish to change their Mac's computer name/hostname in macOS System Settings.
1839 **Effect:** Changing the Mac's hostname will **not** break Nullexit (which relies on internal routing/localhost
    ). However, Tailscale will automatically detect this change and update the machine name on the Tailnet.
1840 - The underlying Tailscale `100.x.x.x` IPv4 address will remain exactly the same.
1841 - The **MagicDNS name** will change to match the new hostname. Users must remember to update their SFTP/SSH
    clients to use the new MagicDNS address.
1842
1843 ##### 15.11.8 macOS MAC Address Spoofing (Apple Silicon)
1844 **Observation:** macOS prevents setting a new MAC address (`ifconfig en0 ether <mac>`) while the Wi-Fi
    interface is actively associated with an SSID. Additionally, some drivers on Apple Silicon completely
    reject spoofed MAC addresses via `ifconfig`, effectively ignoring the command and retaining the hardware
    MAC.
1845 **Workaround / Fix:**
1846 1. **Disassociate First:** The Wi-Fi interface must be turned off (`networksetup -setairportpower en0 off`)
    before attempting to set the MAC address, and turned back on afterward. This allows the OS to accept the
    change.
1847 2. **Fail-Safe Testing:** The `scripts/device-scramble.sh test-mac` command was implemented to safely verify if
    the underlying hardware and the current network will accept a randomized MAC address, as behavior varies
    significantly between Intel Macs (usually permissive) and Apple Silicon Macs (often restrictive).
1848
1849 ##### 15.11.9 Shell-Language Landmines: macOS `/bin/bash` 3.2 + `set -e` (Field Notes)
1850
1851 This project's lifecycle scripts (`toggle.sh`, `recover.sh`, `common.sh`, ) run under `#!/bin/bash`, which on
    macOS is **GNU bash 3.2.57 (2007)** Apple has never shipped a newer bash because bash 4+ is GPLv3. (
    Homebrew's bash 5 lives at `/opt/homebrew/bin/bash` but is **not** the shebang target, and cannot be assumed
    on PATH a bare `bash --version` on a stock Mac reports 3.2.) Every script here also runs `set -e` (exit-on
    -error) with an `ERR`/`INT`/`TERM`/`HUP` trap. That combination **ancient bash + `set -e` + traps + `set -
    x` xtrace redirection** is a minefield. The traps below were all hit and fixed during development;
    document them so they are not re-discovered.
1852
1853 **A. `set -e` + `if then else fi` can abort at the `else` (bash 3.2 heisenbug).**
1854 The nastiest one. A construct as innocent as:
1855 ```bash
1856 if [ -f "$MARKER_FILE" ]; then
1857     X="yes"
1858 else
1859     X="no"
1860 fi
1861 ~~~
1862 aborted `toggle.sh` at init under `set -e` whenever the file was **absent** the false status of the `[ -f ]`
    test leaked through the compound and `set -e` killed the script (`EXIT: code=1 phase=init`), before any
    real work ran. Two things made this vicious: (1) it is intermittent/context-dependent it did **not**
    reproduce in isolated `bash -c` snippets of the identical block; it only manifested inside the full

```



```

script with the `set -x` xtrace fd-redirect (`exec 2> >(tee)` active. We only proved causation by **
bisecting the live script** (replace the block with a trivial assignment the script sailed past init). (2)
Static tools (`bash -n`, `shellcheck`) cannot see it it is a runtime interaction. **Safe pattern:**
default-assign, then an `if` with **no `else`** (an `if` whose false condition has no else-branch yields
status 0):
1863 ```bash
1864 X="no"
1865 if [ -f "$MARKER_FILE" ]; then X="yes"; fi
1866 ```
1867 **Confirmed root cause (2026-07-13, reproduced head-to-head).** The trigger is specifically **`set -x` plus a
PS4` that contains a command substitution** here the `DEBUG_TRACE` prompt `PS4='+ [$(date)] `. Forcing
the false-condition/else path (`gateway_state` `stopped`, so `if [ "$(gateway_state)" = "running" ]` takes
the else/START branch) on stock `/bin/bash` 3.2.57: with the **default `PS4`** it survives (3/3, exit 0);
with toggle's **`$(date)` `PS4`** the `ERR` trap fires on the else branch (`rc=1` from the false condition,
3/3, exit 42). The `$(date)` runs a subshell on *every* traced command; while tracing the `if`-condition
it perturbs ` $? ` so the condition's non-zero status leaks past the normal `if`-exemption and trips the `ERR`
trap. This is a true **heisenbug** enabling tracing (`DEBUG_TRACE=true`) is what *causes* the abort,
which is exactly why it's invisible with `DEBUG_TRACE=false` (the default) and why every isolated repro on
the *default* `PS4` passes (this misled an entire debugging session into briefly "disproving" the landmine)
. It aborted the gateway START decision (`toggle.sh:689`) under `DEBUG_TRACE=true`; with `DEBUG_TRACE=false`
the gateway starts cleanly (verified: 192 s, all five containers healthy). **Fixed (2026-07-14).**
`DEBUG_TRACE`'s `PS4` now uses the subshell-free `${SECONDS}` (elapsed seconds since the shell started)
instead of `$(date "+%H:%M:%S")`, removing the per-traced-command subshell that perturbed ` $? `. bash 3.2
has no subshell-free *wall-clock* token for `PS4` (`$EPOCHSECONDS` is 5.0+, `printf '%()T'` is 4.2+), so
the per-line stamp is now elapsed-seconds the absolute start time is still logged once in the `DEBUG_TRACE`
enabled `header` line, so wall-clock is recoverable. Bonus: this also drops a `date` fork on *every*
traced line (previously thousands). **Verified head-to-head on stock `/bin/bash` 3.2.57:** `toggle.sh --
restart` with `DEBUG_TRACE=true` now completes `code=0` with tracing active straight through the exact
decision that aborted before the trace shows `gateway_state stopped` (`common.sh:229`) feeding the false
`if [ "$(gateway_state)" = "running" ]`, then the else branch reaching `Action: STARTUP GATEWAY` (1436
traced lines, no `ERR` trap firing, all five containers healthy). `DEBUG_TRACE` is now safe to use on the
STOP/START decision path.
1868
1869 More generally, any command whose "failure" is normal control flow (`grep -q`, `[ ]`, `diff`, `kill -0`) that
ends up as the *last* command of a function/branch/script under `set -e` is a landmine; guard it (`|| true`
`) or restructure so a non-zero status can't be the tail value.
1870
1871 **B. `BASH_XTRACEFD` and the `exec {var}>>file` dynamic-fd form are bash 4.1+ (hard-fail on 3.2).**
1872 Our `DEBUG_TRACE` feature wants xtrace (`set -x`) sent to `output.log` while keeping the terminal clean the
clean way is `exec {BASH_XTRACEFD}>>"$LOG_FILE"`. On 3.2 this is a **syntax/runtime error**: `exec: {
BASH_XTRACEFD}: not found` (dynamic-fd allocation `{var}>>` didn't exist until 4.1, and `BASH_XTRACEFD`
itself is 4.1+; on 3.2 it's just an ordinary variable that `set -x` ignores). Left unguarded it would abort
the script at startup i.e. the debug feature would break the very thing it's meant to observe. **Safe
pattern:** branch on `${BASH_VERSINFO[0]}.${BASH_VERSINFO[1]}` on 4.1 use `exec 9>>"$LOG_FILE"; export
BASH_XTRACEFD=9`; on 3.2 fall back to `exec 2> >(tee -a "$LOG_FILE" >&2)` (xtrace goes to stderr, which we
duplicate to the log; the terminal also shows it, which is acceptable). Never use the `exec {var}>>` form
unless you *know* you're on 4.1.
1873
1874 **C. `trap ERR` + `${LINENO}` misattributes the failing line on 3.2.**
1875 When a command fails under `set -e`, our trap runs `cleanup_handler ERR $LINENO`. On bash 3.2, `${LINENO}` inside
an `ERR` trap frequently reports the line of the **enclosing `fi`/closing brace of a compound statement**,
not the true failing line. During the STOP/START-decision bug this produced `Script failed at line 1202`
where 1202 is a bare `fi` actively misleading. **Mitigation:** don't trust the trap's line number as
gospel on macOS; corroborate with the `EXEC:/EXIT:` lifecycle breadcrumbs and the `DEBUG_TRACE` xtrace (
which prints `file:line:function` via a custom `PS4`), which pin the *actual* last-executed command.
1876
1877 **D. `set -e` is not inherited by command substitutions / subshells the way people expect, and is suppressed
inside `if`/`&&`/`||` conditions.**
1878 This cuts both ways and is a frequent source of "why didn't it fail?" and "why *did* it fail?": a non-zero
command used as an `if` condition (or left/right of `&&`/`||`) is *exempt* from `set -e` (by design), but
the moment that same call is a bare statement it aborts. `is_gateway_active()` relies on this (it's always
called as `if is_gateway_active; then`, which is precisely why moving the decision to a marker file
instead of a probe that must be `if`-guarded to be safe is more robust (see 15.12.19). Also note `local X=
$(cmd)` masks `cmd`'s exit status (the `local` builtin's status wins) a `shellcheck` SC2155 we see
repeatedly; harmless when we don't check ` $? `, but a real footgun if you do.
1879
1880 **E. Process substitution (`>()`, `<()`) is a bashism, and its lifecycle is loose.**
1881 Our 3.2 xtrace fallback (`exec 2> >(tee -a "$LOG_FILE" >&2)`) uses process substitution fine under bash, but
(1) it is **not** POSIX `sh`, so these scripts must never be invoked as `sh script.sh`; and (2) the
background `tee` it spawns is reaped asynchronously, so the very last few lines written just before exit
can occasionally be truncated in the log. Acceptable for a debug trace; do not rely on it for critical last
-gasp logging (use a direct `>> "$LOG_FILE" append for must-capture lines, as the `EXIT` breadcrumb does).
1882
1883 **F. GNU-vs-BSD userland divergence (not bash itself, but same class of pain).**
1884 macOS ships BSD variants of the core tools, so idioms differ from Linux: `sed -i ''` (BSD requires the empty
backup-suffix arg) vs `sed -i` (GNU); `stat -f%z` (BSD) vs `stat -c%s` (GNU); `date` format flags; `jot` (
BSD) vs `shuf` (GNU) for random port generation; `route`/`networksetup`/`dscacheutil` (macOS-only) vs `ip
`/`resolvectl` (Linux). This is *why* the codebase branches on `[ "$(OSTYPE)" == "darwin" ]` everywhere
`toggle.sh` and `recover.sh` are each single cross-platform scripts that dispatch on `OSTYPE` (an early
Linux block runs and exits before the macOS body). `timeout(1)` doesn't exist on stock macOS at all hence
the pure-bash `run_with_timeout` in `common.sh`.

```

```

1885
1886 **Working rules distilled from the above:**
1887 - Assume **bash 3.2** semantics for anything under #!/bin/bash on macOS; gate any 4.0+ feature (BASH_XTRACEFD`, dynamic {var}` fds, associative arrays, {var}` case-conversion, mapfile`/`readarray`,
negative array indices) behind a BASH_VERSINFO` check or avoid it.
1888 - Under set -e`, never let a "benign non-zero" command ([ ]`, grep -q`, kill -0`, diff`) be the tail
statement of a branch/function/script; use no-`else` if`s, || true`, or explicit `${` handling.
1889 - Prefer **persisted state (marker files)** over **live probes** for control-flow decisions a probe forces an
if`-guard and can hang/abort; a file read cannot (see 15.12.19).
1890 - bash -n` and shellcheck` catch syntax and many smells but **cannot** catch set -e`/trap runtime
interactions a **live run is mandatory** to validate lifecycle-script changes; the DEBUG_TRACE` +
breadcrumb instrumentation exists precisely to make those live runs diagnosable.
1891 - Never run these scripts as sh` they are bashisms end to end.
1892
1893 #### 15.11.10 The Live Wi-Fi SSID Is Unreadable From Any Script (Darwin 25) Confirmed Including Root
1894 **Finding (2026-07-23, while building Wi-Fi auto-rejoin, 15.5.9).** On this Mac (Darwin 25 / macOS 26) there is
**no** script- or root-accessible path to the *currently associated* SSID. This was verified empirically
against every candidate:
1895
1896 - networksetup -getairportnetwork en0` "You are not associated with an AirPort network" *even while connected
*.
1897 - ipconfig getsummary en0`, system_profiler SPAirPortDataType`, wdutil info`, scutil` the SSID field
prints <redacted>`. wdutil` redacts **as root too**. The legacy airport` binary was removed back in 14.4
(15.11.6).
1898 - The on-disk store /Library/Preferences/com.apple.wifi.known-networks.plist` (per-network SSIDs + timestamps)
is 0600 root:wheel` with **no** restricted` flag, yet sudo python3 open()` still returns 
PermissionError` it lives in a macOS **data vault** (same protection class as TCC.db`), which denies even
root regardless of flags. defaults`/`plutil` can't reach it either. airport.preferences.plist` is SIP-
protected.
1899
1900 The **only** dynamic path to the live SSID is a **Location-Services-authorized** process (shortcuts` CLI, or a
CoreWLAN+CoreLocation helper). The user explicitly refused to grant Location, so the live SSID is treated
as permanently unobtainable. This is *why* Wi-Fi auto-rejoin (15.5.9) is built on a user-**remembered**
name list rather than reading the current network joining *by name* is the one operation Apple leaves
ungated. Note this is distinct from 15.11.6, which is specifically about the airport`-binary removal
breaking detect_lan_p2p_mode`'s security probe; system_profiler`'s Security:` field still works, only
the SSID itself is redacted.
1901
1902 ### 15.12 Misc Resolved
1903
1904 #### 15.12.1 Gluetun Resets nftables FORWARD Rules
1905 **Symptom:** FORWARD RELATED,ESTABLISHED rule disappears after Gluetun health check.
1906
1907 **Fix:** scripts/routing-fix.sh` re-adds to both iptables backends every 30 seconds. Legacy backend (policy
ACCEPT) acts as safety net.
1908
1909 #### 15.12.2 Native SSH & SFTP Security (Zero Local Attack Surface)
1910 **Context:** macOS "Remote Login" (sshd`) and "File Sharing" (smbd`) open ports on the local network (e.g.
172.x.x.x`), exposing the host to brute-force attacks on public Wi-Fi.
1911
1912 **Implementation:** The script strictly enforces tailscale up --ssh=true` whenever the mesh state is reset.
This empowers the user to completely disable native macOS Remote Login and File Sharing. This provides an
incredible zero-trust security advantage: even if an attacker steals your Mac username and password, they
**cannot** access your files remotely. Disabling the native services completely closes the listening ports
on the local Wi-Fi interface (172.x.x.x`), rendering the Mac inaccessible to local network logins.
Meanwhile, Tailscale intercepts port 22 traffic exclusively over the encrypted mesh and authenticates
cryptographically based on your Tailscale SSO identity. Stolen Mac passwords are mathematically useless
without first bypassing your Tailscale Two-Factor Authentication and registering a device onto the mesh.
1913
1914 **Cross-Platform File Exchange:** To easily browse and exchange files securely, simply use an SFTP client and
connect to your Mac's MagicDNS name on port 22. Recommended clients: **WinSCP** or **FileZilla** (Windows),
**Cyberduck** (macOS), and **FE File Explorer** or **Solid Explorer** (iOS/Android).
1915
1916 #### 15.12.3 Logging Architecture
1917 For debugging, logs are strictly segmented based on the component's lifecycle:
1918 - **`output.log` (Host-side):** Contains all standard error (stderr`) and verbose output from the host scripts
(toggle.sh`, recover.sh`, setup.sh`). Since the rule-compiler` container is ephemeral and deleted
after running, its logs (and any Python errors from scripts/sync-rules.py`) are extracted and appended to
this file before deletion.
1919 - **Log Rotation Policy:** On startup, toggle.sh` checks if output.log` exceeds **1GB** (1,073,741,824`
bytes). If it does, it performs a metadata-only rename (mv output.log output.log.old`), preserving history
while resetting the active log to 0 bytes. This rename-based rotation avoids the heavy disk I/O and CPU
overhead of rewriting a massive file line-by-line. The threshold is intentionally large because with 
DEBUG_TRACE=true` the log accumulates a full command-by-command execution trace (effectively the framework'
s entire compilation/warning history), which is valuable to retain for post-mortems; a smaller cap would
rotate that history away too quickly.
1920 - **Egress Probe Logging:** The background host-egress leak probe checks if stdout is a TTY ([ -t 1 ]`) and
will only print live status updates (using carriage return \r`) or duplicate console warnings in
interactive mode. It automatically detects macOS/BSD vs. Linux environments to dynamically construct
timestamps with date and time (omitting milliseconds on macOS to prevent high CPU utilization from spawning
sub-second processes).

```

```

1921 - **docker logs <container>` (Guest-side):** The persistent containers (`warp`, `tailscale`, `routing-fix`, `
    adguardhome`) use the Docker `json-file` logging driver with a strict `max-size` (1m-10m) to prevent VM
    disk exhaustion. Use standard `docker logs` to view them.
1922
1923 #### 15.12.4 Pre-Flight Check 1: Wrong `tailscale ping` Flag
1924 **Problem:** Check 1 of the pre-flight connectivity checks used `--c 1` (double dash) but `tailscale ping`
    accepts `-c 1` (single dash). The invalid flag caused the command to exit with code 1, marking the check as
    FAIL even though the gateway was reachable.
1925
1926 **Fix:** Changed `--c 1` to `-c 1`.
1927
1928 #### 15.12.5 Pre-Flight Check 1: DERP Relay Exit Code
1929 **Problem:** Even with the correct `-c 1` flag, `tailscale ping` exits with code 1 when the connection goes
    through a DERP relay (`direct connection not established`) even though a pong was successfully received
    (28ms via DERP(tor)). The check treated relayed connections as failures.
1930
1931 **Fix:** Changed the check from relying on exit code to piping stdout through `grep -q "pong"`. This accepts
    relayed connections as valid (they work fine for the exit node use case), while still failing on true
    timeouts or unreachable peers.
1932
1933 #### 15.12.6 IPv6 Exit-Node Leak
1934 **Problem:** Tailscale supports IPv6, but WARP/ghuetun is IPv4-only. IPv6 packets forwarded through the exit
    node would leak through the Docker bridge unencrypted.
1935
1936 **Fix:** Added `ip6tables -A FORWARD -i tailscale0 -j DROP` to `post-rules.txt`, blocking all IPv6 forwarded
    traffic from the Tailscale interface. *(This is the fix referenced by the IPv6-leak scenario in 15.6.1.)*
1937
1938 #### 15.12.7 SOCKS5 Proxy Threading Race Condition
1939 **Problem:** The `forward` function in `scripts/socks5-proxy.py` called `select.select([src, dst], [], [])`
    which raised `ValueError: file descriptor cannot be a negative integer (-1)` when one thread closed a
    socket while the other thread was selecting on it.
1940
1941 **Fix:** Added `ValueError` exception handling around `select.select()` and `recv()` calls. When a file
    descriptor becomes negative, the thread breaks out of the forwarding loop cleanly instead of crashing.
1942
1943 #### 15.12.8 Docker Healthcheck: Multi-line Python in CMD Array
1944 **Problem:** The socks-proxy healthcheck used a `CMD` array with a multi-line Python string. YAML collapsed the
    newlines, causing the `#` comment to comment out the rest of the code and the `assert` statement to be
    mangled into `as`.
1945
1946 **Fix:** Changed from `["CMD", "python3", "-c", "..."]` to `["CMD-SHELL", "python3 -c '...']` with a single-
    line Python expression. Uses a real SOCKS5 handshake (sends `[5, 1, 0]`, expects `[5, 0]`) instead of a
    simple TCP port check.
1947
1948 #### 15.12.9 Graceful Shutdowns Leave DNS Hijacked
1949 **Problem:** The gateway overrides the macOS host's DNS settings (via `networksetup`) to route queries to the
    AdGuard container. Because macOS persists `networksetup` changes to disk, shutting down the Mac while the
    gateway is running causes the Mac to boot up later with hijacked DNS. Since the Colima VM and Docker
    containers don't auto-start on boot, the user would have no internet access until they manually run the
    toggle script.
1950
1951 **Root Cause:** The `toggle.sh` script exits after the gateway starts. There was no background daemon listening
    for macOS system shutdown signals to revert the network settings.
1952
1953 **Fix:** Wrapped the background `caffeinate` process (used to prevent system sleep) in a bash `trap` command.
    The trap listens for `SIGTERM`, `SIGINT`, and `SIGHUP`. When the user initiates a graceful shutdown, macOS
    sends `SIGTERM` to the background trap, which instantly executes `recover.sh` to flush the host DNS back to
    `1.1.1.1` right before the machine powers off.
1954 *Note:** We explicitly use `recover.sh` instead of `toggle.sh` because during a shutdown, the OS limits process
    cleanup time and is already independently sending termination signals to Docker and Colima. Trying to run
    `docker compose down` inside a shutdown trap creates a race condition that can hang the script until macOS
    forcefully `SIGKILL`s it. `recover.sh` abandons container management and surgically flushes the macOS
    network settings in milliseconds, guaranteeing completion before the OS pulls the plug.
1955
1956 #### 15.12.10 Linux Native Wi-Fi Bouncing
1957 **Problem:** The generated `scripts/recover-linux.sh` incorrectly retained the macOS `networksetup` commands,
    which would crash and fail to bounce Wi-Fi on Linux hosts.
1958
1959 **Fix:** Swapped the macOS logic for native Linux commands. The script now attempts `nmcli radio wifi off/on`
    first, and gracefully falls back to `ip link set wlan... down/up` if NetworkManager is unavailable.
1960
1961 #### 15.12.11 TS_AUTH_ONCE Cloud Footgun & Ephemeral Keys
1962 **Decision:** The compose file uses `TS_AUTH_ONCE=true` to prevent authentication loops. However, on headless
    cloud VPS deployments, if a standard auth key expires (usually 90 days), the container will silently drop
    off the mesh. We documented this trade-off explicitly in `docker-compose.yml` and recommended the use of **
    Tailscale Ephemeral Keys** for cloud deployments, which automatically clean themselves up and bypass this
    issue.
1963
1964 #### 15.12.12 Hardcoded AdGuard Credentials
1965 **Decision:** AdGuard is deployed with the hardcoded credentials `admin / nullexit`. This is perfectly safe
    behind a NAT on a local macOS laptop. However, if deployed on a cloud host with exposed ports, this becomes

```

a severe security risk. We added a stark warning comment in `docker-compose.yml` to ensure cloud users change this before deployment.

15.12.13 Routing Fix: 30-second polling vs Netlink Socket

Decision: Instead of building a complex Netlink socket listener (`ip monitor`) to react instantly to iptables and routing flushes from Gluetun, we retained the 5-second `sleep` loop in `scripts/routing-fix.sh`.

Reasoning: Building a netlink listener in pure bash on Alpine is incredibly brittle and complex (especially for iptables events). The 30-second polling loop is bulletproof. The only trade-off is a potential 1-4 second stutter if Gluetun reconnects, which was deemed an acceptable edge case for absolute reliability.

15.12.14 Cache Poisoning and URL Rot in scripts/sync-rules.py

Problem: If a remote blocklist URL went permanently offline (404 Not Found), the Python `urllib` request would return the 404 HTML string. The script would aggressively regex-parse this HTML, find no domains, and overwrite the healthy local cache with a 0-domain file. This effectively disabled ad-blocking until the URL was fixed.

Fix: Implemented a defensive programming sanity check in `scripts/sync-rules.py`. Before overwriting the cache, the script now verifies that the compiled domain count is greater than 10 (and 1 for IP feeds). If it falls below this threshold (indicating a catastrophic 404 failure across multiple URLs), it aborts the cache overwrite, raises a `ValueError`, and keeps the previous day's healthy cache intact.

15.12.15 Gluetun nf_tables Parser Crash (Bash Interpolation Failure)

Problem: Attempting to make the TCP MSS dynamically configurable by using a standard bash variable inside `post-rules.txt` (`iptables ... --set-mss \${GATEWAY\_MSS:-1120}`) caused the Gluetun container to catastrophically crash during startup. Gluetun's internal parser executes `post-rules.txt` line by line without a shell, meaning the variable was never expanded. It literally attempted to inject the string `\${GATEWAY\_MSS:-1120}` into iptables, resulting in a fatal `bad value for option "--set-mss"` error.

Fix: Removed the bash variable from `post-rules.txt` and reverted it to a pure hardcoded integer. To retain dynamic `.env` configuration, we moved the interpolation logic to `toggle.sh`. Right before Docker starts, the host script parses `GATEWAY\_MSS` from the `.env` file and uses a cross-platform `sed` command to dynamically overwrite the integer directly inside `post-rules.txt`. This perfectly mimics manual configuration and completely bypasses Gluetun's strict parser.

15.12.16 AdGuard Filter Redundancy & Memory Optimization

Problem: AdGuard Home does not perform cross-list deduplication in memory. When it loads its own native subscription filters (e.g., `AdGuard DNS filter`) alongside our massive `compiled\_rules.txt`, overlapping rules are loaded twice, wasting significant RAM in the Colima VM. The UI would show ~500k rules, representing the sum of all lists rather than unique domains.

Fix: Updated `scripts/sync-rules.py` to intelligently cross-reference AdGuard's configuration and deduplicate our list against it.

- The script now reads `adguard/conf/AdGuardHome.yaml` to dynamically fetch the exact URLs of any enabled native AdGuard filters.
- The parsing engine was upgraded to normalize basic AdGuard syntax (`||domain`) back into raw base domains during the build process.
- The script subtracts the native AdGuard domains from our custom blocklist *before* compiling, immediately purging ~83,000 completely redundant rules.
- The normalized base domains then pass through our subdomain optimizer, which squashes an additional ~50% of the remaining rules.

This reduces the final compiled output from ~325k to ~281k rules on the `heavy` profile, saving ~45k rules from being loaded into memory twice and reducing the overall RAM footprint of the `adguardhome` container.

15.12.17 Kernel-Level IP Blocklist (Threat Intelligence Firewall)

Problem: DNS sinkholing cannot stop malware or botnets that bypass DNS entirely by hardcoding direct IP addresses (a common technique for C2 communication). These connections pass straight through AdGuard unseen.

Fix: Added a second compilation pipeline to `scripts/sync-rules.py` that runs on every startup alongside the DNS pipeline.

- The compiler concurrently fetches four curated threat-intelligence feeds: **Feodo Tracker** (abuse.ch botnet C2 IPs), **Spamhaus DROP** (IPs allocated to criminal organizations), **Emerging Threats** (Proofpoint active C2 and scanners), and **CINS** (active brute-force sources).
- All entries are normalized via Python's `ipaddress` module. Any IP or CIDR that overlaps with RFC1918 private ranges, loopback, link-local, or the Tailscale CGNAT range (`100.64.0.0/10`) is stripped to prevent accidentally locking users out of their own LAN or mesh.
- The cleaned list (16,721 unique IPs/CIDRs) is written as an `ipset restore` file using an **atomic swap** pattern: `create\_new` populate swap with live `destroy\_new`. This guarantees the live `block\_malicious` ipset is never empty during a reload.
- `scripts/routing-fix.sh` watches the file's mtime every 30 seconds. On change it runs `ipset restore` and idempotently re-injects `FORWARD DROP` rules for both `src` and `dst` into both iptables backends (nftables + legacy).
- `docker-compose.yml` mounts `adguard/work/userfilters/` into `routing-fix` as `/userfilters:ro` and adds `rule-compiler: service\_completed\_successfully` to its `depends\_on`, eliminating the race condition where routing-fix could start before the file existed.

Memory cost: The entire 16,721-entry ipset costs only ~1.6 MiB of kernel memory negligible in the 600MB VM budget.

2003
2004 ##### 15.12.18 Incident Post-Mortem: Discarded Docker Exec Errors in logs (July 10, 2026)
2005 **Symptom:** When the `warp` container is stopped, dead, or failing to connect to its endpoints, the `toggle.sh`
and `recover.sh` connectivity health checks fail, but no details are printed to `output.log`. The logs
only show generic `FAIL` outputs, making it hard to see if the failure was a network timeout, Docker daemon
error, or a missing binary.

2006
2007 **Root Cause:** Overly broad error silencing. The `docker compose exec` commands in the health checks (Check 3
in `toggle.sh` and the traffic check in `recover.sh`) both redirected stderr to `/dev/null` (`>/dev/null`
and `2>/dev/null` respectively), completely throwing away any diagnostic error messages produced by Docker
or `wget`.

2008
2009 **Fix (July 10, 2026):** Redirected stderr of the `docker compose exec` check commands to `output.log` (`2>>
output.log`). This ensures that any command execution failures or connection timeout details are logged.

2010
2011 ##### 15.12.19 Incident Post-Mortem: Toggle Decided STOP/START From a Live Probe, Aborting When Docker Was Down
(July 13, 2026)
2012 **Symptom:** Running `./toggle.sh` (or `--restart`) while the Colima VM / Docker daemon was **not** running
caused the script to hang for ~15 seconds and then abort during the START phase without ever booting Colima
. With `DEBUG\_TRACE` the lifecycle breadcrumb showed `toggle.sh EXIT: code=1 phase=start`, and the trace
showed execution jumping straight from the START-branch entry to the `ERR` trap (`cleanup\_handler ERR`)
with the START body never executing. Historically this silent failure occurred 57 in `output.log`. Spam-
clicking the toggle during the resulting window (each retry rejected by the lock guard, then finally
succeeding) is what produced the earlier "pile of stacked toggle processes" and Wi-Fi-bounce confusion.

2013
2014 **Root Cause:** `toggle.sh` decided whether to STOP or START by calling `is\_gateway\_active()` (`scripts/common.
sh`), which **live-probes** the running system its first step is `run_with_timeout 15 docker compose ps --
status running`. When `/var/run/docker.sock` is absent (Colima down), that call blocks for the full 15 s
timeout and returns non-zero. Combined with the script's global `set -e` and `ERR` trap, a non-zero result
evaluated at the `if is\_gateway\_active; then else fi` boundary aborted the whole script **before** the
`else` (START) branch body ran precisely the branch whose job is to start Colima. Design-wise the deeper
flaw is that a **disable** should not depend on whether the gateway is **currently** healthy; it should act
on whether the gateway is **supposed** to be running (persisted intent). The correct signal already existed
`MARKER\_FILE=/tmp/nullexit-gateway-active.marker`, written at the end of a successful START and cleared on
STOP but `toggle.sh` only wrote/cleared it and never read it for this decision. **Note:** this was a long-
standing latent bug, not a regression from the July 2026 `common.sh` platform-function unification or the
`DEBUG\_TRACE` work; those changes only made the previously-silent abort **visible** (via the `EXEC:/'EXIT:`
breadcrumbs and xtrace).

2015
2016 **Fix (July 13, 2026):** Introduced `gateway\_should\_be\_running()` in `common.sh` and routed all four decision
sites through it (`toggle.sh` main decision + `--restart` pre-check; `scripts/toggle-linux.sh` main
decision + `--restart` pre-check). It decides from persisted intent:

2017 1. `toggle.sh` captures the marker's existence into `GATEWAY\_MARKER\_AT\_STARTUP` **before** the top-of-script "
defensive stale-marker clear" wipes it, and the helper honors that captured value (falling back to the live
marker file for callers that run before the clear, e.g. the `--restart` pre-check).

2018 2. Marker present STOP path; marker absent START path. This is instant and cannot hang or abort.

2019 3. Only when the marker is absent **and** the Docker socket is actually reachable (`/var/run/docker.sock` or
`~/colima/default/docker.sock` on macOS) does it consult `is\_gateway\_active` as a non-fatal reconciliation
fallback (covers a marker lost to a `/tmp` wipe while containers are genuinely up). A dead socket short-
circuits to "stopped" with no probe, so `set -e` can never be tripped.

2020 4. The STOP path's `docker compose down` is now `|| true`-guarded so a disable always completes even with
Docker down; the START path's `docker compose up` is deliberately left unguarded (a failed start **should**
trigger cleanup). On Linux the marker isn't maintained, so the helper degrades to the `is\_gateway\_active`
fallback fast there because native Docker has no Colima VM to hang.

2021
2022 **How we got here (methodology note):** This one is worth recording because the **process** mattered as much as
the patch. The failure had been happening silently for a long time the log simply ended after a teardown
with no error, so there was nothing to chase. We first attacked the **observability gap** rather than
guessing at the cause: we wrapped the heavyweight lifecycle commands (`colima start`, `docker compose up/
down`) so their output and exit codes always land in `output.log`, added an `EXIT: code= phase=` breadcrumb
that fires even when the process is killed, and put a full opt-in xtrace behind `DEBUG\_TRACE`. Only **then**
did we reproduce the failure and this time the trace said, unambiguously, `EXIT: code=1 phase=start` with
the START body never entered. The instrumentation didn't fix anything, but it converted an invisible abort
into a precise, reproducible fact. (Lesson, consistent with 15.12.18 and the LUKS post-mortem referenced
in 15.11: **a mechanism that fails silently is worse than one that fails loudly** so we make it fail
loudly first.)

2023
2024 The root-cause leap, though, came from a **design-smell** intuition, not the stack trace: the observation that
it is nonsensical for a **disable** to first check whether the gateway is **currently** online a teardown
should act on whether the gateway is **supposed** to be running, not probe a subsystem to find out. That
instinct turned out to name the bug exactly. The smell was a **conflated responsibility** with an inverted
dependency: to decide whether to **start** the gateway, the code first had to **talk to** the gateway (via
Docker) the very thing START is responsible for bringing up so the check was guaranteed to be unavailable
in precisely the cold-start case that mattered. A second tell was **false symmetry**: enable (builds state
) and disable (tears it down) are not symmetric operations, yet both ran the identical expensive probe;
whenever two operations that should differ share suspiciously identical machinery, the code is usually
lying about the structure of the problem, and reality eventually calls the bluff. The fix invented nothing
the honest signal (`MARKER\_FILE`, literally "the gateway is supposed to be up") already existed and was
maintained in all the right places; it was simply never consulted for the decision it was made for. We just
pointed the decision at the answer the codebase already knew. General principle worth keeping: **design**
smells here are load-bearing they tend to **be** latent bugs, not merely cosmetic, and are worth chasing

on intuition even before a trace confirms them.

****Acceptance verified (static + live):**** with marker present STOP (instant, even with Docker down); with marker absent + Docker down START (0 s, no `set -e` abort); a real run now proceeds past init to the gateway-status decision. `bash -n` clean on all three scripts; no new `shellcheck` findings; re-signed signatures (toggle.sh, common.sh, toggle-linux.sh are in the signed set) and `crypto.sh --verify` exits 0.

****Follow-up (same day)** a `set -e` regression the fix introduced, and why only a live run caught it. The first cut of the marker capture wrote the intent as a full `if [ -f "\$MARKER\_FILE" ]; then GATEWAY\_MARKER\_AT\_STARTUP="yes"; else GATEWAY\_MARKER\_AT\_STARTUP="no"; fi`. On macOS `/bin/bash` 3.2, with `set -e` active and the `DEBUG\_TRACE` xtrace redirect (`exec 2> >(tee)`) in effect, that construct **aborted the script at init** the moment the marker was absent the false `[ -f ]` status leaked through the compound and `set -e` killed the run (`EXIT: code=1 phase=init`), before the gateway logic ran at all. Notably this did **not** reproduce in isolated `bash -c` snippets of the same block it only surfaced in the script's real runtime context so we had to bisect against the actual `toggle.sh` (swap the block for a trivial assignment the script sailed past init) to prove causation. Fix: write it as a default assignment plus an `if` **without an `else`** (`GATEWAY\_MARKER\_AT\_STARTUP="no" then `if [-f "\$MARKER_FILE"]; then GATEWAY_MARKER_AT_STARTUP="yes"; fi`), which yields status 0 when the condition is false. Lesson reinforced: `set -e` on legacy bash is genuinely treacherous around conditionals, and the observability work paid for itself again the breadcrumb pinned the abort to init instantly, and a live run (not static checks) was the only thing that exposed a heisenbug invisible to isolated reproduction.

15.12.20 Honey-Port Tripwire Accumulation (July 14, 2026)

****Symptom:**** After many toggle-ons, `ps` showed a pile of idle `bash ./toggle.sh` subshells, each blocked on an `nc -l localhost <port>` listener one leaked per toggle-on, never reaped.

****Root Cause:**** The Honey-Port Tripwire arms a fresh random port each START via a disowned `( nc -l ) &` subshell. `nc -l` blocks until a connection that (by design) almost never comes, so each toggle-on leaks one idle listener; because the port is random, successive listeners never collide and simply stack up over time.

****Fix:**** `toggle.sh` now reaps the previous tripwire before arming a new one `pkill -f "nc -l localhost" ( macOS ) / `pkill -f "nc -l .127.0.0.1" + "nc -l localhost" (Linux) immediately before `HONEY\_PORT=\$(get\_free\_port)` at both arming sites. No `sudo` needed (the listeners are user-owned). Net effect: at most one tripwire alive at a time instead of one per toggle. (The same `pkill` pattern cleared the backlog that this session's ~15 test toggles produced.)

15.12.21 Post-Wake Self-Feedback Loop: The Watcher Re-Fires On The Route Change It Caused (July 14, 2026)

****Symptom:**** After a container restart (or wake), `output.log` showed `recover.sh --post-wake` launching ~6 times in a row, one launch roughly every ~40s, each exiting `exit=0` recovery *worked* every time yet kept re-firing "until it worked." Every launch was triggered by `NET: changedKey State:/Network/Global/IPv4`.

****Root Cause** a self-feedback loop, NOT PF. ****scripts/watcher.sh** Listener 2 watches `State:/Network/Global/IPv4` via `scutil n.watch` and launches `recover.sh --post-wake` on any change. But post-wake re-asserts the exit node (`recover.sh` ~L367, `tailscale set --exit-node`), and even though `set` is deliberately used instead of `--reset` to minimise churn (recover.sh ~L360-366), **re-asserting the exit node re-writes the host default route and the default route IS `State:/Network/Global/IPv4`**. So each run mutates the exact key the watcher triggers on. The self-triggered change lands ~15-45s after the run finishes; `run\_recover()`'s old 10s debounce (reset to the run's *finish* time at the tail of the function) had long expired by then, so the watcher re-fired. The loop self-terminates only once `tailscale set` becomes a no-op (route already correct) no route change no re-trigger.

****Why the debounce couldn't stop it:**** the loop period (~40s = one full post-wake run + settle) far exceeds the 10s debounce, which therefore only ever caught macOS's rapid *duplicate* scutil notifications for a single change (the ~0.2s doublets), not the self-trigger.

****Not PF:**** post-wake never calls `pfctl` (the kill-switch is armed once, in `scripts/common.sh`, not per cycle); no PF-enable line falls inside the loop window. The "No ALTQ support in kernel" warning is normal macOS pfctl noise (Apple ships pfctl without ALTQ compiled in), and "Could not capture PF reference token" is a separate benign ref-count bookkeeping wart both unrelated to the loop.

****Fix:**** `scripts/watcher.sh` `run\_recover()` now arms a **post-recovery settle cooldown** (`/tmp/nullexit-watcher.settle-until`, `NULLEXIT\_POSTWAKE\_SETTLE\_SECONDS` default 45s) *after* each post-wake run, and skips launching while `now < settle-until`. Because it is armed only after a recovery runs, a genuine first wake/roam is still handled immediately; only the Global/IPv4 change the run caused itself is absorbed. A real new roam during the window is delayed at most `POSTWAKE\_SETTLE\_SECONDS`, an acceptable trade for killing the loop. Watcher-side only `recover.sh` is unchanged. Severity was low (self-correcting, healthy every iteration, no leak/outage; cost was wasted ~28s runs + log noise).

15.12.22 FaceTime (and Real-Time P2P Media) Over the Double-Tunnel: Why It Breaks + the Split-Route Plan (July 15, 2026)

****Symptom:**** A FaceTime call placed while nullexit is up fails to connect.

****Diagnosis** nullexit is NOT blocking it; every layer passes the traffic (all verified live):

- ****DNS:**** no Apple/FaceTime domain is sinkholed `dig` via the gateway (`100.100.21.8`) returns real Apple A/CNAME records, none `0.0.0.0`/`127.0.0.1`. The `<no answer>` domains (`gateway.push.apple.com`, `fmn.apple.com`, `identityservices.apple.com`, `fdr.apple.com`) return empty on `1.1.1.1` too CNAME/non-A, not our block.

- ****routing-fix FORWARD:**** the exit-path rule `-i tailscale0 -o tun0 -j ACCEPT` carries ~203K pkts / 73M bytes and accepts *everything*.

2054 - ****Country / threat blocklists**** (`block\_il`, `block\_kp`, `block\_malicious`): ****0**** packets dropped.

2055 - ****WARP (gluetun) egress:**** `-A OUTPUT -o tun0 -j ACCEPT` + `-A POSTROUTING -o tun0 -j MASQUERADE` all
outbound UDP (any port) leaves and is SNAT'd through WARP.

2056

2057 ****Root cause** double symmetric NAT ending at Cloudflare WARP. Path: `host Tailscale (MASQUERADE) WARP (MASQUERADE) Cloudflare shared egress`. WARP is a CGNAT-style shared egress with ****no** inbound UDP mappings
******, and the double-MASQUERADE presents as ****symmetric NAT**** the exact condition that defeats FaceTime's
STUN/ICE UDP hole-punching. No P2P path can form; the Apple-relay fallback is unreliable through the
stacked tunnel, and the 12801200 MTU squeeze degrades media. This is inherent to routing real-time P2P
media through WARP, ****not**** a firewall rule. (Zoom/WhatsApp ***calls*** hit the same wall.)

2058

2059 ****Latent finding (separate bug) Closed:**** the `FORWARD` chain's `-i tailscale0 -p udp --dport 443` (QUIC) and
`--dport 853` (DoT) `REJECT` rules were **shadowed** by the earlier -i tailscale0 -o tun0 -j ACCEPT` (
first-match wins) verified **0 pkts** on each at the time. Fixed by moving those port-REJECT` rules in `
post-rules.txt` **before** the tailscale0 tun0` / `tun0 tailscale0` ACCEPT rules. Re-verified live post-
fix: the REJECTs now sit ahead of the ACCEPTs in iptables -L FORWARD --line-numbers` and accumulate hits.`

2060

2061 ****Fix plan (chosen: Option 2** narrow, always-on, opt-in split-route). Implemented and shipped enabled by
default:******

2062 - ****Mechanism:**** add more-specific host routes for FaceTime's Apple `/16`s` via the ****physical**** gateway
(`172.17.0.1/16en0``). Longest-prefix match beats Tailscale's exit-node `0.0.0.0/1` + `128.0.0.0/1` routes,`
so only those `/16`s` leave ****direct**** while everything else stays double-tunneled. Mirrors ```
`add_warp_bypass_routes`.`

2063 - ****Wiring:**** `add_apple_split_routes()` / `remove_apple_split_routes()` in `common.sh``; called in toggle START
(after exit-node assert) + `recover.sh --post-wake` (survives roam) + removed on STOP; guarded by ``
`FACETIME_SPLIT_ROUTE` in .env` .env.example` ships it false` (opt-in, privacy-first default for new
installs), enabled here after the tradeoff below was accepted and the CIDR verified against a real call.`

2064 - ****Privacy tradeoff:**** whatever `/16`s` go direct see the ****real ISP IP**** (not WARP). Apple already ties
traffic to the Apple ID, so Apple-direct is the accepted scope; user chose ****narrow**** (FaceTime-infra `/16`s`
only) over all-`17.0.0.0/8` to minimise the surface.`

2065 - ****CIDR list** finalised from an empirical capture: ****host-side` sudo tcpdump -n -i <exit-utun, e.g. utun6> net**
`17.0.0.0/8` during a real FaceTime call. The capture identified **17.249.0.0/16** as the load-bearing`
media/relay range this is the value shipped in `.env.example`s` FACETIME_DIRECT_SUBNETS``, verified
working end-to-end on a real call. (Earlier DNS-only guesses `17.57/16` APNs push/ringing, `17.248/16``
iCloud/FaceTime gateway, `17.253/16` aapling push CDN, `17.157/16` GSA auth` were signaling-only ranges and`
did not carry media; the capture superseded them.) See P2P note 16.4.

2066

2067 **####** 15.12.23 setup.sh Never Generated NULLEXIT_SEED Every Fresh Install Hard-Failed Its First toggle.sh (July
18, 2026)

2068 ****Symptom:**** README "Cryptographic Integrity Verification" and this doc's own `.env` variable table both`
claimed `NULLEXIT_SEED` is "generated by `setup.sh`." It wasn't. `scripts/setup-common.sh`s` .env`-writing
step never referenced NULLEXIT_SEED` at all, and .env.example` only offered a placeholder telling the
user to run openssl rand -hex 32` by hand. .signatures` is committed to git (tracking the maintainer's
own local seed), so a fresh clone's toggle.sh` would call scripts/crypto.sh --verify`, find no`
NULLEXIT_SEED` in the new .env` at all, and hard-fail with a cryptic seed error on the very first run
with nothing in the setup flow or its closing instructions telling the user a crypto.sh --sign` step was
needed, or why.`

2069

2070 ****Compounding bug found in the same code path:**** `setup-common.sh`s` .env` writer also defaulted KILL_SWITCH`
to false` for brand-new installs (EXISTING_KILL="false", only overridden if an .env` already existed
with a different value) the exact inverse of the Leave true, this is the core leak guarantee" invariant
enforced everywhere else (.env.example`, `scripts/profiles.sh`)). Every fresh install would have shipped
with the kill-switch off by default, not just failed on first boot.`

2071

2072 ****Fix:**** `setup-common.sh` now generates NULLEXIT_SEED` via openssl rand -hex 32` when .env` doesn't already
have one (preserving it on re-runs, same pattern as every other preserved flag), writes it into the new .
env`, and calls scripts/crypto.sh --sign` immediately after writing .env` so a fresh install's first`
toggle.sh` finds a valid seed and matching signatures instead of failing cold. EXISTING_KILL` now defaults
to "true".`

2073

2074 **####** 15.12.24 Honey-Port Subshell Holds stdout/stderr Open Piping `toggle.sh` Hangs Long After It Finishes (July
18, 2026)

2075 ****Symptom:**** `./toggle.sh --restart 2>&1 | tail -120` appeared to hang indefinitely 10+ minutes with zero`
output even though `output.log` showed the restart itself completed cleanly (EXIT: code=0 phase=start`)
in about a minute and all 5 containers were healthy. ps` showed the top-level bash ./toggle.sh` process
still alive, reparented to launchd (ppid=1`), sitting in a __wait4` syscall (confirmed via sample` with
its only child being the [[Honey-Port Tripwire]]`'s nc -l localhost <port>` listener.`

2076

2077 ****Root cause:**** the tripwire's arming code (`toggle.sh` ~L1704) redirects only the *inner*` nc -l` command's
own stdout/stderr to /dev/null` the **subshell process itself** (( if nc -l ; then fi ) &`)) inherits
fd 1/2 unchanged from its parent. Since nc -l` never returns (by design it's waiting for a port-scan that
almost never comes), that subshell stays alive indefinitely, and because it still holds the *write end*` of
whatever pipe its parent's stdout was connected to, any reader on the other end (| tail`, `| tee`, a CI
log capture, anything) never sees EOF even though the disowned process is functionally idle and the script
that spawned it exited normally minutes earlier. This is a distinct manifestation of the listener-lifetime
issue already known from 15.12.20 (accumulation): that fix (kill -f "nc -l localhost` before arming)
reaps the *previous*` listener, but does nothing about the *current*` one's inherited file descriptors.`

2078

2079 ****Workaround used:**** killed only the reader (`tail` on the stuck end of the pipe safe, since it doesn't touch`
the gateway, Docker, `caffeinate`, or the tripwire itself, which is meant to keep running. **Not yet fixed`
at the source** see `[[Future - Pluggable Egress|18 TODO]]` candidate: redirect the subshell's own stdout/`

```

stderr (`( ) >/dev/null 2>&1 &`) at arm time so it can no longer hold a caller's pipe open. Already
flagged as a sharp edge in the [[toggle.sh]] component doc ("Never pipe it through a reader") this
incident is the first confirmed real-world hit, not just a theoretical gotcha.
2080
2081 ##### 15.12.25 Incident Post-Mortem: WARP Watcher False-Kills the Gateway on a Wi-Fi Drop; Wi-Fi Auto-Rejoin (
      July 2223, 2026)
2082 **Symptom:** Overnight the gateway went fully down and stayed down (~22:46 EDT until it was manually brought
      back). No mesh device including the S24 had connectivity during the window. The user flagged it as
      suspicious ("what killed the watcher").
2083
2084 **Root cause (not malicious):** the in-container WARP Watcher (15.6.2) polls `cdn-cgi/trace` and cannot
      distinguish "WARP tunnel failed" from "the host physical link (en0) is down" both surface as `warp=off`,
      because with no internet there is no tunnel *and* no trace response. The host lost Wi-Fi (`
      ipconfig_get_packet(en0) failed: not found` repeated in `output.log`), the watcher counted its 6
      consecutive `off` polls, and `recover.sh` tore the whole gateway down a gateway that would have healed on
      its own once Wi-Fi returned. A secondary design gap amplified it: `recover.sh --post-wake` releases the
      watcher inhibit marker on a *timeout* (after `wait_for_dhcp_settle` gives up) rather than waiting for `warp
      =on`, so on a drop the inhibit lifted while WARP was still legitimately down.
2085
2086 **Fix, part 1 pause, don't kill.** The WARP Watcher now checks the physical uplink (`ipconfig getifaddr
      $phys_iface`) before treating a `warp=off` as a tunnel failure; empty/`169.254.*` PAUSE (reset counter,
      sleep, `continue`) instead of running `recover.sh`. Leak-safe by construction (no routable IP = no egress
      path). Full detail in 15.6.2 point 5.
2087
2088 **Fix, part 2 actually reconnect the Wi-Fi.** Pausing stops the false kill but the Mac still has no internet
      if macOS auto-join is off; it "just sits there." macOS (Darwin 25) redacts the live SSID from every script
      unless the process holds Location, and the user explicitly refused Location (15.11.10), so the *current*
      network is undetectable. Solution: don't detect it **remember** it. `scripts/wifi-rejoin.sh` keeps a user-
      managed list of network names in `.last_wifi_ssid` (`set`/`forget`/`list`, gitignored, unsigned) and re-
      associates the first remembered one via `sudo -n networksetup -setairportnetwork "$iface" "$ssid"`, which
      pulls the WPA password from the login Keychain automatically (confirmed by live drop test) and only ever
      targets a network the user explicitly remembered no scanning, no public-AP hopping. `rejoin` arms a down-
      timer on first call, holds a 15s **grace** period (let DHCP self-heal a same-AP blip first), then attempts
      with **progressive backoff** (3060120240cap 300s) and **gives up after 6 tries** per down-episode so it
      never churns a network that can't come up (captive portal, DHCP blocked, out of range); `reset` clears the
      timer once the link returns.
2089
2090 **Fix, part 3 put the driver where it can't die with the gateway.** The first cut wired `rejoin` into the WARP
      Watcher only but that watcher runs *solely while the gateway is up*, so a Wi-Fi drop while the gateway
      was down/off left nothing to reconnect the Mac (observed directly: a retest "just didn't do shit" because
      the gateway had shut down at 06:54:55Z and its watcher was gone). The rejoin driver was therefore added as
      a 4th listener in the **always-on** `scripts/watcher.sh` daemon (15.5.9): a 15s poll of the Wi-Fi interface
      that calls `wifi-rejoin.sh rejoin` when it has no routable IP and `reset` when it does. The WARP Watcher
      still calls the same helper while the gateway is up; the two coordinate through shared `/tmp` markers
      rather than double-associating. Net result: Wi-Fi reconnects whether the gateway is up, down, on or off which
      is what the user's original report ("sometimes Wi-Fi disconnects and it just sits there without internet")
      actually required.
2091
2092 **Self-inflicted detour worth recording:** an intermediate retest broke the network *after* reconnecting
      because editing signed core files (`toggle.sh`, `docker-compose.yml`) without re-running `./scripts/crypto.
      sh --sign` made `recover.sh --post-wake` fail its integrity check and abort (exit 1) so recovery never ran
      and `wifi-rejoin` churned every 30s. Lesson reinforced: **re-sign after every edit to a signed file** ( "
      Cryptographic Integrity Verification"), and the give-up cap above exists partly to make such a churn self-
      limiting.
2093
2094 ##### 15.12.26 Logging Unified to UTC (July 23, 2026)
2095 **Symptom:** `output.log` interleaved UTC and local (EDT) timestamps, and the AdGuard query log was in local
      time while `blocked.log` was UTC making forensics (e.g. "was the S24 connected at 23:00?") error-prone,
      because the same wall-clock instant appeared under two different hour labels. The user asked for "one thing
      , utc or est, only one."
2096
2097 **Root cause:** `TZ=America/New_York` was set on every service in `docker-compose.yml`, but it is only honored
      by images that ship `tzdata` (AdGuard rendered EDT); Alpine-based images without `tzdata` (routing-fix,
      the Go logger) silently fell back to UTC. So the timezone of a line depended on which container emitted it.
2098
2099 **Fix:** standardized on **UTC everywhere**. All `TZ=` entries in `docker-compose.yml` were set to `UTC`, and `
      scripts/logger/main.go` now formats with `time.Now().UTC()` and appends an explicit `UTC` marker to each `
      blocked.log` line so the zone is unambiguous on the page. `docker-compose.yml` is a signed file and was re-
      signed. Host scripts (`watcher.sh`, `wifi-rejoin.sh`, `recover.sh`) already emit UTC via `date -u`. **
      Forensics caveat:** log lines written *before* 2026-07-23 still carry the old UTC/local split see the [[
      log-timezones]] note when reading historical entries.
2100
2101 ---
2102
2103 # Part IV Observations, Changelog & TODO
2104
2105 ## 16. Unverified Observations
2106
2107 ### 16.1 AdGuard Returns Two Different Sinkhole IPs (Unverified)
2108
2109 > **Status:** Observed but not formally verified. Needs a deeper audit of the rule-compiler sources to confirm.**

```

```

2110
2111 **Observation (July 10, 2026):** When querying AdGuard Home for blocked ad domains, some domains resolve to
    `0.0.0.0` and others resolve to `127.0.0.1`. Both are blocked, both cause connection failure, but the
    different responses were unexpected.
2112
2113 ```
2114 doubleclick.net          0.0.0.0    (blocked)
2115 ads.google.com           127.0.0.1 (blocked)
2116 pagead2.googlesyndication.com 0.0.0.0 (blocked)
2117 scorecardresearch.com    127.0.0.1 (blocked)
2118 ```
2119
2120 **Likely Explanation (Unverified):** There are three historical schools of thought on the "correct" DNS
    sinkhole response, and AdGuard faithfully preserves whichever the source blocklist used:
2121
2122 | Sinkhole IP | Convention | Origin |
2123 |---|---|---|
2124 | `0.0.0.0` | AdBlock-style lists (`\|\|domain`) | Modern DNS-level blocking; AdGuard's native format. Fails
    fast OS doesn't attempt a TCP connection. Works for both IPv4 and IPv6 without a separate rule. |
2125 | `127.0.0.1` | Hosts-file-style lists (`127.0.0.1 domain`) | 90s-era `/etc/hosts` tradition. Steven Black's
    hosts list (500k+ domains) still uses this. |
2126 | `NXDOMAIN` | Purist / Pi-hole default | Returns "domain doesn't exist." Semantically most accurate but
    requires browsers to handle NXDOMAIN gracefully. |
2127
2128 The dual-response behavior in this project is because `rule-compiler` pulls from both AdBlock-format sources (
    Hagezi, uBlock origin lists) and hosts-format sources (Spamhaus DROP, Steven Black, Feodo Tracker), and
    AdGuard returns whatever sinkhole IP the matching rule specifies.
2129
2130 **To Verify:** Run `docker compose exec tailscale cat /etc/hosts` to confirm no hosts-file rules are being
    injected at the container level, and check which specific AdGuard rule matched each domain via the AdGuard
    query log at `http://100.100.21.8:3000/#logs`.
2131
2132 ### 16.2 AGY CLI Appears to Generate Native macOS Notification Pop-ups (Unverified)
2133
2134 > **Status: Observed but source unverified. macOS security logs ruled out system-level origin.**
2135
2136 **Observation (July 10, 2026):** While running a DNS blocklist verification via the Antigravity CLI (`agy`), a
    command was proposed that included `malware.wicar.org` (a known-safe DNS blocklist test domain) in a `dig`
    loop. Shortly after, a macOS-style notification popup appeared describing a "malicious" or "blocked" event.
    The AGY CLI terminal session then closed.
2137
2138 **Investigation:**
2139 - **macOS XProtect / Gatekeeper logs:** Empty. Zero events in the 30-minute window around the incident.
2140 - **macOS Endpoint Security / System Policy logs:** Empty.
2141 - **Broad `log show` search for "malicious", "malware", "blocked":** Empty.
2142 - **Conclusion:** macOS itself did not generate the notification. No system security subsystem fired.
2143
2144 **Likely Explanation (Unverified):** The notification appears to have originated from **AGY CLI's own internal
    safety system**. AGY CLI is a native macOS application (not just a terminal process) and has access to the
    macOS Notification Center API (`UNUserNotificationCenter`). When its guardrails detected a domain
    containing the word "malware" (`malware.wicar.org`) in a proposed shell command, it likely:
2145 1. Blocked the command from executing
2146 2. Posted a native macOS-style notification via the Notification Center
2147
2148 This is surprising behavior for a CLI tool most terminal programs don't send GUI notifications but AGY CLI
    appears to bridge both worlds: it runs in a terminal but is packaged as a native app with full system API
    access. The notification would be indistinguishable from a system security alert at a glance.
2149
2150 **Notes:**
2151 - `malware.wicar.org` is the DNS/AV equivalent of the EICAR test file a deliberately benign domain that
    triggers blocklist/AV systems to verify they work. It was included in the test set to confirm the AdGuard
    blocklist was catching it. AGY's safety system is not aware of this distinction.
2152 - For future DNS blocklist testing, use only ad/tracking domains (e.g., `doubleclick.net`, `adnxs.com`) to
    avoid triggering AGY's guardrails.
2153
2154 ### 16.3 `system_profiler SPAirPortDataType` Longevity (Wi-Fi Detection)
2155 The Wi-Fi security detection used by `watcher.sh`'s `detect_lan_p2p_mode()` currently relies on `
    system_profiler SPAirPortDataType` (after the `airport` binary removal full history in 15.11.6). This
    works as of macOS Sequoia (15.x) without sudo and without redaction, but given Apple's ongoing Wi-Fi
    privacy clampdown, it may be locked down in a future release. If `security` comes back empty on a future
    macOS version while on Wi-Fi, detection silently falls back to `allow=false` (a safe default). The long-
    term fix would be a small Swift CoreWLAN helper compiled and bundled in the repo.
2156
2157 ### 16.4 Tailscale LAN P2P Succeeds Cross-AP But Fails Same-AP (Client Isolation + No NAT Hairpin)
2158
2159 **Observation (July 15, 2026):** On the residence Wi-Fi, a Galaxy S24 in the lobby reception 6 floors away,
    associated to a *different* AP establishes a **direct, low-ping** Tailscale P2P session to an iPhone 11.
    The **same* pair on the *same* AP / same floor / close range does **not** go direct. Counterintuitively, **
    farther apart = P2P works**. The effect shows for the iPhone 11 (a plain client) but **not** for S24macOS
    or S24the gateway container.
2160

```

2161 ****Mechanism:****

2162 - ****Same AP:**** the AP enforces ****client isolation**** (L2 device/device block on one radio). Both devices then share one public IP; STUN returns identical external mappings, so reaching each other would need the residence router to ****NAT-hairpin**** which most campus/consumer gear does not do direct P2P fails and falls back to a DERP relay.

2163 - ****Different AP (floors away):**** large residence networks ****segment per AP/area into distinct VLANs/subnets****, so the two devices sit on different L3 networks routed through the campus core, where client isolation does not apply and the STUN path works ****without**** hairpin clean direct P2P.

2164 - ****Why macOS / the container are immune:**** nullexit pins a ****direct WireGuard endpoint**** (Tailscale port `41642`, `direct 172.x`), so those peers already hold a negotiated direct path and do not depend on same-AP L2 discovery. A plain iPhone 11 has no pinned endpoint, so its path is entirely at the mercy of the Wi-Fi topology which is exactly why ***it*** exposes the isolation effect.

2165 ****Relevance:**** refines `watcher.sh`'s `detect\_lan\_p2p\_mode()` (which probes same-subnet AP isolation via a broadcast ping). The probe correctly flags same-AP isolation, but this observation adds that isolation is ****per-AP**** and cross-VLAN P2P can succeed even when same-AP fails so a single "LAN P2P allowed?" boolean under-describes a multi-AP network. No code change; documents expected behavior. Related: the FaceTime NAT analysis in 15.12.22 (same STUN/hairpin physics, different symptom).

2166 ****Follow-up (July 18, 2026)** measuring the same-AP DERP jitter, and whether P2P can be forced anyway:

2167 Burst-pinged (`tailscale ping -c 8`) an S24 on this exact same-AP-isolated network: 8/8 pong, 0% loss, but RTT swung 38-703ms (665ms spread) over `DERP(tor)`. The gateway's own path to the same Toronto DERP relay measured a clean, stable 29.1ms (`tailscale netcheck`), ruling out anything on the nullexit/gateway side the variance is entirely downstream, between the S24 and the relay.

2171 Switched the S24 to cellular (same physical location, different link) for a controlled comparison: 78-500ms (422ms spread) somewhat better, but still substantial. If the jitter were purely Android's Wi-Fi power-saving radio behavior, cellular should have looked dramatically cleaner; it didn't, which points at the real shared cause instead: both Wi-Fi and cellular radios drop to a low-power idle state between packets and pay a wake-up latency penalty on each new one, and DERP relaying rides over TCP (its own retransmit/ACK timing adds further variance that's invisible as "loss" at the WireGuard layer). This is inherent to relaying sparse/bursty traffic over ***any*** mobile radio link not a nullexit configuration issue, and not fixable from the gateway side.

2173 ****Can P2P be forced anyway, despite confirmed AP isolation?**** Confirmed with the user this specific network genuinely has AP isolation (not a `detect\_lan\_p2p\_mode()` false positive). Per the mechanism above: the block is enforced by the AP at Layer 2, below anything WireGuard/Tailscale operates at no VPN-layer trick un-isolates two clients the radio itself refuses to bridge. The only theoretical bypass is the residence router performing ****NAT hairpin**** (loopback) so both devices' identical STUN-mapped public IP routes back down through the router instead of laterally through the AP and this section already established "most campus/consumer gear does not do" that. AP-isolated deployments are, if anything, ***less*** likely to support hairpinning than typical consumer gear, since both are usually part of the same client-isolation security posture. ****Conclusion:** no, not reliably forceable. ****TAILSCALE_ALLOW_LAN_P2P=true** would not restore P2P here it would only reproduce the 15.7 SNAT endpoint-poisoning blackhole, trading "jittery over DERP" for "briefly offline entirely." Leave it `false`. (One untested, unguaranteed exception: some AP isolation implementations only block wireless/wireless traffic at the radio level, not wired/wireless a ****wired**** Ethernet connection for one side might bypass it on hardware that isolates that way. Depends entirely on how this specific AP implements isolation; not something to assume works.)

2175 **### 16.5 Working Theory iPhone Cross-Device Visibility Reaches Other Phones, Not macOS (Instagram Auto-Open, July 15, 2026)**

2177 ****Observation:**** Opening Instagram on a Galaxy S24 (specifically via ****Chrome**** on the S24, not Samsung Internet) auto-opens a tab on a nearby MacBook, but ****only when the S24 is physically close**** to the Mac it does not happen when the phone is away. ****Chrome** on the S24 has zero granted Android permissions (no Bluetooth, no location, no nearby-devices) this rules out Chrome itself performing any BLE/Nearby proximity scan, since Android enforces those permissions at the app level regardless of what the browser ships. This pushes the likely actor down to ****Google Play Services****, which holds system-level Bluetooth/nearby-device access independent of per-app grants and could be doing the proximity detection on Chrome's behalf (e.g. via Android's "Nearby" APIs) then handing the result to Chrome only for the actual tab-open. A purely local, short-range channel is still required to explain the proximity gating regardless of which component performs it; this also rules out any cloud-mediated mechanism (Chrome cloud tab sync, FCM push, Tailscale/WAN P2P per 16.4) since those are distance-independent. Live probes during the session (`system\_profiler SPBluetoothDataType` for a paired device, `dns-sd -B \_services.\_dns-sd.\_udp` for 8s) found ****no paired Bluetooth device and no Android/Google/Instagram mDNS service**** only the Mac's own Bonjour services (AirPlay, RFB, `\_companion-link`, Spotify Connect). Inconclusive: a BLE-advertisement-only channel (no formal pairing) would not show up in either probe.

2179 ****User's working theory:**** the iPhone (also on hand, on the same network/Apple ID as the Mac) makes ***other devices connected to it*** even an Android phone visible to nearby devices, but this visibility channel reaches ***other phones more readily than it reaches macOS****. I.e., the iPhone may be acting as a discovery bridge/relay between the S24 and the Mac rather than the S24 and Mac talking directly. Combined with the Play Services angle above, a refined version is: ****Play Services** on the S24 detects proximity to an Apple mesh (iPhone and/or Mac both signed into iCloud/AWDL-visible) via some cross-ecosystem discovery Google has built, rather than the S24 and Mac negotiating directly.

2181 ****Cross-ecosystem refinement:**** this isn't Google-only. Apple runs an equivalent OS-level background BLE scanning layer for its own Continuity stack (Handoff, Universal Clipboard, AirDrop discovery, Find My the latter using ***every nearby Apple device***, not just the owner's, as a passive BLE relay), implemented as system daemons (`bluetoothd`/`sharingd`/`rapportd`, already implicated in the unrelated AWDL wedge bug at

15.10) rather than anything gated by app-level permission. The key mechanism that makes the two ecosystems mutually visible: ****BLE advertisement packets are broadcast in the clear over open air**** Apple's Continuity BLE frames are semi-proprietary but unencrypted at the advertisement layer, so any BLE radio (including Android's, via Play Services once Android's own "Wi-Fi & Bluetooth scanning" system-level toggle is on) can passively observe "an Apple device is here" and react, without pairing or explicit permission from Apple's side. So both ecosystems maintain a BLE mesh-discovery layer that sits below their own per-app permission model, and each side's layer is passively readable by the other's radio which is a plausible bridge for a Play-Services-mediated Chrome trigger reacting to iPhone/Mac Continuity broadcasts.

****Status:**** Unverified. Not yet reproduced under an active capture the July 15 session ran the mDNS browse * before* confirming the phone was actively broadcasting, so it's not a clean negative result. To confirm: capture ``tcpdump -i en0 udp port 5353`` (mDNS) ****and a raw BLE advertisement capture**** (e.g. ``nRF Connect`` or similar BLE scanner app on a third device, or ``sudo btmon``-equivalent on Linux macOS does not expose raw BLE advertisement capture without a hardware sniffer) at the exact moment Instagram is opened on the S24 while close, with the iPhone present vs. absent, to isolate whether the iPhone's Continuity broadcasts are the trigger or the S24Mac channel exists independently of it. On the Android side, checking ``Settings Location Location Services Wi-Fi & Bluetooth scanning`` and whether ****Google Play Services**** (not Chrome) holds Bluetooth/nearby-device/location permissions would confirm or kill the Play-Services-as-actor theory. Related: 16.4 (same physical-proximity family of P2P/discovery quirks, different mechanism and inverted distance relationship).

17. Changelog / Recent Updates

Reflects the current branch's state. Each entry is a one-line summary + commit hash; read the commit message (and the linked Incident Log entry) for full detail. Full post-mortems live in 15.

- ****July 19, 2026 docs: dense-cellular field test**** Live burst-ping test of a mesh device on cellular in a very dense crowd: 0% loss, DERP-relayed the whole time (expected on cellular NAT, not congestion-related), a couple of RF-driven latency spikes traced to the cell tower rather than the gateway. See 15.3.7.
- ****July 18, 2026 `fix(toggle): harden sleep prevention with caffeinate -s`**** Four real 'Idle Sleep' cycles in a 38-minute window blacked out every mesh device's exit node because ``caffeinate -i`` alone doesn't reliably hold sleep off. Added ``-s`` (blocks sleep on AC power, no-op on battery) alongside the existing ``-i``. See 15.5.8.
- ****July 18, 2026 docs: honey-port pipe-hang incident**** Confirmed in practice (not just the known theoretical gotcha) that the Honey-Port subshell's inherited stdout/stderr keeps any ``| reader`` on ``toggle.sh`` blocked long after the script itself exits. Worked around by killing the reader; source fix still open. See 15.12.24.
- ****July 10, 2026 `fix(race): suppress WARP Watcher during post-wake force-recreate`**** Fixed a race where switching Wi-Fi caused the host IP to leak as the raw ISP IP (``132.x``) on every roam; ``recover.sh --post-wake``'s warp force-recreate tripped the WARP Watcher into a nuclear shutdown. Fixed via ``/tmp/nullexit-warp -inhibit.marker``. See 15.2.7.
- ****July 10, 2026 startup/roaming stabilization suite**** Pre-flight Tailscale race (15.5.3), STUN-block cold-boot false negative (15.5.4), restart DNS-build race via ``wait_for_dhcp_settle`` (15.5.5), nuclear/post-wake concurrency lock (15.2.8), infinite auto-recovery loop marker leak (15.2.9), PF kill-switch bricking SOCKS5 fallback (15.7.3), ``--reset`` host logouts (15.7.4), missing-flags abort (15.7.5), discarded docker-exec errors (15.12.18), and the SNAT endpoint poisoning fix (15.7.1).
- ****July 6, 2026 `fix: resolve edge-case vulnerabilities and system state leaks`**** Security-review hardening suite:
 1. ****DNS Watcher Interface Independence:**** ``start_dns_watcher`` now detects the active interface via ``get_active_service()`` instead of hardcoding a ``grep`` for "Wi-Fi".
 2. ****Robust Lock Verification:**** ``toggle.sh`` lock-file logic uses ``ps -p`` verification to prevent permanent lockouts from a recycled PID.
 3. ****Fail-Closed Crypto:**** ``crypto.sh`` integrity check switched from ``[ -x ]`` to ``[ -f ]`` so it runs even if git strips the ``+x`` bit.
 4. ****Strict `iptables` Pinning:**** ``post-rules.txt`` FORWARD-chain rules explicitly pin ``tailscale0``/``tun0`` both directions, preventing escape via ``eth0`` during WARP drops.
 5. ****Docker Local Network Isolation:**** ``docker-compose.yml`` binds exposed WARP ports (``1080``, ``5354``) to ``127.0.0.1`` to prevent LAN access.
 6. ****Routing Verification & Sudoers:**** Added a ``netstat -nr`` default-route override check and ensured ``/usr/bin/python3`` is permitted in the sudoers template.
- ****July 4, 2026 `cc789c5`**** Concurrency mutex on ``toggle.sh`` (``/tmp/nullexit-toggle.lock``); defensive ``TS_AUTHKEY`` init under ``set -u``; Go ``go vet``/``go build`` steps in CI; AdGuard log capping; macOS BSD ``grep`` fixes in ``setup-common.sh``; ``recover-linux.sh`` quoting/PID fixes; removed redundant bypass-routes block in ``recover.sh``.
- ****July 4, 2026 `fix: post-wake routing loop & destructive recovery`**** (1) ``add_warp_bypass_routes`` now accepts explicit interface args so post-wake stops routing WARP traffic back into ``tun0`` (was an infinite loop killing internet on every wake). (2) Replaced ``sudo route -n flush`` with targeted deletions of ``0.0.0.0/1``/``128.0.0.0/1`` + WARP bypass routes, so recovery no longer destroys loopback/multicast routes and wedges ``config``/``mDNSResponder`` until reboot.
- ****July 4, 2026 `feat: secure execution`**** Restricted the blanket ``/usr/bin/python3`` NOPASSWD sudo to exactly ``/usr/bin/python3 -I ../scripts/dns-proxy.py``; locked ``dns-proxy.py`` with ``chown root:wheel`` + ``chmod 755``; added native auth prompts to ``Toggle Gateway.app``/``Recover Gateway.app``; ``toggle.sh`` now captures ``warp`` logs on failure before teardown.
- ****July 4, 2026 Go ports + refactor**** ``logger`` and ``rule-compiler`` ported to Go multi-stage Docker builds (15.8.6); ~200 lines of duplicated bash extracted to ``scripts/common.sh`` (15.9.4); ``crypto.sh`` HMAC-SHA256 script integrity added.
- ****July 3, 2026 `fix: resolve numerous system regressions`**** Fixed truncated ``/etc/sudoers.d/nullexit`` line; temporary ``tmp``+``os.replace()`` atomic writes in ``sync-rules.py`` (later reverted, see below); ``recover.sh`` ``ts_args`` quoting; ANSI ``%b``/``-e`` output in ``diagnose-host-leak.sh``/``fix-docker-bridge-collision.sh``; ``toggle-linux.sh`` using ``resolvectl dns`` instead of macOS ``networksetup``; AdGuard REST refresh POST targeting port ``3000``; restored ``START_GATEWAY=true``; purged stale port ``80`` mapping; verified ``output.log``


```

in .gitignore.
2207 - **July 3, 2026 `unlock-files.sh` + revert atomic writes** Reverted all 5 `tmp`+`os.replace()` sites in `
sync-rules.py` to direct writes; created `scripts/unlock-files.sh` to permanently fix stale `0444`/`000`
file permissions via atomic rename (no `chmod`). Covers `.env`, `compiled-rules.txt`, `ip_blocklist.ipset`,
`cache/*.txt`, `cache/ip/*.txt`, `data/filters/*.txt`. See 15.9.6 + 3.
2208 - **July 3, 2026 Overnight leak + geo-blocking** Introduced `scripts/host-leak-probe.sh` (sub-second host-
egress detector, 15.6.1) and the statistical-threshold WARP auto-shutdown watcher (15.6.2); fixed the
overnight silent IP leak (15.6.3).
2209 - **July 2, 2026 `8c5fae1` + `181e1e1`** Two infinite routing loops fixed: host-VM tunnel loop (WARP bypass
routes via physical gateway IP + `setup_exit_node_routing` forcing default to `utun`) and Docker Compose
subnet takeover (`10.200.1.0/24` lock). `--accept-routes=true` added explicitly to all `tailscale up --
reset` calls. See 15.2.2.
2210 - **July 2, 2026 auto-recovery daemon** `scripts/watcher.sh` + launchd LaunchAgent documented. See 15.5.1.
2211 - **July 1, 2026 `c5b92c0` (refactor: personal-leak cleanup)** Eliminated every residual personal-username
leak across source + config; launchd plist uses `WorkingDirectory` + relative program arg with the `
__NULLEXIT_HOME__` sentinel (install recipe gains a single `sed -i 's|__NULLEXIT_HOME__|$(pwd)|'` step);
8 devref prose sites genericized to `~/colima/...`, `~/Library/LaunchAgents/...`, `$USER`, ``).
2212 - **July 1, 2026 `a5e2a5a` (refactor: script-relative paths)** Replaced 6 hardcoded `~/Users/<user>/.../
nullexit/...` literals with script-relative resolution. `recover.sh`/`watcher.sh` inject `SCRIPT_DIR=$(cd
$(dirname "${BASH_SOURCE[0]}") && pwd)`; `RECOVER`=`$SCRIPT_DIR/./recover.sh` resolves without launch-
path dependency.
2213 - **June 28, 2026 `f8f8e4e` (M1: scripts/ move)** 8 internal scripts consolidated into `scripts/`; the 4 user
-facing/orchestrator/config files (`toggle.sh`, `recover.sh`, `setup.sh`, `docker-compose.yml`) stayed at
repo root.
2214 - **June 28, 2026 `0875ef3` (ci: glob)** CI `py_compile` switched to `python3 -m py_compile scripts/*.py`
after M1 left root with zero `.py` files.
2215 - **June 28, 2026 `25b37a1` (docs: scripts/ prefix)** `devref.md` + `README.md` updated to the `scripts/`
prefix for all 8 moved files (-31 patches).
2216
2217 ### End-to-end verification (commit `c5b92c0`)
2218 Manual START STOP cycle ran clean on macOS after path relativization + personal-leak cleanup:
2219 - `bash toggle.sh` START: exit 0, ~135s (cold Colima boot); marker present (`2026-07-02T03:41:41Z`), all 5
containers healthy, host DNS hijacked to `100.100.21.8` (no fallback), exit node selected, Cloudflare trace
through WARP succeeds.
2220 - `bash toggle.sh` STOP: exit 0, ~12s; marker cleared, all nullexit containers gone, host DNS restored to
`1.1.1.1`, Tailscale disconnected.
2221
2222 ## 18. TODO & Future Work
2223
2224 ### 18.1 TODO
2225 * **FaceTime / real-time P2P split-route (Option 2, narrow):** --Implement `add_apple_split_routes()` / `
remove_apple_split_routes()` in `common.sh` (route FaceTime's Apple `/16`'s direct via the physical gateway,
opt-in `.env` flag, default OFF), wired into toggle START/STOP + `recover.sh --post-wake`. **Blocked on**
an empirical `tcpdump` capture of the relay/STUN `/16`'s during a real call (user-triggered).~ **Closed**
implemented and shipped enabled by default. The `tcpdump` capture from a real FaceTime call confirmed
`17.249.0.0/16` as the load-bearing media/relay range; `FACETIME_DIRECT_SUBNETS` in `.env.example` ships
that value. Full design + diagnosis in 15.12.22.
2226 * **Fix the shadowed exit-path port REJECTs (DoT/QUIC bypass):** --The `FORWARD` `-i tailscale0 -p udp --dport
853`/`--dport 443` REJECT rules are shadowed by the earlier `tailscale0 tun0` ACCEPT (0 pkts), so the
intended block-DoT / force-AdGuard and QUIC-block are **not enforced** a mesh client can bypass AdGuard
via DoT (853). Re-order the REJECTs before the ACCEPT (or scope `-o tun0`).~ **Closed** moved the REJECT
rules in `post-rules.txt` before the `tailscale0 tun0` / `tun0 tailscale0` ACCEPT rules. Live `iptables -
L FORWARD -n -v --line-numbers` now shows the REJECTs ahead of the ACCEPTs and actually accumulating hits.
See 15.12.22.
2227 * **Decide toggle STOP/START from persisted *intent*, not a live probe:** --`toggle.sh` decided STOP-vs-START
by calling `is_gateway_active()`, which live-probes the running system (`docker compose ps` + DNS + SOCKS
checks). When Colima/Docker was down the `docker compose ps` blocked for the full 15 s `run_with_timeout`
window and, under `set -e` + the `ERR` trap, aborted the script *before the START body ran* so the path
that would boot Colima never executed (`EXIT: code=1 phase=start`, seen 57 historically).~ **Closed**
added `gateway_should_be_running()` in `common.sh`, which reads persisted intent (`MARKER_FILE`) captured
into `GATEWAY_MARKER_AT_STARTUP` before the defensive stale-marker clear, and only falls back to `
is_gateway_active` when the marker is absent *and* the Docker socket is actually reachable (so a dead-
socket probe can neither hang nor trip `set -e`). All four decision sites (`toggle.sh` main + `--restart`,
`toggle-linux.sh` main + `--restart`) now use it, and the STOP `docker compose down` is `|| true`-guarded
so a disable always completes even with Docker down. See 15.12 Misc Resolved.
2228 * **Verify Wi-Fi Roaming:** Investigate and test whether Switching Wi-Fi networks now successfully and smoothly
recovers the gateway end-to-end without any tailscale flag errors or dead tunnels. (Pending user
verification).
2229 * **Fix Honey-Port subshell fd inheritance (15.12.24):** The tripwire's arming subshell (`toggle.sh` -L1704)
only redirects the inner `nc -l` command's stdout/stderr, not its own so it holds any caller's pipe open
forever once armed. Redirect the subshell itself (`( ) >/dev/null 2>&1 &`) at both arming sites (macOS +
Linux) so piping `toggle.sh`'s output no longer hangs post-completion.
2230 * **Fix Diagnostic Script Check 5/8:** --Investigate why `diagnose-host-leak.sh` check 5/8 (`Host default route
`) sometimes reports "default route goes via physical Wi-Fi Tailscale route assertion failed" on macOS
even though the `warp=on` egress checks confirm that traffic is successfully tunneling.~ **Closed** fixed
by switching check 5 from `netstat -rn | grep default` to `route -n get 1.1.1.1`. macOS Tailscale uses
interface-scoped routes and does not replace the DHCP default route entry. See 15.7.1 Known Unknowns.
2231 * **Expose Tor Control Port (9051):** --Consider mapping the Tor Control port to localhost and adding `nyx` (
the Tor terminal status monitor) to allow users to inspect their entry, middle, and exit node IPs in real-
time.~ **Closed** mapped the Tor Control Port to a procedurally generated, localhost-bound port `
TOR_CONTROL_PORT`, configured `ControlPort` and disabled cookie authentication in `docker/tor/entrypoint.sh

```



```

    `, and integrated it into the `/sweep` verification protocol.
2232 * Host-Only Tor Mode (Pluggable Egress): Implement an `EGRESS_TYPE=tor_host_only` mode for users in heavily
    censored (DPI-restricted) environments. This mode would skip booting `warp` and `tailscale`, boot `
    adguardhome` and `tor` (using Telegram `obfs4` bridges), set AdGuard's upstream to Tor's `DNSPort`, and use
    the `pf` kill-switch to force all host Mac traffic strictly through the Tor network. This provides a 100%
    free, DPI-resistant evasion path without requiring an external VPS.
2233 * Real-activity decoy traffic (upgrade to `noise.sh`): `noise.sh`'s current cover traffic is one-
    directional random UDP padding toward the exit it blurs uplink volume/timing but does nothing for downlink
    (the larger, more revealing direction: page loads, images, video) and can't hide any real spike bigger
    than its own modest rate (15.3.1's battery-driven decision not to saturate the link). The upgrade: instead
    of junk padding, periodically fire real outbound requests (page fetches, DNS lookups) that get real
    responses this naturally covers downlink too, since a genuine response comes back at a genuine size/timing
    , not a synthetic one. Not a clean win, two open problems: (1) making the decoy request shape (timing
    , correlation between a page load and its embedded assets) statistically indistinguishable from genuine
    human browsing is itself the open research problem of website fingerprinting published attacks already
    achieve real, nontrivial classification accuracy against far more mature systems than this one; a naive
    independent-random-fetch generator would likely be distinguishable from genuine sessions by the same
    techniques. (2) still costs real bandwidth to be big enough to mask a genuinely large spike same
    fundamental cost as raw padding, just spent fetching real content instead of junk. Also carries a practical
    /etiquette concern raw padding doesn't: firing synthetic requests at real third-party servers at a "human-
    like" rate is closer to bot traffic than background noise, and could trigger rate-limiting or abuse
    detection on the destination's side. Research-grade, not a quick toggle.
2234 * The Technical Realities (What You Need to Watch Out For):
2235 * The UDP Problem: Tor only transports TCP traffic. It does not support UDP (aside from DNS requests
    passed directly to its DNSPort). macOS is incredibly chatty over UDP (FaceTime, mDNS, QUIC protocols). Your
    pf rules defined in `scripts/pf.conf` must be configured to aggressively and silently DROP all host UDP
    traffic (except for local DNS queries to AdGuard). If you don't drop it cleanly, macOS services might hang
    indefinitely while waiting for timeouts.
2236 * The "Daily Driver" Friction: Routing a whole host OS through Tor is brutal on performance. Background
    daemons (iCloud sync, macOS software updates, Spotlight indexing) will blindly attempt to download
    gigabytes of data over Tor, which could saturate the circuits or time out completely.
2237 * Bridge Rot: State firewalls actively hunt and block obfs4 bridges. When a user's bridges burn out,
    they will lose all internet access due to the pf kill-switch. You will need to ensure the user has a
    frictionless way to update the `tor-bridges.txt` file and restart the Tor container without exposing their
    IP.
2238 * Exit Node Discrimination: Because all traffic will exit from known Tor nodes, the user will face an
    onslaught of Cloudflare CAPTCHAs, and many banking or streaming services will flat-out reject the
    connections.
2239
2240 ## 18.2 Future Work
2241 - Direct P2P on VM Hosts: Non-Linux hosts run Docker inside a VM, making true UDP hole-punching to
    arbitrary external peers impossible; those peers fall back to the DERP relay unless bridged mode is
    enabled on a trusted LAN. The only fully general solution is a native Linux host (Raspberry Pi, Intel NUC)
    where Docker runs without VM hypervisor translation. (The one case that works regardless is direct
hostcontainerP2P the gateway and its own Mac, the same physical machine re-permitted via routing-fix
.host_ips` RETURN rules and confirmed `direct` even in plain user-mode networking; see 15.3.5 and 15.8.7.)
    *
2242 - Native Linux Deployment: Benchmark on a Raspberry Pi native container footprint is ~75MB without macOS
    hypervisor overhead.
2243 - Mesh-Wide Filesystem (SFTP over Tailscale): Any mesh device passively exposes its filesystem over SFTP.
    Android via Termux + OpenSSH + wakelock; iOS is client-only due to background process limits.
2244 - Post-Quantum Cryptography (PQC): Tailscale uses Curve25519, theoretically vulnerable to "harvest now,
    decrypt later" quantum attacks. A future iteration could replace the Tailscale container with raw WireGuard
    + [Rosenpass](https://rosenpass.eu/) to negotiate post-quantum PSKs.
2245 - Egress Compartmentalization (Pluggable Architecture): See 12.9 for the full plan to make the entire
    egress layer swappable via a single `.env` variable.

```

5 diagrams.md

```

1 # nullexit Flow Diagrams & Architecture
2
3 ---
4
5 ## 1. System Architecture What's Running & Where
6
7 ```mermaid
8 graph TB
9     subgraph INTERNET[" Internet / Cloudflare Edge"]
10         CF["Cloudflare WARP\n(Exit Point)\nwarp=on"]
11     end
12
13     subgraph TAILNET[" Tailscale Mesh (100.x.x.x)"]
14         PHONE[" Phone / Laptop\n(Tailnet Client)"]
15     end
16
17     subgraph MACHOST[" macOS Host"]

```

```

18     direction TB
19     PFFIREWALL["macOS pf Firewall\n(Kill-Switch & TCP MSS Clamping)"]
20     TSDAEMON["tailscaled\n(brew service)\nhost Tailscale"]
21     HOSTDNS["Host DNS\n Gateway IP\n(hijacked by toggle.sh)"]
22     HOSTSOCKS["Host SOCKS5\n127.0.0.1:1080\n(fallback only)"]
23     HOSTTOR["Host Tor SOCKS5\n127.0.0.1:$TOR SOCKS_PORT\n(randomized)"]
24     HOSTTORCTRL["Host Tor Control\n127.0.0.1:$TOR_CONTROL_PORT\n(randomized)"]
25
26     subgraph MONITORS[" Background Daemons (toggle.sh children)"]
27         WARPWATCH["WARP Watcher\n(every 30s, via docker exec)\n output.log"]
28         DNSWATCHER["DNS Watcher\n(re-hijacks if drift detected)\n output.log"]
29         CAFFEINATE["caffeinate -i\n(sleep prevention)"]
30     end
31
32     subgraph COLIMAVM[" Colima VM (Linux, 600MB RAM)"]
33         subgraph NETNS["warp container network namespace (shared by ALL containers)"]
34             GLUETUN["warp / Gluetun\nWireGuard tun0\n(owns the namespace)"]
35             TS["tailscale container\nAdvertises exit node\ntailscale0 interface"]
36             SOCKS["tailscale\nSOCKS5\nport 1080"]
37             ADGUARD["adguardhome\nDNS sinkhole\nport 5335"]
38             ROUTINGFIX["routing-fix sidecar\nGeo-IP Blocking & Go Logger\n(writes blocked.log)"]
39             TOR["tor container\nSOCKS5: port 9050\nControl: port 9051\nTransparent Proxy: port 9040\n
nDNSPort: port 5353"]
40         end
41     end
42
43     subgraph LAUNCHD[" launchd (always-on)"]
44         WATCHER["scripts/watcher.sh\n(sleep/wake + Wi-Fi roam\n+ LAN P2P auto-detection\n+ Wi-Fi auto-
rejoin always on)"]
45     end
46 end
47
48 subgraph RECOVERY[" Recovery"]
49     RECOVER["recover.sh\n(nuclear reset)"]
50     RECOVERPOST["recover.sh --post-wake\n(light refresh)"]
51 end
52
53 %% Traffic flow
54 PHONE -->|"WireGuard / Tailscale mesh"| TSDAEMON
55 TSDAEMON -->|"exit-node route utun*"| TS
56 TS -->|"FORWARD tun0 (MASQUERADE)"| GLUETUN
57 GLUETUN -->|"WireGuard UDP (macOS Host Egress)"| PFFIREWALL
58 PFFIREWALL -->|"TCP MSS Clamped / Kill-Switch check"| CF
59 TS -->|"198.18.0.0/15 PREROUTING redirect"| TOR
60 TOR -->|"internal path / exit nodes via tun0"| GLUETUN
61
62 %% DNS
63 HOSTDNS -->|"UDP:53 TCP:5354"| ADGUARD
64 ADGUARD -->|"DNS upstream via tun0"| CF
65 ADGUARD -->|"onion queries localhost:5353"| TOR
66
67 %% Monitoring
68 WARPWATCH -->|"docker exec warp wget cdn-cgi/trace"| GLUETUN
69 DNSWATCHER -->|"networksetup -getdnsservers"| HOSTDNS
70
71 %% Logging
72 ROUTINGFIX -->|"writes"| BLOCKEDLOG["blocked.log\n(Host-side)"]
73 HOSTTOR -->|"port mapping"| TOR
74 HOSTTORCTRL -->|"port mapping"| TOR
75
76 %% Recovery triggers
77 WARPWATCH -->|"6 consecutive warp=off"| RECOVER
78 WATCHER -->|"sleep/wake / roam"| RECOVERPOST
79 RECOVERPOST -->|"if gateway unhealthy"| RECOVER
80 WATCHER -->|"writes .lan_p2p_detected"| ROUTINGFIX
81
82 style MONITORS fill:#1a1a2e,color:#e0e0ff
83 style COLIMAVM fill:#0d2137,color:#e0e0ff
84 style NETNS fill:#0a1628,color:#e0e0ff
85 style RECOVERY fill:#2d0a0a,color:#ffe0e0
86 style INTERNET fill:#0a2d1a,color:#e0ffe0
87 style TAILNET fill:#1a2d0a,color:#e8ffe0
88 style LAUNCHD fill:#2d1a2d,color:#ffe0ff
89 ...
90
91 ---
92
93 ## 2. toggle.sh Full START Flow
94
95 ```mermaid
96 flowchart TD

```

```

97 START(["./toggle.sh"])
98 CHECK{"is_gateway_active?\n(containers running OR\nDNS 1.1.1.1)}
99
100 START --> CHECK
101 CHECK -->|"YES already ON"| STOPFLOW[" See STOP Flow"]
102 CHECK -->|"NO start it"| S1
103
104 S1["1. Reset DNS 1.1.1.1\n(prevent deadlocks)"]
105 S2["2. tailscale down\n(prevent exit-node routing during boot)"]
106 S3["3. Check Colima VM\n(colima start --memory 0.6 --vm-type vz --network-address --network-mode bridged)"]
107 S4["4. Configure VM swap\n(400MB, prevent OOM)"]
108 S5["5. Clean corrupted AdGuard config\n(prevent container crash loop)"]
109 S6["6. docker compose up -d --build\n(compile Go threat rules & boot containers)"]
110 S7["7. Poll tailscale status\n(wait for 'offers exit node'\nup to 60s)"]
111 TSREADY{"container\nTailscale ready?"}
112 ABORT(["ABORT\n cleanup_handler"])
113 S8["8. Resolve gateway Tailscale IP\n(.gateway_ip or docker exec)"]
114 S9["9. Verify tailscaled running\n(auto-start if needed)"]
115 S10["10. tailscale up --reset\n--exit-node=\n--accept-routes=true\n--ssh=true --accept-dns=false"]
116
117 PF1{"Pre-flight 1/3\ntailscale ping gateway"}
118 PF2{"Pre-flight 2/3\ndig +tcp AdGuard DNS"}
119 PF3{"Pre-flight 3/3\nWARP internet check"}
120 ALLPASS{"All 3 pass?"}
121
122 EXITPATH["EXIT NODE PATH\n tailscale up --exit-node=\n force_dns_to_gateway (3 retry)\n SOCKS5 disabled"]
123 SOCKSPATH["SOCKS5 FALLBACK PATH\n macOS SOCKS5 127.0.0.1:1080\n DNS proxy 127.0.0.1:53\n No exit node (SKIP_EXIT_NODE=true)"]
124
125 DAEMONS["Start background daemons\n caffeinate -i (sleep prevention)\n DNS Watcher\n WARP Watcher\n(all write output.log)"]
126
127 RULECOMPILE["Rule compiler status\n(log rule/IP counts)"]
128 WRITEMARKER["write_gateway_active_marker\n(/tmp/nullexit-gateway-active.marker)"]
129 DONE(["Gateway UP"])
130
131 S1 --> S2 --> S3 --> S4 --> S5 --> S6 --> S7
132 S7 --> TSREADY
133 TSREADY -->|"NO (40 NoState)"| ABORT
134 TSREADY -->|"YES"| S8
135 S8 --> S9 --> S10
136 S10 --> PF1 --> PF2 --> PF3
137 PF1 & PF2 & PF3 --> ALLPASS
138 ALLPASS -->|"YES"| EXITPATH
139 ALLPASS -->|"NO"| SOCKSPATH
140 EXITPATH --> DAEMONS
141 SOCKSPATH --> DAEMONS
142 DAEMONS --> RULECOMPILE --> WRITEMARKER --> DONE
143
144 style ABORT fill:#5c1010,color:#fff
145 style EXITPATH fill:#0a3d1a,color:#c0ffc0
146 style SOCKSPATH fill:#3d2d00,color:#ffe0a0
147 style DAEMONS fill:#0a1f3d,color:#c0d8ff
148 style DONE fill:#0a3d1a,color:#fff
149
150
151 ---
152
153 ## 3. toggle.sh STOP Flow
154
155 ``mermaid
156 flowchart TD
157     STOP(["./toggle.sh\n(gateway already ON)"])
158
159     ST1["1. tailscale down\n(disconnect from mesh)"]
160     ST2["2. docker compose down -t 5\n(stop all containers)"]
161     ST3["3. Reset DNS 1.1.1.1\n(networksetup on all services)"]
162     ST4["4. stop_local_dns_proxy\n(kill Python UDP proxy)"]
163     ST5["5. stop_pidfile_daemon\n(kill caffeinate, rm PID)"]
164     ST6["6. stop_pidfile_daemon\n(kill DNS Watcher, rm PID)"]
165     ST7["7. stop_warp_watcher\n(kill WARP Watcher, rm PID)"]
166     ST8["8. clear_gateway_active_marker\n(rm /tmp/nullexit-gateway-active.marker)"]
167     ST9["9. cleanup_network_state\n disable_all_proxies\n Flush DNS cache (dscacheutil)\n Flush stale routes\n Power-cycle Wi-Fi (offon)"]
168
169     DONE(["Gateway DOWN\nInternet restored"])
170
171     STOP --> ST1 --> ST2 --> ST3 --> ST4
172     ST4 --> ST5 --> ST6 --> ST7 --> ST8 --> ST9 --> DONE
173
174     style DONE fill:#0a3d1a,color:#fff

```

```

175 ---
176 ---
177 ---
178
179 ## 4. Monitoring Layer The 3 Background Daemons
180
181 mermaid
182 graph LR
183     subgraph DAEMONS["Background Daemons (all start with gateway, stop on teardown)"]
184         direction TB
185
186         subgraph D1[" Sleep Prevention"]
187             CAFF["caffeinate -i\nPID: /tmp/nullexit-caffeinate.pid\n\nKeeps Mac awake while gateway is active"]
188         end
189
190         subgraph D2[" DNS Watcher"]
191             DNSW["Polls networksetup every -30s\nPID: /tmp/nullexit-dns-watcher.pid\n\nIf DNS drifts from gateway IP\n re-hijacks automatically\n output.log"]
192         end
193
194         subgraph D3[" WARP Watcher"]
195             WARPW["Polls every 30s via:\nndocker compose exec warp wget cdn-cgi/trace\nPID: /tmp/nullexit-warp-watcher.pid\n\nChecks physical uplink first:\nnno routable IP PAUSE (rejoin Wi-Fi),\nnot a kill (15.6.2)\n\nElse counts consecutive warp=off\n WARP_FAIL_THRESHOLD (default 6=30s)\n runs recover.sh (nuclear)\n output.log"]
196         end
197
198     end
199
200     subgraph BLIND["What each monitor can/can't see"]
201         B1["WARP Watcher sees:\n Container WARP tunnel state\n Host NIC traffic\n Flash leaks < 5s"]
202         B2["Host-NIC blind spot covered by:\n PF kill-switch (kernel, fail-closed)\n on-demand: diagnose-host-leak.sh\n(--watch to monitor) / sweep.sh"]
203     end
204
205     D3 -->|"covers"| B1
206     B1 -->|"gap closed by"| B2
207
208     style D1 fill:#1a2d0a
209     style D2 fill:#0a1a2d
210     style D3 fill:#2d0a2d
211     style BLIND fill:#1a1a1a,color:#aaa
212 ---
213 ---
214 ---
215
216 ## 5. Traffic Flow Data Path (Normal Operation)
217
218 mermaid
219 sequenceDiagram
220     participant PHONE as Phone<br/>(Tailnet client)
221     participant TS_HOST as tailscaled<br/>(macOS host)
222     participant TS_CONTAINER as tailscale<br/>(container)
223     participant GLUETUN as Gluetun/warp<br/>(container)
224     participant CF as Cloudflare<br/>WARP Edge
225     participant INTERNET as Internet
226
227     Note over PHONE,INTERNET: Normal EXIT NODE path (all 3 pre-flights passed)
228
229     PHONE->>TS_HOST: WireGuard packet<br/>(encrypted, UDP 41641)
230     TS_HOST->>TS_CONTAINER: route via tun* tailscale0
231     TS_CONTAINER->>GLUETUN: FORWARD chain<br/>(iptables MASQUERADE on tun0)
232     GLUETUN->>TS_HOST: WireGuard tunnel (tun0) egresses macOS Host
233     Note over TS_HOST: macOS pf Firewall applies<br/>Kill-Switch & TCP MSS Clamping
234     TS_HOST->>CF: re-encrypted UDP packet (MSS 1160)
235     CF->>INTERNET: Plain HTTPS from<br/>Cloudflare IP
236
237     Note over PHONE,INTERNET: DNS Resolution path
238     PHONE->>TS_HOST: DNS query
239     TS_HOST->>TS_CONTAINER: UDP:53 TCP:5354 AdGuard :5335
240     TS_CONTAINER->>GLUETUN: AdGuard upstream DNS via tun0
241     GLUETUN->>CF: Encrypted DNS query
242     CF-->>GLUETUN: DNS response
243     GLUETUN-->>TS_CONTAINER: response
244     TS_CONTAINER-->>TS_HOST: response
245     TS_HOST-->>PHONE: DNS answer
246 ---
247 ---
248 ---
249

```

```

250 ## 6. recover.sh Decision Tree
251
252 ```mermaid
253 flowchart TD
254     INVOKE(["recovered.sh called"])
255     MODE{"--post-wake\nflag?"}
256
257     subgraph POSTWAKE["--post-wake (light refresh, gateway stays UP)"]
258         PW1["Re-hijack DNS gateway IP"]
259         PW2["Flush DNS cache"]
260         PW3["Refresh tailscale exit-node"]
261         PW4["Force-recreate warp container\n(if Gluetun healthcheck failed)"]
262         PW5["Leave: containers, Wi-Fi,\ncaffeinate, DNS watcher untouched"]
263     end
264
265     subgraph NUCLEAR["Default (nuclear gateway torn down)"]
266         N0["Sudo keep-alive loop (background)"]
267         N1["1. tailscale down\n(disconnect from mesh)"]
268         N2["2. Reset DNS 1.1.1.1\n(ALL network services)"]
269         N3["3. disable_all_proxies\n(all interfaces)"]
270         N4["4. Flush DNS cache\n(dscacheutil -flushcache)"]
271         N5["5. Flush stale routes\n(WARP endpoint routes)"]
272         N6a["6a. docker compose down -t 30\n(stop all containers)"]
273         N6b["6b. stop_pidfile_daemon\n(kill caffeinate)"]
274         N6c["6c. stop_pidfile_daemon\n(kill DNS Watcher)"]
275         N7["7. Power-cycle Wi-Fi\n(networksetup offslepp 2on)"]
276         N8["8. Verify internet working\n(curl ifconfig.io)"]
277     end
278
279     DONE_PW(["Gateway refreshed\n(still running)"])
280     DONE_NUC(["Internet restored\n(gateway fully stopped)"])
281
282     INVOKE --> MODE
283     MODE -->|"yes"| PW1
284     PW1 --> PW2 --> PW3 --> PW4 --> PW5 --> DONE_PW
285     MODE -->|"no"| N0
286     N0 --> N1 --> N2 --> N3 --> N4 --> N5
287     N5 --> N6a --> N6b --> N6c --> N7 --> N8 --> DONE_NUC
288
289     style POSTWAKE fill:#0a2d1a,color:#c0ffc0
290     style NUCLEAR fill:#2d0505,color:#ffc0c0
291     style DONE_PW fill:#0a3d1a,color:#fff
292     style DONE_NUC fill:#3d1a00,color:#ffe0c0
293
294 ---
295
296 ## 7. Failure Paths & Self-Healing
297
298 ```mermaid
299 flowchart LR
300     subgraph EVENTS["Failure / Event"]
301         E1["WARP tunnel drops\n(warp=off)"]
302         E2["Mac wakes from sleep\nor Wi-Fi roams"]
303         E3["DNS drifts from\ngateway IP"]
304         E4["toggle.sh crashes /\nCtrl-C / SIGTERM"]
305         E5["Host-side leak\n(non-tunnel HOST NIC egress)"]
306         E6["Silent NAT timeout\n(ping stops working)"]
307         E7["Wi-Fi uplink drops\n(auto-join off no internet)"]
308     end
309
310     subgraph DETECTORS["Detector"]
311         D1["WARP Watcher\n(30s poll, docker exec)"]
312         D2["scripts/watcher.sh\n(launchd, always on)"]
313         D3["DNS Watcher\n(30s poll, networksetup)"]
314         D4["cleanup_handler\n(ERR/INT/TERM/HUP trap)"]
315         D5["PF kill-switch\n(kernel, always-on) +\nndiagnose-host-leak.sh --watch\n(on-demand)"]
316         D6["routing-fix.sh\n(30s curl over tun0)"]
317     end
318
319     subgraph ACTIONS["Response"]
320         A1["uplink up + threshold off\nrecover.sh (nuclear).\nuplink DOWN PAUSE, not kill"]
321         A2["recover.sh --post-wake\n(gentle refresh)"]
322         A3["Re-hijack DNS\n(networksetup setdnsservers)"]
323         A4["Stop all daemons\nReset DNS 1.1.1.1\nTear down Tailscale"]
324         A5["Non-tunnel egress blocked\nfail-closed at kernel;\n--watch alerts on state change"]
325         A6["Blackhole internet traffic\n+ Drop TUNNEL_FAILED_CLOSED.marker"]
326         A7["Push macOS Desktop Notification\n(via watcher.sh listener)"]
327         A8["watcher.sh Listener 4\nwifi-rejoin.sh: re-associate a\nremembered network (grace+backoff)"]
328     end
329
330

```

```

331 E1 --> D1 --> A1
332 E2 --> D2 --> A2
333 E3 --> D3 --> A3
334 E4 --> D4 --> A4
335 E5 --> D5 --> A5
336 E6 --> D6 --> A6
337 E7 --> D2 --> A8
338 A6 -->|"marker detected"| D2
339 D2 --> A7
340
341 style EVENTS fill:#2d1010,color:#ffcccc
342 style DETECTORS fill:#0a1a2d,color:#cce0ff
343 style ACTIONS fill:#0a2d0a,color:#ccffcc
344 ...
345
346 ---
347
348 ## 8. output.log Who Writes What
349
350 All events converge into a single `output.log` at the repo root.
351
352 | Writer | Event Types | Format |
353 |-----|-----|-----|
354 | `toggle.sh` | All startup/shutdown steps, errors, pre-flight results | Plain text with timestamps |
355 | `recover.sh` | Every recovery step (nuclear or post-wake) | Plain text |
356 | **WARP Watcher** | `WARP DOWN`, `WARP RECOVERED`, `WARP SHUTDOWN` | `[UTC timestamp]` prefix |
357 | **DNS Watcher** | DNS re-hijack events | Appended inline |
358 | `docker compose` | Container logs on failure (last 100 lines of warp) | Dumped on ERR path |
359 | `routing-fix.sh` | (via `nullexit-logger` inside container) | Structured |
360
361 **Grep cheatsheet:**
362 ```bash
363 grep 'WARP DOWN' output.log # Container-side tunnel drops
364 grep 'WARP SHUTDOWN' output.log # Auto-recovery triggered
365 grep 'WARP RECOVERED' output.log # Self-healed before threshold
366 grep -E 'EXEC:|EXIT ' output.log # Lifecycle breadcrumbs (last command + exit)
367 ```

```

6 toggle.sh

```

1 #!/bin/bash
2 # toggle.sh nullexit gateway toggle script (macOS + Linux)
3 # Automatically handles Docker containers, Colima, DNS hijacking, and Tailscale exit node routing.
4 #
5 # One file, both OSes: the dispatch below runs the self-contained Linux
6 # implementation (shuf / ss / nmcli / ip / systemd) and exits; on macOS it
7 # falls through to the macOS implementation (jot / netstat / networksetup,
8 # launchd / pf).
9
10 set -e
11
12 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
13
14 #
15 # LINUX TOGGLE self-contained; runs the full toggle then exits before the
16 # macOS body below. Native Linux tooling (shuf / ss / nmcli / ip / systemd).
17 # (Body inlined verbatim at column 0 it contains multiline `bash -c ""`
18 # strings that must NOT be re-indented.)
19 #
20 if [[ "$OSTYPE" != "darwin"* ]]; then
21 # Define log file (used by the lifecycle breadcrumb + run_logged helper)
22 LOG_FILE="$PWD/output.log"
23
24 # --- LIFECYCLE PHASE TRACKING + EXIT BREADCRUMB ---
25 # TOGGLE_PHASE is updated as the script moves through STOP/START so that if the
26 # process is killed (terminal closed, pkill, OOM) or exits on error, the EXIT
27 # trap records the final exit code AND the phase it died in making an
28 # interrupted START traceable in output.log instead of ending abruptly.
29 TOGGLE_PHASE="init"
30 _log_exit_breadcrumb() {
31     local rc=$?
32     echo "[$(date +%Y-%m-%d %H:%M:%S)] toggle.sh EXIT: code=$rc phase=$TOGGLE_PHASE (pid $$)" >> "$LOG_FILE"
33     2>/dev/null || true
34 }
35 trap '_log_exit_breadcrumb' EXIT

```



```

36 # --- DEBUG TRACE (opt-in via .env: DEBUG_TRACE=true) ---
37 # When enabled, mirror a full command-by-command xtrace to output.log. Off by
38 # default. Read directly from .env because common.sh (read_env_var) isn't
39 # sourced yet. Branch on bash version: BASH_XTRACEFD needs >= 4.1 (Linux
40 # usually has bash 5); fall back to teeing stderr on older bash. We never use
41 # the `exec {var}>>` dynamic-fd form (4.1+ only).
42 DEBUG_TRACE=$(grep -E '^DEBUG_TRACE=' .env 2>/dev/null | cut -d '=' -f2- | tr -d '\n' | tr '[:upper:]' '[:lower:]' | tr -d '[:space:]')
43 if [ "$DEBUG_TRACE" = "true" ]; then
44     echo "[$(date '+%Y-%m-%d %H:%M:%S')] DEBUG_TRACE enabled xtrace mirrored to output.log (bash ${BASH_VERSION} [0]).${BASH_VERSION[1]}" >> "$LOG_FILE"
45     export PS4='+ [$(SECONDS)s] ${BASH_SOURCE##*/}:${LINENO}:${FUNCNAME[0]:-main}: ' # subshell-free clock: a $
46     () in PS4 trips the bash 3.2 set -x/ERR heisenbug (devref 15.11.8)
47     if [ "${BASH_VERSION[0]}" -gt 4 ] || { [ "${BASH_VERSION[0]}" -eq 4 ] && [ "${BASH_VERSION[1]}" -ge 1 ];
48     }; then
49         exec 9>>"$LOG_FILE"
50         export BASH_XTRACEFD=9
51         set -x
52     else
53         # bash < 4.1: plain stderrfile redirect. NOT a process substitution under
54         # set -e, xtrace flowing through a backgrounded `tee` can abort the script
55         # (failed write/SIGPIPE trips set -e). See devref 15.11.8.
56         exec 2>>"$LOG_FILE"
57         set -x
58     fi
59 fi
60 # --- PROCEDURAL PORT GENERATION ---
61 # To prevent static port fingerprinting by malware, we randomize exposed ports
62 # on every boot and persist them to a hidden environment file.
63 PORTS_FILE="/tmp/nullexit-ports.env"
64 if [[ "$1" == "" || "$1" == "--restart" || "$1" == "restart" ]]; then
65     # Collision-avoidance function
66     GENERATED_PORTS=""
67     get_free_port() {
68         local port
69         while true; do
70             # Linux uses shuf or $RANDOM instead of jot
71             port=$(shuf -i 10000-65000 -n 1 2>/dev/null || echo $((10000 + RANDOM % 55000)))
72             # 1. Prevent self-collision within this loop
73             if [[ "$GENERATED_PORTS" == *"$port"* ]]; then
74                 continue
75             fi
76             # 2. Prevent collision with ANY process on the machine (root daemons, terminal apps, etc)
77             # We use ss to read the kernel socket table directly.
78             if ! ss -ltn | awk '{print $4}' | grep -q ":$port"; then
79                 echo "$port"
80                 return
81             fi
82         done
83     }
84
85     export TOR SOCKS_PORT=$(get_free_port)
86     GENERATED_PORTS="$GENERATED_PORTS $TOR SOCKS_PORT"
87
88     export TOR_CONTROL_PORT=$(get_free_port)
89     GENERATED_PORTS="$GENERATED_PORTS $TOR_CONTROL_PORT"
90
91     export SOCKS_PROXY_PORT=$(get_free_port)
92     GENERATED_PORTS="$GENERATED_PORTS $SOCKS_PROXY_PORT"
93
94     export DNS_PROXY_PORT=$(get_free_port)
95
96     echo "export TOR SOCKS_PORT=$TOR SOCKS_PORT" > "$PORTS_FILE"
97     echo "export TOR_CONTROL_PORT=$TOR_CONTROL_PORT" >> "$PORTS_FILE"
98     echo "export SOCKS_PROXY_PORT=$SOCKS_PROXY_PORT" >> "$PORTS_FILE"
99     echo "export DNS_PROXY_PORT=$DNS_PROXY_PORT" >> "$PORTS_FILE"
100 elif [ -f "$PORTS_FILE" ]; then
101     # On STOP, source the existing ports so docker-compose down matches
102     source "$PORTS_FILE"
103 else
104     # Fallbacks if file is lost
105     export TOR SOCKS_PORT=9050
106     export TOR_CONTROL_PORT=9051
107     export SOCKS_PROXY_PORT=1080
108     export DNS_PROXY_PORT=5354
109 fi
110
111 # Start execution timer
112 TOGGLE_START_TIME=$SECONDS

```

```

113
114 if [[ "$1" == "restart" ]] || [[ "$1" == "--restart" ]]; then
115     echo "Executing Gateway Restart Sequence..."
116     # We must source common.sh here temporarily to check gateway state before we proceed
117     source "$SCRIPT_DIR/scripts/common.sh"
118     # Use persisted intent via gateway_state (echoes a string, never a return code;
119     # set -e-safe under set -x see devref 15.11.8). On Linux the marker is not
120     # maintained, so this degrades to the is_gateway_active fallback (fast native Docker).
121     if [ "$(gateway_state)" = "running" ]; then
122         echo "Gateway is currently running. Stopping it first..."
123         bash "$0"
124         echo "Gateway stopped. Now starting it..."
125     else
126         echo "Gateway is already stopped. Starting it..."
127     fi
128     bash "$0"
129     exit 0
130 fi
131
132 # Global flag to track if the script completed successfully
133 SUCCESS_RUN=false
134
135 # Global variable to track the currently running background command PID
136 CURRENT_BG_PID=""
137
138 # Source common bash functions
139 source "$SCRIPT_DIR/scripts/common.sh"
140
141
142 PID_FILE="/tmp/nullexit-cafeinate.pid"
143
144 # Start macOS caffeinate to prevent sleep while gateway is running
145 start_sleep_prevention() {
146     if ! command -v systemd-inhibit >/dev/null 2>&1; then
147         echo " [Warning] systemd-inhibit tool not found. Sleep prevention unavailable."
148         return 1
149     fi
150
151     # Stop any existing sleep prevention process first to avoid duplicates
152     stop_sleep_prevention
153
154     echo " Enabling system sleep prevention (systemd-inhibit)..."
155     # Run systemd-inhibit wrapped in a bash trap to prevent idle system sleep.
156     # Critically, this traps shutdown signals (SIGTERM) and automatically
157     # flushes hijacked DNS back to normal right before the PC powers off.
158     # We do not use recover.sh here because it is a heavy recovery script
159     # which takes too long and gets terminated before it can reset the DNS. Instead,
160     # we surgically and instantly restore normal DNS settings here.
161     nohup bash -c "
162     trap '
163         echo \"[Shutdown Trap] Reverting network settings...\\" >> \"\$PWD/output.log\" 2>&1
164         kill \$! 2>/dev/null || true
165         sudo -n resolvectl domain \"\$ACTIVE_SERVICE\" \"\" >> \"\$PWD/output.log\" 2>&1 || true
166         sudo -n resolvectl revert \"\$ACTIVE_SERVICE\" >> \"\$PWD/output.log\" 2>&1 || true
167         resolvectl domain \"\$ACTIVE_SERVICE\" \"\" >> \"\$PWD/output.log\" 2>&1 || true
168         resolvectl revert \"\$ACTIVE_SERVICE\" >> \"\$PWD/output.log\" 2>&1 || true
169         if command -v tailscale >/dev/null 2>&1; then
170             tailscale up --accept-dns=false --exit-node= >> \"\$PWD/output.log\" 2>&1 &
171             TS_DOWN_ARGS=\"\"
172             if [ \"\$KILL_SWITCH\" = \"true\" ]; then TS_DOWN_ARGS=\"--accept-risk=lose-ssh\"; fi
173             tailscale down \$TS_DOWN_ARGS >> \"\$PWD/output.log\" 2>&1 &
174         fi
175         sleep 1
176         echo \"[Shutdown Trap] Cleanup complete.\" >> \"\$PWD/output.log\" 2>&1
177         exit 0
178     ' SIGTERM SIGINT SIGHUP
179     systemd-inhibit --what=sleep --who=nullexit --why=\"gateway running\" --mode=block sleep infinity &
180     wait \$!
181     \" >> output.log 2>&1 &
182     local caffe_pid=$!
183     echo \"\$caffe_pid\" > \"\$PID_FILE\"
184     echo \" Sleep prevention active (PID \$caffe_pid). Your PC won't sleep while the gateway is running.\"
185 }
186
187 # Stop sleep prevention when gateway is stopped
188 stop_sleep_prevention() {
189     if [ -f \"\$PID_FILE\" ]; then
190         local caffe_pid
191         caffe_pid=$(cat \"\$PID_FILE\")
192         if [ -n \"\$caffe_pid\" ] && kill -0 \"\$caffe_pid\" 2>/dev/null && ps -p \"\$caffe_pid\" -o command= 2>/dev/null |
            grep -q -E \"systemd-inhibit|recover.sh\"; then

```

```

193     echo " Stopping system sleep prevention (systemd-inhibit wrapper PID $caffe_pid)..."
194     kill "$caffe_pid" 2>/dev/null || true
195     fi
196     rm -f "$PID_FILE"
197 fi
198 }
199
200 DNS_WATCHER_PID_FILE="/tmp/nullexit-dns-watcher.pid"
201
202 stop_dns_watcher() {
203     if [ -f "$DNS_WATCHER_PID_FILE" ]; then
204         local watcher_pid
205         watcher_pid=$(cat "$DNS_WATCHER_PID_FILE")
206         if [ -n "$watcher_pid" ] && kill -0 "$watcher_pid" 2>/dev/null; then
207             echo " Stopping background DNS Watcher (PID $watcher_pid)..."
208             kill -9 "$watcher_pid" 2>/dev/null || true
209         fi
210         rm -f "$DNS_WATCHER_PID_FILE"
211     fi
212 }
213
214 # Cleanup handler to restore DNS to 1.1.1.1 on error or user interrupt (Ctrl+C / SIGTERM / SIGHUP)
215 cleanup_handler() {
216     local exit_code=$?
217     local trigger_type="$1"
218     local line_no="$2"
219
220     if [ "$SUCCESS_RUN" = "false" ]; then
221         echo -e "\n[Self-Correction] Script interrupted or failed ($trigger_type). Restoring host DNS to 1.1.1.1..."
222         if [ -n "$ACTIVE_SERVICE" ]; then
223             reset_dns
224         fi
225
226         if [ -n "$CURRENT_BG_PID" ]; then
227             echo "Terminating active command (PID $CURRENT_BG_PID)..."
228             kill -15 "$CURRENT_BG_PID" 2>> output.log || true
229         fi
230
231         # Disconnect host Tailscale and close the GUI application on failure
232         disconnect_tailscale_host
233
234         # Stop local DNS proxy if running
235         stop_local_dns_proxy
236
237         # Stop sleep prevention
238         stop_sleep_prevention
239         stop_dns_watcher
240
241         # Nuke leftover network state (proxies, routes, DNS cache, Wi-Fi)
242         if [ -n "$ACTIVE_SERVICE" ]; then
243             cleanup_network_state
244         fi
245
246         if [ "$trigger_type" = "ERR" ]; then
247             echo -e "\n=====
248             echo "ERROR: Script failed at line $line_no with exit code $exit_code."
249             echo "=====
250             echo "Troubleshooting steps:"
251             echo "1. Run 'colima status' to verify the VM is active."
252             echo "2. Run 'docker compose ps' to check container health."
253             echo "3. Run 'docker compose logs' to view application logs."
254             echo "4. Run 'tailscale status' to check host Tailscale status."
255             echo "=====
256         fi
257     fi
258 }
259
260
261 trap 'cleanup_handler ERR $LINENO' ERR
262 trap 'cleanup_handler INT' INT
263 trap 'cleanup_handler TERM' TERM
264 trap 'cleanup_handler HUP' HUP
265
266 # NOPASSWD sudo
267 # The user has NOPASSWD in /etc/sudoers.d/nullexit for the specific commands
268 # needed (resolvectl, ip, systemctl, systemd-inhibit, pkill, and
269 # python3 scripts/dns-proxy.py for the fallback DNS proxy).
270 # All privileged calls below use `sudo -n` which will run without prompting.
271 # No credential caching loop is needed.
272

```

```

273 # Add Homebrew and standard paths
274 export PATH="/opt/homebrew/bin:/opt/homebrew/sbin:/usr/local/bin:$PATH"
275
276 # Check for local DNS proxy tools (used when tailnet data plane is unavailable)
277 # Uses a Python DNS proxy (handles TCP wire format correctly).
278 # Python3 is built into macOS no external dependencies.
279 DNS_PROXY_BIN=""
280 if command -v python3 >/dev/null 2>&1; then
281     DNS_PROXY_BIN="python3 -I"
282 elif command -v python >/dev/null 2>&1; then
283     DNS_PROXY_BIN="python -I"
284 fi
285
286 # Path to the DNS proxy script (sibling of this script)
287 DNS_PROXY_SCRIPT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/dns-proxy.py"
288
289 # Resolve Tailscale CLI on host. We rely on the standalone Homebrew formula
290 # (`brew install tailscale` + `systemctl start tailscaled`) NOT the
291 # Tailscale.app GUI so there is no .app bundle or Network Extension sandbox
292 # to launch, no menu-bar click to perform, and no scutil Network Service to
293 # start manually. tailscaled runs as a per-user LaunchAgent (or LaunchDaemon
294 # if you ran the install with sudo) and the control socket is always available.
295 TS_BIN=""
296 if command -v tailscale >> output.log 2>&1; then
297     TS_BIN="tailscale"
298 fi
299
300 # Baseline: disable Tailscale DNS management at the daemon level.
301 # `tailscale set` writes a persistent preference. However, `tailscale up --reset`
302 # (used in steps 7 and 9) RESETS all unspecified flags to defaults, which would
303 # re-enable `--accept-dns=true` and nullify this `set`.
304 # Therefore, EVERY `tailscale up --reset` call in this file MUST also pass
305 # `--accept-dns=false` explicitly. See steps 7 and 9 for the fix.
306 #
307 # This initial `set` is still useful as a baseline for non-reset operations
308 # (e.g. if the user runs `tailscale up` manually without --reset) and covers
309 # the brief window between this line and the first `--reset` call.
310 #
311 # Runs once at script start while tailscaled is still connected (if it was
312 # left running from a previous toggle), so the pref takes effect before any
313 # disconnection or reconnection logic.
314 if [ -n "$TS_BIN" ]; then
315     $TS_BIN set --accept-dns=false >> output.log 2>&1 || true
316 fi
317
318 # Disconnect host Tailscale from the mesh. With the standalone daemon, this is the
319 # *entire* teardown no need to mess with system network managers.
320 # tailscaled keeps running (as a systemd service), which is intentional:
321 # the next `tailscale up` then reconnects in seconds rather than waiting for a
322 # slow daemon launch. The `tailscale down` call also retains the user's
323 # auth state, so we don't force a browser re-login every cycle.
324 #
325 # Defined early (before the if/else branches and referenced by cleanup_handler's
326 # ERR trap) so that any failure before the main logic can still tear down Tailscale.
327 restart_tailscaled_daemon() {
328     echo " [Tailscale Recovery] tailscaled daemon appears to be wedged/unresponsive."
329     echo " [Tailscale Recovery] Attempting to restart tailscaled service via systemctl..."
330     if run_with_timeout 15 sudo -n systemctl restart tailscaled >> output.log 2>&1; then
331         echo " [Tailscale Recovery] Successfully restarted tailscaled service."
332     else
333         echo " [Tailscale Recovery] WARNING: Failed to restart tailscaled. Manual intervention may be needed."
334     fi
335     sleep 3
336 }
337 disconnect_tailscale_host() {
338     if [ -n "$TS_BIN" ]; then
339         echo "Disconnecting host Tailscale from mesh (tailscaled stays running as system service)..."
340         local ts_args=""
341         if is_kill_switch_enabled; then ts_args="--accept-risk=lose-ssh"; fi
342         if ! run_with_timeout 10 $TS_BIN down $ts_args >> output.log 2>&1; then
343             restart_tailscaled_daemon
344             run_with_timeout 10 $TS_BIN down $ts_args >> output.log 2>&1 || true
345         fi
346     fi
347 }
348
349 ACTIVE_SERVICE=$(get_active_service)
350
351 # Helper to nuke leftover network state after teardown (proxy settings, routing
352 # table, DNS cache, Wi-Fi). Prevents the "had to write a whole recovery script"
353 # problem where stale state kills internet after the gateway stops.

```

```

354 cleanup_network_state() {
355     echo -e "\nCleaning up network state (proxies, routes, DNS cache, Wi-Fi)..."
356
357     # 1. Disable any leftover proxy settings (needs root on many macOS versions)
358     # (Skipped for Linux since we don't set global SOCKS proxy)
359     echo "  Proxies disabled."
360
361     # 2. Flush DNS cache
362     if command -v resolvectl >> output.log 2>&1; then
363         sudo -n resolvectl flush-caches 2>> output.log || true
364     elif command -v systemd-resolve >> output.log 2>&1; then
365         sudo -n systemd-resolve --flush-caches 2>> output.log || true
366     fi
367     echo "  DNS cache flushed."
368
369     # 3. Flush stale routing table entries
370     if command -v ip >> output.log 2>&1; then
371         sudo -n ip route flush cache >> output.log 2>&1 || true
372     fi
373     echo "  Routing table flushed."
374
375     # 4. Power-cycle Wi-Fi to clear any lingering interface state
376     echo "  Restarting network interface..."
377     sudo -n ip link set dev "$ACTIVE_SERVICE" down 2>> output.log || true
378     sleep 1
379     sudo -n ip link set dev "$ACTIVE_SERVICE" up 2>> output.log || true
380     echo "  Interface power-cycled ($ACTIVE_SERVICE)."
381     sleep 3
382
383     echo "Network state cleanup complete."
384 }
385
386 # Ensure DNS is set to 1.1.1.1 immediately at script start to prevent any DNS deadlocks/hangs
387 # during status checks, container startup, VM boot, or teardown. We *also* clear any leftover
388 # `ts.net` search domain from a previous `tailscale up --accept-dns=true` run, otherwise macOS
389 # would prepend it to every lookup and most public DNS queries would resolve to NXDOMAIN.
390 # Capture current DNS before resetting so we can check if it was hijacked
391 INITIAL_DNS=$(resolvectl dns "$ACTIVE_SERVICE" 2>/dev/null | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | head
392 -1 || true)
393
394 echo "Initializing DNS to 1.1.1.1 to ensure reliable internet access..."
395 reset_dns
396
397 # Local Network Surveillance Check
398 echo -e "\n"
399 echo "Local Network Surveillance Check"
400 echo ""
401 # Count populated ARP cache entries to detect network scanning or high congestion
402 if command -v ip >> /dev/null 2>&1; then
403     ARP_COUNT=$(ip neigh | grep -iv 'incomplete' | wc -l | tr -d ' ')
404 else
405     # Using '-an' avoids hanging on DNS reverse resolution when the network state is in transition.
406     ARP_COUNT=$(arp -an 2>/dev/null | grep -iv 'incomplete' | wc -l | tr -d ' ')
407 fi
408 if [ "$ARP_COUNT" -gt 15 ]; then
409     echo -e " \033[1;33m[!] WARNING: High ARP activity detected ($ARP_COUNT devices in cache).\033[0m"
410     echo -e " \033[1;33m[!] This Wi-Fi network may be heavily congested or actively scanned (e.g., arp-scan)
411     .\033[0m"
412     echo -e " [] Your traffic remains fully encrypted and invisible.\n"
413 else
414     echo -e " [] Local network looks quiet ($ARP_COUNT devices). No aggressive scanning detected.\n"
415 fi
416
417 echo "Checking Gateway Status..."
418 # Decide STOP vs START from persisted intent via gateway_state, which echoes a
419 # string (never a return code) to stay set -e-safe under set -x. See devref 15.11.8.
420 # On Linux this falls back to is_gateway_active (fast native Docker).
421 if [ "$(gateway_state)" = "running" ]; then
422     TOGGLE_PHASE="stop"
423     echo -e "\n=====
424     echo "Gateway is RUNNING. Stopping it now..."
425     echo -e "=====
426
427 # 1. Disconnect host Tailscale cleanly first before stopping the exit-node container
428 disconnect_tailscale_host
429 echo ""
430
431 echo "Stopping Docker containers..."
432 # `|| true`: a disable must always complete even if the Docker daemon is down.
433 run_logged docker compose down -t 5 || true

```

```

433
434 echo -e "\nDocker daemon handles container teardown automatically."
435
436 # The host's DNS was hijacked to the gateway IP during ENABLE; now that the
437 # gateway is down, restore DNS to 1.1.1.1 immediately so subsequent lookups
438 # don't stall for the macOS DNS timeout before the 1.1.1.1 fallback engages.
439 echo -e "\nRestoring host DNS to 1.1.1.1 (gateway is gone)... "
440 reset_dns
441
442 # 3. Stop local DNS proxy if running
443 stop_local_dns_proxy
444
445 # Stop sleep prevention
446 stop_sleep_prevention
447 stop_dns_watcher
448
449 # 4. Nuke leftover network state so internet actually works after teardown
450 cleanup_network_state
451
452 ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
453 echo -e "\nGateway has been successfully STOPPED in ${ELAPSED} seconds."
454 else
455 TOGGLE_PHASE="start"
456 echo -e "\n=====
457 echo "Gateway is STOPPED. Starting it now..."
458 echo -e "=====
459
460 # 1. Prevent host exit-node deadlock during VM / Container startup
461 disconnect_tailscale_host
462
463 echo -e "\nUsing native Linux Docker..."
464
465 # 4. Clean up corrupted AdGuardHome configurations
466 if [ -f "adguard/conf/AdGuardHome.yaml" ] && [ ! -s "adguard/conf/AdGuardHome.yaml" ]; then
467     rm -f "adguard/conf/AdGuardHome.yaml"
468     echo "Removed corrupted empty AdGuardHome.yaml to prevent container crash loop."
469 fi
470
471
472 # 5. Start compose services
473 write_host_ips
474 # Procedurally generated ports have already been exported at the top of the script.
475 echo " [Security] Procedurally randomized proxy ports generated to evade fingerprinting."
476
477 # --- HONEY-PORT TRIPWIRE ---
478 # Generate a random trap port. If any malware does a sequential port scan on localhost,
479 # it will inevitably hit this port. When it does, we instantly fire a desktop alert.
480 # Reap the honey-port from any previous toggle before arming a new one, so the
481 # `nc -l` tripwire never accumulates one idle listener per toggle-on (15.12.20).
482 # `nc -l` tripwire never accumulates one idle listener per toggle-on (15.12.20).
483 pkill -f "nc -l .*127.0.0.1" 2>/dev/null || true
484 pkill -f "nc -l localhost" 2>/dev/null || true
485 HONEY_PORT=$(get_free_port)
486 (
487     if nc -l -p "$HONEY_PORT" 127.0.0.1 >/dev/null 2>&1 || nc -l localhost "$HONEY_PORT" >/dev/null 2>&1; then
488         echo "[$(date '+%Y-%m-%d %H:%M:%S')] [CRITICAL SECURITY ALERT] PORT SCAN DETECTED! An unknown application
489             just pinged the local Honey-Port ($HONEY_PORT)!" >> "$PWD/output.log"
490         if command -v notify-send >/dev/null 2>&1; then
491             notify-send -u critical "nullexit OPSEC Alert" "An unknown application is aggressively scanning your
492                 local ports. Malware port-scan suspected."
493         fi
494     fi
495 ) &
496 disown %1 2>/dev/null || true
497 echo " [Security] Honey-Port Tripwire armed on random port to detect malware port-scans."
498
499 echo -e "\nStarting Docker containers..."
500 run_logged docker compose up -d
501
502 # Clean up the one-off rule compiler container so it doesn't clutter the Docker UI
503 docker compose rm -s -f rule-compiler >> output.log 2>&1
504
505 # 6. Wait for the gateway container's Tailscale connection to be ready
506 BYPASS_PING=$(read_env_var GATEWAY_BYPASS_PING | tr '[:upper:]' '[:lower:]')
507 USE_EXIT_NODE=$(read_env_var GATEWAY_USE_EXIT_NODE | tr '[:upper:]' '[:lower:]')
508
509 echo "Waiting for gateway container's Tailscale to connect to the tailnet..."
510 echo " (This can take 30-60s Tailscale goes through: NoState Starting Running Online)"
511 connected=false
512 consecutive_failures=0
513 for i in {1..60}; do

```



```

512 # Run tailscale status and capture its output so the user can see what's happening.
513 # Previously this was hidden behind 2>> output.log, making it impossible to debug hangs.
514 ts_output=$(run_with_timeout 3 docker compose exec -T tailscale tailscale status 2>&1 || true)
515
516 if echo "$ts_output" | grep -q "offers exit node"; then
517     connected=true
518     break
519 fi
520
521 # Track consecutive failures to detect a stuck state.
522 # If we see 40+ consecutive 'NoState' responses, give up early
523 # the daemon is alive but can't connect to the coordination server.
524 if echo "$ts_output" | grep -q "NoState"; then
525     consecutive_failures=$((consecutive_failures + 1))
526     if [ "$consecutive_failures" -ge 40 ]; then
527         echo ""
528         echo " [attempt $i/60] Stuck in 'NoState' for $consecutive_failures checks."
529         echo " Tailscale daemon is running but can't reach the coordination server."
530         break
531     fi
532 else
533     consecutive_failures=0
534 fi
535
536 # Print a condensed status every 10 seconds so the user can see progress
537 if [ $((i % 10)) -eq 0 ]; then
538     ts_short=$(echo "$ts_output" | head -1 | tr -d '\r\n')
539     echo " [attempt $i/60] $ts_short"
540 elif [ $((i % 5)) -eq 0 ]; then
541     echo -n "[i]"
542 else
543     echo -n "."
544 fi
545 sleep 1
546 done
547 echo ""
548
549 if [ "$connected" = "true" ]; then
550     echo "Gateway container is online on the Tailscale mesh."
551 elif [ "$BYPASS_PING" = "true" ]; then
552     echo "[Warning] Gateway container check timed out. Proceeding anyway (GATEWAY_BYPASS_PING is true)..."
553 else
554     echo "ERROR: Gateway container failed to initialize Tailscale (timed out after 60s)." >&2
555     echo " The container was seen in the following states during the wait:" >&2
556     echo "$ts_output" | sed 's/^/ /' >&2
557     echo "" >&2
558     echo " This could mean:" >&2
559     echo " 1. Tailscale auth key has expired check TS_AUTHKEY in .env" >&2
560     echo " 2. Network issue inside the container check: docker compose logs tailscale" >&2
561     echo " 3. gluetun VPN tunnel not connecting check: docker compose logs warp | tail -20" >&2
562     echo "" >&2
563     echo " Diagnose manually:" >&2
564     echo " docker compose exec -T tailscale tailscale status" >&2
565     echo " docker compose exec -T tailscale tailscale netcheck" >&2
566     cleanup_handler ERR $LINENO
567     exit 1
568 fi
569
570 # 6b. Resolve the gateway's Tailscale IP.
571 # Dynamically query the container for the IP (handles IP changes after re-auth)
572 # and cache it in .gateway_ip for fast retrieval by recover.sh during Wi-Fi roaming.
573 #
574 # CRITICAL: `docker compose exec -T` on macOS/Colima injects carriage
575 # returns (\r) into captured output. If $TS_IP ends with \r, then
576 # `networksetup -setdnsservers` silently rejects the malformed IP and
577 # DNS stays at 1.1.1.1 the primary bug this project was hitting.
578 # `tr -d '\r'` strips the CR; `awk 'NR==1{print $1; exit}'` takes only
579 # the first field of the first line.
580 TS_IP=""
581 if [ "$connected" = "true" ]; then
582     TS_IP=$(run_with_timeout 5 docker compose exec -T tailscale tailscale ip -4 2>> output.log | tr -d '\r' |
583         awk 'NR==1{print $1; exit}' || true)
584     if [ -n "$TS_IP" ]; then
585         echo "Resolved gateway Tailscale IP dynamically: $TS_IP"
586         echo "$TS_IP" > .gateway_ip
587     else
588         echo " [!] Failed to resolve gateway IP dynamically. Trying fallback cache..." >&2
589         TS_IP=$(read_adguard_ip || true)
590     fi
591 else
592     TS_IP=$(read_adguard_ip || true)

```

```

592 fi
593
594 # 7. Connect host Mac to Tailscale mesh.
595 # With the standalone tailscaled daemon (registered as a per-user LaunchAgent or
596 # system LaunchDaemon via 'systemctl start tailscaled'), the previous five-
597 # phase flow collapses into two trivial CLI calls. There is no .app GUI to launch,
598 # no menu bar item to click, and no Accessibility permission required.
599 #
600 # CRITICAL: If tailscaled is not running, `tailscale up` will silently fail.
601 # We auto-start it before proceeding, and SKIP the rest of the Tailscale setup
602 # if it can't be started (DNS hijack to a 100.x.x.x IP and exit-node routing
603 # are impossible without the host on the mesh).
604 #
605 # We connect in two phases:
606 # Phase A Join mesh WITHOUT exit node (so pre-flight checks can run)
607 # Phase B Set exit node only after pre-flight checks confirm the gateway works
608 HOST_ON_MESH=false
609 SKIP_EXIT_NODE=false
610 if [ -n "$TS_BIN" ]; then
611     echo -n "Verifying tailscaled is reachable"
612     daemon_ready=false
613     for i in {1..10}; do
614         # `tailscale status` returns exit code 1 when the daemon is alive but
615         # disconnected ("Tailscale is stopped."). We check the daemon socket
616         # directly as a fallback if the socket exists, tailscaled is running
617         # even if the host isn't connected to the mesh yet.
618         if run_with_timeout 5 $TS_BIN status >> output.log 2>&1 || [ -S /var/run/tailscaled.socket ]; then
619             daemon_ready=true
620             break
621         fi
622         echo -n "."
623         sleep 1
624     done
625     echo ""
626
627     if [ "$daemon_ready" != "true" ]; then
628         echo -e "\033[0;33m[!] tailscaled daemon is not running. Attempting to start it...\033[0m"
629
630         # Try system-wide daemon (with timeout sudo may prompt for password)
631         if [ "$daemon_ready" != "true" ]; then
632             if run_with_timeout 10 sudo systemctl start tailscaled 2>> output.log; then
633                 echo "Started tailscaled as system LaunchDaemon."
634                 for i in {1..15}; do
635                     if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
636                         daemon_ready=true
637                         break
638                     fi
639                     sleep 1
640                 done
641             fi
642         fi
643     fi
644
645     if [ "$daemon_ready" = "true" ]; then
646         echo "tailscaled is responsive (running as a system service)."
647
648         # Phase A: Join mesh without exit node
649         echo "Connecting host to Tailscale mesh (no exit node yet)..."
650         if $TS_BIN up --reset --ssh=true --accept-dns=false --exit-node=; then
651             HOST_ON_MESH=true
652             echo "Host is on Tailscale mesh."
653         else
654             echo -e "\033[0;33m[!] tailscale up failed even though the daemon is running.\033[0m"
655             echo "This usually means the host hasn't authenticated with your tailnet yet."
656             echo "Run this command in a terminal to authenticate:"
657             echo ""
658             echo "    sudo tailscale up"
659             echo ""
660             echo "A browser will open. Log in to your Tailscale account, then re-run toggle.sh."
661         fi
662     else
663         echo -e "\033[0;31m[!] tailscaled could NOT be started automatically.\033[0m"
664         echo "Run this in a terminal to start and authenticate Tailscale:"
665         echo ""
666         echo "    sudo systemctl start tailscaled"
667         echo "    sudo tailscale up"
668         echo ""
669         echo "Then re-run toggle.sh."
670     fi
671 else
672     echo -e "\n[Warning] tailscale CLI not found on PATH. Skipping host Tailscale configuration..."

```

```

673 fi
674
675 # Phase B: Pre-flight checks + enable exit node only if safe
676 if [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" != "false" ] && [ -n "$TS_IP" ]; then
677     echo -e "\n"
678     echo "Pre-flight connectivity check"
679     echo ""
680
681     # Check 1: Can we reach the gateway via Tailscale (uses tailscale ping, not ICMP)?
682     # NOTE: tailscale ping exits 1 when the connection goes through a DERP relay
683     # ("direct connection not established") even though the pong was received.
684     # We check for 'pong' in the output (stderr discarded) to accept relayed connections.
685     echo -n " [1/3] Gateway reachable via Tailscale... "
686     ping_ok=false
687     # The host tailscaled needs a few seconds to propagate the new network map and
688     # establish a connection route after 'tailscale up --reset'. We retry up to 45 times.
689     for attempt in {1..45}; do
690         if $TS_BIN ping --until-direct=false -c 2 --timeout 2s "$TS_IP" 2>/dev/null | grep -q "pong"; then
691             ping_ok=true
692             break
693         fi
694         sleep 1
695     done
696     if [ "$ping_ok" = "true" ]; then
697         echo "PASS"
698     else
699         echo "FAIL"
700         SKIP_EXIT_NODE=true
701     fi
702
703     # Check 2: Can we reach AdGuard via localhost (exposed port ${DNS_PROXY_PORT})?
704     echo -n " [2/3] AdGuard DNS via localhost... "
705     if command -v dig >/dev/null 2>&1; then
706         if dig +tcp @127.0.0.1 -p ${DNS_PROXY_PORT} google.com +short +timeout=5 >/dev/null 2>&1; then
707             echo "PASS"
708         else
709             echo "FAIL"
710             SKIP_EXIT_NODE=true
711         fi
712     elif command -v nslookup >/dev/null 2>&1; then
713         if nslookup -port=${DNS_PROXY_PORT} google.com 127.0.0.1 >/dev/null 2>&1; then
714             echo "PASS"
715         else
716             echo "FAIL"
717             SKIP_EXIT_NODE=true
718         fi
719     else
720         echo "SKIP (dig/nslookup not available)"
721     fi
722
723     # Check 3: Does the WARP container have external internet?
724     echo -n " [3/3] WARP container internet... "
725     if docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace &>/dev/null;
726     then
727         echo "PASS"
728     else
729         echo "FAIL"
730         SKIP_EXIT_NODE=true
731     fi
732
733     # Act based on check results
734     if [ "$SKIP_EXIT_NODE" != "true" ]; then
735         echo -e "\nAll checks passed. Enabling exit node $TS_IP..."
736         if $TS_BIN up --reset --ssh=true --accept-dns=false --exit-node="$TS_IP" --exit-node-allow-lan-access=
737         true; then
738             echo "Exit node enabled."
739         else
740             echo "[Warning] Failed to set exit node."
741             SKIP_EXIT_NODE=true
742         fi
743     fi
744
745     if [ "$SKIP_EXIT_NODE" = "true" ]; then
746         echo -e "\n\033[0;33m[!] Pre-flight checks failed EXIT NODE + DNS HIJACK SKIPPED.\033[0m"
747         echo " Host stays on Tailscale mesh but your internet routes through your normal connection."
748         echo " Troubleshoot with:"
749         echo "   docker compose logs warp | tail -20"
750         echo "   docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace"
751         echo " Falling back to SOCKS5 proxy for traffic routing..."
752     fi

```

```

752 elif [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" = "false" ]; then
753     echo "USE_EXIT_NODE is false. Skipping exit node and DNS hijack."
754     SKIP_EXIT_NODE=true
755 fi
756
757 # 8. Apply DNS Hijacking.
758 # If exit node is enabled: hijack DNS to gateway's Tailscale IP (routes through tailnet).
759 # Otherwise, leave DNS at 1.1.1.1 (already set by reset at the top).
760 # AdGuard is also available at localhost:${DNS_PROXY_PORT} for manual configuration in apps.
761 if [ -n "$TS_IP" ] && [ "$HOST_ON_MESH" = "true" ] && [ "$SKIP_EXIT_NODE" != "true" ]; then
762     echo -e "\nHijacking host DNS to point ONLY at AdGuard Home ($TS_IP)..."
763     echo "Setting MagicDNS search domain 'ts.net' so tailnet hostnames resolve..."
764     if force_dns_to_gateway "$TS_IP"; then
765         echo "DNS hijack verified: single server = $TS_IP."
766         start_dns_watcher "$TS_IP"
767     else
768         echo "[Warning] DNS hijack could not be verified."
769         echo "    If DNS is broken, run manually:"
770         echo "        networksetup -setdnsservers \"\$ACTIVE_SERVICE\" \"$TS_IP"
771         echo "        networksetup -setsearchdomains \"\$ACTIVE_SERVICE\" ts.net"
772     fi
773 elif [ -n "$TS_IP" ]; then
774     echo -e "\n[Info] Exit node not enabled (pre-flight checks failed)."
775     echo "    Trying local DNS proxy for ad-blocking..."
776     if start_local_dns_proxy; then
777         echo "    Ad-blocking active via local DNS proxy through AdGuard."
778     else
779         echo "    Local DNS proxy unavailable. DNS stays at 1.1.1.1."
780         echo "    AdGuard available at http://localhost:80 (port ${DNS_PROXY_PORT} for DNS via TCP)."
781     fi
782
783 # Enable SOCKS5 proxy for traffic routing through WARP
784 # (enabled whenever exit node is skipped, regardless of HOST_ON_MESH)
785 echo -e "\n    Enabling SOCKS5 proxy for TCP traffic routing through gateway WARP tunnel..."
786 echo "    (SOCKS5 proxy runs inside gateway container, routes through tun0 -> WARP)"
787 enable_socks_proxy "$ACTIVE_SERVICE"
788
789 # Verify the SOCKS5 proxy works through WARP
790 echo -n "    Verifying SOCKS5 proxy routes through WARP... "
791 if curl --socks5-hostname 127.0.0.1:$SOCKS_PROXY_PORT --max-time 10 -s https://www.cloudflare.com/cdn-cgi/
792 trace 2>/dev/null | grep -q "warp=on"; then
793     echo "ok (warp=on)."
794     echo "    All TCP traffic now encrypted through Cloudflare WARP tunnel."
795 else
796     echo "FAILED (proxy not routing through WARP)."
797     echo "    Disabling SOCKS5 proxy internet stays on direct connection."
798     disable_socks_proxy
799     echo "    SOCKS proxy available at localhost:$SOCKS_PROXY_PORT for manual configuration."
800 fi
801 else
802     echo -e "\n[Warning] Could not resolve gateway Tailscale IP."
803 fi
804
805 # 9. Verify host connectivity
806 if [ "$HOST_ON_MESH" = "true" ] && [ -n "$TS_BIN" ]; then
807     if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
808         echo "Host is online on the Tailscale mesh."
809     else
810         echo "[Warning] Host is not responding on Tailscale."
811     fi
812 fi
813
814 ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
815 echo -e "\nGateway has been successfully STARTED in ${ELAPSED} seconds."
816
817 # Prevent system from going to sleep while gateway is running
818 start_sleep_prevention
819 fi
820
821 SUCCESS_RUN=true
822
823 # Final DNS state summary
824 if [ -n "$ACTIVE_SERVICE" ]; then
825     FINAL_DNS=$(resolvectl dns "$ACTIVE_SERVICE" 2>/dev/null | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | tr '
826 ' ' || true)
827     echo -e "\n"
828     echo -e "DNS STATE: $FINAL_DNS"
829     echo -e ""
830 fi
831
832 if [ "$START_GATEWAY" = "true" ] && command -v docker >/dev/null 2>&1; then

```

```

832 if docker compose logs rule-compiler 2>/dev/null | grep -q "Warning: Failed to fetch"; then
833     echo -e "\n[Warning] One or more blocklist URLs failed to download (404/Offline)."
```

834 echo " The gateway gracefully fell back to yesterday's cached blocklist."

835 echo " (Run 'docker logs rule-compiler' later to investigate which link died.)"

836 fi

837 fi

838

839 echo -e "\n"

840 read -rp "Press [r] and Enter to instantly reverse state, or just press Enter to exit: " USER_CHOICE

841 if [["\${USER_CHOICE}" == "r" || "\${USER_CHOICE}" == "R"]]; then

842 echo "Reversing gateway state..."

843 exec bash "\$0"

844 fi

845

846 echo -e "\nYou can close this terminal window now."

847

848 exit 0

849 fi

850

851 #

852 # macOS TOGGLE jot / netstat / networksetup, launchd / pf.

853 #

854

855 # Enforce Cryptographic Script Integrity

856 if [-f "scripts/crypto.sh"]; then

857 if ! bash scripts/crypto.sh --verify; then

858 exit 1

859 fi

860 fi

861

862 # Define log file

863 LOG_FILE="\$PWD/output.log"

864

865 # Rotate log file if it exceeds 1GB (1073741824 bytes).

866 # Sized large on purpose: with DEBUG_TRACE the log holds a full command-by-command

867 # trace effectively the entire compilation/warning history of the framework

868 # which is valuable to keep for post-mortems. Raised from 50MB so always-on

869 # tracing doesn't churn that history away prematurely.

870 if [-f "\$LOG_FILE"]; then

871 log_size=\$(stat -f%z "\$LOG_FILE" 2>/dev/null || stat -c%s "\$LOG_FILE" 2>/dev/null || echo 0)

872 if ["\$log_size" -gt 1073741824]; then

873 mv "\$LOG_FILE" "\${LOG_FILE}.old" 2>/dev/null || rm -f "\$LOG_FILE"

874 echo "[\$(date '+%Y-%m-%d %H:%M:%S')] Log file exceeded 1GB; rotated to output.log.old" > "\$LOG_FILE"

875 fi

876 fi

877

878 # --- LIFECYCLE PHASE TRACKING + EXIT BREADCRUMB ---

879 # TOGGLE_PHASE is updated as the script moves through STARTUP/STOP/START so that

880 # if the process is killed (window closed, pkill, OOM) or exits with an error,

881 # the EXIT trap records the final exit code AND the phase it died in. Previously

882 # an interrupted START left output.log ending abruptly after the teardown with no

883 # indication of what failed this makes every exit traceable.

884 TOGGLE_PHASE="init"

885 _log_exit_breadcrumb() {

886 local rc=\$?

887 echo "[\$(date '+%Y-%m-%d %H:%M:%S')] toggle.sh EXIT: code=\$rc phase=\$TOGGLE_PHASE (pid \$\$)" >> "\$LOG_FILE"

888 2>/dev/null || true

889 }

890 trap '_log_exit_breadcrumb' EXIT

891

892 # --- DEBUG TRACE (opt-in via .env: DEBUG_TRACE=true) ---

893 # When enabled, mirror a full command-by-command xtrace to output.log for a

894 # complete post-mortem trail (e.g. a race during --restart while Wi-Fi is

895 # re-associating). Off by default. Read directly from .env here because

896 # common.sh (read_env_var) isn't sourced yet.

897 #

898 # BASH_XTRACEFD (send xtrace to its own fd, keeping the terminal clean) needs

899 # bash >= 4.1. macOS ships /bin/bash 3.2, so we branch:

900 # - bash >= 4.1 (e.g. Linux): dedicate fd 9 to the log and point xtrace there;

901 # the terminal stays clean and normal stderr is untouched.

902 # - bash 3.2 (stock macOS): BASH_XTRACEFD is unsupported, so xtrace always goes

903 # to stderr. We redirect stderr *straight* to the log file* with a plain

904 # `exec 2>>"\$LOG\_FILE"`. This deliberately does NOT use a process

905 # substitution (`2>>(tee)`): under `set -e`, stderr flowing through the

906 # backgrounded `tee` proved to abort the script mid-START on 3.2 (a failed

907 # write / SIGPIPE in a pipeline returned non-zero and tripped `set -e`, with

908 # \$LINENO mis-blaming the enclosing `fi` see devref 15.11.8-A/E). The

909 # tradeoff: on 3.2 the terminal no longer shows live progress while tracing

910 # (everything is in output.log instead). Acceptable for an opt-in debug mode.

911 # We NEVER use the `exec {var}>>` dynamic-fd form (4.1+ only), which hard-fails on 3.2.

```

911 DEBUG_TRACE=$(grep -E '^DEBUG_TRACE=' .env 2>/dev/null | cut -d '=' -f2- | tr -d '\n' | tr '[:upper:]' '[:lower:]' | tr -d '[:space:]')
912 if [ "$DEBUG_TRACE" = "true" ]; then
913   echo "[$(date '+%Y-%m-%d %H:%M:%S')] DEBUG_TRACE enabled xtrace mirrored to output.log (bash ${BASH_VERSINFO
914     [0]}.${BASH_VERSINFO[1]})" >> "$LOG_FILE"
915   # Timestamped, location-aware xtrace prompt (works on 3.2 and 4.x).
916   export PS4='+ [${SECONDS}s] ${BASH_SOURCE##*/}:${LINENO}:${FUNCNAME[0]:-main}: ' # subshell-free clock: a $
917     () in PS4 trips the bash 3.2 set -x/ERR heisenbug (devref 15.11.8)
918   if [ "${BASH_VERSINFO[0]}" -gt 4 ] || { [ "${BASH_VERSINFO[0]}" -eq 4 ] && [ "${BASH_VERSINFO[1]}" -ge 1 ];
919     }; then
920     exec 9>>"$LOG_FILE"
921     export BASH_XTRACEFD=9
922     set -x
923   else
924     # bash 3.2 (stock macOS): plain stderrfile redirect (no process substitution).
925     exec 2>>"$LOG_FILE"
926     set -x
927   fi
928 fi
929
930 # --- MAIN EXECUTION LOGGING ---
931 echo -e "\n===== " >> "$LOG_FILE"
932 echo "[$(date '+%Y-%m-%d %H:%M:%S')] toggle.sh invoked" >> "$LOG_FILE"
933 echo "===== " >> "$LOG_FILE"
934
935 # (Previously this script chmod 600'd .env at start to "unlock" it for docker compose,
936 # and chmod 000'd it on exit. That pattern broke docker compose on macOS/Colima when
937 # the umask produced 0600 the script removed its own ability to read .env before it could even proceed.)
938
939 # --- PROCEDURAL PORT GENERATION ---
940 # To prevent static port fingerprinting by malware, we randomize exposed ports
941 # on every boot and persist them to a hidden environment file.
942 PORTS_FILE="/tmp/nullexit-ports.env"
943 if [[ "$1" == "" || "$1" == "--restart" ]]; then
944
945   # Collision-avoidance function
946   GENERATED_PORTS=""
947   get_free_port() {
948     local port
949     while true; do
950       port=$(jot -r 1 10000 65000)
951       # 1. Prevent self-collision within this loop
952       if [[ "$GENERATED_PORTS" == *"$port"* ]]; then
953         continue
954       fi
955       # 2. Prevent collision with ANY process on the Mac (root daemons, terminal apps, etc)
956       # We use netstat instead of lsof because netstat reads the kernel socket table
957       # directly and doesn't require sudo to see ports bound by other users/root.
958       if ! netstat -an -p tcp | awk '{print $4}' | grep -q "\.${port}"; then
959         echo "$port"
960         return
961       fi
962     done
963   }
964
965   export TOR SOCKS_PORT=$(get_free_port)
966   GENERATED_PORTS="$GENERATED_PORTS $TOR SOCKS_PORT"
967
968   export TOR_CONTROL_PORT=$(get_free_port)
969   GENERATED_PORTS="$GENERATED_PORTS $TOR_CONTROL_PORT"
970
971   export SOCKS_PROXY_PORT=$(get_free_port)
972   GENERATED_PORTS="$GENERATED_PORTS $SOCKS_PROXY_PORT"
973
974   export DNS_PROXY_PORT=$(get_free_port)
975
976   echo "export TOR SOCKS_PORT=$TOR SOCKS_PORT" > "$PORTS_FILE"
977   echo "export TOR_CONTROL_PORT=$TOR_CONTROL_PORT" >> "$PORTS_FILE"
978   echo "export SOCKS_PROXY_PORT=$SOCKS_PROXY_PORT" >> "$PORTS_FILE"
979   echo "export DNS_PROXY_PORT=$DNS_PROXY_PORT" >> "$PORTS_FILE"
980   ADGUARD_PASS=$(grep -E "^ADGUARD_PASSWORD=" .env 2>/dev/null | cut -d '=' -f2- | tr -d '\n')
981   echo "export TOR_PASSWORD=${ADGUARD_PASS:-nullexit}" >> "$PORTS_FILE"
982 elif [ -f "$PORTS_FILE" ]; then
983   # On STOP, source the existing ports so docker-compose down matches
984   source "$PORTS_FILE"
985 else
986   # Fallbacks if file is lost
987   export TOR SOCKS_PORT=9050
988   export TOR_CONTROL_PORT=9051
989   export SOCKS_PROXY_PORT=1080
990   export DNS_PROXY_PORT=5354

```



```

988 fi
989
990 # Start execution timer
991 TOGGLE_START_TIME=$SECONDS
992
993 if [[ "$1" == "--restart" ]]; then
994     echo "Executing Gateway Restart Sequence..."
995     # We must source common.sh here temporarily to check gateway state before we proceed
996     source "$SCRIPT_DIR/scripts/common.sh"
997     # Use persisted intent (marker) via gateway_state, which echoes a string (never
998     # a return code) to stay set -e-safe under set -x. See devref 15.11.8.
999     if [ "$(gateway_state)" = "running" ]; then
1000         echo "Gateway is currently running. Stopping it first..."
1001         bash "$0"
1002         echo "Gateway stopped. Now starting it..."
1003     else
1004         echo "Gateway is already stopped. Starting it..."
1005     fi
1006     bash "$0"
1007     exit 0
1008 fi
1009
1010 # Global flag to track if the script completed successfully
1011 SUCCESS_RUN=false
1012
1013 # Global variable to track the currently running background command PID
1014 CURRENT_BG_PID=""
1015
1016 # Source common bash functions
1017 source "$SCRIPT_DIR/scripts/common.sh"
1018
1019 # Prevent concurrent execution of lifecycle scripts (toggle.sh / recover.sh)
1020 LOCK_FILE="/tmp/nullexit-toggle.lock"
1021 if [ -f "$LOCK_FILE" ]; then
1022     LOCK_PID=$(cat "$LOCK_FILE" 2>/dev/null || echo "")
1023     if [ -n "$LOCK_PID" ] && [ "$LOCK_PID" != "$$" ] && kill -0 "$LOCK_PID" 2>/dev/null; then
1024         if ps -p "$LOCK_PID" -o args= 2>/dev/null | grep -q -E "toggle\.sh|recover\.sh"; then
1025             die "Another instance of toggle.sh or recover.sh (PID $LOCK_PID) is already running."
1026         fi
1027     fi
1028 fi
1029 echo "$$" > "$LOCK_FILE"
1030
1031 # Helper function to run a GUI command as the logged-in console user
1032 # (Prevents permission failures if the script is run with sudo)
1033 run_gui_cmd() {
1034     local console_user
1035     console_user=$(stat -f '%Su' /dev/console 2>> output.log || echo "$SUDO_USER")
1036     if [ -z "$console_user" ] || [ "$console_user" = "root" ]; then
1037         console_user=$(logname 2>> output.log || echo "$USER")
1038     fi
1039
1040     if [ -n "$console_user" ] && [ "$console_user" != "root" ] && [ "$EUID" -eq 0 ]; then
1041         sudo -u "$console_user" "$@"
1042     else
1043         "$@"
1044     fi
1045 }
1046
1047 # Active-state marker: written at end of START path so external watchers
1048 # (see launchd/com.nullexit.wake-recovery.plist + scripts/watcher.sh +
1049 # devref 10.29) know the gateway is currently up. They only fire
1050 # recover.sh --post-wake when this file is present. Cleared at the top
1051 # of the STOP path and inside cleanup_handler on any error/signal path
1052 # so a half-broken gateway never re-triggers post-wake refreshes.
1053 write_gateway_active_marker() {
1054     date -u +%FT%TZ > /tmp/nullexit-gateway-active.marker
1055     # Set the watcher debounce timestamp to now to prevent a redundant post-startup recovery run.
1056     date +%s > /tmp/nullexit-watcher.last-recovery 2>/dev/null || true
1057 }
1058
1059 clear_gateway_active_marker() {
1060     rm -f /tmp/nullexit-gateway-active.marker
1061     rm -f "$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/TUNNEL_FAILED_CLOSED.marker"
1062 }
1063
1064 # Capture whether the gateway-active marker exists BEFORE the defensive clear
1065 # below wipes it. The STOP-vs-START decision (gateway_should_be_running) reads
1066 # this captured value persisted *intent* instead of a live Docker probe, so
1067 # a disable is instant and works even when Colima/Docker is down. Must be read
1068 # before clear_gateway_active_marker or it would always look "stopped".

```

```

1069 #
1070 # Written as default-assign + `if` WITHOUT an `else`: under `set -e` (with the
1071 # DEBUG_TRACE xtrace redirect active) an `if [ -f ]; then ; else ; fi` whose
1072 # condition is false was aborting the script here at init on macOS /bin/bash 3.2
1073 # the false `[ -f ]` status leaked through the compound. An `if` with no `else`
1074 # yields status 0 when the condition is false, so nothing leaks.
1075 GATEWAY_MARKER_AT_STARTUP="no"
1076 if [ -f "$MARKER_FILE" ]; then
1077     GATEWAY_MARKER_AT_STARTUP="yes"
1078 fi
1079
1080 # Defensive: clear any stale marker from a prior crashed/aborted run. If a
1081 # previous toggle.sh wrote the marker but was OOM-killed in the middle of the
1082 # START block (before the END-of-START write) or if the user rebooted mid
1083 # session this prevents the watcher from firing post-wake against a
1084 # non-running gateway on every wake event. Placed AFTER the function
1085 # definitions so bash resolves the call at parse time.
1086 clear_gateway_active_marker
1087
1088 PID_FILE="$PID_CAFFEINATE"
1089
1090 # Start macOS caffeine to prevent sleep while gateway is running
1091 start_sleep_prevention() {
1092     if ! command -v caffeine >/dev/null 2>&1; then
1093         echo " [Warning] caffeine tool not found. Sleep prevention unavailable."
1094         return 1
1095     fi
1096
1097     # Stop any existing sleep prevention process first to avoid duplicates
1098     stop_pidfile_daemon "/tmp/nullexit-caffeinate.pid" "system sleep prevention"
1099
1100     echo " Enabling system sleep prevention (caffeinate)..."
1101     # Run caffeine wrapped in a bash trap to prevent idle system sleep.
1102     # Critically, this traps macOS shutdown signals (SIGTERM) and automatically
1103     # flushes hijacked DNS back to normal right before the Mac powers off.
1104     # We do not use recover.sh here because it is a heavy recovery script (managing
1105     # containers, restarting tailscaled, etc.) which takes too long and gets
1106     # terminated by macOS before it can reset the DNS. Instead, we surgically and
1107     # instantly restore normal DNS and proxy settings here.
1108     #
1109     # -i alone only blocks *idle* sleep a stopped/crashed gateway process,
1110     # a hung docker command, or any other assertion lapse still lets the Mac
1111     # sleep and drop every mesh device's exit node. -s additionally blocks
1112     # sleep outright whenever the Mac is on AC power (it's a no-op on battery,
1113     # so unplugged runs still sleep normally and don't burn charge).
1114     nohup bash -c "
1115     trap '
1116         echo \"[Shutdown Trap] Reverting network settings...\" >> \"\$PWD/output.log\" 2>&1
1117         kill \$! 2>/dev/null || true
1118         sudo -n networksetup -setdnsservers \"\$ACTIVE_SERVICE\" 1.1.1.1 >> \"\$PWD/output.log\" 2>&1 || true
1119         sudo -n networksetup -setdnsservers \"\$ENO_SERVICE\" 1.1.1.1 >> \"\$PWD/output.log\" 2>&1 || true
1120         sudo -n networksetup -setsearchdomains \"\$ACTIVE_SERVICE\" \"Empty\" >> \"\$PWD/output.log\" 2>&1 || true
1121         sudo -n networksetup -setsearchdomains \"\$ENO_SERVICE\" \"Empty\" >> \"\$PWD/output.log\" 2>&1 || true
1122         sudo -n networksetup -setsocksfirewallproxystate \"\$ACTIVE_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 ||
1123         true
1124         sudo -n networksetup -setsocksfirewallproxystate \"\$ENO_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 || true
1125         sudo -n networksetup -setwebproxystate \"\$ACTIVE_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 || true
1126         sudo -n networksetup -setwebproxystate \"\$ENO_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 || true
1127         sudo -n networksetup -setsecurewebproxystate \"\$ACTIVE_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 || true
1128         sudo -n networksetup -setsecurewebproxystate \"\$ENO_SERVICE\" off >> \"\$PWD/output.log\" 2>&1 || true
1129         if command -v tailscale >/dev/null 2>&1; then
1130             tailscale up >> \"\$PWD/output.log\" 2>&1 || true
1131             tailscale set --accept-dns=false --exit-node= >> \"\$PWD/output.log\" 2>&1 || true
1132             TS_DOWN_ARGS=""
1133             if [ \"\$KILL_SWITCH\" = \"true\" ]; then TS_DOWN_ARGS="--accept-risk=lose-ssh\"; fi
1134             tailscale down \$TS_DOWN_ARGS >> \"\$PWD/output.log\" 2>&1 &
1135         fi
1136         sleep 1
1137         echo \"[Shutdown Trap] Cleanup complete.\" >> \"\$PWD/output.log\" 2>&1
1138         exit 0
1139     ' SIGTERM SIGINT SIGHUP
1140     caffeine -i -s &
1141     wait \$!
1142     \" >> output.log 2>&1 &
1143     local caffe_pid=$!
1144     echo \"$caffe_pid\" > \"$PID_FILE"
1145     echo " Sleep prevention active (PID $caffe_pid). Your Mac won't sleep while the gateway is running."
1146 }
1147
1148

```

```

1149 DNS_WATCHER_PID_FILE="$PID_DNS_WATCHER"
1150 WARP_WATCHER_PID_FILE="/tmp/nullexit-warp-watcher.pid"
1151
1152
1153
1154 # Background WARP tunnel liveness monitor. Polls cdn-cgi/trace every 30s
1155 # while healthy, but accelerates to polling every 5s if a failure is detected.
1156 # Always logs state transitions to output.log. When warp=off persists for
1157 # WARP_FAIL_THRESHOLD consecutive polls (default 6 = 30s of downtime), the watcher
1158 # triggers a nuclear recovery: runs recover.sh to tear down the entire
1159 # gateway disconnecting Tailscale, stopping containers, resetting DNS,
1160 # and power-cycling Wi-Fi. Note: This minimizes exposure window, but
1161 # only the pf kill switch guarantees absolutely zero leakage on drop.
1162 #
1163 # The threshold (default 6) is statistically chosen: even at an
1164 # unrealistically high 10% false-positive rate per poll, P(6 consecutive
1165 # false positives) = 0.1^6 0.0001%. Real-world false-positive rates
1166 # are far lower, making 30s of continuous downtime overwhelmingly likely
1167 # to be a genuine outage.
1168 #
1169 # Configurable via .env: WARP_FAIL_THRESHOLD=3 (15s, more aggressive)
1170 start_warp_watcher() {
1171     stop_warp_watcher
1172     # Parse threshold from .env (same pattern as GATEWAY_MSS, GATEWAY_BYPASS_PING, etc.)
1173     local threshold
1174     threshold=$(read_env_var WARP_FAIL_THRESHOLD)
1175     if [ -z "$threshold" ]; then
1176         threshold="${WARP_FAIL_THRESHOLD:-6}"
1177     fi
1178     local trigger_file="/tmp/nullexit-warp-shutdown-triggered"
1179     rm -f "$trigger_file"
1180     # Resolve the physical uplink interface (Wi-Fi/Ethernet) once, same convention
1181     # as wait_for_dhcp_settle(). The watcher uses it to tell "no uplink at all"
1182     # (e.g. Wi-Fi dropped with auto-join off) apart from a genuine WARP failure,
1183     # so a transient link loss can't false-trigger the nuclear gateway teardown.
1184     local phys_iface
1185     phys_iface=$(networksetup -listallhardwareports 2>/dev/null | awk '/Hardware Port: (Wi-Fi|Ethernet)/{getline;
1186         print $2; exit}')
1187     [ -z "$phys_iface" ] && phys_iface="en0"
1188     echo " Starting background WARP Watcher (polling every 30s [accelerating to 5s on failure], shutdown after $
1189     {threshold} consecutive failures; paused while ${phys_iface} has no uplink)..."
1190     nohup bash -c "
1191         trap 'exit 0' SIGTERM SIGINT SIGHUP
1192         last_state='on'
1193         consec_off=0
1194         paused=0
1195         threshold='$threshold'
1196         trigger_file='$trigger_file'
1197         phys_iface='$phys_iface'
1198         out_log='$PWD/output.log'
1199         recover_bin='$PWD/recover.sh'
1200         inhibit_file='/tmp/nullexit-warp-inhibit.marker'
1201         wifi_helper='$PWD/scripts/wifi-rejoin.sh'
1202         while true; do
1203             # Inhibit check
1204             # recover.sh --post-wake writes this marker while force-recreating the
1205             # warp container. During that window, the container is intentionally
1206             # down don't count it as a failure or we'll fire nuclear recover.sh
1207             # and kill a gateway that was in the process of healing itself.
1208             if [ -f "\$inhibit_file" ]; then
1209                 consec_off=0
1210                 sleep 5
1211                 continue
1212             fi
1213             # Physical-uplink check
1214             # If the Mac has no physical uplink (e.g. Wi-Fi dropped and auto-join
1215             # is off), the cdn-cgi/trace probe below reads warp=off only because
1216             # there's no internet NOT because the WARP tunnel failed. With no
1217             # uplink there is also no egress path, so nothing can leak. Pause
1218             # failure-counting until the link returns instead of nuking a gateway
1219             # that heals itself once Wi-Fi reconnects. (Link up + warp=off is a
1220             # genuine failure and still trips the kill-switch below, unchanged.)
1221             phys_ip=$(ipconfig getifaddr "\$phys_iface" 2>/dev/null)
1222             case "\$phys_ip" in
1223                 ''|169.254.*)
1224                     if [ "\$paused" = 0 ]; then
1225                         echo "\[$(date -u +%FT%TZ)] WARP watcher PAUSED no physical uplink on \$phys_iface (Wi-Fi down /
1226                         auto-join off?). Not counting warp=off as a failure; gateway left intact until the link returns.\" >> \"\$
1227                         $out_log\"
1228                         paused=1

```

```

1226         fi
1227         # Re-associate ONLY the last-known-good network (never any other),
1228         # after a grace period so a DHCP-only blip can self-heal first.
1229         bash "\$wifi_helper\" rejoin >/dev/null 2>&1 || true
1230         consec_off=0
1231         sleep 10
1232         continue
1233     ;;
1234     esac
1235     if [ \"\$paused\" = 1 ]; then
1236         echo \"[\$(date -u +%FT%TZ)] WARP watcher RESUMED  physical uplink back on \$phys_iface (\$phys_ip).
Normal WARP monitoring restored.\" >> \"\$out_log\"
        paused=0
1237         # Link is back clear the drop-timer so the next drop re-arms the grace window.
1238         bash \"\$wifi_helper\" reset >/dev/null 2>&1 || true
1239     fi
1240
1241     state='off'
1242     if docker compose exec -T warp wget -q0- --timeout=5 \
1243         https://www.cloudflare.com/cdn-cgi/trace 2>/dev/null \
1244         | grep -q 'warp=on'; then
1245         state='on'
1246     fi
1247
1248     # State-transition logging
1249     if [ \"\$state\" != \"\$last_state\" ]; then
1250         if [ \"\$state\" = 'off' ]; then
1251             echo \"[\$(date -u +%FT%TZ)] WARP DOWN  cdn-cgi/trace reports warp=off\" >> \"\$out_log\"
1252         fi
1253         last_state=\"\$state\"
1254     fi
1255
1256     # Consecutive-failure tracking + auto-shutdown
1257     if [ \"\$state\" = 'off' ]; then
1258         consec_off=$((consec_off + 1))
1259         if [ \"\$consec_off\" -ge \"\$threshold\" ] && [ ! -f \"\$trigger_file\" ]; then
1260             touch \"\$trigger_file\"
1261             echo \"[\$(date -u +%FT%TZ)] WARP SHUTDOWN  \$consec_off consecutive failures (threshold=\$threshold)
. Running recover.sh to kill the gateway and restore normal internet. Your IP is NO LONGER Cloudflare.\" >>
\"\$out_log\"
1262             # Nuclear recovery: tear down the entire gateway.
1263             # recover.sh resets DNS 1.1.1.1, disconnects Tailscale,
1264             # stops Docker containers, flushes routes, power-cycles Wi-Fi.
1265             bash \"\$recover_bin\" --auto >> \"\$out_log\" 2>&1 || true
1266             echo \"[\$(date -u +%FT%TZ)] WARP SHUTDOWN  recover.sh completed, watcher exiting. Your IP is now
your real ISP IP.\" >> \"\$out_log\"
1267             exit 0
1268         fi
1269     else
1270         if [ \"\$consec_off\" -gt 0 ]; then
1271             echo \"[\$(date -u +%FT%TZ)] WARP RECOVERED  warp=on after \$consec_off consecutive off readings (
threshold was \$threshold)\" >> \"\$out_log\"
1272         fi
1273         consec_off=0
1274     fi
1275
1276     if [ \"\$state\" = \"off\" ]; then
1277         sleep 5
1278     else
1279         sleep 30
1280     fi
1281     done
1282     \" >> output.log 2>&1 &
1283     echo \$! > \"$WARP_WATCHER_PID_FILE\"
1284 }
1285
1286 stop_warp_watcher() {
1287     if [ -f \"$WARP_WATCHER_PID_FILE\" ]; then
1288         local wp
1289         wp=$(cat \"$WARP_WATCHER_PID_FILE\")
1290         if [ -n \"$wp\" ] && kill -0 \"$wp\" 2>/dev/null; then
1291             echo \" Stopping background WARP Watcher (PID $wp)...\"
1292             kill \"$wp\" 2>/dev/null || true
1293         fi
1294         rm -f \"$WARP_WATCHER_PID_FILE\"
1295     fi
1296 }
1297
1298
1299
1300
1301

```

```

1302
1303 # Cleanup handler to restore DNS to 1.1.1.1 on error or user interrupt (Ctrl+C / SIGTERM / SIGHUP)
1304 cleanup_handler() {
1305     local exit_code=$?
1306     local trigger_type="$1"
1307     local line_no="$2"
1308
1309     if [ "$SUCCESS_RUN" = "false" ]; then
1310         echo -e "\n[Self-Correction] Script interrupted or failed ($trigger_type). Restoring host DNS to 1.1.1.1..."
1311         if [ -n "$ACTIVE_SERVICE" ]; then
1312             reset_dns
1313         fi
1314
1315         if [ -n "$CURRENT_BG_PID" ]; then
1316             echo "Terminating active command (PID $CURRENT_BG_PID)..."
1317             kill -15 "$CURRENT_BG_PID" 2>> output.log || true
1318         fi
1319
1320         # Disconnect host Tailscale and close the GUI application on failure
1321         disconnect_tailscale_host
1322
1323         # Stop local DNS proxy if running
1324         stop_local_dns_proxy
1325
1326         # Stop sleep prevention
1327         stop_pidfile_daemon "/tmp/nullexit-caffeinate.pid" "system sleep prevention"
1328         stop_pidfile_daemon "/tmp/nullexit-dns-watcher.pid" "background DNS Watcher"
1329         stop_warp_watcher
1330         clear_gateway_active_marker
1331
1332         # Capture warp logs on failure for debugging before teardown
1333         if [ "$trigger_type" = "ERR" ]; then
1334             echo -e "\n--- WARP FAILURE LOGS ---" >> output.log
1335             docker compose logs warp --tail=100 >> output.log 2>&1 || true
1336             echo "-----" >> output.log
1337         fi
1338
1339         # Best-effort container teardown on failure
1340         docker compose down --remove-orphans -t 30 2>> output.log || true
1341
1342         # Nuke leftover network state (proxies, routes, DNS cache, Wi-Fi)
1343         if [ -n "$ACTIVE_SERVICE" ]; then
1344             cleanup_network_state
1345         fi
1346
1347
1348         if [ "$trigger_type" = "ERR" ]; then
1349             echo -e "\n=====
1350             echo "ERROR: Script failed at line $line_no with exit code $exit_code."
1351             echo "=====
1352             echo "Troubleshooting steps:"
1353             echo "1. Run 'colima status' to verify the VM is active."
1354             echo "2. Run 'docker compose ps' to check container health."
1355             echo "3. Run 'docker compose logs' to view application logs."
1356             echo "4. Run 'tailscale status' to check host Tailscale status."
1357             echo "=====
1358         fi
1359         rm -f "${LOCK_FILE:-/tmp/nullexit-toggle.lock}"
1360     fi
1361 }
1362
1363 trap 'cleanup_handler ERR $LINENO' ERR
1364 trap 'cleanup_handler INT' INT
1365 trap 'cleanup_handler TERM' TERM
1366 trap 'cleanup_handler HUP' HUP
1367
1368 # NOPASSWD sudo
1369 # The user has NOPASSWD in /etc/sudoers.d/nullexit for the specific commands
1370 # needed (networksetup, pfctl, route, ifconfig, dscacheutil, killall, pkill,
1371 # and python3 -I scripts/dns-proxy.py for the fallback DNS proxy).
1372 # All privileged calls below use `sudo -n` which will run without prompting.
1373 # No credential caching loop is needed.
1374
1375 # Add Homebrew and standard paths
1376 setup_standard_path
1377
1378 # Check for local DNS proxy tools (used when tailnet data plane is unavailable)
1379 # Uses a Python DNS proxy (handles TCP wire format correctly).
1380 # Python3 is built into macOS no external dependencies.
1381 DNS_PROXY_BIN=""

```

```

1382 if command -v python3 >> output.log 2>&1; then
1383     DNS_PROXY_BIN="python3 -I"
1384 elif command -v python >> output.log 2>&1; then
1385     DNS_PROXY_BIN="python -I"
1386 fi
1387
1388 # Path to the DNS proxy script (sibling of this script)
1389 DNS_PROXY_SCRIPT="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/scripts/dns-proxy.py"
1390
1391 # Resolve Tailscale CLI on host. We rely on the standalone Homebrew formula
1392 # (`brew install tailscale` + `brew services start tailscale`) NOT the
1393 # Tailscale.app GUI so there is no .app bundle or Network Extension sandbox
1394 # to launch, no menu-bar click to perform, and no scutil Network Service to
1395 # start manually. tailscaled runs as a per-user LaunchAgent (or LaunchDaemon
1396 # if you ran the install with sudo) and the control socket is always available.
1397 TS_BIN=""
1398 if command -v tailscale >> output.log 2>&1; then
1399     TS_BIN="tailscale"
1400 fi
1401
1402 # Baseline: disable Tailscale DNS management at the daemon level.
1403 # `tailscale set` writes a persistent preference. However, `tailscale up --reset`
1404 # (used in steps 7 and 9) RESETS all unspecified flags to defaults, which would
1405 # re-enable `--accept-dns=true` and nullify this `set`.
1406 # Therefore, EVERY `tailscale up --reset` call in this file MUST also pass
1407 # `--accept-dns=false` explicitly. See steps 7 and 9 for the fix.
1408 #
1409 # This initial `set` is still useful as a baseline for non-reset operations
1410 # (e.g. if the user runs `tailscale up` manually without --reset) and covers
1411 # the brief window between this line and the first `--reset` call.
1412 #
1413 # Runs once at script start while tailscaled is still connected (if it was
1414 # left running from a previous toggle), so the pref takes effect before any
1415 # disconnection or reconnection logic.
1416 if [ -n "$TS_BIN" ]; then
1417     $TS_BIN set --accept-dns=false >> output.log 2>&1 || true
1418 fi
1419
1420 # Disconnect host Tailscale from the mesh. With the standalone daemon, this is the
1421 # *entire* teardown no scutil tunnel stop, no Tailscale.app GUI quit, no pkill.
1422 # tailscaled keeps running (as a LaunchAgent / LaunchDaemon), which is intentional:
1423 # the next `tailscale up` then reconnects in seconds rather than waiting for a
1424 # slow .app launch + Network Extension activation. The `tailscale down` call also
1425 # retains the user's auth state, so we don't force a browser re-login every cycle.
1426 #
1427 # Defined early (before the if/else branches and referenced by cleanup_handler's
1428 # ERR trap) so that any failure before the main logic can still tear down Tailscale.
1429
1430
1431 # Wait for Network Readiness
1432 # On --restart, the gateway tears down while the host network interface may
1433 # still be reconnecting (Wi-Fi, Ethernet, USB tethering, etc.). If we proceed
1434 # too early, get_active_service returns nothing, routing targets the wrong
1435 # interface, and DNS/WARP setup silently fails.
1436 #
1437 # We use `route get default` to detect a valid default gateway rather than
1438 # checking a specific interface (e.g. en0) this generalizes across all
1439 # connection types: Wi-Fi, Ethernet, USB-C adapters, iPhone tethering, etc.
1440 # The moment the OS has any routable default gateway, we proceed.
1441 _net_wait_secs=0
1442 _net_timeout=60
1443 echo "Waiting for network connectivity before starting..."
1444 while true; do
1445     if route get default 2>/dev/null | grep -q 'gateway: '; then
1446         _gw=$(route get default 2>/dev/null | awk '/gateway:/{print $2}')
1447         echo " Network ready (default gateway: $_gw)."
1448         break
1449     fi
1450     if [ "$_net_wait_secs" -ge "$_net_timeout" ]; then
1451         echo " [Warning] No default gateway after ${_net_timeout}s proceeding anyway."
1452         break
1453     fi
1454     sleep 2
1455     _net_wait_secs=$(( _net_wait_secs + 2 ))
1456 done
1457
1458 ACTIVE_SERVICE=$(get_active_service)
1459
1460 # Resolve the service name for en0 (usually "Wi-Fi") macOS scutil DNS resolver
1461 # is commonly scoped to en0, so per-service DNS changes on other interfaces are
1462 # ignored unless en0's service is also updated.

```



```

1463 ENO_SERVICE=$(get_en0_service)
1464
1465 # Helper to add host-side bypass routes for the WARP WireGuard endpoints.
1466 # This prevents an infinite recursive routing loop (tunnel loop) where
1467 # the container's WireGuard packets exit the VM, reach the host, match the
1468 # default route (the exit node), and get routed back into the tunnel.
1469 # We route via the gateway IP (not interface) to ensure packets are routed
1470 # properly even when the host's default route changes to the Tailscale interface.
1471
1472 # Helper to manually override the host's default route to point through the
1473 # Tailscale utun* interface. Standalone tailscaled on macOS (Homebrew version)
1474 # lacks the platform-specific integration to override the default gateway
1475 # automatically when --exit-node is used.
1476 setup_exit_node_routing() {
1477     local ts_iface
1478     ts_iface=$(ifconfig | awk '/^[a-z0-9]+:/{iface=$1} /inet 100\./{print iface; exit}' | tr -d ':')
1479
1480     if [ -n "$ts_iface" ]; then
1481         echo "Re-routing default gateway to $ts_iface using 0.0.0.0/1 split..."
1482         local host_mtu
1483         host_mtu=$(read_env_var HOST_MTU)
1484         host_mtu=${host_mtu:-1200}
1485
1486         # Lower the host Tailscale MTU (defaults to 1200) to prevent fragmentation when passing
1487         # through the container's WARP tunnel (which also has an MTU of 1280).
1488         sudo -n ifconfig "$ts_iface" mtu "$host_mtu" >> output.log 2>&1 || true
1489
1490         # DO NOT delete the physical default route! It crashes Colima.
1491         # Use the /1 trick to mathematically override the default route.
1492         sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
1493         sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
1494         sudo -n route add -net 0.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
1495         sudo -n route add -net 128.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
1496
1497         if netstat -nr | grep -E -q "^(0\.\0\.\0/1|0/1)[[:space:]]+.*$ts_iface"; then
1498             echo " [] Default route successfully overridden via $ts_iface."
1499         else
1500             echo " [!] Failed to verify exit node routing on $ts_iface."
1501         fi
1502     else
1503         echo "[Warning] Could not detect Tailscale utun interface for host routing."
1504     fi
1505 }
1506
1507 # Helper to nuke leftover network state after teardown (proxy settings, routing
1508 # table, DNS cache, Wi-Fi). Prevents the "had to write a whole recovery script"
1509 # problem where stale state kills internet after the gateway stops.
1510 cleanup_network_state() {
1511     echo -e "\nCleaning up network state (proxies, routes, DNS cache, Wi-Fi)..."
1512
1513     # 1. Disable any leftover proxy settings (needs root on many macOS versions)
1514     for svc in "$ACTIVE_SERVICE" "$ENO_SERVICE"; do
1515         disable_all_proxies "$svc"
1516     done
1517     echo " Proxies disabled."
1518
1519     # 2. Flush DNS cache
1520     if command -v dscacheutil >> output.log 2>&1; then
1521         sudo -n dscacheutil -flushcache 2>> output.log || true
1522     fi
1523     sudo -n killall -HUP mDNSResponder 2>> output.log || true
1524     echo " DNS cache flushed."
1525
1526     # 3. Flush stale routing table entries
1527     if command -v route >> output.log 2>&1; then
1528         sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
1529         sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
1530     fi
1531     remove_warp_bypass_routes
1532     remove_apple_split_routes
1533     disable_killswitch
1534     echo " Routing table and firewall flushed."
1535
1536     # 4. Power-cycle Wi-Fi to clear any lingering interface state
1537     WIFI_PORT=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: Wi-Fi/{getline; print $2}'
1538     )
1539     if [ -n "$WIFI_PORT" ]; then
1540         sudo -n networksetup -setairportpower "$WIFI_PORT" off 2>> output.log || true
1541         sleep 2
1542         sudo -n networksetup -setairportpower "$WIFI_PORT" on 2>> output.log || true
1543         echo " Wi-Fi power-cycled (interface $WIFI_PORT)."

```

```

1543     sleep 3
1544 else
1545     echo " Wi-Fi interface not detected; bouncing en0..."
1546     sudo -n ifconfig en0 down 2>> output.log || true
1547     sudo -n ifconfig en0 up 2>> output.log || true
1548 fi
1549
1550 # Force restart sharing services to prevent AirDrop / AirPlay discovery freezes
1551 # after network interface/DNS changes.
1552 echo " Resetting macOS sharing services (AirDrop/AirPlay)..."
1553 reset_sharing_services
1554
1555 echo "Network state cleanup complete."
1556 }
1557
1558 SOCKS_PROXY_PORT=${SOCKS_PROXY_PORT}
1559
1560 # Ensure DNS is set to 1.1.1.1 immediately at script start to prevent any DNS deadlocks/hangs
1561 # during status checks, container startup, VM boot, or teardown. We *also* clear any leftover
1562 # `ts.net` search domain from a previous `tailscale up --accept-dns=true` run, otherwise macOS
1563 # would prepend it to every lookup and most public DNS queries would resolve to NXDOMAIN.
1564
1565
1566 echo "Initializing DNS to 1.1.1.1 to ensure reliable internet access..."
1567 reset_dns
1568
1569
1570
1571
1572 echo "Checking Gateway Status..."
1573 HIJACK_HOST=$(read_env_var GATEWAY_HIJACK_HOST | tr '[:upper:]' '[:lower:]')
1574
1575 # Decide STOP vs START from persisted intent (marker), not a live Docker probe.
1576 # This makes *disable* instant and robust even when Colima/Docker is down.
1577 #
1578 # CRITICAL (set -e + set -x on bash 3.2): gateway_state ECHOES "running"/"stopped"
1579 # and always returns 0, so we branch on the STRING never on a function's exit
1580 # code. A helper that `return 1`'s trips a spurious `set -e` abort under `set -x`
1581 # (DEBUG_TRACE=true) even when guarded, hence the echo contract. See devref 15.11.8.
1582 if [ "$(gateway_state)" = "running" ]; then
1583     TOGGLE_PHASE="stop"
1584     echo -e "\n=====
1585     echo "Gateway is RUNNING. Stopping it now..."
1586     echo -e "=====
1587
1588 # 1. Disconnect host Tailscale cleanly first before stopping the exit-node container
1589 if [[ "$HIJACK_HOST" != "false" ]]; then
1590     disconnect_tailscale_host
1591     echo ""
1592 fi
1593
1594 echo "Stopping Docker containers..."
1595 # `|| true`: a disable must always complete its teardown even if the Docker
1596 # daemon / Colima VM is already down (compose down would exit non-zero and,
1597 # under set -e, abort the STOP). The START path intentionally does NOT guard
1598 # its `docker compose up` a failed start SHOULD trigger cleanup.
1599 run_logged docker compose down --remove-orphans -t 30 || true
1600
1601 # Only stop Colima if the user explicitly opted in (false by default) to prevent breaking their other Docker
1602 # dev projects
1603 STOP_COLIMA=$(read_env_var STOP_COLIMA_ON_EXIT | tr '[:upper:]' '[:lower:]')
1604 if [[ "$STOP_COLIMA" == "true" ]]; then
1605     echo -e "\nStopping Colima VM to free up host RAM and battery..."
1606     run_with_timeout 30 colima stop >> output.log 2>&1 || echo "Warning: Failed to stop Colima gracefully."
1607 else
1608     echo -e "\nLeaving Colima running. (Set STOP_COLIMA_ON_EXIT=true in .env to change this)"
1609 fi
1610
1611 if [[ "$HIJACK_HOST" != "false" ]]; then
1612     # The host's DNS was hijacked to the gateway IP during ENABLE; now that the
1613     # gateway is down, restore DNS to 1.1.1.1 immediately so subsequent lookups
1614     # don't stall for the macOS DNS timeout before the 1.1.1.1 fallback engages.
1615     echo -e "\nRestoring host DNS to 1.1.1.1 (gateway is gone)..."
1616     reset_dns
1617
1618 # 3. Stop local DNS proxy if running
1619 stop_local_dns_proxy
1620
1621 # Stop background daemons
1622 stop_pidfile_daemon "/tmp/nullexit-cafeinate.pid" "system sleep prevention"
1623 stop_pidfile_daemon "/tmp/nullexit-dns-watcher.pid" "background DNS Watcher"

```

```

1623 stop_warp_watcher
1624 clear_gateway_active_marker
1625
1626 # 4. Nuke leftover network state so internet actually works after teardown
1627 cleanup_network_state
1628 fi
1629
1630 ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
1631 echo -e "\nGateway has been successfully STOPPED in ${ELAPSED} seconds."
1632 else
1633 TOGGLE_PHASE="start"
1634 START_GATEWAY=true
1635 echo "[$(date +%Y-%m-%d %H:%M:%S)] Action: STARTUP GATEWAY" >> "$LOG_FILE"
1636 echo -e "\n=====
1637 echo "Gateway is STOPPED. Starting it now..."
1638 echo -e "=====\\n"
1639
1640 # 1. Prevent host exit-node deadlock during VM / Container startup
1641 if [[ "$HIJACK_HOST" != "false" ]]; then
1642     disconnect_tailscale_host
1643 fi
1644
1645 # 2. Wait for physical DHCP lease to settle (crucial for restarts/wake-up/roaming)
1646 wait_for_dhcp_settle
1647
1648 # 3. Boot Colima VM if it is not already running
1649 echo -e "\nChecking Colima VM status..."
1650 if ! run_with_timeout 15 colima status >> output.log 2>&1; then
1651     # Colima networking is gated on the SAME LAN-P2P signal routing-fix uses
1652     # (.lan_p2p_detected overrides TAILSCALE_ALLOW_LAN_P2P in .env). A LAN-reachable
1653     # VM (--network-address + bridged/shared) needs the socket_vmnet helper and is
1654     # only worthwhile on a trusted home network, so we request it only when P2P is
1655     # allowed; otherwise AP-isolated / enterprise, the common case we boot plain
1656     # user-mode networking. Either way host<->container P2P is UNAFFECTED: it rides
1657     # routing-fix's Docker-gateway / .host_ips RETURN rules, not the VM network mode
1658     # (see devref 15.3.5). colima_start_until_ready returns as soon as Docker answers
1659     # (~15s) instead of blocking on colima's ~2min post-provision hang (see 15.8.7);
1660     # that hang is colima-internal and mode-independent (it also occurs in user mode).
1661     allow_p2p=$(read_env_var "TAILSCALE_ALLOW_LAN_P2P")
1662     [ -z "$allow_p2p" ] && allow_p2p="false"
1663     if [ -f "$SCRIPT_DIR/.lan_p2p_detected" ]; then
1664         allow_p2p=$(tr -d '[:space:]' < "$SCRIPT_DIR/.lan_p2p_detected" 2>/dev/null || echo "false")
1665     fi
1666
1667     if [ "$allow_p2p" = "true" ]; then
1668         colima_network_mode="bridged"
1669         if [[ "$OSTYPE" == "darwin*" ]]; then
1670             security_type=$(system_profiler SPAirPortDataType 2>/dev/null | awk '/Current Network Information:/{
1671             found=1} found && /Security:/{print $NF; exit}')
1672             if echo "$security_type" | grep -qi "Enterprise"; then
1673                 echo -e "\n [!] WPA2-Enterprise Wi-Fi detected! Bridged mode will cause MAC-spoofing
1674                 deauthentication."
1675                 echo "      Falling back to '--network-mode shared' for Colima."
1676                 colima_network_mode="shared"
1677             fi
1678         fi
1679         echo "Colima is not running. Starting Colima (600MB RAM, vz VM, LAN P2P on: --network-address
1680         $colima_network_mode)..."
1681         colima_start_until_ready --memory 0.6 --vm-type vz --network-address --network-mode "$colima_network_mode
1682         " \
1683         || echo " [!] Colima Docker not confirmed ready; continuing (downstream docker-ps auto-heal will retry
1684         )."
1685     else
1686         colima_network_mode="user"
1687         echo "Colima is not running. Starting Colima (600MB RAM, vz VM, LAN P2P off: fast user-mode networking)
1688         ..."
1689         colima_start_until_ready --memory 0.6 --vm-type vz \
1690         || echo " [!] Colima Docker not confirmed ready; continuing (downstream docker-ps auto-heal will retry
1691         )."
1692     fi
1693     echo "$colima_network_mode" > /tmp/nullexit_colima_mode.txt
1694 else
1695     echo "Colima is already running."
1696 fi
1697
1698 # 3b. Auto-detect and fix Docker Subnet Collisions (e.g. if the user travels to a 172.17.x.x Wi-Fi)
1699 if [ -f "scripts/fix-docker-bridge-collision.sh" ]; then
1700     if bash scripts/fix-docker-bridge-collision.sh --dry | grep -q "collision detected"; then
1701         echo -e "\n[!] WARNING: Docker Subnet Collision Detected with your current Wi-Fi!"
1702         echo -e "      Auto-fixing Colima config to prevent internet jitter..."
1703         bash scripts/fix-docker-bridge-collision.sh --skip-toggle || true

```

```

1697     fi
1698 fi
1699
1700 # 3c. Configure swap file inside the VM to prevent OOM on low-memory limits
1701 # We also set vm.swappiness=10 so the Linux kernel strictly prefers physical RAM
1702 # and avoids unnecessarily wearing out the SSD with proactive background swapping.
1703 if ! run_with_timeout 15 colima ssh -- grep -q 'swapfile' /proc/swaps >> output.log 2>&1; then
1704     echo "Configuring 400MB swap file inside the VM to prevent OOM..."
1705     run_with_timeout 30 colima ssh -- sudo sh -c "if [ ! -f /swapfile ]; then dd if=/dev/zero of=/swapfile bs=1
        M count=400 status=none && mkswap /swapfile; fi && swapon /swapfile && sysctl vm.swappiness=10" >> output.
        log 2>&1 || echo "Warning: Failed to enable swap file inside the VM."
1706 fi
1707
1708 # 4. Clean up corrupted AdGuardHome configurations
1709 if [ -f "adguard/conf/AdGuardHome.yaml" ] && [ ! -s "adguard/conf/AdGuardHome.yaml" ]; then
1710     rm -f "adguard/conf/AdGuardHome.yaml"
1711     echo "Removed corrupted empty AdGuardHome.yaml to prevent container crash loop."
1712 fi
1713
1714 # 4b. Auto-heal stale Docker socket from hard reboots
1715 if ! run_with_timeout 15 docker ps >/dev/null 2>&1; then
1716     echo "Docker daemon is unresponsive (likely a stale socket from a crash). Auto-healing Colima..."
1717     run_with_timeout 120 colima restart >> output.log 2>&1
1718 fi
1719
1720 # 4c. Inject TCP MSS clamp from .env into post-rules.txt before starting
1721 if [ -f .env ] && grep -q "^GATEWAY_MSS=" .env; then
1722     GATEWAY_MSS=$(read_env_var GATEWAY_MSS)
1723     if [[ "$GATEWAY_MSS" =~ ^[0-9]+$ ]]; then
1724         # macOS sed syntax
1725         sed -i '' "s/--set-mss [0-9]*/--set-mss ${GATEWAY_MSS}/g" post-rules.txt 2>/dev/null || \
1726         # Linux sed fallback
1727         sed -i "s/--set-mss [0-9]*/--set-mss ${GATEWAY_MSS}/g" post-rules.txt 2>/dev/null
1728     fi
1729 fi
1730
1731 # 5. Start compose services
1732 write_host_ips
1733
1734 # Procedurally generated ports have already been exported at the top of the script.
1735 echo " [Security] Procedurally randomized proxy ports generated to evade fingerprinting."
1736
1737 # --- HONEY-PORT TRIPWIRE ---
1738 # Generate a random trap port. If any malware does a sequential port scan on localhost,
1739 # it will inevitably hit this port. When it does, we instantly fire a macOS alert.
1740 # Reap the honey-port from any previous toggle before arming a new one, so the
1741 # 'nc -l' tripwire never accumulates one idle listener per toggle-on (15.12.20).
1742 pkill -f "nc -l localhost" 2>/dev/null || true
1743 HONEY_PORT=$(get_free_port)
1744 (
1745     if nc -l localhost "$HONEY_PORT" >/dev/null 2>&1; then
1746         echo "[$(date +%Y-%m-%d %H:%M:%S)] [CRITICAL SECURITY ALERT] PORT SCAN DETECTED! An unknown application
            just pinged the local Honey-Port ($HONEY_PORT)!" >> "$LOG_FILE"
1747         osascript -e 'display notification "An unknown application is aggressively scanning your local ports.
            Malware port-scan suspected." with title "nullexit OPSEC Alert" sound name "Basso" 2>/dev/null || true
1748     fi
1749 ) &
1750 disown %1 2>/dev/null || true
1751 echo " [Security] Honey-Port Tripwire armed on random port to detect malware port-scans."
1752
1753 echo -e "\nStarting Docker containers..."
1754
1755 # --- Ad-block rule compilation: hash-gated one-off, off the critical path (15.8.8) ---
1756 # The rule-compiler (a `compile`-profiled container, the SOLE writer of adguard/work)
1757 # re-dedupes ~340k rules inside the 600MB VM (~28s) far too slow to run every toggle.
1758 # It is gated on a hash of the lists YOU control (black_list.txt + white_list.txt):
1759 # recompile only when they change, when compiled_rules.txt / ip_blocklist.ipset are
1760 # missing, or when they exceed RULES_MAX_AGE_DAYS (default 7, so the remote blocklists
1761 # still refresh occasionally). Otherwise AdGuard/routing-fix boot on the existing
1762 # artifacts and we skip the ~28s they no longer `depends_on` the rule-compiler
1763 # container (see docker-compose.yml). A compile failure is non-fatal: we keep the
1764 # previous compiled rules rather than bricking the tunnel (ad-block != the kill-switch).
1765 RULES_HASH_FILE="$SCRIPT_DIR/.rules_hash"
1766 rules_hash=$(compute_rules_hash)
1767 stored_rules_hash=""
1768 [ -f "$RULES_HASH_FILE" ] && stored_rules_hash=$(cat "$RULES_HASH_FILE" 2>/dev/null)
1769 rules_max_age_days=$(read_env_var RULES_MAX_AGE_DAYS)
1770 [ -z "$rules_max_age_days" ] && rules_max_age_days=7
1771 need_compile=false
1772 if [ ! -f adguard/work/userfilters/compiled_rules.txt ] || [ ! -f adguard/work/userfilters/ip_blocklist.ipset
]; then

```

```

1773 need_compile=true
1774 elif [ -z "$rules_hash" ] || [ "$rules_hash" != "$stored_rules_hash" ]; then
1775     need_compile=true
1776     elif [ -n "$(find adguard/work/userfilters/compiled_rules.txt -mtime +"$rules_max_age_days" 2>/dev/null)" ];
1777         then
1778             need_compile=true
1779 fi
1780 if [ "$need_compile" = "true" ]; then
1781     echo " Ad-block lists changed (or artifacts missing/stale)  compiling rules (one-off, ~28s)..."
1782     if run_logged docker compose run --build --rm rule-compiler >> output.log 2>&1; then
1783         if [ -n "$rules_hash" ]; then echo "$rules_hash" > "$RULES_HASH_FILE"; fi
1784     else
1785         echo " [!] Rule compile failed  continuing with existing compiled ad-block rules."
1786     fi
1787 else
1788     echo " Ad-block lists unchanged  reusing compiled rules (skipping the ~28s recompile)."
1789 fi
1790
1791 # --- Gate `--build` for the long-running images (routing-fix, tor) ---
1792 # BuildKit re-checking these on the 600MB VM costs ~30s even when every layer is
1793 # CACHED (15.8.8). Only pass --build when the hashed inputs change or an image is
1794 # missing; otherwise plain `up -d` reuses cached images. `up` starts everything
1795 # except the `compile`-profiled rule-compiler.
1796 BUILD_HASH_FILE="$SCRIPT_DIR/.build_hash"
1797 build_inputs_hash=$(compute_build_inputs_hash)
1798 stored_build_hash=""
1799 [ -f "$BUILD_HASH_FILE" ] && stored_build_hash=$(cat "$BUILD_HASH_FILE" 2>/dev/null)
1800 local_images_present=true
1801 for _img in nullexit-routing-fix:v1.0.0 nullexit-tor:v1.0.0; do
1802     docker image inspect "$_img" >> output.log 2>&1 || local_images_present=false
1803 done
1804 if [ -n "$build_inputs_hash" ] && [ "$build_inputs_hash" = "$stored_build_hash" ] && [ "$local_images_present"
1805     " = "true" ]; then
1806     echo " Build inputs unchanged  skipping image build (reusing cached images)."
1807     run_logged docker compose up -d
1808 else
1809     echo " Build inputs changed or a local image is missing  building images..."
1810     run_logged docker compose up -d --build
1811     # Record the hash only after a successful build (set -e aborts above on failure).
1812     if [ -n "$build_inputs_hash" ]; then echo "$build_inputs_hash" > "$BUILD_HASH_FILE"; fi
1813 fi
1814
1815 RULE_COUNT=$(grep "! Total Block Rules:" adguard/work/userfilters/compiled_rules.txt 2>/dev/null | awk -F': '
1816     '{print $2}' || echo "0")
1817 NATIVE_COUNT=$(grep "! Native AdGuard Rules:" adguard/work/userfilters/compiled_rules.txt 2>/dev/null | awk -
1818     F': ' '{print $2}' || echo "0")
1819
1820 if [ "$RULE_COUNT" != "0" ] && [ "$RULE_COUNT" != "" ]; then
1821     if [ "$NATIVE_COUNT" != "0" ] && [ "$NATIVE_COUNT" != "" ]; then
1822         TOTAL_COUNT=$((RULE_COUNT + NATIVE_COUNT))
1823         # Format with commas for readability (e.g. 441,578)
1824         if command -v printf &> /dev/null; then
1825             FORMATTED_RULE=$(printf "%'d" "$RULE_COUNT")
1826             FORMATTED_TOTAL=$(printf "%'d" "$TOTAL_COUNT")
1827             echo " DNS: Compiled $FORMATTED_RULE unique custom rules (Total active in AdGuard: ~$FORMATTED_TOTAL
1828                 rules)."
1829         else
1830             echo " DNS: Compiled $RULE_COUNT unique custom rules (Total active in AdGuard: ~$TOTAL_COUNT rules)."
1831         fi
1832     else
1833         echo " DNS: Compiled and loaded $RULE_COUNT optimized rules."
1834     fi
1835 fi
1836
1837 # Report IP blocklist compilation results
1838 IP_COUNT=$(grep "^# Entries:" adguard/work/userfilters/ip_blocklist.ipset 2>/dev/null | awk -F': ' '{print $2
1839     }' || echo "0")
1840
1841 if [ "$IP_COUNT" != "0" ] && [ "$IP_COUNT" != "" ]; then
1842     if command -v printf &> /dev/null; then
1843         FORMATTED_IP=$(printf "%'d" "$IP_COUNT")
1844         echo " IP:  Loaded $FORMATTED_IP threat intelligence IPs/CIDRs into kernel firewall."
1845     else
1846         echo " IP:  Loaded $IP_COUNT threat intelligence IPs/CIDRs into kernel firewall."
1847     fi
1848 fi
1849
1850 # 6. Wait for the gateway container's Tailscale connection to be ready
1851 BYPASS_PING=$(read_env_var GATEWAY_BYPASS_PING | tr '[:upper:]' '[:lower:]')
1852 USE_EXIT_NODE=$(read_env_var GATEWAY_USE_EXIT_NODE | tr '[:upper:]' '[:lower:]')
1853
1854 echo "Waiting for gateway container's Tailscale to connect to the tailnet..."

```

```

1848 echo " (This can take 30-60s Tailscale goes through: NoState Starting Running Online)"
1849 connected=false
1850 consecutive_failures=0
1851 for i in {1..60}; do
1852     # Run tailscale status and capture its output so the user can see what's happening.
1853     # Previously this was hidden behind 2>> output.log, making it impossible to debug hangs.
1854     ts_output=$(run_with_timeout 3 docker compose exec -T tailscale tailscale status 2>&1 || true)
1855
1856     if echo "$ts_output" | grep -q "offers exit node"; then
1857         connected=true
1858         break
1859     fi
1860
1861     # Track consecutive failures to detect a stuck state.
1862     # If we see 40+ consecutive 'NoState' responses, give up early
1863     # the daemon is alive but can't connect to the coordination server.
1864     if echo "$ts_output" | grep -q "NoState"; then
1865         consecutive_failures=$((consecutive_failures + 1))
1866         if [ "$consecutive_failures" -ge 40 ]; then
1867             echo ""
1868             echo " [attempt $i/60] Stuck in 'NoState' for $consecutive_failures checks."
1869             echo " Tailscale daemon is running but can't reach the coordination server."
1870             break
1871         fi
1872     else
1873         consecutive_failures=0
1874     fi
1875
1876     # Print a condensed status every 10 seconds so the user can see progress
1877     if [ $((i % 10)) -eq 0 ]; then
1878         ts_short=$(echo "$ts_output" | head -1 | tr -d '\r\n')
1879         echo " [attempt $i/60] $ts_short"
1880     elif [ $((i % 5)) -eq 0 ]; then
1881         echo -n "$[i]"
1882     else
1883         echo -n "."
1884     fi
1885     sleep 1
1886 done
1887 echo ""
1888
1889 if [ "$connected" = "true" ]; then
1890     echo "Gateway container is online on the Tailscale mesh."
1891 elif [ "$BYPASS_PING" = "true" ]; then
1892     echo "[Warning] Gateway container check timed out. Proceeding anyway (GATEWAY_BYPASS_PING is true)..."
1893 else
1894     echo "ERROR: Gateway container failed to initialize Tailscale (timed out after 60s)." >&2
1895     echo " The container was seen in the following states during the wait:" >&2
1896     echo "$ts_output" | sed 's/^/ /' >&2
1897     echo "" >&2
1898     echo " This could mean:" >&2
1899     echo " 1. Tailscale auth key has expired check TS_AUTHKEY in .env" >&2
1900     echo " 2. Network issue inside the container check: docker compose logs tailscale" >&2
1901     echo " 3. gluetun VPN tunnel not connecting check: docker compose logs warp | tail -20" >&2
1902     echo "" >&2
1903     echo " Diagnose manually:" >&2
1904     echo " docker compose exec -T tailscale tailscale status" >&2
1905     echo " docker compose exec -T tailscale tailscale netcheck" >&2
1906     cleanup_handler ERR $LINENO
1907     exit 1
1908 fi
1909
1910 # 6b. Resolve the gateway's Tailscale IP.
1911 # Dynamically query the container for the IP (handles IP changes after re-auth)
1912 # and cache it in .gateway_ip for fast retrieval by recover.sh during Wi-Fi roaming.
1913 #
1914 # CRITICAL: `docker compose exec -T` on macOS/Colima injects carriage
1915 # returns (\r) into captured output. If $TS_IP ends with \r, then
1916 # `networksetup -setdnsservers` silently rejects the malformed IP and
1917 # DNS stays at 1.1.1.1 the primary bug this project was hitting.
1918 # `tr -d '\r'` strips the CR; `awk 'NR==1{print $1; exit}'` takes only
1919 # the first field of the first line.
1920 TS_IP=""
1921 if [ "$connected" = "true" ]; then
1922     TS_IP=$(run_with_timeout 5 docker compose exec -T tailscale tailscale ip -4 2>> output.log | tr -d '\r' |
1923         awk 'NR==1{print $1; exit}' || true)
1924     if [ -n "$TS_IP" ]; then
1925         echo "Resolved gateway Tailscale IP dynamically: $TS_IP"
1926         echo "$TS_IP" > .gateway_ip
1927     else
1928         echo " [!] Failed to resolve gateway IP dynamically. Trying fallback cache..." >&2

```



```

1928     TS_IP=$(read_adguard_ip || true)
1929     fi
1930 else
1931     TS_IP=$(read_adguard_ip || true)
1932 fi
1933
1934 # 7. Connect host Mac to Tailscale mesh.
1935 # With the standalone tailscaled daemon (registered as a per-user LaunchAgent or
1936 # system LaunchDaemon via 'brew services start tailscale'), the previous five-
1937 # phase flow collapses into two trivial CLI calls. There is no .app GUI to launch,
1938 # no menu bar item to click, and no Accessibility permission required.
1939 #
1940 # CRITICAL: If tailscaled is not running, `tailscale up` will silently fail.
1941 # We auto-start it before proceeding, and SKIP the rest of the Tailscale setup
1942 # if it can't be started (DNS hijack to a 100.x.x.x IP and exit-node routing
1943 # are impossible without the host on the mesh).
1944 #
1945 # We connect in two phases:
1946 # Phase A Join mesh WITHOUT exit node (so pre-flight checks can run)
1947 # Phase B Set exit node only after pre-flight checks confirm the gateway works
1948 HOST_ON_MESH=false
1949 SKIP_EXIT_NODE=false
1950 if [ "$HIJACK_HOST" = "false" ]; then
1951     echo -e "\n[Info] HIJACK_HOST is false (Headless Mode). Host networking will remain untouched."
1952     SKIP_EXIT_NODE=true
1953 elif [ -n "$TS_BIN" ]; then
1954     echo -n "Verifying tailscaled is reachable"
1955     daemon_ready=false
1956     status_exit=0
1957     for i in {1..10}; do
1958         # Test if the daemon responds to status check.
1959         # If status returns non-zero, check if it was a normal "stopped/disconnected" state
1960         # (exit code 1 is normal for disconnected). If the command exited quickly (not killed by timeout),
1961         # and the error is just "Tailscale is stopped", then the daemon is alive and ready.
1962         # But if it timed out (exit code 143) or is completely unresponsive, it's wedged.
1963         status_exit=0
1964         if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
1965             daemon_ready=true
1966             break
1967         else
1968             status_exit=$?
1969             if [ "$status_exit" -ne 143 ] && [ -S /var/run/tailscaled.socket ]; then
1970                 daemon_ready=true
1971                 break
1972             fi
1973         fi
1974         echo -n "."
1975         sleep 1
1976     done
1977     echo ""
1978
1979     # If the daemon was unresponsive or socket check failed, attempt auto-restart
1980     if [ "$daemon_ready" != "true" ]; then
1981         if [ "$status_exit" -eq 143 ]; then
1982             warn "tailscaled daemon is wedged/unresponsive. Restarting it..."
1983         else
1984             warn "tailscaled daemon is not running. Attempting to start it..."
1985         fi
1986         restart_tailscaled_daemon
1987
1988         # Re-verify
1989         if run_with_timeout 5 $TS_BIN status >> output.log 2>&1 || [ -S /var/run/tailscaled.socket ]; then
1990             daemon_ready=true
1991         fi
1992     fi
1993
1994     if [ "$daemon_ready" = "true" ]; then
1995         echo "tailscaled is responsive (running as a system service)."
1996
1997         # Phase A: Join mesh without exit node
1998         echo "Connecting host to Tailscale mesh (no exit node yet)..."
1999         if $TS_BIN up; then
2000             $TS_BIN set --ssh=true --accept-dns=false --accept-routes=true --exit-node= || true
2001             HOST_ON_MESH=true
2002             echo "Host is on Tailscale mesh."
2003         else
2004             warn "tailscale up failed even though the daemon is running."
2005             echo " This usually means the host hasn't authenticated with your tailnet yet."
2006             echo " Run this command in a terminal to authenticate:"
2007             echo ""
2008             echo "         sudo tailscale up"

```

```

2009     echo ""
2010     echo "  A browser will open. Log in to your Tailscale account, then re-run toggle.sh."
2011   fi
2012 else
2013   fail "tailscaled could NOT be started automatically."
2014   echo "  Run this in a terminal to start and authenticate Tailscale:"
2015   echo ""
2016   echo "    sudo brew services start tailscale"
2017   echo "    sudo tailscale up"
2018   echo ""
2019   echo "  Then re-run toggle.sh."
2020 fi
2021 else
2022   echo -e "\n[Warning] tailscale CLI not found on PATH. Skipping host Tailscale configuration..."
2023 fi
2024
2025 # Phase B: Pre-flight checks + enable exit node only if safe
2026 if [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" != "false" ] && [ -n "$TS_IP" ]; then
2027   echo -e "\n"
2028   echo "Pre-flight connectivity check"
2029   echo ""
2030
2031   # Check 1: Can we reach the gateway via Tailscale (uses tailscale ping, not ICMP)?
2032   # NOTE: tailscale ping exits 1 when the connection goes through a DERP relay
2033   # ("direct connection not established") even though the pong was received.
2034   # We check for 'pong' in the output (stderr discarded) to accept relayed connections.
2035   echo -n " [1/3] Gateway reachable via Tailscale... "
2036   ping_ok=false
2037   # The host tailscaled needs a few seconds to propagate the new network map and
2038   # establish a connection route after 'tailscale up --reset'. We retry up to 45 times.
2039   for attempt in {1..45}; do
2040     if $TS_BIN ping --until-direct=false -c 2 --timeout 2s "$TS_IP" 2>/dev/null | grep -q "pong"; then
2041       ping_ok=true
2042       break
2043     fi
2044     sleep 1
2045   done
2046   if [ "$ping_ok" = "true" ]; then
2047     echo "PASS"
2048   else
2049     echo "FAIL"
2050     SKIP_EXIT_NODE=true
2051   fi
2052
2053   # Check 2: Can we reach AdGuard via localhost (exposed port ${DNS_PROXY_PORT})?
2054   # Colima's SSH tunnel only forwards TCP (not UDP), so we use +tcp.
2055   echo -n " [2/3] AdGuard DNS via localhost... "
2056   if command -v dig &>/dev/null; then
2057     if dig +tcp @127.0.0.1 -p ${DNS_PROXY_PORT} google.com +short +timeout=5 &>/dev/null; then
2058       echo "PASS"
2059     else
2060       echo "FAIL"
2061       SKIP_EXIT_NODE=true
2062     fi
2063   elif command -v nslookup &>/dev/null; then
2064     if nslookup -port=${DNS_PROXY_PORT} google.com 127.0.0.1 &>/dev/null; then
2065       echo "PASS"
2066     else
2067       echo "FAIL"
2068       SKIP_EXIT_NODE=true
2069     fi
2070   else
2071     echo "SKIP (dig/nslookup not available)"
2072   fi
2073
2074   # Check 3: Does the WARP container have external internet?
2075   echo -n " [3/3] WARP container internet... "
2076   tmp_err=$(mktemp)
2077   if docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace >/dev/null 2>
2078   "$tmp_err"; then
2079     echo "PASS"
2080   else
2081     echo "FAIL"
2082     if [ -s "$tmp_err" ]; then
2083       err_msg=$(cat "$tmp_err" | tr -d '\r')
2084       echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Check 3] WARP container check failed: $err_msg" >> output.log
2085     fi
2086     SKIP_EXIT_NODE=true
2087   fi
2088   rm -f "$tmp_err"

```

```

2089 # Act based on check results
2090 if [ "$SKIP_EXIT_NODE" != "true" ]; then
2091     echo -e "\nAll checks passed. Enabling exit node $TS_IP..."
2092     $TS_BIN up || true
2093     if $TS_BIN set --ssh=true --accept-dns=false --accept-routes=true --exit-node="$TS_IP" --exit-node-allow-
lan-access=true; then
2094         echo "Exit node enabled."
2095         add_warp_bypass_routes
2096         setup_exit_node_routing
2097         enable_killswitch
2098         # Opt-in FaceTime split-route (no-op unless FACETIME_SPLIT_ROUTE=true).
2099         # After the kill-switch so the com.apple/nullexit anchor + table exist.
2100         add_apple_split_routes
2101     else
2102         echo "[Warning] Failed to set exit node."
2103         SKIP_EXIT_NODE=true
2104     fi
2105 fi
2106
2107 if [ "$SKIP_EXIT_NODE" = "true" ]; then
2108     warn "Pre-flight checks failed. EXIT NODE + DNS HIJACK SKIPPED."
2109     echo "Host stays on Tailscale mesh but your internet routes through your normal connection."
2110     echo "Troubleshoot with:"
2111     echo "    docker compose logs warp | tail -20"
2112     echo "    docker compose exec -T warp wget -qO- --timeout=5 https://www.cloudflare.com/cdn-cgi/trace"
2113     echo ""
2114     echo "Falling back to SOCKS5 proxy for traffic routing..."
2115 fi
2116 elif [ "$HOST_ON_MESH" = "true" ] && [ "$USE_EXIT_NODE" = "false" ]; then
2117     echo "USE_EXIT_NODE is false. Skipping exit node and DNS hijack."
2118     SKIP_EXIT_NODE=true
2119 fi
2120
2121 # 8. Apply DNS Hijacking.
2122 # If exit node is enabled: hijack DNS to gateway's Tailscale IP (routes through tailnet).
2123 # Otherwise, leave DNS at 1.1.1.1 (already set by reset at the top).
2124 # AdGuard is also available at localhost:${DNS_PROXY_PORT} for manual configuration in apps.
2125 if [ -n "$TS_IP" ] && [ "$HOST_ON_MESH" = "true" ] && [ "$SKIP_EXIT_NODE" != "true" ]; then
2126     echo -e "\nHijacking host DNS to point ONLY at AdGuard Home ($TS_IP)..."
2127     echo "Setting MagicDNS search domain 'ts.net' so tailnet hostnames resolve..."
2128     if force_dns_to_gateway "$TS_IP"; then
2129         echo "DNS hijack verified: single server = $TS_IP."
2130     else
2131         echo "[Warning] DNS hijack could not be verified."
2132         echo "If DNS is broken, run manually:"
2133         echo "    networksetup -setdnsservers \"$ACTIVE_SERVICE\" $TS_IP"
2134         echo "    networksetup -setsearchdomains \"$ACTIVE_SERVICE\" ts.net"
2135     fi
2136 elif [ -n "$TS_IP" ] && [ "$HIJACK_HOST" != "false" ]; then
2137     echo -e "\n[Info] Exit node not enabled (pre-flight checks failed)."
2138     echo "Trying local DNS proxy for ad-blocking..."
2139     if start_local_dns_proxy; then
2140         echo "Ad-blocking active via local DNS proxy through AdGuard."
2141     else
2142         echo "Local DNS proxy unavailable. DNS stays at 1.1.1.1."
2143         echo "AdGuard available at http://localhost:80 (port ${DNS_PROXY_PORT} for DNS via TCP)."
2144     fi
2145
2146 # Enable SOCKS5 proxy for traffic routing through WARP
2147 # (enabled whenever exit node is skipped, regardless of HOST_ON_MESH)
2148 echo -e "\nEnabling SOCKS5 proxy for TCP traffic routing through gateway WARP tunnel..."
2149 echo "(SOCKS5 proxy runs inside gateway container, routes through tun0 -> WARP)"
2150 enable_socks_proxy "$ACTIVE_SERVICE"
2151
2152 # Verify the SOCKS5 proxy works through WARP
2153 echo -n "Verifying SOCKS5 proxy routes through WARP... "
2154 if curl --socks5-hostname 127.0.0.1:$SOCKS_PROXY_PORT --max-time 10 -s https://www.cloudflare.com/cdn-cgi/
trace 2>/dev/null | grep -q "warp=on"; then
2155     echo "ok (warp=on)."
2156     echo "All TCP traffic now encrypted through Cloudflare WARP tunnel."
2157 else
2158     echo "FAILED (proxy not routing through WARP)."
2159     echo "Disabling SOCKS5 proxy internet stays on direct connection."
2160     disable_socks_proxy
2161     echo "SOCKS proxy available at localhost:$SOCKS_PROXY_PORT for manual configuration."
2162 fi
2163 else
2164     echo -e "\n[Warning] Could not resolve gateway Tailscale IP."
2165 fi
2166
2167 # 9. Verify host connectivity

```

```

2168 if [ "$HOST_ON_MESH" = "true" ] && [ -n "$TS_BIN" ]; then
2169     if run_with_timeout 5 $TS_BIN status >> output.log 2>&1; then
2170         echo "Host is online on the Tailscale mesh."
2171     else
2172         echo "[Warning] Host is not responding on Tailscale."
2173     fi
2174 fi
2175
2176 ELAPSED=$(( SECONDS - TOGGLE_START_TIME ))
2177 echo -e "\nGateway has been successfully STARTED in ${ELAPSED} seconds."
2178
2179 # Prevent system from going to sleep while gateway is running
2180 start_sleep_prevention
2181
2182 if [[ "$HIJACK_HOST" != "false" ]]; then
2183     if [ -n "$TS_IP" ] && [ "$TS_IP" != "1.1.1.1" ]; then
2184         start_dns_watcher "$TS_IP"
2185     fi
2186
2187     # Start WARP liveness monitor (logs to output.log on warp flip)
2188     start_warp_watcher
2189 fi
2190
2191 # Reset sharing services after network configuration to prevent AirDrop freezes
2192 echo "Resetting macOS sharing services (AirDrop/AirPlay)..."
2193 reset_sharing_services
2194
2195 # Tell external watchers (post-wake / network-change) the gateway is up.
2196 write_gateway_active_marker
2197
2198 # Local Network Surveillance Check
2199 echo -e "\n"
2200 echo "Local Network Surveillance Check"
2201 echo ""
2202 # Count populated ARP cache entries to detect network scanning or high congestion
2203 # Using '-an' avoids hanging on DNS reverse resolution when the network state is in transition.
2204 ARP_COUNT=$(arp -an 2>/dev/null | grep -iv 'incomplete' | wc -l | tr -d ' ')
2205 if [ "$ARP_COUNT" -gt 15 ]; then
2206     warn "High ARP activity detected ($ARP_COUNT devices in cache)."
2207     warn "This Wi-Fi network may be heavily congested or actively scanned (e.g., arp-scan)."
2208     echo -e " [] Your traffic remains fully encrypted and invisible.\n"
2209 else
2210     echo -e " [] Local network looks quiet ($ARP_COUNT devices). No aggressive scanning detected.\n"
2211 fi
2212 fi
2213
2214 SUCCESS_RUN=true
2215
2216 # Final DNS state summary
2217 if [ -n "$ACTIVE_SERVICE" ]; then
2218     FINAL_DNS=$(networksetup -getdnsservers "$ACTIVE_SERVICE" 2>> output.log || true)
2219     echo -e "\n"
2220     echo -e "DNS STATE: $FINAL_DNS"
2221     echo -e ""
2222 fi
2223
2224 if [ "$START_GATEWAY" = "true" ] && command -v docker >/dev/null 2>&1; then
2225     if docker compose logs rule-compiler 2>/dev/null | grep -q "Warning: Failed to fetch"; then
2226         echo -e "\n[Warning] One or more blocklist URLs failed to download (404/Offline)."
2227         echo " The gateway gracefully fell back to yesterday's cached blocklist."
2228         echo " (Run 'docker logs rule-compiler' later to investigate which link died.)"
2229     fi
2230 fi
2231
2232 rm -f "${LOCK_FILE:-/tmp/nullexit-toggle.lock}"
2233 if [ -t 0 ]; then
2234     read -rp "Press [r] and Enter to instantly reverse state, or just press Enter to exit: " USER_CHOICE
2235     if [[ "${USER_CHOICE}" == "r" || "${USER_CHOICE}" == "R" ]]; then
2236         echo "Reversing gateway state..."
2237         exec bash "$0"
2238     fi
2239 fi
2240
2241 echo -e "\nYou can close this terminal window now."

```

7 recover.sh

```

1 #!/bin/bash
2 # recover.sh nullexit KILL-SWITCH recovery script (macOS + Linux)
3 # Fixes internet when toggle.sh leaves you stranded.
4 #
5 # This is a NUCLEAR option: it undoes EVERYTHING toggle.sh does by:
6 #   1. Disconnecting host Tailscale from the mesh (exit-node off)
7 #   2. Resetting DNS to default on ALL network services
8 #   3. Disabling rogue proxy settings (SOCKS, web, secure web)
9 #   4. Flushing the DNS cache
10 #   5. Flushing stale routing table entries
11 #   6. Stopping gateway Docker containers
12 #   7. Power-cycling Wi-Fi
13 #   8. Verifying internet is working
14 #
15 # One file, both OSes: the dispatch below runs the self-contained Linux
16 # recovery and exits; on macOS it falls through to the richer dual-mode
17 # implementation (default teardown / --post-wake refresh).
18 #
19 # Usage: double-click Recover-Gateway.applescript
20 #         OR ./recover.sh
21 #         OR ./recover.sh --post-wake    (macOS only lighter refresh, keeps gateway live)
22
23 set -e
24
25 # Resolve script directory (used for path-relative refs)
26 # recover.sh lives at the repo root; SCRIPT_DIR lets us launch toggle.sh
27 # (and reference any other repo-root file) from the same directory no
28 # matter where the user invoked recover.sh from.
29 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
30 cd "$SCRIPT_DIR" || exit 1
31
32 #
33 # LINUX RECOVERY self-contained; runs the full teardown then exits before the
34 # macOS body below. Native Linux tooling (resolvectl / nmcli / ip) throughout.
35 #
36 if [[ "$OSTYPE" != "darwin*" ]]; then
37     # Source common formatting and helper functions
38     source "$SCRIPT_DIR/scripts/common.sh"
39
40     # Always add Homebrew path
41     export PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:$PATH"
42
43     echo ""
44     echo -e "${RED}${BOLD}${NC}"
45     echo -e "${RED}${BOLD}      Nullexit Gateway  RECOVERY MODE      ${NC}"
46     echo -e "${RED}${BOLD}${NC}"
47     echo ""
48     echo "This script will tear down the gateway and restore normal internet."
49     echo ""
50
51     # Cache sudo credentials upfront
52     # Prevents the script from hanging halfway through when it needs to run
53     # privileged commands (route flush, Wi-Fi power cycle, etc.).
54     echo -e "${YELLOW}${BOLD}Authentication required for network recovery...${NC}"
55     sudo -v
56     (
57         while true; do
58             sudo -n true 2>/dev/null
59             sleep 60
60         done
61     ) &
62     SUDO_KEEPER_PID=$!
63     trap 'kill $SUDO_KEEPER_PID 2>/dev/null' EXIT
64
65     # 1. Disconnect host Tailscale
66     step "Disconnecting host Tailscale from mesh"
67     if command -v tailscale >> output.log 2>&1; then
68         # Check if actually connected first (with timeout tailscaled could be wedged)
69         if run_with_timeout 5 tailscale status >> output.log 2>&1; then
70             # Also explicitly reset any exit-node so it doesn't linger.
71             if run_with_timeout 10 tailscale up --reset --ssh=true --accept-dns=false --exit-node= >> output.log
72             2>&1; then
73                 ok "Exit-node preference cleared"
74             else
75                 warn "tailscale up --reset didn't respond (tailscaled may be wedged)"
76             fi
77             ts_args=""
78             if is_kill_switch_enabled; then ts_args="--accept-risk=lose-ssh"; fi
79             if run_with_timeout 10 tailscale down $ts_args >> output.log 2>&1; then

```

```

80     else
81         warn "tailscale down didn't respond"
82     fi
83 else
84     warn "tailscaled not reachable skipping (timeout after 5s)"
85 fi
86 else
87     warn "tailscale CLI not found skipping"
88 fi
89
90 # 2. Reset DNS to default on ALL network services
91 step "Resetting DNS to default on active interface"
92
93 ACTIVE_SERVICE=$(ip route get 1.1.1.1 2>/dev/null | awk '{print $5; exit}')
94 if [ -n "$ACTIVE_SERVICE" ]; then
95     sudo resolvectl revert "$ACTIVE_SERVICE" 2>> output.log || true
96     ok "DNS reset on $ACTIVE_SERVICE"
97 else
98     warn "Could not determine active network interface"
99 fi
100
101 # 3. Disable rogue proxy settings
102 step "Disabling proxy settings (SOCKS, web, secure web)"
103 echo " (Linux global SOCKS proxy configuration skipped.)"
104 ok "Proxy settings cleared"
105
106
107 # 4. Flush DNS cache
108 step "Flushing DNS cache"
109 if command -v resolvectl >> output.log 2>&1; then
110     sudo resolvectl flush-caches 2>> output.log || true
111     ok "DNS cache flushed"
112 else
113     warn "resolvectl not found"
114 fi
115
116 # 5. Clear stale routing table
117 step "Flushing stale routing table entries"
118 sudo ip route flush cache >> output.log 2>&1 || true
119 ok "Routing table flushed"
120
121 # 6. Stop gateway Docker containers
122 step "Stopping gateway Docker containers"
123 if command -v docker >> output.log 2>&1 && docker info >> output.log 2>&1; then
124     if [ -f "docker-compose.yml" ]; then
125         docker compose down -t 5 2>> output.log && ok "Containers stopped" || warn "No containers were running"
126     else
127         warn "docker-compose.yml not found in current directory"
128     fi
129 else
130     warn "Docker not available skipping"
131 fi
132
133 # 6b. Stopping sleep prevention (systemd-inhibit)
134 step "Stopping sleep prevention"
135 PID_FILE="/tmp/nullexit-cafeinate.pid"
136 if [ -f "$PID_FILE" ]; then
137     INHIBIT_PID=$(cat "$PID_FILE")
138     if [ -n "$INHIBIT_PID" ] && kill -0 "$INHIBIT_PID" 2>/dev/null; then
139         kill "$INHIBIT_PID" 2>/dev/null || true
140         ok "Sleep prevention stopped (PID $INHIBIT_PID)"
141     else
142         warn "Sleep prevention process not running, cleaning up stale PID file"
143     fi
144     rm -f "$PID_FILE"
145 else
146     ok "Sleep prevention was not active"
147 fi
148
149 # 7. Power-cycle Wi-Fi
150 step "Power-cycling Wi-Fi"
151 if command -v nmcli &>/dev/null; then
152     sudo nmcli radio wifi off 2>> output.log || true
153     sleep 2
154     sudo nmcli radio wifi on 2>> output.log || true
155     ok "Wi-Fi bounced via nmcli"
156     sleep 3
157 else
158     warn "nmcli not found. Falling back to ip link..."
159     WIFI_IFACE=$(ip link | grep -E '[0-9]+: (wl[a-zA-Z0-9]+|wlan[0-9]+):' | awk -F': ' '{print $2}')
160     if [ -n "$WIFI_IFACE" ]; then

```



```

161     sudo ip link set "$WIFI_IFACE" down 2>> output.log || true
162     sleep 2
163     sudo ip link set "$WIFI_IFACE" up 2>> output.log || true
164     ok "Wi-Fi bounced via ip link (interface $WIFI_IFACE)"
165     sleep 3
166     else
167         warn "No Wi-Fi interface detected."
168     fi
169 fi
170
171 # 8. Verify internet connectivity
172 verify_internet_connectivity
173
174 # Summary
175 echo ""
176 echo -e "${BOLD}${NC}"
177 echo -e "${BOLD}RECOVERY SUMMARY${NC}"
178 echo -e "${BOLD}${NC}"
179
180 # Show final DNS state
181 FINAL_DNS=$(resolvectl dns "$ACTIVE_SERVICE" 2>> output.log | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | tr
182 '\n' ' ' || echo "N/A")
183 echo " DNS ($ACTIVE_SERVICE): $FINAL_DNS"
184
185 echo ""
186
187 if [ "$INTERNET_OK" = "true" ]; then
188     echo -e " ${GREEN}${BOLD} Internet is working.${NC}"
189 else
190     echo -e " ${YELLOW}${BOLD} Could not verify internet connectivity.${NC}"
191     echo " This might mean:"
192     echo " - Your Wi-Fi is disconnected from the router"
193     echo " - Tailscale is still interfering (try restarting tailscaled:"
194     echo "     systemctl restart tailscaled)"
195     echo " - You need to re-connect to Wi-Fi in the menu bar"
196     echo " - Try toggling Wi-Fi off/on in the menu bar"
197     echo ""
198     echo " Or try manually:"
199     echo "     resolvectl revert $ACTIVE_SERVICE"
200     extra_args=""
201     if is_kill_switch_enabled; then extra_args="--accept-risk=lose-ssh"; fi
202     echo "     tailscale down $extra_args"
203     echo "     sudo ip route flush cache"
204     echo "     systemctl restart tailscaled"
205 fi
206
207 echo ""
208 echo -e "${GREEN}${BOLD}Recovery complete.${NC}"
209 echo ""
210
211 exit 0
212 fi
213
214 #
215 # macOS RECOVERY (dual-mode: default teardown / --post-wake refresh)
216 #
217 # Enforce Cryptographic Script Integrity
218 if [ -f "scripts/crypto.sh" ]; then
219     if ! bash scripts/crypto.sh --verify; then
220         exit 1
221     fi
222 fi
223
224 # Argument parsing
225 # --post-wake: lightweight refresh after sleep/wake or Wi-Fi roam.
226 # Keeps the gateway live while HUPing DNS caches, refreshing the
227 # Tailscale exit-node leak, re-hijacking host DNS, and force-
228 # recreating the warp container if its gluetun healthcheck failed.
229 # Does NOT tear down Docker, Tailscale, sleep prevention, or Wi-Fi.
230 # Called from scripts/watcher.sh on every wake / network-change event
231 # while /tmp/nullexit-gateway-active.marker is present (i.e. the
232 # gateway is currently up). See devref.md 10.29 for the why.
233 POST_WAKE=false
234 AUTO_MODE=false
235 for arg in "$@"; do
236     if [ "$arg" = "--post-wake" ]; then
237         POST_WAKE=true
238     elif [ "$arg" = "--auto" ] || [ "$arg" = "--non-interactive" ]; then
239         AUTO_MODE=true
240     fi

```

```

241 done
242
243 rm -f "$SCRIPT_DIR/TUNNEL_FAILED_CLOSED.marker"
244
245 # Prevent concurrent execution of lifecycle scripts (toggle.sh / recover.sh)
246 LOCK_FILE="/tmp/nullexit-toggle.lock"
247 if [ -f "$LOCK_FILE" ]; then
248     LOCK_PID=$(cat "$LOCK_FILE" 2>/dev/null || echo "")
249     if [ -n "$LOCK_PID" ] && [ "$LOCK_PID" != "$$" ] && kill -0 "$LOCK_PID" 2>/dev/null; then
250         if ps -p "$LOCK_PID" -o args= 2>/dev/null | grep -q -E "toggle\.sh|recover\.sh"; then
251             if [ "$POST_WAKE" = "true" ]; then
252                 echo "[$(date -u +%FT%TZ)] POST-WAKE skipped: another lifecycle script (PID $LOCK_PID) is running." >>
                output.log
                exit 0
253             else
254                 echo "Error: Another instance of toggle.sh or recover.sh (PID $LOCK_PID) is already running." >&2
255                 exit 1
256             fi
257         fi
258     fi
259 fi
260 fi
261 echo "$$" > "$LOCK_FILE"
262
263 # Clear the active-state marker in nuclear recovery mode since the gateway is being
264 # completely torn down. This prevents scripts/watcher.sh from triggering post-wake
265 # recoveries in response to network changes caused by the teardown itself.
266 if [ "$POST_WAKE" = "false" ]; then
267     rm -f "/tmp/nullexit-gateway-active.marker"
268 fi
269
270 # WARP Watcher inhibit marker: written immediately in --post-wake mode so the
271 # background WARP Watcher (in toggle.sh) skips failure counting while we are
272 # intentionally tearing down / recreating the warp container. Without this, the
273 # watcher accumulates 6 consecutive warp=off readings during force-recreate (~35s)
274 # and fires nuclear recover.sh, killing a gateway that would have been healthy.
275 WARP_INHIBIT_FILE="/tmp/nullexit-warp-inhibit.marker"
276 if [ "$POST_WAKE" = "true" ]; then
277     touch "$WARP_INHIBIT_FILE"
278     echo "[$(date -u +%FT%TZ)] POST-WAKE started WARP Watcher inhibited to prevent spurious shutdown during warp
    recreate." >> output.log
279 fi
280
281 # Ensure the inhibit marker and lock file are always removed when the script exits (success or error)
282 cleanup_recover() {
283     rm -f "$WARP_INHIBIT_FILE"
284     if [ -f "$LOCK_FILE" ] && [ "$(cat "$LOCK_FILE" 2>/dev/null)" = "$$" ]; then
285         rm -f "$LOCK_FILE"
286     fi
287 }
288 trap cleanup_recover EXIT
289
290 # Source common formatting and helper functions
291 source "$SCRIPT_DIR/scripts/common.sh"
292
293 # Always add Homebrew path
294 setup_standard_path
295
296 echo ""
297 if [ "$POST_WAKE" = "true" ]; then
298     echo -e "${YELLOW}${BOLD}${NC}"
299     echo -e "${YELLOW}${BOLD} Nullexit POST-WAKE / POST-ROAM LIGHT ${NC}"
300     echo -e "${YELLOW}${BOLD} (keeps the gateway live, refreshes) ${NC}"
301     echo -e "${YELLOW}${BOLD}${NC}"
302     echo ""
303     echo "Re-hijacking DNS, refreshing Tailscale exit-node, force-recreating"
304     echo "warp if unhealthy, and resetting sharing services. The gateway stays"
305     echo "up. docker compose / Wi-Fi / caffeinate are NOT touched."
306     echo ""
307 else
308     echo -e "${RED}${BOLD}${NC}"
309     echo -e "${RED}${BOLD} Nullexit Gateway RECOVERY MODE ${NC}"
310     echo -e "${RED}${BOLD}${NC}"
311     echo ""
312     echo "This script will tear down the gateway and restore normal internet."
313     echo ""
314 fi
315
316 # Cache sudo credentials upfront (default mode ONLY)
317 # Skip in --post-wake: the LaunchAgent-launched watcher has no TTY, so `sudo -v`
318 # would block forever waiting for a password. All post-wake commands use `sudo -n`
319 # which is non-blocking and relies on /etc/sudoers.d/nullexit NOPASSWD entries.

```

```

320 SUDO_KEEPER_PID=""
321 if [ "$POST_WAKE" = "false" ] && [ "$AUTO_MODE" = "false" ] && [ -t 0 ]; then
322     echo -e "${YELLOW}${BOLD}Authentication required for network recovery...${NC}"
323     sudo -v
324     (
325         while true; do
326             sudo -n true 2>/dev/null
327             sleep 60
328         done
329     ) &
330     SUDO_KEEPER_PID=$!
331     trap 'kill $SUDO_KEEPER_PID 2>/dev/null' EXIT
332 fi
333
334 # Resolve physical default interface and gateway upfront
335 PHYSICAL_IFACE=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: (Wi-Fi|Ethernet)/{
336     getline; print $2; exit}')
337 [ -z "$PHYSICAL_IFACE" ] && PHYSICAL_IFACE="en0"
338
339 if [ "$POST_WAKE" = "true" ]; then
340     wait_for_dhcp_settle
341 else
342     # Quick resolution in non-post-wake mode (non-blocking)
343     PHYSICAL_GW=$(ipconfig getpacket "$PHYSICAL_IFACE" 2>> output.log | awk -F'[{}]' '/router/{print $2}')
344 fi
345
346 # 1. Disconnect host Tailscale (default) / Refresh exit-node (post-wake)
347 if [ "$POST_WAKE" = "true" ]; then
348     step "Refreshing host Tailscale exit-node routing (post-wake)"
349     if command -v tailscale >> output.log 2>&1; then
350         # Re-issue the SAME exit-node up command toggle.sh START uses. This refreshes
351         # the DERP relay mapping and re-asserts the exit-node preference without
352         # dropping the host's mesh connection (which `tailscale down` would).
353         TS_IP=""
354         for src in .gateway_ip; do
355             [ -f "$src" ] || continue
356             TS_IP=$(cat "$src" 2>> output.log | tr -d '\r' | awk 'NR==1{print $1; exit}' || true)
357             [ -n "$TS_IP" ] && break
358         done
359
360         if [ -n "$TS_IP" ]; then
361             step "Re-establishing exit node $TS_IP via 'tailscale set --exit-node'"
362             # `tailscale set` only mutates the named preference (exit-node here).
363             # Crucially it does NOT call --reset, which would re-apply ALL flags and
364             # trigger a sub-second route-table re-evaluation cycle each post-wake.
365             # On already-connected nodes, the lighter `set` keeps the existing tunnel
366             # alive while only changing the exit-node preference.
367             if run_with_timeout 15 tailscale set --exit-node="$TS_IP" \
368                 --exit-node-allow-lan-access=true \
369                 >> output.log 2>&1; then
370                 ok "Exit-node $TS_IP re-asserted via 'tailscale set'"
371             else
372                 warn "tailscale set --exit-node=$TS_IP didn't respond; falling back to mesh-only"
373                 run_with_timeout 10 tailscale set --exit-node= \
374                     >> output.log 2>&1 || true
375             fi
376         else
377             warn ".gateway_ip missing cannot re-assert exit node"
378             run_with_timeout 10 tailscale up --reset --ssh=true --accept-dns=false --exit-node= \
379                 >> output.log 2>&1 || true
380         fi
381     else
382         warn "tailscale CLI not found"
383     fi
384 else
385     step "Disconnecting host Tailscale from mesh"
386     disconnect_tailscale_host "tailscale"
387 fi
388
389 # 2. Reset DNS to default on ALL network services (default) / Re-hijack gateway DNS (post-wake)
390 if [ "$POST_WAKE" = "true" ]; then
391     step "Re-hijacking host DNS to gateway IP (post-wake / post-roam)"
392     TS_IP=""
393     for src in .gateway_ip; do
394         [ -f "$src" ] || continue
395         TS_IP=$(cat "$src" 2>> output.log | tr -d '\r' | awk 'NR==1{print $1; exit}' || true)
396         [ -n "$TS_IP" ] && break
397     done
398
399     if [ -n "$TS_IP" ]; then

```

```

400 # Detect the active network service (using helper in common.sh)
401 ACTIVE_SVC=$(get_active_service)
402 ENO_SVC=$(get_en0_service)
403
404 networksetup -setsearchdomains "$ACTIVE_SVC" "ts.net" 2>> output.log || true
405 networksetup -setdnsservers "$ACTIVE_SVC" "$TS_IP" 2>> output.log || true
406 networksetup -setsearchdomains "$ENO_SVC" "ts.net" 2>> output.log || true
407 networksetup -setdnsservers "$ENO_SVC" "$TS_IP" 2>> output.log || true
408
409 HITS=$(networksetup -getdnsservers "$ACTIVE_SVC" 2>> output.log | grep -Fx "$TS_IP" || true)
410 if [ -n "$HITS" ]; then
411     ok "DNS re-hijacked to $TS_IP on services: $ACTIVE_SVC, $ENO_SVC"
412 else
413     warn "networksetup accepted but DNS read-back didn't include $TS_IP"
414 fi
415 else
416     warn ".gateway_ip missing cannot re-hijack DNS (user must run toggle.sh START)"
417 fi
418 else
419     step "Resetting DNS to default on all network services (empty = DHCP)"
420
421 # Get ALL network services (handles spaces in names, skips disabled ones)
422 SERVICES=$(networksetup -listallnetworkservices 2>> output.log | grep -v "An asterisk" | grep -v "^$" ||
423     true)
424
425 if [ -z "$SERVICES" ]; then
426     # Fallback to common names
427     SERVICES="Wi-Fi Ethernet Thunderbolt Ethernet USB 10/100 LAN"
428 fi
429
430 RESET_COUNT=0
431 while IFS= read -r service; do
432     # Strip leading asterisk (disabled services)
433     service_clean=$(echo "$service" | sed 's/^\*//')
434     [ -z "$service_clean" ] && continue
435
436     # Check if this service has a hardware port (skip VPN/service-only entries)
437     if networksetup -listallhardwareports 2>> output.log | grep -A1 "Port: $service_clean$" | grep -q "Device:"
438     || [ "$service_clean" = "Wi-Fi" ]; then
439         (
440             networksetup -setsearchdomains "$service_clean" "Empty" 2>> output.log
441             # Set to "empty" so macOS uses DHCP-assigned DNS (not hardcoded 1.1.1.1)
442             sudo networksetup -setdnsservers "$service_clean" "empty" 2>> output.log
443         ) && RESET_COUNT=$((RESET_COUNT + 1)) || true
444     fi
445 done <<< "$SERVICES"
446
447 ok "DNS reset on $RESET_COUNT service(s)"
448 fi
449
450 # 3. Disable rogue proxy settings (default only gateway uses proxy in non-exit-node path)
451 if [ "$POST_WAKE" = "false" ]; then
452     step "Disabling proxy settings (SOCKS, web, secure web)"
453     for svc in $(networksetup -listallnetworkservices 2>> output.log | grep -v "An asterisk" | grep -v "^$" ||
454         echo "Wi-Fi"); do
455         svc_clean=$(echo "$svc" | sed 's/^\*//')
456         [ -z "$svc_clean" ] && continue
457         disable_all_proxies "$svc_clean"
458     done
459     ok "Proxy settings cleared"
460 else
461     step "Proxy settings: leaving untouched (post-wake keeps gateway SOCKS wiring alive)"
462 fi
463
464 # 4. Flush macOS DNS cache (always safe and useful in both modes)
465 step "Flushing DNS cache"
466 if command -v dscacheutil >> output.log 2>&1; then
467     flush_dns_cache
468     ok "DNS cache flushed (mDNSResponder HUP)"
469 else
470     warn "dscacheutil not found"
471 fi
472
473 # 5. Clear stale routing table (always safe in both modes)
474 step "Flushing stale routing table entries"
475 # Resolve physical default gateway IP before flushing
476 # We cannot trust `route get default` here because Tailscale may have already hijacked
477 # the default route to utunX. We must read the actual DHCP router assignment from the hardware.
478 # PHYSICAL_IFACE and PHYSICAL_GW resolved upfront
479
480 # Instead of full route -n flush (which destroys macOS loopback/multicast and wedges the OS until reboot),

```

```

478 # we cleanly delete the Tailscale overrides and Cloudflare WARP bypass routes in ALL modes.
479 # This ensures tailscaled isn't blocked from reaching its control plane when waking from sleep.
480 sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
481 sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
482 remove_warp_bypass_routes
483 ok "Routing overrides cleared"
484
485 if [ "$POST_WAKE" = "false" ]; then
486     disable_killswitch
487     ok "Routing table flush completed via targeted deletes and firewall disabled"
488 else
489     ok "Routing overrides cleared (post-wake mode to preserve Colima bridge100)"
490 fi
491
492 # In post-wake mode we don't bounce Wi-Fi, so we MUST manually restore the physical default route
493 if [ "$POST_WAKE" = "true" ] && [ -n "$PHYSICAL_GW" ]; then
494     # Ensure physical gateway is set just in case it was dropped
495     sudo -n route add default "$PHYSICAL_GW" >> output.log 2>&1 || true
496 fi
497
498 # CONDITIONAL BYPASS ROUTE RESTORATION:
499 # If this was a post-wake recovery and the exit node is active, the route flush
500 # just wiped the static bypass routes for the WARP endpoints. We must re-add
501 # them immediately to prevent a routing loop when Tailscale starts sending traffic.
502 if [ "$POST_WAKE" = "true" ]; then
503     if [ -z "${ACTIVE_IF:-}" ]; then
504         ACTIVE_IF=$(route get default 2>> output.log | awk '/interface:/{print $2; exit}')
505         if [ -z "$ACTIVE_IF" ] || [[ "$ACTIVE_IF" =~ ^tun ]] || [[ "$ACTIVE_IF" =~ ^tun ]]; then
506             for i in en0 en1 en2 en3; do
507                 if ifconfig "$i" 2>> output.log | grep -q 'status: active'; then
508                     ACTIVE_IF="$i"; break
509                 fi
510             done
511         fi
512         [ -z "$ACTIVE_IF" ] && ACTIVE_IF="en0"
513     fi
514
515     echo "Re-adding host bypass routes for Cloudflare WARP endpoints..."
516     remove_warp_bypass_routes
517     if [ -n "${PHYSICAL_GW:-}" ]; then
518         add_warp_bypass_routes "$PHYSICAL_GW"
519     else
520         add_warp_bypass_routes "$ACTIVE_IF"
521     fi
522     # Re-assert the opt-in FaceTime split-route so it survives roam/wake
523     # (no-op unless FACETIME_SPLIT_ROUTE=true). Physical gateway is known here.
524     add_apple_split_routes "${PHYSICAL_GW:-}"
525
526     # Also re-apply the default route pointing to the Tailscale interface
527     ts_iface=$(ifconfig 2>> output.log | grep -B4 "inet 100." | grep -E '[a-z0-9]+' | cut -d: -f1 | head -n 1)
528     if [ -n "$ts_iface" ]; then
529         echo "Re-routing default gateway to $ts_iface using 0.0.0.0/1 split..."
530         # DO NOT delete the physical default route! It crashes Colima.
531         # Use the /1 trick to mathematically override the default route.
532         sudo -n route delete -net 0.0.0.0/1 >> output.log 2>&1 || true
533         sudo -n route delete -net 128.0.0.0/1 >> output.log 2>&1 || true
534         sudo -n route add -net 0.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
535         sudo -n route add -net 128.0.0.0/1 -interface "$ts_iface" >> output.log 2>&1 || true
536         enable_killswitch
537     fi
538 fi
539
540 # 6. Stop gateway Docker containers (default) / Force-recreate warp if unhealthy (post-wake)
541 if [ "$POST_WAKE" = "true" ]; then
542     # 6a. Check for Roaming Network Mismatch (Bridged vs Enterprise Wi-Fi)
543     if [ -f "/tmp/nullexit_colima_mode.txt" ] && [ -f "$SCRIPT_DIR/../../lan_p2p_detected" ]; then
544         current_colima_mode=$(cat /tmp/nullexit_colima_mode.txt)
545         p2p_allowed=$(cat "$SCRIPT_DIR/../../lan_p2p_detected")
546         required_colima_mode="bridged"
547         [ "$p2p_allowed" == "false" ] && required_colima_mode="shared"
548
549         if [ "$current_colima_mode" != "$required_colima_mode" ]; then
550             warn "Network roaming mismatch detected!"
551             warn "Colima is running in '$current_colima_mode' mode but new network requires '$required_colima_mode'."
552             warn "Executing full gateway restart to protect against MAC-spoofing deauthentication..."
553             echo "[$(date +%Y-%m-%d %H:%M:%S)] [Roam] Mismatch ($current_colima_mode vs $required_colima_mode). Triggering toggle.sh --restart." >> output.log
554             nohup bash "$SCRIPT_DIR/../../toggle.sh" --restart >/dev/null 2>&1 &
555             exit 0
556         fi
557     fi

```

```

558 step "Checking Colima VM state"
559 colima_status=$(colima status 2>&1)
560 if echo "$colima_status" | grep -qi "running"; then
561     ok "Colima VM is running"
562     echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Colima status check passed." >> output.log
563 else
564     warn "Colima VM is NOT running or unresponsive"
565     echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Colima is unhealthy or dead! Output: $colima_status"
566     >> output.log
567     echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Attempting to restart Colima..." >> output.log
568     colima restart >> output.log 2>&1 || warn "Failed to restart Colima"
569 fi
570
571 step "Checking warp container health (gluetun UDP tunnel)"
572 echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Inspecting warp container state..." >> output.log
573 # The warp container is the most failure-prone link in the chain at wake/roam:
574 # its UDP NAT binding to Cloudflare's edge (162.159.192.1:2408) silently dies
575 # during a Wi-Fi roam. We verify the tunnel is actually passing traffic via curl.
576 WARP_STATE=$(docker inspect --format '{{.State.Health.Status}}' warp 2>> output.log || echo "missing")
577 echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container status: $WARP_STATE" >> output.log
578
579 if [ "$WARP_STATE" = "missing" ]; then
580     warn "warp container is missing leaving gateway containers alone (full relaunch requires \"$SCRIPT_DIR/
toggle.sh\")"
581     echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container missing from Docker engine. Requires
full toggle.sh launch." >> output.log
582 else
583     tmp_err=$(mktemp)
584     if docker compose exec -T warp wget -q0- --timeout=3 https://www.cloudflare.com/cdn-cgi/trace 2>"$tmp_err"
| grep -q "warp=on"; then
585         ok "warp container is healthy and actively tunneling traffic (no recreate needed)"
586         echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container passed live traffic healthcheck (
Cloudflare trace successful)." >> output.log
587     else
588         warn "warp container tunnel is dead (Docker status: $WARP_STATE) force-recreating all gateway containers
to restore network namespace"
589         echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container is DEAD or STUCK. Forcing recreation
of gateway stack..." >> output.log
590         if [ -s "$tmp_err" ]; then
591             err_msg=$(cat "$tmp_err" | tr -d '\r')
592             echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] Healthcheck failure details: $err_msg" >> output.
log
593         fi
594         docker compose up -d --force-recreate >> output.log 2>&1 || warn "force-recreate failed"
595
596         NEW_STATE=$(docker inspect --format '{{.State.Health.Status}}' warp 2>> output.log || echo "missing")
597         ok "warp container health after recreate: $NEW_STATE"
598         echo "[$(date '+%Y-%m-%d %H:%M:%S')] [Docker/Colima] warp container health after recreation: $NEW_STATE"
>> output.log
599     fi
600     rm -f "$tmp_err"
601 fi
602 else
603     step "Stopping gateway Docker containers"
604     if command -v docker >> output.log 2>&1 && docker info >> output.log 2>&1; then
605         if [ -f "docker-compose.yml" ]; then
606             docker compose down -t 5 2>> output.log && ok "Containers stopped" || warn "No containers were running"
607         else
608             warn "docker-compose.yml not found in current directory"
609         fi
610     else
611         warn "Docker not available skipping"
612     fi
613 fi
614
615 # 6b. Stopping sleep prevention (caffeinate) default only
616 if [ "$POST_WAKE" = "false" ]; then
617     step "Stopping sleep prevention"
618     stop_pidfile_daemon "$PID_CAFFEINATE" "system sleep prevention"
619 else
620     step "Sleep prevention: leaving untouched (caffeinate already re-acquired by parent shell)"
621 fi
622
623 # 6c. Stopping DNS Watcher default only
624 if [ "$POST_WAKE" = "false" ]; then
625     step "Stopping DNS Watcher"
626     stop_pidfile_daemon "$PID_DNS_WATCHER" "background DNS Watcher"
627 else
628     step "DNS Watcher: leaving untouched (it keeps re-hijacking DNS on every roam)"
629 fi

```



```

630
631 # 7. Power-cycle Wi-Fi default only
632 if [ "$POST_WAKE" = "false" ]; then
633     step "Power-cycling Wi-Fi"
634     bounce_wifi_interfaces
635 else
636     step "Wi-Fi: leaving untouched (post-wake keeps the current Wi-Fi association)"
637 fi
638
639 # 8. Verify (default checks direct internet; post-wake verifies the gateway)
640 if [ "$POST_WAKE" = "true" ]; then
641     step "Verifying gateway is healthy after post-wake refresh"
642
643     # Wait a moment for tailscale + warp healthcheck to settle
644     sleep 3
645
646     GATEWAY_OK=false
647     if command -v dig >> output.log 2>&1; then
648         # Source procedural ports if available
649         if [ -f "/tmp/nullexit-ports.env" ]; then
650             source "/tmp/nullexit-ports.env"
651         fi
652         DNS_PORT=${DNS_PROXY_PORT:-5354}
653
654         # AdGuard is exposed at localhost:${DNS_PORT}. If this resolves, the warp tunnel
655         # is functioning properly because AdGuard routes upstream to warp.
656         if dig +tcp @127.0.0.1 -p "$DNS_PORT" google.com +short +timeout=5 >> output.log 2>&1; then
657             ok "Gateway DNS (AdGuard via localhost:${DNS_PORT}) works"
658             GATEWAY_OK=true
659         else
660             warn "Gateway DNS not responding yet"
661         fi
662     fi
663
664     # Direct host gateway check via Tailscale ping (relayed pong counts as success)
665     if command -v tailscale >> output.log 2>&1; then
666         TS_IP=""
667         [ -f .gateway_ip ] && TS_IP=$(read_adguard_ip || true)
668         if [ -n "$TS_IP" ] && tailscale ping --until-direct=false -c 1 --timeout 4s "$TS_IP" 2>> output.log | grep
669             -q pong; then
670             ok "Gateway $TS_IP reachable via Tailscale"
671             GATEWAY_OK=true
672         else
673             warn "Tailscale ping to gateway did not pong exit-node may still be re-establishing"
674         fi
675     fi
676
677     if [ "$GATEWAY_OK" = "true" ]; then
678         echo -e " ${GREEN}${BOLD} Gateway is healthy after post-wake refresh.${NC}"
679     else
680         echo -e " ${YELLOW}${BOLD} Gateway recovery is in-progress.${NC}"
681         echo "     The route may need another 30s before queries succeed."
682         echo "     If it stays broken for >60s, run: bash \"$SCRIPT_DIR/toggle.sh\""
683     fi
684 else
685     verify_internet_connectivity
686 fi
687
688 # Summary
689 echo ""
690 if [ "$POST_WAKE" = "true" ]; then
691     echo -e "${BOLD}POST-WAKE SUMMARY${NC}"
692 else
693     echo -e "${BOLD}RECOVERY SUMMARY${NC}"
694 fi
695 echo -e "${BOLD}${NC}"
696
697 # Show final DNS state
698 FINAL_DNS=$(networksetup -getdnsservers "Wi-Fi" 2>/dev/null || echo "N/A")
699 echo " DNS (Wi-Fi): $FINAL_DNS"
700
701 FINAL_DNS2=$(networksetup -getdnsservers "Ethernet" 2>/dev/null || true)
702 if [[ -z "$FINAL_DNS2" || "$FINAL_DNS2" == *"not a recognized"* || "$FINAL_DNS2" == *"Error"* ]]; then
703     FINAL_DNS2="N/A (Not configured)"
704 fi
705 echo " DNS (Ethernet): $FINAL_DNS2"
706
707 echo ""
708
709 if [ "$POST_WAKE" = "false" ]; then

```

```

710 if [ "$INTERNET_OK" = "true" ]; then
711     echo -e " ${GREEN}${BOLD} Internet is working.${NC}"
712 else
713     echo -e " ${YELLOW}${BOLD} Could not verify internet connectivity.${NC}"
714     echo "      This might mean:"
715     echo "      - Your Wi-Fi is disconnected from the router"
716     echo "      - Tailscale is still interfering (try restarting tailscaled:"
717     echo "        brew services restart tailscale)"
718     echo "      - You need to re-connect to Wi-Fi in the menu bar"
719     echo "      - Try toggling Wi-Fi off/on in the menu bar"
720     echo ""
721     echo "      Or try manually:"
722     echo "      networksetup -setdnsservers Wi-Fi empty"
723     echo "      tailscale down"
724     echo "      sudo route delete -net 0.0.0.0/1"
725     echo "      sudo route delete -net 128.0.0.0/1"
726     echo "      brew services restart tailscale"
727 fi
728 fi
729
730 # Reset sharing services to prevent AirDrop freezes after interface changes
731 # This is the SAME step in BOTH modes (post-wake inherits the previous fix from 10.27).
732 echo -e "  Resetting macOS sharing services (AirDrop/AirPlay)..."
733 reset_sharing_services
734
735 echo ""
736 if [ "$POST_WAKE" = "true" ]; then
737     echo -e "${YELLOW}${BOLD}Post-wake refresh complete.${NC}"
738     # Remove the WARP Watcher inhibit marker now that the gateway is fully healthy.
739     # The watcher will resume normal liveness monitoring on its next 5s poll.
740     rm -f "$WARP_INHIBIT_FILE"
741     echo "[$(date -u +%FT%TZ)] POST-WAKE complete WARP Watcher inhibit released." >> output.log
742 else
743     echo -e "${GREEN}${BOLD}Recovery complete.${NC}"
744 fi
745 echo ""

```

8 setup.sh

```

1  #!/bin/bash
2  # setup.sh nullexit one-shot setup script (macOS)
3  # Configures Cloudflare WARP + Tailscale + AdGuard Home from scratch.
4  set -euo pipefail
5
6  # Source common formatting and helper functions
7  source "$(dirname "$0")/scripts/common.sh"
8
9  # Run common installation logic
10 source "$(dirname "$0")/scripts/setup-common.sh"
11
12 # Compile macOS Shortcuts
13 echo -e "\n${BOLD}Compiling macOS Desktop Shortcuts...${NC}"
14 if command -v osacompile &> /dev/null; then
15     osacompile -o "Toggle Gateway.app" "Toggle-Gateway.applescript" >/dev/null 2>&1
16     osacompile -o "Recover Gateway.app" "Recover-Gateway.applescript" >/dev/null 2>&1
17     echo "  Created 'Toggle Gateway.app' and 'Recover Gateway.app'"
18 fi
19
20 # Done
21 echo ""
22 echo -e "${GREEN}${BOLD}${NC}"
23 echo -e "${GREEN}${BOLD}          Setup complete!          ${NC}"
24 echo -e "${GREEN}${BOLD}${NC}"
25 echo ""
26 echo -e "${BOLD}One manual step remaining approve the exit node:${NC}"
27 echo "  https://login.tailscale.com/admin/machines"
28 echo "  Find your gateway '...' 'Edit route settings' enable Exit Node"
29 echo ""
30
31 echo -e "${YELLOW}${BOLD}Sudoers / Background Execution Requirements:${NC}"
32 echo "  To allow silent, non-interactive execution of the firewall and network routes (especially during sleep/
wake recovery), you must configure passwordless sudo."
33 echo "  Run this exact command in your terminal (grants are scoped to least privilege):"
34 echo "    echo \"\$USER ALL=(root) NOPASSWD: /sbin/pfctl, /usr/sbin/networksetup, /sbin/route, /sbin/ifconfig,
/usr/bin/dscacheutil -flushcache, /usr/bin/killall -HUP mDNSResponder, /usr/bin/killall sharingd rapportd,
/usr/bin/pkill -9 tailscaled, /usr/bin/pkill -f dns-proxy.py, /bin/kill, /usr/bin>true, /opt/homebrew/bin/

```

```

brew services * tailscale, /opt/homebrew/bin/brew uninstall tailscale, /usr/local/bin/brew services *
tailscale, /usr/local/bin/brew uninstall tailscale, /usr/bin/python3 -I \${PWD}/scripts/dns-proxy.py, /opt/
homebrew/bin/python3 -I \${PWD}/scripts/dns-proxy.py, /usr/local/bin/python3 -I \${PWD}/scripts/dns-proxy.py\"
| sudo tee /etc/sudoers.d/nullexit >/dev/null && sudo visudo -cf /etc/sudoers.d/nullexit"
35 echo ""
36
37 if [[ -n "${TS_IP:-}" ]]; then
38     echo -e "${BOLD}AdGuard Home dashboard:${NC} http://${TS_IP}:3000 (username: admin)"
39     echo ""
40 fi
41
42 echo -e "${BOLD}To toggle the gateway on/off:${NC}"
43 echo " macOS: double-click the 'Toggle Gateway' app icon!"
44 echo " Windows: double-click Toggle-Gateway.bat"
45 echo ""
46
47 if [[ "$OSTYPE" == "darwin"* ]]; then
48     # Install Taildrop auto-receive watcher
49     echo -e "\n${BOLD}Installing Taildrop auto-receive watcher...${NC}"
50     mkdir -p "$HOME/Library/LaunchAgents"
51     NULLEXIT_HOME="$(cd "$(dirname "$0")" && pwd)"
52     TAILDROP_PLIST="$HOME/Library/LaunchAgents/com.nullexit.taildrop-watch.plist"
53     launchctl unload "$TAILDROP_PLIST" >/dev/null 2>&1 || true
54     cp "$NULLEXIT_HOME/launchd/com.nullexit.taildrop-watch.plist" "$TAILDROP_PLIST"
55     sed -i '' "s|__NULLEXIT_HOME_|$NULLEXIT_HOME|" "$TAILDROP_PLIST"
56     launchctl load -w "$TAILDROP_PLIST"
57     echo " Incoming Taildrop files now land in ~/Downloads automatically."
58
59     # LuLu localhost filtering check
60     if [ -d "/Applications/LuLu.app" ] || systemextensionsctl list 2>/dev/null | grep -q "com.objective-see.
lulu"; then
61         echo -e "${YELLOW}${BOLD}LuLu Firewall Detected:${NC}"
62         echo " LuLu's localhost (loopback) filtering is OFF by default. To properly protect the"
63         echo " local nullexit proxy ports from malicious local processes, you must manually enable"
64         echo " 'Allow Loopback' (or block it selectively) in LuLu's Preferences -> Rules."
65         echo ""
66     fi
67
68     echo -e "${YELLOW}${BOLD}Note on macOS Tailscale Sandboxing:${NC}"
69     echo " The Tailscale GUI app (tailscale-app) installed on macOS is sandboxed."
70     echo " While you cannot run a Tailscale SSH server on this Mac host, you can"
71     echo " still SSH outbound to other devices on the mesh using their MagicDNS"
72     echo " addresses directly (e.g. ssh user@device.tailnet-name.ts.net)."
73     echo ""
74 fi

```

9 scripts/setup-linux.sh

```

1 #!/bin/bash
2 # scripts/setup-linux.sh nullexit one-shot setup script (Linux)
3 # Configures Cloudflare WARP + Tailscale + AdGuard Home from scratch.
4 set -euo pipefail
5
6 # Source common formatting and helper functions
7 source "$(dirname "$0")/common.sh"
8
9 # Run common installation logic
10 source "$(dirname "$0")/setup-common.sh"
11
12 # Compile Linux Desktop Shortcuts
13 echo -e "\n${BOLD}Creating Linux Desktop Shortcuts...${NC}"
14 DIR="$(pwd)"
15 cat <<EOF > "Toggle Gateway.desktop"
16 [Desktop Entry]
17 Version=1.0
18 Name=Toggle Gateway
19 Comment=Start or Stop Nullexit Gateway
20 Exec=bash -c "cd '$DIR' && ./toggle.sh; read -p 'Press Enter to close...' dummy"
21 Icon=utilities-terminal
22 Terminal=true
23 Type=Application
24 Categories=Network;Security;
25 EOF
26
27 cat <<EOF > "Recover Gateway.desktop"
28 [Desktop Entry]

```

```

29 Version=1.0
30 Name=Recover Gateway
31 Comment=Emergency DNS and Network Recovery
32 Exec=bash -c "cd '$DIR' && ./recover.sh; read -p 'Press Enter to close...' dummy"
33 Icon=utilities-terminal
34 Terminal=true
35 Type=Application
36 Categories=Network;Security;
37 EOF
38 echo "    Created 'Toggle Gateway.desktop' and 'Recover Gateway.desktop'"
39
40 # Done
41 echo ""
42 echo -e "${GREEN}${BOLD}${NC}"
43 echo -e "${GREEN}${BOLD}                Setup complete!                ${NC}"
44 echo -e "${GREEN}${BOLD}${NC}"
45 echo ""
46 echo -e "${BOLD}One manual step remaining  approve the exit node:${NC}"
47 echo "    https://login.tailscale.com/admin/machines"
48 echo "    Find your gateway '...' 'Edit route settings'  enable Exit Node"
49 echo ""
50
51 if [[ -n "${TS_IP:-}" ]]; then
52     echo -e "${BOLD}AdGuard Home dashboard:${NC}  http://${TS_IP}:3000  (username: admin)"
53     echo ""
54 fi
55
56 echo -e "${BOLD}To toggle the gateway on/off:${NC}"
57 echo "    Linux: double-click the 'Toggle Gateway' desktop icon!"
58 echo "    (or run ./toggle.sh in terminal)"
59 echo ""

```

10 docker-compose.yml

```

1 services:
2   warp:
3     image: qmcgaw/gluetun:v3.41.1
4     container_name: warp
5     hostname: nullexit-gateway
6     cap_add:
7       - NET_ADMIN
8     devices:
9       - /dev/net/tun:/dev/net/tun
10    ports:
11      - "127.0.0.1:${DNS_PROXY_PORT:-5354}:5335/udp" # AdGuard DNS: host binds 5354 (the mapped DNS-proxy
12        port); 5335 is AdGuard's in-container port
13      - "127.0.0.1:${DNS_PROXY_PORT:-5354}:5335/tcp" # AdGuard DNS (TCP)
14      - "41642:41642/udp" # Tailscale WireGuard (direct connection to host)
15      - "127.0.0.1:${SOCKS_PROXY_PORT:-1080}:1080/tcp" # SOCKS5 proxy (routed through WARP tunnel)
16      - "127.0.0.1:${TOR SOCKS_PORT:-9050}:9050/tcp" # Tor SOCKS5 proxy (procedurally generated)
17      - "127.0.0.1:${TOR_CONTROL_PORT:-9051}:9051/tcp" # Tor Control port (procedurally generated)
18    environment:
19      - TZ=UTC
20      - VPN_SERVICE_PROVIDER=custom
21      - VPN_TYPE=wireguard
22      - WIREGUARD_PRIVATE_KEY=${WIREGUARD_PRIVATE_KEY}
23      - WIREGUARD_PUBLIC_KEY=${WIREGUARD_PUBLIC_KEY}
24      - WIREGUARD_ADDRESSES=${WIREGUARD_ADDRESSES}
25      - WIREGUARD_ENDPOINT_IP=${WARP_ENDPOINT_1:-162.159.192.1}
26      - WIREGUARD_ENDPOINT_PORT=2408
27      # Set to 25s (WireGuard default) to beat standard 30s hardware NAT/firewall UDP timeouts
28      - WIREGUARD_PERSISTENT_KEEPLIVE_INTERVAL=25s
29      # MTU Fix (Critical): Prevent packet fragmentation from double-encryption
30      - WIREGUARD_MTU=1280
31      # Give WARP more time to establish the connection before internal restart (default is 6s which is too
32      short)
33      - HEALTH_VPN_DURATION_INITIAL=30s
34      # Use plaintext DNS resolvers instead of DNS-over-TLS (DoT) to prevent startup failures on networks where
35      port 853 is blocked
36      - DNS_UPSTREAM_RESOLVER_TYPE=plain
37      - DNS_UPSTREAM_PLAIN_ADDRESSES=1.1.1.1:53,8.8.8.8:53
38      # Allow local network traffic to bypass WARP so Tailscale can establish fast, direct peer-to-peer
39      connections (no DERP relays) on the LAN
40      - FIREWALL_OUTBOUND_SUBNETS=192.168.0.0/16,172.16.0.0/12,10.0.0.0/8
41      # Built-in DNS blocking for ads, malicious domains, and tracking
42      # NOTE: This is the massive "Hagezi-style" built-in blocklist.

```

```

39     # You can turn these to 'on' for maximum security if you have spare RAM
40     # or are running this on a cloud exit node with adequate autonomous resources (requires approx 1-2GB of
41     # Disabled intentionally: AdGuard Home provides the DNS filtering layer instead.
42     # If AdGuard fails, there is no fallback filtering, but the healthcheck
43     # dependency chain ensures Tailscale won't come up if AdGuard is down.
44     - BLOCK_MALICIOUS=off
45     - BLOCK_SURVEILLANCE=off
46     - BLOCK_ADS=off
47 dns:
48     - 1.1.1.1
49 sysctls:
50     - net.ipv4.ip_forward=1
51 volumes:
52     - ./post-rules.txt:/iptables/post-rules.txt:ro
53 restart: unless-stopped
54 healthcheck:
55     test: ["CMD-SHELL", "/gluetun-entrypoint healthcheck"]
56     interval: 2s
57     timeout: 5s
58     retries: 15
59     start_period: 60s
60 logging:
61     driver: "json-file"
62     options:
63         max-size: "10m"
64         max-file: "3"
65
66 tailscale:
67     image: tailscale/tailscale:v1.98.4
68     container_name: tailscale
69     network_mode: service:warp
70     depends_on:
71         warp:
72             condition: service_healthy
73         adguardhome:
74             condition: service_healthy
75     cap_add:
76         - NET_ADMIN
77         - NET_RAW
78         - SYS_MODULE
79     sysctls:
80         - net.ipv4.ip_forward=1
81         - net.ipv6.conf.all.forwarding=1
82     environment:
83         - TZ=UTC
84         - TS_EXTRA_ARGS=--advertise-exit-node
85         - PORT=41642
86         - TS_USERSPACE=false
87         - TS_AUTHKEY=${TS_AUTHKEY}
88         - TS_STATE_DIR=/var/lib/tailscale
89         # (Note: For headless/cloud deployments, you can bypass this footgun by
90         # using Tailscale Ephemeral Keys which auto-clean themselves).
91         - TS_AUTH_ONCE=true
92         # MTU Optimization: Force Tailscale packets to be smaller than WARP's 1280 MTU
93         # to prevent WireGuard-in-WireGuard fragmentation, completely eliminating bufferbloat
94         - TS_DEBUG_MTU=1200
95         # Enable Tailscale's built-in SOCKS5 proxy to route out of the WARP tunnel
96         - TS_SOCKS5_SERVER=:1080
97     devices:
98         - /dev/net/tun:/dev/net/tun
99     volumes:
100         - tailscale_state:/var/lib/tailscale
101 restart: unless-stopped
102 logging:
103     driver: "json-file"
104     options:
105         max-size: "10m"
106         max-file: "3"
107
108 routing-fix:
109     image: nullexit-routing-fix:v1.0.0
110     build:
111         context: ./scripts
112         dockerfile: Dockerfile.routing-fix
113     container_name: routing-fix
114     network_mode: service:warp
115     # NOTE: Deliberately NOT sharing tailscale's PID namespace the shared
116     # namespace caused routing-fix to be killed whenever tailscale restarted
117     # (e.g. during gluetun health-check flaps), which dropped all routing
118     # table changes and left traffic leaking through eth0 instead of tun0.

```

```

119 cap_add:
120   - NET_ADMIN
121   - SYSLOG
122 depends_on:
123   warp:
124     condition: service_healthy
125     # rule-compiler dependency removed: it is now a hash-gated one-off run by
126     # toggle.sh before `up` (15.8.8). This service boots on the existing
127     # compiled_rules.txt / ip_blocklist.ipset already present in ./adguard/work.
128 environment:
129   - TZ=UTC
130   - WARP_ENDPOINT=${WARP_ENDPOINT_1:-162.159.192.1}
131   - BLOCKED_COUNTRIES=${BLOCKED_COUNTRIES}
132   - TAILSCALE_ALLOW_LAN_P2P=${TAILSCALE_ALLOW_LAN_P2P:-false}
133 volumes:
134   - ./scripts/routing-fix.sh:/routing-fix.sh:ro
135   # SECURITY: ./adguard/work/ is bind-mounted into this container.
136   # Never write to it from the host or another container. The
137   # single allowed writer is `rule-compiler` container manual
138   # edits corrupt the AdGuard cache + BoltDB session store.
139   - ./adguard/work/userfilters:/userfilters:ro
140   - ./adguard/work:/adguard_work:ro
141   - ./app
142 restart: unless-stopped
143 stop_grace_period: 1s
144 logging:
145   driver: "json-file"
146   options:
147     max-size: "1m"
148     max-file: "1"
149
150 adguardhome:
151   # WARNING: AdGuard is pre-configured with hardcoded credentials (admin/nullexit).
152   # This is fine for a local Mac running behind a NAT, but if you are deploying
153   # this on a public cloud VPS with port 80/3000 exposed, change this immediately!
154   image: adguard/adguardhome:v0.107.77
155   container_name: adguardhome
156   network_mode: service:warp
157   depends_on:
158     warp:
159       condition: service_healthy
160       # rule-compiler dependency removed: it is now a hash-gated one-off run by
161       # toggle.sh before `up` (15.8.8). This service boots on the existing
162       # compiled_rules.txt / ip_blocklist.ipset already present in ./adguard/work.
163   environment:
164     - TZ=UTC
165   healthcheck:
166     test: ["CMD-SHELL", "nc -z 127.0.0.1 5335 || exit 1"]
167     interval: 2s
168     timeout: 5s
169     retries: 10
170   volumes:
171     # SECURITY: ./adguard/work/ is bind-mounted into this container.
172     # Never write to it from the host or another container. The
173     # single allowed writer is `rule-compiler` container manual
174     # edits corrupt the AdGuard cache + BoltDB session store.
175     - ./adguard/work:/opt/adguardhome/work
176     - ./adguard/conf:/opt/adguardhome/conf
177   restart: unless-stopped
178   stop_grace_period: 2s
179   logging:
180     driver: "json-file"
181     options:
182       max-size: "10m"
183       max-file: "3"
184
185 rule-compiler:
186   image: nullexit-rule-compiler:v1.0.0
187   build:
188     context: ./scripts
189     dockerfile: Dockerfile.rule-compiler
190   container_name: rule-compiler
191   # `compile` profile: excluded from `docker compose up`. toggle.sh runs it
192   # on-demand via `docker compose run --build --rm rule-compiler`, hash-gated on
193   # black_list.txt/white_list.txt, so ~340k rules are re-deduped only when a list
194   # actually changes instead of every toggle (~28s saved on the common path, 15.8.8).
195   profiles: ["compile"]
196   volumes:
197     - ./app
198
199 tor:

```



```

200 build: ./docker/tor
201 image: nullexit-tor:v1.0.0
202 container_name: tor
203 network_mode: service:warp
204 depends_on:
205     warp:
206         condition: service_healthy
207 environment:
208     - TZ=UTC
209     - TOR_USE_BRIDGES=${TOR_USE_BRIDGES:-false}
210     - TOR_BRIDGE_LINES_FILE=${TOR_BRIDGE_LINES_FILE:-./tor-bridges.txt}
211     - TOR_PASSWORD=${ADGUARD_PASSWORD:-nullexit}
212     - TOR_EXCLUDE_EXIT_NODES=${TOR_EXCLUDE_EXIT_NODES:-{us},{gb},{fr},{de},{it},{ca},{jp},{kp},{il}}
213 working_dir: /nullexit
214 volumes:
215     - ./:/nullexit:ro
216     - ./output.log:/output.log:rw
217 restart: unless-stopped
218 logging:
219     driver: "json-file"
220     options:
221         max-size: "10m"
222         max-file: "3"
223
224 volumes:
225     tailscale_state:
226
227 networks:
228     default:
229         ipam:
230             config:
231                 - subnet: 10.200.1.0/24

```

11 scripts/common.sh

```

1 #!/bin/bash
2 # common.sh shared bash utilities for nullexit scripts
3
4 # Formatting
5 RED='\033[0;31m'
6 GREEN='\033[0;32m'
7 YELLOW='\033[1;33m'
8 BLUE='\033[0;34m'
9 BOLD='\033[1m'
10 NC='\033[0m'
11
12 # Constants & Environment
13 export MARKER_FILE="/tmp/nullexit-gateway-active.marker"
14 export PID_CAFFEINATE="/tmp/nullexit-cafeinate.pid"
15 export PID_DNS_WATCHER="/tmp/nullexit-dns-watcher.pid"
16 export DEFAULT_WARP_ENDPOINT_1="162.159.192.1"
17 export DEFAULT_WARP_ENDPOINT_2="162.159.193.1"
18
19 setup_standard_path() {
20     export PATH="/opt/homebrew/bin:/opt/homebrew/sbin:/usr/local/bin:$PATH"
21 }
22
23 step() { echo -e "\n[ $(date '+%Y-%m-%d %H:%M:%S') ]" ${BLUE}${BOLD} ${NC}; }
24 ok() { echo -e "[ $(date '+%Y-%m-%d %H:%M:%S') ]" ${GREEN} ${NC}; }
25 warn() { echo -e "[ $(date '+%Y-%m-%d %H:%M:%S') ]" ${YELLOW} ${NC}; }
26 fail() { echo -e "[ $(date '+%Y-%m-%d %H:%M:%S') ]" ${RED} ${NC}; }
27 die() { echo -e "\n[ $(date '+%Y-%m-%d %H:%M:%S') ]" ${RED} ${NC}\n"; exit 1; }
28
29 # Helper Functions
30
31 # Append a timestamped line to output.log (best-effort; never fails the caller).
32 # Used to record lifecycle breadcrumbs (EXEC/EXIT markers, phase changes) so a
33 # START/STOP that is killed or window-closed still leaves a retraceable trail.
34 log_line() {
35     echo "[ $(date '+%Y-%m-%d %H:%M:%S') ]" "*" >> "${LOG_FILE:-output.log}" 2>/dev/null || true
36 }
37
38 # Run a lifecycle command with FULL observability: log the command, stream its
39 # combined stdout+stderr to the terminal AND append it to output.log via tee,
40 # then log the exit code explicitly. Returns the command's real exit code (not
41 # tee's). This closes the blind spot where `docker compose up` / `colima start`

```

```

42 # printed only to the terminal so when they fail (or the run is interrupted),
43 # output.log shows exactly what the last command was and whether it succeeded.
44 #
45 # Usage: run_logged docker compose up -d --build
46 run_logged() {
47     local logf="${LOG_FILE:-output.log}"
48     log_line "EXEC: $"
49     # PIPESTATUS[0] is the command's exit code; tee (PIPESTATUS[1]) is ignored.
50     "$@" 2>&1 | tee -a "$logf"
51     local rc=${PIPESTATUS[0]}
52     log_line "EXIT $rc: $"
53     return "$rc"
54 }
55
56 # Pure-bash timeout (no dependency on GNU coreutils' timeout)
57 # Runs a command with a safety cutoff so a wedged daemon can't hang the script.
58 run_with_timeout() {
59     local timeout_sec="$1"
60     shift
61     if [ $# -eq 0 ]; then return 1; fi
62
63     "$@" &
64     local cmd_pid=$!
65     CURRENT_BG_PID="$cmd_pid"
66
67     (
68         sleep "$timeout_sec"
69         if kill -0 "$cmd_pid" 2>> output.log; then
70             echo -e "\n[Timeout] Command '$*' exceeded $timeout_sec seconds. Terminating..." >&2
71             kill -15 "$cmd_pid" 2>> output.log || true
72             sleep 2
73             if kill -0 "$cmd_pid" 2>> output.log; then
74                 kill -9 "$cmd_pid" 2>> output.log || true
75             fi
76         fi
77     ) &
78     local watcher_pid=$!
79
80     # Disable set -e temporarily to capture exit status of wait
81     set +e
82     wait "$cmd_pid" 2>> output.log
83     local exit_status=$?
84     set -e
85
86     # Kill the watcher since the command finished
87     kill "$watcher_pid" 2>> output.log && wait "$watcher_pid" 2>> output.log || true
88
89     CURRENT_BG_PID=""
90     return "$exit_status"
91 }
92
93 # Start Colima and return the moment Docker is actually reachable, instead of
94 # blocking on colima 0.10.3's post-provision hang. Observed on macOS/vz: the VM
95 # reaches READY and creates the `colima` docker context in ~10-15s, then
96 # `colima start` frequently fails to return for ~110s (until a watchdog kills it)
97 # even though Docker is fully usable the entire time. We poll the colima docker
98 # context and proceed as soon as it answers, then reap the (usually hung) starter.
99 # The VM keeps running and `colima status` stays accurate. $@ = colima start args.
100 # Returns 0 once Docker responds, 1 if it never does within the cap.
101 colima_start_until_ready() {
102     local max_wait=90 waited=0 ready=false start_pid
103     echo "[$(date '+%Y-%m-%d %H:%M:%S')] EXEC(bg): colima start $" >> output.log
104     colima start "$@" >> output.log 2>&1 &
105     start_pid=$!
106     while [ "$waited" -lt "$max_wait" ]; do
107         if run_with_timeout 5 docker --context colima info >/dev/null 2>&1; then
108             ready=true
109             break
110         fi
111         # If colima start exited on its own (genuine finish or hard failure), stop waiting.
112         if ! kill -0 "$start_pid" 2>/dev/null; then
113             run_with_timeout 5 docker --context colima info >/dev/null 2>&1 && ready=true
114             break
115         fi
116         sleep 3
117         waited=$((waited + 3))
118     done
119     # Reap the starter (harmless if it already exited); this does NOT stop the VM.
120     kill "$start_pid" 2>/dev/null || true
121     wait "$start_pid" 2>/dev/null || true
122     if [ "$ready" = "true" ]; then

```

```

123     docker context use colima >/dev/null 2>&1 || true
124     echo "[$(date '+%Y-%m-%d %H:%M:%S')] Colima Docker ready after -${waited}s (starter reaped)." >> output.log
125     return 0
126 fi
127 echo "[$(date '+%Y-%m-%d %H:%M:%S')] Colima Docker NOT ready after ${max_wait}s." >> output.log
128 return 1
129 }
130
131 # Hash the files that feed the long-running locally-built images that `docker
132 # compose up` starts (routing-fix, tor). toggle.sh uses this to skip
133 # `docker compose --build` when nothing changed: BuildKit re-evaluating those
134 # images on the 600MB Colima VM costs ~30s per toggle even when every layer is
135 # CACHED (15.8.8). Includes the Dockerfiles, everything they COPY (routing-fix.sh,
136 # tor/entrypoint.sh, the logger/ Go source), and docker-compose.yml. The
137 # rule-compiler image is NOT here it is a `compile`-profiled one-off, built+run
138 # by the gated compile step (15.8.8). Prints a sha256 hex digest, or empty.
139 compute_build_inputs_hash() {
140     local root="${SCRIPT_DIR:-.}" f
141     {
142         for f in \
143             "$root/scripts/Dockerfile.routing-fix" \
144             "$root/scripts/routing-fix.sh" \
145             "$root/docker/tor/Dockerfile" \
146             "$root/docker/tor/entrypoint.sh" \
147             "$root/docker-compose.yml"; do
148             printf ':%s::' "$f"; cat "$f" 2>/dev/null
149         done
150         find "$root/scripts/logger" -type f 2>/dev/null \
151             | LC_ALL=C sort | while IFS= read -r f; do printf ':%s::' "$f"; cat "$f" 2>/dev/null; done
152     } | openssl dgst -sha256 2>/dev/null | awk '{print $NF}'
153 }
154
155 # Hash the ad-block lists the user controls: black_list.txt (custom blocks) and
156 # white_list.txt (the allow-list that governs what DNS resolves through). toggle.sh
157 # gates the ~28s rule recompile on this editing either list changes the digest
158 # and triggers a one-off recompile of compiled_rules.txt + ip_blocklist.ipset;
159 # an unchanged toggle skips it entirely (15.8.8). Prints a sha256 hex digest.
160 compute_rules_hash() {
161     local root="${SCRIPT_DIR:-.}"
162     { printf ':black::'; cat "$root/black_list.txt" 2>/dev/null
163       printf ':white::'; cat "$root/white_list.txt" 2>/dev/null
164     } | openssl dgst -sha256 2>/dev/null | awk '{print $NF}'
165 }
166
167 # Check if the gateway is active (either containers are running, or host DNS is hijacked)
168 is_gateway_active() {
169     # 1. Check if containers are running (suppress stderr to avoid errors if docker is down)
170     if run_with_timeout 15 docker compose ps --status running 2>/dev/null | grep -q 'warp'; then
171         echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: TRUE (warp container is running)" >> "$LOG_FILE"
172         return 0
173     fi
174
175     # 2. Check if host DNS was hijacked (not 1.1.1.1 and not default/empty)
176     local current_dns=""
177     local active_svc=""
178     if [[ "$OSTYPE" == "darwin"* ]]; then
179         active_svc=$(get_active_service)
180         current_dns=$(networksetup -getdnsservers "$active_svc" 2>> output.log || true)
181     else
182         current_dns=$(resolvectl dns 2>/dev/null | awk '/Global|Link/ {for(i=4;i<=NF;i++) print $i}' | head -n1)
183     fi
184
185     if [[ -n "$current_dns" && "$current_dns" != "1.1.1.1" && ! "$current_dns" =~ "There aren't any DNS Servers"
186         ]]; then
187         echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: TRUE (DNS was hijacked: $current_dns)" >> "
188             $LOG_FILE"
189         return 0
190     fi
191
192     # 3. Check if SOCKS proxy is enabled (macOS only)
193     if [[ "$OSTYPE" == "darwin"* ]]; then
194         local socks_proxy
195         socks_proxy=$(networksetup -getsocksfirewallproxy "$active_svc" 2>> output.log || true)
196         if echo "$socks_proxy" | grep -q "Enabled: Yes"; then
197             echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: TRUE (SOCKS proxy enabled)" >> "$LOG_FILE"
198             return 0
199         fi
200     fi
201
202     echo "[$(date '+%Y-%m-%d %H:%M:%S')] is_gateway_active: FALSE (Containers down, DNS clean, SOCKS disabled)"
203     >> "$LOG_FILE"

```

```

201     return 1
202 }
203
204 # Decide whether the gateway SHOULD be running, from persisted *intent* rather
205 # than a live health probe. This is the correct signal for toggle.sh's
206 # STOP-vs-START decision: a disable should tear down whatever the last START
207 # left behind it should NOT depend on whether Docker/Colima happens to be
208 # reachable right now.
209 #
210 # Why this exists: using is_gateway_active() for the decision means every toggle
211 # (including *disable*) runs `docker compose ps` first. When Colima is down the
212 # socket is absent, that call blocks for the full run_with_timeout window, and
213 # under `set -e` + the ERR trap a non-zero result at the `if then else fi`
214 # boundary could abort the script BEFORE the START body runs so the very path
215 # that would boot Colima never executed (observed as `EXIT: code=1 phase=start`).
216 #
217 # Signal source: MARKER_FILE is written at the end of a successful START
218 # (write_gateway_active_marker) and cleared at the top of STOP and in
219 # cleanup_handler. Reading it is instant and cannot hang or abort.
220 #
221 # ECHOES "running" or "stopped" to stdout (and ALWAYS returns 0). Callers read
222 # the string: [ "$(gateway_state)" = "running" ] &&
223 #
224 # CRITICAL why this echoes instead of using a 0/1 return code: on macOS
225 # /bin/bash 3.2, with `set -e` + an `ERR` trap + `set -x` (DEBUG_TRACE=true) all
226 # active, a function that does `return 1` triggers a SPURIOUS `set -e` abort in
227 # the caller even when the call is guarded (`if f; then`, or even `f || rc=$?`).
228 # The `return N` from the function, traced by `set -x`, trips the ERR trap. This
229 # is a genuine bash-3.2 interaction (see devref 15.11.8). Echoing a result and
230 # always returning 0 sidesteps `return`/`set -e`/`set -x` entirely the function
231 # can never contribute a non-zero status, so no caller can be aborted by it.
232 gateway_state() {
233     # Primary: persisted intent. Marker present => a START completed and no STOP
234     # has cleared it yet.
235     #
236     # NOTE: toggle.sh runs a "defensive stale-marker clear" near the top of every
237     # invocation (before this decision) to stop the wake-watcher firing against a
238     # crashed gateway. That would wipe the marker before we read it, so toggle.sh
239     # captures the marker's existence into GATEWAY_MARKER_AT_STARTUP *before* that
240     # clear and we honor it here. If the variable is unset (other callers, or the
241     # --restart pre-check which runs before the defensive clear), fall back to the
242     # live marker file.
243     if [ -n "${GATEWAY_MARKER_AT_STARTUP:-}" ]; then
244         if [ "$GATEWAY_MARKER_AT_STARTUP" = "yes" ]; then echo "running"; return 0; fi
245     elif [ -f "$MARKER_FILE" ]; then
246         echo "running"; return 0
247     fi
248
249     # Marker absent. Normally this means "stopped". But cover the edge case where
250     # the marker was lost (e.g. /tmp cleared, or gateway started by an older build)
251     # yet containers are genuinely up reconcile via is_gateway_active, but ONLY
252     # when Docker is actually reachable, so a dead-socket probe can never hang. On
253     # macOS Docker runs in the Colima VM; if that socket is absent, the gateway
254     # definitively is not running.
255     local docker_up="no"
256     if [[ "$OSTYPE" == "darwin"* ]]; then
257         if [ -S /var/run/docker.sock ] || [ -S "$HOME/.colima/default/docker.sock" ]; then docker_up="yes"; fi
258     else
259         if [ -S /var/run/docker.sock ]; then docker_up="yes"; fi
260     fi
261
262     if [ "$docker_up" = "yes" ] && is_gateway_active; then
263         echo "running"; return 0
264     fi
265     echo "stopped"
266     return 0
267 }
268
269 # Reads an environment variable from a file (default: .env), strips quotes and whitespace
270 read_env_var() {
271     local var_name="$1"
272     local env_file="${2:-.env}"
273     grep -E "^${var_name}= " "$env_file" 2>/dev/null | cut -d '=' -f2- | tr -d '\n\' | sed -e 's/^[[:space:]]*//'
274     -e 's/[[:space:]]*$//\' || echo ""
275 }
276
277 # Load KILL_SWITCH variable from .env (defaults to false if not found/empty)
278 export KILL_SWITCH
279 KILL_SWITCH=$(read_env_var "KILL_SWITCH" 2>/dev/null | tr '[:upper:]' '[:lower:]')
280 if [ -z "$KILL_SWITCH" ]; then
281     KILL_SWITCH="false"

```

```

281 fi
282
283 # Function to get the active network service name (e.g., "Wi-Fi" or "USB 10/100 LAN")
284 get_active_service() {
285     if [[ "$OSTYPE" == "darwin*" ]]; then
286         local iface
287         iface=$(route get default 2>> output.log | awk '/interface:/ {print $2}')
288
289         # If the default interface is empty or a VPN tunnel (like utunX), fallback to the active physical interface
290         if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
291             for i in en0 en1 en2 en3; do
292                 if ifconfig "$i" 2>> output.log | grep -q "status: active" && ifconfig "$i" 2>> output.log | grep -q "
inet "; then
293                     iface="$i"
294                     break
295                 fi
296             done
297         fi
298
299         # Default fallback if still empty or not enX
300         if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
301             iface="en0"
302         fi
303
304         # Map interface (e.g., en0) to service name (e.g., Wi-Fi)
305         local service
306         service=$(networksetup -listnetworkserviceorder 2>> output.log | grep -B 1 "Device: $iface" | head -n 1 |
sed -E 's/^\([0-9\*]+\)\s+ //' )
307
308         if [ -n "$service" ]; then
309             echo "$service"
310         else
311             echo "Wi-Fi"
312         fi
313     else
314         # Get the interface used for default internet routing
315         local iface
316         iface=$(ip route get 1.1.1.1 2>/dev/null | awk '{print $5; exit}')
317         if [ -z "$iface" ]; then
318             echo ""
319             return 1
320         fi
321         echo "$iface"
322     fi
323 }
324
325 get_en0_service() {
326     local service
327     service=$(networksetup -listnetworkserviceorder 2>> output.log | grep -B1 "Device: en0" | head -1 | sed -E 's
/^\([0-9\*]+\)\s+ //' || true)
328     if [ -z "$service" ]; then
329         echo "Wi-Fi"
330     else
331         echo "$service"
332     fi
333 }
334
335 get_warp_endpoint_1() { read_env_var "WARP_ENDPOINT_1" ".env" | grep -Eo '[0-9\.]+' || echo "
$DEFAULT_WARP_ENDPOINT_1"; }
336 get_warp_endpoint_2() { read_env_var "WARP_ENDPOINT_2" ".env" | grep -Eo '[0-9\.]+' || echo "
$DEFAULT_WARP_ENDPOINT_2"; }
337
338 restart_tailscaled_daemon() {
339     echo " [Tailscale Recovery] tailscaled daemon appears to be wedged/unresponsive."
340     echo " [Tailscale Recovery] Attempting to restart tailscaled service..."
341
342     if run_with_timeout 15 brew services restart tailscale >> output.log 2>&1; then
343         echo " [Tailscale Recovery] Successfully restarted tailscaled (user service)."
344     elif run_with_timeout 15 sudo -n brew services restart tailscale >> output.log 2>&1; then
345         echo " [Tailscale Recovery] Successfully restarted tailscaled (system service)."
346     else
347         echo " [Tailscale Recovery] WARNING: Failed to restart tailscaled. Manual intervention may be needed."
348     fi
349     sleep 3
350 }
351
352 disconnect_tailscale_host() {
353     local ts_bin="${1:-tailscale}"
354     if command -v "$ts_bin" >> output.log 2>&1; then
355         echo " Disconnecting host Tailscale from mesh..."
356

```

```

357 # Check if actually connected first (with timeout tailscaled could be wedged)
358 local status_ok=false
359 if run_with_timeout 5 "$ts_bin" status >> output.log 2>&1; then
360     status_ok=true
361 else
362     local status_exit=$?
363     # If it was a quick exit code 1 (disconnected), it is still reachable
364     if [ "$status_exit" -ne 143 ] && [ -S /var/run/tailscaled.socket ]; then
365         status_ok=true
366     fi
367 fi
368
369 if [ "$status_ok" = "false" ]; then
370     restart_tailscaled_daemon
371 fi
372
373 # Explicitly reset any exit-node so it doesn't linger.
374 "$ts_bin" up >> output.log 2>&1 || true
375 if run_with_timeout 10 "$ts_bin" set --ssh=true --accept-dns=false --exit-node= >> output.log 2>&1; then
376     echo " [] Exit-node preference cleared."
377 else
378     echo " [!] tailscale up didn't respond (tailscaled may be wedged)"
379 fi
380
381 local ts_args=""
382 if is_kill_switch_enabled; then ts_args="--accept-risk=lose-ssh"; fi
383 if run_with_timeout 10 "$ts_bin" down $ts_args >> output.log 2>&1; then
384     echo " [] Tailscale disconnected."
385 else
386     echo " [!] tailscale down didn't respond"
387 fi
388 else
389     echo " [!] tailscale CLI not found skipping"
390 fi
391 }
392
393 stop_pidfile_daemon() {
394     local pid_file="$1"
395     local label="$2"
396     if [ -f "$pid_file" ]; then
397         local pid
398         pid=$(cat "$pid_file")
399         if kill -0 "$pid" 2>/dev/null; then
400             echo " Stopping $label (PID $pid)..."
401             sudo -n kill "$pid" 2>> output.log || kill "$pid" 2>> output.log || true
402             sleep 0.5
403             if kill -0 "$pid" 2>/dev/null; then
404                 sudo -n kill -9 "$pid" 2>> output.log || kill -9 "$pid" 2>> output.log || true
405             fi
406         fi
407         rm -f "$pid_file"
408     fi
409 }
410
411 disable_all_proxies() {
412     local svc="$1"
413     if [ -n "$svc" ]; then
414         sudo -n networksetup -setsocksfirewallproxystate "$svc" off 2>> output.log || true
415         sudo -n networksetup -setwebproxystate "$svc" off 2>> output.log || true
416         sudo -n networksetup -setsecurewebproxystate "$svc" off 2>> output.log || true
417     fi
418 }
419
420 flush_dns_cache() {
421     if [[ "$OSTYPE" == "darwin*" ]]; then
422         sudo -n dscacheutil -flushcache 2>> output.log || true
423         sudo -n killall -HUP mDNSResponder 2>> output.log || true
424     else
425         resolvectl flush-caches 2>> output.log || true
426     fi
427 }
428
429 wait_for_dhcp_settle() {
430     # Resolve physical interface (Wi-Fi or Ethernet)
431     local physical_iface
432     physical_iface=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: (Wi-Fi|Ethernet)/{
433         getline; print $2; exit}')
434     [ -z "$physical_iface" ] && physical_iface="en0"
435
436     echo -n "Waiting for physical network interface ($physical_iface) DHCP lease to settle..."
437     local settled=false

```



```

437 for attempt in {1..60}; do
438     PHYSICAL_GW=$(ipconfig getpacket "$physical_iface" 2>> output.log | awk -F'[{ }]' '/router /{print $2}')
439     local local_ip
440     local_ip=$(ifconfig "$physical_iface" 2>/dev/null | awk '/inet /{print $2}')
441
442     if [ -n "$PHYSICAL_GW" ] && [[ "$PHYSICAL_GW" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]] && \
443         [ -n "$local_ip" ] && [[ "$local_ip" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]] && \
444         [[ "$local_ip" != "169.254.*" ]]; then
445         echo "settled (Interface IP: $local_ip, Router IP: $PHYSICAL_GW)."
446         settled=true
447         break
448     fi
449
450     if [ "$attempt" -gt 15 ] && [ -n "$local_ip" ] && [[ "$local_ip" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]] && \
451         [[ "$local_ip" != "169.254.*" ]]; then
452         local fallback_gw
453         fallback_gw=$(route get default 2>> output.log | awk '/gateway:/ {print $2}')
454         if [ -n "$fallback_gw" ] && [[ "$fallback_gw" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]]; then
455             PHYSICAL_GW="$fallback_gw"
456             fi
457             echo "static/settled detected (Interface IP: $local_ip, Gateway IP: ${PHYSICAL_GW:-unknown})."
458             settled=true
459             break
460         fi
461         echo -n "."
462         sleep 0.5
463     done
464     if [ "$settled" = "false" ]; then
465         echo "timed out or no active link. Proceeding anyway."
466     fi
467 }
468
469 reset_sharing_services() {
470     if [[ "$OSTYPE" == "darwin"* ]]; then
471         sudo -n killall sharingd rapportd 2>> output.log || true
472     fi
473 }
474
475 read_adguard_ip() {
476     if [ -f ".gateway_ip" ]; then
477         tr -d '\r' < ".gateway_ip" | awk 'NR==1{print $1;exit}'
478     fi
479 }
480
481 is_kill_switch_enabled() {
482     [ "$KILL_SWITCH" = "true" ]
483 }
484
485 add_warp_bypass_routes() {
486     local target="${1:-}"
487     local ep1
488     ep1=$(get_warp_endpoint_1)
489     local ep2
490     ep2=$(get_warp_endpoint_2)
491
492     if [[ "$OSTYPE" == "darwin"* ]]; then
493         local routing_arg=""
494         local msg_via=""
495         if [ -n "$target" ]; then
496             if [[ "$target" =~ ^en[0-9]+$ || "$target" == "bridge"* ]]; then
497                 routing_arg="-interface $target"
498                 msg_via="interface $target"
499             else
500                 routing_arg="$target"
501                 msg_via="gateway $target"
502             fi
503         else
504             local gateway_ip
505             gateway_ip=$(route get default 2>> output.log | awk '/gateway:/ {print $2}')
506             if [ -z "$gateway_ip" ]; then
507                 local iface
508                 iface=$(route get default 2>> output.log | awk '/interface:/ {print $2}')
509                 if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
510                     iface="en0"
511                 fi
512                 routing_arg="-interface $iface"
513                 msg_via="interface $iface"
514             else
515                 routing_arg="$gateway_ip"
516                 msg_via="gateway $gateway_ip"
517             fi
518         fi
519     fi

```

```

517 fi
518
519 echo -e "\nAdding host bypass routes for Cloudflare WARP endpoints via $msg_via..."
520 sudo -n route delete -host "$ep1" 2>/dev/null || true
521 sudo -n route delete -host "$ep2" 2>/dev/null || true
522 sudo -n route add -host "$ep1" $routing_arg >> output.log 2>&1 || true
523 sudo -n route add -host "$ep2" $routing_arg >> output.log 2>&1 || true
524
525 echo -e "Adding host bypass routes for Tailscale control plane via $msg_via..."
526 sudo -n route delete -net 192.200.0.0/24 2>/dev/null || true
527 sudo -n route add -net 192.200.0.0/24 $routing_arg >> output.log 2>&1 || true
528
529 echo -e "Adding host bypass routes for Tailscale DERP relays via $msg_via..."
530 local derp_ips
531 derp_ips=$(curl -s --connect-timeout 5 https://login.tailscale.com/derpmap/default | grep -oE 'IPv4'[:space:]]*[:space:]]*"[0-9.]+" | cut -d'"' -f4 | grep -E '^([0-9]+\.[0-9]+\.[0-9]+\.[0-9])+$' || true)
532 if [ -n "$derp_ips" ]; then
533     echo "$derp_ips" > /tmp/nullexit-derp-ips.txt
534     for ip in $derp_ips; do
535         sudo -n route delete -host "$ip" 2>/dev/null || true
536         sudo -n route add -host "$ip" $routing_arg >> output.log 2>&1 || true
537     done
538 fi
539 else
540     local gateway_ip="${target}"
541     if [ -z "$gateway_ip" ]; then
542         gateway_ip=$(ip route show default 2>/dev/null | awk '/default/ {print $3}')
543     fi
544     if [ -n "$gateway_ip" ]; then
545         sudo ip route add "$ep1" via "$gateway_ip" >> output.log 2>&1 || true
546         sudo ip route add "$ep2" via "$gateway_ip" >> output.log 2>&1 || true
547     fi
548 fi
549 }
550
551 remove_warp_bypass_routes() {
552     local ep1
553     ep1=$(get_warp_endpoint_1)
554     local ep2
555     ep2=$(get_warp_endpoint_2)
556
557     if [[ "$OSTYPE" == "darwin*" ]]; then
558         echo -e "\nRemoving host bypass routes for Cloudflare WARP endpoints..."
559         sudo -n route delete -host "$ep1" >> output.log 2>&1 || true
560         sudo -n route delete -host "$ep2" >> output.log 2>&1 || true
561         echo -e "Removing host bypass routes for Tailscale control plane..."
562         sudo -n route delete -net 192.200.0.0/24 >> output.log 2>&1 || true
563         if [ -f /tmp/nullexit-derp-ips.txt ]; then
564             echo -e "Removing host bypass routes for Tailscale DERP relays..."
565             while read -r ip; do
566                 sudo -n route delete -host "$ip" >> output.log 2>&1 || true
567             done < /tmp/nullexit-derp-ips.txt
568             rm -f /tmp/nullexit-derp-ips.txt
569         fi
570     else
571         sudo ip route del "$ep1" >> output.log 2>&1 || true
572         sudo ip route del "$ep2" >> output.log 2>&1 || true
573     fi
574 }
575
576 # FaceTime / real-time P2P split-route (opt-in, default OFF)
577 # Mitigation for the FaceTime Double-Tunnel bug (15.12.22). Adds more-specific
578 # host routes for Apple's FaceTime media/relay /16s via the PHYSICAL gateway, so
579 # longest-prefix match beats the exit-node 0.0.0.0/1 + 128.0.0.0/1 and ONLY those
580 # /16s leave direct real-time media can then hole-punch from the real IP instead
581 # of dying behind WARP's symmetric NAT. Mirrors add_warp_bypass_routes.
582 #
583 # PRIVACY: the direct /16s expose the real ISP IP (not WARP) for that traffic
584 # a deliberate, narrow hole in the Double-Tunnel promise, scoped to Apple infra
585 # (which already ties traffic to your Apple ID). Inert unless FACETIME_SPLIT_ROUTE
586 # =true. Needs BOTH a route AND a PF pass: 'block out on en0 all' would otherwise
587 # drop the packets, so we populate the <apple_direct> table pf.conf permits.
588 facetime_split_enabled() {
589     [ "$(read_env_var FACETIME_SPLIT_ROUTE | tr '[:upper:]' '[:lower:]')" = "true" ]
590 }
591
592 facetime_direct_subnets() {
593     local s; s=$(read_env_var FACETIME_DIRECT_SUBNETS)
594     echo "${s:-17.249.0.0/16}"
595 }
596

```

```

597 add_apple_split_routes() {
598     facetime_split_enabled || return 0
599     [[ "$OSTYPE" == "darwin"* ]] || return 0
600     local gw="${1:-}"
601     if [ -z "$gw" ]; then
602         # The physical default route survives alongside the exit-node /1 override,
603         # so the real gateway is still the 'default' entry in the table.
604         gw=$(netstat -rn -f inet 2>/dev/null | awk ' $1=="default"{print $2; exit}')
605     fi
606     if [ -z "$gw" ] || [[ "$gw" =~ ^utun ]] || [[ "$gw" =~ ^link ]]; then
607         echo " [FaceTime split-route] could not resolve a physical gateway skipping."
608         return 0
609     fi
610     local subnets; subnets=$(facetime_direct_subnets)
611     echo -e "\n[FaceTime split-route] routing Apple subnets DIRECT via $gw (real IP exposed for these): $subnets"
612     for net in $subnets; do
613         sudo -n route delete -net "$net" >> output.log 2>&1 || true
614         sudo -n route add -net "$net" "$gw" >> output.log 2>&1 || true
615         # Kill-switch pass: the <apple_direct> table is what pf.conf lets out on en0.
616         sudo -n pfctl -a com.apple/nullexit -t apple_direct -T add "$net" >> output.log 2>&1 || true
617     done
618 }
619
620 remove_apple_split_routes() {
621     [[ "$OSTYPE" == "darwin"* ]] || return 0
622     local subnets; subnets=$(facetime_direct_subnets)
623     for net in $subnets; do
624         sudo -n route delete -net "$net" >> output.log 2>&1 || true
625     done
626     # Flush the whole table so no direct Apple exception lingers after teardown.
627     sudo -n pfctl -a com.apple/nullexit -t apple_direct -T flush >> output.log 2>&1 || true
628 }
629
630 bounce_wifi_interfaces() {
631     if [[ "$OSTYPE" == "darwin"* ]]; then
632         local wifi_port
633         wifi_port=$(networksetup -listallhardwareports 2>> output.log | awk '/Hardware Port: Wi-Fi/{getline; print $2}')
634         if [ -n "$wifi_port" ]; then
635             sudo -n networksetup -setairportpower "$wifi_port" off 2>> output.log || true
636             sleep 2
637             sudo -n networksetup -setairportpower "$wifi_port" on 2>> output.log || true
638             echo "Wi-Fi bounced (interface $wifi_port)"
639             sleep 3
640         else
641             echo "Could not detect Wi-Fi interface. Bouncing fallback interfaces..."
642             set +e
643             for iface in en0 en1 en2 en3 en4 en5; do
644                 if ifconfig "$iface" >> output.log 2>&1; then
645                     sudo -n ifconfig "$iface" down 2>> output.log || true
646                     sudo -n ifconfig "$iface" up 2>> output.log || true
647                 fi
648             done
649             set -e
650             echo "Fallback interfaces bounced"
651         fi
652     fi
653 }
654
655 get_active_interface() {
656     local iface
657     iface=$(route get default 2>> output.log | awk '/interface:/ {print $2}')
658
659     # If the default interface is empty or a VPN tunnel (like utunX), fallback to the active physical interface
660     if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
661         for i in en0 en1 en2 en3; do
662             if ifconfig "$i" 2>> output.log | grep -q "status: active" && ifconfig "$i" 2>> output.log | grep -q "
663                 inet "; then
664                     iface="$i"
665                     break
666                 fi
667             done
668         fi
669         # Default fallback if still empty or not enX
670         if [[ -z "$iface" || ! "$iface" =~ ^en[0-9]+$ ]]; then
671             iface="en0"
672         fi
673         echo "$iface"
674     fi
675 }

```

```

676 enable_killswitch() {
677     if [[ "$OSTYPE" == "darwin"* ]]; then
678         if ! is_kill_switch_enabled; then
679             return 0
680         fi
681     fi
682
683     local ext_if
684     ext_if=$(get_active_interface)
685     local ep1
686     ep1=$(get_warp_endpoint_1)
687     local ep2
688     ep2=$(get_warp_endpoint_2)
689     local pf_conf_path
690     pf_conf_path="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/pf.conf"
691
692     # Keep the host's scrub max-mss in sync with GATEWAY_MSS (same variable that drives
693     # post-rules.txt's --set-mss for the container). The host egresses through the same
694     # double-tunnel (Tailscale+WARP) as forwarded phone traffic, so it needs the same
695     # empirically-safe clamp a stale/higher value here reproduces the 15.3.2 stalling
696     # bug for the host's own connections (see devref.md).
697     local gateway_mss
698     gateway_mss=$(read_env_var GATEWAY_MSS)
699     if [[ "$gateway_mss" =~ ^[0-9]+$ ]]; then
700         sed -i 's/max-mss [0-9]*/max-mss ${gateway_mss}/g' "$pf_conf_path" 2>/dev/null || true
701     fi
702
703     echo -e "\nEnabling macOS Packet Filter (PF) Kill-Switch on $ext_if..."
704
705     # Enable PF and capture the reference token
706     local token_out
707     token_out=$(sudo -n pfctl -E 2>> output.log || true)
708     local token
709     token=$(echo "$token_out" | awk '/Token/{print $NF}')
710     if [ -n "$token" ]; then
711         echo "$token" > /tmp/nullexit-pf-token.txt
712         echo " [] PF enabled with token $token."
713     else
714         echo " [!] WARNING: Could not capture PF reference token (may already be enabled or sudo failed)."
715     fi
716
717     # Load the rules into the anchor
718     if sudo -n pfctl -D "ext_if=$ext_if" -a com.apple/nullexit -f "$pf_conf_path" >> output.log 2>&1; then
719         echo " [] Ruleset loaded into anchor com.apple/nullexit."
720     else
721         echo " [!] ERROR: Failed to load ruleset into anchor."
722     fi
723
724     # Populate tables in the anchor
725     sudo -n pfctl -a com.apple/nullexit -t vpn_endpoints -T flush >> output.log 2>&1 || true
726     sudo -n pfctl -a com.apple/nullexit -t vpn_endpoints -T add "$ep1" >> output.log 2>&1 || true
727     sudo -n pfctl -a com.apple/nullexit -t vpn_endpoints -T add "$ep2" >> output.log 2>&1 || true
728
729     if [ -f /tmp/nullexit-derp-ips.txt ]; then
730         local derp_ips
731         derp_ips=$(tr '\n' ' ' < /tmp/nullexit-derp-ips.txt)
732         if [ -n "$derp_ips" ]; then
733             sudo -n pfctl -a com.apple/nullexit -t derp_relays -T flush >> output.log 2>&1 || true
734             sudo -n pfctl -a com.apple/nullexit -t derp_relays -T add $derp_ips >> output.log 2>&1 || true
735         fi
736     fi
737     echo " [] Kill-switch tables populated."
738 fi
739 }
740
741 disable_killswitch() {
742     if [[ "$OSTYPE" == "darwin"* ]]; then
743         if ! is_kill_switch_enabled; then
744             return 0
745         fi
746         echo -e "\nDisabling macOS PF Kill-Switch..."
747
748         # Flush rules from anchor com.apple/nullexit
749         sudo -n pfctl -a com.apple/nullexit -F all >> output.log 2>&1 || true
750
751         # Release the enable reference if token is present
752         if [ -f /tmp/nullexit-pf-token.txt ]; then
753             local token
754             token=$(cat /tmp/nullexit-pf-token.txt)
755             if [ -n "$token" ]; then
756                 sudo -n pfctl -X "$token" >> output.log 2>&1 || true

```

```

757     echo " [] Released PF enable reference token $token."
758     fi
759     rm -f /tmp/nullexit-pf-token.txt
760 else
761     echo " [] Rules flushed from anchor com.apple/nullexit."
762     fi
763 fi
764 }
765
766
767 write_host_ips() {
768     # Gather all active IPv4 addresses on the host and write to .host_ips
769     # routing-fix.sh will read this file to explicitly whitelist them for direct Tailscale P2P
770     local repo_dir
771     repo_dir="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
772
773     if [[ "$OSTYPE" == "darwin"* ]]; then
774         ifconfig | awk '/inet /{print $2}' | grep -v '127.0.0.1' > "$repo_dir/.host_ips" 2>/dev/null || true
775     else
776         ip -4 addr show 2>/dev/null | awk '/inet /{print $2}' | cut -d/ -f1 | grep -v '127.0.0.1' > "$repo_dir/.
777         host_ips" 2>/dev/null || true
778     fi
779 }
780
781 start_dns_watcher() {
782     if [[ "$OSTYPE" == "darwin"* ]]; then
783         local target_ip=$1
784         stop_pidfile_daemon "/tmp/nullexit-dns-watcher.pid" "background DNS Watcher"
785         echo " Starting background DNS Watcher for seamless Wi-Fi roaming..."
786         nohup bash -c "
787             source \"\$SCRIPT_DIR/scripts/common.sh\"
788             trap 'exit 0' SIGTERM SIGINT SIGHUP
789             while true; do
790                 ACTIVE_IF=$(get_active_service)
791                 if [ -n \"\$ACTIVE_IF\" ]; then
792                     CURRENT_DNS=$(networksetup -getdnsservers \"\$ACTIVE_IF\" 2>/dev/null)
793                     if [ \"\$CURRENT_DNS\" != \"\$target_ip\" ]; then
794                         networksetup -setdnsservers \"\$ACTIVE_IF\" \"\$target_ip\" >/dev/null 2>&1
795                     fi
796                 fi
797                 sleep 30
798             done
799             " >> output.log 2>&1 &
800         echo $! > "$DNS_WATCHER_PID_FILE"
801     else
802         local target_ip=$1
803         stop_dns_watcher
804         echo " Starting background DNS Watcher for seamless Wi-Fi roaming..."
805         nohup bash -c "
806             trap 'exit 0' SIGTERM SIGINT SIGHUP
807             while true; do
808                 if [ -n \"\$ACTIVE_SERVICE\" ]; then
809                     CURRENT_DNS=$(resolvectl dns \"\$ACTIVE_SERVICE\" 2>/dev/null | grep -oE
810                     '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | head -1)
811                     if [ \"\$CURRENT_DNS\" != \"\$target_ip\" ]; then
812                         resolvectl dns \"\$ACTIVE_SERVICE\" \"\$target_ip\" >/dev/null 2>&1 || true
813                     fi
814                 fi
815                 sleep 30
816             done
817             " >> output.log 2>&1 &
818         echo $! > "$DNS_WATCHER_PID_FILE"
819     fi
820 }
821
822 # SOCKS5 Proxy (WARP Tunnel)
823 # When Tailscale's exit node can't route through WARP (due to userspace
824 # forwarding), we use a SOCKS5 proxy running inside the warp container's
825 # network namespace. Connections created by this proxy go through the
826 # kernel routing table (table 200 -> tun0 -> WARP), unlike Tailscale's
827 # userspace exit node which bypasses it.
828 #
829 # The SOCKS5 proxy is exposed on localhost:${SOCKS_PROXY_PORT} via Docker port mapping.
830 # `networksetup -setsocksfirewallproxy` on macOS routes system TCP traffic
831 # through the proxy, which then goes through WARP.
832 #
833 # IMPORTANT: CLI tools like curl do NOT respect macOS system proxy settings.
834 # They must use --socks5-hostname or the ALL_PROXY env variable explicitly.
835
836 enable_socks_proxy() {
837     if [[ "$OSTYPE" == "darwin"* ]]; then

```

```

836 local svc="$1"
837 if [ -z "$svc" ]; then
838     svc="$ACTIVE_SERVICE"
839 fi
840
841 echo -n " Enabling system-wide SOCKS5 proxy on $svc (localhost:$SOCKS_PROXY_PORT)... "
842 sudo -n networksetup -setsocksfirewallproxy "$svc" 127.0.0.1 $SOCKS_PROXY_PORT 2>> output.log || true
843 sudo -n networksetup -setsocksfirewallproxystate "$svc" on 2>> output.log || true
844
845 # Also set on en0 service for macOS resolver consistency
846 if [ "$svc" != "$ENO_SERVICE" ]; then
847     sudo -n networksetup -setsocksfirewallproxy "$ENO_SERVICE" 127.0.0.1 $SOCKS_PROXY_PORT 2>> output.log ||
848     true
849     sudo -n networksetup -setsocksfirewallproxystate "$ENO_SERVICE" on 2>> output.log || true
850 fi
851
852 # IMPORTANT: Exclude local LAN and mDNS traffic from the proxy so P2P works natively
853 # without source-IP masking from the Colima VM bridge.
854 sudo -n networksetup -setproxybypassdomains "$svc" 127.0.0.1 localhost 192.168.0.0/16 10.0.0.0/8
855 172.16.0.0/12 "*.local" 2>> output.log || true
856 if [ "$svc" != "$ENO_SERVICE" ]; then
857     sudo -n networksetup -setproxybypassdomains "$ENO_SERVICE" 127.0.0.1 localhost 192.168.0.0/16 10.0.0.0/8
858     172.16.0.0/12 "*.local" 2>> output.log || true
859 fi
860
861 echo "done."
862 echo " All TCP traffic now routed through gateway -> WARP tunnel -> internet."
863 echo " (CLI tools: export ALL_PROXY=socks5://127.0.0.1:$SOCKS_PROXY_PORT)"
864 else
865     local svc="$1"
866     echo " (Linux global SOCKS proxy configuration is desktop-environment specific, skipping.)"
867     echo " SOCKS5 proxy is active and listening on 127.0.0.1:$SOCKS_PROXY_PORT"
868 fi
869 }
870
871 disable_socks_proxy() {
872     if [[ "$OSTYPE" == "darwin"* ]]; then
873         for svc in "$ACTIVE_SERVICE" "$ENO_SERVICE"; do
874             sudo -n networksetup -setsocksfirewallproxystate "$svc" off 2>> output.log || true
875         done
876         echo " SOCKS5 proxy disabled."
877     else
878         local svc="$1"
879         echo " (Linux global SOCKS proxy configuration skipped.)"
880     fi
881 }
882
883 # Helper to restore host DNS to a clean state (used on script start, on failure,
884 # and after a successful STOP). Without this, a successful toggle-off leaves the
885 # host's resolver pinned to a now-dead gateway IP every lookup stalls for macOS's
886 # DNS timeout (~5s) before falling through to whatever next server is configured.
887 # With it, we always leave the host in `1.1.1.1 / no search domain`.
888 #
889 # We set DNS on BOTH the active service AND the en0 service because macOS's scutil DNS
890 # resolver is commonly scoped to en0 (Wi-Fi). A per-service change on e.g.
891 # "USB 10/100 LAN" is ignored by the system resolver unless the en0 service is also updated.
892 reset_dns() {
893     if [[ "$OSTYPE" == "darwin"* ]]; then
894         if [ -n "$ACTIVE_SERVICE" ]; then
895             for dns_svc in "$ACTIVE_SERVICE" "$ENO_SERVICE"; do
896                 networksetup -setsearchdomains "$dns_svc" "Empty" 2>> output.log || true
897                 networksetup -setdnsservers "$dns_svc" 1.1.1.1 2>> output.log || true
898             done
899         fi
900     else
901         if [ -n "$ACTIVE_SERVICE" ]; then
902             resolvectl domain "$ACTIVE_SERVICE" "" 2>/dev/null || true
903             resolvectl revert "$ACTIVE_SERVICE" 2>/dev/null || true
904         fi
905     fi
906 }
907
908 # Verify basic direct-internet connectivity after a recovery/teardown: DNS via
909 # 1.1.1.1, then HTTP (curl) or ping to 1.1.1.1. Sets the global INTERNET_OK to
910 # "true" on any successful check (callers read $INTERNET_OK in their summary).
911 # Always returns 0 so a bare call can't trip `set -e`. Used by recover.sh's macOS
912 # and Linux recovery paths (extracted from a formerly-duplicated block).
913 verify_internet_connectivity() {
914     step "Verifying internet connectivity"
915
916     # Wait a moment for the network to settle

```



```

914 sleep 2
915
916 INTERNET_OK=false
917
918 # Try DNS resolution first
919 if host -W 3 google.com 1.1.1.1 >> output.log 2>&1; then
920     ok "DNS resolution works (google.com via 1.1.1.1)"
921     INTERNET_OK=true
922 elif nslookup google.com 1.1.1.1 >> output.log 2>&1; then
923     ok "DNS resolution works (nslookup google.com via 1.1.1.1)"
924     INTERNET_OK=true
925 else
926     warn "DNS resolution check failed will still try ping/curl"
927 fi
928
929 # Try actual HTTP connectivity
930 if command -v curl >> output.log 2>&1; then
931     if curl -sf --max-time 5 https://1.1.1.1 >> output.log 2>&1; then
932         ok "Internet reachable via HTTP"
933         INTERNET_OK=true
934     elif curl -sf --max-time 5 http://1.1.1.1 >> output.log 2>&1; then
935         ok "Internet reachable via HTTP (plain)"
936         INTERNET_OK=true
937     else
938         warn "HTTP check failed"
939     fi
940 elif command -v ping >> output.log 2>&1; then
941     if ping -c 1 -W 3 1.1.1.1 >> output.log 2>&1; then
942         ok "Internet reachable via ping"
943         INTERNET_OK=true
944     else
945         warn "Ping check failed"
946     fi
947 fi
948
949 return 0
950 }
951
952 # Local DNS Proxy (Python)
953 # When the tailnet data plane cannot establish, we use a Python DNS proxy.
954 # It listens on UDP:53, receives DNS queries, forwards them over TCP to
955 # AdGuard at 127.0.0.1:${DNS_PROXY_PORT} (Docker port mapping), and returns the response.
956 # The Python script handles the DNS-over-TCP wire format (2-byte length prefix)
957 # correctly unlike socat which just forwards raw bytes.
958 #
959 # Python3 is built into macOS no external dependencies.
960 DNS_PROXY_PID=""
961
962 # Kill any leftover DNS proxy process
963 stop_local_dns_proxy() {
964     if [ -n "$DNS_PROXY_PID" ]; then
965         kill "$DNS_PROXY_PID" 2>/dev/null || true
966         wait "$DNS_PROXY_PID" 2>/dev/null || true
967         DNS_PROXY_PID=""
968     fi
969     # Clean up any stale Python DNS proxy processes
970     sudo -n pkill -f "dns-proxy.py" 2>/dev/null || true
971     rm -f /tmp/nullexit-dns-proxy.conf 2>/dev/null || true
972 }
973
974 # Start local DNS proxy (Python)
975 start_local_dns_proxy() {
976     if [ -z "$DNS_PROXY_BIN" ]; then
977         echo " Python not found. Python3 is built into macOS this shouldn't happen."
978         return 1
979     fi
980
981     if [ ! -f "$DNS_PROXY_SCRIPT" ]; then
982         echo " DNS proxy script not found at $DNS_PROXY_SCRIPT"
983         return 1
984     fi
985
986     # Kill any leftover process first
987     stop_local_dns_proxy
988
989     echo -n " Starting local DNS proxy via Python (UDP:53 TCP:${DNS_PROXY_PORT})... "
990     # The shipped sudoers NOPASSWD rule only covers the literal `python3
991     # dns-proxy.py` command no bash -c wrapper, no env prefix so sudo -n's
992     # env_reset strips TARGET_PORT before it ever reaches the script. Drop it in
993     # a file dns-proxy.py reads itself instead (see its own comment for why).
994     echo "TARGET_PORT=${DNS_PROXY_PORT}" > /tmp/nullexit-dns-proxy.conf

```

```

995 chmod 600 /tmp/nullexit-dns-proxy.conf 2>/dev/null || true
996 sudo -n "$DNS_PROXY_BIN" "$DNS_PROXY_SCRIPT" &
997 DNS_PROXY_PID=$!
998 disown "$DNS_PROXY_PID" 2>/dev/null || true
999
1000 # Give it a moment to bind
1001 sleep 0.5
1002
1003 if kill -0 "$DNS_PROXY_PID" 2>/dev/null; then
1004     echo "started (PID $DNS_PROXY_PID)."
1005
1006     # Hijack host DNS to localhost
1007     echo -n " Hijacking host DNS to 127.0.0.1 for ad-blocking... "
1008     if [[ "$OSTYPE" == "darwin"* ]]; then
1009         networksetup -setsearchdomains "$ACTIVE_SERVICE" "Empty" 2>/dev/null || true
1010         networksetup -setsearchdomains "$ENO_SERVICE" "Empty" 2>/dev/null || true
1011         if ! networksetup -setdnsservers "$ACTIVE_SERVICE" 127.0.0.1; then
1012             echo "FAILED (networksetup error check permissions)."
1013             stop_local_dns_proxy
1014             return 1
1015         fi
1016         networksetup -setdnsservers "$ENO_SERVICE" 127.0.0.1 2>/dev/null || true
1017         sudo -n dscacheutil -flushcache 2>/dev/null || true
1018         sudo -n killall -HUP mDNSResponder 2>/dev/null || true
1019     else
1020         resolvectl domain "$ACTIVE_SERVICE" "" 2>/dev/null || true
1021         resolvectl domain "$ACTIVE_SERVICE" "" 2>/dev/null || true
1022         if ! sudo -n resolvectl dns "$ACTIVE_SERVICE" 127.0.0.1; then
1023             echo "FAILED (resolvectl error check permissions)."
1024             stop_local_dns_proxy
1025             return 1
1026         fi
1027         resolvectl dns "$ACTIVE_SERVICE" 127.0.0.1 2>/dev/null || true
1028         sudo -n resolvectl flush-caches 2>/dev/null || true
1029     fi
1030
1031     # Verify DNS actually works through the proxy
1032     echo -n " Verifying DNS resolution... "
1033     if dig @127.0.0.1 google.com +short +timeout=5 &>/dev/null; then
1034         echo "ok."
1035         return 0
1036     else
1037         echo "FAILED (DNS queries not reaching AdGuard)."
1038         echo " Restoring DNS to 1.1.1.1..."
1039         reset_dns
1040         stop_local_dns_proxy
1041         return 1
1042     fi
1043 else
1044     echo "FAILED (could not bind port 53 is it in use?)."
1045     DNS_PROXY_PID=""
1046     return 1
1047 fi
1048 }
1049
1050 # Helper to FORCIBLY hijack host DNS to a single server (the gateway's AdGuard IP)
1051 # and VERIFY via `networksetup -getdnsservers` that the only name server is exactly
1052 # $1. Returns 0 iff the live state matches; non-zero if it can't be confirmed.
1053 #
1054 # This deliberately does NOT append a 1.1.1.1 fallback: macOS's resolver queries
1055 # the list in order and falls back to the next entry on timeout, so on a
1056 # misbehaving AdGuard (filter compile crash, OOM kill, etc.) the resolver still
1057 # leaks to 1.1.1.1 and silently bypasses ad-blocking. With no fallback, a broken
1058 # gateway manifests as a *visible* DNS outage that prompts the user to
1059 # investigate which is the correct failure mode.
1060 force_dns_to_gateway() {
1061     if [[ "$OSTYPE" == "darwin"* ]]; then
1062         local target_ip="$1"
1063         if [ -z "$target_ip" ] || [ -z "$ACTIVE_SERVICE" ]; then
1064             echo " [force_dns] ERROR: target_ip or ACTIVE_SERVICE is empty" >&2
1065             return 1
1066         fi
1067
1068         local attempt
1069         for attempt in 1 2 3; do
1070             echo -n " [force_dns] Attempt $attempt/3: setting DNS to $target_ip on \"\$ACTIVE_SERVICE\" + \"\$ENO_SERVICE\"... "
1071
1072             # Apply to BOTH the active service AND the en0 service the scutil DNS resolver
1073             # is scoped to en0 (Wi-Fi) and ignores per-service changes that don't include it.
1074             # Fail fast on ACTIVE_SERVICE; best-effort on ENO_SERVICE (it may not exist).

```

```

1075 networksetup -setsearchdomains "$ACTIVE_SERVICE" "ts.net" || true
1076 if ! networksetup -setdnsservers "$ACTIVE_SERVICE" "$target_ip"; then
1077     echo "FAILED (networksetup error check permissions)"
1078     sleep 1
1079     continue
1080 fi
1081 networksetup -setsearchdomains "$ENO_SERVICE" "ts.net" || true
1082 networksetup -setdnsservers "$ENO_SERVICE" "$target_ip" || true
1083
1084 local entries
1085 entries=$(networksetup -getdnsservers "$ACTIVE_SERVICE" 2>> output.log \
1086     | awk '/^(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)
1087     \.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\.(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)$/ {print}')
1088 if echo "$entries" | grep -qFx "$target_ip"; then
1089     echo "VERIFIED"
1090     return 0
1091 fi
1092
1093 local current
1094 current=$(echo "$entries" | tr '\n' ' ' | sed 's/ $//')
1095 echo "NOT YET (current DNS: [${current:-none}])"
1096
1097 if [ "$attempt" = "3" ]; then sleep 2; else sleep 1; fi
1098 done
1099
1100 echo " [force_dns] FAILED after 3 attempts"
1101 return 1
1102 else
1103     local target_ip="$1"
1104
1105     for attempt in {1..3}; do
1106         echo -n " [force_dns] Attempt $attempt/3: setting DNS to $target_ip on $ACTIVE_SERVICE... "
1107         sudo -n resolvectl domain "$ACTIVE_SERVICE" "~ts.net" 2>/dev/null || true
1108         if ! sudo -n resolvectl dns "$ACTIVE_SERVICE" "$target_ip"; then
1109             echo "FAILED (resolvectl error)"
1110             else
1111                 # Verify
1112                 entries=$(resolvectl dns "$ACTIVE_SERVICE" 2>> output.log | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+')
1113                 if [ "$entries" == "$target_ip" ]; then
1114                     echo "VERIFIED"
1115                     return 0
1116                 else
1117                     echo "MISMATCH (Resolver refused lock)"
1118                 fi
1119             fi
1120             sleep 2
1121         done
1122         return 1
1123     fi
1124 }

```

12 scripts/watcher.sh

```

1 #!/bin/bash
2 # scripts/watcher.sh nullexit post-wake / post-roam recovery watcher
3 #
4 # Long-running daemon. Launched by launchd as a LaunchAgent at
5 # ~/Library/LaunchAgents/com.nullexit.wake-recovery.plist
6 # (label com.nullexit.wake-recovery).
7 #
8 # Two independent listeners run as backgrounded pipelines:
9 #
10 # (1) WAKE listener `log stream` live-follows the macOS unified log for
11 # power-management events (lid closewake, manual wake, DarkWake from
12 # clamshell). Each event triggers run_recover (with a global debounce so
13 # a single wake that emits 2-3 log lines only causes ONE post-wake run).
14 #
15 # (2) NETWORK listener `scutil n.watch` on `State:/Network/Global/IPv4`
16 # fires on every Wi-Fi roam, ethernet swap, hotspot join, captive-portal
17 # re-bind, VPN up/down, etc. Same debounce.
18 #
19 # Both listeners call run_recover, which:
20 # - checks /tmp/nullexit-gateway-active.marker (only act when toggle.sh left
21 # the gateway up)
22 # - debounces (10s default) so multiple events don't pile up
23 # - shells out to `bash <repo>/recover.sh --post-wake` directly (no GUI auth prompt needed due to NOPASSWD
    sudoers)

```

```

24 #
25 # See devref.md 10.29 for the full Apple power management primer and the
26 # rationale behind using `log stream` + `scutil n.watch` from a shell daemon.
27
28 # NOTE: errexit (`set -e`) is intentionally NOT enabled. This is a long-running
29 # daemon, not a discrete-step script, and errexit actively breaks the reconnect
30 # loops below: any failed pipe (log stream on rotation, scutil on state churn)
31 # would kill the outer subshell before `echo "reconnecting..."; sleep 5;` runs,
32 # AND the parent's final `wait` would propagate a non-zero child exit and kill
33 # the entire watcher. Every error path is already wrapped in `|| true` /
34 # `2>/dev/null` fallbacks.
35 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
36 source "$SCRIPT_DIR/common.sh"
37 setup_standard_path
38
39 LOG="$SCRIPT_DIR/../output.log"
40 # Resolve our own directory; recover.sh lives one level up at the repo root.
41 # This makes the daemon install-agnostic no need to template the path
42 # in launchd, no need to keep a deploy-specific hardcoded location in sync.
43 RECOVER="$SCRIPT_DIR/../recover.sh"
44 MARKER="$MARKER_FILE"
45 DEBOUNCE_FILE="/tmp/nullexit-watcher.last-recovery"
46 DEBOUNCE_SECONDS="${NULLEXIT_DEBOUNCE_SECONDS:-10}"
47 # Post-recovery SETTLE cooldown. A post-wake run re-asserts the exit node
48 # (recover.sh: `tailscale set --exit-node`), which re-writes the host default
49 # route and that route IS State:/Network/Global/IPv4, the exact key Listener 2
50 # watches. The self-triggered change lands ~15-45s AFTER the run finishes, past
51 # the 10s debounce, so without a longer window the watcher re-fires post-wake in
52 # a ~40s self-feedback loop that only stops once routing happens to reach a fixed
53 # point (devref 15.12.21). This cooldown is armed ONLY after a recovery runs, so
54 # a genuine first wake/roam is still handled immediately; only the change we
55 # caused ourselves gets absorbed. A real new roam during the window is delayed at
56 # most this many seconds, which is acceptable.
57 COOLDOWN_FILE="/tmp/nullexit-watcher.settle-until"
58 POSTWAKE_SETTLE_SECONDS="${NULLEXIT_POSTWAKE_SETTLE_SECONDS:-45}"
59
60 exec >> "$LOG" 2>&1
61
62 # Single-instance lock
63 # Without this, repeated lid-closewake cycles under macOS Sonoma+ accumulate
64 # orphan watcher.sh processes: launchd suspends on sleep and SIGCONT-resumes
65 # on wake, and KeepAlive.Crashed=true caused relaunch on any non-zero exit;
66 # the listener grandchildren (log stream | while ... and (echo n.add ...;
67 # sleep 86400) | scutil | while ...) stay alive because pipe producers don't
68 # reliably respond to plain SIGTERM if the consumer is in a half-closed state.
69 #
70 # We solve it with a PID-file lock (POSIX-portable; macOS does NOT ship `flock`).
71 # The trap below removes the lock file on every exit so a stale lock never
72 # outlives a crashed watcher. On launch: read the existing PID, check it is
73 # alive AND is actually a `scripts/watcher.sh` process; if so, exit 0; else
74 # overwrite with our PID and proceed. The check-then-write window has a TOCTOU
75 # race but is microseconds wide and only matters for hand-launched duplicate
76 # test invocations; production is single-launch via launchctl load -w.
77 LOCK_FILE="/tmp/nullexit-watcher.lock"
78 LOCK_PID=""
79 if [ -e "$LOCK_FILE" ]; then
80     LOCK_PID=$(cat "$LOCK_FILE" 2>/dev/null || echo "")
81     if [ -n "$LOCK_PID" ] && kill -0 "$LOCK_PID" 2>/dev/null; then
82         # Verify the holder is actually a scripts/watcher.sh process (not some
83         # unrelated process that got the recycled PID). The exact-launchd pattern
84         # `bash <abs-path>/scripts/watcher.sh` is hard to accidentally match with
85         # a `grep scripts/watcher.sh` invocation because we anchor on `^bash`
86         # followed by whitespace and end-of-line on the script path.
87         if ps -p "$LOCK_PID" -o args= 2>/dev/null | grep -qE "(^|[:space:])bash[:space:]+.*scripts/watcher\\.sh$"; then
88             echo "[$(date -u +%FT%TZ)] watcher.sh: another instance holds $LOCK_FILE (pid=$LOCK_PID), exiting"
89             exit 0
90         fi
91     fi
92 fi
93 echo $$ > "$LOCK_FILE"
94
95 echo "[$(date -u +%FT%TZ)] watcher.sh start (pid=$$ ppid=$PPID user=$(id -un) lock=acquired)"
96
97 # Treat launchd-resume as a wake signal. Apple's launchd SUSPENDS LaunchAgents
98 # during system sleep and restores them on wake; during that suspended gap,
99 # macOS's `log stream` cannot catch the wake event that prompted the resume.
100 # The next thing that happens after wake IS our own startup, so we always
101 # fire one post-wake run unconditionally here. The marker check + debounce
102 # in run_recover() make this idempotent: if the gateway isn't up or we just
103 # debounced, it no-ops. Without this, the FIRST wake-after-install goes

```

```

104 # unseen and the watcher appears broken until the SECOND wake.
105 # Helper: run recover.sh --post-wake if the gateway is active
106 run_recover() {
107     local why="$1"
108     if [ ! -f "$MARKER" ]; then
109         echo "[$(date -u +%FT%TZ)] $why no $MARKER, skip"
110         return 0
111     fi
112     local now last settle_until
113     now=$(date +%s)
114     # Post-recovery settle window: ignore the Global/IPv4 change our own last
115     # post-wake run caused (exit-node re-assert) so we don't self-feedback loop.
116     settle_until=$(cat "$COOLDOWN_FILE" 2>/dev/null || echo 0)
117     if [ "$now" -lt "$settle_until" ]; then
118         echo "[$(date -u +%FT%TZ)] $why settling ($(settle_until - now)s left in post-recovery cooldown), skip"
119         return 0
120     fi
121     last=$(cat "$DEBOUNCE_FILE" 2>/dev/null || echo 0)
122     if [ "$((now - last))" -lt "$DEBOUNCE_SECONDS" ]; then
123         echo "[$(date -u +%FT%TZ)] $why debounced ($(now - last)s since last, threshold ${DEBOUNCE_SECONDS}s)"
124         return 0
125     fi
126     echo "$now" > "$DEBOUNCE_FILE"
127     echo "[$(date -u +%FT%TZ)] $why bash $RECOVER --post-wake"
128     bash "$RECOVER" --post-wake
129     local exit_code=$?
130     now=$(date +%s)
131     echo "$now" > "$DEBOUNCE_FILE"
132     # Arm the settle cooldown so the route change this run just caused can't re-fire us.
133     echo "$((now + POSTWAKE_SETTLE_SECONDS))" > "$COOLDOWN_FILE"
134     echo "[$(date -u +%FT%TZ)] $why exit=$exit_code (settle until +${POSTWAKE_SETTLE_SECONDS}s, now=$now)"
135     return $exit_code
136 }
137
138 if [ -f "$MARKER" ]; then
139     run_recover "initial post-wake (launchd resume = wake signal)"
140 fi
141
142 # LAN P2P Auto-Detection
143 # Detects whether the current network allows safe Tailscale LAN P2P by:
144 # 1. Checking WPA2-Enterprise (802.1x) via system_profiler SPAirPortDataType always forces P2P=false
145 #    because enterprise networks almost universally have AP Client Isolation.
146 # 2. AP Isolation probe sends a broadcast ping and checks if ANY LAN host responds.
147 #    If no LAN hosts respond on a non-empty network, AP Isolation is active.
148 # Writes 'true' or 'false' to .lan_p2p_detected in the repo root.
149 # routing-fix.sh reads this file dynamically every 30s loop iteration.
150 P2P_OVERRIDE_FILE="$SCRIPT_DIR/../../lan_p2p_detected"
151
152
153 detect_lan_p2p_mode() {
154     local reason="unknown" allow="false"
155
156     # Step 1: Detect Wi-Fi security type via system_profiler (airport binary removed in macOS 14.4+)
157     # SPAirPortDataType lists all known networks; the first "Security:" line is the current association.
158     local security
159     security=$(system_profiler SPAirPortDataType 2>/dev/null \
160         | awk '/Current Network Information:/{found=1} found && /Security:/{print $NF; exit}')
161
162     if [ -z "$security" ]; then
163         # Not on Wi-Fi or system_profiler unavailable fail safe
164         reason="no-wifi" allow="false"
165     elif echo "$security" | grep -qi "Enterprise"; then
166         # WPA2/WPA3 Enterprise = 802.1x always AP isolated
167         reason="802.1x-enterprise" allow="false"
168     elif echo "$security" | grep -qi "Personal\|None\|Open"; then
169         # WPA2/WPA3 Personal or open probe for AP isolation
170         local gw
171         gw=$(route -n get 1.1.1.1 2>/dev/null | awk '/gateway:/{print $2}')
172         if [ -z "$gw" ]; then
173             reason="no-default-route" allow="false"
174         else
175             local responses
176             responses=$(ping -c 3 -t 1 -q "$gw" 2>/dev/null | awk '/packets transmitted/{print $4}')
177             if [ "${responses:-0}" -gt 0 ]; then
178                 reason="wpa-personal-lan-reachable" allow="true"
179             else
180                 reason="wpa-personal-ap-isolated" allow="false"
181             fi
182         fi
183     else
184         # Unknown security type fail safe

```

```

185     reason="unknown-security-type" allow="false"
186 fi
187
188 echo "[$(date -u +%FT%TZ)] P2P detect: security='${security:-none}' allow=${allow} (reason: ${reason})"
189 echo "$allow" > "$P2P_OVERRIDE_FILE"
190 write_host_ips
191 }
192
193 # Run once on startup
194 detect_lan_p2p_mode
195
196 # Listener 1: WAKE events via unified log
197 # Predicate picks up:
198 # * "didWake"          regular kernel wake events (lid open, RTC alarm)
199 # * "Wake from Sleep"  powerd-driven normal wake
200 # * "DarkWake"         clamshell-display wake that didn't fully wake the
201 #                       Mac (relevant when lid opens during display sleep)
202 # * "Waking from"      generic catch-all for variations in newer macOS
203 # `[c]` makes the match case-insensitive (the unified log uses mixed-case
204 # phrases). --info reduces noise by dropping debug/trace log levels.
205 # Subsystem-based predicate catches every powermanagement emission (willSleep,
206 # didWake, didDim, didUndim, DarkWake, etc.) regardless of message phrasing
207 # across kernel versions. Phrasing-based predicates ('didWake', 'Wake from Sleep')
208 # are fragile across macOS releases. We accept the extra log noise because the
209 # run_recover() debounce + marker check filter out anything that isn't a real
210 # gateway-relevant event.
211 (
212 # Reconnect loop: macOS rotates the unified log buffer periodically; when
213 # that happens, this `log stream` subscriber's pipe EOFs, the inner
214 # `while read` consumer terminates, and without this wrapper the listener
215 # would be silently dead for the rest of the watcher's lifetime.
216 while ;; do
217     log stream --predicate \
218         'subsystem == "com.apple.powermanagement"' \
219         --style compact --info 2>/dev/null \
220         | while IFS= read -r line; do
221             [ -z "$line" ] && continue
222             run_recover "WAKE: $(echo "$line" | head -c 100)"
223         done
224     echo "[$(date -u +%FT%TZ)] log stream subshell exited; reconnecting in 5s"
225     sleep 5
226 done
227 ) &
228
229 # Listener 2: NETWORK state changes via scutil
230 # `scutil n.watch` on a state key prints SCEventUpdate lines whenever the key
231 # changes. We watch Global/IPv4 because that's the umbrella that catches link,
232 # address, route, and DNS changes for ALL interfaces in one namespace.
233 # The pipe is kept alive by an `sleep 86400` loop so scutil never sees EOF
234 # from us (which would silently terminate the watch).
235 (
236 # Reconnect loop: scutil can exit noisily on certain state changes (rare,
237 # but observed). Without the wrapper, the network listener's pipe EOFs and
238 # every subsequent roam goes unseen until launchd restarts us.
239 while ;; do
240     (
241         # Watch both IPv4 and IPv6 umbrella state IPv6-only or dual-stack
242         # networks never fire on the IPv4 key alone, and the user's first roam
243         # in a hotel / airport / on cellular hotspot is often IPv6-first under
244         # NAT64.
245         echo "n.add State:/Network/Global/IPv4"
246         echo "n.add State:/Network/Global/IPv6"
247         echo "n.watch"
248         while true; do sleep 86400; done
249     ) | script -q /dev/null scutil 2>/dev/null \
250     | while IFS= read -r line; do
251         case "$line" in
252             *changedKey*|*State:/Network/Global/IPv*|*n.state*|*SCEventUpdate*)
253                 detect_lan_p2p_mode
254                 run_recover "NET: $(echo "$line" | head -c 100)"
255                 ;;
256             *) ;;
257         esac
258     done
259     echo "[$(date -u +%FT%TZ)] scutil pipe exited; reconnecting in 5s"
260     sleep 5
261 done
262 ) &
263 # Listener 3: TUNNEL_FAILED_CLOSED.marker Watcher
264 # Polls for the fail-closed marker dropped by the routing-fix container.
265 (

```



```

266 last_notified=0
267 while ;; do
268     if [ -f "$SCRIPT_DIR/./TUNNEL_FAILED_CLOSED.marker" ]; then
269         now=$(date +%s)
270         # Notify once every 5 minutes (300s) if it stays failed
271         if [ "$((now - last_notified))" -ge 300 ]; then
272             osascript -e 'display notification "VPN Tunnel Health Check Failed. Internet traffic is blackholed to
prevent IP leak." with title "NullExit Alert"' || true
273             last_notified=$now
274         fi
275     else
276         last_notified=0
277     fi
278     sleep 3
279 done
280 ) &
281
282 # Listener 4: Wi-Fi uplink keepalive
283 # Re-associates a remembered network (scripts/wifi-rejoin.sh) whenever the
284 # Wi-Fi interface has no routable IP. This lives HERE, in the always-on
285 # wake-recovery daemon, on purpose: the WARP watcher (toggle.sh) that used to
286 # be the ONLY driver of wifi-rejoin runs solely while the gateway is UP, so a
287 # Wi-Fi drop while the gateway was down/off left the Mac "just sitting there
288 # without internet" nothing rejoined. This poller is independent of gateway
289 # state, so it recovers Wi-Fi either way. Leak-safe: it only acts when the
290 # interface has NO address (empty or 169.254.* self-assigned) i.e. no egress
291 # path exists, so there is nothing to leak. wifi-rejoin.sh enforces its own
292 # grace period, backoff, and give-up cap, and only ever joins a network you
293 # explicitly remembered. When the gateway IS up, the WARP watcher may also call
294 # wifi-rejoin; the two share the same /tmp markers so they coordinate rather
295 # than double-associate.
296 (
297     WIFI_HELPER="$SCRIPT_DIR/wifi-rejoin.sh"
298     wifi_iface=$(networksetup -listallhardwareports 2>/dev/null \
299         | awk '/Hardware Port: Wi-Fi/{getline; print $2; exit}')
300     [ -z "$wifi_iface" ] && wifi_iface=en0
301     while ;; do
302         phys_ip=$(ipconfig getifaddr "$wifi_iface" 2>/dev/null || true)
303         case "${phys_ip:-}" in
304             ''|169.254.*) bash "$WIFI_HELPER" rejoin >/dev/null 2>&1 || true ;;
305             *) bash "$WIFI_HELPER" reset >/dev/null 2>&1 || true ;;
306         esac
307         sleep 15
308     done
309 ) &
310
311 # Lifetime
312 # `wait` blocks until both background pipelines exit (which they normally
313 # never do). launchctl `bootout`/SIGTERM triggers the trap, which kills the
314 # whole process group so the sleep-forever in the scutil pipe also dies.
315 # We catch TERM/INT/HUP for explicit signals and EXIT for any normal-exit path
316 # (so listeners always get cleaned up even if `exit 0` runs somewhere).
317 # `rm -f $LOCK_FILE` first so a stale lock can never block the next launch.
318 # `pkill -TERM -P $$` then targets direct listener-children, then `kill 0`
319 # takes care of any backgrounded group-mates not reachable via the parent.
320 trap 'echo "[$(date -u +%FT%TZ)] watcher.sh terminating (signal/exit=$?)"; rm -f "$LOCK_FILE" 2>/dev/null ||
true; pkill -TERM -P $$ 2>/dev/null || true; kill 0 2>/dev/null || true; exit 0' TERM INT HUP EXIT
321 wait

```

13 scripts/crypto.sh

```

1 #!/usr/bin/env bash
2 # crypto.sh - Cryptographic Integrity Check
3
4 set -euo pipefail
5 cd "$(dirname "$0")/.."
6
7 if [ "$#" -ne 1 ] || { [ "$1" != "--sign" ] && [ "$1" != "--verify" ]; }; then
8     echo "Usage: $0 --sign | --verify"
9     exit 1
10 fi
11
12 if [ ! -f .env ]; then
13     echo "Error: .env not found."
14     exit 1
15 fi

```

```

16 SEED=$(grep "^NULLEXIT_SEED=" .env | cut -d '=' -f2)
17 if [ -z "$SEED" ]; then
18     echo "Error: NULLEXIT_SEED not found in .env."
19     exit 1
20 fi
21
22 if [ "$1" == "--sign" ]; then
23     echo "Generating cryptographic signatures using seed..."
24     rm -f .signatures
25     for file in toggle.sh recover.sh scripts/common.sh scripts/routing-fix.sh scripts/diagnose-host-leak.sh
26         scripts/sweep.sh scripts/noise.sh scripts/watcher.sh scripts/pf.conf post-rules.txt docker-compose.yml
27         black_list.txt white_list.txt; do
28         if [ -f "$file" ]; then
29             hash=$(openssl dgst -sha256 -hmac "$SEED" "$file" | awk '{print $NF}')
30             echo "$file:$hash" >> .signatures
31             echo " [Signed] $file"
32         else
33             echo " [Skipped] $file not found"
34         fi
35     done
36     echo "Signatures saved to .signatures"
37     exit 0
38 fi
39
40 if [ "$1" == "--verify" ]; then
41     if [ ! -f .signatures ]; then
42         echo "Error: .signatures file missing. Run $0 --sign first."
43         exit 1
44     fi
45     FAIL=0
46     while IFS=: read -r file expected_hash; do
47         if [ -z "$file" ]; then continue; fi
48         if [ ! -f "$file" ]; then
49             echo "[FAIL] $file is missing!"
50             FAIL=1
51             continue
52         fi
53         actual_hash=$(openssl dgst -sha256 -hmac "$SEED" "$file" | awk '{print $NF}')
54         if [ "$actual_hash" != "$expected_hash" ]; then
55             echo "[FAIL] $file has been modified! Hash mismatch."
56             FAIL=1
57         fi
58     done < .signatures
59
60     if [ "$FAIL" -eq 1 ]; then
61         echo ""
62         echo "CRITICAL: Script integrity verification failed!"
63         echo "One or more core files have been modified without authorization."
64         echo "To authorize these changes, run: ./scripts/crypto.sh --sign"
65         echo ""
66         exit 1
67     fi
68     exit 0
69 fi

```

14 .aiignore

```

1 # Environment variables and secrets
2 .env
3 tor-bridges.txt

```

15 .claude/settings.local.json

```

1 {
2     "permissions": {
3         "allow": [
4             "Bash(grep *)",
5             "Bash(python3 /tmp/gen_nullexit_graph.py)",
6             "Bash(python3 -)",
7             "Bash(\"/Applications/Google Chrome.app/Contents/MacOS/Google Chrome\" --headless=new --disable-gpu --
            window-size=1280,800 --virtual-time-budget=8000 --dump-dom \"file:///tmp/kg_debug.html\")",

```

```

8      "Bash(echo \"dangling-check-exit=$?\")",
9      "Bash(bash scripts/wifi-rejoin.sh reset)",
10     "Bash(launchctl kickstart *)",
11     "Bash(perl -0pi -e 's/see 15\\\\.11\\\\.7\\\\\\\\b/see 15.11.10/g; s/\\\\\\\\\\\\\\\\(15\\\\\\\\.11\\\\\\\\.7\\\\\\\\\\\\\\\\)\\\\\\\\(15.11.10\\\\\\\\)/g; s/15\\\\\\\\.11\\\\\\\\.7\\\\\\\\\\\\\\\\(SSID redaction\\\\\\\\\\\\\\\\)/15.11.10\\\\\\\\(SSID redaction\\\\\\\\)/g' devref.md)",
12     "Bash(perl -0pi -e 's/see `devref\\\\\\\\.md 15\\\\\\\\.11\\\\\\\\.7`/see `devref.md 15.11.10`/g' README.md)"
13 ]
14 }
15 }

```

16 .env.example

```

1 # nullexit configuration template copy to .env and fill in.
2 # cp .env.example .env
3 # .env is gitignored (it holds secrets); this template documents every option
4 # with its shipping default. For coherent bundles, see scripts/profiles.sh.
5
6 # Secrets (fill these in)
7 # WARP WireGuard keys from your Cloudflare WARP registration.
8 WIREGUARD_PRIVATE_KEY=<your-warp-wireguard-private-key>
9 WIREGUARD_PUBLIC_KEY=<your-warp-wireguard-public-key>
10 WIREGUARD_ADDRESSES=172.16.0.2/32
11 # Tailscale auth key Admin Console Settings Keys Generate auth key.
12 TS_AUTHKEY=tskey-auth-xxxxxxxxxxxxxxxxxxxxxxxxxxxx
13 # HMAC seed for script-integrity signatures generate with: openssl rand -hex 32
14 NULLEXIT_SEED=<run: openssl rand -hex 32>
15 # AdGuard Home UI + Tor ControlPort credentials.
16 ADGUARD_USER=admin
17 ADGUARD_PASSWORD=<choose-a-password>
18
19 # Gateway core
20 # Rule-compiler memory profile: light (~52k) / medium (~167k) / heavy (~253k).
21 GATEWAY_RULE_PROFILE=medium
22 STOP_COLIMA_ON_EXIT=true
23 # PF fail-closed kill-switch. Leave true this is the core leak guarantee.
24 KILL_SWITCH=true
25 # true only on trusted home networks (no AP isolation); false on campus/enterprise
26 # WPA2-Enterprise (forces DERP, avoids gvproxy SNAT poisoning).
27 TAILSCALE_ALLOW_LAN_P2P=false
28 GATEWAY_MSS=1120
29 GATEWAY_BYPASS_PING=false
30 GATEWAY_USE_EXIT_NODE=true
31 # Consecutive warp-off polls before auto-shutdown (6 = 30s; 3 = faster/stricter).
32 WARP_FAIL_THRESHOLD=6
33 BLOCKED_COUNTRIES="kp il"
34 HOST_MTU=1200
35 # Debug xtrace of toggle.sh to output.log (keep false for normal use).
36 DEBUG_TRACE=false
37
38 # Tor hardening
39 # true = Tor must use obfs4 bridges only (never public guards).
40 TOR_USE_BRIDGES=false
41 TOR_BRIDGE_LINES_FILE=./tor-bridges.txt
42 TOR_EXCLUDE_EXIT_NODES={us},{gb},{fr},{de},{it},{ca},{jp},{kp},{il}
43
44 # Cover traffic / dummy padding (scripts/noise.sh) opt-in, default off
45 NOISE_ENABLED=false
46 NOISE_PAD_TARGET=
47 NOISE_PAD_PORT=9999
48 NOISE_PAD_RATE_KBPS=64
49 NOISE_PAD_PACKET_BYTES=1024
50 NOISE_PAD_JITTER_MS=40
51 # constant = always emit the target rate; topup = fill real-traffic gaps to keep
52 # the observed rate flat (stronger anti-correlation, no waste when already busy).
53 NOISE_PAD_MODE=constant
54 NOISE_PAD_IFACE=en0
55 # Optional: target as a % of LINK CAPACITY (overrides NOISE_PAD_RATE_KBPS).
56 NOISE_PAD_PCT=
57 NOISE_LINK_MBPS=100
58
59 # FaceTime split-route (scripts/common.sh) opt-in, default off
60 # true routes the Apple /16s below DIRECT on your REAL IP (not WARP) so
61 # FaceTime can hole-punch. A deliberate, narrow hole scoped to Apple infra.
62 FACETIME_SPLIT_ROUTE=false
63 FACETIME_DIRECT_SUBNETS="17.249.0.0/16"

```

```

64
65 # LAN Pi-hole mode (scripts/pihole-mode.sh) opt-in, default off
66 # Serve AdGuard DNS filtering to non-Tailscale LAN devices (trusted networks only).
67 PIHOLE_LAN_MODE=false
68 PIHOLE_LAN_LISTEN=
69
70 # Sweep throughput check (scripts/sweep.sh, check 7/7) all optional
71 # Uncomment to override the defaults baked into sweep.sh (a fast CDN, 30s cap).
72 # SWEEP_THROUGHPUT_URL=https://ash-speed.hetzner.com/100MB.bin
73 # SWEEP_THROUGHPUT_TIMEOUT=30
74 # SWEEP_SKIP_THROUGHPUT=false

```

17 .gitignore

```

1 # Environment variables and secrets
2 .env
3
4 # wgcf generated files containing keys and account info
5 wgcf-account.toml
6 wgcf-profile.conf
7
8 # Tailscale persistent state containing node identity
9 tailscale-state/
10
11 # Binaries
12 wgcf
13
14 # macOS App bundles
15 *.app/
16 *.app
17
18 # AdGuard Home persistent state
19 adguard/
20 .gateway_ip
21 logs.txt
22
23 #
24 stability_incident_report.md
25 output.log
26 .DS_Store
27 *.desktop
28 blocked.log
29 wtf.md
30 host-leak-diagnostic-*.txt
31 nullexit_review.md
32 nullexit_unified.tex
33 nullexit_unified.txt
34
35 nullexit_unified.synctex.gz
36 tor-bridges.txt
37
38 # Runtime state files
39 .host_ips
40 .lan_p2p_detected
41 .build_hash
42 .rules_hash
43 .last_wifi_ssid
44 .last_wifi_ssid.tmp
45 .signatures
46 TUNNEL_FAILED_CLOSED.marker
47
48 # Python cache
49 __pycache__/*
50 *.py[cod]
51
52 # LaTeX build artifacts
53 *.aux
54 *.log
55 *.out
56 *.toc
57 *.xdv
58 *.fdb_latexmk
59 *.fls
60 *.synctex*
61 refactor.md
62 sweep.md

```

```

63
64 # Obsidian Knowledge graph
65 nullexit-knowledge-graph/
66
67 # DNS anomaly detector per-network model (regenerated by 'learn')
68 .dns_baseline.json
69
70 # .env backups written by profiles.sh (contain secrets never commit)
71 .env.bak*
72
73 # device-scramble saved real identity (never commit)
74 .device-identity.orig

```

18 Recover-Gateway.applescript

```

1 tell application "Finder" to set scriptDir to POSIX path of (container of (path to me) as alias)
2 try
3     do shell script "true" with administrator privileges
4 on error
5     return
6 end try
7 tell application "Terminal"
8     activate
9     do script "cd " & quoted form of scriptDir & "; bash recover.sh"
10 end tell

```

19 Sweep-Gateway.applescript

```

1 tell application "Finder" to set scriptDir to POSIX path of (container of (path to me) as alias)
2 try
3     do shell script "cd " & quoted form of scriptDir & " && ./scripts/crypto.sh --verify"
4 on error errMsg
5     display dialog "nullexit integrity check FAILED sweep will not run." & return & return & "A signed file (
        scripts/sweep.sh, scripts/common.sh, ) no longer matches .signatures. If you edited it on purpose, re-sign
        with:" & return & "./scripts/crypto.sh --sign" & return & return & errMsg buttons {"Abort"} default button
        "Abort" with icon stop
6     return
7 end try
8 tell application "Terminal"
9     activate
10    do script "cd " & quoted form of scriptDir & " && bash scripts/sweep.sh"
11 end tell

```

20 Toggle-Gateway.applescript

```

1 tell application "Finder" to set scriptDir to POSIX path of (container of (path to me) as alias)
2 try
3     do shell script "true" with administrator privileges
4 on error
5     return
6 end try
7 tell application "Terminal"
8     activate
9     do script "cd " & quoted form of scriptDir & "; ./toggle.sh"
10 end tell

```

21 benchmarks/benchmark.sh

```

1 #!/usr/bin/env bash
2
3 # benchmark.sh
4 # Automates the full benchmarking suite (both Python download test and Lighthouse HTML test)
5 # Runs the suite with Gateway ON, toggles OFF, runs again, then toggles back ON.

```

```

6
7 set -e
8
9 # Change to project root so we can access toggle.sh reliably
10 cd "$(dirname "$0")/.."
11
12 # Ensure jq is installed
13 if ! command -v jq &> /dev/null; then
14     echo "Error: 'jq' is not installed. Please run 'brew install jq'"
15     exit 1
16 fi
17
18 run_lighthouse() {
19     local url=$1
20     local file_prefix=$2
21     echo " [Lighthouse] Testing $url ..."
22
23     # Run lighthouse headlessly, silencing standard output
24     npx -y lighthouse "$url" --chrome-flags="--headless" --only-categories=performance --output=json --output-path="${file_prefix}.json" >/dev/null 2>&1
25
26     # Parse the results
27     local score=$(jq -r '.categories.performance.score' "${file_prefix}.json")
28     local tti=$(jq -r '.audits["interactive"].displayValue' "${file_prefix}.json")
29     local si=$(jq -r '.audits["speed-index"].displayValue' "${file_prefix}.json")
30
31     # Convert score to percentage
32     local score_pct=$(awk "BEGIN {print $score*100}")
33
34     echo "    -> Performance Score: ${score_pct}%"
35     echo "    -> Time to Interactive: $tti"
36     echo "    -> Speed Index: $si"
37
38     # Cleanup
39     rm -f "${file_prefix}.json"
40 }
41
42 echo "=====
43 echo "    PHASE 1: Testing with Gateway ON"
44 echo "=====
45 # 1. Run raw Python download test
46 python3 benchmarks/test_load.py
47
48 # 2. Run Lighthouse HTML render test
49 echo ""
50 echo "Starting Lighthouse HTML Tests (this takes ~30-60s per site)..."
51 run_lighthouse "https://www.cnet.com/" "on_cnet"
52 run_lighthouse "https://www.independent.co.uk/" "on_ind"
53
54
55 echo ""
56 echo "=====
57 echo "    PHASE 2: Toggling Gateway OFF"
58 echo "=====
59 ./toggle.sh
60 echo "Waiting 10 seconds for macOS Wi-Fi to stabilize after DNS reset..."
61 sleep 10
62
63
64 echo ""
65 echo "=====
66 echo "    PHASE 3: Testing with Gateway OFF"
67 echo "=====
68 # 1. Run raw Python download test
69 python3 benchmarks/test_load.py
70
71 # 2. Run Lighthouse HTML render test
72 echo ""
73 echo "Starting Lighthouse HTML Tests (this takes ~30-60s per site)..."
74 run_lighthouse "https://www.cnet.com/" "off_cnet"
75 run_lighthouse "https://www.independent.co.uk/" "off_ind"
76
77
78 echo ""
79 echo "=====
80 echo "    PHASE 4: Restoring Gateway ON"
81 echo "=====
82 ./toggle.sh
83
84 echo ""
85 echo "=====

```



```

86 echo " BENCHMARKS COMPLETE!"
87 echo "You can scroll up to compare Phase 1 (AdGuard ON) vs Phase 3 (AdGuard OFF)."
```

22 benchmarks/test_load.py

```

1 import urllib.request
2 import urllib.error
3 import time
4 from html.parser import HTMLParser
5 import concurrent.futures
6
7 class ResourceParser(HTMLParser):
8     def __init__(self):
9         super().__init__()
10        self.urls = set()
11
12    def handle_starttag(self, tag, attrs):
13        attrs = dict(attrs)
14        url = None
15        if tag in ['img', 'script']:
16            url = attrs.get('src')
17        elif tag == 'link' and attrs.get('rel') == 'stylesheet':
18            url = attrs.get('href')
19
20        if url and url.startswith('http'):
21            self.urls.add(url)
22
23    def fetch_url(url):
24        try:
25            req = urllib.request.Request(url, headers={'User-Agent': 'Mozilla/5.0'})
26            urllib.request.urlopen(req, timeout=3)
27            return True
28        except Exception:
29            return False
30
31    def measure_url(target_url):
32        print(f"\nTesting {target_url} Load...")
33        start_total = time.time()
34
35        # Fetch main HTML
36        try:
37            req = urllib.request.Request(target_url, headers={'User-Agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36'})
38            response = urllib.request.urlopen(req, timeout=5)
39            html = response.read().decode('utf-8', errors='ignore')
40        except Exception as e:
41            print(f"Failed to fetch {target_url}: {e}")
42            return
43
44        parser = ResourceParser()
45        parser.feed(html)
46        urls = list(parser.urls)
47        print(f"Found {len(urls)} subresources to fetch (scripts, images, css)...")
48
49        # Concurrently fetch subresources
50        success_count = 0
51        fail_count = 0
52        max_threads = min(64, max(1, len(urls)))
53        with concurrent.futures.ThreadPoolExecutor(max_workers=max_threads) as executor:
54            results = list(executor.map(fetch_url, urls))
55
56        success_count = sum(1 for r in results if r)
57        fail_count = len(results) - success_count
58
59        total_time = time.time() - start_total
60        print(f"Time taken: {total_time:.2f} seconds")
61        print(f"Successfully fetched: {success_count}, Failed/Blocked: {fail_count}")
62
63    urls_to_test = [
64        "https://www.dailymail.co.uk/home/index.html",
65        "https://www.cnet.com/",
66        "https://www.independent.co.uk/"
67    ]
68
69    for u in urls_to_test:
```

23 black_list.txt

```
1 ad.doubleclick.net
2 adclick.g.doubleclick.net
3 adeventtracker.spotify.com
4 adfstat.yandex.ru
5 ads-api.tiktok.com
6 ads-api.twitter.com
7 ads-fa.spotify.com
8 ads-sg.tiktok.com
9 ads.tiktok.com
10 adserver.unityads.unity3d.com
11 adsfs.oppomobile.com
12 an.facebook.com$important
13 connect.facebook.net$important
14 pixel.facebook.com$important
15 analytics.query.yahoo.com
16 analytics.s3.amazonaws.com
17 analytics.spotify.com
18 analyticsengine.s3.amazonaws.com
19 appmetrica.yandex.ru
20 audio2.spotify.com
21 bdapi-ads.realmemobile.com
22 bdapi-in-ads.realmemobile.com
23 bounceexchange.com
24 bs.serving-sys.com
25 business-api.tiktok.com
26 ck.ads.oppomobile.com
27 crashdump.spotify.com
28 data.ads.oppomobile.com
29 data.mistat.india.xiaomi.com
30 data.mistat.rus.xiaomi.com
31 data.mistat.xiaomi.com
32 desktop.spotify.com
33 doubleclick.net
34 ds.metric.spotify.com
35 gemini.yahoo.com
36 googleads.g.doubleclick.net
37 googleadservices.com
38 googlesyndication.com
39 grs.hicloud.com
40 gtm.spotify.com
41 iot-eu-logser.realme.com
42 iot-logser.realme.com
43 log.byteoversea.com
44 log.fc.yahoo.com
45 log.spotify.com
46 m.doubleclick.net
47 mediavisor.doubleclick.net
48 metrics.spotify.com
49 metrika.yandex.ru
50 omaze.com
51 pagead2.googlesyndication.com
52 securepubads.g.doubleclick.net
53 static.doubleclick.net
54 stats.g.doubleclick.net
55 tracking.rus.miui.com
56 udcms.yahoo.com
57 v.jwpcdn.com
58 weblb-wg.gslb.spotify.com
59 webview.unityads.unity3d.com
60 diagassets.apple.com
61 iphonesubmissions.apple.com
62 radarsubmissions.apple.com
63 metrics.apple.com
64 xp.apple.com
65 heads-fa.spotify.com
66 heads4.spotify.com
67 pixel.spotify.com
68 segs.spotify.com
69 audio-fa.spotify.com
70 events.data.microsoft.com
71 datarouter-prod.ak.epicgames.com
72 nel.cloudflare.com
```

```

73 eu-mobile.events.data.microsoft.com
74 stocks-analytics-events.apple.com
75 settings-win.data.microsoft.com
76 choice.microsoft.com
77 watson.live.com$important
78 gdmf.apple.com
79 deviceenrollment.apple.com
80 deviceservices-external.apple.com
81 gpm.samsungqbe.com
82 daisy.ubuntu.com
83 popcon.ubuntu.com
84 popcon.debian.org
85 mdm-enrollment.googleapis.com$important
86 device-provisioning.googleapis.com$important

```

24 cloud-init.yaml

```

1 #cloud-config
2 # Generic Cloud-Init Bootstrap Script for Remote Exit Nodes (Ubuntu/Debian)
3 # Paste this into the "User Data" or "Initialization Script" section when creating a VPS.
4
5 package_update: true
6 package_upgrade: true
7
8 packages:
9   - apt-transport-https
10  - ca-certificates
11  - curl
12  - gnupg
13  - lsb-release
14  - git
15  - python3
16  - python3-pip
17
18 runcmd:
19   # 1. Install Docker natively
20   - curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-
      archive-keyring.gpg
21   - echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]
      https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/
      docker.list > /dev/null
22   - sudo apt-get update
23   - sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
24
25   # 2. Add default user (ubuntu) to docker group
26   - usermod -aG docker ubuntu || true
27   - usermod -aG docker debian || true
28
29   # 3. Create the installation directory
30   - mkdir -p /opt/nullexit
31   - chown -R ubuntu:ubuntu /opt/nullexit || chown -R debian:debian /opt/nullexit
32
33   # 4. Enable IPv4 and IPv6 forwarding for VPN routing
34   - echo "net.ipv4.ip_forward=1" >> /etc/sysctl.d/99-tailscale.conf
35   - echo "net.ipv6.conf.all.forwarding=1" >> /etc/sysctl.d/99-tailscale.conf
36   - sysctl -p /etc/sysctl.d/99-tailscale.conf
37
38 final_message: "The remote exit node has been fully bootstrapped. SSH in, clone the repo to /opt/nullexit, add
      your .env, and run ./setup-linux.sh!"

```

25 docker/tor/Dockerfile

```

1 FROM debian:bookworm-slim
2
3 RUN apt-get update && \
4     apt-get install -y tor obfs4proxy gosu bash && \
5     rm -rf /var/lib/apt/lists/*
6
7 COPY entrypoint.sh /entrypoint.sh
8 RUN chmod +x /entrypoint.sh
9
10 ENTRYPOINT ["/entrypoint.sh"]

```

26 docker/tor/entrypoint.sh

```
1 #!/bin/bash
2 set -e
3
4 TORRC="/etc/tor/torrc"
5 mkdir -p /etc/tor /var/lib/tor
6
7 # Base configuration: SOCKS 9050, Control 9051, TransPort 9040, DNSPort 5353 all bound to
8 # 0.0.0.0 (required for Docker/Colima host port-mapping). The ControlPort is secured via a
9 # dynamic HashedControlPassword (added below when TOR_PASSWORD is set); see devref.md 15.10.2.
10 cat <<EOF > $TORRC
11 SocksPort 0.0.0.0:9050
12 ControlPort 0.0.0.0:9051
13 CookieAuthentication 0
14 Log notice stdout
15 DataDirectory /var/lib/tor
16
17 # Transparent proxying & DNS resolution for .onion mapping
18 TransPort 0.0.0.0:9040
19 DNSPort 0.0.0.0:5353
20 AutomapHostsOnResolve 1
21 VirtualAddrNetworkIPv4 198.18.0.0/15
22 EOF
23
24 if [ -n "$TOR_PASSWORD" ]; then
25     HASH=$(tor --hash-password "$TOR_PASSWORD" | tail -n 1)
26     echo "HashedControlPassword $HASH" >> $TORRC
27 fi
28
29 if [ -n "$TOR_EXCLUDE_EXIT_NODES" ]; then
30     echo "ExcludeExitNodes $TOR_EXCLUDE_EXIT_NODES" >> $TORRC
31     echo "StrictNodes 1" >> $TORRC
32 fi
33
34 if [ "${TOR_USE_BRIDGES:-false}" = "true" ]; then
35     BRIDGE_FILE="${TOR_BRIDGE_LINES_FILE:-/tor-bridges.txt}"
36
37     if ! grep -vE "^[[:space:]]*#" "$BRIDGE_FILE" | grep -q '^[[:space:]]'; then
38         MSG="[$(date '+%Y-%m-%d %H:%M:%S')] [CRITICAL] [Tor Proxy] TOR_USE_BRIDGES is enabled, but no bridge
39         lines found at $BRIDGE_FILE. Proxy startup ABORTED. Get bridges from https://bridges.torproject.org."
40         echo "$MSG"
41         if [ -w "/output.log" ]; then
42             echo "$MSG" >> /output.log
43         fi
44         # Sleep indefinitely to prevent docker-compose 'unless-stopped' from triggering a crash loop
45         exec sleep infinity
46     fi
47
48     echo "UseBridges 1" >> $TORRC
49     echo "ClientTransportPlugin obfs4 exec /usr/bin/obfs4proxy" >> $TORRC
50
51     while IFS= read -r line; do
52         # Ignore comments and empty lines
53         [[ -z "$line" || "$line" == \#* ]] && continue
54         echo "Bridge $line" >> $TORRC
55     done < "$BRIDGE_FILE"
56 fi
57
58 # Debian tor package uses 'debian-tor' user
59 chown -R debian-tor:debian-tor /var/lib/tor /etc/tor
60
61 exec gosu debian-tor /usr/bin/tor -f $TORRC
```

27 launchd/com.nullexit.noise.plist

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
3 <plist version="1.0">
4 <dict>
5     <key>Label</key>
6     <string>com.nullexit.noise</string>
7
8     <!-- Same install-agnostic pattern as com.nullexit.wake-recovery: WorkingDirectory
9         is _NULLEXIT_HOME_ (substitute your absolute install root at install time
```

```

10     via sed), and the program arg is scripts/noise.sh RELATIVE to it, so
11     BASH_SOURCE still resolves SCRIPT_DIR correctly. -->
12     <key>WorkingDirectory</key>
13     <string>__NULLEXIT_HOME__</string>
14
15     <!-- `__boot`: governed SOLELY by NOISE_ENABLED in .env. It exits 0 cleanly when
16     disabled (so KeepAlive never tight-loops) and runs the padding loop in the
17     foreground when enabled. Loading this plist adds NO new option it just means
18     "auto-start cover traffic whenever NOISE_ENABLED=true". -->
19     <key>ProgramArguments</key>
20     <array>
21         <string>/bin/bash</string>
22         <string>scripts/noise.sh</string>
23         <string>__boot</string>
24     </array>
25
26     <!-- Start immediately on launchctl load and re-run at every login/boot. -->
27     <key>RunAtLoad</key>
28     <true/>
29
30     <!-- Same restart policy rationale as wake-recovery:
31     SuccessfulExit=false a clean exit 0 (the disabled-idle path) is NOT
32                          relaunched, so a disabled job doesn't spin.
33     Crashed=false       a SIGTERM from `launchctl bootout` is not treated as
34                          a relaunch trigger, so stopping is clean and loop-free.
35     Net effect: load once padding runs at boot when enabled; bootout stops it. -->
36     <key>KeepAlive</key>
37     <dict>
38         <key>SuccessfulExit</key>
39         <false/>
40         <key>Crashed</key>
41         <false/>
42     </dict>
43
44     <!-- Cover traffic must not be suspended by App Nap when the user is idle. -->
45     <key>ProcessType</key>
46     <string>Background</string>
47
48     <key>StandardOutPath</key>
49     <string>/tmp/nullexit-noise.out.log</string>
50     <key>StandardErrorPath</key>
51     <string>/tmp/nullexit-noise.err.log</string>
52 </dict>
53 </plist>

```

28 launchd/com.nullexit.taildrop-watch.plist

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
3 <plist version="1.0">
4 <dict>
5     <key>Label</key>
6     <string>com.nullexit.taildrop-watch</string>
7
8     <!-- Same install-agnostic pattern as com.nullexit.wake-recovery /
9     com.nullexit.noise: WorkingDirectory is __NULLEXIT_HOME__ (substituted
10     with the absolute install root at install time), and the program arg
11     is scripts/taildrop-watch.sh RELATIVE to it, so BASH_SOURCE inside the
12     script still resolves correctly. -->
13     <key>WorkingDirectory</key>
14     <string>__NULLEXIT_HOME__</string>
15
16     <key>ProgramArguments</key>
17     <array>
18         <string>/bin/bash</string>
19         <string>scripts/taildrop-watch.sh</string>
20     </array>
21
22     <key>RunAtLoad</key>
23     <true/>
24
25     <!-- Unlike noise/wake-recovery there's no "disabled, sit idle" exit path
26     here `tailscale file get --loop` only returns on a real failure
27     (tailscaled restarting, socket briefly gone), so always respawning is
28     correct. launchd's built-in crash throttling covers a persistent
29     failure without spinning. -->

```

```

30     <key>KeepAlive</key>
31     <true/>
32
33     <key>ProcessType</key>
34     <string>Background</string>
35
36     <key>StandardOutPath</key>
37     <string>/tmp/nullexit-taildrop.out.log</string>
38     <key>StandardErrorPath</key>
39     <string>/tmp/nullexit-taildrop.err.log</string>
40 </dict>
41 </plist>

```

29 launchd/com.nullexit.wake-recovery.plist

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
3 <plist version="1.0">
4 <dict>
5     <key>Label</key>
6     <string>com.nullexit.wake-recovery</string>
7
8     <!-- WorkingDirectory + relative program arg keeps this plist install-agnostic.
9          The source-of-truth for the install path is __NULLEXIT_HOME__, which
10         setup.sh or the user must substitute with their absolute nullexit
11         install root at install time. Program arg is `scripts/watcher.sh`
12         RELATIVE to WorkingDirectory, so launchd's evaluated cwd IS the
13         install root and BASH_SOURCE inside scripts/watcher.sh still
14         resolves to the real <install>/scripts/watcher.sh, preserving the
15         SCRIPT_DIR pattern. After install-time `sed -i 's|__NULLEXIT_HOME__|<abs_path>|'`,
16         the plist is portable to any clone location. -->
17     <key>WorkingDirectory</key>
18     <string>__NULLEXIT_HOME__</string>
19
20     <key>ProgramArguments</key>
21     <array>
22         <string>/bin/bash</string>
23         <string>scripts/watcher.sh</string>
24     </array>
25
26     <!-- Start the daemon immediately on launchctl load (instead of waiting
27          for the next user login). Combined with keepAlive below this means
28          once installed: load once, runs forever across logouts/reboots. -->
29     <key>RunAtLoad</key>
30     <true/>
31
32     <!-- Restart policy:
33          SuccessfulExit=false    SIGTERM / clean exit does NOT relaunch (so
34                                `launchctl bootout` cleanly stops, not loops).
35          Crashed=false          HARD crashes ALSO do NOT relaunch. We've seen
36                                that repeated sleep/wake cycles on macOS
37                                Sonoma+ accumulate orphan watcher.sh PIDs
38                                because SIGCONT/resume doesn't reliably kill
39                                the prior instance. The watcher.sh script
40                                now enforces single-instance via a manual
41                                PID-file and process-identity check at the
42                                top, so a re-launch on crash is
43                                actively dangerous (it would skip the lock
44                                and exit; better to surface the crash in
45                                logs than to silently 0-process). -->
46     <key>KeepAlive</key>
47     <dict>
48         <key>SuccessfulExit</key>
49         <false/>
50         <key>Crashed</key>
51         <false/>
52     </dict>
53
54     <!-- Background hint to App Nap so it doesn't get suspended when
55          the user is inactive. We're the entire reason this LaunchAgent
56          needs to NOT nap. -->
57     <key>ProcessType</key>
58     <string>Background</string>
59
60     <!-- launchd-level stdout/stderr (the bash launcher shell). The script
61          itself writes to /tmp/nullexit-watcher.log via exec >>. These

```

```

62         separate channels help us tell "launchd crashed" from "the bash
63         wrapper piped to recover.sh errored". -->
64         <key>StandardOutPath</key>
65         <string>/tmp/nullexit-watcher.out.log</string>
66         <key>StandardErrorPath</key>
67         <string>/tmp/nullexit-watcher.err.log</string>
68     </dict>
69 </plist>

```

30 post-rules.txt

```

1 # NOTE: the REJECT rules below (DNS-over-TLS, QUIC) must be added BEFORE the blanket
2 # FORWARD ACCEPT rules. iptables stops at the first match, so if the ACCEPT rules ran
3 # first, all tailscale0 traffic (including port 853/443) would already be accepted and
4 # these REJECT rules would never be reached they'd sit dead with a permanent 0-packet counter.
5
6 # Block DNS-over-TLS (Port 853) to force Android Private DNS to fallback to Port 53
7 iptables -A FORWARD -i tailscale0 -p tcp --dport 853 -j REJECT --reject-with tcp-reset
8 iptables -A FORWARD -i tailscale0 -p udp --dport 853 -j REJECT
9
10 # Block QUIC (UDP 443) to force Facebook/Google to fallback to TCP.
11 # UDP bypasses TCP MSS clamping, causing fragmentation. Over weak Wi-Fi (far from gateway),
12 # dropping a single fragment destroys the entire packet, stalling image downloads.
13 iptables -A FORWARD -i tailscale0 -p udp --dport 443 -j REJECT
14
15 iptables -A FORWARD -i tailscale0 -o tun0 -j ACCEPT
16 iptables -A FORWARD -i tun0 -o tailscale0 -j ACCEPT
17 iptables -A INPUT -i tailscale0 -j ACCEPT
18 iptables -A INPUT -i eth0 -p udp --dport 41642 -j ACCEPT
19 iptables -t nat -A POSTROUTING -o tun0 -j MASQUERADE
20 # Clamp MSS to 1120. With Tailscale overhead on top of WARP (each WireGuard encapsulation adds ~60 bytes),
21 # the safe MSS for double-tunneled traffic is 1120 to prevent strict-MSS endpoints from stalling.
22 # NOTE: this 1120 integer is rewritten at boot by toggle.sh from GATEWAY_MSS in .env
23 # (Gluetun's parser can't interpolate variables) change GATEWAY_MSS in .env, not this line.
24 iptables -t mangle -A POSTROUTING -p tcp --tcp-flags SYN,RST SYN -j TCPMSS --set-mss 1120
25
26 iptables -t nat -A PREROUTING -i tailscale0 -p udp --dport 53 -j REDIRECT --to-port 5335
27 iptables -t nat -A PREROUTING -i tailscale0 -p tcp --dport 53 -j REDIRECT --to-port 5335
28
29 # Also redirect DNS from Docker bridge interface so the host can reach AdGuard via localhost:53
30 iptables -t nat -A PREROUTING -i eth0 -p udp --dport 53 -j REDIRECT --to-port 5335
31 iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 53 -j REDIRECT --to-port 5335
32
33 # Block IPv6 exit-node traffic to prevent leakage (Gluetun/WARP is IPv4-only)
34 ip6tables -A FORWARD -i tailscale0 -j DROP

```

31 scripts/Dockerfile.routing-fix

```

1 FROM golang:1.22-alpine AS builder
2 WORKDIR /app
3 COPY logger/ ./logger/
4 RUN cd logger && go build -o logger main.go
5
6 FROM alpine:3.20
7 RUN apk add --no-cache iptables iproute2 curl ipset
8 COPY --from=builder /app/logger/logger /usr/local/bin/nullexit-logger
9 COPY routing-fix.sh /routing-fix.sh
10 CMD ["sh", "/routing-fix.sh"]

```

32 scripts/Dockerfile.rule-compiler

```

1 FROM golang:1.22-alpine AS builder
2 WORKDIR /app
3 COPY rule-compiler/ ./rule-compiler/
4 RUN cd rule-compiler && go build -o sync-rules main.go
5
6 FROM alpine:3.20
7 COPY --from=builder /app/rule-compiler/sync-rules /usr/local/bin/sync-rules

```



```

8 WORKDIR /app
9 CMD ["sync-rules"]

```

33 scripts/Toggle-Gateway.bat

```

1 <# :
2 @echo off
3 powershell -NoProfile -ExecutionPolicy Bypass -Command "Invoke-Expression ([System.IO.File]::ReadAllText('%-f0
4 ') -replace '^(?s).*?#\s*>\s*\r?\n','')
5 exit /b
6 #>
7 # Toggle-Gateway.bat (Hybrid Batch-PowerShell Toggler) nullexit Gateway Toggler for Windows
8 # Mirrors the sophisticated error handling, checks, and automation of toggle.sh
9
10 # 1. Administrator Check & Elevation
11 $isAdmin = ([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([
12 Security.Principal.WindowsBuiltInRole]::Administrator)
13 if (-not $isAdmin) {
14 Write-Host "Requesting Administrator privileges..." -ForegroundColor Yellow
15 Start-Process PowerShell -ArgumentList "-NoProfile -ExecutionPolicy Bypass -Command `Invoke-Expression ([
16 System.IO.File]::ReadAllText('%PSCommandPath') -replace '^(?s).*?#\s*>\s*\r?\n','')`"" -Verb RunAs
17 Exit
18 }
19
20 # Set console output encoding to UTF8 for clean symbols
21 [Console]::OutputEncoding = [System.Text.Encoding]::UTF8
22 Clear-Host
23
24 Write-Host "" -ForegroundColor Green
25 Write-Host " nullexit Windows Toggler " -ForegroundColor Green
26 Write-Host "" -ForegroundColor Green
27 Write-Host ""
28
29 # 2. Helpers
30 function Reset-HostDNS {
31 Write-Host "Restoring default DNS (DHCP/DHCP-assigned DNS)..." -ForegroundColor Cyan
32 $activeAdapters = Get-NetAdapter | Where-Object { $_.Status -eq 'Up' }
33 foreach ($adapter in $activeAdapters) {
34 Write-Host " -> Resetting DNS on adapter: $($adapter.Name)" -ForegroundColor Gray
35 Set-DnsClientServerAddress -InterfaceIndex $adapter.InterfaceIndex -ResetServerAddresses -ErrorAction
36 SilentlyContinue
37 }
38 }
39
40 function Hijack-HostDNS ($ip) {
41 Write-Host "Hijacking DNS to route through AdGuard ($ip)..." -ForegroundColor Yellow
42 $activeAdapters = Get-NetAdapter | Where-Object { $_.Status -eq 'Up' }
43 foreach ($adapter in $activeAdapters) {
44 Write-Host " -> Pointing DNS to $ip on adapter: $($adapter.Name)" -ForegroundColor Gray
45 Set-DnsClientServerAddress -InterfaceIndex $adapter.InterfaceIndex -ServerAddresses $ip -ErrorAction
46 SilentlyContinue
47 }
48 }
49
50 function Get-HostTailscalePath {
51 $paths = @(
52 "$env:ProgramFiles\Tailscale\tailscale.exe",
53 "$env:ProgramFiles (x86)\Tailscale\tailscale.exe",
54 "tailscale.exe"
55 )
56 foreach ($path in $paths) {
57 if (Test-Path $path) { return $path }
58 }
59 return $null
60 }
61
62 # 3. Detect State
63 # Prevent deadlocks by resetting DNS to default temporarily during checks
64 Reset-HostDNS
65
66 Write-Host "Checking Docker daemon status..." -ForegroundColor Cyan
67 & docker info >$null 2>&1
68 if ($LASTEXITCODE -ne 0) {
69 Write-Host "Docker is not running. Starting Docker Desktop..." -ForegroundColor Yellow
70 $dockerPath = "C:\Program Files\Docker\Docker\Desktop.exe"
71 if (Test-Path $dockerPath) {

```

```

67     Start-Process $dockerPath
68     Write-Host "Waiting for Docker daemon to become responsive..." -ForegroundColor Gray
69     $timeout = 60
70     while ($timeout -gt 0) {
71         Start-Sleep -Seconds 2
72         & docker info >$null 2>&1
73         if ($LASTEXITCODE -eq 0) {
74             Write-Host "Docker is ready!" -ForegroundColor Green
75             break
76         }
77         $timeout -= 2
78     }
79     if ($timeout -le 0) {
80         Write-Error "Docker failed to start in time. Aborting."
81         Exit 1
82     }
83 } else {
84     Write-Error "Docker Desktop executable not found at standard path. Please start Docker manually."
85     Exit 1
86 }
87 }
88
89 # Detect if gateway is already running
90 $runningWarp = docker compose ps --status running --format json | ConvertFrom-Json -ErrorAction
    SilentlyContinue | Where-Object { $_.Service -eq "warp" }
91 $isGatewayActive = ($null -ne $runningWarp)
92
93 # 4. Execution
94 if ($isGatewayActive) {
95     # STOP PATH
96     Write-Host "Gateway is currently RUNNING. Disabling now..." -ForegroundColor Yellow
97     Write-Host ""
98
99     # Disconnect host Tailscale from exit node if installed
100    $tsCli = Get-HostTailscalePath
101    if ($null -ne $tsCli) {
102        Write-Host "Disconnecting host Tailscale from exit node..." -ForegroundColor Cyan
103        & $tsCli up --exit-node= --accept-dns=true >$null 2>&1
104    }
105
106    Write-Host "Stopping Docker containers..." -ForegroundColor Cyan
107    docker compose down -t 5
108
109    Reset-HostDNS
110    Write-Host "Gateway has been successfully STOPPED." -ForegroundColor Green
111 } else {
112     # START PATH
113     Write-Host "Gateway is currently STOPPED. Enabling now..." -ForegroundColor Yellow
114     Write-Host ""
115
116     # Clean up corrupted AdGuardHome configurations
117     $confFile = "adguard\conf\AdGuardHome.yaml"
118     if (Test-Path $confFile) {
119         $fileSize = (Get-Item $confFile).Length
120         if ($fileSize -eq 0) {
121             Remove-Item $confFile -Force
122             Write-Host "Removed corrupted empty AdGuardHome.yaml to prevent container crash loop." -
    ForegroundColor Yellow
123         }
124     }
125
126    Write-Host "Starting Docker containers..." -ForegroundColor Cyan
127    docker compose up -d
128
129    Write-Host "Waiting for container's Tailscale connection to offer Exit Node..." -ForegroundColor Cyan
130    Write-Host " (This can take up to 60 seconds..." -ForegroundColor Gray
131
132    $elapsed = 0
133    $maxWait = 60
134    $resolvedIP = $null
135
136    while ($elapsed -lt $maxWait) {
137        $statusJson = docker compose exec -T tailscale tailscale status --json 2>$null
138        if ($null -ne $statusJson) {
139            $status = $statusJson | ConvertFrom-Json -ErrorAction SilentlyContinue
140            if ($null -ne $status -and $null -ne $status.Self) {
141                # Check if it has a valid Tailscale IP and is running
142                if ($status.Self.Online -and $status.Self.TailscaleIPs.Count -gt 0) {
143                    $resolvedIP = $status.Self.TailscaleIPs[0]
144                    break
145                }
146            }
147        }
148        $elapsed += 1
149    }

```

```

146     }
147 }
148 Start-Sleep -Seconds 2
149 $elapsed += 2
150 }
151
152 if ($null -ne $resolvedIP) {
153     Write-Host "Tailscale IP resolved: $resolvedIP" -ForegroundColor Green
154     Write-Host ""
155
156     # Configure host Tailscale client to route through this exit node
157     $tsCli = Get-HostTailscalePath
158     if ($null -ne $tsCli) {
159         Write-Host "Configuring host Tailscale to use Exit Node ($resolvedIP)..." -ForegroundColor Cyan
160         & $tsCli up --exit-node=$resolvedIP --accept-dns=false >$null 2>&1
161     }
162
163     # Hijack DNS
164     Hijack-HostDNS $resolvedIP
165     Write-Host ""
166     Write-Host "Gateway has been successfully STARTED." -ForegroundColor Green
167 } else {
168     Write-Host "Warning: Could not resolve gateway Tailscale IP. DNS hijacking skipped." -ForegroundColor
    Red
169     # Fallback to .gateway_ip if it exists
170     if (Test-Path ".gateway_ip") {
171         $fallbackIP = Get-Content ".gateway_ip" -TotalCount 1
172         if (-not [string]::IsNullOrEmpty($fallbackIP)) {
173             Write-Host "[Fallback] Using static IP from .gateway_ip: $fallbackIP" -ForegroundColor Yellow
174             Hijack-HostDNS $fallbackIP
175             Write-Host "Gateway STARTED (with static fallback IP)." -ForegroundColor Green
176         }
177     } else {
178         Write-Host "Gateway started in offline/unauthenticated state. Run setup.sh if this is a fresh setup
179         ." -ForegroundColor Yellow
180     }
181 }
182
183 Write-Host ""
184 Write-Host "Press any key to exit..."
185 [void]$Host.UI.RawUI.ReadKey("NoEcho,IncludeKeyDown")

```

34 scripts/check_circuits.py

```

1  #!/usr/bin/env python3
2  import socket
3  import os
4  import re
5
6  def query_tor(port, command, password=""):
7      try:
8          s = socket.socket()
9          s.settimeout(2)
10         s.connect(('127.0.0.1', port))
11         s.sendall(f"AUTHENTICATE \r\n{password}\r\n{command}\r\nQUIT\r\n".encode())
12         response = b""
13         while True:
14             chunk = s.recv(4096)
15             if not chunk:
16                 break
17             response += chunk
18         return response.decode()
19     except Exception as e:
20         return f"Error connecting to Tor Control Port: {e}"
21
22 def get_ports():
23     ports = {}
24     ports_file = "/tmp/nullexit-ports.env"
25     if os.path.exists(ports_file):
26         with open(ports_file, 'r') as f:
27             for line in f:
28                 if line.startswith("export "):
29                     # Split at first '=' to handle potential multiple '=' in value
30                     parts = line.replace("export ", "").strip().split("=", 1)
31                     if len(parts) == 2:

```

```

32         key, val = parts
33         try:
34             ports[key] = int(val)
35         except ValueError:
36             ports[key] = val
37     return ports
38
39 def get_password(ports):
40     # 1. Try ports.env
41     password = ports.get("TOR_PASSWORD")
42     if password:
43         return password
44
45     # 2. Try .env
46     if os.path.exists(".env"):
47         try:
48             with open(".env", "r") as f:
49                 for line in f:
50                     if line.startswith("ADGUARD_PASSWORD="):
51                         return line.split("=", 1)[1].strip().strip('\n')
52                     if line.startswith("TOR_PASSWORD="):
53                         return line.split("=", 1)[1].strip().strip('\n')
54         except Exception:
55             pass
56
57     # 3. Fallback
58     return "nullexit"
59
60 def main():
61     ports = get_ports()
62     control_port = ports.get("TOR_CONTROL_PORT", 9051)
63     password = get_password(ports)
64
65     print(f"Connecting to Tor Control Port on 127.0.0.1:{control_port}...")
66     status = query_tor(control_port, "GETINFO circuit-status", password)
67
68     if "Error" in status:
69         print(status)
70         return
71
72     print("\nActive Tor Circuits:")
73     circuits = re.findall(r"(\d+)\s+BUILT\s+(.+?)\s+PURPOSE", status)
74     if not circuits:
75         print("No active built circuits found yet. Tor might still be bootstrapping.")
76         return
77
78     for circ_id, path_str in circuits:
79         print(f"\nCircuit #{circ_id}:")
80         nodes = path_str.split(",")
81         for idx, node in enumerate(nodes):
82             fingerprint = node.split("~")[0].replace("$", "")
83             nickname = node.split("~")[1] if "~" in node else "unknown"
84
85             # Fetch node IP
86             ns_info = query_tor(control_port, f"GETINFO ns/id/{fingerprint}", password)
87             ip = "unknown"
88             for line in ns_info.split("\n"):
89                 if line.startswith("r "):
90                     parts = line.split(" ")
91                     if len(parts) >= 7:
92                         ip = parts[6]
93                         break
94
95             # Fetch country
96             country = "unknown"
97             if ip != "unknown":
98                 country_info = query_tor(control_port, f"GETINFO ip-to-country/{ip}", password)
99                 for line in country_info.split("\n"):
100                     if line.startswith("250-ip-to-country/"):
101                         country = line.split("=")[1].strip().upper()
102                         break
103
104             role = "Guard" if idx == 0 else ("Exit" if idx == len(nodes) - 1 else "Middle")
105             print(f"    [{role}] {nickname} ({ip}) - Country: {country}")
106
107 if __name__ == "__main__":
108     main()

```

35 scripts/device-scramble.sh

```
1 #!/bin/bash
2 # scripts/device-scramble.sh present a random, anonymous DEVICE identity on Wi-Fi.
3 #
4 # Randomizes the identifiers a network/observer uses to fingerprint your device:
5 #   hostname      ComputerName / LocalHostName / HostName (the DHCP hostname +
6 #                 the mDNS/Bonjour ".local" name broadcast on the LAN)
7 #   MAC (opt)     the layer-2 device address
8 #
9 # WHERE IT HELPS: open / no-auth Wi-Fi (caf, hotel, airport) where these IDs are
10 # the ONLY handle on you, and against co-located sniffers on any network. It does
11 # NOT hide you from a network you 802.1X-authenticate to that login ties every
12 # session to your account regardless of MAC (see the FaceTime/threat-model notes).
13 #
14 # SAFETY: `scramble` saves your real identity first; `restore` puts it back.
15 # `test-mac` is FAIL-SAFE it always restores your original MAC at the end, and
16 # detects Apple-Silicon spoof-blocking and MAC-gated networks without stranding you.
17 #
18 # Usage (privileged actions need sudo):
19 #   bash scripts/device-scramble.sh status           # show current + saved identity (no sudo)
20 #   sudo bash scripts/device-scramble.sh scramble    # random hostname now (add --mac to also rotate MAC)
21 #   sudo bash scripts/device-scramble.sh restore     # put your real identity back
22 #   sudo bash scripts/device-scramble.sh test-mac    # can this network take a random MAC? (auto-reverts)
23 #   bash scripts/device-scramble.sh --dry-run        # print the random values it would use (no sudo, no
24 #   change)
25
26 set -uo pipefail
27 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
28 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
29 cd "$PROJECT_ROOT" || exit 1
30 IFACE=en0
31 SAVE="$PROJECT_ROOT/.device-identity.orig" # gitignored
32 WARP_INHIBIT="/tmp/nullexit-warp-inhibit.marker" # WARP Watcher skips shutdown while this exists
33
34 rand_mac() { # locally-administered (0x02) + unicast (clear 0x01) first octet
35     printf '%02x:%02x:%02x:%02x:%02x:%02x' \
36         $(( (RANDOM & 0xFC) | 0x02 )) $((RANDOM&255)) $((RANDOM&255)) $((RANDOM&255)) $((RANDOM&255)) $((RANDOM
37         &255))
38 }
39 rand_host() { printf 'MacBook-%04X' $((RANDOM & 0xFFFF)); } # Apple-ish, blends in
40
41 cur_mac() { ifconfig "$IFACE" ether 2>/dev/null | awk '/ether/{print $2}'; }
42 cur_ip() { ipconfig getifaddr "$IFACE" 2>/dev/null; }
43 need_root(){ [ "$(id -u)" = 0 ] || { echo "device-scramble: this needs root run with: sudo bash $0 $" >&2;
44     exit 1; }; }
45
46 cmd_status() {
47     echo "current:"
48     echo "  ComputerName : $(scutil --get ComputerName 2>/dev/null)"
49     echo "  LocalHostName : $(scutil --get LocalHostName 2>/dev/null)"
50     echo "  HostName      : $(scutil --get HostName 2>/dev/null || echo '(unset)')"
51     echo "  MAC ($IFACE)   : $(cur_mac)   IP: $(cur_ip)"
52     if [ -f "$SAVE" ]; then echo; echo "saved original (restore target):"; sed 's/~ / /' "$SAVE"; fi
53 }
54
55 cmd_dry_run() {
56     echo "dry-run would set (no changes made, no sudo needed):"
57     echo "  hostname $(rand_host)"
58     echo "  MAC      $(rand_mac) (locally-administered, unicast)"
59 }
60
61 save_original() {
62     [ -f "$SAVE" ] && return 0 # don't overwrite a real original with an already-scrambled one
63     {
64         echo "ComputerName=$(scutil --get ComputerName 2>/dev/null)"
65         echo "LocalHostName=$(scutil --get LocalHostName 2>/dev/null)"
66         echo "HostName=$(scutil --get HostName 2>/dev/null)"
67         echo "MAC=$(cur_mac)"
68     } > "$SAVE"
69 }
70
71 cmd_scramble() {
72     need_root scramble
73     save_original
74     local h; h=$(rand_host)
75     echo "device-scramble: hostname $h"
76     scutil --set ComputerName "$h"
77     scutil --set LocalHostName "$h"
78 }
```

```

75 scutil --set HostName "$h"
76 dscacheutil -flushcache 2>/dev/null || true
77 if [ "${1:-}" = "--mac" ]; then
78     local m; m=$(rand_mac)
79     echo "device-scramble: MAC $m (disassociate set reconnect)"
80     networksetup -setairportpower "$IFACE" off; sleep 3 # can't set MAC while associated
81     ifconfig "$IFACE" ether "$m" 2>/dev/null
82     [ "$(cur_mac)" = "$m" ] || echo " [!] MAC did not change this driver refuses ifconfig spoofing on this
hardware"
83     networksetup -setairportpower "$IFACE" on
84 fi
85 echo "done. Restore your real identity with: sudo bash $0 restore"
86 }
87
88 cmd_restore() {
89     need_root restore
90     [ -f "$SAVE" ] || { echo "device-scramble: no saved identity at $SAVE nothing to restore."; exit 1; }
91     local cn lh hn mac
92     cn=$(sed -n 's/~ComputerName=//p' "$SAVE"); lh=$(sed -n 's/~LocalHostName=//p' "$SAVE")
93     hn=$(sed -n 's/~HostName=//p' "$SAVE"); mac=$(sed -n 's/~MAC=//p' "$SAVE")
94     [ -n "$cn" ] && scutil --set ComputerName "$cn"
95     [ -n "$lh" ] && scutil --set LocalHostName "$lh"
96     [ -n "$hn" ] && scutil --set HostName "$hn"
97     if [ -n "$mac" ] && [ "$(cur_mac)" != "$mac" ]; then
98         networksetup -setairportpower "$IFACE" off; sleep 3 # disassociate before setting MAC
99         ifconfig "$IFACE" ether "$mac" 2>/dev/null
100         networksetup -setairportpower "$IFACE" on
101     fi
102     rm -f "$SAVE"
103     echo "restored: ComputerName=$cn MAC=$(cur_mac)"
104 }
105
106 # The safe experiment: does THIS network accept a random MAC? Always reverts.
107 # CRITICAL: macOS refuses to change the MAC while the interface is ASSOCIATED to
108 # an SSID, so we disassociate (Wi-Fi radio off) BEFORE setting it, then reconnect.
109 # (This is the fix for the naive first attempt that set it while connected.)
110 cmd_test_mac() {
111     need_root test-mac
112     # Protect the running gateway: inhibit the WARP Watcher so the brief Wi-Fi blip
113     # isn't counted as tunnel death (which would auto-shutdown a healthy gateway).
114     # Only manage the marker if we created it don't clobber recover.sh's.
115     local owned_inhibit=0
116     if [ ! -f "$WARP_INHIBIT" ]; then touch "$WARP_INHIBIT" && owned_inhibit=1; fi
117     trap '[ "$owned_inhibit" = 1 ] && rm -f "$WARP_INHIBIT" EXIT INT TERM
118     echo "gateway guard: WARP Watcher inhibited during the test (containers stay up regardless)."'
119     local orig; orig=$(cur_mac)
120     [ -n "$orig" ] || { echo "no IFACE MAC found"; exit 1; }
121     echo "$orig" > /tmp/nullexit-mac.orig
122     local rnd; rnd=$(rand_mac)
123     echo "original MAC: $orig trying random: $rnd"
124     echo "step 1: disassociate (Wi-Fi off) you can't set the MAC while connected"
125     networksetup -setairportpower "$IFACE" off; sleep 3
126     echo "step 2: set MAC while disassociated"
127     ifconfig "$IFACE" ether "$rnd" 2>/dev/null
128     local after; after=$(cur_mac)
129     if [ "$after" != "$rnd" ]; then
130         echo "RESULT: MAC still $after even when disassociated this driver refuses ifconfig spoofing on neo."
131         echo " (Only Apple's native 'Private Wi-Fi Address' or a USB Wi-Fi adapter would work.)"
132         networksetup -setairportpower "$IFACE" on; exit 0
133     fi
134     echo " MAC CHANGED to $after with disassociate-first. Reconnecting to test the network"
135     networksetup -setairportpower "$IFACE" on
136     local ip=""; for _ in $(seq 1 15); do sleep 2; ip=$(cur_ip); [ -n "$ip" ] && break; done
137     if [ -n "$ip" ]; then
138         echo "RESULT: SUCCESS random MAC works AND got IP $ip. neo can spoof + this network accepts it."
139     else
140         echo "RESULT: MAC spoof works, but no IP in ~30s this network gates on your registered MAC (or 802.1X re
-auth needed)."

```

```

154 restore) cmd_restore ;;
155 test-mac) cmd_test_mac ;;
156 --help|-h) sed -n '2,26p' "$0" ;;
157 *) echo "Unknown: $1 (status|scramble [--mac]|restore|test-mac|--dry-run)" >&2; exit 2 ;;
158 esac

```

36 scripts/diagnose-host-leak.sh

```

1 #!/bin/bash
2 # scripts/diagnose-host-leak.sh nullexit host-routing diagnostic
3 #
4 # Symptom this addresses: tests like https://www.whatismyip.com on the
5 # *HOST MAC* return the underlying physical ISP/ASN
6 # local ISP / campus network") even though the nullexit gateway is active and remote
7 # tailnet clients correctly egress via Cloudflare WARP. The container and
8 # the gateway itself are healthy only the host's own routing is leaking.
9 #
10 # This script runs 7 checks in sequence, classifies the result into ONE
11 # of the three known leak scenarios, writes the full report to a
12 # timestamped file in the project root, and prints a verdict + a
13 # ready-to-run fix command on stdout. Pass --fix to apply the matching
14 # remediation in the same invocation and re-verify egress afterwards.
15 # See devref.md 10.30 for the deep-dive rationale.
16 #
17 # Usage:
18 #   bash scripts/diagnose-host-leak.sh           # diagnose + write report
19 #   bash scripts/diagnose-host-leak.sh --fix      # diagnose + fix + re-verify
20 #   bash scripts/diagnose-host-leak.sh --watch    # full baseline then loop checks every 60s
21 #   bash scripts/diagnose-host-leak.sh --watch 30 # same, with custom interval (seconds)
22 #   bash scripts/diagnose-host-leak.sh --help    # this message
23 #
24 # Multiple runs produce timestamped files (one per run) so historical
25 # reports can be diffed. The newest file is the most recent.
26
27 set -uo pipefail
28 # NOTE: errexit (`set -e`) is intentionally NOT enabled. Several checks
29 # below are EXPECTED to fail and that's diagnostic data (e.g. `tailscale
30 # status` returns non-zero when the daemon is wedged). Every command
31 # pipes to a helper that records the exit code without killing the script.
32
33 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
34 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
35 TIMESTAMP="$(date -u +%Y%m%dT%H%M%SZ)"
36 source "$SCRIPT_DIR/common.sh"
37 LOG_FILE="$PROJECT_ROOT/output.log"
38 REPORT_FILE="$PROJECT_ROOT/host-leak-diagnostic-$TIMESTAMP.txt"
39
40 # Argument parsing
41 DO_FIX=false
42 DO_WATCH=false
43 WATCH_INTERVAL=60
44 WATCH_LOG="$PROJECT_ROOT/host-leak-watch.log"
45 case "${1:-}" in
46   --fix) DO_FIX=true ;;
47   --watch)
48     DO_WATCH=true
49     # Optional second argument: custom interval in seconds
50     if [ -n "${2:-}" ] && [[ "$2" =~ ^[0-9]+$ ]]; then
51       WATCH_INTERVAL="$2"
52     fi
53   ;;
54   --help|-h)
55     sed -n '2,31p' "$0"
56     exit 0
57   ;;
58   *)
59     : # default: diagnose only
60   ;;
61 *)
62   echo "Unknown argument: $1" >&2
63   echo "Run with --help for usage." >&2
64   exit 2
65 ;;
66 esac
67
68 # Colours (auto-disabled when stdout is not a tty, e.g. piped)

```



```

69 if [ -t 1 ]; then
70     RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
71     BOLD='\033[1m'; NC='\033[0m'
72 else
73     RED=''; GREEN=''; YELLOW=''; BOLD=''; NC=''
74 fi
75
76 # Output helpers write to BOTH stdout and the report file
77 exec 3>> "$REPORT_FILE" || {
78     printf '\n[FATAL] cannot open %s for write\n' "$REPORT_FILE" >&2
79     exit 1
80 }
81 section() {
82     local title="$1"
83     printf '\n%b %s %b\n' "$BOLD" "$title" "$NC" >&3
84     printf '\n%b %s %b\n' "$BOLD" "$title" "$NC"
85 }
86 line() { printf ' %b\n' "$1" >&3; printf ' %b\n' "$1"; }
87 ok() { line "${GREEN}${NC} $1"; }
88 warn() { line "${YELLOW}${NC} $1"; }
89 fail() { line "${RED}${NC} $1"; }
90 note() { line "(no ${1})"; }
91
92 # Watch-mode header (only shown when entering watch loop)
93 if [ "$DO_WATCH" = "true" ]; then
94     echo ""
95     echo ""
96     echo " n u l l e x i t   w a t c h   m o d e "
97     echo ""
98     echo " Interval:          ${WATCH_INTERVAL}s"
99     echo " Watch log:         $WATCH_LOG"
100     echo ""
101 else
102     echo ""
103     echo " n u l l e x i t   h o s t - r o u t i n g   d i a g n o s t i c "
104     echo ""
105     echo " Timestamp (UTC): $TIMESTAMP"
106     echo " Project root:    $PROJECT_ROOT"
107     echo " Report file:     $REPORT_FILE"
108     echo " Mode:            $([ "$DO_FIX" = true ] && echo "diagnose + apply fix" || echo "diagnose only (pass --
fix to remediate)")"
109     echo ""
110     echo "" >&3
111     echo "" >&3
112     echo " n u l l e x i t   h o s t - r o u t i n g   d i a g n o s t i c " >&3
113     echo "" >&3
114     echo " Timestamp (UTC): $TIMESTAMP" >&3
115     echo " Mode:            $([ "$DO_FIX" = true ] && echo "diagnose + apply fix" || echo "diagnose only")" >&3
116 fi
117
118 # Always add Homebrew to PATH (same as toggle.sh / recover.sh)
119 export PATH="/opt/homebrew/bin:/usr/local/bin:$PATH"
120
121 # Pure-bash timeout (macOS lacks GNU `timeout`)
122 run_with_timeout() {
123     local timeout_sec="$1"
124     shift
125     if [ "$#" -eq 0 ]; then return 1; fi
126     "$@" &
127     local cmd_pid=$!
128     (
129         sleep "$timeout_sec"
130         if kill -0 "$cmd_pid" 2>> "$LOG_FILE"; then
131             kill -9 "$cmd_pid" 2>> "$LOG_FILE" || true
132         fi
133     ) >> "$LOG_FILE" 2>&1 &
134     local watcher_pid=$!
135     set +e
136     wait "$cmd_pid"
137     local exit_status=$?
138     set -uo pipefail
139     kill "$watcher_pid" 2>> "$LOG_FILE" || true
140     return "$exit_status"
141 }
142
143 #
144 # Quick-check functions (used by --watch loop; return simple status strings)
145 # Each echoes its result so the caller can capture and compare.
146 #
147
148 quick_warp() {

```

```

149 # Returns: "on", "off", "unknown"
150 local out
151 out=$(run_with_timeout 8 curl -s --max-time 6 \
152     https://www.cloudflare.com/cdn-cgi/trace 2>/dev/null || echo "")
153 local warp
154 warp=$(printf '%s' "$out" | awk -F=' ' /^warp={print $2; exit}')
155 printf '%s' "${warp:-unknown}"
156 }
157
158 quick_ipv6() {
159     # Returns: the IPv6 address if leaking, or empty string if clean
160     local out
161     out=$(run_with_timeout 7 curl -6 -s --max-time 5 ifconfig.co 2>/dev/null || echo "")
162     if [ -n "$out" ] && printf '%s' "$out" | grep -qE '[0-9a-fA-F:]+$'; then
163         printf '%s' "$out"
164     fi
165 }
166
167 quick_default_route() {
168     # Returns: "utun" if default route is via Tailscale utun*,
169     #           "wifi" if via physical Wi-Fi (192.168.x.x),
170     #           "other" otherwise
171     local route
172     route=$(netstat -rn 2>/dev/null | awk '/^default/ {print $0; exit}')
173     if [ -z "$route" ]; then
174         printf '%s' "none"
175     elif printf '%s' "$route" | grep -qE 'utun[0-9]+'; then
176         printf '%s' "utun"
177     elif printf '%s' "$route" | grep -qE '^(default|0\.0\.0\.0).*\b192\.168\.'; then
178         printf '%s' "wifi"
179     else
180         printf '%s' "other"
181     fi
182 }
183
184 # Watch loop (runs after baseline diagnostic when --watch is passed)
185 run_watch_loop() {
186     local baseline_warp="$1"
187     local baseline_default="$2"
188     local baseline_ipv6_leak="$3"
189     local interval="$4"
190
191     echo ""
192     echo ""
193     echo " Baseline established. Monitoring every ${interval}s..."
194     echo "  warp:      $baseline_warp"
195     echo "  default:    $baseline_default"
196     echo ""
197
198     # Safety: warn if interval is very short (avoids hammering APIs)
199     if [ "$interval" -lt 30 ]; then
200         printf '%b' Short interval (%ss) rate-limiting risk on external APIs%b\n' "$YELLOW" "$interval" "$NC"
201         printf ' Consider 30s or longer for production use.\n'
202     fi
203     printf '[%s] WATCH START warp=%s default=%s interval=%ss\n' \
204         "$(date -u +%Y-%m-%dT%H:%M:%SZ)" "$baseline_warp" "$baseline_default" "$interval" \
205         >> "$WATCH_LOG"
206
207     local iteration=0
208     local last_warp="$baseline_warp"
209     local last_default="$baseline_default"
210     local last_ipv6_ok=$(( [ "$baseline_ipv6_leak" = "false" ] && echo true || echo false ))
211
212     trap 'printf "\n%bWatch stopped by user. Log at: %s%b\n" "$YELLOW" "$WATCH_LOG" "$NC"; exit 0' INT TERM
213
214     while true; do
215         iteration=$((iteration + 1))
216
217         local warp_now ipv6_now default_now
218         warp_now=$(quick_warp)
219         ipv6_now=$(quick_ipv6)
220         default_now=$(quick_default_route)
221
222         local ts
223         ts=$(date -u +%H:%M:%S)
224
225         # WARP check
226         if [ "$warp_now" != "$last_warp" ]; then
227             if [ "$warp_now" = "off" ] || [ "$warp_now" = "unknown" ]; then
228                 printf '\n%b ALERT %b\n' "$RED$BOLD" "$NC"
229                 printf '%b[%s] %bWARP FLIPPED: %s %s%b YOUR IP IS LEAKING!\n' \

```

```

230     "$RED" "$ts" "$BOLD" "$last_warp" "$warp_now" "$NC"
231     printf '[%s] ALERT warp flipped %s%s\n' "$ts" "$last_warp" "$warp_now" >> "$WATCH_LOG"
232 else
233     printf '\n\b[%s] warp recovered: %s %s\b\n' \
234     "$GREEN" "$ts" "$last_warp" "$warp_now" "$NC"
235     printf '[%s] OK warp recovered %s%s\n' "$ts" "$last_warp" "$warp_now" >> "$WATCH_LOG"
236 fi
237 last_warp="$warp_now"
238 fi
239
240 # IPv6 check
241 if [ -n "$ipv6_now" ]; then
242     if [ "$last_ipv6_ok" = true ]; then
243         printf '\n\b ALERT %b\n' "$RED$BOLD" "$NC"
244         printf '%b[%s] %bIPv6 LEAK DETECTED %s\b\n' \
245         "$RED" "$ts" "$BOLD" "$ipv6_now" "$NC"
246         printf '[%s] ALERT IPv6 leak %s\n' "$ts" "$ipv6_now" >> "$WATCH_LOG"
247         last_ipv6_ok=false
248     fi
249 else
250     if [ "$last_ipv6_ok" = false ]; then
251         printf '%b[%s] IPv6 leak resolved\b\n' "$GREEN" "$ts" "$NC"
252         printf '[%s] OK IPv6 resolved\n' "$ts" >> "$WATCH_LOG"
253     fi
254     last_ipv6_ok=true
255 fi
256
257 # Default route check
258 if [ "$default_now" != "$last_default" ]; then
259     if [ "$default_now" != "utun" ]; then
260         printf '\n\b ALERT %b\n' "$RED$BOLD" "$NC"
261         printf '%b[%s] %bDEFAULT ROUTE CHANGED: %s %s\b traffic may bypass WARP!\n' \
262         "$RED" "$ts" "$BOLD" "$last_default" "$default_now" "$NC"
263         printf '[%s] ALERT route changed %s%s\n' "$ts" "$last_default" "$default_now" >> "$WATCH_LOG"
264     else
265         printf '%b[%s] default route recovered: %s %s\b\n' \
266         "$GREEN" "$ts" "$last_default" "$default_now" "$NC"
267         printf '[%s] OK route recovered %s%s\n' "$ts" "$last_default" "$default_now" >> "$WATCH_LOG"
268     fi
269     last_default="$default_now"
270 fi
271
272 # Heartbeat (every 10th iteration or if all OK)
273 if [ $((iteration % 10)) -eq 0 ]; then
274     printf '[%s] #d warp=%s ipv6=%s route=%s\n' \
275     "$ts" "$iteration" "$warp_now" "${ipv6_now:-none}" "$default_now" >> "$WATCH_LOG"
276     printf '\r\b[%s] #d warp=%s route=%s (Ctrl+C to stop)%b' \
277     "$GREEN" "$ts" "$iteration" "$warp_now" "$default_now" "$NC"
278 fi
279
280 sleep "$interval"
281 done
282 }
283
284 #
285 # Begin main diagnostic (skipped in non-watch modes if we're just watching)
286 #
287
288 ACTIVE_SERVICE=$(get_active_service)
289 ENO_SERVICE=$(networksetup -listnetworkserviceorder 2>> "$LOG_FILE" \
290     | grep -B 1 "Device: en0" | head -1 \
291     | sed -E 's/^\([0-9\*]+\)/ /' || true)
292 [ -z "$ENO_SERVICE" ] && ENO_SERVICE="Wi-Fi"
293
294 # Resolve the gateway's Tailscale IP.
295 GATEWAY_TS_IP=""
296 if [ -f "$PROJECT_ROOT/.gateway_ip" ]; then
297     GATEWAY_TS_IP=$(cat "$PROJECT_ROOT/.gateway_ip" 2>> "$LOG_FILE" \
298         | tr -d '\r' | awk 'NR==1{print $1; exit}')
299 fi
300 if [ -z "$GATEWAY_TS_IP" ] && command -v docker >> "$LOG_FILE" 2>&1 \
301     && docker compose ps --status running 2>/dev/null | grep -q 'tailscale'; then
302     GATEWAY_TS_IP=$(run_with_timeout 10 docker compose exec -T tailscale \
303         tailscale ip -4 2>> "$LOG_FILE" \
304         | tr -d '\r' | awk 'NR==1{print $1; exit}')
305 fi
306
307 # Scenario classification state
308 SCENARIO=""
309 WARP_STATUS=""
310 SOCKS5_ENABLED=false

```

```

311 TS_ON_MESH=false
312 DEFAULT_VIA_UTUN=false
313 IPV6_LEAK=false
314
315 #
316 section "1/8 Host Tailscale mesh status"
317 #
318 if ! command -v tailscale >> "$LOG_FILE" 2>&1; then
319     fail "tailscale CLI not on PATH install via 'brew install tailscale' then re-run"
320     echo " (a Tailscale daemon is required for the exit-node path to work)"
321 else
322     set +e
323     TS_OUT=$(run_with_timeout 5 tailscale status 2>&1)
324     TS_EXIT=$?
325     set -uo pipefail
326     TS_HEAD=$(printf '%s\n' "$TS_OUT" | head -5 | tr -d '\r')
327     if [ "$TS_EXIT" -eq 137 ]; then
328         fail "tailscaled daemon is wedged/unresponsive (5s timeout)"
329         echo " Fix path: 'sudo brew services restart tailscale'"
330     elif printf '%s' "$TS_OUT" | grep -qE 'Logged out|not logged in|expired'; then
331         fail "tailscale is logged out / auth expired on this host"
332         echo " Fix path: 'sudo tailscale up' (browser auth once)"
333     elif printf '%s' "$TS_OUT" | grep -q '100\.'; then
334         ok "Host is on tailnet (100.x.x.x visible in status)"
335         TS_ON_MESH=true
336         if printf '%s' "$TS_OUT" | grep -q "$GATEWAY_TS_IP"; then
337             ok "Gateway $GATEWAY_TS_IP visible in host's peer list"
338         elif [ -n "$GATEWAY_TS_IP" ]; then
339             warn "Gateway $GATEWAY_TS_IP NOT in host's peer list possible auth/key mismatch"
340         fi
341         line " first 5 lines of 'tailscale status' "
342         line "$(printf '%s' "$TS_HEAD" | awk '{print " " "$0}')"
343     else
344         fail "tailscale output unrecognized:"
345         line "$(printf '%s' "$TS_OUT" | head -3 | awk '{print " " "$0}')"
346     fi
347 fi
348
349 #
350 section "2/8 Active SOCKS5 proxy on Wi-Fi"
351 #
352 SOCKS_OUT=$(networksetup -getsocksfirewallproxy "$ACTIVE_SERVICE" 2>> "$LOG_FILE" \
353     || networksetup -getsocksfirewallproxy Wi-Fi 2>> "$LOG_FILE" \
354     || echo "Could not query SOCKS5 proxy state")
355 if printf '%s' "$SOCKS_OUT" | grep -q "Enabled: Yes"; then
356     fail "SOCKS5 proxy IS enabled on $ACTIVE_SERVICE toggle.sh fell back to SOCKS5 mode last run"
357     SOCKS5_ENABLED=true
358     line " Full state:"
359     line "$(printf '%s' "$SOCKS_OUT" | awk '{print " " "$0}')"
360     line " Browse on macOS ignores system SOCKS5 by default traffic bypasses WARP."
361 else
362     ok "SOCKS5 proxy is OFF on $ACTIVE_SERVICE exit-node path should be active"
363 fi
364
365 #
366 section "3/8 Egress IP + WARP tunnel verification (definitive test)"
367 #
368 CDN_OUT=$(run_with_timeout 8 curl -s --max-time 6 \
369     https://www.cloudflare.com/cdn-cgi/trace 2>&1 || echo "(curl failed)")
370 CDN_WARP=$(printf '%s' "$CDN_OUT" | awk -F=' ' /^warp/{print $2; exit}')
371 CDN_IP=$(printf '%s' "$CDN_OUT" | awk -F=' ' /^ip/{print $2; exit}')
372 CDN_COLO=$(printf '%s' "$CDN_OUT" | awk -F=' ' /^colo/{print $2; exit}')
373 WARP_STATUS="$CDN_WARP"
374 if [ "$CDN_WARP" = "on" ]; then
375     ok "warp=on tunnel is hot, host traffic IS going through Cloudflare WARP"
376     line " Public IP: $CDN_IP"
377     line " Cloudflare: ${CDN_COLO:-unknown colo}"
378     line " whatismyip.com's 'local ISP' result is its ISP-database error, not yours."
379 elif [ "$CDN_WARP" = "off" ]; then
380     fail "warp=off host traffic is NOT going through WARP"
381     line " Public IP: $CDN_IP (NOT Cloudflare)"
382     warn "This is the actual leak."
383 else
384     warn "could not determine warp status (curl failed or returned unexpected shape)"
385     line " Raw output: $(printf '%s' "$CDN_OUT" | head -3 | tr '\n' '|')"
386 fi
387
388 #
389 section "4/8 IPv6 leak probe (bypasses IPv4 exit-node on campus networks)"
390 #
391 IPV6_OUT=$(run_with_timeout 7 curl -6 -s --max-time 5 ifconfig.co 2>&1 || echo "")

```

```

392 if [ -n "$IPV6_OUT" ] && printf '%s' "$IPV6_OUT" | grep -qE '[0-9a-fA-F:]+$'; then
393     fail "host has live IPv6 egress $IPV6_OUT (A6 record bypasses WARP)"
394     IPV6_LEAK=true
395     line " Tailscale exit-node advertisement is IPv4-only, so IPv6 traffic skips it."
396     line " Quick-fix: 'sudo networksetup -setv6off $ACTIVE_SERVICE'"
397 else
398     ok "no IPv6 egress detected (good IPv6 won't bypass the tunnel)"
399 fi
400
401 #
402 section "5/8 Host egress route for real traffic (route -n get 1.1.1.1)"
403 #
404 # WHY NOT 'netstat -rn | grep default':
405 # On macOS, Tailscale does NOT replace the physical default route installed by
406 # DHCP. Instead it adds a second interface-scoped route bound to the utun
407 # interface that the kernel prefers for real traffic forwarding. The literal
408 # "default" entry (192.168.x.x via en0) is left untouched in the table as a
409 # BSD routing quirk it's not lying, it's just answering a different question.
410 # The correct question is: "where does a real destination actually resolve?"
411 # `route -n get 1.1.1.1` asks the kernel's forwarding logic directly.
412 ROUTE_GET=$(route -n get 1.1.1.1 2>/dev/null)
413 ROUTE_IFACE=$(echo "$ROUTE_GET" | awk '/interface:/{print $2}')
414 ROUTE_GATEWAY=$(echo "$ROUTE_GET" | awk '/gateway:/{print $2}')
415 if [ -z "$ROUTE_IFACE" ]; then
416     warn "route -n get 1.1.1.1 returned no interface routing table may be empty"
417 else
418     line " interface: $ROUTE_IFACE gateway: ${ROUTE_GATEWAY:-n/a}"
419     if echo "$ROUTE_IFACE" | grep -qE '^utun[0-9]+'; then
420         ok "1.1.1.1 resolves via $ROUTE_IFACE Tailscale interface-scoped route is winning"
421         DEFAULT_VIA_UTUN=true
422     elif echo "$ROUTE_IFACE" | grep -qE '^en[0-9]+'; then
423         fail "1.1.1.1 resolves via $ROUTE_IFACE (physical Wi-Fi) exit-node interface route not active"
424     else
425         warn "1.1.1.1 resolves via unexpected interface: $ROUTE_IFACE"
426     fi
427 fi
428
429 #
430 section "6/8 Last toggle.sh START-relevant lines in output.log (if any)"
431 #
432 if [ ! -f "$LOG_FILE" ]; then
433     note "output.log"
434     line " toggle.sh was never run from this directory. Run './toggle.sh' first,"
435     line " then re-run this diagnostic."
436 else
437     HITS=$(grep -E 'Exit node enabled|SKIP_EXIT_NODE|Pre-flight|Falling back to SOCKS|Starting|Successfully' \
438         "$LOG_FILE" 2>/dev/null | tail -8 || true)
439     if [ -z "$HITS" ]; then
440         note "matching lines in output.log"
441     else
442         line "$(printf '%s\n' "$HITS" | awk '{print " " "$0}')"
443     fi
444 fi
445
446 #
447 section "7/8 Gateway IP we should be pointing at"
448 #
449 if [ -n "$GATEWAY_TS_IP" ]; then
450     ok "gateway Tailscale IP: $GATEWAY_TS_IP"
451 else
452     warn "could not resolve gateway Tailscale IP"
453     line " (looked in .gateway_ip and tried docker compose exec tailscale ip -4)"
454 fi
455
456 #
457 section "8/8 Tailscale System Extension (App Store conflict check)"
458 #
459 SE_PENDING_UNINSTALL=false
460 SE_ACTIVE=false
461 if command -v systemextensionsctl >/dev/null 2>&1; then
462     SE_OUT=$(systemextensionsctl list 2>/dev/null | grep -iE '(io|com)\.tailscale')
463     if [ -n "$SE_OUT" ]; then
464         line " Tailscale System Extension(s) detected:"
465         line "$(printf '%s' "$SE_OUT" | awk '{print " " "$0}')"
466         if printf '%s' "$SE_OUT" | grep -q 'waiting to uninstall'; then
467             SE_PENDING_UNINSTALL=true
468             warn "System Extension is pending uninstall on next reboot."
469         else
470             SE_ACTIVE=true
471             warn "Active Tailscale System Extension detected. This will conflict with Homebrew Tailscale."
472         fi
473     fi
474 fi

```

```

473     else
474         ok "no Tailscale System Extension detected"
475     fi
476 else
477     note "systemextensionsctl not available (not on macOS or permission restricted)"
478 fi
479
480 #
481 section "CLASSIFICATION    applies ONE of the four known scenarios"
482 #
483
484 if [ "$WARP_STATUS" = "on" ] && [ "$IPV6_LEAK" = "false" ]; then
485     SCENARIO="OK"
486 elif [ "$SE_PENDING_UNINSTALL" = "true" ]; then
487     SCENARIO="SE_PENDING"
488 elif [ "$SOCKS5_ENABLED" = "true" ]; then
489     SCENARIO="A"
490 elif [ "$IPV6_LEAK" = "true" ]; then
491     SCENARIO="B"
492 elif [ "$DEFAULT_VIA_UTUN" = "false" ] && [ "$TS_ON_MESH" = "true" ]; then
493     SCENARIO="C"
494 else
495     SCENARIO="UNKNOWN"
496 fi
497
498 case "$SCENARIO" in
499     SE_PENDING)
500         echo ""
501         echo -e " ${RED}${BOLD}VERDICT: Scenario SE_PENDING Tailscale System Extension Pending Uninstall${NC}"
502         echo " A prior App Store Tailscale System Extension is pending uninstall."
503         echo " Since System Integrity Protection (SIP) is enabled, macOS blocks manual"
504         echo " uninstallation of terminated system extensions until the next reboot."
505         echo " Brew-managed tailscaled cannot engage the Network Extension slot until this is resolved."
506         echo ""
507         echo "${BOLD}Recommended fix:${NC}"
508         echo "     sudo shutdown -r now"
509         echo "     # After reboot, run: ./toggle.sh"
510         ;;
511     A)
512         echo ""
513         echo -e " ${RED}${BOLD}VERDICT: Scenario A SOCKS5 fallback lane${NC}"
514         echo " toggle.sh's pre-flight checks failed last run. The script activated"
515         echo " SOCKS5 + local DNS proxy as a fallback. DNS gets hijacked but"
516         echo " browsers on macOS ignore the system SOCKS5 traffic egresses"
517         echo " direct through the local network."
518         echo ""
519         echo "${BOLD}Recommended fix:${NC}"
520         echo "     echo 'GATEWAY_BYPASS_PING=true' >> $PROJECT_ROOT/.env"
521         echo "     sudo tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true \\"
522         if [ -n "$GATEWAY_TS_IP" ]; then
523             echo "         --exit-node=$GATEWAY_TS_IP --exit-node-allow-lan-access=true"
524         else
525             echo "         --exit-node=$GATEWAY_TS_IP --exit-node-allow-lan-access=true # fill in $GATEWAY_TS_IP"
526         fi
527         echo "     sudo dscacheutil -flushcache && sudo killall -HUP mDNSResponder"
528         echo "     ./toggle.sh"
529         ;;
530     B)
531         echo ""
532         echo -e " ${RED}${BOLD}VERDICT: Scenario B IPv6 leak over dual-stack campus/local IPv6${NC}"
533         echo " Tailscale exit-node advertises only IPv4. macOS sends AAAA queries"
534         echo " straight out en0, which is dual-stack on most campus APs IPv6"
535         echo " traffic bypasses the WARP tunnel entirely."
536         echo ""
537         echo "${BOLD}Recommended fix:${NC}"
538         echo "     sudo networksetup -setv6off $ACTIVE_SERVICE"
539         echo "     sudo dscacheutil -flushcache && sudo killall -HUP mDNSResponder"
540         echo "     # Re-enable IPv6 later with: sudo networksetup -setv6automatic $ACTIVE_SERVICE"
541         ;;
542     C)
543         echo ""
544         echo -e " ${RED}${BOLD}VERDICT: Scenario C Tailscale route-freeze${NC}"
545         echo " Host's default route points at the Wi-Fi gateway instead of the"
546         echo " Tailscale utun* interface. The exit-node preference is set but the"
547         echo " routing-table assertion did not take effect (devref 10.26)."
548         echo ""
549         echo "${BOLD}Recommended fix:${NC}"
550         echo "     sudo brew services restart tailscale"
551         echo "     sudo route delete -net 0.0.0.0/1"
552         echo "     sudo route delete -net 128.0.0.0/1"
553         echo "     sudo dscacheutil -flushcache && sudo killall -HUP mDNSResponder"

```

```

554     if [ -n "$GATEWAY_TS_IP" ]; then
555     echo "      sudo tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true \\"
556     echo "          --exit-node=$GATEWAY_TS_IP --exit-node-allow-lan-access=true"
557     else
558     echo "      sudo tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true --exit-node=\$TS_IP
559     --exit-node-allow-lan-access=true"
560     fi
561     echo "      ./toggle.sh"
562     ;;
563 OK)
564     echo ""
565     echo -e "  ${GREEN}${BOLD}VERDICT: No host leak detected tunnel is working${NC}"
566     echo "  The local ISP string from whatismyip.com is its IP-database"
567     echo "  misclassifying the Cloudflare egress IP. CDN-CGI's trace endpoint"
568     echo "  (which is what we use here) reports warp=on because that endpoint"
569     echo "  is hosted by Cloudflare itself and can see its own edge."
570     echo "  If you want a third-party confirmation:"
571     echo "      curl -s https://api.ipify.org; echo"
572     ;;
573 *)
574     echo ""
575     echo -e "  ${YELLOW}${BOLD}VERDICT: Could not classify automatically${NC}"
576     echo "  The diagnostic data did not match any known scenario cleanly."
577     echo "  Likely causes:"
578     echo "    - tailscaled is logged out / expired (see Section 1)"
579     echo "    - Docker is not running (gateway containers down)"
580     echo "    - .gateway_ip missing AND docker compose exec failed"
581     echo "  Inspect the report at: $REPORT_FILE"
582     ;;
583 esac
584 echo ""
585 #
586 # Write the verdict block to the report file
587 #
588 SECOND_BLOCK=$(cat <<EOF
589
590 C L A S S I F I C A T I O N   R E S U L T
591
592 Scenario: $SCENARIO
593 WARP egress: $WARP_STATUS (cdn-cgi/trace)
594 SOCKS5 lane: $([ "$SOCKS5_ENABLED" = true ] && echo "ENABLED (browsers bypass)" || echo "off")
595 IPv6 leak: $([ "$IPV6_LEAK" = true ] && echo "YES (campus IPv6 egress)" || echo "no")
596 Default via: $([ "$DEFAULT_VIA_UTUN" = true ] && echo "utun* (Tailscale)" || echo "Wi-Fi gateway")
597 Host on mesh: $([ "$TS_ON_MESH" = true ] && echo "yes" || echo "no")
598 SE pending: $([ "$SE_PENDING_UNINSTALL" = true ] && echo "yes" || echo "no")
599 Gateway IP: ${GATEWAY_TS_IP:-unresolved}
600 EOF
601 )
602 printf '%s\n' "$SECOND_BLOCK" >&3
603 #
604 # --fix mode
605 #
606 if [ "$DO_FIX" = "true" ] && [ "$SCENARIO" != "OK" ] && [ "$SCENARIO" != "UNKNOWN" ]; then
607     echo ""
608     echo ""
609     echo "  A P P L Y I N G   F I X   (scenario $SCENARIO)"
610     echo ""
611     echo ""
612     case "$SCENARIO" in
613     SE_PENDING)
614         warn "System Extension uninstall is pending on next reboot."
615         line "Please run 'sudo shutdown -r now' to reboot the system."
616         ;;
617     A)
618         if [ -n "$GATEWAY_TS_IP" ]; then
619             line "writing GATEWAY_BYPASS_PING=true to .env"
620             ENV="$PROJECT_ROOT/.env"
621             [ -f "$ENV" ] || touch "$ENV"
622             if grep -q '^GATEWAY_BYPASS_PING=' "$ENV" 2>/dev/null; then
623                 sed -i 's/^GATEWAY_BYPASS_PING=.*GATEWAY_BYPASS_PING=true/' "$ENV"
624             else
625                 printf '\nGATEWAY_BYPASS_PING=true\n' >> "$ENV"
626             fi
627             line "re-asserting tailscale exit-node (with --reset)"
628             sudo -n tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true \
629                 --exit-node="$GATEWAY_TS_IP" --exit-node-allow-lan-access=true \
630                 >> "$LOG_FILE" 2>&1 \

```



```

634     && ok "tailscale exit-node re-asserted for $GATEWAY_TS_IP" \
635     || warn "tailscale up did not return success (check $LOG_FILE)"
636 else
637     warn "no gateway Tailscale IP resolved skipping tailscale up"
638     warn "fix manually: 'sudo tailscale up --reset ... --exit-node=<TS_IP>'"
639 fi
640 ;;
641 B)
642     line " disabling IPv6 on $ACTIVE_SERVICE while gateway is up"
643     sudo -n networksetup -setv6off "$ACTIVE_SERVICE" >> "$LOG_FILE" 2>&1 \
644     && ok "IPv6 disabled on $ACTIVE_SERVICE (re-enable with 'setv6automatic')\" \
645     || warn "networksetup -setv6off failed (check $LOG_FILE)"
646 ;;
647 C)
648     line " restarting tailscaled (route-table fix)"
649     run_with_timeout 15 brew services restart tailscale >> "$LOG_FILE" 2>&1 \
650     || run_with_timeout 15 sudo -n brew services restart tailscale >> "$LOG_FILE" 2>&1
651     sleep 3
652     line " flushing stale host routes + DNS cache"
653     sudo -n route delete -net 0.0.0.0/1 >> "$LOG_FILE" 2>&1 || true
654     sudo -n route delete -net 128.0.0.0/1 >> "$LOG_FILE" 2>&1 || true
655     sudo -n dscacheutil -flushcache >> "$LOG_FILE" 2>&1 || true
656     sudo -n killall -HUP mDNSResponder >> "$LOG_FILE" 2>&1 || true
657     if [ -n "$GATEWAY_TS_IP" ]; then
658         line " re-asserting exit-node after restart"
659         sudo -n tailscale up --reset --ssh=true --accept-dns=false --accept-routes=true \
660         --exit-node="$GATEWAY_TS_IP" --exit-node-allow-lan-access=true \
661         >> "$LOG_FILE" 2>&1 \
662         && ok "tailscale exit-node re-asserted for $GATEWAY_TS_IP" \
663         || warn "tailscale up did not return success (check $LOG_FILE)"
664     fi
665 ;;
666 esac
667
668 echo ""
669 echo "Re-verifying after fix..."
670 sleep 3
671 CDN_WARP_AFTER=$(run_with_timeout 8 curl -s --max-time 6 \
672     https://www.cloudflare.com/cdn-cgi/trace 2>&1 \
673     | awk -F'=' '/^warp={print $2; exit}')
674 IPV6_AFTER=$(run_with_timeout 7 curl -6 -s --max-time 5 ifconfig.co 2>&1 || echo "")
675 printf '\n[AFTER FIX]\n' >&3
676 printf ' warp status: %s\n' "$CDN_WARP_AFTER" >&3
677 printf ' IPv6 egress: %s\n' "${IPV6_AFTER:-none}" >&3
678 line " warp status: ${CDN_WARP_AFTER:-unknown}"
679 line " IPv6 egress: ${IPV6_AFTER:-none}"
680 if [ "$CDN_WARP_AFTER" = "on" ] && [ -z "$IPV6_AFTER" ]; then
681     echo ""
682     ok "Fix verified host is now routing through WARP. Remote clients unaffected."
683 else
684     echo ""
685     warn "Fix did not fully take effect on first try. Likely causes:"
686     line " tailscaled needed a longer settling window (re-run diagnostic in 30s)"
687     line " The classified scenario was wrong; paste this report for triage."
688     line " Tailscale needs a full auth refresh: 'sudo tailscale up'"
689 fi
690 fi
691
692 echo ""
693 echo ""
694 echo " Full report written to:"
695 echo " $REPORT_FILE"
696 echo ""
697 echo ""
698
699 #
700 # --watch mode: enter the continuous monitoring loop after baseline diagnostic
701 #
702 if [ "$DO_WATCH" = "true" ]; then
703     # Build baseline from the full diagnostic just completed
704     BASELINE_WARP="$WARP_STATUS"
705     BASELINE_DEFAULT=$( [ "$DEFAULT_VIA_UTUN" = "true" ] && echo "utun" || echo "wifi/other")
706
707     if [ "$SCENARIO" != "OK" ]; then
708         warn "Baseline diagnostic found issues (scenario: $SCENARIO)."
709         line " Watch loop will still run alerts fire on any STATE CHANGE from current baseline."
710         line " Fix the issue first? Re-run with: bash scripts/diagnose-host-leak.sh --fix"
711         echo ""
712     fi
713
714     run_watch_loop "$BASELINE_WARP" "$BASELINE_DEFAULT" "$IPV6_LEAK" "$WATCH_INTERVAL"

```

```

715 fi
716
717 exit 0

```

37 scripts/dns-proxy.py

```

1  #!/usr/bin/env python3
2  """
3  dns-proxy.py Local DNS proxy for nullexit gateway.
4
5  Listens on UDP port 53, receives DNS queries, forwards them over TCP
6  to AdGuard at 127.0.0.1:5354 (the Docker port mapping), and returns
7  the response via UDP. Handles the DNS-over-TCP wire format (2-byte
8  length prefix) correctly unlike socat which just forwards raw bytes.
9  """
10
11 import socket
12 import struct
13 import sys
14 import os
15 from concurrent.futures import ThreadPoolExecutor
16
17 LISTEN_ADDR = os.environ.get("LISTEN_ADDR", "127.0.0.1")
18 LISTEN_PORT = int(os.environ.get("LISTEN_PORT", 53))
19 TARGET_HOST = os.environ.get("TARGET_HOST", "127.0.0.1")
20 TARGET_PORT = int(os.environ.get("TARGET_PORT", 5354))
21
22 # When invoked via `sudo -n`, custom env vars set by the caller are stripped by
23 # sudo's env_reset unless allowlisted in sudoers which the shipped NOPASSWD
24 # rule for this exact script (no bash -c wrapper, no env prefix) does not do.
25 # A caller that needs non-default values drops them here *before* sudo'ing the
26 # literal whitelisted command; this process picks them up on top of the
27 # env-var/default values above. Same bounded local-TOCTOU class already
28 # accepted for /tmp/nullexit-ports.env (see devref.md 15.9.5) restrictive
29 # perms, not a hard security boundary against a co-resident attacker.
30 _CONF = "/tmp/nullexit-dns-proxy.conf"
31 if os.path.isfile(_CONF):
32     with open(_CONF) as _f:
33         for _line in _f:
34             _line = _line.strip()
35             if not _line or _line.startswith("#") or "=" not in _line:
36                 continue
37             _k, _v = _line.split("=", 1)
38             if _k == "LISTEN_ADDR":
39                 LISTEN_ADDR = _v
40             elif _k == "LISTEN_PORT":
41                 LISTEN_PORT = int(_v)
42             elif _k == "TARGET_HOST":
43                 TARGET_HOST = _v
44             elif _k == "TARGET_PORT":
45                 TARGET_PORT = int(_v)
46
47 def handle_query(data: bytes, client_addr: tuple, sock: socket.socket) -> None:
48     """Forward a single DNS query over TCP and send the response back via UDP."""
49     try:
50         tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
51         tcp.settimeout(5)
52         tcp.connect((TARGET_HOST, TARGET_PORT))
53
54         # DNS over TCP: prepend 2-byte length (big-endian)
55         tcp.sendall(struct.pack("!H", len(data)) + data)
56
57         # Read the 2-byte response length
58         raw_len = tcp.recv(2)
59         if len(raw_len) < 2:
60             tcp.close()
61             return
62         resp_len = struct.unpack("!H", raw_len)[0]
63
64         # Read the full response
65         response = b""
66         while len(response) < resp_len:
67             chunk = tcp.recv(resp_len - len(response))
68             if not chunk:
69                 break
70

```

```

71         response += chunk
72
73         tcp.close()
74
75         # Send the response back over UDP (strip TCP length prefix)
76         if response:
77             sock.sendto(response, client_addr)
78     except Exception:
79         # Silently drop failed queries (malformed, timeout, etc.)
80         pass
81
82
83 def main() -> None:
84     # Create UDP socket
85     sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
86     sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
87
88     try:
89         sock.bind((LISTEN_ADDR, LISTEN_PORT))
90     except PermissionError:
91         print(f"dns-proxy: ERROR need root to bind port {LISTEN_PORT}", flush=True)
92         sys.exit(1)
93     except OSError as e:
94         print(f"dns-proxy: ERROR {e}", flush=True)
95         sys.exit(1)
96
97     print(f"dns-proxy: listening on UDP {LISTEN_ADDR}:{LISTEN_PORT} TCP {TARGET_HOST}:{TARGET_PORT}", flush=True)
98
99     # Use a thread pool to handle concurrent DNS queries efficiently without thread exhaustion
100    executor = ThreadPoolExecutor(max_workers=64)
101
102    while True:
103        try:
104            data, client_addr = sock.recvfrom(4096) # Support EDNS0 (up to 4096 bytes) to prevent truncation
105            executor.submit(handle_query, data, client_addr, sock)
106        except KeyboardInterrupt:
107            break
108        except Exception:
109            pass
110
111    executor.shutdown(wait=False)
112    sock.close()
113
114
115 if __name__ == "__main__":
116     main()

```

38 scripts/dns_anomaly_detector.py

```

1  #!/usr/bin/env python3
2  """
3  nullexit DNS anomaly detector (v2)  report-only DNS-tunneling / DGA / C2 finder.
4
5  Replaces the v1 whole-FQDN Shannon-entropy + mean3 demo (which measured the
6  wrong scope, was fooled by short encoded labels, assumed a normal distribution
7  DNS names don't have, and ignored the volumetric signal that actually defines
8  tunneling). v2 keeps the "statistical, data-based, 100% local, no third-party
9  deps" spirit but detects far better:
10
11  1. n-gram (character bigram) improbability of the SUBDOMAIN labels only,
12     trained on YOUR querylog. Robust on short strings where Shannon entropy
13     fails (entropy is capped at log2(len)); scores "doesn't look like real
14     naming", which is what encoded tunneling labels are.
15  2. per-registered-domain AGGREGATION (unique-subdomain count, query volume,
16     suspicious record types) a stream of high-improbability names under ONE
17     domain is the real tunneling tell, not any single name.
18  3. robust thresholds (median + kMAD, resistant to CDN/cloud hash outliers)
19     instead of mean3 on a non-normal distribution.
20  4. a data-driven allowlist of YOUR frequent domains, so CloudFront/S3/telemetry
21     hashes don't false-positive.
22
23  Report-only by design: a privacy gateway must never silently drop DNS on a
24  statistical trip (a false positive = "my internet broke"). It surfaces findings;
25  promoting a domain to an actual block stays a human/AdGuard-blocklist decision.
26

```

```

27 Footprint: single streaming pass (never loads the log into RAM); the model is a
28 ~4040 bigram table + a few floats + a small allowlist (a few KB on disk); per-
29 domain state is a handful of ints plus a subdomain set capped at 64.
30
31 Exit codes (scan): 0 = ran, clean 1 = ran, anomalies found 2 = FAILED to run
32 (no/corrupt model, unreadable or empty querylog) a failure is never reported as
33 "clean". selftest exits 3 if the detection logic itself is broken.
34
35 Usage:
36 dns_anomaly_detector.py learn [--log PATH] [--out PATH] # train from querylog
37 dns_anomaly_detector.py scan [--log PATH] [--model PATH] [--recent N] [--quiet]
38 dns_anomaly_detector.py selftest # prove it detects (exit 3 if broken)
39 dns_anomaly_detector.py demo # human-readable showcase
40 """
41 import argparse
42 import json
43 import math
44 import os
45 import random
46 import re
47 import sys
48 import time
49 from collections import defaultdict
50
51 DEFAULT_LOG = os.path.join(os.path.dirname(os.path.abspath(__file__)),
52                             "..", "adguard", "work", "data", "querylog.json")
53 DEFAULT_MODEL = os.path.join(os.path.dirname(os.path.abspath(__file__)),
54                               "..", ".dns_baseline.json")
55 OUTPUT_LOG = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "output.log")
56
57
58 def log_fail(msg):
59     """Persist a hard failure to output.log (besides stderr) so a broken detector
60     is on the record even when run headless (launchd/sweep/CI). Heisenbug-safe by
61     construction: it only ever APPENDS to a separate file it touches neither
62     stdout/stderr (which sweep captures) nor any detection state, and a logging
63     error is swallowed so it can never change what the detector does or returns."""
64     try:
65         with open(OUTPUT_LOG, "a", encoding="utf-8") as f:
66             f.write(f"[{time.strftime('%Y-%m-%d %H:%M:%S')}] [dns-anomaly] FAIL: {msg}\n")
67     except OSError:
68         pass
69
70 ALPHABET = "abcdefghijklmnopqrstuvwxyz0123456789-_"
71 AIDX = {c: i for i, c in enumerate(ALPHABET)}
72 OTHER = len(ALPHABET) # bucket for any char outside the alphabet
73 NSYM = len(ALPHABET) + 1
74 BEG = NSYM # start-of-label marker
75 NSTATES = NSYM + 1
76
77 # Minimal two-level public suffixes so eTLD+1 grouping is stable without shipping
78 # the full ~200 KB Public Suffix List. Exact PSL isn't needed we only need a
79 # consistent grouping key per registered domain.
80 TWO_LEVEL = {
81     "co.uk", "org.uk", "ac.uk", "gov.uk", "co.jp", "co.kr", "co.in", "co.za",
82     "co.nz", "com.au", "net.au", "org.au", "com.br", "com.mx", "com.tr",
83     "com.cn", "com.hk", "com.sg", "com.tw", "com.ar", "co.il", "ne.jp",
84 }
85
86
87 def sym(ch):
88     return AIDX.get(ch, OTHER)
89
90
91 def registered_and_sub(qh):
92     """Split a query host into (registered_domain, subdomain_labels_string)."""
93     qh = qh.rstrip(".").lower()
94     parts = qh.split(".")
95     if len(parts) <= 2:
96         return qh, ""
97     last2 = ".".join(parts[-2:])
98     reg_n = 3 if last2 in TWO_LEVEL else 2
99     reg = ".".join(parts[-reg_n:])
100     sub = ".".join(parts[:-reg_n])
101     return reg, sub
102
103
104 def iter_querylog(path, recent=0):
105     """Stream (QH, QT) from an AdGuard querylog. recent>0 keeps only the last N."""
106     if recent > 0:
107         # Bounded tail without loading the file: keep a ring buffer of raw lines.

```

```

108         from collections import deque
109         buf = deque(maxlen=recent)
110         with open(path, "r", encoding="utf-8", errors="ignore") as f:
111             for line in f:
112                 if line.strip():
113                     buf.append(line)
114         lines = buf
115     else:
116         lines = _line_reader(path)
117     for line in lines:
118         try:
119             rec = json.loads(line)
120         except (json.JSONDecodeError, ValueError):
121             continue
122         qh = rec.get("QH")
123         if not qh:
124             continue
125         qh = qh.strip().lower()
126         if qh.endswith("in-addr.arpa") or qh.endswith("ip6.arpa"):
127             continue
128         yield qh, rec.get("QT", "")
129
130
131 def _line_reader(path):
132     with open(path, "r", encoding="utf-8", errors="ignore") as f:
133         for line in f:
134             if line.strip():
135                 yield line
136
137
138 # n-gram model
139 def label_improbability(label, logp):
140     """Mean -log2 P(char|prev) over a label. High = doesn't look like real naming."""
141     if not label:
142         return 0.0
143     prev = BEG
144     total = 0.0
145     for ch in label:
146         s = sym(ch)
147         total += logp[prev][s]
148         prev = s
149     return total / len(label)
150
151
152 def name_score(sub, logp):
153     """Worst-label improbability across the subdomain (tunneling hides in one label)."""
154     if not sub:
155         return 0.0
156     return max(label_improbability(lbl, logp) for lbl in sub.split(".") if lbl)
157
158
159 def finalize_logp(counts):
160     """Add-1 smoothed transition matrix -log2 probabilities."""
161     logp = [[0.0] * NSYM for _ in range(NSTATES)]
162     for a in range(NSTATES):
163         row_total = sum(counts[a]) + NSYM # +NSYM for add-1 smoothing
164         for b in range(NSYM):
165             p = (counts[a][b] + 1) / row_total
166             logp[a][b] = -math.log2(p)
167     return logp
168
169
170 def learn(log_path, out_path):
171     counts = [[0] * NSYM for _ in range(NSTATES)]
172     reg_counts = defaultdict(int)
173     reservoir = [] # sampled subdomain strings for threshold calibration
174     RES_MAX = 6000
175     seen = 0
176
177     for qh, _qt in iter_querylog(log_path):
178         reg, sub = registered_and_sub(qh)
179         reg_counts[reg] += 1
180         if not sub:
181             continue
182         for lbl in sub.split("."):
183             prev = BEG
184             for ch in lbl:
185                 s = sym(ch)
186                 counts[prev][s] += 1
187                 prev = s
188     # Reservoir-sample subdomains (unbiased) for a robust score distribution.

```

```

189     seen += 1
190     if len(reservoir) < RES_MAX:
191         reservoir.append(sub)
192     else:
193         j = random.randint(0, seen - 1)
194         if j < RES_MAX:
195             reservoir[j] = sub
196
197 if seen == 0:
198     print("learn: no subdomain queries found nothing to train on.", file=sys.stderr)
199     return 1
200
201 logp = finalize_logp(counts)
202
203 # Robust threshold from the score distribution: median + kMAD (MAD1.4826,
204 # so k=6 mean+4 but resistant to the CDN-hash outliers that wreck plain ).
205 scores = sorted(name_score(s, logp) for s in reservoir if s)
206 median = _median(scores)
207 mad = _median(sorted(abs(x - median) for x in scores)) or 1e-9
208 threshold = median + 6.0 * mad
209
210 # Data-driven allowlist: the user's frequent registered domains (their normal
211 # traffic, incl. their habitual CDNs) tunneling to a NEW domain isn't here.
212 top = sorted(reg_counts.items(), key=lambda kv: kv[1], reverse=True)
213 allow_cut = max(20, int(0.002 * seen)) # "frequent" = seen a lot for this net
214 allowlist = [r for r, c in top if c >= allow_cut][:400]
215
216 model = {
217     "version": 2,
218     "counts": counts, # compact ints; logp recomputed on load
219     "threshold": threshold,
220     "median": median,
221     "mad": mad,
222     "allowlist": allowlist,
223     "trained_on": seen,
224     "unique_registered": len(reg_counts),
225 }
226 with open(out_path, "w", encoding="utf-8") as f:
227     json.dump(model, f, separators=(",", ":"))
228 print(f"learn: trained on {seen} subdomain queries across "
229       f"{len(reg_counts)} registered domains.")
230 print(f" score median={median:.3f} MAD={mad:.3f} threshold={threshold:.3f}")
231 print(f" allowlist: {len(allowlist)} frequent domains {out_path} "
232       f"f"{os.path.getsize(out_path)} bytes)")
233 return 0
234
235
236 def _median(xs):
237     n = len(xs)
238     if n == 0:
239         return 0.0
240     xs = sorted(xs)
241     m = n // 2
242     return xs[m] if n % 2 else (xs[m - 1] + xs[m]) / 2.0
243
244
245 # scan
246 SUSPICIOUS_QT = {"TXT", "NULL", "CNAME", "ANY"} # classic tunneling data channels
247 SUB_CAP = 64 # bound per-domain memory
248
249 # High-entropy-but-legit CDN/cloud suffixes: their hostnames LOOK random but are
250 # structured, not encoded payload. A tiny static safety net beside the data-driven
251 # frequency allowlist (which only catches domains queried a lot in THIS log).
252 CDN_SUFFIXES = (
253     "nflxvideo.net", "nflxso.net", "googleusercontent.com", "usercontent.goog",
254     "cloudfront.net", "akamai.net", "akamaihd.net", "akamaiedge.net", "edgekey.net",
255     "edgesuite.net", "fastly.net", "fbcdn.net", "yimg.com", "ggpht.com", "gvt1.com",
256     "gvt2.com", "ie100.net", "azureedge.net", "cloudflare.net", "cloudflarestream.com",
257     "digitaloceanspaces.com", "b-cdn.net", "llnwd.net", "wac.edgecastcdn.net", "cdn77.org",
258 )
259
260 _HEX = re.compile(r"^[0-9a-f]{16}$")
261 _B32 = re.compile(r"^[a-z2-7]{16}$")
262
263
264 def looks_encoded(label):
265     """True if a label looks like an encoded payload (hex/base32/long base64),
266     NOT just a random-ish structured CDN name or a long dictionary word. This is
267     the strongest single discriminator between DNS tunneling and legitimate
268     high-entropy hostnames."""
269     if _HEX.match(label): # 16+ hex chars

```

```

270     return True
271 # base32 alphabet [a-z2-7] fully overlaps lowercase letters, so a long word
272 # like "visualstudioexptteam" would match require digits, which real base32
273 # payload always carries (~19% of chars) but dictionary words don't.
274 if _B32.match(label) and sum(c in "234567" for c in label) >= 2:
275     return True
276 # long base64-ish: high char diversity AND several digits (excludes words).
277 if len(label) >= 20 and len(set(label)) >= 13 and sum(c.isdigit() for c in label) >= 3:
278     return True
279 return False
280
281
282 def is_allowlisted(reg, allow):
283     return reg in allow or any(reg == s or reg.endswith("." + s) for s in CDN_SUFFIXES)
284
285
286 def scan(log_path, model_path, recent, quiet):
287     # Fail LOUD, never silently: a corrupt/absent model or an unreadable log must
288     # NOT look like a clean result that would be a security detector reporting
289     # "all clear" while blind. Each failure prints to stderr and exits non-zero.
290     try:
291         with open(model_path, "r", encoding="utf-8") as f:
292             model = json.load(f)
293             counts = model["counts"]
294             threshold = float(model["threshold"])
295             if (not isinstance(counts, list) or len(counts) != NSTATES
296                 or any(not isinstance(r, list) or len(r) != NSYM for r in counts)):
297                 raise ValueError("bigram matrix has the wrong shape")
298     except (OSError, json.JSONDecodeError, KeyError, ValueError, TypeError) as e:
299         msg = (f"could not load a usable baseline model at {model_path} "
300             f"({type(e).__name__}: {e}) run 'learn' first")
301         print(f"scan: FAILED to {msg}.", file=sys.stderr)
302         log_fail(msg)
303         return 2
304     logp = finalize_logp(counts)
305     allow = set(model.get("allowlist", []))
306
307     agg = {} # reg -> [queries, {subdomains cap64}, sum_score, n_scored, susp_qt, maxlen, encoded]
308     for qh, qt in iter_querylog(log_path, recent=recent):
309         reg, sub = registered_and_sub(qh)
310         a = agg.get(reg)
311         if a is None:
312             a = agg[reg] = [0, set(), 0.0, 0, 0, 0, 0]
313         a[0] += 1
314         if qt in SUSPICIOUS_QT:
315             a[4] += 1
316         if sub:
317             if len(a[1]) < SUB_CAP:
318                 a[1].add(sub)
319             sc = name_score(sub, logp)
320             a[2] += sc
321             a[3] += 1
322             for l in sub.split("."):
323                 if len(l) > a[5]:
324                     a[5] = len(l)
325                 if not a[6] and looks_encoded(l):
326                     a[6] = 1
327
328     if not agg:
329         msg = (f"the querylog at {log_path} yielded 0 parseable DNS records "
330             f"the detector did NOT run; this is NOT a clean result")
331         print(f"scan: FAILED {msg}.", file=sys.stderr)
332         log_fail(msg)
333         return 2
334
335     findings = []
336     for reg, (q, subs, ssum, sn, susp, maxlen, encoded) in agg.items():
337         if sn == 0 or is_allowlisted(reg, allow):
338             continue
339         mean_score = ssum / sn
340         uniq = len(subs)
341         improbable = mean_score > threshold
342         # Three independent tunneling signatures (any one flags), each tunneling-
343         # SHAPED so structured CDN names don't trip it. Encoded-payload volume is
344         # primary because encoded labels score like CDN hashes under the bigram
345         # model the improbability threshold alone can't separate them, but their
346         # SHAPE (pure hex/base32/base64) and per-domain multiplicity can:
347         strong_encoded = encoded and uniq >= 8 # many unique encoded payloads
348         txt_abuse = susp >= 5 # sustained TXT/NULL channel
349         improbable_vol = improbable and (uniq >= 8 or maxlen >= 24)
350         if strong_encoded or txt_abuse or improbable_vol:

```



```

351     corro = min(1.0, uniq / 30.0 + susp / 12.0 + encoded * 0.45 + (maxlen >= 40) * 0.2)
352     over = min(1.0, max(0.0, mean_score - threshold) / (threshold + 1e-9))
353     findings.append((round(0.55 * corro + 0.45 * over, 3), reg, round(mean_score, 2),
354                     uniq + (SUB_CAP if uniq >= SUB_CAP else 0), susp, maxlen, q))
355
356 findings.sort(reverse=True)
357 if quiet:
358     print(f"DNS anomaly scan: {len(findings)} suspicious domain(s) "
359           f"(threshold={threshold:.2f}, {len(agg)} domains scanned)")
360 else:
361     print(f"\n=== DNS anomaly scan {len(findings)} suspicious domain(s) "
362           f"of {len(agg)} scanned (threshold {threshold:.2f}) ===")
363     if findings:
364         print(f"{'conf':>5} {'registered domain':>32} {'score':>6} {'uniqusub':>7} "
365               f"{'txt/null':>8} {'maxlbl':>6} {'queries':>7}")
366         for conf, reg, sc, uniq, susp, maxlen, q in findings[:25]:
367             cap = "+" if uniq > SUB_CAP else " "
368             print(f"{'conf':>5} {'reg':>32.32} {'sc':>6} {'min(uniq, SUB_CAP):>6}{cap} "
369                   f"{'susp':>8} {'maxlen':>6} {'q':>7}")
370         else:
371             print(" clean no domain combined improbable naming with a volumetric signal.")
372 return 1 if findings else 0
373
374 # demo (self-test, no querylog needed)
375 def demo():
376     normal = ["google.com", "api.github.com", "mail.google.com", "cdn.jsdelivr.net",
377              "en.wikipedia.org", "outlook.office365.com", "i.redd.it", "apple.com",
378              "static.cloudflareinsights.com", "play.googleapis.com"] * 40
379     counts = [[0] * NSYM for _ in range(NSTATES)]
380     for qh in normal:
381         _reg, sub = registered_and_sub(qh)
382         for lbl in sub.split("."):
383             prev = BEG
384             for ch in lbl:
385                 s = sym(ch)
386                 counts[prev][s] += 1
387                 prev = s
388     logp = finalize_logp(counts)
389     samples = [registered_and_sub(q)[1] for q in normal]
390     scores = sorted(name_score(s, logp) for s in samples if s)
391     med = _median(scores)
392     mad = _median(sorted(abs(x - med) for x in scores)) or 1e-9
393     thr = med + 6.0 * mad
394     print(f"demo threshold = {thr:.3f} (median {med:.3f}, MAD {mad:.3f})")
395     tests = [("api.github.com", "normal"), ("mail.google.com", "normal"),
396             ("A8F93BD72Q9X.attacker.com", "base64 tunnel"),
397             ("c732488a09b2e4f6d1.tunnel.evil.org", "hex tunnel"),
398             ("kZ9x-Qw82-Lp01-Vn55.dns.io", "high-entropy tunnel")]
399     for qh, label in tests:
400         _reg, sub = registered_and_sub(qh)
401         sc = name_score(sub, logp)
402         flag = "anomalous" if sc > thr else "normal"
403         print(f" {flag:14} score={sc:5.2f} {qh} ({label})")
404
405 def selftest():
406     """Prove the detection logic works end-to-end. Exit 0 = healthy, 3 = BROKEN
407     (loud). Lets sweep/CI catch a regression instead of silently trusting a
408     detector that no longer detects. Validates BOTH signal paths: the n-gram
409     language model (improbability) and the encoding-shape detector."""
410     normal = ["google.com", "api.github.com", "mail.google.com", "cdn.jsdelivr.net",
411              "en.wikipedia.org", "outlook.office365.com", "i.redd.it", "apple.com"] * 40
412     counts = [[0] * NSYM for _ in range(NSTATES)]
413     for qh in normal:
414         for lbl in registered_and_sub(qh)[1].split("."):
415             prev = BEG
416             for ch in lbl:
417                 s = sym(ch)
418                 counts[prev][s] += 1
419                 prev = s
420     logp = finalize_logp(counts)
421     scores = sorted(name_score(registered_and_sub(q)[1], logp) for q in normal
422                     if registered_and_sub(q)[1])
423     med = _median(scores)
424     mad = _median(sorted(abs(x - med) for x in scores)) or 1e-9
425     thr = med + 6.0 * mad
426
427     fails = []
428     for g in ("mail.google.com", "api.github.com", "cdn.jsdelivr.net"):
429         sub = registered_and_sub(g)[1]

```

```

432     if name_score(sub, logp) > thr or any(looks_encoded(1) for 1 in sub.split(".")):
433         fails.append(f"false-positive on known-good '{g}')"
434     # hex/base32 encoded payloads must be caught by the SHAPE detector:
435     for enc in ("a8f93bd72c9e4f16.evil.com", "mzxw6ytb2do4zb3q7a.tunnel.io"):
436         lbl = registered_and_sub(enc)[1].split(".")[0]
437         if not looks_encoded(lbl):
438             fails.append(f"encoding detector missed '{lbl}')"
439     # an unpronounceable non-encoded label must be caught by the n-gram model:
440     imp = registered_and_sub("xqzjvkwbmqpzrfxn.dga.net")[1].split(".")[0]
441     if name_score(imp, logp) <= thr:
442         fails.append(f"n-gram model missed improbable label '{imp}')"
443
444     if fails:
445         print("SELFTEST FAILED  detector is not working:", file=sys.stderr)
446         for f in fails:
447             print(f" - {f}", file=sys.stderr)
448         log_fail("selftest  detector is not working: " + " ".join(fails))
449         return 3
450     print("selftest OK  flags hex/base32 payloads + improbable labels, clears known-good.")
451     return 0
452
453
454 def main():
455     ap = argparse.ArgumentParser(description="nullexit DNS anomaly detector (report-only)")
456     sub = ap.add_subparsers(dest="cmd", required=True)
457     pl = sub.add_parser("learn"); pl.add_argument("--log", default=DEFAULT_LOG); pl.add_argument("--out",
458         default=DEFAULT_MODEL)
459     ps = sub.add_parser("scan")
460     ps.add_argument("--log", default=DEFAULT_LOG); ps.add_argument("--model", default=DEFAULT_MODEL)
461     ps.add_argument("--recent", type=int, default=0, help="scan only the last N records (0=all)")
462     ps.add_argument("--quiet", action="store_true")
463     sub.add_parser("demo")
464     sub.add_parser("selftest")
465     args = ap.parse_args()
466
467     if args.cmd == "learn":
468         if not os.path.exists(args.log):
469             print(f"learn: querylog not found at {args.log}", file=sys.stderr); return 1
470         return learn(args.log, args.out)
471     if args.cmd == "scan":
472         if not os.path.exists(args.log):
473             print(f"scan: querylog not found at {args.log}", file=sys.stderr); return 2
474         return scan(args.log, args.model, args.recent, args.quiet)
475     if args.cmd == "demo":
476         demo(); return 0
477     if args.cmd == "selftest":
478         return selftest()
479
480 if __name__ == "__main__":
481     sys.exit(main())

```

39 scripts/fix-docker-bridge-collision.sh

```

1  #!/bin/bash
2  # scripts/fix-docker-bridge-collision.sh  nullexit fix for devref 9.13
3  #
4  # Symptom: host default route points at Docker's bridge gateway (172.17.0.1)
5  # instead of the real Wi-Fi gateway. ALL host internet egress fails. Remote
6  # tailnet clients route through the gateway correctly (their tunnel is
7  # independent of the host's broken route table), so they look healthy while
8  # the host itself cannot reach anything.
9  #
10 # Root cause: Docker's default bridge claims 172.17.0.0/16 by default. When the
11 # host's physical Wi-Fi also lands in the same /16  extremely common on
12 # university networks which hand out 172.16.0.0/12  the kernel
13 # installs Docker's bridge as the default route, and every packet bound for
14 # the internet is silently absorbed by docker0 instead of egressing.
15 #
16 # This script:
17 # 1. Detects the actual collision (Docker's 172.17.0.0/16 vs the host's
18 #    Wi-Fi subnet, queried live via the ACTIVE interface not hardcoded en0)
19 # 2. Picks a non-colliding Docker bip using /8-family routing (works for
20 #    172.16.0.0/12 campus networks; naive prefix-match would pick 172.26/24
21 #    which still lives INSIDE a /12 of 172.16.0.0/12)
22 # 3. Backs up ~/.colima/default/colima.yaml with an ISO-8601 timestamp

```

```

23 # 4. Edits the file in place, idempotent (replace existing docker.bip /
24 #    append new docker.bip without disturbing other keys; skip YAML comments)
25 # 5. Restarts Colima so the VM rebuilds with the new bridge subnet
26 # 6. Rebinds Wi-Fi DHCP so the host re-learns its real Wi-Fi gateway
27 # 7. Verifies the new default route is no longer the Docker bridge
28 # 8. Re-runs toggle.sh to bring the gateway back up cleanly
29 # 9. Re-runs scripts/diagnose-host-leak.sh and prints the new verdict
30 #
31 # Usage:
32 #   bash scripts/fix-docker-bridge-collision.sh          # apply the fix
33 #   bash scripts/fix-docker-bridge-collision.sh --dry    # show what would change, then exit
34 #   bash scripts/fix-docker-bridge-collision.sh --skip-toggle # fix but don't restart gateway
35 #   bash scripts/fix-docker-bridge-collision.sh --help   # usage
36 #
37 set -uo pipefail
38 # NOTE: errexit (`set -e`) is intentionally NOT enabled so intermediate
39 # `command || warn` patterns can continue past non-fatal failures.
40 # Every critical failure (colima restart, sed, cp backup, toggle.sh)
41 # uses an explicit `|| die` so a half-applied state never escapes.
42 #
43 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
44 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
45 LOG_FILE="$PROJECT_ROOT/output.log"
46 source "$SCRIPT_DIR/common.sh"
47 TIMESTAMP="$(date -u +%Y%m%dT%H%M%SZ)"
48 COLIMA_YAML="$HOME/.colima/default/colima.yaml"
49 PLATFORM="$(uname -s)"
50 #
51 # Argument parsing
52 DRY_RUN=false
53 SKIP_TOGGLE=false
54 case "${1:-}" in
55     --dry|-n)          DRY_RUN=true ;;
56     --skip-toggle)    SKIP_TOGGLE=true ;;
57     --help|-h)
58         sed -n '28,37p' "$0"
59         exit 0
60     ;;
61     "")
62         : # default: apply
63     ;;
64     *)
65         printf 'Unknown argument: %s\n' "$1" >&2
66         exit 2
67     ;;
68 esac
69 #
70 #
71 # Helper: print "[dry-run] $cmd" or actually run $cmd. Wraps the redirect
72 # INSIDE the function so outer `>> $FILE` statements never accidentally
73 # append dry-run noise to the real file. (Earlier draft didn't and the
74 # dry-run polluted $COLIMA_YAML.)
75 run() {
76     if [ "$DRY_RUN" = "true" ]; then
77         printf '[dry-run] %s\n' "$*"
78         return 0
79     fi
80     printf '%s\n' "$*"
81     "$@"
82 }
83 #
84 # Platform-aware sed in-place. macOS BSD sed needs an empty argument;
85 # GNU sed (Linux) does not. Earlier draft used `-i ''` everywhere and
86 # would have failed on Linux silently.
87 sed_inplace() {
88     local expr="$1"
89     local file="$2"
90     case "$PLATFORM" in
91         Darwin) run sed -i '' "$expr" "$file" ;;
92         *)      run sed -i "$expr" "$file" ;;
93     esac
94 }
95 #
96 # 0. Resolve the ACTIVE physical interface once (used everywhere)
97 # Replaces the earlier `en0` hardcoding. Same algorithm recover.sh uses:
98 # `route get default` if utun/tun, walk en0..en5 first with `status: active`
99 # + `inet` fall back to en0. This means the script also works on hosts
100 # whose primary NIC is en1/en2 (Thunderbolt Ethernet, USB-LAN, etc.).
101 resolve_active_iface() {
102     local iface
103 
```

```

104 iface=$(route get default 2>> "$LOG_FILE" | awk '/interface:/{print $2; exit}')
105 if [ -z "$iface" ] || [ "${iface#tun}" != "$iface" ] || [ "${iface#tun}" != "$iface" ]; then
106     for i in en0 en1 en2 en3 en4 en5; do
107         if ifconfig "$i" 2>> "$LOG_FILE" | grep -q "status: active" \
108             && ifconfig "$i" 2>> "$LOG_FILE" | grep -q "inet "; then
109             iface="$i"; break
110         fi
111     done
112 fi
113 [ -z "$iface" ] && iface="en0"
114 printf '%s\n' "$iface"
115 }
116 ACTIVE_IFACE=$(resolve_active_iface)
117
118 # 1. Detect collision
119 step "1/9 Detecting Docker-bridge vs Wi-Fi subnet collision"
120
121 # Host subnet on the ACTIVE interface (not hardcoded en0).
122 HOST_SUBNET=$(ifconfig "$ACTIVE_IFACE" 2>> "$LOG_FILE" \
123     | awk '/inet /{print $2; exit}')
124 HOST_GATEWAY=$(route get default 2>> "$LOG_FILE" \
125     | awk '/gateway:/{print $2; exit}')
126 DEFAULT_DEV=$(route get default 2>> "$LOG_FILE" \
127     | awk '/interface:/{print $2; exit}')
128
129 printf ' active iface: %s\n' "$ACTIVE_IFACE"
130 printf ' iface inet: %s\n' "${HOST_SUBNET:-none}"
131 printf ' default via: %s\n' "${HOST_GATEWAY:-none}"
132 printf ' default dev: %s\n' "${DEFAULT_DEV:-none}"
133
134 # Collision fires when the host's default-route gateway is 172.17.0.1 (Docker's
135 # bridge) OR when the host subnet is in 172.17.0.0/16. The full-bridge-collision
136 # case (gateway is literally the docker bridge IP) is the smoking gun.
137 COLLISION=false
138 if [ -n "$HOST_GATEWAY" ] && [ "${HOST_GATEWAY#172.17.}" != "$HOST_GATEWAY" ]; then
139     COLLISION=true
140     fail "default gateway $HOST_GATEWAY collides with Docker's default bridge (172.17.0.0/16)"
141 elif [ -n "$HOST_SUBNET" ] && [ "${HOST_SUBNET#172.17.}" != "$HOST_SUBNET" ]; then
142     COLLISION=true
143     fail "host subnet $HOST_SUBNET overlaps Docker's 172.17.0.0/16 bridge"
144 else
145     ok "no collision detected between host Wi-Fi and Docker's default bridge"
146 fi
147
148 if [ "$COLLISION" = "false" ]; then
149     echo ""
150     echo "Nothing to fix. If you still see egress issues, run:"
151     echo "    bash scripts/diagnose-host-leak.sh"
152     exit 0
153 fi
154
155 # 2. Pick non-colliding bip via /8-family routing
156 step "2/9 Picking a non-colliding Docker bip for ~/.colima/default/colima.yaml"
157
158 # Earlier draft tried naive prefix-match: "if candidate's 3-octet prefix is
159 # not a literal prefix of host gateway, pick it". That breaks on /12 campus
160 # networks (172.16.0.0/12 covers 172.16-172.31): a candidate like
161 # 172.26.0.1/24 LIVES INSIDE the host's /12 even though the prefix test
162 # passes. Fix: classify the host's address family FIRST, then choose ALL
163 # candidates from a DIFFERENT RFC1918 family. /8 boundaries are coarse
164 # enough that no /24 picked this way can ever collide.
165 case "$HOST_GATEWAY" in
166     172.16.*|172.17.*|172.18.*|172.19.*|172.20.*|172.21.*|172.22.*|172.23.*|\
167     172.24.*|172.25.*|172.26.*|172.27.*|172.28.*|172.29.*|172.30.*|172.31.*)
168         HOST_FAMILY="172"
169         # Host is in 172.x jump past 172.31 entirely. Pick from 10.x / 192.168.x.
170         CANDIDATES="10.200.0.1/24 10.201.0.1/24 192.168.99.1/24"
171         ;;
172     10.*)
173         HOST_FAMILY="10"
174         # Host is in 10.x pick from 172.x (well outside any commonly-deployed
175         # 10.0.0.0/8 slice) / 192.168.x.
176         CANDIDATES="172.26.0.1/24 172.28.0.1/24 192.168.99.1/24"
177         ;;
178     192.168.*)
179         HOST_FAMILY="192"
180         CANDIDATES="172.26.0.1/24 10.200.0.1/24 10.201.0.1/24"
181         ;;
182     *)
183         HOST_FAMILY="unknown"
184         warn "host gateway $HOST_GATEWAY is not in any RFC1918 range (/8 family)"

```

```

185     warn "falling back to abstract candidate set; verify collision-free manually"
186     CANDIDATES="172.26.0.1/24 10.200.0.1/24 192.168.99.1/24"
187     ;;
188 esac
189
190 printf '  host address family: %s\n' "$HOST_FAMILY"
191 printf '  candidate pool:      %s\n' "$CANDIDATES"
192
193 # Sanity-check the chosen pool has at least one entry that doesn't literally
194 # share a /24 with the host gateway. Naive check (just /24 prefix) is fine
195 # here because we already picked across families.
196 CHOSEN=""
197 for cand in $CANDIDATES; do
198     PREFIX3=$(printf '%s' "$cand" | cut -d. -f1-3)
199     if [ -n "$HOST_GATEWAY" ] && [ "${HOST_GATEWAY#"$PREFIX3"}" != "$HOST_GATEWAY" ]; then
200         warn "skip $cand (literal /24 collision with host gateway $HOST_GATEWAY)"
201         continue
202     fi
203     CHOSEN="$cand"
204     ok "chose bip=$CHOSEN (cross-family, no /24 literal collision)"
205     break
206 done
207
208 if [ -z "$CHOSEN" ]; then
209     CHOSEN="172.26.0.1/24"
210     warn "could not find a clean candidate; defaulting to $CHOSEN"
211     warn "verify with 'route get default' after restart"
212 fi
213
214 # 3. Backup colima.yaml
215 step "3/9 Backing up $COLIMA_YAML"
216 mkdir -p "$(dirname "$COLIMA_YAML")"
217 BACKUP="$HOME/.colima/default/colima.yaml.bak.$TIMESTAMP"
218 if [ -f "$COLIMA_YAML" ]; then
219     # `cp` is safe in dry-run because run() suppresses the inner command
220     # entirely no chance of accidentally writing a real backup.
221     run cp -p "$COLIMA_YAML" "$BACKUP" || die "backup failed"
222     if [ "$DRY_RUN" = "false" ]; then ok "backup: $BACKUP"; fi
223 else
224     warn "no existing $COLIMA_YAML (will create fresh)"
225 fi
226
227 # 4. Edit colima.yaml in place (idempotent)
228 step "4/9 Setting docker.bip=$CHOSEN in colima.yaml (in place)"
229
230 # Resolve the existing docker.bip (if any) without picking up commented-out
231 # YAML keys. awk-based extraction; the negated-comment check (~! /^[[:space:]]*#`)
232 # ensures `# bip: 1.2.3.4/24` is treated as a comment, not the active value.
233 EXISTING_BIP=""
234 if [ -f "$COLIMA_YAML" ]; then
235     EXISTING_BIP=$(awk '
236         /^[[:space:]]*#/{next}
237         /^[[:space:]]*bip:[[:space:]]*/{print $2; exit}
238     ' "$COLIMA_YAML" 2>> "$LOG_FILE" || true)
239 fi
240
241 if [ -n "$EXISTING_BIP" ] && [ "$EXISTING_BIP" = "$CHOSEN" ]; then
242     ok "colima.yaml already has docker.bip=$CHOSEN no edit needed"
243 elif [ -n "$EXISTING_BIP" ]; then
244     printf '  (replacing existing docker.bip=%s with %s)\n' "$EXISTING_BIP" "$CHOSEN"
245     # Use sed_inplace so Linux / macOS get the right syntax. Skip YAML comment
246     # lines so existing `# bip:` comments aren't rewired.
247     sed_inplace "s|~\([[:space:]]*bip:[[:space:]]*\).*\|~1$CHOSEN|" "$COLIMA_YAML" \
248     || die "sed replacement failed (colima.yaml malformed?)"
249     ok "colima.yaml updated"
250 else
251     # No existing `bip:` key. Branch:
252     # - file has `docker:` block append ` bip:` under it
253     # - file has no `docker:` block append new docker: section
254     # - file does not exist create fresh
255     # Each branch is dry-run-aware so no branch can accidentally leak noise
256     # into $COLIMA_YAML during a preview.
257     HAS_DOCKER_BLOCK=false
258     if [ -f "$COLIMA_YAML" ] && grep -q '^docker:' "$COLIMA_YAML"; then
259         HAS_DOCKER_BLOCK=true
260     fi
261
262     if [ "$HAS_DOCKER_BLOCK" = "true" ]; then
263         if [ "$DRY_RUN" = "true" ]; then
264             printf '  [dry-run] would append to %s:\n bip: %s\n' "$COLIMA_YAML" "$CHOSEN"
265         else

```

```

266     printf '\n bip: %s\n' "$CHOSEN" >> "$COLIMA_YAML" \
267     || die "append failed"
268     ok "appended bip under existing docker: block"
269 fi
270 elif [ -f "$COLIMA_YAML" ]; then
271     if [ "$DRY_RUN" = "true" ]; then
272         printf '        [dry-run] would append to %s:\\ndocker:\\n bip: %s\\n' "$COLIMA_YAML" "$CHOSEN"
273     else
274         printf '\\ndocker:\\n bip: %s\\n' "$CHOSEN" >> "$COLIMA_YAML" \
275         || die "append failed"
276         ok "appended new docker: block at end of file"
277     fi
278 else
279     if [ "$DRY_RUN" = "true" ]; then
280         printf '        [dry-run] would create %s with:\\ndocker:\\n bip: %s\\n' "$COLIMA_YAML" "$CHOSEN"
281     else
282         printf 'docker:\\n bip: %s\\n' "$CHOSEN" > "$COLIMA_YAML" \
283         || die "create failed"
284         ok "created fresh $COLIMA_YAML with docker.bip=$CHOSEN"
285     fi
286 fi
287 fi
288
289 # Always show the final colima.yaml docker section for visibility
290 printf '\n%b colima.yaml `docker:` section now reads:%b\n' "$BOLD" "$NC"
291 awk '
292     /^docker:/{p=1}
293     p && /^[^ ]/ && !/^docker:/{exit}
294     p
295 ' "$COLIMA_YAML" 2>/dev/null | sed 's/^ / /' || true
296
297 if [ "$DRY_RUN" = "true" ]; then
298     printf '\n %b(dry-run: exiting before colima restart / dhcp rebind / toggle.sh)%b\n' "$YELLOW" "$NC"
299     exit 0
300 fi
301
302 # 5. Restart Colima
303 step "5/9 Restarting Colima VM with new bip"
304 if ! command -v colima >> "$LOG_FILE" 2>&1; then
305     die "colima not on PATH install with 'brew install colima' first"
306 fi
307 # No timeout wrapper (would mask actual VM startup time). If colima hangs,
308 # the user kills with Ctrl+C and can manually `colima restart` from a fresh
309 # terminal the rest of this script is idempotent so re-running works.
310 colima restart >> "$LOG_FILE" 2>&1 || die "colima restart failed; check $LOG_FILE"
311 ok "colima restarted"
312
313 # 6. Rebind DHCP on the ACTIVE interface (not hardcoded en0)
314 step "6/9 Rebinding DHCP on $ACTIVE_IFACE so host re-learns its real gateway"
315
316 # Map ifname macOS network service name (Wi-Fi, USB 10/100 LAN, etc.).
317 # Same logic as the diagnostic + toggle.sh.
318 ACTIVE_SVC=""
319 if [ "$PLATFORM" = "Darwin" ]; then
320     ACTIVE_SVC=$(get_active_service)
321 fi
322
323 case "$PLATFORM" in
324     Darwin)
325         sudo -n networksetup -setdhcp "$ACTIVE_SVC" renew >> "$LOG_FILE" 2>&1 \
326         && ok "DHCP rebind issued on $ACTIVE_SVC" \
327         || warn "setdhcp renew failed; try 'sudo ipconfig set $ACTIVE_IFACE DHCP' manually"
328         ;;
329     Linux)
330         sudo -n dhclient -r "$ACTIVE_IFACE" >> "$LOG_FILE" 2>&1 || true
331         sudo -n dhclient "$ACTIVE_IFACE" >> "$LOG_FILE" 2>&1 \
332         && ok "dhclient rebind on $ACTIVE_IFACE" \
333         || warn "dhclient rebind failed"
334         ;;
335     *)
336         warn "unsupported platform $PLATFORM; skip DHCP rebind"
337         ;;
338 esac
339
340 # 7. Verify new default route
341 step "7/9 Verifying the new default route is NOT the Docker bridge"
342 sleep 2 # give DHCP + kernel a beat
343 NEW_DEFAULT=$(netstat -rn 2>> "$LOG_FILE" | awk '/^default/ {print $0; exit}')
344 printf ' %s\n' "${NEW_DEFAULT:-no default route detected}"
345 NEW_GATEWAY=$(printf '%s' "$NEW_DEFAULT" | awk '{print $2; exit}')
346

```

```

347 if [ -n "$NEW_GATEWAY" ] && [ "${NEW_GATEWAY#172.17.}" != "$NEW_GATEWAY" ]; then
348     fail "default route STILL points at 172.17.x.x DHCP rebind may need manual touch"
349     warn "try: sudo ipconfig set $ACTIVE_IFACE DHCP (or reconnect Wi-Fi in the menu bar)"
350 elif [ -n "$NEW_GATEWAY" ]; then
351     ok "default route is now via $NEW_GATEWAY (no longer Docker's bridge)"
352 else
353     warn "could not determine new gateway; verify with 'route get default'"
354 fi
355
356 # 8. Re-run toggle.sh to bring the gateway back up
357 if [ "$SKIP_TOGGLE" = "true" ]; then
358     step "8/9 Skipping toggle.sh (--skip-toggle)"
359     warn "gateway is currently down run ./toggle.sh manually when ready"
360 else
361     step "8/9 Re-running toggle.sh to bring the gateway back up"
362     if bash "$PROJECT_ROOT/toggle.sh" 2>&1 | tee -a "$LOG_FILE"; then
363         ok "toggle.sh completed"
364     else
365         warn "toggle.sh returned non-zero see $LOG_FILE for details"
366     fi
367 fi
368
369 # 9. Re-run the diagnostic + show verdict
370 step "9/9 Re-running host-leak diagnostic to confirm fix"
371 if bash "$SCRIPT_DIR/diagnose-host-leak.sh" 2>&1 | tee -a "$LOG_FILE"; then
372     LATEST_REPORT=$(ls -t "$PROJECT_ROOT"/host-leak-diagnostic-*.txt 2>/dev/null | head -1)
373     if [ -n "$LATEST_REPORT" ]; then
374         VERDICT=$(awk '/^ Scenario:/ {print $2; exit}' "$LATEST_REPORT" 2>/dev/null)
375         WARP=$(awk -F': *' '/^ WARP egress:/ {print $2; exit}' "$LATEST_REPORT" 2>/dev/null)
376         printf '\n Most recent verdict: Scenario=%s WARP=%s\n' \
377             "${VERDICT:-?}" "${WARP:-?}"
378         if [ "$VERDICT" = "OK" ]; then
379             ok "fix confirmed by diagnostic host traffic is going through WARP"
380         fi
381     fi
382 else
383     warn "diagnostic exited non-zero; verify manually"
384 fi
385
386 printf '\n%b%b\n' "$BOLD" "$NC"
387 printf '%bDone.%b Full transcript at %s\n' "$BOLD" "$NC" "$LOG_FILE"
388 printf '%b%b\n' "$BOLD" "$NC"
389 exit 0

```

40 scripts/fix-ssh-delay.sh

```

1 #!/bin/bash
2 # scripts/fix-ssh-delay.sh Fix ~20s SSH connection delay caused by UseDNS
3 #
4 # macOS OpenSSH defaults UseDNS to "yes". When Tailscale SSH connects from
5 # a peer (phone/laptop), sshd performs a reverse-DNS (PTR) lookup on the
6 # client's 100.x.x.x Tailscale IP. Since DNS is hijacked to the nullexit
7 # gateway AdGuard Cloudflare WARP, and 100.64.0.0/10 is CGNAT private
8 # space, the PTR query times out (~15-20s) before SSH proceeds.
9 #
10 # This script uncomments "UseDNS no" in /etc/ssh/sshd_config and reloads
11 # sshd. Existing SSH sessions are NOT dropped.
12 #
13 # Usage:
14 #     sudo bash scripts/fix-ssh-delay.sh
15
16 set -euo pipefail
17
18 SSHD_CONFIG="/etc/ssh/sshd_config"
19
20 # Check we're root
21 if [ "$(id -u)" -ne 0 ]; then
22     echo "ERROR: This script must be run with sudo (it edits $SSHD_CONFIG)"
23     echo "Usage: sudo bash scripts/fix-ssh-delay.sh"
24     exit 1
25 fi
26
27 # Apply config (idempotent)
28 if grep -qE '^UseDNS no' "$SSHD_CONFIG" 2>/dev/null; then
29     echo "UseDNS is already set to 'no' in $SSHD_CONFIG"
30 elif grep -qE '^#UseDNS' "$SSHD_CONFIG" 2>/dev/null; then

```



```

31     echo " Uncommenting 'UseDNS no' in $SSHD_CONFIG"
32     sed -i ' ' 's/^#UseDNS no/UseDNS no/' "$SSHD_CONFIG"
33 elif grep -qEi '^UseDNS' "$SSHD_CONFIG" 2>/dev/null; then
34     echo " Changing existing UseDNS line to 'UseDNS no'"
35     sed -i ' ' 's/^UseDNS.*/UseDNS no/' "$SSHD_CONFIG"
36 else
37     echo " Appending 'UseDNS no' to $SSHD_CONFIG"
38     echo "" >> "$SSHD_CONFIG"
39     echo "UseDNS no" >> "$SSHD_CONFIG"
40 fi
41
42 # Verify
43 if ! grep -qE '^UseDNS no' "$SSHD_CONFIG"; then
44     echo " Failed to apply UseDNS no check $SSHD_CONFIG manually"
45     exit 1
46 fi
47
48 # Reload sshd (does NOT drop existing connections)
49 echo " Reloading sshd..."
50
51 SSHD_PID=$(pgrep -f /usr/sbin/sshd 2>/dev/null | head -1 || true)
52 if [ -n "$SSHD_PID" ]; then
53     kill -HUP "$SSHD_PID"
54     echo " Sent SIGHUP to sshd (PID $SSHD_PID) config reloaded"
55 else
56     echo " No running sshd found. macOS starts sshd on-demand per connection."
57     echo " The new 'UseDNS no' config will take effect on the next SSH connection."
58 fi
59
60 echo ""
61 echo " Done. SSH connections should now connect instantly."
62 echo " Existing sessions were not interrupted."

```

41 scripts/generate_tex.py

```

1  #!/usr/bin/env python3
2  import os
3
4  PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))
5
6  IGNORE_DIRS = {'.git', '.github', 'adguard', '__pycache__', 'Recover Gateway.app', 'Toggle Gateway.app', 'Sweep
7  Gateway.app', 'nullexit-knowledge-graph'}
8  IGNORE_FILES = {'.env', 'nullexit_unified.tex', 'nullexit_unified.pdf', 'output.log', 'blocked.log', '.DS_Store
9  ', '.lan_p2p_detected', '.signatures', '.gateway_ip', 'TUNNEL_FAILED_CLOSED.marker', '.host_ips', 'refactor.
10 md', 'sweep.md', '.build_hash', '.rules_hash', 'knowledge-graph.html', '.dns_baseline.json', '.last_wifi_ssid', '.
11 last_wifi_ssid.tmp'}
12 IGNORE_EXTS = {'.pdf', '.tex', '.png', '.jpg', '.jpeg', '.zip', '.tar', '.gz', '.log', '.aux', '.fls', '.
13 fdb_latexmk', '.out', '.toc', '.xdv', '.synctex(busy)'}
14 # Transient runtime reports (timestamped names) carry live egress/peer IPs never
15 # embed them in the published quine. Matched by filename prefix.
16 IGNORE_PREFIXES = ('sweep-', 'host-leak-diagnostic-')
17
18 # Exclude from code block inclusion, but keep in tree
19 SKIP_CONTENT_FILES = {'LICENSE'}
20
21 PRIORITY_FILES = [
22     'README.md',
23     'agent.md',
24     'devref.md',
25     'diagrams.md',
26     'toggle.sh',
27     'recover.sh',
28     'setup.sh',
29     'scripts/setup-linux.sh',
30     'docker-compose.yml',
31     'scripts/common.sh',
32     'scripts/watcher.sh',
33     'scripts/crypto.sh'
34 ]
35
36 def get_language(filename):
37     ext = os.path.splitext(filename)[1].lower()
38     if ext == '.sh' or ext == '.applescript':
39         return 'bash'
40     elif ext == '.py':
41         return 'Python'

```



```

116
117 \section{Project Directory Tree}
118 \begin{verbatim}
119 nullexit/
120 """
121
122     tree = generate_tree(PROJECT_ROOT)
123     midamble = r"""\end{verbatim}
124 \newpage
125 """
126
127 all_relpaths = []
128 for root, dirs, files in os.walk(PROJECT_ROOT):
129     dirs[:] = sorted([d for d in dirs if d not in IGNORE_DIRS])
130     for file in sorted(files):
131         if file in IGNORE_FILES or file in SKIP_CONTENT_FILES or os.path.splitext(file)[1].lower() in
IGNORE_EXTS:
132             continue
133         if file.startswith(IGNORE_PREFIXES):
134             continue
135
136         filepath = os.path.join(root, file)
137         relpath = os.path.relpath(filepath, PROJECT_ROOT)
138         all_relpaths.append(relpath)
139
140 def sort_key(path):
141     if path in PRIORITY_FILES:
142         return (0, PRIORITY_FILES.index(path))
143     return (1, path)
144
145 all_relpaths.sort(key=sort_key)
146
147 with open(tex_path, 'w', encoding='utf-8') as f:
148     f.write(preamble)
149     f.write(tree)
150     f.write(midamble)
151
152     for relpath in all_relpaths:
153         lang = get_language(relpath)
154         lang_attr = f"[language={lang}]" if lang else ""
155
156         f.write(f"\\section{{{escape_tex(relpath)}}}\n")
157         f.write(f"\\begin{{{lstlisting}}}{lang_attr}\n")
158         try:
159             with open(os.path.join(PROJECT_ROOT, relpath), 'r', encoding='utf-8', errors='ignore') as src:
160                 content = src.read()
161                 import re
162                 # Strip all non-ASCII characters (emojis, box drawings, em dashes, etc)
163                 # AND ASCII control characters except \t\n (binary payloads, ANSI escapes,
164                 # form feeds all invisible, all fatal to LaTeX compilation)
165                 content = re.sub(r'[\x09\x0A\x20-\x7E]+' , '', content)
166                 f.write(content)
167                 if not content.endswith('\n'):
168                     f.write('\n')
169             except Exception as e:
170                 f.write(f"Error reading file: {e}\n")
171             f.write("\\end{lstlisting}\n\n")
172
173         f.write("\\end{document}\n")
174
175     print(f"Generated {tex_path}")
176
177 if __name__ == '__main__':
178     main()

```

42 scripts/logger/go.mod

```

1 module nullexit/logger
2
3 go 1.22

```

43 scripts/logger/main.go

```

1 package main
2
3 import (
4     "bufio"
5     "encoding/json"
6     "fmt"
7     "io"
8     "log"
9     "os"
10    "regexp"
11    "strings"
12    "time"
13 )
14
15 const (
16     queryLogPath = "/adguard_work/data/querylog.json"
17     procKmsgPath = "/proc/kmsg"
18     outputLogPath = "/app/blocked.log"
19 )
20
21 func logMessage(msg string) {
22     // Always log in UTC with an explicit marker. The routing-fix image is
23     // Alpine (no tzdata), so time.Now() already fell back to UTC but pin it
24     // so the format can't silently flip to local time if tzdata is ever added.
25     // AdGuard's querylog.json is now UTC too (TZ=UTC in docker-compose.yml).
26     timestamp := time.Now().UTC().Format("2006-01-02 15:04:05") + " UTC"
27     formattedMsg := fmt.Sprintf("%s %s\n", timestamp, msg)
28     fmt.Print(formattedMsg)
29
30     f, err := os.OpenFile(outputLogPath, os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0644)
31     if err != nil {
32         log.Printf("Error writing to log file: %v\n", err)
33         return
34     }
35     defer f.Close()
36     f.WriteString(formattedMsg)
37 }
38
39 func tailFile(filepath string, out chan<- string) {
40     for {
41         if _, err := os.Stat(filepath); os.IsNotExist(err) {
42             time.Sleep(1 * time.Second)
43             continue
44         }
45         break
46     }
47
48     f, err := os.Open(filepath)
49     if err != nil {
50         log.Printf("Error opening file %s: %v\n", filepath, err)
51         return
52     }
53     defer func() {
54         if f != nil {
55             f.Close()
56         }
57     }()
58
59     // Seek to end
60     f.Seek(0, io.SeekEnd)
61     reader := bufio.NewReader(f)
62
63     for {
64         line, err := reader.ReadString('\n')
65         if err != nil {
66             if err == io.EOF {
67                 stat, err := os.Stat(filepath)
68                 if err == nil {
69                     currentOffset, _ := f.Seek(0, io.SeekCurrent)
70                     if stat.Size() < currentOffset {
71                         // File truncated or rotated
72                         f.Close()
73                         for {
74                             if _, err := os.Stat(filepath); os.IsNotExist(err) {
75                                 time.Sleep(1 * time.Second)
76                                 continue
77                             }
78                             break
79                         }
80                     }
79                     f, _ = os.Open(filepath)

```

```

81         reader = bufio.NewReader(f)
82         continue
83     }
84 }
85     time.Sleep(500 * time.Millisecond)
86     continue
87 }
88     log.Printf("Error reading file %s: %v\n", filepath, err)
89     time.Sleep(1 * time.Second)
90     continue
91 }
92     out <- line
93 }
94 }
95
96 type adguardLogEntry struct {
97     IP      string `json:"IP"`
98     QH      string `json:"QH"`
99     QT      string `json:"QT"`
100     Result struct {
101         IsFiltered bool `json:"IsFiltered"`
102         Reason      int  `json:"Reason"`
103         Rules       []struct {
104             Text string `json:"Text"`
105         } `json:"Rules"`
106     } `json:"Result"`
107 }
108
109 func monitorDNS() {
110     logMessage("[System] Starting DNS block logger...")
111     lines := make(chan string)
112     go tailFile(queryLogPath, lines)
113
114     reasonMap := map[int]string{
115         2: "CustomRule",
116         3: "BlockList",
117         4: "SafeBrowsing",
118         5: "ParentalControl",
119         6: "SafeSearch",
120         7: "BlockedService",
121     }
122
123     for line := range lines {
124         var data adguardLogEntry
125         if err := json.Unmarshal([]byte(line), &data); err != nil {
126             continue
127         }
128
129         res := data.Result
130         if res.IsFiltered || (res.Reason != 0 && res.Reason != 1) {
131             qh := data.QH
132             if qh == "" {
133                 qh = "unknown"
134             }
135             qt := data.QT
136             if qt == "" {
137                 qt = "unknown"
138             }
139             clientIP := data.IP
140             if clientIP == "" {
141                 clientIP = "unknown"
142             }
143
144             ruleText := "unknown rule"
145             if len(res.Rules) > 0 && res.Rules[0].Text != "" {
146                 ruleText = res.Rules[0].Text
147             }
148
149             reasonStr, ok := reasonMap[res.Reason]
150             if !ok {
151                 reasonStr = fmt.Sprintf("Reason-%d", res.Reason)
152             }
153
154             logMessage(fmt.Sprintf("[DNS] Blocked %s (Type: %s) for client %s | Reason: %s | Rule: %s", qh, qt,
155                 clientIP, reasonStr, ruleText))
156         }
157     }
158
159 func monitorIPs() {
160     logMessage("[System] Starting IP block logger...")

```

```

161
162 prefixRe := regexp.MustCompile(`(IP_BLOCK_[A-Z_]+):`)
163 srcRe := regexp.MustCompile(`SRC=([0-9a-fA-F\.:]+)`)
164 dstRe := regexp.MustCompile(`DST=([0-9a-fA-F\.:]+)`)
165 protoRe := regexp.MustCompile(`PROTO=([A-Z0-9]+)`)
166 sptRe := regexp.MustCompile(`SPT=(\d+)`)
167 dptRe := regexp.MustCompile(`DPT=(\d+)`)
168 inRe := regexp.MustCompile(`IN=([a-zA-Z0-9\.\_]+)`)
169 outRe := regexp.MustCompile(`OUT=([a-zA-Z0-9\.\_]+)`)
170
171 f, err := os.Open(procKmsgPath)
172 if err != nil {
173     if os.IsPermission(err) {
174         logMessage("[System] ERROR: Insufficient permissions to read /proc/kmsg. Enable CAP_SYSLOG.")
175     } else {
176         logMessage(fmt.Sprintf("[System] ERROR in IP logger: %v", err))
177     }
178     return
179 }
180 defer f.Close()
181
182 reader := bufio.NewReader(f)
183 for {
184     line, err := reader.ReadString('\n')
185     if err != nil {
186         if err == io.EOF {
187             time.Sleep(100 * time.Millisecond)
188             continue
189         }
190         logMessage(fmt.Sprintf("[System] ERROR reading kmsg: %v", err))
191         time.Sleep(1 * time.Second)
192         continue
193     }
194
195     if strings.Contains(line, "IP_BLOCK") {
196         match := prefixRe.FindStringSubmatch(line)
197         if len(match) > 1 {
198             prefix := match[1]
199
200             src := "unknown"
201             if m := srcRe.FindStringSubmatch(line); len(m) > 1 {
202                 src = m[1]
203             }
204
205             dst := "unknown"
206             if m := dstRe.FindStringSubmatch(line); len(m) > 1 {
207                 dst = m[1]
208             }
209
210             proto := "unknown"
211             if m := protoRe.FindStringSubmatch(line); len(m) > 1 {
212                 proto = m[1]
213             }
214
215             spt := ""
216             if m := sptRe.FindStringSubmatch(line); len(m) > 1 {
217                 spt = m[1]
218             }
219
220             dpt := ""
221             if m := dptRe.FindStringSubmatch(line); len(m) > 1 {
222                 dpt = m[1]
223             }
224
225             inIf := ""
226             if m := inRe.FindStringSubmatch(line); len(m) > 1 {
227                 inIf = m[1]
228             }
229
230             outIf := ""
231             if m := outRe.FindStringSubmatch(line); len(m) > 1 {
232                 outIf = m[1]
233             }
234
235             direction := "inbound"
236             if strings.HasSuffix(prefix, "DST") {
237                 direction = "outbound"
238             }
239
240             listType := "MALICIOUS"
241             if !strings.Contains(prefix, "MALICIOUS") {

```

```

242     parts := strings.Split(prefix, "_")
243     if len(parts) >= 3 {
244         listType = "GEO_" + parts[2]
245     }
246 }
247
248 portStr := ""
249 if dpt != "" {
250     portStr = ":" + dpt
251 }
252
253 srcPortStr := ""
254 if spt != "" {
255     srcPortStr = ":" + spt
256 }
257
258 ifStr := ""
259 if inIf != "" && outIf != "" {
260     ifStr = fmt.Sprintf("IF: %s->%s", inIf, outIf)
261 } else if inIf != "" || outIf != "" {
262     ifStr = fmt.Sprintf("IF: %s%s", inIf, outIf)
263 }
264
265 logMessage(fmt.Sprintf("[IP] Blocked %s to %s%s (Proto: %s) from %s%s (%s) | List: %s", direction, dst,
portStr, proto, src, srcPortStr, ifStr, listType))
266 }
267 }
268 }
269 }
270
271 func main() {
272     logMessage("[System] Nullexit Block Logger starting...")
273
274     go monitorDNS()
275     monitorIPs()
276 }

```

44 scripts/noise.sh

```

1  #!/bin/bash
2  # scripts/noise.sh nullexit verifiable-random noise + cover-traffic padding
3  #
4  # OPT-IN and ACTIVE. Unlike sweep.sh (read-only), this module PUTS TRAFFIC ON
5  # THE WIRE. It stays completely inert unless NOISE_ENABLED=true in .env, so the
6  # default posture is unchanged and the sweep's read-only guarantee is intact.
7  #
8  # Two jobs, one primitive (CSPRNG bytes from the OS, /dev/urandom via os.urandom):
9  #
10 # 1. verify prove a freshly generated buffer is statistically random using
11 # three standard tests (Shannon entropy, NIST monobit, chi-square over the
12 # 256 byte values). This is the "we can verify it's truly random
13 # mathematically" requirement it is harmless and always allowed.
14 #
15 # 2. pad a cover-traffic daemon. It emits verified-random UDP datagrams
16 # toward the exit at a configured rate + jitter so a passive on-path
17 # observer (residence / campus / hostile network) sees a continuously
18 # varying encrypted flow instead of your real traffic envelope. This is
19 # the "dummy padding" defence the Threat Model flags as the countermeasure
20 # to metadata / traffic-flow correlation. It ONLY runs with NOISE_ENABLED=true.
21 #
22 # HONEST LIMITS: this is constant/random one-way padding, not a full
23 # traffic-analysis-resistant shaper. It blurs volume + timing on the uplink; it
24 # does not mimic a specific protocol, pad bidirectionally, or defeat a global
25 # active adversary. It costs bandwidth and (on battery devices) power hence
26 # opt-in, modest default rate. The DNS/IP anomaly detector is deliberately NOT
27 # here (left for later).
28 #
29 # Usage:
30 #   bash scripts/noise.sh verify [BYTES]    # randomness self-test (default 1 MiB)
31 #   bash scripts/noise.sh start              # launch the padding daemon (background)
32 #   bash scripts/noise.sh pad                # run the padding loop in the foreground
33 #   bash scripts/noise.sh status             # is the daemon running? target + rate
34 #   bash scripts/noise.sh stop               # stop the daemon
35 #   bash scripts/noise.sh --help            # this message
36 #
37 # Persistence: install launchd/com.nullexit.noise.plist (see README) so padding

```



```

38 # auto-starts at boot/login. It is governed by the SAME NOISE_ENABLED switch no
39 # extra knob: the launchd job is a clean no-op whenever NOISE_ENABLED=false.
40 #
41 # .env knobs (all optional; sane defaults):
42 # NOISE_ENABLED          master on/off switch (default false)
43 # NOISE_PAD_TARGET       where to aim padding (default: the gateway Tailscale IP)
44 # NOISE_PAD_PORT         UDP port on the target (default 9999; no listener needed)
45 # NOISE_PAD_RATE_KBPS    target padding bitrate (default 64)
46 # NOISE_PAD_PACKET_BYTES datagram payload size (default 1024)
47 # NOISE_PAD_JITTER_MS    +/- timing jitter per packet (default 40)
48 # NOISE_PAD_MODE         'constant' = always emit the target rate (default);
49 #                         'topup' = measure real egress each second and inject
50 #                         only the deficit, so the OBSERVED total stays flat
51 #                         whether idle or busy (stronger anti-correlation, no
52 #                         wasted bandwidth when you're already sending)
53 # NOISE_PAD_PCT           set the target as a % of LINK CAPACITY (overrides
54 #                         NOISE_PAD_RATE_KBPS). % of capacity, NOT of current
55 #                         throughput which would drop padding to zero when idle
56 # NOISE_LINK_MBPS        link capacity for NOISE_PAD_PCT (default 100)
57 # NOISE_PAD_IFACE        interface topup measures egress on (default en0)
58
59 set -uo pipefail
60
61 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
62 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
63 cd "$PROJECT_ROOT" || { echo "[FATAL] cannot cd to $PROJECT_ROOT" >&2; exit 1; }
64 source "$SCRIPT_DIR/common.sh"
65 export PATH="/opt/homebrew/bin:/usr/local/bin:$PATH"
66
67 SELF="$SCRIPT_DIR/noise.sh"
68 NOISE_PID="/tmp/nullexit-noise.pid"
69 NOISE_LOG="$PROJECT_ROOT/output.log"
70
71 # .env-driven config
72 NOISE_ENABLED=$(read_env_var "NOISE_ENABLED" | tr '[:upper:]' '[:lower:]')
73 [ -z "$NOISE_ENABLED" ] && NOISE_ENABLED="false"
74
75 PAD_TARGET=$(read_env_var "NOISE_PAD_TARGET")
76 [ -z "$PAD_TARGET" ] && PAD_TARGET=$(read_adguard_ip) # gateway Tailscale IP
77 PAD_PORT=$(read_env_var "NOISE_PAD_PORT"); [ -z "$PAD_PORT" ] && PAD_PORT=9999
78 PAD_RATE=$(read_env_var "NOISE_PAD_RATE_KBPS"); [ -z "$PAD_RATE" ] && PAD_RATE=64
79 PAD_BYTES=$(read_env_var "NOISE_PAD_PACKET_BYTES"); [ -z "$PAD_BYTES" ] && PAD_BYTES=1024
80 PAD_JITTER=$(read_env_var "NOISE_PAD_JITTER_MS"); [ -z "$PAD_JITTER" ] && PAD_JITTER=40
81 # 'constant' = always emit the target rate. 'topup' = measure real egress each
82 # second and inject only the deficit, so the OBSERVED total rate stays flat
83 # whether you're idle or busy (the strong anti-correlation property).
84 PAD_MODE=$(read_env_var "NOISE_PAD_MODE" | tr '[:upper:]' '[:lower:]'); [ -z "$PAD_MODE" ] && PAD_MODE=constant
85 PAD_IFACE=$(read_env_var "NOISE_PAD_IFACE"); [ -z "$PAD_IFACE" ] && PAD_IFACE=en0
86 # Target rate: NOISE_PAD_PCT (a % of LINK CAPACITY a stable reference, NOT of
87 # current throughput, which would collapse padding to zero when idle) overrides
88 # the absolute NOISE_PAD_RATE_KBPS when set.
89 PAD_PCT=$(read_env_var "NOISE_PAD_PCT")
90 PAD_LINK_MBPS=$(read_env_var "NOISE_LINK_MBPS"); [ -z "$PAD_LINK_MBPS" ] && PAD_LINK_MBPS=100
91 if [ -n "$PAD_PCT" ]; then
92     PAD_RATE=$(awk "BEGIN{printf \"%.0f\\", ($PAD_PCT/100.0)*$PAD_LINK_MBPS*1000}")
93 fi
94
95 log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] [noise] $" >> "$NOISE_LOG" 2>/dev/null || true; }
96
97 require_enabled() {
98     if [ "$NOISE_ENABLED" != "true" ]; then
99         echo "noise: disabled (NOISE_ENABLED != true in .env) refusing to emit traffic." >&2
100        echo "        Set NOISE_ENABLED=true to arm cover-traffic padding." >&2
101        exit 3
102    fi
103 }
104
105 # Randomness self-test (stdlib python3; no third-party deps)
106 # Exits 0 (PASS) / 1 (FAIL). Prints a human summary unless $2 == "quiet".
107 verify_random() {
108     local nbytes="${1:-1048576}" mode="${2:-loud}"
109     python3 - "$nbytes" "$mode" <<'PY'
110 import os, sys, math
111 from collections import Counter
112
113 n = int(sys.argv[1])
114 mode = sys.argv[2] if len(sys.argv) > 2 else "loud"
115 buf = os.urandom(n)
116 bits = n * 8
117
118 # 1) Shannon entropy over byte values (bits/byte, ideal = 8.0)

```

```

119 c = Counter(buf)
120 H = -sum((v / n) * math.log2(v / n) for v in c.values())
121
122 # 2) NIST monobit: 1-bit count should be ~half. z = (ones - bits/2)/sqrt(bits/4)
123 ones = sum(bin(b).count("1") for b in buf)
124 z = (ones - bits / 2) / math.sqrt(bits / 4)
125
126 # 3) Chi-square over the 256 byte values (df = 255, mean 255, sd ~22.6)
127 exp = n / 256.0
128 chi2 = sum((c.get(i, 0) - exp) ** 2 / exp for i in range(256))
129
130 # PASS windows chosen wide enough that true CSPRNG output passes -always,
131 # yet a broken/patterned source (constant, low-entropy, biased) fails hard.
132 H_OK = H >= 7.99 if n >= 1 << 16 else H >= 7.5
133 Z_OK = abs(z) < 5.0
134 CHI_OK = 185.0 < chi2 < 345.0
135 ok = H_OK and Z_OK and CHI_OK
136
137 if mode != "quiet":
138     print(f"    sample      : {n} bytes ({bits} bits) from os.urandom (/dev/urandom CSPRNG)")
139     print(f"    shannon H    : {H:.5f} bits/byte (ideal 8.0) [{ 'ok' if H_OK else 'FAIL' }]")
140     print(f"    monobit z    : {z:+.3f} (|z|<5, bias {ones/bits*100:.3f}% ones) [{ 'ok' if Z_OK else 'FAIL' }]")
141     print(f"    chi-square   : {chi2:.1f} (df=255, expect ~255, window 185-345) [{ 'ok' if CHI_OK else 'FAIL' }]")
142     print(f"    verdict      : [{ 'PASS statistically random' if ok else 'FAIL not random enough' }]")
143 sys.exit(0 if ok else 1)
144 PY
145 }
146
147 # The cover-traffic loop (foreground; `start` backgrounds this)
148 pad_loop() {
149     require_enabled
150     # Own the PID file for BOTH run modes (manual `start` and launchd `__boot`), so
151     # `status`/`stop` behave identically however padding was launched. Cleared on exit.
152     echo $$ > "$NOISE_PID"
153     trap 'rm -f "$NOISE_PID"' EXIT INT TERM
154     # Gate: only ever emit noise we have just proven is statistically random.
155     if ! verify_random 1048576 quiet; then
156         echo "noise: randomness self-test FAILED refusing to emit non-random padding." >&2
157         log "pad aborted: randomness self-test failed"
158         exit 1
159     fi
160     log "pad start ${PAD_TARGET}:${PAD_PORT} mode=${PAD_MODE} rate=${PAD_RATE}kbps pkt=${PAD_BYTES}B jitter=${PAD_JITTER}ms iface=${PAD_IFACE}"
161     python3 - "$PAD_TARGET" "$PAD_PORT" "$PAD_RATE" "$PAD_BYTES" "$PAD_JITTER" "$PAD_MODE" "$PAD_IFACE" <<'PY'
162 import os, sys, socket, time, random, subprocess
163
164 target = sys.argv[1]
165 port = int(sys.argv[2])
166 target_bps = float(sys.argv[3]) * 1000.0 / 8.0 # kbps bytes/sec (target TOTAL on the wire)
167 pkt = int(sys.argv[4])
168 jitter = float(sys.argv[5]) / 1000.0 # ms sec
169 mode = sys.argv[6] # 'constant' | 'topup'
170 iface = sys.argv[7]
171
172 s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
173
174
175 def obytes(ifc):
176     """Cumulative output bytes for an interface what an on-path observer sees
177     leave the NIC. Returns None if the counter can't be read (fail toward MORE cover)."""
178     try:
179         out = subprocess.run(["netstat", "-ibn", "-I", ifc],
180                               capture_output=True, text=True, timeout=2).stdout
181         for line in out.splitlines():
182             f = line.split()
183             # the "<Link#N>" row carries cumulative counters; Obytes is column 9
184             if len(f) >= 11 and f[0] == ifc and f[2].startswith("<Link"):
185                 return int(f[9])
186     except Exception:
187         pass
188     return None
189
190
191 def emit(budget_bytes):
192     """Send ~budget_bytes of fresh CSPRNG padding, spread across ~1s with jitter.
193     Returns bytes actually sent (so topup can subtract its own contribution)."""
194     n = max(0, int(budget_bytes // pkt))
195     interval = 1.0 / n if n > 0 else 1.0
196     sent = 0

```

```

197     for _ in range(n):
198         try:
199             s.sendto(os.urandom(pkt), (target, port)); sent += pkt
200         except OSError:
201             pass # dropped is fine the bytes still left the NIC
202         time.sleep(max(0.0, interval + random.uniform(-jitter, jitter)))
203     return sent
204
205
206 try:
207     prev = obytes(iface) if mode == "topup" else None
208     pad_last = 0
209     while True:
210         t0 = time.time()
211         if mode == "topup":
212             now = obytes(iface)
213             if prev is not None and now is not None:
214                 total = max(0, now - prev) # all egress on iface this window
215                 real = max(0, total - pad_last) # minus our own last-window padding
216                 budget = max(0.0, target_bps - real)
217             else:
218                 budget = target_bps # counters unavailable full target
219             prev = now
220         else: # constant
221             budget = target_bps
222             pad_last = emit(budget)
223             rem = 1.0 - (time.time() - t0) # keep a ~1s accounting window
224             if rem > 0:
225                 time.sleep(rem)
226 except KeyboardInterrupt:
227     pass
228 PY
229 }
230
231 daemon_running() {
232     [ -f "$NOISE_PID" ] || return 1
233     local pid; pid="$(cat "$NOISE_PID" 2>/dev/null)"
234     [ -n "$pid" ] && kill -0 "$pid" 2>/dev/null
235 }
236
237 cmd_start() {
238     require_enabled
239     if daemon_running; then
240         echo "noise: padding daemon already running (pid $(cat "$NOISE_PID"))."
241         exit 0
242     fi
243     echo "noise: verifying randomness before arming padding"
244     verify_random 1048576 loud || { echo "noise: aborting self-test failed." >&2; exit 1; }
245     nohup "$SELF" pad >>"$NOISE_LOG" 2>&1 &
246     local npid=$!
247     sleep 0.3 # let pad_loop write its PID file so `status` is immediately accurate
248     echo "noise: padding daemon armed (pid $npid) ${PAD_TARGET}:${PAD_PORT}, ${PAD_RATE} kbps."
249 }
250
251 cmd_stop() {
252     if daemon_running; then
253         local pid; pid="$(cat "$NOISE_PID")"
254         kill "$pid" 2>/dev/null && echo "noise: stopped padding daemon (pid $pid)."
255         log "pad stop (pid $pid)"
256     else
257         echo "noise: no padding daemon running."
258     fi
259     rm -f "$NOISE_PID"
260 }
261
262 cmd_status() {
263     echo "NOISE_ENABLED = $NOISE_ENABLED"
264     if daemon_running; then
265         echo "padding : RUNNING (pid $(cat "$NOISE_PID"))"
266         echo "target : ${PAD_TARGET}:${PAD_PORT}"
267         echo "mode : ${PAD_MODE}${[ "$PAD_MODE" = topup ] && echo " (fills to target on ${PAD_IFACE};
padding backs off as real traffic rises)"}"
268         echo "rate / packet : ${PAD_RATE} kbps${PAD_PCT:+ (=${PAD_PCT}% of ${PAD_LINK_MBPS} Mbps)} / ${PAD_BYTES} B
, +/-${PAD_JITTER} ms jitter"
269     else
270         echo "padding : stopped"
271     fi
272 }
273
274 # launchd entry point (see launchd/com.nullexit.noise.plist). Governed by the
275 # SAME NOISE_ENABLED switch NOT a new option. When disabled it exits 0 cleanly

```

```

276 # so KeepAlive never tight-loops; when enabled it runs the padding loop in the
277 # foreground so launchd supervises it directly. This is why persistence adds no
278 # extra knob: loading the plist once means "auto-start whenever NOISE_ENABLED=true".
279 cmd_boot() {
280     if [ "$NOISE_ENABLED" != "true" ]; then
281         log "boot: NOISE_ENABLED != true idle (padding not started)"
282         exit 0
283     fi
284     log "boot: NOISE_ENABLED=true starting padding loop under launchd"
285     pad_loop
286 }
287
288 case "${1:-}" in
289     verify)      verify_random "${2:-1048576}" loud ;;
290     start)       cmd_start ;;
291     pad)         pad_loop ;;
292     __boot)      cmd_boot ;;
293     stop)        cmd_stop ;;
294     status)      cmd_status ;;
295     --help|-h|"") sed -n '2,59p' "$0" ;;
296     *) echo "Unknown argument: $1 (try --help)" >&2; exit 2 ;;
297 esac

```

45 scripts/pf.conf

```

1 # scripts/pf.conf - Packet Filter configuration for nullexit killswitch
2 # Dynamically parameterized by pfctl macro override
3
4 # ext_if must be passed via command line, e.g. -D ext_if=en0
5 # If not passed, default to en0
6 ext_if = "en0"
7 ts_if = "utun+"
8
9 # Tables populated dynamically via pfctl
10 table <vpn_endpoints> persist
11 table <derp_relays> persist
12 # FaceTime split-route (opt-in): Apple media/relay /16s that egress DIRECT via the
13 # physical gateway. EMPTY unless FACETIME_SPLIT_ROUTE=true, so the kill-switch
14 # posture is unchanged by default. Populated by add_apple_split_routes().
15 table <apple_direct> persist
16
17 # Core Rules
18 set skip on lo0 # Ensure local loopback is never blocked
19 # TCP MSS Clamping: prevents fragmentation on host TCP flows (like Tailscale control plane)
20 # by keeping the host in sync with the container's double-tunnel (Tailscale+WARP) overhead.
21 # NOTE: this 1120 integer is rewritten at kill-switch enable time by scripts/common.sh's
22 # enable_killswitch() from GATEWAY_MSS in .env change GATEWAY_MSS in .env, not this line.
23 scrub out all max-mss 1120
24 pass quick on $ts_if all # Allow all Tailscale traffic (essential to prevent SSH lockout)
25
26 # Default deny outbound WAN traffic
27 block out on $ext_if all
28
29 # Block inbound Screen Sharing (VNC) and SSH from local Wi-Fi, forcing them to use Tailscale
30 block in on $ext_if proto tcp from any to any port { 22, 5900 }
31
32 # Exceptions for tunnel handshakes and coordination
33 pass out quick on $ext_if proto udp to <vpn_endpoints> port 2408
34 pass out quick on $ext_if proto udp to <derp_relays>
35 pass out quick on $ext_if proto tcp to 192.200.0.0/24 port 443
36
37 # FaceTime split-route (opt-in, table empty unless FACETIME_SPLIT_ROUTE=true):
38 # let real-time media/relay traffic to Apple's /16s leave DIRECT (real IP) so
39 # FaceTime can hole-punch instead of dying behind WARP's symmetric NAT.
40 pass out quick on $ext_if proto { tcp udp } to <apple_direct>
41
42 # Allow local LAN traffic (AirDrop, local Tailscale P2P, etc)
43 pass out quick on $ext_if proto tcp to { 192.168.0.0/16, 10.0.0.0/8, 172.16.0.0/12 }
44 pass out quick on $ext_if proto udp to { 192.168.0.0/16, 10.0.0.0/8, 172.16.0.0/12 }
45 pass out quick on $ext_if proto icmp to { 192.168.0.0/16, 10.0.0.0/8, 172.16.0.0/12 }
46
47 # Allow DHCP outbound (client port 68 to server port 67) so we can obtain/renew IP address
48 pass out quick on $ext_if proto udp from any port 68 to any port 67

```

46 scripts/pihole-mode.sh

```
1 #!/bin/bash
2 # scripts/pihole-mode.sh LAN Pi-hole mode (opt-in)
3 #
4 # nullexit is already a Pi-hole for MESH devices: AdGuard Home sinkholes DNS for
5 # everything that routes through the gateway over Tailscale. This mode extends
6 # that to NON-Tailscale devices on the same LAN a smart TV, a guest laptop, an
7 # IoT gadget the classic "point your DNS at this box" Pi-hole. It binds a DNS
8 # forwarder on the LAN interface that relays to AdGuard, so any device that sets
9 # its DNS server to this Mac's LAN IP gets the same ad/tracker/threat filtering.
10 #
11 # Mechanism: reuse the tested dns-proxy.py, but LISTEN_ADDR = the LAN IP instead
12 # of loopback, forwarding to AdGuard at 127.0.0.1:${DNS_PROXY_PORT} (the Docker
13 # port mapping). No kill-switch changes are needed pf.conf has no inbound-53
14 # block, and DNS responses to RFC1918 LAN clients are already permitted.
15 #
16 # NETWORK REALITY: on an AP-isolated network (WPA2-Enterprise / hotel), other
17 # devices CANNOT reach this Mac at all the same isolation that forces phones
18 # onto DERP. This mode only does anything useful on a TRUSTED, non-isolated
19 # network (home / your own AP). It is report-clear here but won't serve remote
20 # clients until you're on such a network. It also filters DNS only it does NOT
21 # route those devices' traffic through WARP.
22 #
23 # Usage:
24 #   bash scripts/pihole-mode.sh enable    # start the LAN DNS forwarder (needs PIHOLE_LAN_MODE=true)
25 #   bash scripts/pihole-mode.sh disable  # stop it
26 #   bash scripts/pihole-mode.sh status    # running? listen addr, AdGuard reachability
27 #   bash scripts/pihole-mode.sh test     # prove it resolves AND sinkholes, locally
28 #
29 # .env:
30 #   PIHOLE_LAN_MODE      master on/off (default false)
31 #   PIHOLE_LAN_LISTEN    LAN address to bind (blank = this Mac's en0 IP)
32 #   DNS_PROXY_PORT       AdGuard's host-published DNS port (default 5354)
33
34 set -uo pipefail
35
36 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
37 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
38 cd "$PROJECT_ROOT" || { echo "[FATAL] cannot cd to $PROJECT_ROOT" >&2; exit 1; }
39 source "$SCRIPT_DIR/common.sh"
40 export PATH="/opt/homebrew/bin:/usr/local/bin:$PATH"
41
42 DNS_PROXY="$SCRIPT_DIR/dns-proxy.py"
43 PID_FILE="/tmp/nullexit-pihole.pid"
44 LOG_FILE="$PROJECT_ROOT/output.log"
45
46 PIHOLE_ON=$(read_env_var "PIHOLE_LAN_MODE" | tr '[:upper:]' '[:lower:]'); [ -z "$PIHOLE_ON" ] && PIHOLE_ON="false"
47
48 # AdGuard's DNS host port is PROCEDURAL (randomized per boot see Procedural
49 # Ports), so discover it; never assume 5354. Docker is the source of truth, then
50 # the ports file toggle.sh writes, then the compose default.
51 resolve_adguard_port() {
52     local p
53     p=$(docker compose port warp 5335 2>/dev/null | grep -oE '[0-9]+' | head -1)
54     [ -n "$p" ] && { echo "$p"; return; }
55     p=$(grep -oE 'DNS_PROXY_PORT=[0-9]+' /tmp/nullexit-ports.env 2>/dev/null | grep -oE '[0-9]+' | head -1)
56     [ -n "$p" ] && { echo "$p"; return; }
57     echo 5354
58 }
59 ADGUARD_PORT=$(resolve_adguard_port)
60
61 lan_ip() {
62     local a; a=$(read_env_var "PIHOLE_LAN_LISTEN")
63     [ -n "$a" ] && { echo "$a"; return; }
64     ipconfig getifaddr en0 2>/dev/null || echo ""
65 }
66
67 log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] [pihole-lan] $" >> "$LOG_FILE" 2>/dev/null || true; }
68
69 running() {
70     [ -f "$PID_FILE" ] || return 1
71     local p; p=$(cat "$PID_FILE" 2>/dev/null)
72     [ -n "$p" ] && kill -0 "$p" 2>/dev/null
73 }
74
75 adguard_reachable() {
76     # Probe over TCP: Colima user-mode net forwards UDP-to-container unreliably (the
```

```

77 # very reason dns-proxy.py relays over TCP), so a UDP probe false-negatives even
78 # when AdGuard is healthy. TCP is exactly the path the forwarder uses.
79 dig +tcp +short +tries=1 +time=2 -p "$ADGUARD_PORT" @127.0.0.1 example.com >/dev/null 2>&1
80 }
81
82 cmd_enable() {
83     if [ "$PIHOLE_ON" != "true" ]; then
84         echo "pihole-lan: disabled (PIHOLE_LAN_MODE != true in .env) refusing to serve LAN DNS." >&2
85         exit 3
86     fi
87     if running; then echo "pihole-lan: already running (pid $(cat "$PID_FILE"))."; exit 0; fi
88     local addr; addr=$(lan_ip)
89     if [ -z "$addr" ]; then echo "pihole-lan: could not determine a LAN IP (en0 down?)." >&2; exit 1; fi
90     if ! adguard_reachable; then
91         echo "pihole-lan: AdGuard not reachable at 127.0.0.1:$ADGUARD_PORT is the gateway up?" >&2
92         exit 1
93     fi
94     echo "pihole-lan: starting LAN DNS forwarder on ${addr}:53 AdGuard 127.0.0.1:$ADGUARD_PORT"
95     # :53 needs root. The shipped sudoers NOPASSWD rule only covers the literal
96     # `python3 dns-proxy.py` command no bash -c wrapper, no env prefix so
97     # config is dropped in a file dns-proxy.py reads itself, not via env vars
98     # that `sudo -n` would strip. See dns-proxy.py's own comment for why.
99     {
100         echo "LISTEN_ADDR=$addr"
101         echo "LISTEN_PORT=53"
102         echo "TARGET_HOST=127.0.0.1"
103         echo "TARGET_PORT=$ADGUARD_PORT"
104     } > /tmp/nullexit-dns-proxy.conf
105     chmod 600 /tmp/nullexit-dns-proxy.conf 2>/dev/null || true
106     nohup sudo -n "$(command -v python3)" "$DNS_PROXY" >>"$LOG_FILE" 2>&1 &
107     disown "$!" 2>/dev/null || true
108     echo "$!" > "$PID_FILE"
109     sleep 0.4
110     if running; then
111         log "enabled on ${addr}:53 127.0.0.1:$ADGUARD_PORT"
112         echo "pihole-lan: running (pid $(cat "$PID_FILE")). Point a LAN device's DNS at ${addr} to filter it."
113     else
114         echo "pihole-lan: listener did not stay up check $LOG_FILE (port 53 already bound?)." >&2; exit 1
115     fi
116 }
117
118 cmd_disable() {
119     if running; then
120         local p; p=$(cat "$PID_FILE")
121         sudo -n kill "$p" 2>/dev/null || kill "$p" 2>/dev/null
122         log "disabled (pid $p)"; echo "pihole-lan: stopped (pid $p)."
123     else
124         echo "pihole-lan: not running."
125     fi
126     sudo -n rm -f "$PID_FILE" 2>/dev/null || rm -f "$PID_FILE" 2>/dev/null || true
127     rm -f /tmp/nullexit-dns-proxy.conf 2>/dev/null || true
128 }
129
130 cmd_status() {
131     echo "PIHOLE_LAN_MODE = $PIHOLE_ON"
132     echo "AdGuard (127.0.0.1:$ADGUARD_PORT): ${adguard_reachable} && echo reachable || echo UNREACHABLE)"
133     if running; then
134         echo "forwarder      : RUNNING (pid $(cat "$PID_FILE")) on $(lan_ip):53"
135     else
136         echo "forwarder      : stopped"
137     fi
138 }
139
140 cmd_test() {
141     local addr; addr=$(lan_ip)
142     if ! running; then echo "pihole-lan: not running 'enable' first." >&2; exit 1; fi
143     echo "Testing LAN Pi-hole at ${addr}:53 (querying locally)"
144     local good bad
145     good=$(dig +short +tries=1 +time=3 @"$addr" example.com 2>/dev/null | head -1)
146     bad=$(dig +short +tries=1 +time=3 @"$addr" doubleclick.net 2>/dev/null | head -1)
147     echo "  resolve example.com      ${good:-<none>}"
148     echo "  filter doubleclick.net    ${bad:-<blocked/empty>}"
149     if [ -n "$good" ] && [ [ -z "$bad" ] || [ "$bad" = "0.0.0.0" ] ]; then
150         echo "  RESULT: PASS resolves clean domains and sinkholes blocked ones."
151     else
152         echo "  RESULT: check clean resolve=${good:+yes}, sinkhole=${bad:-empty}. (If both empty, AdGuard/gateway may be down.)"
153     fi
154 }
155
156 case "${1:-}" in

```

```

157 enable) cmd_enable ;;
158 disable) cmd_disable ;;
159 status) cmd_status ;;
160 test) cmd_test ;;
161 --help|-h|"") sed -n '2,40p' "$0" ;;
162 *) echo "Unknown argument: $1 (try --help)" >&2; exit 2 ;;
163 esac

```

47 scripts/profiles.sh

```

1 #!/bin/bash
2 # scripts/profiles.sh named config profiles that tame the .env flag sprawl.
3 #
4 # CONFIG-ONLY BY DESIGN: this tool ONLY reads/writes .env. It NEVER touches the
5 # live gateway no routing, no PF, no containers, no restart so it cannot break
6 # your running network. Changes take effect only on your NEXT ./toggle.sh --restart.
7 #
8 # A profile is a coherent, known-good bundle of the intent-level switches, so you
9 # pick ONE intent instead of reasoning about ~7 interacting flags. KILL_SWITCH is
10 # forced true in every profile the fail-closed invariant is never negotiable.
11 #
12 # Usage:
13 #   bash scripts/profiles.sh show           # current config, grouped + explained (read-only)
14 #   bash scripts/profiles.sh list          # available profiles and what each sets (read-only)
15 #   bash scripts/profiles.sh preview <name> # exactly what 'apply' would change (read-only)
16 #   bash scripts/profiles.sh apply <name>   # write the profile to .env (backs up; never restarts)
17 #
18 # Profiles: campus (AP-isolated enterprise, conservative) home (trusted, fast)
19 #            hardened (max privacy: cover traffic + Tor bridges, no real-IP exposure)
20 #
21 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
22 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
23 cd "$PROJECT_ROOT" || exit 1
24 case "${1:-}" in --help|-h) sed -n '2,25p' "$0"; exit 0 ;; esac
25
26 python3 - "$@" <<'PY'
27 import sys, os, shutil, time
28
29 ENV = os.environ.get("PROFILES_ENV_FILE", ".env")
30
31 # Intent-level switches each profile manages. KILL_SWITCH is a forced invariant.
32 PROFILES = {
33     "campus": {"KILL_SWITCH":"true", "TAILSCALE_ALLOW_LAN_P2P":"false", "FACETIME_SPLIT_ROUTE":"false",
34               "PIHOLE_LAN_MODE":"false", "NOISE_ENABLED":"false", "TOR_USE_BRIDGES":"false", "WARP_FAIL_THRESHOLD":
35               "6"},
36     "home": {"KILL_SWITCH":"true", "TAILSCALE_ALLOW_LAN_P2P":"true", "FACETIME_SPLIT_ROUTE":"true",
37              "PIHOLE_LAN_MODE":"true", "NOISE_ENABLED":"false", "TOR_USE_BRIDGES":"false", "WARP_FAIL_THRESHOLD":
38              "6"},
39     "hardened": {"KILL_SWITCH":"true", "TAILSCALE_ALLOW_LAN_P2P":"false", "FACETIME_SPLIT_ROUTE":"false",
40                  "PIHOLE_LAN_MODE":"false", "NOISE_ENABLED":"true", "NOISE_PAD_MODE":"topup",
41                  "TOR_USE_BRIDGES":"true", "WARP_FAIL_THRESHOLD":"3"},
42 }
43 DESC = {
44     "campus": "AP-isolated enterprise/campus: DERP (no SNAT poisoning), everything conservative, no real-IP exposure.",
45     "home": "Trusted network: direct LAN P2P (fast), FaceTime split-route + LAN Pi-hole on, no cover-traffic battery cost.",
46     "hardened": "Max privacy on a hostile net: cover traffic (topup) + Tor bridges, fail-fast WARP, never expose the real IP.",
47 }
48 META = {
49     "KILL_SWITCH": "PF fail-closed kill-switch (forced true never negotiable)",
50     "TAILSCALE_ALLOW_LAN_P2P": "direct LAN P2P vs forced DERP relay",
51     "FACETIME_SPLIT_ROUTE": "route Apple /16s direct (fixes FaceTime; exposes real IP for them)",
52     "PIHOLE_LAN_MODE": "serve AdGuard DNS to non-mesh LAN devices",
53     "NOISE_ENABLED": "cover-traffic padding",
54     "NOISE_PAD_MODE": "padding rate mode (constant / topup)",
55     "TOR_USE_BRIDGES": "Tor via obfs4 bridges only",
56     "WARP_FAIL_THRESHOLD": "consecutive warp=off polls before auto-shutdown",
57 }
58
59 def read_env():
60     if not os.path.exists(ENV):
61         print(f"profiles: {ENV} not found", file=sys.stderr); sys.exit(1)
62     return open(ENV, encoding="utf-8").read().splitlines()

```



```

62 def get_val(lines, key):
63     for l in lines:
64         s = l.strip()
65         if s.startswith(key + "="):
66             return s.split("=", 1)[1]
67     return None
68
69 def which_profile(lines):
70     """Name the profile that matches the current .env, or None."""
71     for name, flags in PROFILES.items():
72         if all(get_val(lines, k) == v for k, v in flags.items()):
73             return name
74     return None
75
76 def cmd_show():
77     lines = read_env()
78     active = which_profile(lines)
79     print(f"\n== nullexit config (current){'    profile: ' + active if active else '    custom (no exact profile match)'} ==\n")
80     for k, d in META.items():
81         v = get_val(lines, k)
82         print(f"    {k:26} = {str(v):9} {d}")
83     print("\n Read-only. 'list' to see profiles, 'apply <name>' for a known-good bundle.")
84
85 def cmd_list():
86     lines = read_env()
87     print()
88     for name, flags in PROFILES.items():
89         print(f"    {name} {DESC[name]}")
90         for k, v in flags.items():
91             cur = get_val(lines, k)
92             print(f"        {k:26} {v}{' if cur == v else '    (now: ' + str(cur) + ')'}")
93     print()
94
95 def diff_for(name):
96     lines = read_env()
97     return [(k, get_val(lines, k), v) for k, v in PROFILES[name].items() if get_val(lines, k) != v]
98
99 def cmd_preview(name):
100     if name not in PROFILES:
101         print(f"unknown profile '{name}' (try: {' ', '.join(PROFILES)}", file=sys.stderr); sys.exit(2)
102     ch = diff_for(name)
103     print(f"\nprofile '{name}' {DESC[name]}\n")
104     if not ch:
105         print("    already applied no changes."); return
106     print("    would change:")
107     for k, cur, v in ch:
108         print(f"        {k:26} {cur} {v}")
109     print("\n Preview only nothing written, network untouched.")
110
111 def set_key(lines, key, val):
112     for i, l in enumerate(lines):
113         if l.strip().startswith(key + "="):
114             lead = l[:len(l) - len(l.strip())]
115             lines[i] = f"{lead}{key}={val}"
116     return
117     lines.append(f"{key}={val}")
118
119 def cmd_apply(name):
120     if name not in PROFILES:
121         print(f"unknown profile '{name}' (try: {' ', '.join(PROFILES)}", file=sys.stderr); sys.exit(2)
122     ch = diff_for(name)
123     if not ch:
124         print(f"profile '{name}' already applied no changes."); return
125     lines = read_env()
126     bak = f"{ENV}.bak.{time.strftime('%Y%m%d%H%M%S')}"
127     shutil.copy2(ENV, bak)
128     for k, v in PROFILES[name].items():
129         set_key(lines, k, v)
130     with open(ENV, "w", encoding="utf-8") as f:
131         f.write("\n".join(lines) + "\n")
132     print(f"\n\napplied '{name}' to {ENV} (backup: {bak})\n")
133     for k, cur, v in ch:
134         print(f"        {k:26} {cur} {v}")
135     print("\n .env only your LIVE gateway is UNCHANGED. Apply with: ./toggle.sh --restart")
136
137 cmd = sys.argv[1] if len(sys.argv) > 1 else "show"
138 arg = sys.argv[2] if len(sys.argv) > 2 else ""
139 {"show": cmd_show, "list": cmd_list,
140  "preview": (lambda: cmd_preview(arg)), "apply": (lambda: cmd_apply(arg))}.get(cmd, cmd_show)()
141 PY

```

48 scripts/reinstall-host-tailscale.sh

```
1 #!/usr/bin/env bash
2 #
3 # scripts/reinstall-host-tailscale.sh
4 #
5 # Complete uninstall + reinstall of the host-side Homebrew Tailscale.
6 #
7 # Use this when the brew LaunchAgent is wedged, Login Items entries are
8 # duplicated, "Tailscale is stopped" persists despite
9 # `brew services restart tailscale`, or you simply want a clean baseline
10 # before re-engaging the gateway as exit node.
11 #
12 # Idempotent. Safe to re-run. Use --dry to preview every step with zero
13 # changes. Use --yes to skip the per-step confirmation prompts.
14 #
15 # This script ONLY touches the HOST-side Tailscale (the one installed by
16 # Homebrew `brew install tailscale` and managed by launchd as
17 # homebrew.mxcl.tailscale). The nullexit gateway container's tailscaled
18 # (which lives inside the Colima VM at 100.100.21.8 and offers the exit
19 # node to this host) is unaffected.
20
21 set -u
22
23 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
24
25 DRY=0
26 YES=0
27
28 for arg in "$@"; do
29     case "$arg" in
30         --dry) DRY=1 ;;
31         --yes) YES=1 ;;
32         --help|-h) # NOTE: heredoc is UNQUOTED so $SCRIPT_DIR expands in the body. If you
33                 # add new $-style placeholders here, they'll expand the same way.
34                 cat <USAGE
35 Usage: bash "$SCRIPT_DIR/reinstall-host-tailscale.sh" [--dry] [--yes]
36
37 --dry Print every step, make zero changes. Use to audit before
38 committing.
39 --yes Auto-confirm the prompts at every destructive step. Useful in
40 CI / fully-scripted recovery; do not use until you've read
41 the script.
42
43 After this script completes cleanly, you still must (MANUALLY):
44
45 1. Open System Settings Privacy & Security Local Network and
46 enable Tailscale. macOS will prompt automatically the first time
47 the fresh install of `tailscale` is invoked.
48
49 2. Re-engage the tailnet + exit node:
50     sudo tailscale up --ssh=true --accept-dns=false \
51       --exit-node="$(cat .gateway_ip | tr -d '\r' | awk 'NR==1{print $1; exit}')" \
52       --exit-node-allow-lan-access=true
53
54 3. Re-run ./toggle.sh from the project root.
55
56 4. Final verification:
57     curl -s https://www.cloudflare.com/cdn-cgi/trace | grep -E '^(warp|ip|colo)=*'
58     expected: warp=on, ip=<Cloudflare>, colo=YYZ/YUL
59 USAGE
60     exit 0
61     ;;
62     *)
63     echo "Unknown flag: $arg" >&2
64     exit 2
65     ;;
66     esac
67 done
68
69 # ----- helpers -----
70
71 confirm() {
72     if [ "$YES" = 1 ] || [ "$DRY" = 1 ]; then
73         return 0
```

```

74 fi
75 printf "      %s [y/N] " "$1"
76 read -r ans
77 case "$ans" in
78     y|Y|yes|YES) return 0 ;;
79     *)
80         echo "      aborted by user"
81         exit 1
82         ;;
83     esac
84 }
85
86 header() {
87     printf '\n %s\n' "$1"
88 }
89
90 # with_timeout <seconds> <cmd...>
91 # Run a command in the background and kill it if it exceeds <seconds>.
92 # Returns the command's exit code if it finished in time, 124 (matching
93 # GNU `timeout` convention) if it timed out. macOS doesn't ship GNU
94 # coreutils `timeout`, so this is the bash-3.2-friendly replacement.
95 # stdout + stderr are discarded (we only care about exit code); redirect
96 # or pipe externally if you need the output.
97 with_timeout() {
98     local timeout_s="${1:-30}"
99     shift
100     "$@" >/dev/null 2>&1 &
101     local cmdpid=$!
102     local elapsed=0
103     while kill -0 "$cmdpid" 2>/dev/null; do
104         if [ "$elapsed" -ge "$timeout_s" ]; then
105             kill -9 "$cmdpid" 2>/dev/null
106             wait "$cmdpid" 2>/dev/null
107             return 124
108         fi
109         sleep 1
110         elapsed=$((elapsed+1))
111     done
112     wait "$cmdpid" 2>/dev/null
113     return $?
114 }
115
116 # ----- 0. Pre-flight -----
117 header "Pre-flight current host tailscale state"
118 echo "      brew: $(command -v brew >/dev/null 2>&1 && brew --version | head -1 || echo 'not on
119     PATH')"
119 echo "      tailscaled running: $(pgrep -lf tailscaled 2>/dev/null | wc -l | tr -d ' ') process(es)"
120 echo "      brew service tailscale: $(brew services list 2>/dev/null | awk '1=="tailscale"{{print $2, $3}}' | tr
121     '\n' ' ' | sed 's/ $//')
122 echo "      LaunchAgent plists (under ~/Library/LaunchAgents):"
123 ls -l ~/Library/LaunchAgents/ 2>/dev/null | grep -i 'tail' | sed 's/~/ /' || echo "      (none)"
124
125 # Detect Tailscale's macOS Network Extension (residual from prior App Store
126 # Tailscale.app installs). Lives under /Library/Apple/SystemExtensions/ and
127 # is managed by `systemextensionsctl` NOT launchd. If loaded, brew
128 # tailscaled will NOT engage the Network Extension framework slot because
129 # the SE still has it claimed. Symptom: 'sudo tailscale status' returns
130 # "Tailscale is stopped" even when the daemon is alive and listening.
131 se_found=0
132 if command -v systemextensionsctl >/dev/null 2>&1; then
133     # Grab any line matching tailscale and network-extension
134     se_line=$(systemextensionsctl list 2>/dev/null | grep -iE '(io|com)\.tailscale\..*network-extension' | head -
135         n 1)
136     if [ -n "$se_line" ]; then
137         se_found=1
138         # Extract the teamID (10-char uppercase/numeric word)
139         se_teamID=$(echo "$se_line" | tr -s ' \t' '\n' | grep -E '^[A-Z0-9]{10}$' | head -n 1)
140         # Extract the bundleID (starts with io.tailscale or com.tailscale)
141         se_bundleID=$(echo "$se_line" | tr -s ' \t' '\n' | grep -E '^(io|com)\.tailscale\.' | head -n 1 | sed 's
142             /(.*/)/')
143         # If teamID is not found, fallback to '-'
144         if [ -z "$se_teamID" ]; then
145             se_teamID="-"
146         fi
147     fi
148 fi
149 if [ "$se_found" = 1 ] && [ -n "$se_bundleID" ]; then
150     echo "      Tailscale Network Extension System Extension is loaded (residual from App Store Tailscale.app
151         install)"

```

```

150 echo "      Team ID: $se_teamID, Bundle ID: $se_bundleID"
151 echo "      brew tailscale cannot engage the NE slot while this SE has it claimed."
152 if [ "$DRY" = 1 ]; then
153     echo "      [dry] (would prompt) sudo systemextensionsctl uninstall $se_teamID $se_bundleID"
154 else
155     confirm "Run 'sudo systemextensionsctl uninstall $se_teamID $se_bundleID'? (frees the NE slot for brew
156     tailscale; non-fatal if it errors)"
157     sudo systemextensionsctl uninstall "$se_teamID" "$se_bundleID" 2>&1 \
158     || echo "      ! uninstall returned non-zero (may require reboot, non-fatal)"
159 fi
160 else
161     echo "      no Tailscale System Extension loaded"
162 fi
163 # Pattern used by the Step-1 bootout loops, in both dry + real branches.
164 # Widened from a bare /tailscale/ so the same loops also pick up any
165 # nullexit-labeled LaunchAgents (e.g., com.nullexit.wake-recovery, the
166 # caffeinate + post-wake recovery hook). The wake-recovery LaunchAgent
167 # is re-installed by ./toggle.sh / setup.sh on re-engage, so wiping it
168 # here is non-fatal and is part of restoring a clean baseline.
169 TAILSCALE_LABEL_RE='tailscale'
170 NULLEXIT_LABEL_RE='nullexit'
171
172 # ----- 1. brew services stop -----
173 header "Step 1 Stop the brew-managed service + unload LaunchAgents"
174 # Detect whether the brew-managed service is registered as $USER (gui/$UID
175 # domain) or as root (system domain). NOPASSWD sudoers can leave a stale
176 # root-owned registration after a prior 'sudo brew services ...' round;
177 # 'brew services stop' without sudo refuses to touch a root-owned plist
178 # (Error: Service 'tailscale' is started as `root`).
179 brew_status_line="$(brew services list 2>/dev/null | awk '$1=="tailscale"')"
180 brew_status_user="$(echo "$brew_status_line" | awk '{print $3}')"
181 if [ -n "$brew_status_line" ]; then
182     if [ "$DRY" = 1 ]; then
183         echo "      [dry] brew services stop tailscale (user=$brew_status_user)"
184     else
185         if [ "$brew_status_user" = "root" ]; then
186             confirm "Run 'sudo brew services stop tailscale' (service is registered as root)?"
187             sudo brew services stop tailscale 2>&1 || echo "      (sudo brew services stop returned non-zero non-fatal)"
188         else
189             echo "      brew services stop tailscale (user=$brew_status_user, no sudo needed)"
190             brew services stop tailscale 2>&1 || echo "      (brew reported it wasn't actually running)"
191         fi
192     fi
193     echo "      unload any tailscale-related LaunchAgents (prevents launchd from respawning after kill)"
194     if [ "$DRY" = 1 ]; then
195         echo "      [dry] for each 'tailscale|nullexit'-matched label in gui/$UID domain:"
196         while IFS= read -r label; do
197             [ -z "$label" ] && continue
198             echo "      [dry] launchctl bootout gui/$UID/$label"
199         done <<(launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$NULLEXIT_LABEL_RE" 'NR > 1 && ($3 ~ ts || $3 ~ nx) {print $3}')
200         echo "      [dry] for each 'tailscale|nullexit'-matched label in system domain (root-owned brew plist):"
201         while IFS= read -r label; do
202             [ -z "$label" ] && continue
203             echo "      [dry] (would prompt) sudo launchctl bootout system/$label"
204         done <<(sudo -n launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$NULLEXIT_LABEL_RE" 'NR > 1 && ($3 ~ ts || $3 ~ nx) {print $3}')
205         if launchctl print system/com.tailscale.ipn.macos >/dev/null 2>&1; then
206             echo "      [dry] (would prompt) sudo launchctl bootout system/com.tailscale.ipn.macos"
207         fi
208         if launchctl print system/com.tailscale.ipn.macsys >/dev/null 2>&1; then
209             echo "      [dry] (would prompt) sudo launchctl bootout system/com.tailscale.ipn.macsys"
210         fi
211     else
212         # Bootout every gui-domain LaunchAgent whose label matches 'tailscale'
213         # OR 'nullexit'. The nullexit arm catches com.nullexit.wake-recovery,
214         # which holds caffeinate/watcher.sh alive across a daemon reset not
215         # desired while the script is establishing a fresh baseline. Awake-
216         # recovery is re-installed by ./toggle.sh / setup.sh on re-engage, so
217         # wiping here is non-fatal. The pattern stays narrower than a bare
218         # /tail/ or /null/ so unrelated Apple services are not unloaded.
219         while IFS= read -r label; do
220             [ -z "$label" ] && continue
221             echo "      launchctl bootout gui/$UID/$label"
222             launchctl bootout "gui/$UID/$label" 2>/dev/null || true
223         done <<(launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$NULLEXIT_LABEL_RE" 'NR > 1 && ($3 ~ ts || $3 ~ nx) {print $3}')
224         # Bootout every system-domain LaunchDaemon/Agent whose label matches
225         # 'tailscale' OR 'nullexit'. This catches the root-owned brew plist left

```

```

226 # behind by prior 'sudo brew services ...' invocations that was the
227 # case that left tailscaled PID 47447 57734 respawning between sudo
228 # pkill runs. Defensive nullexit catch in case the user previously
229 # `sudo ln -s`d the wake-recovery plist into /Library/LaunchDaemons.
230 while IFS= read -r label; do
231     [ -z "$label" ] && continue
232     confirm "Run 'sudo launchctl bootout system/$label'?"
233     sudo launchctl bootout "system/$label" 2>/dev/null || true
234     done < <(sudo -n launchctl list 2>/dev/null | awk -v ts="$TAILSCALE_LABEL_RE" -v nx="$NULLEXIT_LABEL_RE" '
235     NR > 1 && ($3 ~ ts || $3 ~ nx) {print $3}')
236     # Legacy Tailscale.app system-domain entries. Each one is checked and
237     # confirmed separately so we don't sudo-prompt or sudo-bootout for nothing.
238     if launchctl print system/com.tailscale.ipn.macos >/dev/null 2>&1; then
239         confirm "Run 'sudo launchctl bootout system/com.tailscale.ipn.macos'?"
240         sudo launchctl bootout system/com.tailscale.ipn.macos 2>/dev/null || true
241     fi
242     if launchctl print system/com.tailscale.ipn.macsys >/dev/null 2>&1; then
243         confirm "Run 'sudo launchctl bootout system/com.tailscale.ipn.macsys'?"
244         sudo launchctl bootout system/com.tailscale.ipn.macsys 2>/dev/null || true
245     fi
246     echo "    waiting up to 10s for tailscaled to exit"
247     for i in 1 2 3 4 5 6 7 8 9 10; do
248         if [ "$DRY" = 1 ]; then
249             echo "    [dry] sleep 1; loop check would terminate here"
250             break
251         fi
252         sleep 1
253         if ! pgrep -lf tailscaled >/dev/null 2>&1; then
254             echo "    tailscaled exited in ${i}s"
255             break
256         fi
257         [ "$i" = 10 ] && echo "    tailscaled still alive after 10s Step 2 will deal with it"
258     done
259 else
260     echo "    brew reports no tailscale service loaded"
261 fi
262
263 # ----- 2. Kill any orphan tailscaled -----
264 header "Step 2 Kill any orphan tailscaled process(es) (may escalate to sudo)"
265 if ! pgrep -lf tailscaled >/dev/null 2>&1; then
266     echo "    no orphans"
267 else
268     echo "    Stray process(es):"
269     pgrep -lf tailscaled | sed 's/^/    /'
270     echo "    no-sudo: pkill tailscaled"
271     if [ "$DRY" = 1 ]; then
272         echo "    [dry] pkill tailscaled on failure: sudo pkill -9 tailscaled"
273     else
274         if pkill tailscaled 2>/dev/null; then
275             echo "    no-sudo pkill succeeded"
276         else
277             echo "    ! no-sudo pkill insufficient; cannot escalate yet"
278         fi
279         sleep 1
280     fi
281 fi
282 # Recheck; if still alive, offer sudo escalation
283 if { [ "$DRY" = 0 ] && pgrep -lf tailscaled >/dev/null 2>&1; } || [ "$DRY" = 1 ]; then
284     if [ "$DRY" = 1 ]; then
285         echo "    [dry] sudo pkill -9 tailscaled (escalation would fire here)"
286     else
287         confirm "Run 'sudo pkill -9 tailscaled' to escalate?"
288         sudo pkill -9 tailscaled 2>&1
289         sleep 1
290     fi
291 fi
292
293 if [ "$DRY" = 0 ]; then
294     if pgrep -lf tailscaled >/dev/null 2>&1; then
295         echo "    Failed to kill stragglers bailing out"
296         pgrep -lf tailscaled | sed 's/^/    /'
297         exit 3
298     fi
299     echo "    all tailscaled processes gone"
300 fi
301 fi
302
303 # ----- 3. brew uninstall -----
304 header "Step 3 brew uninstall tailscale"
305 if ! brew list tailscale >/dev/null 2>&1; then

```

```

306     echo "        not currently installed via brew skip"
307 else
308     # Detect whether the keg directory is owner=$USER or owner=root. If the
309     # keg is root-owned (typically a residual of some prior 'sudo brew ...'
310     # cycle that did perms fixups), 'brew uninstall' without sudo refuses
311     # to remove the files and prints: 'Error: Could not remove tailscale
312     # keg! Do so manually: sudo rm -rf /opt/homebrew/Cellar/tailscale/<v>'.
313     # Detect that and escalate to a sudoed uninstall, with a manual rm -rf
314     # fallback if `sudo brew uninstall` itself errors out (e.g., partial
315     # state where the keg dir is gone but brew still thinks it's installed).
316     # Detect keg state. Treat `(dir absent)` AND `(dir present + empty)` as the
317     # same `<unknown>` bucket so both escalate to sudo. The empty-parent case
318     # is what you get after a manual `sudo rm -rf /opt/homebrew/Cellar/tailscale/<v>`
319     # from inside: brew's parent dir is left present-but-empty and brew still
320     # thinks the package is installed.
321     keg_dir="/opt/homebrew/Cellar/tailscale"
322     if [ -d "$keg_dir" ] && [ -n "$(ls -A "$keg_dir" 2>/dev/null)" ]; then
323         keg_owner="$(stat -f '%Su' "$keg_dir" 2>/dev/null || echo '<unknown>')"
324     else
325         keg_owner="<unknown>"
326     fi
327     if [ "$SDRY" = 1 ]; then
328         echo "        [dry] brew uninstall tailscale (keg_owner=$keg_owner)"
329     else
330         # 'root' AND '<unknown>' both require sudo escalation. The '<unknown>'
331         # case is the partial-state path: keg dir was already removed (manually
332         # or by a prior failed uninstall) but brew's internal state still says
333         # installed. Without this branch the script would fall through to a
334         # non-sudo brew uninstall that fails again with the same 'Could not
335         # remove tailscale keg!' error.
336         if [ "$keg_owner" = "root" ] || [ "$keg_owner" = "<unknown>" ]; then
337             confirm "Run 'sudo brew uninstall tailscale' (keg=$keg_owner)?"
338             sudo brew uninstall tailscale 2>&1 || {
339                 echo "        ! sudo brew uninstall failed; offering manual rm -rf fallback"
340                 # Guard the glob so an empty/absent dir doesn't literal-expand the
341                 # '*' and confuse the user with a sudo error about a non-existent path.
342                 if [ -d /opt/homebrew/Cellar/tailscale ] && [ -n "$(ls -A /opt/homebrew/Cellar/tailscale 2>/dev/null)" ]; then
343                     confirm "Run 'sudo rm -rf /opt/homebrew/Cellar/tailscale/*'?"
344                     sudo rm -rf /opt/homebrew/Cellar/tailscale/* 2>&1
345                 else
346                     echo "        keg dir already empty or absent  nothing more to rm"
347                 fi
348             }
349         else
350             confirm "Run 'brew uninstall tailscale' (removes LaunchAgent plist + linked symlinks)?"
351             brew uninstall tailscale 2>&1
352         fi
353     fi
354 fi
355
356 # ----- 4. Wipe state directories -----
357 header "Step 4 Wipe stale state"
358 # User-owned
359 declare -a USER_PATHS
360 USER_PATHS=(
361     "$HOME/.local/share/tailscale"
362     "$HOME/.local/run/tailscaled.socket"
363     "$HOME/.local/run/tailscaled.state"
364     "$HOME/.local/state/tailscale"
365     "$HOME/Library/Application Support/Tailscale"
366     "$HOME/Library/Caches/com.tailscale.ipn.macos"
367     "$HOME/Library/Caches/Tailscale"
368     "$HOME/Library/Logs/Tailscale"
369 )
370 echo "        User-owned paths (no sudo):"
371 for p in "${USER_PATHS[@]}; do
372     if [ -e "$p" ]; then
373         if [ "$SDRY" = 1 ]; then
374             echo "        [dry] rm -rf '$p'"
375         else
376             rm -rf "$p"
377             echo "        rm -rf '$p'"
378         fi
379     fi
380 done
381
382 # System
383 declare -a SUDO_PATHS
384 SUDO_PATHS=(
385     "/var/lib/tailscale"

```

```

386 "/var/cache/tailscale"
387 )
388 echo "    System paths under /var (will prompt for sudo if any are present):"
389 eligible_sudo=0
390 for p in "${SUDO_PATHS[@]"; do
391     if [ -e "$p" ]; then
392         eligible_sudo=1
393         if [ "$DRY" = 1 ]; then
394             echo "        [dry] sudo rm -rf '$p'"
395         else
396             if confirm "Run 'sudo rm -rf $p'?" ; then
397                 sudo rm -rf "$p" && echo "            rm -rf '$p'"
398             fi
399         fi
400     fi
401 done
402 [ "$eligible_sudo" = 0 ] && echo "        (none of /var/lib/tailscale, /var/cache/tailscale are present)"
403
404 # ----- 5. Wipe stale LaunchAgent plists -----
405 header "Step 5 Wipe stale LaunchAgent plist files"
406 declare -a PLIST_FILES
407 PLIST_FILES=(
408     "$HOME/Library/LaunchAgents/homebrew.mxcl.tailscale.plist"
409     "$HOME/Library/LaunchAgents/com.tailscale.ipn.macos.plist"
410     "$HOME/Library/LaunchAgents/homebrew.mxcl.tailscale.plist.bak"
411     "$HOME/Library/LaunchAgents/com.nullexit.wake-recovery.plist"
412 )
413 for f in "${PLIST_FILES[@]"; do
414     if [ -e "$f" ]; then
415         if [ "$DRY" = 1 ]; then
416             echo "        [dry] rm -f '$f'"
417         else
418             rm -f "$f"
419             echo "            rm -f '$f'"
420         fi
421     fi
422 done
423 echo "    Remaining tailscale|nullexit plists (should be empty):"
424 ls -l ~/Library/LaunchAgents/ 2>/dev/null | grep -i 'tail' | sed 's/~/ /' || echo "        (none)"
425
426 # ----- 5.5. Drain Background-Items (Login Items) -----
427 header "Step 5.5 Drain Background-Items (Login Items)"
428 # macOS 13+ keeps a parallel "Background Items" database alongside
429 # classic LaunchAgents. Even when we wipe ~/Library/LaunchAgents/homebrew.mxcl.tailscale.plist
430 # in Step 5, the .btm file at
431 # ~/Library/Application Support/com.apple.backgroundtaskmanagementagent.backgrounditems.btm
432 # can still hold a Background-Items entry pointing at the (now-dead) plist
433 # path. Next login: macOS sees the entry, tries to bootstrap, finds the
434 # plist missing, and either silently regenerates it via brew's post-install
435 # hook (causing tailscaled respawn at next login) or refuses outright.
436 # `sfltool reset-login-items` is Apple's canonical API for wiping the
437 # entire .btm file. It DOES wipe all Login Items the user re-adds what
438 # they want via System Settings General Login Items. Acceptable here
439 # because the goal of this script is "fresh baseline for Tailscale".
440 BTM="$HOME/Library/Application Support/com.apple.backgroundtaskmanagementagent/backgrounditems.btm"
441 if [ -f "$BTM" ]; then
442     if [ "$DRY" = 1 ]; then
443         echo "        [dry] found $BTM; would prompt sfltool reset-login-items"
444     else
445         confirm "Run 'sfltool reset-login-items' (wipes ALL Login Items; rebuild via System Settings General Login Items)?"
446         if sfltool reset-login-items 2>&1; then
447             echo "            Login Items drained"
448         else
449             echo "            ! sfltool reset-login-items failed (non-fatal; do manually: System Settings General Login Items)"
450         fi
451     fi
452 else
453     echo "        no backgrounditems.btm present"
454 fi
455
456 # ----- 6. brew install -----
457 header "Step 6 brew install tailscale (fresh)"
458 if brew list tailscale >/dev/null 2>&1; then
459     echo "        already installed skip"
460 else
461     confirm "Run 'brew install tailscale'?"
462     if [ "$DRY" = 1 ]; then
463         echo "        [dry] brew install tailscale"
464     else

```



```

465     brew install tailscale 2>&1
466 fi
467 fi
468
469 # ----- 6.5. Strip quarantine xattr from keg -----
470 header "Step 6.5 Strip quarantine xattr from tailscale install"
471 # brew's freshly poured bottles don't carry com.apple.quarantine (the
472 # bottles are content-addressed and Homebrew-verified), but the upstream
473 # installer (Tailscale.app from App Store) and any manual download do.
474 # Strip the xattr from the entire keg as a defensive safety net so the
475 # freshly-installed tailscaled binary and any helper binaries are not
476 # Gatekeeper-blocked on first launch. Idempotent no-op if no quarantine
477 # xattr is present.
478 quarantine_count="$(xattr -lr /opt/homebrew/Cellar/tailscale 2>/dev/null | grep -c '^[^ ]* com.apple.quarantine
    ' || echo 0)"
479 if [ "${quarantine_count:-0}" -gt 0 ]; then
480     if [ "$DRY" = 1 ]; then
481         echo "    [dry] found $quarantine_count file(s) with com.apple.quarantine; would prompt xattr -dr"
482     else
483         confirm "Run 'xattr -dr com.apple.quarantine /opt/homebrew/Cellar/tailscale'? (strips quarantine xattr from
            $quarantine_count file(s); non-fatal if it errors)"
484         if xattr -dr com.apple.quarantine /opt/homebrew/Cellar/tailscale 2>&1; then
485             echo "        quarantine xattr stripped"
486         else
487             echo "        ! xattr -dr failed (non-fatal; open /opt/homebrew/Cellar/tailscale in Finder, right-click
                tailscaled Open Anyway)"
488         fi
489     fi
490 else
491     echo "        no com.apple.quarantine xattr on tailscale install"
492 fi
493
494 # ----- 7. brew services start (NO sudo!) -----
495 header "Step 7 Start the service as $USER (NO sudo, with multi-tier self-healing)"
496 if ! command -v tailscale >/dev/null 2>&1; then
497     echo "        tailscale not on PATH after install brew install errored; aborting"
498     exit 4
499 fi
500
501 # Aliveness fan-out: ANY one of these is taken as proof of life. None of
502 # them alone is reliable pgrep misses tailscaled if it does setproctitle
503 # or fork-detach; launchctl's reported PID can be a shim that exited
504 # within 1s; tailscale-cli hangs while macOS Local-Network permission is
505 # unresolved. Combined, they cover each other's blind spots.
506
507 tailscaled_api_up() {
508     # Strategy A: tailscale CLI can talk to the local API. Canonical proof.
509     # Bound at 8s tailscale status itself returns quickly on success OR
510     # failure, but if the post-install state is wedged it can hang.
511     with_timeout 8 /opt/homebrew/bin/tailscale status >/dev/null 2>&1
512 }
513
514 tailscaled_proc_up() {
515     # Strategy B: any process has 'tailscaled' substring in argv.
516     pgrep -lf tailscaled >/dev/null 2>&1
517 }
518
519 tailscaled_lc_up() {
520     # Strategy C: launchctl-managed service has a non-dash, non-zero PID.
521     local lc_pid
522     lc_pid="$(launchctl list 2>/dev/null | awk '$3 == "homebrew.mxcl.tailscale" {print $1; exit}')"
523     if [ -n "$lc_pid" ] && [ "$lc_pid" != "-" ] && [ "$lc_pid" -gt 0 ] 2>/dev/null; then
524         return 0
525     fi
526     return 1
527 }
528
529 tailscaled_is_alive() {
530     tailscaled_api_up && return 0
531     tailscaled_proc_up && return 0
532     tailscaled_lc_up && return 0
533     return 1
534 }
535
536 confirm "Run 'brew services start tailscale'?"
537 if [ "$DRY" = 1 ]; then
538     echo "    [dry] brew services start tailscale"
539     echo "    [dry] (then poll up to 30s for tailscaled_is_alive)"
540     echo "    [dry] (recovery tier 1: launchctl kickstart)"
541     echo "    [dry] (recovery tier 2: launchctl bootout + bootstrap of explicit plist)"
542     echo "    [dry] (recovery tier 3: brew services restart (stop+start cycle))"

```

```

543 echo " [dry] (recovery tier 4: direct /opt/homebrew/bin/tailscaled spawn (bypasses launchd))"
544 else
545 brew services start tailscale 2>&1
546 # Primary poll tailscaled_is_alive covers all 3 detection strategies.
547 echo " polling for tailscaled to come up (up to ~30s)"
548 tailscaled_up=0
549 for i in 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15; do
550     if tailscaled_is_alive; then
551         tailscaled_up=1
552         echo " tailscaled alive after ${i}*2s"
553         break
554     fi
555     sleep 2
556 done
557
558 # Recovery tier 1: kickstart the launchd-managed service. This is the
559 # cheapest and most-aligned-with-launchd fix when brew has loaded the
560 # LaunchAgent but tailscaled didn't fork. Kickstart tells launchd
561 # "re-spawn if not running" without forcing a teardown.
562 if [ "$tailscaled_up" = 0 ]; then
563     echo " ! Recovery tier 1: launchctl kickstart"
564     if launchctl kickstart -kp "gui/$UID/homebrew.mxcl.tailscale" 2>&1; then
565         for i in 1 2 3 4 5 6 7 8 9 10; do
566             if tailscaled_is_alive; then
567                 tailscaled_up=1
568                 echo " tailscaled alive after kickstart (${i}*2s)"
569                 break
570             fi
571             sleep 2
572         done
573     else
574         echo " ! kickstart returned non-zero (LaunchAgent may not be loaded)"
575     fi
576 fi
577
578 # Recovery tier 2: explicit plist bootout + bootstrap. Forces a fresh
579 # launchd registration cycle for the brew-managed service, which fixes
580 # the wider class of "brew says started but daemon never spawned"
581 # wedge states that kickstart alone can't dig out of.
582 if [ "$tailscaled_up" = 0 ]; then
583     echo " ! Recovery tier 2: launchctl bootout + bootstrap of explicit plist"
584     launchctl bootout "gui/$UID/homebrew.mxcl.tailscale" 2>/dev/null || true
585     sleep 1
586     # Brew installs the canonical plist template at $HOMEBREW_PREFIX/Library/.
587     # On Apple Silicon that's /opt/homebrew/Library/LaunchAgents/. On Intel
588     # it's /usr/local/Library/LaunchAgents/. Detect which and pick the
589     # right one so this script works across both architectures.
590     if [ -f /opt/homebrew/Library/LaunchAgents/homebrew.mxcl.tailscale.plist ]; then
591         plist_path="/opt/homebrew/Library/LaunchAgents/homebrew.mxcl.tailscale.plist"
592     elif [ -f /usr/local/Library/LaunchAgents/homebrew.mxcl.tailscale.plist ]; then
593         plist_path="/usr/local/Library/LaunchAgents/homebrew.mxcl.tailscale.plist"
594     else
595         plist_path=""
596     fi
597     if [ -n "$plist_path" ]; then
598         if launchctl bootstrap "gui/$UID" "$plist_path" 2>&1; then
599             for i in 1 2 3 4 5 6 7 8 9 10; do
600                 if tailscaled_is_alive; then
601                     tailscaled_up=1
602                     echo " tailscaled alive after bootstrap (${i}*2s)"
603                     break
604                 fi
605                 sleep 2
606             done
607         else
608             echo " ! explicit bootstrap returned non-zero"
609         fi
610     else
611         echo " ! could not locate brew plist template; skipping bootstrap tier"
612     fi
613 fi
614
615 # Recovery tier 3: brew services restart (forces a stop + start cycle,
616 # which unsticks LaunchAgents with stale 'started' state from prior
617 # half-cycles). This is the canonical brew-doc fix for 'services don't
618 # actually start even though brew says started'.
619 if [ "$tailscaled_up" = 0 ]; then
620     echo " ! Recovery tier 3: brew services restart (forces stop+start cycle)"
621     if brew services restart tailscale 2>&1; then
622         for i in 1 2 3 4 5 6 7 8 9 10; do
623             if tailscaled_is_alive; then

```

```

624         tailscaled_up=1
625         echo "         tailscaled alive after brew restart (${i}*2s)"
626         break
627     fi
628     sleep 2
629     done
630     else
631         echo "         ! brew services restart returned non-zero"
632     fi
633 fi
634
635 # Recovery tier 4: last-resort direct spawn of the daemon binary,
636 # bypassing launchd entirely. If tailscaled crashes here too, we'll
637 # capture its actual stderr in /tmp for the user to read (NOPASSWD
638 # sudoers often have local-network permission denied the captured
639 # stderr makes it obvious what to fix in System Settings).
640 if [ "$tailscaled_up" = 0 ]; then
641     echo "         ! Recovery tier 4: direct spawn (bypasses launchd; tail stderr to /tmp)"
642     /opt/homebrew/bin/tailscaled >/tmp/nullexit-tailscaled-spawn.log 2>&1 &
643     spawned_pid=$!
644     disown "$spawned_pid" 2>/dev/null || true
645     sleep 3
646     if tailscaled_is_alive; then
647         tailscaled_up=1
648         echo "         tailscaled alive via direct spawn (pid=$spawned_pid)"
649         echo "         (NOTE: not under launchd supervision a reboot, re-run of"
650         echo "         this script, or ./toggle.sh is required to recreate."
651         echo "         Last 5 lines of stderr captured to /tmp/nullexit-tailscaled-spawn.log:)"
652         tail -n 5 /tmp/nullexit-tailscaled-spawn.log 2>/dev/null | sed 's/~/ /' || true
653     fi
654 fi
655
656 if [ "$tailscaled_up" = 0 ]; then
657     echo "         HARD FAIL: tailscaled refuses to stay alive across every recovery tier"
658     echo "         launchctl-managed state (if available):"
659     launchctl print "gui/$UID/homebrew.mxcl.tailscale" 2>/dev/null \
660     | grep -E 'state =|last exit code =|runs =|path =' \
661     | sed 's/~/ /' \
662     || echo "         (no launchctl state available for homebrew.mxcl.tailscale)"
663     echo "         Last 10 lines of /tmp/nullexit-tailscaled-spawn.log (if any):"
664     tail -n 10 /tmp/nullexit-tailscaled-spawn.log 2>/dev/null | sed 's/~/ /' || echo "         (no log)"
665     echo "         Most likely remaining cause: macOS Local Network permission denied."
666     echo "         System Settings Privacy & Security Local Network toggle Tailscale ON."
667     exit 7
668 fi
669 fi
670 sleep 3
671
672 # ----- 8. Verify -----
673 header "Step 8 Verify fresh install"
674 echo "         brew service tailscale: $(brew services list 2>/dev/null | awk ' $1=="tailscale" {print $2, $3}' | tr
675 '\n' ' ' | sed 's/ $//')
676 echo "         tailscaled process: $(pgrep -lf tailscaled 2>/dev/null | head -1 || echo 'NONE (check brew logs)
677 ')
678 ts_ver=""
679 if [ -x /opt/homebrew/bin/tailscale ]; then
680     ts_ver="$(/opt/homebrew/bin/tailscale version 2>/dev/null | head -1 | tr -d '\n')
681 elif command -v tailscale >/dev/null 2>&1; then
682     ts_ver="$(tailscale version 2>/dev/null | head -1 | tr -d '\n')
683 fi
684 echo "         tailscale version: ${ts_ver:-binary missing}"
685 ts_status=""
686 if [ -x /opt/homebrew/bin/tailscale ]; then
687     ts_status="$(/opt/homebrew/bin/tailscale status 2>/dev/null | head -3)"
688 fi
689 if [ -n "$ts_status" ]; then
690     echo "         tailscale status:"
691     printf '%s\n' "$ts_status" | sed 's/~/ /'
692 else
693     echo "         tailscale status: (binary missing or errored)"
694 fi
695 if { [ "$SDRY" = 0 ] && /opt/homebrew/bin/tailscale status 2>/dev/null | grep -q '^Tailscale is stopped'; };
696 then
697     echo
698     echo "         NOTICE: 'tailscale status' still reports stopped after a fresh install."
699     echo "         Two equally likely causes:"
700     echo "         (a) macOS hasn't yet shown the first-run 'Local Network' prompt for"
701     echo "         Tailscale open System Settings Privacy & Security Local"
702     echo "         Network. Toggle Tailscale ON."
703     echo "         (b) macOS denied Local Network permission at some prior point."

```

```

702 echo "      Either way: System Settings Privacy & Security Local Network."
703 echo "      If the entry isn't present, invoking any tailscale command will"
704 echo "      re-trigger the prompt automatically."
705 fi
706
707 # ----- 9. Done -----
708 header "Done"
709 if [ "$DRY" = 1 ]; then
710     echo "      (dry-mode finished; no follow-up commands were issued.)"
711     exit 0
712 fi
713
714 cat <<'DONE'
715
716 Manual follow-ups (all of these; the script stops short of running sudo
717 tailscale up so you can verify the brew install is actually live first):
718
719 1. System Settings Privacy & Security Local Network Tailscale ON
720 macOS will normally pop the prompt the first time you run `tailscale`.
721 If it didn't, trigger it via the menu bar check.
722
723 2. Re-engage the tailnet + exit node:
724     sudo tailscale up --ssh=true --accept-dns=false \
725       --exit-node="$(cat .gateway_ip | tr -d '\r' | awk 'NR==1{print $1; exit}')" \
726       --exit-node-allow-lan-access=true
727
728 3. Re-run: ./toggle.sh
729 (re-installs the com.nullexit.wake-recovery LaunchAgent wiped
730 in Step 5 to clear the slate and re-engages the gateway)
731
732 4. Final verification:
733     curl -s https://www.cloudflare.com/cdn-cgi/trace | grep -E '^(warp|ip|colo)= '
734     expect: warp=on, ip=<Cloudflare IP>, colo=YYZ/YUL
735
736 Container-side (gateway / 100.100.21.8) Tailscale is unaffected by this
737 script. Other tailnet devices (S24, iPhone, Windows) are unaffected.
738
739 DONE

```

49 scripts/routing-fix.sh

```

1  #!/bin/sh
2  # routing-fix: Maintains routing for SOCKS5 proxy traffic through WARP (tun0)
3  # and FORWARD rules for Tailscale exit-node return traffic.
4  #
5  # Runs inside the warp container's network namespace. This script ensures:
6  # 1. Table 200 has default via tun0 (for SOCKS5 proxy traffic from 172.18.0.2)
7  # with the WARP endpoint exception via eth0 (prevents tunnel loop)
8  # 2. The CGNAT ip rule (100.64.0.0/10 table 52 tailscale routes) is maintained for Tailscale
9  # 3. FORWARD RELATED,ESTABLISHED rule allows return traffic for exit-node
10 # forwarded connections (S24 tailscale0 eth0 host internet)
11 #
12 # CRITICAL: Does NOT touch the main table's default route. Gluetun handles that
13 # internally via its own routing rules and WireGuard fwmark (0xca6c table 51820).
14 # Overriding the main table default would create a loop: encrypted WireGuard
15 # packets exiting tun0 would match default dev tun0 and re-enter the tunnel.
16
17 set -e
18
19 trap 'echo "routing-fix: Caught signal, exiting..."; exit 0' TERM INT QUIT
20
21 sleep 15 &
22 wait $! || true
23
24 echo "routing-fix: Setting up routing for SOCKS5 proxy through WARP (tun0)..."
25
26 # Dynamically detect Docker subnet from eth0
27 DOCKER_NET=$(ip route show dev eth0 | grep -v default | head -1 | awk '{print $1}')
28 DOCKER_GW=$(ip route show default | head -1 | awk '{print $3}')
29 WARP_ENDPOINT="${WARP_ENDPOINT}"
30 TAILSCALE_ALLOW_LAN_P2P="${TAILSCALE_ALLOW_LAN_P2P:-false}"
31 IP_BLOCKLIST_FILE="/userfilters/ip_blocklist.ipset"
32 LAST_IP_MTIME=""
33 echo "routing-fix: Docker network: ${DOCKER_NET}, gateway: ${DOCKER_GW}"
34
35 # Table 200: Used by ip rule 100 (from 172.18.0.2) for SOCKS5 proxy traffic

```

```

36 # WARP endpoint goes via eth0 to avoid tunnel loop, everything else via tun0
37 ip route add "${WARP_ENDPOINT}" via "${DOCKER_GW}" dev eth0 table 200 2>/dev/null || \
38 ip route replace "${WARP_ENDPOINT}" via "${DOCKER_GW}" dev eth0 table 200 2>/dev/null || true
39 ip route replace default dev tun0 table 200 2>/dev/null || true
40
41 # Main table: Just ensure the Docker subnet route exists via eth0
42 # (gluetun owns the default route here do NOT touch it)
43 ip route replace "${DOCKER_NET}" dev eth0 2>/dev/null || true
44
45 # FORWARD RELATED, ESTABLISHED: Allow return traffic for exit-node connections.
46 # The container has BOTH iptables (nftables backend) and iptables-legacy
47 # loaded. Gluetun's post-rules.txt uses the nftables backend, while Tailscale
48 # uses the legacy backend. Packets go through BOTH stacks, so we add the rule
49 # to both to ensure it survives regardless of which backend has policy DROP.
50 #
51 # Packet flow: S24 outgoing (tailscale0->eth0) ACCEPTed by first rule.
52 # Return traffic (eth0->eth0 via table 52 to host) needs RELATED, ESTABLISHED
53 # to match the conntrack entry created by the outgoing flow.
54 add_fwd_related_established() {
55     # nftables backend (used by gluetun's post-rules.txt)
56     if ! iptables -C FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null; then
57         iptables -A FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null || true
58     fi
59     # legacy backend (used by Tailscale's ts-forward)
60     if command -v iptables-legacy >/dev/null 2>&1; then
61         if ! iptables-legacy -C FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null; then
62             iptables-legacy -A FORWARD -m state --state RELATED,ESTABLISHED -j ACCEPT 2>/dev/null || true
63         fi
64     fi
65 }
66
67 add_fwd_related_established
68
69 # Policy Routing for Tailscale P2P:
70 # Tailscale sends its encrypted mesh UDP packets from port 41642.
71 # If these go out tun0 (WARP), Cloudflare's strict NAT drops the returning
72 # hole-punch packets, causing Tailscale to fail P2P and fall back to DERP relays (high latency).
73 # We mark packets originating from port 41642 and force them out the main table (eth0).
74 #
75 # LAN P2P is controlled by two sources (in priority order):
76 # 1. /app/.lan_p2p_detected written by watcher.sh on macOS on every network change
77 #    using system_profiler SPAirPortDataType (802.1x detection) + AP isolation probe. Overrides .env.
78 # 2. TAILSCALE_ALLOW_LAN_P2P env var (from .env) static fallback.
79 # When disabled, we explicitly DROP local RFC1918-destined Tailscale UDP so the
80 # phone cleanly falls back to DERP rather than getting poisoned by macOS gvpnproxy SNAT.
81 add_tailscale_p2p_bypass() {
82     # Dynamic override from watcher.sh (network auto-detection) takes priority over .env
83     local allow_p2p="${TAILSCALE_ALLOW_LAN_P2P:-false}"
84     if [ -f "/app/.lan_p2p_detected" ]; then
85         allow_p2p=$(cat "/app/.lan_p2p_detected" 2>/dev/null | tr -d '[:space:]')
86     fi
87
88     # Always allow P2P directly to the Docker host gateway (the Mac running this container).
89     # The gateway container is always co-located with the host Mac on the same machine,
90     # so direct Tailscale P2P via the Docker bridge is always safe and avoids DERP overhead.
91     # This RETURN rule must be inserted BEFORE the general DROP rules so it takes priority.
92     local default_gw
93     default_gw=$(ip route show default | awk '{print $3}')
94     if [ -n "$default_gw" ]; then
95         if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -d "${default_gw}/32" -j RETURN 2>/dev/null; then
96             iptables -t mangle -I OUTPUT 1 -p udp --sport 41642 -d "${default_gw}/32" -j RETURN 2>/dev/null || true
97         fi
98     fi
99
100     # Also explicitly whitelist all physical IPs of the host Mac (e.g. 172.17.52.223).
101     # The Tailscale control plane registers the Mac using its physical IP, so the
102     # container will attempt P2P handshakes using that physical IP instead of the
103     # Docker bridge IP. If this physical IP falls within 172.16.0.0/12, it would be dropped.
104     if [ -f "/app/.host_ips" ]; then
105         while IFS= read -r host_ip; do
106             [ -z "$host_ip" ] && continue
107             if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -d "${host_ip}/32" -j RETURN 2>/dev/null; then
108                 iptables -t mangle -I OUTPUT 1 -p udp --sport 41642 -d "${host_ip}/32" -j RETURN 2>/dev/null || true
109             fi
110         done < "/app/.host_ips"
111     fi
112
113     if [ "$allow_p2p" != "true" ]; then
114         # Drop Tailscale UDP packets destined for local subnets to prevent macOS gvpnproxy SNAT
115         # endpoint poisoning. When disabled (campus/enterprise WPA2-Enterprise with AP isolation),
116         # dropping these forces the S24 to use DERP relays reliably.

```

```

117     for subnet in "192.168.0.0/16" "10.0.0.0/8" "172.16.0.0/12"; do
118         if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -d "$subnet" -j DROP 2>/dev/null; then
119             iptables -t mangle -A OUTPUT -p udp --sport 41642 -d "$subnet" -j DROP 2>/dev/null || true
120         fi
121     done
122 else
123     # Remove DROP rules if they exist (switching from restricted to allowed)
124     for subnet in "192.168.0.0/16" "10.0.0.0/8" "172.16.0.0/12"; do
125         while iptables -t mangle -D OUTPUT -p udp --sport 41642 -d "$subnet" -j DROP 2>/dev/null; do ;; done
126     done
127 fi
128
129 # Bypass STUN and P2P packets to public IPs (DERP relays and remote peers) out eth0
130 if ! iptables -t mangle -C OUTPUT -p udp --sport 41642 -j MARK --set-mark 0x8888 2>/dev/null; then
131     iptables -t mangle -A OUTPUT -p udp --sport 41642 -j MARK --set-mark 0x8888 2>/dev/null || true
132 fi
133 if ! ip rule show | grep -q 'fwmark 0x8888'; then
134     ip rule add fwmark 0x8888 lookup 254 pref 98 2>/dev/null || true
135 fi
136 }
137
138 add_tailscale_p2p_bypass
139
140 add_onion_transparent_proxy() {
141     for ipt in iptables iptables-legacy; do
142         command -v "$ipt" >/dev/null 2>&1 || continue
143         if ! $ipt -t nat -C PREROUTING -d 198.18.0.0/15 -p tcp -j REDIRECT --to-ports 9040 2>/dev/null; then
144             $ipt -t nat -I PREROUTING -d 198.18.0.0/15 -p tcp -j REDIRECT --to-ports 9040 2>/dev/null || true
145         fi
146         if ! $ipt -C FORWARD -d 198.18.0.0/15 -j ACCEPT 2>/dev/null; then
147             $ipt -I FORWARD -d 198.18.0.0/15 -j ACCEPT 2>/dev/null || true
148         fi
149     done
150 }
151
152 add_tailscale_p2p_bypass
153 add_onion_transparent_proxy
154
155 create_log_drop_chain() {
156     local ipt="$1"
157     local chain="$2"
158     local prefix="$3"
159
160     if ! $ipt -N "$chain" 2>/dev/null; then
161         $ipt -F "$chain" 2>/dev/null || true
162     fi
163     $ipt -A "$chain" -j LOG --log-prefix "$prefix"
164     $ipt -A "$chain" -j DROP 2>/dev/null || true
165 }
166
167 add_country_block() {
168     # To add a country to the blocklist, simply define the 'BLOCKED_COUNTRIES' variable in .env with 2-letter ISO
169     # country codes.
170     # You can find the country code by looking at the filename on http://www.ipdeny.com/ipblocks/data/countries/
171     # For example, to block China (cn) and Russia (ru), set: BLOCKED_COUNTRIES="cn ru" in your .env file
172     for zone in $BLOCKED_COUNTRIES; do
173         set_name="block_${zone}"
174         zone_upper=$(echo "$zone" | tr 'a-z' 'A-Z')
175         chain_name="LOG_DROP_${zone_upper}"
176         log_prefix="IP_BLOCK_${zone_upper}: "
177
178         # Create the ipset if it doesn't exist
179         if ! ipset list "$set_name" >/dev/null 2>&1; then
180             ipset create "$set_name" hash:net 2>/dev/null || true
181             echo "routing-fix: Downloading ${zone_upper} IPs for blocklist..."
182             curl -sS "http://www.ipdeny.com/ipblocks/data/countries/${zone}.zone" | grep -v '^#' | while read -r
183             subnet; do
184                 [ -n "$subnet" ] && ipset add "$set_name" "$subnet" 2>/dev/null || true
185             done
186         fi
187
188         # Apply LOG_DROP rule to both backends
189         for ipt in iptables iptables-legacy; do
190             command -v "$ipt" >/dev/null 2>&1 || continue
191
192             create_log_drop_chain "$ipt" "$chain_name" "$log_prefix"
193
194             # Clean up old plain DROP rules to ensure we hit the log-and-drop chains
195             while $ipt -D FORWARD -m set --match-set "$set_name" dst -j DROP 2>/dev/null; do ;; done
196
197             if ! $ipt -C FORWARD -m set --match-set "$set_name" dst -j "$chain_name" 2>/dev/null; then

```

```

196     $ipt -I FORWARD -m set --match-set "$set_name" dst -j "$chain_name" 2>/dev/null || true
197 fi
198 done
199 done
200 }
201
202 add_ip_blocklist() {
203     # Only reload when the compiled file has actually changed (mtime check).
204     # This prevents a pointless ipset restore on every 30-second loop tick.
205     if [ ! -f "$IP_BLOCKLIST_FILE" ]; then
206         return 0
207     fi
208
209     CURRENT_MTIME=$(stat -c %Y "$IP_BLOCKLIST_FILE" 2>/dev/null || echo "0")
210     if [ "$CURRENT_MTIME" != "$LAST_IP_MTIME" ]; then
211         echo "routing-fix: IP blocklist changed, reloading..."
212         # ipset restore handles the atomic swap internally:
213         # create_new populate swap with live destroy_new
214         if ipset restore < "$IP_BLOCKLIST_FILE" 2>/dev/null; then
215             LAST_IP_MTIME="$CURRENT_MTIME"
216             echo "routing-fix: IP blocklist loaded ($(ipset list block_malicious | grep -c '[0-9]') entries)."
217         else
218             echo "routing-fix: Warning: ipset restore failed. Retrying next cycle."
219             return 1
220         fi
221     fi
222
223     # Apply FORWARD rules in BOTH iptables backends.
224     for ipt in iptables iptables-legacy; do
225         command -v "$ipt" >/dev/null 2>&1 || continue
226
227         create_log_drop_chain "$ipt" LOG_DROP_MALICIOUS_DST "IP_BLOCK_MALICIOUS_DST: "
228         create_log_drop_chain "$ipt" LOG_DROP_MALICIOUS_SRC "IP_BLOCK_MALICIOUS_SRC: "
229
230         # Clean up old plain DROP rules to ensure we hit the log-and-drop chains
231         while $ipt -D FORWARD -m set --match-set block_malicious dst -j DROP 2>/dev/null; do ;; done
232         while $ipt -D FORWARD -m set --match-set block_malicious src -j DROP 2>/dev/null; do ;; done
233
234         if ! $ipt -C FORWARD -m set --match-set block_malicious dst -j LOG_DROP_MALICIOUS_DST 2>/dev/null; then
235             $ipt -I FORWARD -m set --match-set block_malicious dst -j LOG_DROP_MALICIOUS_DST 2>/dev/null || true
236         fi
237
238         if ! $ipt -C FORWARD -m set --match-set block_malicious src -j LOG_DROP_MALICIOUS_SRC 2>/dev/null; then
239             $ipt -I FORWARD -m set --match-set block_malicious src -j LOG_DROP_MALICIOUS_SRC 2>/dev/null || true
240         fi
241     done
242 }
243
244 # Start background block logger (DNS + IP)
245 if ! pgrep -f "nullexit-logger" >/dev/null; then
246     echo "routing-fix: Starting background block logger..."
247     nullexit-logger &
248 fi
249
250 add_country_block
251 add_ip_blocklist
252
253 echo 'routing-fix: Routes applied.'
254
255 UNHEALTHY_COUNT=0
256
257 # Re-assert loop (every 30 seconds)
258 while true; do
259     # Table 200 routes
260     ip route replace "${WARP_ENDPOINT}" via "${DOCKER_GW}" dev eth0 table 200 2>/dev/null || true
261
262     # Tunnel health check
263     if ! curl -m 3 -sS --interface tun0 https://cloudflare.com/cdn-cgi/trace >/dev/null 2>&1; then
264         UNHEALTHY_COUNT=$((UNHEALTHY_COUNT + 1))
265     else
266         UNHEALTHY_COUNT=0
267         if [ -f "/app/TUNNEL_FAILED_CLOSED.marker" ]; then
268             rm -f "/app/TUNNEL_FAILED_CLOSED.marker"
269         fi
270     fi
271
272     if [ "$UNHEALTHY_COUNT" -ge 3 ]; then
273         echo "routing-fix: Tunnel health check failed $UNHEALTHY_COUNT times. Failing closed."
274         ip route del default dev tun0 table 200 2>/dev/null || true
275         ip route replace blackhole default table 200 2>/dev/null || true
276         touch "/app/TUNNEL_FAILED_CLOSED.marker"

```



```

277 else
278     ip route replace default dev tun0 table 200 2>/dev/null || true
279 fi
280
281 # Main table: keep Docker subnet route
282 ip route replace "${DOCKER_NET}" dev eth0 2>/dev/null || true
283
284 # CGNAT ip rule for Tailscale
285 if ! ip rule show | grep -q '100.64.0.0/10'; then
286     ip rule add to 100.64.0.0/10 lookup 52 pref 99 2>/dev/null || true
287     echo 'routing-fix: CGNAT rule missing, re-injected'
288 fi
289
290 # FORWARD RELATED, ESTABLISHED: Allow return traffic in both iptables
291 # backends. Gluetun may reset the nftables ruleset on VPN reconnect,
292 # and Tailscale may reset the legacy ruleset on auth refresh.
293 add_fwd_related_established
294
295 # Ensure P2P packets bypass WARP to maintain low latency
296 add_tailscale_p2p_bypass
297
298 # Ensure transparent proxying for .onion is active
299 add_onion_transparent_proxy
300
301 # Enforce country blocklist
302 add_country_block
303
304 # Enforce IP blocklist (reloads automatically when file changes)
305 add_ip_blocklist
306
307 sleep 30 &
308 wait $! || true
309 done

```

50 scripts/rule-compiler/go.mod

```

1 module nullexit/rule-compiler
2
3 go 1.22

```

51 scripts/rule-compiler/main.go

```

1 package main
2
3 import (
4     "bufio"
5     "bytes"
6     "crypto/md5"
7     "encoding/base64"
8     "encoding/hex"
9     "fmt"
10    "io"
11    "net/http"
12    "net/netip"
13    "os"
14    "os/exec"
15    "path/filepath"
16    "regexp"
17    "sort"
18    "strings"
19    "sync"
20    "time"
21 )
22
23 const (
24     BlackListPath = "black_list.txt"
25     WhiteListPath = "white_list.txt"
26     OutputPath    = "adguard/work/userfilters/compiled_rules.txt"
27     IpOutputPath  = "adguard/work/userfilters/ip_blocklist.ipset"
28     IpCacheDir    = "adguard/work/userfilters/cache/ip"
29     CacheDir      = "adguard/work/userfilters/cache"
30 )

```

```

31
32 var CoreLists = []string{
33     "https://raw.githubusercontent.com/anudeepND/blacklist/master/adservers.txt",
34     "https://raw.githubusercontent.com/anudeepND/blacklist/master/facebook.txt",
35     "https://raw.githubusercontent.com/crazy-max/WindowsSpyBlocker/master/data/hosts/spy.txt",
36     "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/adblock/native.samsung.txt",
37     "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/adblock/native.apple.txt",
38     "https://abp.oisd.nl/basic/",
39     "https://adaway.org/hosts.txt",
40     "https://pgl.yoyo.org/adservers/serverlist.php?hostformat=adblockplus&showintro=1&mimetype=plaintext",
41     "https://raw.githubusercontent.com/nextdns/cname-cloaking-blocklist/master/domains",
42 }
43
44 var MediumAdditions = []string{
45     "https://raw.githubusercontent.com/lightswitch05/hosts/master/docs/lists/facebook-extended.txt",
46     "https://someonewhocares.org/hosts/zero/hosts",
47     "https://urlhaus.abuse.ch/downloads/hostfile/",
48 }
49
50 var HeavyAdditions = []string{
51     "https://raw.githubusercontent.com/StevenBlack/hosts/master/hosts",
52     "https://raw.githubusercontent.com/Perflyst/PiHoleBlocklist/master/SmartTV-AGH.txt",
53     "https://raw.githubusercontent.com/DandelionSprout/adfilt/master/GameConsoleAdblockList.txt",
54 }
55
56 var IpBlockLists = []string{
57     "https://feodotracker.abuse.ch/downloads/ipblocklist.txt",
58     "https://www.spamhaus.org/drop/drop.txt",
59     "https://rules.emergingthreats.net/fwrules/emerging-Block-IPs.txt",
60     "https://cinsscore.com/list/ci-badguys.txt",
61 }
62
63 func getProfiles() map[string][]string {
64     profiles := make(map[string][]string)
65
66     light := append([]string(nil), CoreLists...)
67     light = append(light, "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/domains/light.txt")
68     profiles["light"] = light
69
70     medium := append([]string(nil), CoreLists...)
71     medium = append(medium, MediumAdditions...)
72     medium = append(medium, "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/domains/multi.txt")
73     profiles["medium"] = medium
74
75     heavy := append([]string(nil), CoreLists...)
76     heavy = append(heavy, MediumAdditions...)
77     heavy = append(heavy, HeavyAdditions...)
78     heavy = append(heavy, "https://raw.githubusercontent.com/hagezi/dns-blocklists/main/domains/pro.txt")
79     profiles["heavy"] = heavy
80
81     return profiles
82 }
83
84 func loadEnvProfile() string {
85     profile := "medium"
86
87     if bytesData, err := os.ReadFile(".env"); err == nil {
88         scanner := bufio.NewScanner(strings.NewReader(string(bytesData)))
89         for scanner.Scan() {
90             line := strings.TrimSpace(scanner.Text())
91             if line != "" && !strings.HasPrefix(line, "#") {
92                 parts := strings.SplitN(line, "=", 2)
93                 if len(parts) == 2 {
94                     key := strings.TrimSpace(parts[0])
95                     val := strings.TrimSpace(parts[1])
96                     if key == "GATEWAY_RULE_PROFILE" {
97                         val = strings.Trim(val, "\"'")
98                         profile = strings.ToLower(val)
99                     }
100                 }
101             }
102         }
103     } else if !os.IsNotExist(err) {
104         fmt.Printf("Warning: Failed to read .env file for profile configuration (%v). Using default.\n", err)
105     }
106
107     if envVal := os.Getenv("GATEWAY_RULE_PROFILE"); envVal != "" {
108         profile = strings.ToLower(envVal)
109     }
110
111     profiles := getProfiles()

```

```

112 if _, exists := profiles[profile]; !exists {
113     fmt.Printf("Warning: Profile '%s' is invalid. Falling back to 'medium'.\n", profile)
114     profile = "medium"
115 }
116 return profile
117 }
118
119 func loadDomains(filepath string) map[string]bool {
120     domains := make(map[string]bool)
121     if bytesData, err := os.ReadFile(filepath); err == nil {
122         scanner := bufio.NewScanner(strings.NewReader(string(bytesData)))
123         for scanner.Scan() {
124             line := strings.TrimSpace(scanner.Text())
125             if line != "" && !strings.HasPrefix(line, "#") {
126                 domains[strings.ToLower(line)] = true
127             }
128         }
129     }
130     return domains
131 }
132
133 func parseDomainsFromContent(content string) map[string]bool {
134     domains := make(map[string]bool)
135     scanner := bufio.NewScanner(strings.NewReader(content))
136     ipRegex := regexp.MustCompile(`^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$`)
137     agRegex := regexp.MustCompile(`^\|([a-z0-9.-]+)\|`$`)
138     domainRegex := regexp.MustCompile(`^[a-z0-9.-]+$`)
139
140     for scanner.Scan() {
141         line := strings.TrimSpace(scanner.Text())
142         if line == "" || strings.HasPrefix(line, "!") || strings.HasPrefix(line, "[") {
143             continue
144         }
145         parts := strings.SplitN(line, "#", 2)
146         line = strings.TrimSpace(parts[0])
147         if line == "" {
148             continue
149         }
150
151         fields := strings.Fields(line)
152         var rule string
153         if len(fields) >= 2 {
154             if ipRegex.MatchString(fields[0]) {
155                 rule = fields[1]
156             } else {
157                 rule = fields[0]
158             }
159         } else {
160             rule = fields[0]
161         }
162
163         rule = strings.TrimSpace(strings.ToLower(rule))
164
165         var domain string
166         if m := agRegex.FindStringSubmatch(rule); m != nil {
167             domain = m[1]
168         } else if domainRegex.MatchString(rule) {
169             domain = rule
170         } else {
171             domain = rule
172         }
173
174         if domain != "" && domain != "localhost" && domain != "0.0.0.0" && domain != "127.0.0.1" && domain != "
175             broadcasthost" {
176             domains[domain] = true
177         }
178     }
179     return domains
180 }
181
182 // WARNING: Parsing AdGuardHome.yaml using regex instead of a proper YAML parser
183 // is brittle and assumes the exact format currently emitted by AdGuard.
184 // If an AdGuard version bump updates the configuration schema format,
185 // this parser will fail silently or loudly and should be refactored to use gopkg.in/yaml.v3.
186 func getAdguardNativeLists() []string {
187     yamlPath := "adguard/conf/AdGuardHome.yaml"
188     var enabledUrls []string
189     if _, err := os.Stat(yamlPath); os.IsNotExist(err) {
190         return enabledUrls
191     }

```

```

192 contentBytes, err := os.ReadFile(yamlPath)
193 if err != nil {
194     fmt.Printf("Warning: Failed to parse AdGuardHome.yaml for native lists (%v)\n", err)
195     return enabledUrls
196 }
197 content := string(contentBytes)
198
199 filtersRegex := regexp.MustCompile(`(?ms)^filters:(.*)"?(?^[a-zA-Z_]+:|\z)`
200 match := filtersRegex.FindStringSubmatch(content)
201 if len(match) > 1 {
202     filtersText := match[1]
203     blocks := strings.Split(filtersText, "\n - ")
204     urlRegex := regexp.MustCompile(`url:\s*(\S+)`)
205
206     for _, block := range blocks {
207         if strings.TrimSpace(block) == "" {
208             continue
209         }
210         if strings.Contains(block, "enabled: true") {
211             urlMatch := urlRegex.FindStringSubmatch(block)
212             if len(urlMatch) > 1 {
213                 url := urlMatch[1]
214                 if !strings.Contains(url, "compiled_rules.txt") {
215                     enabledUrls = append(enabledUrls, url)
216                 }
217             }
218         }
219     }
220 }
221 return enabledUrls
222 }
223
224 func getAdguardCompiledRulesCachePath() string {
225     yamlPath := "adguard/conf/AdGuardHome.yaml"
226     if _, err := os.Stat(yamlPath); os.IsNotExist(err) {
227         return ""
228     }
229
230     contentBytes, err := os.ReadFile(yamlPath)
231     if err != nil {
232         fmt.Printf("Warning: Failed to resolve compiled_rules.txt cache path (%v)\n", err)
233         return ""
234     }
235     content := string(contentBytes)
236
237     filtersRegex := regexp.MustCompile(`(?ms)^filters:(.*)"?(?^[a-zA-Z_]+:|\z)`
238     match := filtersRegex.FindStringSubmatch(content)
239     if len(match) > 1 {
240         blocks := strings.Split(match[1], "\n - ")
241         idRegex := regexp.MustCompile(`id:\s*(\d+)`)
242         for _, block := range blocks {
243             if !strings.Contains(block, "compiled_rules.txt") {
244                 continue
245             }
246             idMatch := idRegex.FindStringSubmatch(block)
247             if len(idMatch) > 1 {
248                 return fmt.Sprintf("adguard/work/data/filters/%s.txt", idMatch[1])
249             }
250         }
251     }
252     return ""
253 }
254
255 func handleFetchError(urlStr string, cacheFile string, err error) map[string]bool {
256     if _, statErr := os.Stat(cacheFile); statErr == nil {
257         fmt.Printf("-> Warning: Failed to fetch %s (%v). Falling back to expired local cache.\n", urlStr, err)
258         content, readErr := os.ReadFile(cacheFile)
259         if readErr == nil {
260             domains := parseDomainsFromContent(string(content))
261             fmt.Printf("-> Loaded from expired cache (%d domains).\n", len(domains))
262             return domains
263         }
264         fmt.Printf("-> Warning: Failed to read expired cache (%v). Using local lists only.\n", readErr)
265     } else {
266         fmt.Printf("-> Warning: Failed to fetch %s (%v). Using local lists only for this source.\n", urlStr, err)
267     }
268     return make(map[string]bool)
269 }
270
271 func fetchRemoteDomains(urlStr string) map[string]bool {
272     os.MkdirAll(CacheDir, 0755)

```

```

273 hash := md5.Sum([]byte(urlStr))
274 urlHash := hex.EncodeToString(hash[:])
275 cacheFile := filepath.Join(CacheDir, fmt.Sprintf("%s.txt", urlHash))
276
277
278 if stat, err := os.Stat(cacheFile); err == nil {
279     fileAge := time.Since(stat.ModTime()).Seconds()
280     if fileAge < 86400 {
281         fmt.Printf("Loading remote blacklist from cache: %s\n", urlStr)
282         content, err := os.ReadFile(cacheFile)
283         if err == nil {
284             domains := parseDomainsFromContent(string(content))
285             fmt.Printf(" -> Loaded from cache (copy is %.1f hours old, %d domains).\n", fileAge/3600.0, len(domains))
286         }
287         return domains
288     }
289     fmt.Printf(" -> Warning: Failed to read cache file %s (%v). Re-fetching...\n", cacheFile, err)
290 }
291
292 fmt.Printf("Fetching remote blacklist from: %s ...\n", urlStr)
293
294 req, err := http.NewRequest("GET", urlStr, nil)
295 if err != nil {
296     return handleFetchError(urlStr, cacheFile, err)
297 }
298 req.Header.Set("User-Agent", "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)")
299
300 client := &http.Client{Timeout: 15 * time.Second}
301 resp, err := client.Do(req)
302 if err != nil {
303     return handleFetchError(urlStr, cacheFile, err)
304 }
305 defer resp.Body.Close()
306
307 if resp.StatusCode != 200 {
308     return handleFetchError(urlStr, cacheFile, fmt.Errorf("HTTP %d", resp.StatusCode))
309 }
310
311 bodyBytes, err := io.ReadAll(resp.Body)
312 if err != nil {
313     return handleFetchError(urlStr, cacheFile, err)
314 }
315
316 content := string(bodyBytes)
317 domains := parseDomainsFromContent(content)
318
319 if len(domains) < 10 {
320     return handleFetchError(urlStr, cacheFile, fmt.Errorf("Sanity check failed: only found %d domains. Possible 404 or bad URL", len(domains)))
321 }
322
323 err = os.WriteFile(cacheFile, bodyBytes, 0644)
324 if err != nil {
325     fmt.Printf(" -> Warning: Failed to save cache file (%v)\n", err)
326 }
327
328 fmt.Printf(" -> Successfully fetched and cached %d domains.\n", len(domains))
329 return domains
330 }
331
332 func optimizeSubdomains(domains map[string]bool, listName string) map[string]bool {
333     fmt.Printf("Optimizing %s by removing redundant subdomains...\n", listName)
334     rawCount := len(domains)
335     if rawCount == 0 {
336         return make(map[string]bool)
337     }
338
339     optimized := make(map[string]bool)
340     for domain := range domains {
341         if strings.ContainsAny(domain, "|^@$/") {
342             optimized[domain] = true
343             continue
344         }
345
346         parts := strings.Split(domain, ".")
347         hasParent := false
348
349         for i := 1; i < len(parts)-1; i++ {
350             parent := strings.Join(parts[i:], ".")
351             if domains[parent] {

```

```

352     hasParent = true
353     break
354 }
355 }
356
357 if !hasParent {
358     optimized[domain] = true
359 }
360 }
361
362 saved := rawCount - len(optimized)
363 reduction := float64(0)
364 if rawCount > 0 {
365     reduction = (float64(saved) / float64(rawCount)) * 100
366 }
367 fmt.Printf(" -> Reduced %s from %d to %d domains (-%d / %.1f%% reduction).\n", listName, rawCount, len(
    optimized), saved, reduction)
368 return optimized
369 }
370
371 func parseIpsFromContent(content string) map[string]bool {
372     ips := make(map[string]bool)
373     scanner := bufio.NewScanner(strings.NewReader(content))
374
375     for scanner.Scan() {
376         line := strings.TrimSpace(scanner.Text())
377         if line == "" || strings.HasPrefix(line, "#") || strings.HasPrefix(line, ";") {
378             continue
379         }
380
381         fields := strings.Fields(line)
382         if len(fields) > 0 {
383             token := strings.TrimRight(fields[0], ";")
384             if _, err := netip.ParsePrefix(token); err == nil {
385                 ips[token] = true
386             } else if _, err := netip.ParseAddr(token); err == nil {
387                 ips[token] = true
388             }
389         }
390     }
391     return ips
392 }
393
394 func handleIpFetchError(urlStr string, cacheFile string, err error) map[string]bool {
395     if _, statErr := os.Stat(cacheFile); statErr == nil {
396         fmt.Printf(" -> Warning: fetch failed (%v). Falling back to stale cache.\n", err)
397         content, readErr := os.ReadFile(cacheFile)
398         if readErr == nil {
399             ips := parseIpsFromContent(string(content))
400             fmt.Printf(" -> %d IPs loaded from stale cache.\n", len(ips))
401             return ips
402         }
403         fmt.Printf(" -> Warning: stale cache also failed (%v).\n", readErr)
404     } else {
405         fmt.Printf(" -> Warning: %s failed (%v). No cache available. Skipping.\n", urlStr, err)
406     }
407     return make(map[string]bool)
408 }
409
410 func fetchRemoteIps(urlStr string) map[string]bool {
411     os.MkdirAll(IpCacheDir, 0755)
412
413     hash := md5.Sum([]byte(urlStr))
414     urlHash := hex.EncodeToString(hash[:])
415     cacheFile := filepath.Join(IpCacheDir, fmt.Sprintf("%s.txt", urlHash))
416
417     if stat, err := os.Stat(cacheFile); err == nil {
418         fileAge := time.Since(stat.ModTime()).Seconds()
419         if fileAge < 86400 {
420             fmt.Printf("Loading IP feed from cache: %s\n", urlStr)
421             content, err := os.ReadFile(cacheFile)
422             if err == nil {
423                 ips := parseIpsFromContent(string(content))
424                 fmt.Printf(" -> %d IPs (cache is %.1fh old).\n", len(ips), fileAge/3600.0)
425                 return ips
426             }
427             fmt.Printf(" -> Warning: cache read failed (%v). Re-fetching...\n", err)
428         }
429     }
430
431     fmt.Printf("Fetching IP feed: %s ...\n", urlStr)

```

```

432 req, err := http.NewRequest("GET", urlStr, nil)
433 if err != nil {
434     return handleIpFetchError(urlStr, cacheFile, err)
435 }
436 req.Header.Set("User-Agent", "Mozilla/5.0")
437
438 client := &http.Client{Timeout: 15 * time.Second}
439 resp, err := client.Do(req)
440 if err != nil {
441     return handleIpFetchError(urlStr, cacheFile, err)
442 }
443 defer resp.Body.Close()
444
445 if resp.StatusCode != 200 {
446     return handleIpFetchError(urlStr, cacheFile, fmt.Errorf("HTTP %d", resp.StatusCode))
447 }
448
449 bodyBytes, err := io.ReadAll(resp.Body)
450 if err != nil {
451     return handleIpFetchError(urlStr, cacheFile, err)
452 }
453
454 content := string(bodyBytes)
455 ips := parseIpsFromContent(content)
456 if len(ips) < 1 {
457     return handleIpFetchError(urlStr, cacheFile, fmt.Errorf("Sanity check failed: only %d IPs found. Possible
458         404", len(ips)))
459 }
460
461 err = os.WriteFile(cacheFile, bodyBytes, 0644)
462 if err != nil {
463     fmt.Printf(" -> Warning: cache write failed (%v)\n", err)
464 }
465
466 fmt.Printf(" -> %d IPs fetched and cached.\n", len(ips))
467 return ips
468 }
469
470 func compileIpBlocklist() int {
471     fmt.Println("\n IP Blocklist Compilation ")
472
473     allIps := make(map[string]bool)
474     var mu sync.Mutex
475     var wg sync.WaitGroup
476
477     maxThreads := 8
478     if len(IpBlockLists) < 8 {
479         maxThreads = len(IpBlockLists)
480     }
481     semaphore := make(chan struct{}, maxThreads)
482
483     for _, urlStr := range IpBlockLists {
484         wg.Add(1)
485         go func(u string) {
486             defer wg.Done()
487             semaphore <- struct{}{}
488             defer func() { <-semaphore }()
489
490             ips := fetchRemoteIps(u)
491             mu.Lock()
492             for ip := range ips {
493                 allIps[ip] = true
494             }
495             mu.Unlock()
496         }(urlStr)
497     }
498     wg.Wait()
499
500     fmt.Printf("Total unique entries before filtering: %d\n", len(allIps))
501
502     reservedStr := []string{
503         "10.0.0.0/8",
504         "172.16.0.0/12",
505         "192.168.0.0/16",
506         "127.0.0.0/8",
507         "169.254.0.0/16",
508         "100.64.0.0/10",
509         "0.0.0.0/8",
510     }
511     var reserved []netip.Prefix
512     for _, s := range reservedStr {

```



```

512     p, _ := netip.ParsePrefix(s)
513     reserved = append(reserved, p)
514 }
515
516 cleanIps := make(map[string]bool)
517 for entry := range allIps {
518     var prefix netip.Prefix
519     p, err := netip.ParsePrefix(entry)
520     if err != nil {
521         addr, err2 := netip.ParseAddr(entry)
522         if err2 != nil {
523             continue
524         }
525         prefix = netip.PrefixFrom(addr, addr.BitLen())
526     } else {
527         prefix = p
528     }
529
530     overlaps := false
531     for _, r := range reserved {
532         if r.Overlaps(prefix) {
533             overlaps = true
534             break
535         }
536     }
537     if !overlaps {
538         cleanIps[entry] = true
539     }
540 }
541
542 removed := len(allIps) - len(cleanIps)
543 if removed > 0 {
544     fmt.Printf(" -> Removed %d private/reserved entries.\n", removed)
545 }
546 fmt.Printf(" -> Final IP blocklist: %d entries.\n", len(cleanIps))
547
548 os.MkdirAll(filepath.Dir(IpOutputPath), 0755)
549
550 f, err := os.Create(IpOutputPath)
551 if err != nil {
552     fmt.Printf("Error writing IP output file: %v\n", err)
553     return len(cleanIps)
554 }
555 defer f.Close()
556
557 f.WriteString("# nullexit Compiled IP Blocklist\n")
558 f.WriteString("# Sources: Feodo Tracker, Spamhaus DROP/EDROP, Emerging Threats, CINS\n")
559 f.WriteString(fmt.Sprintf("# Entries: %d\n\n", len(cleanIps)))
560
561 f.WriteString("create block_malicious_new hash:net maxelem 200000 -exist\n")
562
563 var sortedIps []string
564 for ip := range cleanIps {
565     sortedIps = append(sortedIps, ip)
566 }
567 sort.Strings(sortedIps)
568
569 for _, ip := range sortedIps {
570     f.WriteString(fmt.Sprintf("add block_malicious_new %s -exist\n", ip))
571 }
572
573 f.WriteString("create block_malicious hash:net maxelem 200000 -exist\n")
574 f.WriteString("swap block_malicious block_malicious_new\n")
575 f.WriteString("destroy block_malicious_new\n")
576
577 fmt.Printf("IP blocklist written to %s\n", IpOutputPath)
578 return len(cleanIps)
579 }
580
581 func main() {
582     profile := loadEnvProfile()
583     fmt.Printf("Active memory profile: '%s'\n", strings.ToUpper(profile))
584
585     localBlackList := loadDomains(BlackListPath)
586     whiteList := loadDomains(WhiteListPath)
587
588     blackList := make(map[string]bool)
589     for k := range localBlackList {
590         blackList[k] = true
591     }
592

```

```

593 profiles := getProfiles()
594 urlsToFetch := profiles[profile]
595
596 fmt.Printf("\nStarting concurrent downloads for %d lists...\n", len(urlsToFetch))
597 startTime := time.Now()
598
599 var mu sync.Mutex
600 var wg sync.WaitGroup
601
602 maxThreads := 16
603 if len(urlsToFetch) < 16 {
604     maxThreads = len(urlsToFetch)
605 }
606 semaphore := make(chan struct{}, maxThreads)
607
608 for _, urlStr := range urlsToFetch {
609     wg.Add(1)
610     go func(u string) {
611         defer wg.Done()
612         semaphore <- struct{}{}
613         defer func() { <-semaphore }()
614
615         domains := fetchRemoteDomains(u)
616         mu.Lock()
617         for d := range domains {
618             blackList[d] = true
619         }
620         mu.Unlock()
621     }(urlStr)
622 }
623 wg.Wait()
624
625 fmt.Printf("Finished concurrent downloads in %.2f seconds using %d threads.\n", time.Since(startTime).Seconds
626 (), maxThreads)
627
628 adguardNativeUrls := getAdguardNativeLists()
629 var adguardNativeDomains map[string]bool
630 if len(adguardNativeUrls) > 0 {
631     fmt.Printf("\nFetching %d AdGuard native list(s) for deduplication...\n", len(adguardNativeUrls))
632     adguardNativeDomains = make(map[string]bool)
633
634     maxNativeThreads := 4
635     if len(adguardNativeUrls) < 4 {
636         maxNativeThreads = len(adguardNativeUrls)
637     }
638     nativeSemaphore := make(chan struct{}, maxNativeThreads)
639
640     for _, urlStr := range adguardNativeUrls {
641         wg.Add(1)
642         go func(u string) {
643             defer wg.Done()
644             nativeSemaphore <- struct{}{}
645             defer func() { <-nativeSemaphore }()
646
647             domains := fetchRemoteDomains(u)
648             mu.Lock()
649             for d := range domains {
650                 adguardNativeDomains[d] = true
651             }
652             mu.Unlock()
653         }(urlStr)
654     }
655     wg.Wait()
656
657     if len(adguardNativeDomains) > 0 {
658         fmt.Println("Deduplicating compiled rules against AdGuard native lists...")
659         originalSize := len(blackList)
660         for d := range adguardNativeDomains {
661             delete(blackList, d)
662         }
663         fmt.Printf(" -> Removed %d redundant rules already covered by AdGuard.\n", originalSize-len(blackList))
664     }
665 }
666
667 var contradictions []string
668 for d := range whiteList {
669     if blackList[d] {
670         contradictions = append(contradictions, d)
671     }
672 }

```

```

673 if len(contradictions) > 0 {
674     fmt.Printf("Detected %d contradictions. Whitelist taking priority.\n", len(contradictions))
675     sort.Strings(contradictions)
676     for _, domain := range contradictions {
677         if localBlackList[domain] {
678             fmt.Printf(" -> Removed local blacklist domain: %s\n", domain)
679         }
680         delete(blackList, domain)
681     }
682 }
683
684 blackList = optimizeSubdomains(blackList, "blacklist")
685 whiteList = optimizeSubdomains(whiteList, "whitelist")
686
687 os.MkdirAll(filepath.Dir(OutputPath), 0755)
688 outFile, err := os.Create(OutputPath)
689 if err != nil {
690     fmt.Printf("Error creating output file: %v\n", err)
691     return
692 }
693 defer outFile.Close()
694
695 outFile.WriteString("! Custom Compiled Rules (Auto-Generated)\n")
696 outFile.WriteString(fmt.Sprintf("! Memory Profile: %s\n", strings.ToUpper(profile)))
697 outFile.WriteString(fmt.Sprintf("! Total Block Rules: %d\n", len(blackList)))
698 outFile.WriteString(fmt.Sprintf("! Native AdGuard Rules: %d\n", len(adguardNativeDomains)))
699 outFile.WriteString(fmt.Sprintf("! Total Whitelist Rules: %d\n", len(whiteList)))
700
701 outFile.WriteString("! --- Blacklist Rules ---\n")
702
703 var sortedBlacklist []string
704 for d := range blackList {
705     sortedBlacklist = append(sortedBlacklist, d)
706 }
707 sort.Strings(sortedBlacklist)
708
709 for _, domain := range sortedBlacklist {
710     if strings.Contains(domain, "$") {
711         parts := strings.SplitN(domain, "$", 2)
712         dom, mod := parts[0], parts[1]
713         if strings.HasPrefix(dom, "|") {
714             if strings.HasSuffix(dom, "~") {
715                 outFile.WriteString(fmt.Sprintf("%s%s\n", dom, mod))
716             } else {
717                 outFile.WriteString(fmt.Sprintf("%s~%s\n", dom, mod))
718             }
719         } else {
720             outFile.WriteString(fmt.Sprintf("||%s~%s\n", dom, mod))
721         }
722     } else if strings.HasPrefix(domain, "/") || strings.HasPrefix(domain, "|") || strings.HasSuffix(domain, "|") || strings.HasSuffix(domain, "~") {
723         outFile.WriteString(fmt.Sprintf("%s\n", domain))
724     } else {
725         outFile.WriteString(fmt.Sprintf("||%s~\n", domain))
726     }
727 }
728
729 outFile.WriteString("\n! --- Whitelist Rules ---\n")
730 var sortedWhitelist []string
731 for d := range whiteList {
732     sortedWhitelist = append(sortedWhitelist, d)
733 }
734 sort.Strings(sortedWhitelist)
735
736 for _, domain := range sortedWhitelist {
737     if strings.HasPrefix(domain, "/") || strings.HasPrefix(domain, "|") || strings.HasSuffix(domain, "|") || strings.HasSuffix(domain, "~") || strings.HasPrefix(domain, "@@") {
738         rule := domain
739         if !strings.HasPrefix(domain, "@@") {
740             rule = "@@" + domain
741         }
742         outFile.WriteString(fmt.Sprintf("%s\n", rule))
743     } else {
744         outFile.WriteString(fmt.Sprintf("@@||%s~\n", domain))
745     }
746 }
747
748 fmt.Printf("\nSuccessfully compiled %d block rules and %d allow rules to %s\n", len(blackList), len(whiteList), OutputPath)
749
750 cachedFilterPath := getAdguardCompiledRulesCachePath()

```

```

751 if cachedFilterPath != "" {
752     input, err := os.ReadFile(OutputPath)
753     if err == nil {
754         err = os.WriteFile(cachedFilterPath, input, 0644)
755         if err == nil {
756             fmt.Printf("Updated AdGuard filter cache: %s\n", cachedFilterPath)
757         } else {
758             fmt.Printf("Warning: Could not update AdGuard filter cache %s (%v)\n", cachedFilterPath, err)
759         }
760     } else {
761         fmt.Printf("Warning: Could not read OutputPath to copy to cache (%v)\n", err)
762     }
763 } else {
764     fmt.Println("Warning: Could not determine compiled_rules.txt cache path from AdGuardHome.yaml; skipping
       cache update.")
765 }
766
767 adguardReachable := false
768
769 dockerComposeYamlExists := false
770 if _, err := os.Stat("docker-compose.yml"); err == nil {
771     dockerComposeYamlExists = true
772 }
773 dockerPath, err := exec.LookPath("docker")
774 if err == nil && dockerComposeYamlExists {
775     cmd := exec.Command(dockerPath, "compose", "ps", "--status", "running", "-q", "adguardhome")
776     out, err := cmd.Output()
777     if err == nil && strings.TrimSpace(string(out)) != "" {
778         fmt.Println("Force-recreating AdGuard Home container to drop persisted filter state...")
779         upCmd := exec.Command(dockerPath, "compose", "up", "-d", "--force-recreate", "--no-deps", "adguardhome")
780         err := upCmd.Run()
781         if err == nil {
782             fmt.Println("AdGuard Home force-recreated successfully.")
783             time.Sleep(8 * time.Second)
784             adguardReachable = true
785         } else {
786             fmt.Println("Failed to restart AdGuard Home. Is it running?")
787         }
788     }
789 }
790
791 if adguardReachable {
792     agPass := "nullexit"
793     if bytesData, err := os.ReadFile(".env"); err == nil {
794         scanner := bufio.NewScanner(strings.NewReader(string(bytesData)))
795         for scanner.Scan() {
796             line := scanner.Text()
797             if strings.HasPrefix(line, "ADGUARD_PASSWORD=") {
798                 parts := strings.SplitN(line, "=", 2)
799                 if len(parts) == 2 {
800                     agPass = strings.Trim(strings.TrimSpace(parts[1]), "\"'")
801                 }
802                 break
803             }
804         }
805     }
806
807     creds := base64.StdEncoding.EncodeToString([]byte(fmt.Sprintf("admin:%s", agPass)))
808     req, err := http.NewRequest("POST", "http://127.0.0.1:3000/control/filtering/refresh", bytes.NewBuffer([]
       byte("{}")))
809     if err == nil {
810         req.Header.Set("Authorization", fmt.Sprintf("Basic %s", creds))
811         req.Header.Set("Content-Type", "application/json")
812         client := &http.Client{Timeout: 15 * time.Second}
813         resp, err := client.Do(req)
814         if err == nil {
815             fmt.Printf("Triggered AdGuard filter refresh via REST API (HTTP=%d).\n", resp.StatusCode)
816             resp.Body.Close()
817         } else {
818             fmt.Printf("Warning: AdGuard filter refresh API call failed (%v). New whitelist will not load until
       next refresh tick (~24h) or manual refresh.\n", err)
819         }
820     }
821 }
822
823 compileIpBlocklist()
824 }

```

52 scripts/setup-common.sh

```
1 #!/bin/bash
2 # 1. Docker
3 step "Checking Docker"
4
5 if ! command -v docker >> output.log 2>&1; then
6     echo ""
7     if [[ "$OSTYPE" == "darwin"* ]]; then
8         echo " Docker is not installed."
9         # Note: Homebrew aggressively rolls forward and discourages version pinning.
10        # This setup is confirmed stable on Colima v0.10.3 and Docker v29.6.1.
11        read -rp " Would you like to automatically install Colima & Docker via Homebrew? [y/N]: "
12        INSTALL_DOCKER
13        if [[ "$INSTALL_DOCKER" == "y" || "$INSTALL_DOCKER" == "Y" ]]; then
14            step "Installing Colima and Docker..."
15            brew install colima docker docker-compose
16            step "Starting Colima VM..."
17            colima start --memory 0.6 --vm-type vz --network-address --network-mode bridged
18        else
19            echo ""
20            echo " Please install Docker manually. Two options:"
21            echo " A) Colima (Recommended): brew install colima docker docker-compose && colima start --memory
22            0.6 --vm-type vz --network-address --network-mode bridged"
23            echo " B) Docker Desktop: https://www.docker.com/products/docker-desktop/"
24            die "Install Docker, then re-run this script."
25        fi
26    else
27        echo " Docker is not installed."
28        # Note: The Docker convenience script always fetches the latest stable release.
29        # This setup is confirmed stable on Docker Engine v29.6.1.
30        read -rp " Would you like to automatically install Docker? (Requires sudo) [y/N]: " INSTALL_DOCKER
31        if [[ "$INSTALL_DOCKER" == "y" || "$INSTALL_DOCKER" == "Y" ]]; then
32            step "Installing Docker..."
33            curl -fsSL https://get.docker.com | sh
34            sudo usermod -aG docker "$USER"
35            echo -e "\n\033[0;32m[OK] Docker installed successfully!\033[0m"
36            echo -e "\033[0;33m[!] CRITICAL: You must log out of your SSH session and log back in for Docker
37            permissions to apply.\033[0m"
38            die "Please log out, log back in, and re-run setup.sh."
39        else
40            echo " Please install Docker manually:"
41            echo " curl -fsSL https://get.docker.com | sh"
42            echo " sudo usermod -aG docker \"$USER && newgrp docker"
43            die "Install Docker, then re-run this script."
44        fi
45    fi
46 fi
47
48 if ! docker info >> output.log 2>&1; then
49     echo ""
50     if [[ "$OSTYPE" == "darwin"* ]]; then
51         echo " Docker is installed but not running."
52         echo " If using Colima: colima start --memory 0.6 --vm-type vz --network-address --network-mode
53         bridged"
54         echo " If using Docker Desktop: open the app."
55     else
56         echo " Docker is installed but not running."
57         echo " Run: sudo systemctl start docker"
58     fi
59     die "Start Docker, then re-run this script."
60 fi
61
62 if ! docker compose version >> output.log 2>&1; then
63     die "Docker Compose v2 not found. Update Docker or install the compose plugin:\n https://docs.docker.com/
64     compose/install/"
65 fi
66
67 ok "Docker $(docker --version | grep -oE '[0-9.]+' | head -1) is running."
68
69 # 1.5 Python 3
70 step "Checking Python 3"
71
72 if ! command -v python3 >> output.log 2>&1; then
73     echo ""
74     if [[ "$OSTYPE" == "darwin"* ]]; then
75         echo " Python 3 is not installed."
76         echo " macOS includes it via Xcode Command Line Tools, or you can install it via Homebrew:"
77         echo " brew install python3"
```

```

73     die "Install Python 3, then re-run this script."
74 else
75     echo " Python 3 is not installed."
76     echo " Please install it using your system's package manager. For example:"
77     echo " Debian/Ubuntu: sudo apt install python3"
78     echo " Fedora: sudo dnf install python3"
79     echo " Arch: sudo pacman -S python"
80     die "Install Python 3, then re-run this script."
81 fi
82 fi
83
84 # Note: This setup is confirmed stable on Python 3.9.6 (macOS default).
85 PYTHON_VER=$(python3 --version 2>&1 | head -n 1)
86 ok "$PYTHON_VER is installed."
87
88 # 2. Tailscale (host)
89 step "Checking Tailscale (host)"
90 # The host needs its own Tailscale installation separate from the Docker
91 # container so Toggle-Gateway can run 'tailscale down' before stopping
92 # containers and 'tailscale up' after starting them. Without this, the
93 # exit-node deadlock that setup.sh is designed to prevent can still occur.
94
95 # Resolve the tailscale binary: CLI (brew/package) or bundled macOS app
96 TAILSCALE_BIN=""
97 if command -v tailscale >> output.log 2>&1; then
98     TAILSCALE_BIN="tailscale"
99 elif [[ -x "/Applications/Tailscale.app/Contents/MacOS/Tailscale" ]]; then
100     TAILSCALE_BIN="/Applications/Tailscale.app/Contents/MacOS/Tailscale"
101 fi
102
103 RECOMMENDED_TS_VERSION="1.98.5"
104
105 if [[ -z "$TAILSCALE_BIN" ]]; then
106     warn "Tailscale not found on host installing..."
107
108     if [[ "$OSTYPE" == "darwin"* ]]; then
109         if command -v brew >> output.log 2>&1; then
110             step "Installing Tailscale CLI formula via Homebrew (standalone, no .app GUI)..."
111             # Note: We install Tailscale via Homebrew, targeting the stable version (recommended: 1.98.5)
112             brew install tailscale -q
113             TAILSCALE_BIN="tailscale"
114
115             step "Starting tailscaled as a per-user LaunchAgent (auto-starts at login)..."
116             if brew services start tailscale 2>&1 | grep -qE 'Successfully|started'; then
117                 ok "tailscaled LaunchAgent registered."
118             else
119                 warn "Could not auto-start tailscaled. Run 'brew services start tailscale' manually."
120                 warn "For a system-wide LaunchDaemon that runs before login, run instead:"
121                 warn " sudo brew services start tailscale"
122             fi
123             echo ""
124         else
125             die "Homebrew not found. Install Tailscale manually (recommended version: ${RECOMMENDED_TS_VERSION}): \n https://tailscale.com/download/mac\n Then re-run this script."
126         fi
127     elif [[ "$OSTYPE" == "linux-gnu"* ]]; then
128         curl -fsSL https://tailscale.com/install.sh | sh
129         sudo systemctl enable --now tailscaled 2>> output.log || true
130         TAILSCALE_BIN="tailscale"
131     else
132         die "Unsupported OS. Install Tailscale manually (recommended version: ${RECOMMENDED_TS_VERSION}): \n https://tailscale.com/download\n Then re-run this script."
133     fi
134
135     TAILSCALE_VER=$(("${TAILSCALE_BIN}" version 2>> output.log | head -n 1))
136     if [[ "$TAILSCALE_VER" != "$RECOMMENDED_TS_VERSION" ]]; then
137         warn "Tailscale installed, but version ($TAILSCALE_VER) differs from recommended version ($RECOMMENDED_TS_VERSION)."
138     else
139         ok "Tailscale installed (version $TAILSCALE_VER)."
140     fi
141 else
142     TAILSCALE_VER=$(("${TAILSCALE_BIN}" version 2>> output.log | head -n 1))
143     if [[ "$TAILSCALE_VER" != "$RECOMMENDED_TS_VERSION" ]]; then
144         warn "Tailscale is already installed ($TAILSCALE_VER), but recommended version is $RECOMMENDED_TS_VERSION for host/container consistency."
145     else
146         ok "Tailscale is already installed at recommended version ($TAILSCALE_VER)."
147     fi
148 fi
149

```

```

150 fi
151
152 # Authenticate the host machine if it isn't already connected.
153 # This is an interactive step it opens a browser login URL.
154 # We do it now, before containers start, so Toggle-Gateway can
155 # safely run 'tailscale down/up' from day one.
156 if ! ${TAILSCALE_BIN} status >> output.log 2>&1; then
157     echo ""
158     echo " Your host machine needs to join your Tailscale network."
159     echo " A browser window or login URL will appear authenticate, then"
160     echo " return here. The script will continue automatically."
161     echo ""
162     if [[ "$OSTYPE" == "linux-gnu"* ]]; then
163         sudo ${TAILSCALE_BIN} up
164     else
165         ${TAILSCALE_BIN} up
166     fi
167     ok "Host machine connected to Tailscale."
168 else
169     ok "Host machine is already connected to Tailscale."
170 fi
171
172 # 3. wgcf
173 step "Checking wgcf (Cloudflare WARP key tool)"
174
175 if ! command -v wgcf >> output.log 2>&1; then
176     warn "wgcf not found installing..."
177
178     if [[ "$OSTYPE" == "darwin"* ]]; then
179         command -v brew >> output.log 2>&1 || die "Homebrew not found.\nInstall wgcf manually: https://github.com/ViRb3/wgcf/releases"
180         brew install wgcf -q
181
182     elif [[ "$OSTYPE" == "linux-gnu"* ]]; then
183         ARCH=$(uname -m)
184         case $ARCH in
185             x86_64) WGCF_ARCH="amd64" ;;
186             aarch64) WGCF_ARCH="arm64" ;;
187             armv7l) WGCF_ARCH="armv7" ;;
188             *) die "Unsupported architecture: $ARCH.\nInstall wgcf manually: https://github.com/ViRb3/wgcf/releases" ;;
189         esac
190
191         echo " Fetching latest release..."
192         WGCF_VERSION=$(curl -sf https://api.github.com/repos/ViRb3/wgcf/releases/latest \
193             | grep 'tag_name' | cut -d'"' -f4)
194         [[ -z "$WGCF_VERSION" ]] && die "Could not fetch wgcf version. Check your internet connection."
195
196         sudo curl -sL \
197             "https://github.com/ViRb3/wgcf/releases/download/${WGCF_VERSION}/wgcf-${WGCF_VERSION#v}_linux-${WGCF_ARCH}" \
198             -o /usr/local/bin/wgcf
199         sudo chmod +x /usr/local/bin/wgcf
200     else
201         die "Unsupported OS. Install wgcf manually: https://github.com/ViRb3/wgcf/releases"
202     fi
203 fi
204
205 ok "wgcf is ready."
206
207 # 4. WARP profile
208 step "Generating Cloudflare WARP profile"
209
210 if [[ -f "wgcf-profile.conf" ]]; then
211     warn "wgcf-profile.conf already exists skipping key generation."
212 else
213     wgcf register --accept-tos 2>> output.log || die "wgcf register failed. Check your internet connection."
214     wgcf generate 2>> output.log || die "wgcf generate failed."
215     ok "WARP profile generated."
216 fi
217
218 # Parse keys (IPv4 address only gluetun doesn't need the IPv6 one)
219 PRIVATE_KEY=$(awk '/PrivateKey/{print $3}' wgcf-profile.conf)
220 PUBLIC_KEY=$(awk '/PublicKey/{print $3}' wgcf-profile.conf)
221 ADDRESSES=$(awk '/Address/{print $3}' wgcf-profile.conf | grep -v ':' | head -1)
222
223 [[ -z "$PRIVATE_KEY" ]] && die "Could not parse PrivateKey from wgcf-profile.conf"
224 [[ -z "$PUBLIC_KEY" ]] && die "Could not parse PublicKey from wgcf-profile.conf"
225 [[ -z "$ADDRESSES" ]] && die "Could not parse IPv4 Address from wgcf-profile.conf"
226
227 ok "WARP keys extracted."

```



```

228
229 # 5. Tailscale auth key
230 step "Tailscale authentication"
231
232 TS_AUTHKEY=""
233 if [[ -f ".env" ]]; then
234     EXISTING_TS_KEY=$(read_env_var TS_AUTHKEY)
235     if [[ -n "$EXISTING_TS_KEY" ]]; then
236         TS_AUTHKEY="$EXISTING_TS_KEY"
237         warn "Found existing TS_AUTHKEY in .env. Skipping prompt."
238     fi
239 fi
240
241 if [[ -z "$TS_AUTHKEY" ]]; then
242     echo ""
243     echo "    Generate a key at: https://login.tailscale.com/admin/settings/keys"
244     echo "    'Generate auth key' enable Reusable if you plan to re-run setup"
245     echo ""
246     read -rp "    Paste your Tailscale auth key: " TS_AUTHKEY
247     echo ""
248 fi
249
250 [[ -z "$TS_AUTHKEY" ]] && die "Tailscale auth key is required."
251 [[ "$TS_AUTHKEY" != tskey-auth-* ]] && warn "Key doesn't look like a Tailscale auth key proceeding anyway."
252 ok "Auth key accepted."
253
254 # 6. AdGuard Home admin password
255 step "AdGuard Home admin account"
256
257 # Hardcoded default credentials to prevent lockout
258 # Username: admin
259 # Password: nullexit
260 if [[ -f ".env" ]]; then
261     EXISTING_AG_PASS=$(read_env_var ADGUARD_PASSWORD)
262     if [[ -n "$EXISTING_AG_PASS" ]]; then
263         ADGUARD_PASSWORD="$EXISTING_AG_PASS"
264         ADGUARD_PWD_ESC="$EXISTING_AG_PASS"
265         ok "Found existing ADGUARD_PASSWORD in .env."
266     else
267         ADGUARD_PASSWORD="nullexit"
268         ADGUARD_PWD_ESC="nullexit"
269         ok "Default password set to 'nullexit'."
270     fi
271 else
272     ADGUARD_PASSWORD="nullexit"
273     ADGUARD_PWD_ESC="nullexit"
274     ok "Default password set to 'nullexit'."
275 fi
276
277 # 7. Write .env
278 step "Writing .env"
279
280 if [[ -f ".env" ]]; then
281     read -rp "    .env already exists. Overwrite? [y/N]: " OVERWRITE
282     if [[ "$OVERWRITE" != "y" && "$OVERWRITE" != "Y" ]]; then
283         warn "Keeping existing .env."
284     else
285         WRITE_ENV=1
286     fi
287 else
288     WRITE_ENV=1
289 fi
290
291 if [[ "${WRITE_ENV:-0}" == "1" ]]; then
292     # Preserve existing configuration flags if .env already exists
293     EXISTING_PROFILE="medium"
294     EXISTING_MSS="1120"
295     EXISTING_HIJACK="true"
296     EXISTING_EXIT="true"
297     EXISTING_THRESH="6"
298     EXISTING_KILL="true"
299     EXISTING_BLOCKED="kp il"
300     EXISTING_HOST_MTU="1200"
301     EXISTING_SEED=""
302
303     if [[ -f ".env" ]]; then
304         [[ -n "$(read_env_var GATEWAY_RULE_PROFILE)" ]] && EXISTING_PROFILE=$(read_env_var GATEWAY_RULE_PROFILE)
305     )
306     [[ -n "$(read_env_var GATEWAY_MSS)" ]] && EXISTING_MSS=$(read_env_var GATEWAY_MSS)
307     [[ -n "$(read_env_var GATEWAY_HIJACK_HOST)" ]] && EXISTING_HIJACK=$(read_env_var GATEWAY_HIJACK_HOST)

```

```

308 [[ -n "$(read_env_var GATEWAY_USE_EXIT_NODE)" ]] && EXISTING_EXIT=$(read_env_var GATEWAY_USE_EXIT_NODE)
309 [[ -n "$(read_env_var WARP_FAIL_THRESHOLD)" ]] && EXISTING_THRESH=$(read_env_var WARP_FAIL_THRESHOLD)
310 [[ -n "$(read_env_var KILL_SWITCH)" ]] && EXISTING_KILL=$(read_env_var KILL_SWITCH)
311 [[ -n "$(read_env_var BLOCKED_COUNTRIES)" ]] && EXISTING_BLOCKED=$(read_env_var BLOCKED_COUNTRIES)
312 [[ -n "$(read_env_var HOST_MTU)" ]] && EXISTING_HOST_MTU=$(read_env_var HOST_MTU)
313 [[ -n "$(read_env_var NULLEXIT_SEED)" ]] && EXISTING_SEED=$(read_env_var NULLEXIT_SEED)
314 fi
315
316 # README/devref both document NULLEXIT_SEED as "generated by setup.sh" actually
317 # generate it (rather than leaving the .env.example placeholder for the user to
318 # fill by hand), since crypto.sh --verify hard-fails toggle.sh's very first run
319 # without one, and there's nothing else in the setup flow that would create it.
320 if [[ -z "$EXISTING_SEED" ]]; then
321     EXISTING_SEED=$(openssl rand -hex 32)
322     ok "Generated NULLEXIT_SEED for script-integrity signing."
323 fi
324
325 cat > .env <<EOF
326 WIREGUARD_PRIVATE_KEY=${PRIVATE_KEY}
327 WIREGUARD_PUBLIC_KEY=${PUBLIC_KEY}
328 WIREGUARD_ADDRESSES=${ADDRESSES}
329 TS_AUTHKEY=${TS_AUTHKEY}
330 NULLEXIT_SEED=${EXISTING_SEED}
331 GATEWAY_RULE_PROFILE=${EXISTING_PROFILE}
332 # Safe MSS for double-tunneled traffic (Tailscale + WARP). Don't raise this
333 # devref.md 15.3.2 found 1180 (and even 1160) still too large and causes the
334 # exact mysterious mid-transfer stalls this value exists to prevent.
335 GATEWAY_MSS=${EXISTING_MSS}
336 # Lower the host Tailscale MTU to 1200 to prevent fragmentation when passing through the 1280-byte WARP tunnel.
337 HOST_MTU=${EXISTING_HOST_MTU}
338 # Set to false to run as a 'Headless Server' (Docker acts as an exit node, but the host Mac/Linux's own
339 # internet is NOT hijacked).
340 GATEWAY_HIJACK_HOST=${EXISTING_HIJACK}
341 GATEWAY_USE_EXIT_NODE=${EXISTING_EXIT}
342 WARP_FAIL_THRESHOLD=${EXISTING_THRESH}
343 # PF fail-closed kill-switch. Leave true this is the core leak guarantee.
344 KILL_SWITCH=${EXISTING_KILL}
345 ADGUARD_PASSWORD=${ADGUARD_PASSWORD}
346 BLOCKED_COUNTRIES="${EXISTING_BLOCKED}"
347 EOF
348 ok ".env written."
349
350 # Sign the core scripts against the (possibly just-generated) seed now, so the
351 # very first toggle.sh run doesn't hard-fail crypto.sh --verify with no seed/no
352 # .signatures and no indication that a manual `crypto.sh --sign` was needed.
353 if bash scripts/crypto.sh --sign >> output.log 2>&1; then
354     ok "Core scripts signed for integrity verification."
355 else
356     warn "Could not sign core scripts run 'bash scripts/crypto.sh --sign' manually before toggle.sh."
357 fi
358 fi
359
360 # 8. Prepare directories
361 step "Preparing AdGuard directories"
362 mkdir -p adguard/conf adguard/work/userfilters
363
364 # Remove empty AdGuardHome.yaml that would cause a crash loop (same logic as Toggle-Gateway scripts)
365 if [[ -f "adguard/conf/AdGuardHome.yaml" ]] && ! -s "adguard/conf/AdGuardHome.yaml" ]; then
366     rm "adguard/conf/AdGuardHome.yaml"
367     warn "Removed empty AdGuardHome.yaml to prevent crash loop."
368 fi
369
370 ok "Directories ready."
371
372 # 9. Compile DNS filter rules
373 step "Compiling DNS filter rules (this may take a minute)"
374
375 # 10. Start containers
376 step "Starting containers"
377 # Note: tailscale depends on adguardhome being healthy (port 5335 up),
378 # so Docker Compose will hold it back automatically until the wizard is done.
379 docker compose up -d
380 ok "Containers starting."
381
382 # 11. Wait for AdGuard Home setup wizard
383 step "Waiting for AdGuard Home setup wizard"
384 echo " (up to 60 seconds...)"
385
386 MAX=60; ELAPSED=0

```

```

388 until docker exec routing-fix curl -sf http://127.0.0.1:3000 >> output.log 2>&1; do
389     sleep 2; ELAPSED=$((ELAPSED + 2))
390     [[ $ELAPSED -ge $MAX ]] && die "AdGuard Home didn't come up.\nDebug with: docker compose logs adguardhome"
391 done
392
393 ok "AdGuard Home is up."
394
395 # 12. Configure AdGuard Home via API
396 step "Configuring AdGuard Home"
397
398 # Run all API calls in a single docker exec sh -c so the session cookie file
399 # is shared across calls without leaving the container.
400 docker exec routing-fix sh -c "
401     set -e
402
403     # Step 1: Complete the setup wizard (sets admin creds + DNS port to 5335)
404     curl -sf -X POST http://127.0.0.1:3000/control/install/configure \
405         -H 'Content-Type: application/json' \
406         -d '{"web":{"ip":"0.0.0.0"},"port":3000},"dns":{"ip":"0.0.0.0"},"port":5335},"username":"","password":"","password":"'${ADGUARD_PWD_ESC}'" \
407         >/dev/null || true
408
409     # Step 2: Wait for AdGuard to restart after wizard
410     sleep 4
411     WAITED=0
412     until curl -sf http://127.0.0.1:3000 >/dev/null 2>&1; do
413         sleep 2; WAITED=$((WAITED + 2))
414         [ \ $WAITED -ge 30 ] && { echo 'AdGuard did not restart in time'; exit 1; }
415     done
416
417     # Step 3: Login to get session cookie
418     curl -sf -X POST http://127.0.0.1:3000/control/login \
419         -H 'Content-Type: application/json' \
420         -d '{"name":"","admin":"","password":"'${ADGUARD_PWD_ESC}'" \
421         -c /tmp/agh.cookie >/dev/null
422
423     # Step 4: Point upstream DNS back through the WARP tunnel
424     curl -sf -X POST http://127.0.0.1:3000/control/dns_config \
425         -H 'Content-Type: application/json' \
426         -b /tmp/agh.cookie \
427         -d '{"upstream_dns":["127.0.0.1:53","[/onion/]127.0.0.1:5353"],"bootstrap_dns":["1.1.1.1","8.8.8.8"],"upstream_mode":"load_balance"}' \
428         >/dev/null
429
430     # Step 5: Register the compiled rules file as a filter list
431     curl -sf -X POST http://127.0.0.1:3000/control/filtering/add_url \
432         -H 'Content-Type: application/json' \
433         -b /tmp/agh.cookie \
434         -d '{"name":"","Custom Compiled Rules","url":"","/opt/adguardhome/work/userfilters/compiled_rules.txt"}' \
435         >/dev/null || true
436
437     # Step 6: Force a filter refresh to load the rules immediately
438     curl -sf -X POST http://127.0.0.1:3000/control/filtering/refresh \
439         -H 'Content-Type: application/json' \
440         -b /tmp/agh.cookie \
441         -d '{"whitelist":false}' \
442         >/dev/null || true
443
444     rm -f /tmp/agh.cookie
445 "
446
447 ok "AdGuard Home configured."
448
449 # 13. Wait for Tailscale
450 step "Waiting for Tailscale to authenticate"
451 echo " (Tailscale only starts once AdGuard's healthcheck passes up to 90s)"
452
453 MAX=90; ELAPSED=0; TS_IP=""
454 until [[ -n "$TS_IP" ]]; do
455     TS_IP=$(docker exec tailscale tailscale ip -4 2>> output.log || true)
456     if [[ -z "$TS_IP" ]]; then
457         sleep 3; ELAPSED=$((ELAPSED + 3))
458         if [[ $ELAPSED -ge $MAX ]]; then
459             warn "Tailscale hasn't authenticated yet."
460             warn "Once it does, re-run the toggle script it will resolve the IP dynamically."
461             break
462         fi
463     fi
464 done
465

```

```

466 if [[ -n "$TS_IP" ]]; then
467     ok "Tailscale IP: ${TS_IP} (resolved dynamically by the toggle script on each startup)"
468 fi

```

53 scripts/sweep.sh

```

1  #!/bin/bash
2  # scripts/sweep.sh  nullexit gateway health sweep
3  #
4  # Automates the 7-check post-restart health sweep (sweep.md 1 + tailnet ping
5  # + throughput). It answers one question: "after a toggle/restart/wake, is
6  # the gateway actually healthy end-to-end no leak, direct P2P, DNS
7  # filtering live, kill-switch armed, and fast enough to actually use?" It is
8  # READ-ONLY with respect to gateway state: it never mutates routing, PF, DNS,
9  # or containers (the throughput check only pulls a test file over curl the
10 # same kind of traffic any app already sends, nothing is exec'd into or
11 # installed in the container), so it is safe to run against a live gateway at
12 # any time (including from a remote session see the caveat in agent.md L16).
13 #
14 # The 7 checks (1-5 mirror sweep.md 1):
15 # 1. Containers      warp, adguardhome, tailscale, tor, routing-fix up/healthy
16 # 2. Double-tunnel/leak  container WARP exit IP == host exit IP, both warp-on
17 # 3. Hostcontainer path  Tailscale exit-node is a DIRECT P2P conn, not DERP relay
18 # 4. AdGuard filtering  compiled rule counts present + live DNS interception
19 # 5. PF kill-switch     pfctl Enabled + anchor com.apple/nullexit carries rules
20 # 6. Tailnet reachability every ONLINE Tailscale peer answers a tailscale ping
21 # (DERP-relayed peers are flagged as throughput-degraded, not a failure)
22 # 7. Throughput        host path vs. WARP-only baseline (via the container's
23 # SOCKS5 proxy, no exec/install inside the container). Catches "slow AND
24 # dropping mid-transfer" that a one-shot ping/latency check can't see.
25 # Real per-device (phone) throughput can't be measured without an agent
26 # running on the device, so mesh peers get a path/latency flag instead
27 # (see check 6).
28 #
29 # A timestamped report is written to the project root (one file per run, so
30 # runs can be diffed). stdout carries a coloured summary + a final PASS/FAIL
31 # verdict; exit code is 0 when every check passes, 1 otherwise (so it can gate
32 # CI / launchd / a post-restart hook).
33 #
34 # Usage:
35 #   bash scripts/sweep.sh           # run the sweep, write report, print verdict
36 #   bash scripts/sweep.sh --quiet   # only the final verdict line + non-zero on FAIL
37 #   bash scripts/sweep.sh --help    # this message
38 #
39 set -uo pipefail
40 # NOTE: errexit is intentionally OFF. Several probes are EXPECTED to fail on an
41 # unhealthy gateway and that failure IS the diagnostic signal we record each
42 # exit status rather than letting one abort the whole sweep.
43
44 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
45 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
46 TIMESTAMP="$(date -u +%Y%m%dT%H%M%S)"
47 cd "$PROJECT_ROOT" || { echo "[FATAL] cannot cd to $PROJECT_ROOT" >&2; exit 1; }
48 source "$SCRIPT_DIR/common.sh"
49 LOG_FILE="$PROJECT_ROOT/output.log"
50 REPORT_FILE="$PROJECT_ROOT/sweep-$TIMESTAMP.txt"
51
52 # Argument parsing
53 QUIET=false
54 case "${1:-}" in
55     --quiet|-q) QUIET=true ;;
56     --help|-h) sed -n '2,27p' "$0"; exit 0 ;;
57     *)
58         : ;;
59     *)
60         echo "Unknown argument: $1" >&2
61         echo "Run with --help for usage." >&2
62         exit 2 ;;
63 esac
64
65 # Colours (auto-disabled when stdout is not a tty, e.g. piped)
66 if [ -t 1 ]; then
67     RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
68     BOLD='\033[1m'; NC='\033[0m'
69 else
70     RED=''; GREEN=''; YELLOW=''; BOLD=''; NC=''
71 fi

```

```

71 # Homebrew on PATH (same as toggle.sh / recover.sh / diagnose-host-leak)
72 export PATH="/opt/homebrew/bin:/usr/local/bin:$PATH"
73
74 # Output helpers write to BOTH stdout and the report file
75 exec 3>> "$REPORT_FILE" || {
76     printf '\n[FATAL] cannot open %s for write\n' "$REPORT_FILE" >&2
77     exit 1
78 }
79 emit() { [ "$QUIET" = true ] || printf '%b\n' "$1"; printf '%b\n' "$1" | sed 's/\x1b\[[0-9;]*m//g' >&3; }
80 section() { emit ""; emit "${BOLD} $1 ${NC}"; }
81 line() { emit " $1"; }
82 ok() { line "${GREEN}${NC} $1"; }
83 warn() { line "${YELLOW}${NC} $1"; }
84 fail() { line "${RED}${NC} $1"; }
85
86 # common.sh redefines run_with_timeout; it is a pure-bash macOS-safe timeout
87 # (GNU `timeout` is absent on stock macOS). We reuse the sourced version.
88
89 # Result accounting
90 # Each check appends "PASS"/"WARN"/"FAIL" so the verdict is a pure roll-up.
91 declare -a RESULTS=()
92 record() { RESULTS+=("$1|$2"); } # status|label
93
94 # Header
95 if [ "$QUIET" != true ]; then
96     emit ""
97     emit ""
98     emit " n u l l e x i t   h e a l t h   s w e e p "
99     emit ""
100    emit " Timestamp (UTC): $TIMESTAMP"
101    emit " Project root:    $PROJECT_ROOT"
102    emit " Report file:     $REPORT_FILE"
103    emit ""
104 fi
105
106 GATEWAY_TS_IP="$(read_adguard_ip)" # gateway Tailscale IP from .gateway_ip
107
108 #
109 section "1/7 Containers"
110 #
111 # All five services must be running; warp + adguardhome additionally expose a
112 # healthcheck and must be 'healthy'. `docker compose ps` is the source of truth.
113 EXPECTED_SERVICES="warp adguardhome tailscale tor routing-fix"
114 if ! command -v docker >/dev/null 2>&1; then
115     fail "docker CLI not on PATH cannot inspect containers"
116     record FAIL "containers"
117 else
118     PS_OUT=$(run_with_timeout 20 docker compose ps 2>>"$LOG_FILE")
119     missing=""; unhealthy=""
120     for svc in $EXPECTED_SERVICES; do
121         row=$(printf '%s\n' "$PS_OUT" | awk -v s="$svc" ' $1==s || $0 ~ ("[:space:]"s"[:space:]")' )
122         if [ -z "$row" ] || ! printf '%s' "$row" | grep -qiE 'up|running'; then
123             missing="$missing $svc"
124             elif printf '%s' "$row" | grep -qi 'unhealthy'; then
125                 unhealthy="$unhealthy $svc"
126         fi
127     done
128     if [ -z "$missing" ] && [ -z "$unhealthy" ]; then
129         ok "all 5 services up (warp/adguardhome healthy, tailscale/tor/routing-fix running)"
130         record PASS "containers"
131     else
132         [ -n "$missing" ] && fail "not up/running:$missing"
133         [ -n "$unhealthy" ] && fail "reporting unhealthy:$unhealthy"
134         record FAIL "containers"
135     fi
136     line " docker compose ps "
137     printf '%s\n' "$PS_OUT" | awk '{print "    "$0}' >&3
138     [ "$QUIET" = true ] || printf '%s\n' "$PS_OUT" | awk '{print "    "$0}'
139 fi
140
141 #
142 section "2/7 Double-tunnel / IP-leak"
143 #
144 # The container's WARP egress and the host's egress must be the SAME public IP
145 # with warp=on for both. Equal IPs all host traffic is exiting through the
146 # same WARP tunnel the container uses (no split / raw-ISP leak).
147 # The warp container (gluetun) ships wget, not curl fetch the trace from
148 # inside its netns with whichever client is present.
149 CTR_TRACE=$(run_with_timeout 12 docker compose exec -T warp sh -c \
150     'curl -s --max-time 8 https://www.cloudflare.com/cdn-cgi/trace 2>/dev/null \
151     || wget -qO- --timeout=8 https://www.cloudflare.com/cdn-cgi/trace 2>/dev/null' \

```

```

152     2>>>"$LOG_FILE")
153 HOST_TRACE=$(run_with_timeout 12 curl -s --max-time 8 \
154     https://www.cloudflare.com/cdn-cgi/trace 2>>>"$LOG_FILE")
155 CTR_IP=$(printf '%s' "$CTR_TRACE" | awk -F= '/^ip={print $2;exit}')
156 CTR_WARP=$(printf '%s' "$CTR_TRACE" | awk -F= '/^warp={print $2;exit}')
157 HOST_IP=$(printf '%s' "$HOST_TRACE" | awk -F= '/^ip={print $2;exit}')
158 HOST_WARP=$(printf '%s' "$HOST_TRACE" | awk -F= '/^warp={print $2;exit}')
159 line "container exit: ip=${CTR_IP:-?} warp=${CTR_WARP:-?}"
160 line "host exit: ip=${HOST_IP:-?} warp=${HOST_WARP:-?}"
161 if [ -z "$CTR_IP" ] || [ -z "$HOST_IP" ]; then
162     fail "could not obtain both egress IPs (container or host trace failed)"
163     record FAIL "leak"
164 elif [ "$CTR_IP" = "$HOST_IP" ] && [ "$HOST_WARP" = "on" ]; then
165     ok "no leak host egress == container egress ($HOST_IP), warp=on"
166     record PASS "leak"
167 elif [ "$HOST_WARP" != "on" ]; then
168     fail "host warp=$HOST_WARP host traffic is NOT going through WARP (LEAK)"
169     record FAIL "leak"
170 else
171     fail "egress mismatch: host $HOST_IP != container $CTR_IP split/leak"
172     record FAIL "leak"
173 fi
174
175 #
176 section "3/7 Host container path (direct P2P, not DERP relay)"
177 #
178 # `tailscale status` annotates each peer with 'direct <ip:port>' or
179 # 'relay <derp>'. The gateway peer should be DIRECT a DERP relay hop adds
180 # latency and signals NAT traversal failure between host and gateway.
181 if ! command -v tailscale >/dev/null 2>&1; then
182     fail "tailscale CLI not on PATH"
183     record FAIL "p2p"
184 else
185     TS_OUT=$(run_with_timeout 6 tailscale status 2>>>"$LOG_FILE")
186     PEER_LINE=$(printf '%s\n' "$TS_OUT" | grep -E "(^|[:space:])$GATEWAY_TS_IP([[:space:]]|$)" | head -1)
187     if [ -z "$PEER_LINE" ]; then
188         warn "gateway $GATEWAY_TS_IP not found in host peer list"
189         line "$(printf '%s\n' "$TS_OUT" | head -5 | awk '{print "    "$0}')"
190         record WARN "p2p"
191     elif printf '%s' "$PEER_LINE" | grep -q 'direct '; then
192         conn=$(printf '%s' "$PEER_LINE" | grep -oE 'direct [0-9.]+:[0-9.]+')
193         ok "direct P2P to gateway $conn (no DERP relay)"
194         record PASS "p2p"
195     elif printf '%s' "$PEER_LINE" | grep -q 'relay '; then
196         relay=$(printf '%s' "$PEER_LINE" | grep -oE 'relay "[~"]+')
197         warn "gateway reached via DERP $relay (not direct) NAT traversal degraded"
198         record WARN "p2p"
199     else
200         warn "gateway peer state indeterminate (idle/no active conn)"
201         line "$PEER_LINE"
202         record WARN "p2p"
203     fi
204 fi
205
206 #
207 section "4/7 AdGuard filtering"
208 #
209 # Two signals: (a) the compiled ruleset exists with a non-trivial block count,
210 # (b) the gateway resolver actually answers a query (live interception).
211 COMPILED="adguard/work/userfilters/compiled_rules.txt"
212 if [ -f "$COMPILED" ]; then
213     TOTAL=$(grep -m1 '! Total Block Rules:' "$COMPILED" | awk -F': ' '{print $2}' | tr -d '[:space:]')
214     NATIVE=$(grep -m1 '! Native AdGuard Rules:' "$COMPILED" | awk -F': ' '{print $2}' | tr -d '[:space:]')
215     if [ -n "$TOTAL" ] && [ "$TOTAL" -gt 0 ] 2>/dev/null; then
216         ok "compiled ruleset present Total Block Rules: $TOTAL / Native AdGuard: ${NATIVE:-?}"
217         RULES_OK=true
218     else
219         warn "compiled_rules.txt present but block-rule count unreadable"
220         RULES_OK=false
221     fi
222 else
223     warn "compiled_rules.txt missing (rule-compile may not have run)"
224     RULES_OK=false
225 fi
226 # Live interception: the gateway resolver must answer (dig via .gateway_ip).
227 DNS_OK=false
228 if command -v dig >/dev/null 2>&1 && [ -n "$GATEWAY_TS_IP" ]; then
229     DIG_OUT=$(run_with_timeout 8 dig +time=3 +tries=1 @"$GATEWAY_TS_IP" google.com A 2>>>"$LOG_FILE")
230     if printf '%s' "$DIG_OUT" | grep -qE 'ANSWER SECTION|status: NOERROR'; then
231         ok "live DNS interception gateway $GATEWAY_TS_IP resolved google.com"
232         DNS_OK=true

```

```

233     else
234         warn "gateway $GATEWAY_TS_IP did not answer a test query"
235     fi
236 else
237     warn "skipped live-resolution probe (dig missing or gateway IP unknown)"
238 fi
239 if [ "$RULES_OK" = true ] && [ "$DNS_OK" = true ]; then
240     record PASS "adguard"
241 elif [ "$RULES_OK" = true ] || [ "$DNS_OK" = true ]; then
242     record WARN "adguard"
243 else
244     record FAIL "adguard"
245 fi
246
247 #
248 section "5/7 PF kill-switch"
249 #
250 # PF must be Enabled AND the com.apple/nullexit anchor must carry rules.
251 # An empty/absent anchor means the fail-closed lane is not actually armed.
252 PF_STATUS=$(run_with_timeout 6 sudo -n pfctl -s info 2>>"$LOG_FILE" | awk '/^Status:/{print $2;exit}')
253 ANCHOR_RULES=$(run_with_timeout 6 sudo -n pfctl -a com.apple/nullexit -sr 2>>"$LOG_FILE")
254 if [ "$PF_STATUS" = "Enabled" ] && [ -n "$ANCHOR_RULES" ]; then
255     RULE_N=$(printf '%s\n' "$ANCHOR_RULES" | grep -cvE '\s*$')
256     ok "PF Enabled + anchor com.apple/nullexit armed ($RULE_N rules)"
257     line "$(printf '%s\n' "$ANCHOR_RULES" | head -4 | awk '{print "    "$0}')"
258     record PASS "pf"
259 elif [ "$PF_STATUS" = "Enabled" ]; then
260     fail "PF Enabled but anchor com.apple/nullexit has NO rules kill-switch not armed"
261     record FAIL "pf"
262 elif [ -z "$PF_STATUS" ]; then
263     warn "could not read PF status (needs passwordless sudo for 'pfctl -s info')"
264     record WARN "pf"
265 else
266     fail "PF Status: $PF_STATUS kill-switch disabled"
267     record FAIL "pf"
268 fi
269
270 #
271 section "6/7 Tailnet reachability (burst ping loss % + jitter, all online devices)"
272 #
273 # Every peer the coordination server reports as ONLINE should answer a
274 # `tailscale ping` (TSMP probes the WireGuard data path; ICMP is avoided since
275 # phones/laptops often drop it). Offline peers and this host are skipped.
276 # An online-but-unreachable peer means control plane and data path disagree.
277 #
278 # A single ping only proves "reachable right now" it can't see loss or
279 # jitter, which is what actually determines whether a call/video/QUIC session
280 # on that device holds up. There's no way to get a real bits-per-second number
281 # for a phone without something running on it to receive/echo data (neither
282 # phone has that), but TSMP ping is answered by the Tailscale client itself
283 # with zero cooperation needed from the device the one channel that works
284 # identically over DERP or direct P2P. So: send a BURST instead of one ping,
285 # and report loss % + RTT spread per peer, the closest honest signal to
286 # "connection quality" available without an app/listener on the phone.
287 PING_BURST="{SWEEP_PING_BURST:-8}"
288 if ! command -v tailscale >/dev/null 2>&1; then
289     fail "tailscale CLI not on PATH"
290     record FAIL "tailnet"
291 else
292     SELF_TS_IP=$(run_with_timeout 6 tailscale ip -4 2>>"$LOG_FILE")
293     [ -n "${TS_OUT:-}" ] || TS_OUT=$(run_with_timeout 6 tailscale status 2>>"$LOG_FILE")
294     # Online peers = rows starting with a Tailscale IP, minus self and 'offline'.
295     # (A '-' status column means online-but-idle, so it is kept.)
296     PEERS=$(printf '%s\n' "$TS_OUT" | awk -v self="$SELF_TS_IP" \
297         '$1 ~ /^100\. / && $1 != self && $0 !~ /^(|[:space:]]offline(,|[:space:]]|$)/ {print $1, $2}')
298     if [ -z "$PEERS" ]; then
299         warn "no online peers to ping (everything else in the tailnet is offline)"
300         record WARN "tailnet"
301     else
302         unreached=0; lossy=0
303         while read -r peer_ip peer_name; do
304             PONGS=$(run_with_timeout $((PING_BURST + 8)) tailscale ping --timeout=3s --c "$PING_BURST" --until-direct
305 =false "$peer_ip" 2>>"$LOG_FILE")
306             n_ok=$(printf '%s\n' "$PONGS" | grep -c 'pong')
307             if [ "$n_ok" -eq 0 ]; then
308                 fail "$peer_name ($peer_ip) no pong in $PING_BURST attempts (online per control plane, data path dead)"
309             else
310                 unreached=$((unreached+1))
311                 continue
312             fi

```



```

311 path=$(printf '%s\n' "$PONGS" | grep -m1 '^pong' | sed -E 's/^pong from [^ ]+ \([^\)]+\) via ([^ ]+).*/\1/'
312 loss_pct=$(( (PING_BURST - n_ok) * 100 / PING_BURST ))
313 read -r avg_ms min_ms max_ms <<< "$(printf '%s\n' "$PONGS" | grep '^pong' | grep -oE '[0-9]+ms' | tr -d '
ms' | awk '
314 { if (NR==1 || $1<min) min=$1; if (NR==1 || $1>max) max=$1; sum+=$1; n++ }
315 END { if (n>0) printf "%.0f %d %d", sum/n, min, max }')"
316 jitter_ms=$((max_ms - min_ms))
317 relay_note=""
318 case "$path" in
319 DERP*) relay_note=" relayed, expect degraded throughput (see check 7)" ;;
320 esac
321 if [ "$loss_pct" -eq 0 ]; then
322 ok "$peer_name ($peer_ip) $n_ok/$PING_BURST pong (0% loss), RTT ${min_ms}-${max_ms}ms avg ${avg_ms}ms
(jitter ${jitter_ms}ms) via ${path}${relay_note}"
323 elif [ "$loss_pct" -lt 40 ]; then
324 warn "$peer_name ($peer_ip) $n_ok/$PING_BURST pong (${loss_pct}% loss), RTT ${min_ms}-${max_ms}ms avg
${avg_ms}ms via ${path}${relay_note}"
325 lossy=$((lossy+1))
326 else
327 fail "$peer_name ($peer_ip) $n_ok/$PING_BURST pong (${loss_pct}% loss, unstable) via ${path}${
relay_note}"
328 unreachable=$((unreachable+1))
329 fi
330 done <<< "$PEERS"
331 n_peers=$(printf '%s\n' "$PEERS" | wc -l | tr -d '[:space:]')
332 if [ "$unreached" -eq 0 ] && [ "$lossy" -eq 0 ]; then
333 record PASS "tailnet"
334 elif [ "$unreached" -lt "$n_peers" ]; then
335 record WARN "tailnet"
336 else
337 record FAIL "tailnet"
338 fi
339 fi
340 fi
341
342 #
343 section "7/7 Throughput (host path vs. WARP-only baseline)"
344 #
345 # A single ping only proves the path is UP, not that it stays up under real
346 # load devref.md 15.3.1 documents sustained-transfer bufferbloat/reset
347 # behavior a quick latency check can't see. Pulls a real test file twice:
348 # once over the host's normal route (full double-tunnel: Tailscale exit-node
349 # wrap + WARP) and once straight through the warp container's own SOCKS5
350 # proxy (WARP only, no Tailscale wrap) port read from the live
351 # /tmp/nullexit-ports.env, so this never execs into or installs anything in
352 # the container (sweep stays read-only w.r.t. gateway state). Comparing the
353 # two shows whether slowness/resets come from WARP itself or from the extra
354 # Tailscale-wrap hop. Skip with SWEEP_SKIP_THROUGHPUT=true in .env.
355 #
356 # Target: a fast, high-bandwidth CDN, not a slow/distant one. speedtest.net-style
357 # test hosts (e.g. speedtest.tele2.net) were tried first and turned out to be the
358 # bottleneck themselves even RAW, gateway-off ISP speed to tele2.net was ~3.2
359 # Mbps, vs ~32 Mbps raw to a fast CDN so a slow test target would have silently
360 # blamed nullexit for a benchmark artifact. Cloudflare's own speed-test endpoint
361 # 403s WARP egress IPs specifically (abuse-prevention on shared/CGNAT IPs), so
362 # Hetzner (also fast, does not flag WARP IPs) is used instead.
363 THROUGHPUT_SKIP=$(read_env_var "SWEEP_SKIP_THROUGHPUT" | tr '[:upper:]' '[:lower:]')
364 if [ "$THROUGHPUT_SKIP" = "true" ]; then
365 warn "skipped (SWEEP_SKIP_THROUGHPUT=true)"
366 record WARN "throughput"
367 else
368 TP_URL="${SWEEP_THROUGHPUT_URL:-https://ash-speed.hetzner.com/100MB.bin}"
369 TP_TIMEOUT="${SWEEP_THROUGHPUT_TIMEOUT:-30}"
370
371 measure() { # $1 = extra curl args (word-split; may be empty)
372 # curl's own --max-time must win the race against run_with_timeout's outer
373 # SIGTERM watcher a process killed by signal never reaches its own -w
374 # output, so if both timers were equal, the watcher could kill curl right
375 # before it prints stats, and every leg would misreport as "no response".
376 # The outer timeout is a dead-man's-switch (+5s), not the real budget.
377 run_with_timeout $((TP_TIMEOUT + 5)) curl -sS --max-time "$TP_TIMEOUT" $1 -o /dev/null \
378 -w 'code=%{http_code} size=%{size_download} time=%{time_total} bps=%{speed_download}' \
379 "$TP_URL" 2>>"$LOG_FILE"
380 }
381 leg_verdict() { # $1=curl_rc $2=http_code -> ok|stall|blocked|unknown
382 [ -z "$2" ] && { echo unknown; return; }
383 [ "$2" != "200" ] && { echo blocked; return; }
384 case "$1" in
385 0|28) echo ok ;; # 0=completed; 28=hit our own --max-time while still healthy
386 56) echo stall ;; # curl's actual "Recv failure: Connection reset by peer"

```

```

387     *)      echo unknown ;;
388   esac
389 }
390 to_mbps() { awk -v b="${1:-}" 'BEGIN{ if (b=="") print "?"; else printf "%.1f", (b*8)/1000000 }'; }
391
392 HOST_TP=$(measure ""); HOST_RC=?
393 HOST_CODE=$(printf '%s' "$HOST_TP" | grep -oE 'code=[0-9]+' | cut -d= -f2)
394 HOST_SIZE=$(printf '%s' "$HOST_TP" | grep -oE 'size=[0-9]+' | cut -d= -f2)
395 HOST_TIME=$(printf '%s' "$HOST_TP" | grep -oE 'time=[0-9.]+' | cut -d= -f2)
396 HOST_BPS=$(printf '%s' "$HOST_TP" | grep -oE 'bps=[0-9.]+' | cut -d= -f2)
397 HOST_VERDICT=$(leg_verdict "$HOST_RC" "$HOST_CODE")
398 HOST_MBPS=$(to_mbps "$HOST_BPS")
399 case "$HOST_VERDICT" in
400   ok)      ok      "host path: ${HOST_MBPS} Mbps (${HOST_SIZE:-0} bytes in ${HOST_TIME:-?}s)" ;;
401   stall)   warn    "host path: connection reset mid-transfer after ${HOST_SIZE:-0} bytes sustained-load drop,
402     see devref.md 15.3.1" ;;
403   blocked) warn    "host path: test endpoint returned HTTP ${HOST_CODE:-?} (egress IP likely flagged)
404     inconclusive, not a gateway fault" ;;
405   unknown) warn    "host path: no response (timeout after ${TP_TIMEOUT}s)" ;;
406 esac
407
408 SOCKS_PROXY_PORT=""
409 [ -f /tmp/nullexit-ports.env ] && SOCKS_PROXY_PORT=$(grep -oE 'SOCKS_PROXY_PORT=[0-9]+' /tmp/nullexit-ports.
410   env | cut -d= -f2)
411 if [ -z "$SOCKS_PROXY_PORT" ]; then
412   warn "WARP-only baseline: SOCKS_PROXY_PORT unknown (no /tmp/nullexit-ports.env gateway not started via
413     toggle.sh?)"
414   CTR_VERDICT=unknown
415 else
416   CTR_TP=$(measure "--socks5-hostname 127.0.0.1:$SOCKS_PROXY_PORT"); CTR_RC=?
417   CTR_CODE=$(printf '%s' "$CTR_TP" | grep -oE 'code=[0-9]+' | cut -d= -f2)
418   CTR_SIZE=$(printf '%s' "$CTR_TP" | grep -oE 'size=[0-9]+' | cut -d= -f2)
419   CTR_TIME=$(printf '%s' "$CTR_TP" | grep -oE 'time=[0-9.]+' | cut -d= -f2)
420   CTR_BPS=$(printf '%s' "$CTR_TP" | grep -oE 'bps=[0-9.]+' | cut -d= -f2)
421   CTR_VERDICT=$(leg_verdict "$CTR_RC" "$CTR_CODE")
422   CTR_MBPS=$(to_mbps "$CTR_BPS")
423   case "$CTR_VERDICT" in
424     ok)      ok      "WARP-only baseline: ${CTR_MBPS} Mbps (${CTR_SIZE:-0} bytes in ${CTR_TIME:-?}s)" ;;
425     stall)   warn    "WARP-only baseline: connection reset mid-transfer after ${CTR_SIZE:-0} bytes bug is in
426       WARP itself, not the Tailscale wrap" ;;
427     blocked) warn    "WARP-only baseline: test endpoint returned HTTP ${CTR_CODE:-?} inconclusive" ;;
428     unknown) warn    "WARP-only baseline: no response (timeout after ${TP_TIMEOUT}s)" ;;
429   esac
430 fi
431
432 if [ "$HOST_VERDICT" = "ok" ] && [ "$CTR_VERDICT" = "ok" ]; then
433   if [ "$HOST_MBPS" != "?" ] && [ "$CTR_MBPS" != "?" ]; then
434     DELTA=$(awk -v h="$HOST_MBPS" -v c="$CTR_MBPS" 'BEGIN{ if (c<=0) print 0; else printf "%.0f", 100*(c-h)/c
435       }')
436     [ "$DELTA" -ge 30 ] 2>/dev/null && warn "host is ${DELTA}% slower than the WARP-only baseline the
437       Tailscale exit-node wrap is the added cost here, not WARP"
438   fi
439   record PASS "throughput"
440   elif [ "$HOST_VERDICT" = "stall" ] || [ "$CTR_VERDICT" = "stall" ]; then
441     record WARN "throughput"
442   elif [ "$HOST_VERDICT" = "unknown" ] && [ "$CTR_VERDICT" = "unknown" ]; then
443     record FAIL "throughput"
444   else
445     record WARN "throughput"
446 fi
447
448 #
449 # Cover traffic (opt-in) informational only, NOT a scored check.
450 # Shown solely when NOISE_ENABLED=true so the sweep stays read-only and the
451 # 6-check verdict is unchanged. Reports whether the padding daemon is alive.
452 NOISE_ON=$(read_env_var "NOISE_ENABLED" | tr '[:upper:]' '[:lower:]')
453 if [ "$NOISE_ON" = "true" ]; then
454   section "Cover traffic (opt-in, informational)"
455   NOISE_PID_FILE="/tmp/nullexit-noise.pid"
456   npid=$(cat "$NOISE_PID_FILE" 2>/dev/null)
457   if [ -n "$npid" ] && kill -0 "$npid" 2>/dev/null; then
458     ok "dummy-padding daemon running (pid $npid) see 'bash scripts/noise.sh status'"
459   else
460     warn "NOISE_ENABLED=true but no padding daemon running start it: bash scripts/noise.sh start"
461 fi
462
463 #
464 # DNS anomaly scan (informational, read-only, NOT a scored check the 6-check
465 # verdict is unchanged). It only READS the AdGuard querylog; never blocks DNS.

```

```

461 # It FAILS LOUD: a broken detector (crash, corrupt model, empty log, failing
462 # self-test) shows a red with the reason it is NEVER silently reported as
463 # clean, and an untrained detector is announced rather than skipped in silence.
464 QUERYLOG="$PROJECT_ROOT/adguard/work/data/querylog.json"
465 DNS_TOOL="$SCRIPT_DIR/dns_anomaly_detector.py"
466 if command -v python3 >/dev/null 2>&1 && [ -f "$DNS_TOOL" ]; then
467     section "DNS anomaly scan (informational)"
468     # 1) Prove the detection logic itself works before trusting any "clean".
469     ST_ERR=$(python3 "$DNS_TOOL" selftest 2>&1 >/dev/null); ST_RC=$?
470     if [ "$ST_RC" -ne 0 ]; then
471         fail "DNS detector SELF-TEST FAILED (rc=$ST_RC) detector is broken, do NOT trust results: ${ST_ERR:-see
         stderr}"
472     elif [ ! -f "$PROJECT_ROOT/.dns_baseline.json" ]; then
473         warn "DNS detector NOT trained (no .dns_baseline.json) not protecting; run: python3 scripts/
         dns_anomaly_detector.py learn"
474     elif [ ! -f "$QUERYLOG" ]; then
475         warn "DNS detector trained but querylog absent ($QUERYLOG) nothing to scan"
476     else
477         # 2) Real scan. Capture stderr + exit code; do NOT swallow failures.
478         DNS_ERR=$(python3 "$DNS_TOOL" scan --quiet 2>&1 >/tmp/nullexit-dns-scan.out); DNS_RC=$?
479         DNS_OUT=$(cat /tmp/nullexit-dns-scan.out 2>/dev/null); rm -f /tmp/nullexit-dns-scan.out
480         case "$DNS_RC" in
481             0) ok "no DNS-tunneling / DGA signatures $DNS_OUT" ;;
482             1) warn "$DNS_OUT review: python3 scripts/dns_anomaly_detector.py scan" ;;
483             *) fail "DNS scan FAILED to run (rc=$DNS_RC) NOT a clean result: ${DNS_ERR:-unknown error}" ;;
484         esac
485     fi
486 fi
487
488 #
489 section "VERDICT"
490 #
491 pass=0; warnc=0; failc=0
492 for r in "${RESULTS[@]}; do
493     case "${r%[*]}" in PASS) pass=$((pass+1)); WARN) warnc=$((warnc+1)); FAIL) failc=$((failc+1)); esac
494 done
495 for r in "${RESULTS[@]}; do
496     st="${r%[*]}"; lbl="${r##*}"
497     case "$st" in
498         PASS) ok "$lbl" ;;
499         WARN) warn "$lbl" ;;
500         FAIL) fail "$lbl" ;;
501     esac
502 done
503 emit ""
504 if [ "$failc" -eq 0 ] && [ "$warnc" -eq 0 ]; then
505     emit "${GREEN}${BOLD}SWEEP PASS gateway healthy (${pass}/${#RESULTS[@]} checks green){NC}"
506     EXIT=0
507 elif [ "$failc" -eq 0 ]; then
508     emit "${YELLOW}${BOLD}SWEEP PASS with warnings ${pass} pass, ${warnc} warn, 0 fail${NC}"
509     EXIT=0
510 else
511     emit "${RED}${BOLD}SWEEP FAIL ${pass} pass, ${warnc} warn, ${failc} fail${NC}"
512     EXIT=1
513 fi
514 emit ""
515 [ "$QUIET" = true ] || emit "Full report: $REPORT_FILE"
516 exit "$EXIT"

```

54 scripts/taildrop-watch.sh

```

1 #!/bin/bash
2 # scripts/taildrop-watch.sh
3 # launchd entry point for ~/Library/LaunchAgents/com.nullexit.taildrop-watch.plist
4 # (label com.nullexit.taildrop-watch).
5 #
6 # Homebrew's tailscaled CLI (unlike the Tailscale.app GUI) does not auto-drain
7 # incoming Taildrop files they sit in an internal inbox until something calls
8 # `tailscale file get`. This blocks in --wait --loop mode so every incoming
9 # file moves into ~/Downloads within moments of arriving instead of sitting
10 # invisible until someone remembers to pull it manually.
11
12 set -euo pipefail
13 export PATH="/opt/homebrew/bin:/opt/homebrew/sbin:/usr/local/bin:$PATH"
14
15 exec tailscale file get --wait --loop --conflict=rename "$HOME/Downloads"

```

55 scripts/unlock-files.sh

```
1 #!/bin/bash
2 # Resolve project root relative to this script's location, so the script
3 # works regardless of the caller's CWD.
4 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
5 PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
6
7 echo "Unlocking .env..."
8 if [ -f "$PROJECT_ROOT/.env" ]; then
9     sudo cat "$PROJECT_ROOT/.env" > "$PROJECT_ROOT/.env.tmp" && mv "$PROJECT_ROOT/.env.tmp" "$PROJECT_ROOT/.env"
10    echo ".env unlocked"
11 fi
12
13 echo "Unlocking AdGuard cache files..."
14 for f in "$PROJECT_ROOT"/adguard/work/userfilters/compiled_rules.txt \
15         "$PROJECT_ROOT"/adguard/work/userfilters/ip_blocklist.ipset \
16         "$PROJECT_ROOT"/adguard/work/userfilters/cache/*.txt \
17         "$PROJECT_ROOT"/adguard/work/userfilters/cache/ip/*.txt \
18         "$PROJECT_ROOT"/adguard/work/data/filters/*.txt; do
19     if [ -f "$f" ]; then
20         cat "$f" > "$f.tmp" && mv "$f.tmp" "$f"
21         echo "Unlocked $f"
22     fi
23 done
24
25 echo ""
26 echo "All locked files have been successfully bypassed via atomic rename!"
```

56 scripts/wifi-rejoin.sh

```
1 #!/usr/bin/env bash
2 # nullexit Wi-Fi keepalive. Re-associates ONLY to networks you've remembered,
3 # never any other (no public-AP hopping). No Location permission, no root plist
4 # reads, no reading the current SSID.
5 #
6 # Why "remembered" and not "current": macOS hides the live SSID from every
7 # script unless the process holds Location Services access, so nothing can
8 # auto-detect which network you're on. Joining a network *by name* is not
9 # gated, so you remember your network(s) once and we rejoin the first
10 # remembered one that's in range.
11 #
12 # set <SSID>      remember a network (safe to run repeatedly)
13 # forget <SSID>   stop remembering it
14 # list            show remembered networks
15 # reset           (internal) clear the drop-timer once the link is back
16 # rejoin          (internal, called by the WARP watcher) re-associate a
17 #                 remembered network after a grace period, with backoff
18 set -u
19
20 DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
21 ROOT="$(cd "$DIR/.." && pwd)"
22 STATE="$ROOT/.last_wifi_ssid"           # remembered SSIDs, one per line (gitignored)
23 LOG="$ROOT/output.log"
24 DOWN_MARK="/tmp/nullexit-wifi-downsince"
25 JOIN_MARK="/tmp/nullexit-wifi-lastjoin"
26 TRY_MARK="/tmp/nullexit-wifi-tries"
27 GAVEUP_MARK="/tmp/nullexit-wifi-gaveup"
28
29 GRACE=15          # let DHCP self-heal before the first rejoin attempt
30 BACKOFF=30        # base seconds between rejoin attempts (grows per attempt)
31 MAX_TRIES=6       # stop re-associating after this many tries per down-episode
32
33 ts() { date -u +%FT%TZ; }
34 log() { echo "[${ts}] wifi-rejoin: $*" >> "$LOG"; }
35
36 wifi_iface() {
37     local i
38     i=$(networksetup -listallhardwareports 2>/dev/null \
39         | awk '/Hardware Port: Wi-Fi/{getline; print $2; exit}')
40     [ -z "$i" ] && i=en0
41     printf '%s' "$i"
42 }
43
44 case "${1:-}" in
```

```

45 set)
46 ssid="${2:-}"; [ -z "$ssid" ] && { echo "usage: ${0##*/} set <SSID>" >&2; exit 2; }
47 touch "$STATE"
48 if grep -qxF "$ssid" "$STATE" 2>/dev/null; then
49     echo "already remembered: $ssid"
50 else
51     printf '%s\n' "$ssid" >> "$STATE"
52     echo "remembered: $ssid"
53 fi
54 ;;
55
56 forget)
57 ssid="${2:-}"; [ -z "$ssid" ] && { echo "usage: ${0##*/} forget <SSID>" >&2; exit 2; }
58 if [ -f "$STATE" ] && grep -qxF "$ssid" "$STATE"; then
59     grep -vxF "$ssid" "$STATE" > "$STATE.tmp" && mv "$STATE.tmp" "$STATE"
60     echo "forgotten: $ssid"
61 else
62     echo "not remembered: $ssid"
63 fi
64 ;;
65
66 list)
67 if [ -s "$STATE" ]; then cat "$STATE"; else echo "(none run: ${0##*/} set <SSID>); fi
68 ;;
69
70 reset)
71 rm -f "$DOWN_MARK" "$JOIN_MARK" "$TRY_MARK" "$GAVEUP_MARK" 2>/dev/null || true
72 ;;
73
74 rejoin)
75 [ -s "$STATE" ] || exit 0 # nothing remembered do nothing
76 now=$(date +%s)
77
78 # Grace: remember when the uplink first dropped and hold off, so a DHCP-only
79 # blip on the same AP recovers on its own before we force anything.
80 if [ ! -f "$DOWN_MARK" ]; then echo "$now" > "$DOWN_MARK"; echo 0 > "$TRY_MARK"; exit 0; fi
81 downsince=$(cat "$DOWN_MARK" 2>/dev/null || echo "$now")
82 [ $(( now - downsince )) -lt "$GRACE" ] && exit 0
83
84 # Attempt cap: if re-associating hasn't restored the uplink after MAX_TRIES,
85 # STOP. Churning a network that won't come up (captive portal, DHCP blocked,
86 # out of range) only makes things worse which is exactly what happened
87 # once recovery was aborting. Resets when the link returns (see 'reset').
88 tries=$(cat "$TRY_MARK" 2>/dev/null || echo 0)
89 if [ "$tries" -ge "$MAX_TRIES" ]; then
90     [ -f "$GAVEUP_MARK" ] || { log "gave up after $tries attempts reconnect Wi-Fi manually"; : > "$GAVEUP_MARK"; }
91     exit 0
92 fi
93
94 # Progressive backoff between attempts: 30s, 60s, 120s, 240s, then cap 300s
95 # give DHCP time to actually settle instead of hammering every 30s.
96 backoff=$(( BACKOFF * (2 ** (tries < 4 ? tries : 4)) )); [ "$backoff" -gt 300 ] && backoff=300
97 if [ -f "$JOIN_MARK" ]; then
98     last=$(cat "$JOIN_MARK" 2>/dev/null || echo 0)
99     [ $(( now - last )) -lt "$backoff" ] && exit 0
100 fi
101 echo "$now" > "$JOIN_MARK"
102 echo $(( tries + 1 )) > "$TRY_MARK"
103
104 iface=$(wifi_iface)
105 if ! networksetup -getairportpower "$iface" 2>/dev/null | grep -q ': On'; then
106     sudo -n /usr/sbin/networksetup -setairportpower "$iface" on >/dev/null 2>&1 || true
107     sleep 2
108 fi
109
110 log "uplink down rejoin attempt $((tries + 1))/MAX_TRIES, remembered network(s) only (iface $iface)"
111 while IFS= read -r ssid; do
112     [ -z "$ssid" ] && continue
113     # -setairportnetwork joins by name using macOS's saved credentials; it
114     # only ever targets a network you explicitly remembered. Exit 0 just means
115     # the request was accepted DHCP/uplink recovery is confirmed separately
116     # when the watcher sees the interface get an IP and logs RESUMED.
117     if sudo -n /usr/sbin/networksetup -setairportnetwork "$iface" "$ssid" >/dev/null 2>&1; then
118         log "sent re-associate request for a remembered network (waiting for DHCP)"
119         exit 0
120     fi
121 done < "$STATE"
122 log "no remembered network in range (or credentials needed)"
123 ;;
124

```

```

125 *)
126     echo "usage: ${0##*/} {set <SSID>|forget <SSID>|list|rejoin}" >&2
127     exit 2
128 ;;
129 esac

```

57 tor-bridges.txt

58 white_list.txt

```

1 0.client-channel.google.com
2 1drv.com
3 2.android.pool.ntp.org
4 akamaihd.net
5 akamaitechnologies.com
6 akamaized.net
7 amazonaws.com
8 android.clients.google.com
9 api-adservices.apple.com
10 api.ipify.org
11 api.rlje.net
12 app-api.ted.com
13 appleid.apple.com
14 apps.skype.com
15 appsbackup-pa.clients6.google.com
16 appsbackup-pa.googleapis.com
17 appspot-preview.l.google.com
18 apt.sonarr.tv
19 aspnetscdn.com
20 attestation.xboxlive.com
21 ax.phobos.apple.com.edgesuite.net
22 books-analytics-events.apple.com
23 brightcove.net
24 c.s-microsoft.com
25 cdn.cloudflare.net
26 cdn.embedly.com
27 cdn.optimizely.com
28 cdn.vidible.tv
29 cdn2.optimizely.com
30 cdn3.optimizely.com
31 cdnjs.cloudflare.com
32 cert.mgt.xboxlive.com
33 clientconfig.passport.net
34 clients1.google.com
35 clients2.google.com
36 clients3.google.com
37 clients4.google.com
38 clients5.google.com
39 clients6.google.com
40 connectivitycheck.android.com
41 connectivitycheck.gstatic.com
42 continuum.dds.microsoft.com
43 cpms.spop10.ams.plex.bz
44 cpms35.spop10.ams.plex.bz
45 cse.google.com
46 ctldl.windowsupdate.com
47 cws.conviva.com
48 d2c8v52ll5s99u.cloudfront.net
49 d2gatte9o95jao.cloudfront.net
50 dashboard.plex.tv
51 dataplicity.com
52 def-vef.xboxlive.com
53 delivery.vidible.tv
54 dev.virtualearth.net
55 device.auth.xboxlive.com
56 display.ugc.bazaarvoice.com
57 displaycatalog.mp.microsoft.com
58 dl.delivery.mp.microsoft.com
59 dl.dropbox.com
60 dl.dropboxusercontent.com
61 dns.msftncsi.com

```

62 download.sonarr.tv
63 drift.com
64 driftt.com
65 dynupdate.no-ip.com
66 ecn.dev.virtualearth.net
67 edge.api.brightcove.com
68 eds.xboxlive.com
69 fonts.gstatic.com
70 forums.sonarr.tv
71 g.live.com
72 geo-prod.do.dsp.mp.microsoft.com
73 geo3.ggpht.com
74 gfws1.geforce.com
75 giphy.com
76 github.com
77 github.io
78 googleapis.com
79 gravatar.com
80.gstatic.com
81 help.ui.xboxlive.com
82 hls.ted.com
83 i.ytimg.com
84 il.ytimg.com
85 iadsdk.apple.com
86 imagesak.secureserver.net
87 img.vidible.tv
88 imgix.net
89 imgs.xkcd.com
90 instantmessaging-pa.googleapis.com
91 intercom.io
92 jquery.com
93 jsdelivr.net
94 keystone.mwbsys.com
95 lastfm-img2.akamaized.net
96 licensing.xboxlive.com
97 live.com
98 livepassdl.conviva.com
99 login.live.com
100 login.microsoftonline.com
101 manifest.googlevideo.com
102 meta-db-worker02.pop.ric.plex.bz
103 meta.plex.bz
104 meta.plex.tv
105 microsoftonline.com
106 msftncsi.com
107 my.plexapp.com
108 nexusrules.officeapps.live.com
109 nine.plugins.plexapp.com
110 no-ip.com
111 node.plexapp.com
112 notes-analytics-events.apple.com
113 notify.xboxlive.com
114 npr-news.streaming.adswizz.com
115 ns1.dropbox.com
116 ns2.dropbox.com
117 o1.email.plex.tv
118 o2.sg0.plex.tv
119 ocsp.apple.com
120 office.com
121 office.net
122 office365.com
123 officeclient.microsoft.com
124 om.cbsi.com
125 onedrive.live.com
126 outlook.live.com
127 outlook.office365.com
128 pings.conviva.com
129 placeholder.it
130 placeholderit.imgix.net
131 players.brightcove.net
132 pricelist.skype.com
133 products.office.com
134 proxy.plex.bz
135 proxy.plex.tv
136 proxy02.pop.ord.plex.bz
137 pubsub.plex.bz
138 pubsub.plex.tv
139 raw.githubusercontent.com
140 redirector.googlevideo.com
141 res.cloudinary.com
142 s.gateway.messenger.live.com


```

143 s.marketwatch.com
144 s.youtube.com
145 s.ytimg.com
146 sl.wp.com
147 s2.youtube.com
148 s3.amazonaws.com
149 sa.symcb.com
150 secure.avangate.com
151 secure.brightcove.com
152 secure.surveymonkey.com
153 services.sonarr.tv
154 skyhook.sonarr.tv
155 spclient.wg.spotify.com
156 ssl.p.jwpcdn.com
157 staging.plex.tv
158 status.plex.tv
159 t.co
160 t0.ssl.ak.dynamic.tiles.virtualearth.net
161 t0.ssl.ak.tiles.virtualearth.net
162 tawk.to
163 tedcdn.com
164 themoviedb.com
165 thetvdb.com
166 tinyurl.com
167 title.auth.xboxlive.com
168 title.mgt.xboxlive.com
169 traffic.libsyn.com
170 tvdb2.plex.tv
171 tvthemes.plexapp.com
172 twimg.com
173 ui.skype.com
174 video-stats.l.google.com
175 videos.vidible.tv
176 vidtech.cbsinteractive.com
177 weather-analytics-events.apple.com
178 widget-cdn.rpxnow.com
179 win10.ipv6.microsoft.com
180 wp.com
181 ws.audioscrobbler.com
182 www.dataplicity.com
183 www.googleapis.com
184 www.msftconnecttest.com
185 www.msftncsi.com
186 www.no-ip.com
187 www.youtube-nocookie.com
188 xbox.ipv6.microsoft.com
189 xboxexperiencesprod.experimentation.xboxlive.com
190 xflight.xboxlive.com
191 xkms.xboxlive.com
192 xsts.auth.xboxlive.com
193youtu.be
194 youtube-nocookie.com
195 yt3.ggpht.com
196 zee.cws.conviva.com
197
198 # YouTube family base domains so every subdomain (video stream servers on
199 # *.googlevideo.com, image hosts on *.ytimg.com, profile thumbnails on
200 # *.ggpht.com, etc.) is automatically allowlisted via the AdGuard '||domain^'
201 # wildcard. The specific s.youtube.com / manifest.googlevideo.com entries
202 # above are kept for clarity but will be optimized away by rule-compiler
203 # because their parents are now in this list.
204 youtube.com
205 googlevideo.com
206 ytimg.com
207 ggpht.com
208 gvt1.com
209 gvt2.com
210 gvt3.com
211 gvt1.net
212 gvt2.net
213 gvt3.net
214
215 edge-mqtt.facebook.com
216 gateway.instagram.com
217 i.instagram.com
218
219 facebook.com
220 fb.com
221 fbcdn.net
222 fbsbx.com
223 messenger.com

```

```
224 whatsapp.com
225 web.whatsapp.com
226 instagram.com
227 cdninstagram.com
228 static.whatsapp.net
229 media-bos5-1.cdn.whatsapp.net
230 media-yyz1-1.cdn.whatsapp.net
231 webtp.whatsapp.net
232 mmg.whatsapp.net
233 g.whatsapp.net
234 g-fallback.whatsapp.net
```